

ROS2 编程技术

智能网联汽车·具身智能机器人菁英班核心课程

广州科技贸易职业学院交通工程学院

目 录

第 1 章：ROS 2 概述与架构设计	31
1.1 ROS 2 发展背景与设计目标	31
1.2 DDS 中间件架构.....	32
1.3 Quality of Service (QoS) 策略	33
1.4 节点生命周期管理.....	34
1.5 安装与环境搭建	35
1.6 本章小结.....	36
1.7 练习题.....	36
第 1 章 实验指导书：ROS 2 环境搭建与课程仿真入门	40
实验目标.....	40
实验准备.....	40
练习 1.1：ROS 2 安装验证（约 15 分钟）	41
练习 1.2：工作空间创建与编译（约 15 分钟）	42
练习 1.3：DDS 域 ID 实验（约 15 分钟）	43
练习 1.4：课程源码包编译与运行（约 15 分钟）	44
练习 1.5：课程仿真启动与使用（约 15 分钟）	48
练习 1.6：VS Code + ROS 2 插件编程（约 15 分钟）	51

本章实验总结	61
实际运行证据	62
第 2 章：ROS 2 核心编程基础	63
2.1 创建 Python 包与节点.....	63
2.2 编写第一个 ROS 2 Python 节点.....	64
2.3 开发工具链.....	66
2.4 本章小结.....	67
2.5 练习题.....	68
第 2 章 实验指导书：Python 节点编程与工具链.....	69
实验目标.....	69
练习 3.1：创建并运行 Python 节点（约 30 分钟）	69
练习 3.2：日志系统实验（约 30 分钟）	72
练习 3.3：可视化工具使用（约 30 分钟）	74
练习 4：节点与仿真交互 — 订阅 /odom 查看机器人位置（约 15 分钟）	76
实际运行证据	78
第 3 章：话题通信（Topics）	79
3.1 发布-订阅模型原理	79
3.2 标准消息类型	80
3.3 自定义消息接口	81
3.4 QoS 配置实战.....	82

3.5 多线程执行器与回调组.....	83
3.6 本章小结.....	83
3.7 练习题.....	84
第 3 章 实验指导书：话题通信编程与实践.....	89
实验目标.....	89
练习 3.1：Publisher 与 Subscriber 通信（约 30 分钟）	89
练习 3.2：自定义消息接口（约 30 分钟）	93
练习 3.3：QoS 策略对比实验（约 30 分钟）	96
本章实验总结	100
练习 3.4：Twist 控制 XBot-U 走方形轨迹（约 15 分钟）	100
实际运行证据	103
第 4 章：服务通信（Services）	105
4.1 请求-响应模型.....	105
4.2 同步调用 vs 异步调用	106
4.3 服务超时与重试机制.....	107
4.4 本章小结.....	108
4.5 练习题.....	108
新建 client_wait.py.....	109
编译：	109
先不启动 server，只启动 client_wait	109

再启动 server	110
第 4 章 实验指导书：服务通信编程.....	111
实验目标.....	111
练习 4.1: AddTwoInts 服务 (约 30 分钟)	111
1. 创建包.....	111
2. 编写 server.py 和 client.py (参考代码见 lab_code/ch04_lab/)	111
3. 修改 ~/my_ros2_ws/src/service_demo/setup.py	111
4. 编译并运行.....	112
练习 4.2: 自定义 .srv 接口 (约 30 分钟)	112
练习 4.3: 超时与重试 (约 30 分钟)	114
练习 4.4: Service 控制机器人运动 (约 15 分钟)	116
实际运行证据	120
第 5 章: 动作通信 (Actions)	121
5.1 动作通信架构	121
5.2 动作接口定义	122
5.3 取消与抢占机制	124
5.4 本章小结.....	125
5.5 练习题.....	125
第 5 章 实验指导书：动作通信编程.....	126
实验目标.....	126

练习 3.1: DoDishes 动作 (约 30 分钟)	126
创建接口包.....	126
配置 Cmake.....	127
配置 action_demo_interfaces/package.xml	127
编译	127
创建 action_demo.....	127
编译	128
运行	128
练习 3.2: 自定义 Tracking 动作 (约 30 分钟)	128
创建接口包:	128
编写 CMakeLists.txt.....	129
编译	130
创建 Tracking Python 包.....	130
修改 setup.py	134
修改 tracking_server/package.xml.....	134
编译	135
运行	135
改成用 gazebo 仿真.....	136
练习 3.3: 取消与抢占机制 (约 30 分钟)	136
创建新包	137

修改接口	137
接口 CMakeList.....	137
接口 package.xml	137
创建 python 文件.....	138
完善配置	144
编译	145
自动取消	145
拒绝新目标.....	145
练习 4: Action 导航任务 — 发送目标坐标驱动 XBot-U (约 15 分钟)	146
创建包	146
创建并配置接口.....	147
编写导航 Action Server	148
编写导航 Action Client	155
Python package.xml.....	160
setup.py.....	160
setup.cfg.....	161
编译与运行	161
启动 Server	162
发送完整位姿目标	162
自动取消测试.....	162

忙碌拒绝测试.....	163
Action 检查	163
实际运行证据	164
第 6 章：参数系统与 Launch 文件.....	165
6.1 参数系统.....	165
6.2 Launch 文件系统	166
6.3 本章小结.....	168
6.4 练习题.....	168
第 6 章 实验指导书：参数系统与 Launch 文件.....	170
实验目标.....	170
练习 3.1：参数 CRUD 操作（约 30 分钟）	170
创建 param_demo 功能包.....	170
创建 Launch 和 YAML 配置目录：	170
配置功能包公共文件.....	171
编写 param_demo.py.....	172
编译功能包.....	174
运行参数节点	174
动态修改参数	175
练习 3.2：参数回调验证（约 30 分钟）	177
测试 max_speed.....	177

17. 测试 mode	178
YAML 参数文件实验	179
创建 params.yaml	179
通过命令行加载 YAML	179
向运行中的节点加载 YAML	180
练习 3.3: Launch 文件实战 (约 30 分钟)	180
启动 talker、listener 和 RViz	180
使用 Launch 加载 params.yaml	181
编写组合仿真与导航 Launch	182
重新编译 Launch 文件	183
测试 talker 和 listener	183
测试 use_rviz 条件启动	185
测试 Launch 加载 YAML	185
测试组合 Launch	186
练习 4: 动态参数调速 — 运行时修改机器人速度 (约 15 分钟)	187
编写 speed_controller.py	187
创建初始速度 YAML	189
创建 speed.launch.py	189
编译速度控制节点	190
启动 Gazebo 仿真	190

启动速度控制节点	190
检查节点、参数和话题.....	190
动态修改速度	191
暂停和恢复控制	192
测试超限参数	193
实际运行证据	195
第 7 章: TF2 坐标变换系统	196
7.1 坐标系树 (Frame Tree) 设计原理	196
7.2 静态变换与动态变换.....	196
7.3 tf2_ros.Buffer 与 lookup_transform()	197
7.4 时间同步与插值	198
7.5 TF2 调试工具	199
7.6 本章小结.....	199
7.7 练习题.....	199
第 7 章 实验指导书: TF2 坐标变换系统	201
实验目标.....	201
练习 7.1: TF 广播和监听基础 (约 30 分钟)	201
练习 7.2: 多传感器坐标对齐 (约 30 分钟)	202
练习 7.3: TF 调试工具使用 (约 30 分钟)	203
练习 4: TF 仿真查询 — 查询 XBot-U 传感器坐标系 (约 15 分钟)	204

实际运行证据	205
第 8 章: URDF/Xacro 机器人建模	206
8.1 URDF 语法基础	206
8.2 XACRO 宏与参数化	207
8.3 robot_state_publisher + joint_state_publisher	208
8.4 RViz2 RobotModel 显示	209
8.5 本章小结	209
8.6 练习题	209
第 8 章 实验指导书: URDF/Xacro 机器人建模	211
实验目标	211
练习 8.1: 基础 URDF 编写 (约 30 分钟)	211
练习 8.2: XACRO 参数化建模 (约 30 分钟)	212
练习 8.3: RViz2 可视化机器人模型 (约 30 分钟)	212
练习 4: XACRO 自定义标记 — 为 XBot-U 添加视觉标记 (约 15 分钟)	213
第 9 章: Gazebo 仿真	215
9.1 Gazebo 仿真环境概述	215
9.2 Spawn 机器人模型	215
9.3 传感器插件配置	216
9.4 ros2_control 差速驱动	217
9.5 本章小结	218

9.6 练习题.....	218
第 9 章 实验指导书: Gazebo 仿真.....	220
实验目标.....	220
练习 9.1: Gazebo 环境搭建与 Spawn 机器人 (约 30 分钟)	220
练习 9.2: 传感器插件配置 (约 30 分钟)	221
练习 9.3: ros2_control 差速驱动控制 (约 30 分钟)	222
练习 4: Gazebo 传感器数据订阅 — 处理 /scan 和 /camera (约 15 分钟)	223
实际运行证据	224
第 10 章 SLAM 建图与定位.....	227
10.1 知识要点	227
10.2 练习题	234
第 10 章 实验指导书: SLAM 建图与定位.....	236
实验准备: 创建 slam_lab 功能包	236
练习 1: slam_toolbox 在线建图 (约 30 分钟)	252
练习 2: 加载地图并使用 AMCL 定位 (约 30 分钟)	255
练习 3: Cartographer 2D 建图和地图导出 (扩展, 约 30 分钟)	258
练习 4: rosbag 回放建图 (约 15 分钟)	261
第 11 章 Nav2 自主导航.....	266
11.1 知识要点	266
11.2 练习题	276

第 11 章 实验手册: Nav2 自主导航	278
环境说明.....	278
准备工作.....	278
练习 1: goToPose 单点导航 (~15 分钟)	279
练习 2: followWaypoints 多点导航 (~15 分钟)	281
练习 3: 恢复行为与动态障碍 (~20 分钟)	282
练习 4: 导航过程监控 (~15 分钟).....	285
练习 5: 多圈巡逻导航 (~15 分钟).....	287
练习 6: 综合挑战 — 自主巡逻 + 动态避障 (~20 分钟)	288
常用调试命令	289
常见问题.....	290
第 12 章 MoveIt 2 机械臂编程	291
12.1 知识要点	291
12.2 练习题	302
第 12 章 实验手册: MoveIt 2 机械臂编程.....	304
环境说明.....	304
练习 1: MoveIt2 配置加载与单轴运动 (~30 分钟).....	304
练习 2: 笛卡尔路径规划 (~30 分钟).....	307
练习 3: Pick & Place 抓取任务 (~30 分钟)	311
练习 4: MoveItPy 单关节运动规划 (约 15 分钟)	317

第 13 章 YOLO + ROS 2 目标检测	319
13.1 知识要点	319
13.2 练习题	329
第 13 章 实验手册: YOLO + ROS 2 目标检测	331
环境说明.....	331
练习 1: YOLO 模型导出与 ROS 2 推理节点 (~30 分钟).....	331
练习 2: 检测结果可视化与话题发布 (~30 分钟).....	336
练习 3: 目标跟随机器人控制 (~30 分钟)	340
练习 4: YOLO 物体检测 → 视觉跟随控制 (约 15 分钟)	344
第 14 章 视觉大模型 + ROS 2.....	347
14.1 知识要点	347
14.2 练习题	362
第 14 章 实验手册: 视觉大模型 + ROS 2.....	363
环境说明.....	363
练习 1: LLM 文本规划节点 (~30 分钟)	363
练习 2: VLM 场景描述与 ROS 2 集成 (~30 分钟).....	367
练习 3: 自然语言指令驱动机器人导航 (~30 分钟).....	371
练习 4: LLM 指令 → Nav2 导航目标 (约 15 分钟)	376
第 15 章: 综合实训 — 机械臂辅助化学实验系统.....	378
15.1 项目背景	378

15.2 题目 1: LLM 配方验证服务	379
15.3 题目 2: YOLO 试剂瓶检测	380
15.4 题目 3: VLM 标签文字识别.....	382
15.5 题目 4: TF 空间定位	383
15.6 题目 5: MoveIt2 抓取规划.....	385
15.7 题目 6: 全流程编排 (Action Server)	387
15.8 本章小结	390
15.9 综合实训题	390
第 15 章 实验指导书: 综合实训.....	392
实验目标.....	392
题目 1: LLM 配方验证服务 (约 30 分钟)	392
题目 2: YOLO 试剂瓶检测 (约 30 分钟)	393
题目 3: VLM 标签文字识别 (约 30 分钟)	394
题目 4: TF 空间定位 (约 30 分钟)	395
题目 5: MoveIt2 抓取规划 (约 30 分钟)	396
题目 6: 全流程编排 (约 30 分钟)	397
评分标准.....	398
第 16 章 Nav2 架构与核心组件.....	399
学习目标.....	399
16.1 机器人导航概述.....	399

16.2 Nav2 整体架构.....	400
16.3 行为树驱动导航.....	401
16.4 生命周期节点管理.....	404
16.5 核心组件详解.....	407
16.6 Nav2 启动与配置.....	410
课后练习.....	412
第 16 章 实验：机械臂 URDF 建模与状态发布	413
实验目标.....	413
实验环境.....	413
参考代码说明	413
实验步骤.....	414
实验结果与分析	422
思考题.....	422
第 17 章 全局代价地图	423
学习目标.....	423
17.1 代价地图基本原理.....	423
17.2 分层代价地图架构.....	425
17.3 静态层配置	427
17.4 障碍物层配置.....	430
17.5 膨胀层配置	432

17.6 代价地图参数调优.....	435
17.7 实战与调试.....	437
课后练习.....	439
第 17 章 实验：MoveIt2 基础与运动学规划.....	441
实验目标.....	441
实验环境.....	441
参考代码说明	442
17.1 MoveIt2 配置与基础规划	442
17.2 Python 关节空间规划.....	448
实验结果与分析	457
思考题.....	458
第 18 章 全局路径规划	459
学习目标.....	459
18.1 全局路径规划概述.....	459
18.2 Dijkstra 算法	460
18.3 A*算法.....	463
18.4 Nav2 Smac Planner	466
18.5 路径优化	471
18.6 实战案例	475
课后练习.....	476

第 18 章 实验：MoveIt2 规划场景、避障与抓取放置	478
实验目标.....	478
实验环境.....	478
参考代码说明	479
18.1 笛卡尔路径与避障.....	479
18.2 抓取与放置	488
实验结果与分析	496
思考题.....	497
第 19 章 局部路径规划	498
学习目标.....	498
19.1 局部路径规划概述.....	498
19.2 DWA 动态窗口法.....	499
19.3 DWB 控制器	505
19.4 Regulated Pure Pursuit	508
19.5 MPPI 控制器.....	512
19.6 局部规划参数调优.....	513
课后练习.....	515
第 19 章 实验：相机驱动、标定与视觉检测.....	516
实验目标.....	516
实验环境.....	516

参考代码说明	517
19.1 相机驱动与标定.....	518
19.2 颜色检测与 YOLO 检测.....	524
19.3 AR 标签检测与手眼标定.....	533
实验结果与分析	540
思考题.....	541
第 20 章 行为树与恢复行为	542
学习目标.....	542
20.1 行为树基础.....	542
20.2 Nav2 行为树架构.....	544
20.3 恢复行为详解.....	546
20.4 自定义行为树	550
20.5 恢复行为配置与优化	553
20.6 实战案例	555
课后练习.....	557
第 20 章 实验：视觉大模型与 ROS2 应用.....	558
实验目标.....	558
实验环境.....	558
总体设计（依据 `src/lab_code/ch20_lab/README.md`）	558
实验步骤.....	559

实验结果与分析	569
思考题.....	569
第 21 章 视觉 SLAM 导论.....	570
学习目标.....	570
21.1 视觉 SLAM 概述.....	570
21.2 相机模型与 ROS2 视觉接口	572
21.3 ORB-SLAM.....	575
21.4 DSO 直接法	580
21.5 RGB-D SLAM.....	583
21.6 视觉与激光融合 SLAM	585
课后练习.....	588
第 21 章 实验：视觉抓取综合项目	589
实验目标.....	589
实验环境.....	589
参考代码说明	590
21.1 视觉抓取完整流程.....	590
21.2 综合实训：智能机器人产线.....	598
实验结果与分析	609
思考题.....	609
第 22 章 多传感器融合 SLAM.....	610

学习目标.....	610
22.1 多传感器融合概述.....	610
22.2 IMU+LiDAR 融合.....	611
22.3 视觉+IMU 融合(VIO)	617
22.4 Kalman 滤波与图优化融合框架	619
22.5 实际工程案例.....	622
课后练习.....	625
第 22 章 实验指导书：自动驾驶概述与 CARLA 基础.....	627
实验目标.....	627
实验准备.....	627
练习 22.1：环境预检查.....	628
练习 22.2：CARLA 安装验证	629
练习 22.3：探索 CARLA 世界.....	630
练习 22.4：Python API 基础.....	631
实验总结.....	633
补充：Docker 部署方案（备用）	633
思考题.....	634
第 23 章 SLAM 与导航综合实训	635
学习目标.....	635
23.1 综合项目概述.....	635

23.2 SLAM 建图实践.....	636
23.3 自主导航配置.....	640
23.4 多机器人调度.....	646
23.5 项目文档规范.....	649
23.6 性能评估与优化.....	652
23.7 常见问题排查.....	654
课后练习.....	655
第 23 章 实验: CARLA-ROS2 桥接与车辆部署	657
实验目标.....	657
实验环境.....	657
实验准备.....	657
练习 23.1: 启动 CARLA+ROS2 Bridge	658
练习 23.2: 生成 Ego Vehicle 并在 RViz 中可视化.....	659
练习 23.3: 键盘控制车辆移动	660
练习 23.4: 端到端验证与排障	662
实验报告要求	664
思考题.....	664
第 24 章 机械臂基础知识	666
学习目标.....	666
24.1 机械臂分类.....	666

24.2 运动学基础	667
24.3 正运动学与逆运动学	670
24.4 工作空间分析	672
24.5 ROS2 机械臂接口	674
课后练习	676
第 24 章 实验: 多传感器套件与数据采集	677
实验目标	677
实验环境	677
练习 24.1: 配置并启动多传感器套件	677
练习 24.2: 可视化各类传感器数据	679
练习 24.3: 录制并回放 Rosbag	680
实验报告要求	681
常见问题	681
第 25 章 ROS2 机械臂建模	683
学习目标	683
25.1 URDF 建模基础	683
25.2 六自由度机械臂 URDF 建模	685
25.3 Xacro 建模	689
25.4 SRDF 语义描述	692
25.5 在 Rviz 中可视化	694

25.6 3D 模型导入.....	696
25.7 模型验证与调试.....	697
课后练习.....	698
第 25 章 实验手册：全局路径规划与地图导航.....	699
练习 25.1：获取 CARLA 地图 OpenDRIVE 并可视化路网.....	699
练习 25.2：实现 A* 全局路径规划.....	701
练习 25.3：使用 Waypoint 发布者生成导航路径.....	703
实验报告要求	704
第 26 章 MoveIt2 基础	705
学习目标.....	705
26.1 MoveIt2 架构.....	705
26.2 MoveIt2 Setup Assistant.....	707
26.3 MoveGroupInterface.....	710
26.4 OMPL 运动规划器.....	712
26.5 碰撞检测	714
26.6 启动 MoveIt2	717
课后练习.....	717
第 26 章 实验指导：车辆纵横向控制	718
实验目标.....	718
练习 26.1：实现纵向 PID 控制器	718

练习 26.2: 实现 Pure Pursuit 横向控制器	719
练习 26.3: 在 CARLA 中完成路径跟踪测试	720
提交要求.....	721
评分标准.....	721
实际运行证据	722
第 27 章 MoveIt2 Python 规划	723
学习目标.....	723
27.1 MoveItPy API 概览.....	723
27.2 PlanningComponent 详解	725
27.3 关节空间运动规划.....	727
27.4 逆运动学求解.....	729
27.5 完整应用示例	732
课后练习.....	734
第 27 章 实验手册: 多传感器融合定位.....	735
实验环境.....	735
练习 27.1: 构建 TF2 坐标树	736
练习 27.2: 实现 EKF 融合节点.....	737
练习 27.3: 对比 CARLA Ground Truth	738
实验报告要求	740
参考命令速查	741

第 28 章 MoveIt2 笛卡尔空间与避障	742
学习目标.....	742
28.1 笛卡尔路径规划.....	742
28.2 位姿约束	747
28.3 规划场景与障碍物.....	749
28.4 避障规划	753
课后练习.....	756
第 28 章 实验：交通参与者感知.....	758
实验目标.....	758
实验环境.....	758
实验前准备.....	758
练习 28.1: CARLA 相机 YOLO 目标检测	759
练习 28.2: LiDAR 点云聚类检测障碍物.....	760
练习 28.3: 多目标跟踪与可视化.....	762
实验结果总结	764
思考题.....	764
第 29 章 抓取与放置编程	765
学习目标.....	765
29.1 抓取姿态计算.....	765
29.2 夹爪控制	768

29.3 Attach/Detach 物体.....	770
29.4 Pick-and-Place 流程.....	772
29.5 实际案例	778
课后练习.....	780
第 29 章 实验手册：行为决策与交通规则.....	781
实验目标.....	781
前置条件.....	781
练习 29.1: 交通灯检测与停止	781
练习 29.2: 设计 FSM 行为决策节点.....	785
练习 29.3: 配置 Traffic Manager 实现交通流	790
附加挑战.....	795
提交要求.....	795
参考资源.....	795
第 30 章 ROS2 图像接口与相机标定.....	796
学习目标.....	796
30.1 ROS2 图像消息	796
30.2 cv_bridge 图像转换.....	798
30.3 image_transport	800
30.4 usb_cam 驱动	802
30.5 相机标定原理.....	803

30.6 OpenCV 图像处理与 ROS2 集成.....	806
课后练习.....	808
第 30 章 实验手册：安全验证与系统集成.....	810
实验环境.....	810
练习 30.1：验证安全监控节点.....	810
练习 30.2：故障注入.....	811
练习 30.3：集成测试和性能评估.....	812
单元测试.....	812
验收清单.....	812
第 31 章 颜色检测与 YOLO 检测.....	814
学习目标.....	814
31.1 颜色空间与 HSV.....	814
31.2 颜色分割与检测.....	816
31.3 YOLOv8 目标检测集成.....	820
31.4 检测结果发布为 ROS2 消息	825
课后练习.....	827
第 31 章 实验指导书：综合项目——城区自动驾驶.....	829
实验目标.....	829
实验准备.....	829
练习 31.1：验证现有功能包	830

练习 31.2: 运行综合演示脚本	831
练习 31.3: 测试和验收.....	831
结果记录.....	832

Part 1: ROS2 编程基础（智能车终端软件平台）

第 1 章：ROS 2 概述与架构设计

课程： ROS2 Python 编程

章节： 第 1 章

课时： 2 课时 (90 分钟)

**教学
方式**： 讲授 + 演示

1.1 ROS 2 发展背景与设计目标

知识点 1.1.1：从 ROS 1 到 ROS 2

ROS (Robot Operating System) 最初由 Willow Garage 于 2007 年发布，旨在为机器人软件开发提供统一的分布式框架。ROS 1 在学术界和工业界获得了广泛采用，但随着应用场景从实验室扩展到工业级产品，ROS 1 的局限性逐渐暴露：

- **单点故障**：Master 节点 (roscore) 是所有通信的中枢，一旦崩溃整个系统瘫痪
- **实时性不足**：基于 TCP 的通信无法满足硬实时控制需求
- **安全性缺失**：无内置的认证、授权和加密机制
- **嵌入式支持弱**：对资源受限的微控制器支持有限

2014 年，Open Robotics 启动了 ROS 2 项目，完全重新设计了通信架构。

知识点 1.1.2：ROS 2 版本演进

版本名	代号	发布日期	状态
Ardent	Apalone	2017.12	EOL
Bouncy	Bolson	2018.07	EOL
Crystal	Clemmys	2018.12	EOL
Dashing	Diademata	2019.05	EOL
Eloquent	Elusor	2019.11	EOL
Foxy	Fitzroy	2020.06	EOL
Galactic	Geochelone	2021.05	EOL
Humble	Hawksbill	2022.05	**LTS (推荐)**

Iron	Irwini	2023.05	EOL
Jazzy	Jalisco	2024.05	**LTS (最新)**

本课程基于 **ROS 2 Humble** (Ubuntu 22.04) 或 **ROS 2 Jazzy** (Ubuntu 24.04)。

1.2 DDS 中间件架构

知识点 1.2.1 : DDS 概念模型

DDS (Data Distribution Service) 是 OMG 制定的分布式实时通信标准。ROS 2 采用 DDS 替换了 ROS 1 的自定义 TCP/UDP 协议。

DDS 核心概念对应关系:

DDS 概念

→

ROS 2 概念

DomainParticipant

→

Node (通过 context)

DataWriter

→

Publisher

DataReader

→

Subscription

Topic

→

Topic

Domain

→

ROS_DOMAIN_ID

图 1-1 : DDS 去中心化架构

The diagram illustrates a decentralized DDS architecture. It shows two nodes, Node A and Node B, each enclosed in a dashed box. Node A contains a 'Publisher' box, and Node B contains a 'Subscribe' box. A double-headed arrow labeled 'DDS Domain' and 'Topic (自动发现)' connects the Publisher in Node A to the Subscriber in Node B. Below the nodes, a bracket spans both and is labeled '去中心化 (无 Master 节点)', indicating a decentralized system without a central Master node.

说明：ROS 2 使用 DDS 的自动发现机制，节点可直接对等通信，无需中心化的 Master 节点。

知识点 1.2.2 : RMW 抽象层

ROS 2 通过 RMW (ROS MiddleWare) 接口抽象不同的 DDS 实现：

rclcpp / rclpy

← 用户代码 API



图 1-2：ROS 2 分层架构。RMW 层允许用户在不修改应用代码的情况下切换不同的 DDS 实现。

1.3 Quality of Service (QoS) 策略

知识点 1.3.1 : QoS 策略维度

QoS 定义了数据传输的质量保证，ROS 2 提供以下核心策略：

策略维度	选项	含义
Reliability	RELIABLE / BEST Effort	可靠传输（重传） / 尽力传输（允许丢包）
Durability	VOLATILE / TRANSIENT_LOCAL	不保存历史 / 保存历史给后来者
History	KEEP_LAST(n) / KEEP_ALL	保留最后 n 条 / 保留所有
Depth	正整数	队列深度
Deadline	时间间隔	消息更新的最大间隔
Lifespan	时间间隔	消息的有效期
Liveliness	AUTOMATIC / MANUAL	发布者存活检测

知识点 1.3.2 : 预定义 QoS 配置文件

```
# ROS 2 预定义 QoS 配置文件
from rclpy.qos import QoSProfile, qos_profile_sensor_data, qos_profile_services_default

# 传感器数据：高频，允许丢包
```

```

qos_sensor = qos_profile_sensor_data # BEST_EFFORT, KEEP_LAST(5)

# 服务通信：可靠传输
qos_service = qos_profile_services_default # RELIABLE, KEEP_LAST(10)

# 自定义 QoS
qos_custom = QoSProfile(
    reliability=rclpy.qos.ReliabilityPolicy.RELIABLE,
    durability=rclpy.qos.DurabilityPolicy.TRANSIENT_LOCAL,
    depth=10
)

```

1.4 节点生命周期管理

知识点 1.4.1：生命周期状态机

ROS 2 为节点引入了标准化的生命周期状态机：

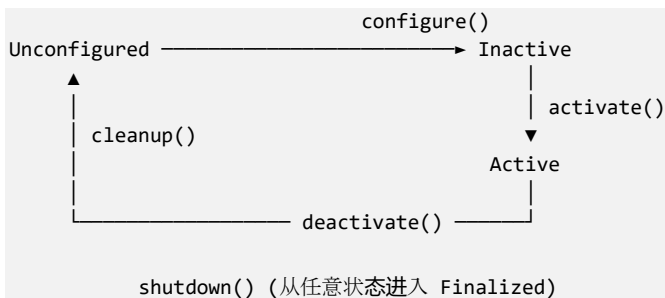


图 1-3：ROS 2 生命周期节点状态转换图。通过状态机管理节点的初始化和资源分配，确保确定的启动和关闭流程。

知识点 1.4.2：生命周期 Python API

```

import rclpy
from rclpy.lifecycle import LifecycleNode, State, TransitionCallbackReturn

class MyLifecycleNode(LifecycleNode):
    def __init__(self):
        super().__init__('my_lifecycle_node')

    def on_configure(self, state: State):
        # 分配资源、设置参数
        self.get_logger().info('on_configure() called')
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state: State):
        # 开始处理数据
        self.get_logger().info('on_activate() called')
        return TransitionCallbackReturn.SUCCESS

```

```
def on_deactivate(self, state: State):
    # 停止处理
    self.get_logger().info('on_deactivate() called')
    return TransitionCallbackReturn.SUCCESS

def on_cleanup(self, state: State):
    # 释放资源
    self.get_logger().info('on_cleanup() called')
    return TransitionCallbackReturn.SUCCESS
```

1.5 安装与环境搭建

知识点 1.5.1 : 安装 ROS 2 Humble

```
# 设置 locale
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# 添加 ROS 2 仓库
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
    | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# 安装 ROS 2 Humble 完整版
sudo apt update
sudo apt install ros-humble-desktop-full

# 安装开发工具
sudo apt install python3-colcon-common-extensions python3-rosdep python3-vcstool
```

知识点 1.5.2 : 配置开发环境

```
# 将 ROS 2 环境变量添加到 ~/.bashrc
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# 初始化 rosdep (用于安装包依赖)
sudo rosdep init
rosdep update
```

知识点 1.5.3 : 工作空间与 colcon 构建

```
# 创建工作空间
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws

# 编译工作空间
colcon build --symlink-install

# 加载工作空间环境
source install/setup.bash

# 验证安装
ros2 run demo_nodes_cpp talker

# 在另一个终端 :
ros2 run demo_nodes_py listener
```

1.6 本章小结

本章核心知识点回顾:

1. ROS 2 是为解决 ROS 1 的单点故障、实时性和安全性问题而重新设计的
2. DDS 中间件提供了去中心化的自动发现通信, 通过 RMW 层实现可替换
3. QoS 策略提供细粒度的通信质量保证 (Reliability、Durability、History 等)
4. 生命周期节点使用状态机管理初始化、激活和关闭流程
5. ROS 2 Humble/Jazzy 通过 apt 安装, colcon 是标准构建工具

1.7 练习题

练习 1.1: 简述 ROS 1 的三个主要局限性和 ROS 2 对应的解决方案。ros1 依赖 master, 存在中心节点——ros2 使用 dds 通信机制, 节点之间可以自动发现彼此 ros1 不太能实时控制并传输, 通信策略单一——Qos 通信, 根据不同数据类型设置可靠性队列深度。。。

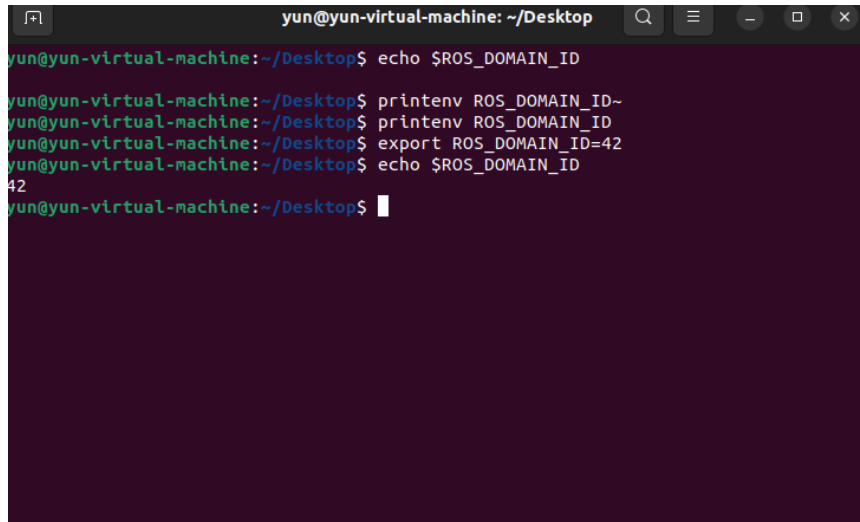
ros1 多机器人、多实验环境相互干扰——通过 ROS_DOMAIN_ID 划分通信领域 **练习**

1.2: DDS 域 ID (ROS_DOMAIN_ID) 的作用是什么? 如果两台机器上的域 ID 不同, 它们的节点能通信吗? 作用是隔离通信环境。id 不同一般不会互相发现和通信 **练习 1.3:** 以下

QoS 配置能否匹配通信?

- Publisher: RELIABLE, VOLATILE
- Subscriber: BEST_EFFORT, VOLATILE 可以。Publisher 使用 RELIABLE，提供可靠传输能力；Subscriber 使用 BEST_EFFORT，只要求尽力接收，因此发布者能力满足订阅者需求。两者 Durability 都为 VOLATILE，也能够匹配。 **练习 1.4：** 查看当前

ROS_DOMAIN_ID 环境变量值，将其修改为 42 并验证。

A terminal window titled 'yun@yun-virtual-machine: ~/Desktop' with standard window controls. The terminal shows a sequence of commands to check and modify the ROS_DOMAIN_ID environment variable. The output shows the current value is 42, and after the export command, it remains 42.

```
yun@yun-virtual-machine:~/Desktop$ echo $ROS_DOMAIN_ID
yun@yun-virtual-machine:~/Desktop$ printenv ROS_DOMAIN_ID~
yun@yun-virtual-machine:~/Desktop$ printenv ROS_DOMAIN_ID~
yun@yun-virtual-machine:~/Desktop$ export ROS_DOMAIN_ID=42
yun@yun-virtual-machine:~/Desktop$ echo $ROS_DOMAIN_ID
42
yun@yun-virtual-machine:~/Desktop$
```

练习 1.5： 创建一

个 ROS 2 工作空间，使用 colcon build 编译，并验证 setup.bash 加载。

```
yun@yun-virtual-machine: ~/ros2_ws
rviz2
rviz_assimp_vendor
rviz_common
rviz_default_plugins
rviz_ogre_vendor
rviz_rendering
sdformat_urdf
sdl2_vendor
sensor_msgs
sensor_msgs_py
shape_msgs
shared_queues_vendor
slam_toolbox
smclib
spdlog_vendor
sqlite3_vendor
sros2
sros2_cmake
statistics_msgs
std_msgs
std_srvs
stereo_msgs
tango_icons_vendor
tcb_span
teleop_twist_joy
teleop_twist_keyboard
tf2
tf2_bullet
tf2_eigen
tf2_eigen_kdl
tf2_geometry_msgs
tf2_kdl
tf2_msgs
tf2_py
tf2_ros
tf2_ros_py
tf2_sensor_msgs
tf2_tools
tf_transformations
theora_image_transport
tinymce_vendor
tinymce_vendor
tl_expected
tlsf
tlsf_cpp
topic_monitor
tracetools
trajectory_msgs
turtlesim
twist_mux
twist_mux_msgs
uncrustify_vendor
unique_identifier_msgs
urdf
urdf_parser_plugin
velocity_controllers
vision_msgs
visualization_msgs
xacro
yaml_cpp_vendor
zstd_vendor
yun@yun-virtual-machine: ~/ros2_ws$
```

练习 1.6: 运行

talker/listener 示例节点, 使用 `ros2 node list` 和 `ros2 topic list` 查看运行状态。

```
yun@yun-virtual-machine: ~/Desktop
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py listener
[INFO] [1783651076.209093902] [listener]: I heard: [Hello World: 55]
[INFO] [1783651077.116607637] [listener]: I heard: [Hello World: 56]
[INFO] [1783651078.116932993] [listener]: I heard: [Hello World: 57]
[INFO] [1783651079.115538103] [listener]: I heard: [Hello World: 58]
[INFO] [1783651080.116638284] [listener]: I heard: [Hello World: 59]
[INFO] [1783651081.116136121] [listener]: I heard: [Hello World: 60]
[INFO] [1783651082.117963226] [listener]: I heard: [Hello World: 61]
[INFO] [1783651083.115699215] [listener]: I heard: [Hello World: 62]
[INFO] [1783651084.115963578] [listener]: I heard: [Hello World: 63]
[INFO] [1783651085.116387246] [listener]: I heard: [Hello World: 64]

[INFO] [1783651049.113522704] [talker]: Publishing: 'Hello World: 28'
[INFO] [1783651050.115207766] [talker]: Publishing: 'Hello World: 29'
[INFO] [1783651051.114256654] [talker]: Publishing: 'Hello World: 30'
[INFO] [1783651052.113385945] [talker]: Publishing: 'Hello World: 31'
[INFO] [1783651053.113406064] [talker]: Publishing: 'Hello World: 32'
[INFO] [1783651054.113430883] [talker]: Publishing: 'Hello World: 33'
[INFO] [1783651055.113459203] [talker]: Publishing: 'Hello World: 34'
[INFO] [1783651056.113486525] [talker]: Publishing: 'Hello World: 35'
[INFO] [1783651057.115557906] [talker]: Publishing: 'Hello World: 36'
[INFO] [1783651058.113544973] [talker]: Publishing: 'Hello World: 37'
[INFO] [1783651059.113574199] [talker]: Publishing: 'Hello World: 38'
[INFO] [1783651060.113630927] [talker]: Publishing: 'Hello World: 39'
[INFO] [1783651061.113960763] [talker]: Publishing: 'Hello World: 40'
[INFO] [1783651062.113651683] [talker]: Publishing: 'Hello World: 41'
[INFO] [1783651063.115871275] [talker]: Publishing: 'Hello World: 42'
[INFO] [1783651064.113817649] [talker]: Publishing: 'Hello World: 43'
[INFO] [1783651065.113765779] [talker]: Publishing: 'Hello World: 44'
[INFO] [1783651066.114425231] [talker]: Publishing: 'Hello World: 45'
[INFO] [1783651067.113754047] [talker]: Publishing: 'Hello World: 46'
[INFO] [1783651068.113839185] [talker]: Publishing: 'Hello World: 47'
[INFO] [1783651069.114887951] [talker]: Publishing: 'Hello World: 48'
[INFO] [1783651070.113947262] [talker]: Publishing: 'Hello World: 49'
[INFO] [1783651071.113940701] [talker]: Publishing: 'Hello World: 50'
[INFO] [1783651072.114845467] [talker]: Publishing: 'Hello World: 51'
[INFO] [1783651073.114076686] [talker]: Publishing: 'Hello World: 52'
[INFO] [1783651074.114015628] [talker]: Publishing: 'Hello World: 53'
[INFO] [1783651075.114040972] [talker]: Publishing: 'Hello World: 53'
[INFO] [1783651076.114180972] [talker]: Publishing: 'Hello World: 54'
[INFO] [1783651077.114180921] [talker]: Publishing: 'Hello World: 55'
[INFO] [1783651078.114528378] [talker]: Publishing: 'Hello World: 56'
[INFO] [1783651079.114133963] [talker]: Publishing: 'Hello World: 57'
[INFO] [1783651080.115408449] [talker]: Publishing: 'Hello World: 58'
[INFO] [1783651081.114815884] [talker]: Publishing: 'Hello World: 59'
[INFO] [1783651082.116752891] [talker]: Publishing: 'Hello World: 60'
[INFO] [1783651083.114328877] [talker]: Publishing: 'Hello World: 61'
[INFO] [1783651084.114844446] [talker]: Publishing: 'Hello World: 62'
```

实验 1:

第 1 章 实验指导书：ROS 2 环境搭建与课程仿真入门

****实验课时****: 2 课时 (90 分钟) ****实验平台****: Ubuntu 22.04 + ROS 2 Humble /
Ubuntu 24.04 + ROS 2 Jazzy

实验目标

完成本实验后，学员应能够：

6. 验证 ROS 2 安装并运行基本节点
7. 编译课程源码包完整工作空间
8. 启动课程仿真 (Gazebo + RViz + LiDAR + 相机)
9. 使用 VS Code + ROS2 插件进行开发调试

实验准备

硬件环境

- PC 一台 (Ubuntu 22.04 或 24.04, ≥8GB RAM, 推荐 16GB)
- 支持 OpenGL 3.3+ 的显卡 (用于 Gazebo 渲染)
- 网络连接正常

软件环境

- ROS 2 Humble (推荐) 或 Jazzy (已安装)
- VS Code (最新版)
- 终端模拟器 (推荐 Terminator 或 tmux, 支持多窗口)

练习 1.1 : ROS 2 安装验证 (约 15 分钟)

目标

验证 ROS 2 安装正确, 环境变量已加载。

步骤

步骤 1: 验证环境变量

```
echo $ROS_DISTRO
# 期望输出: humble 或 jazzy

ros2 --help
# 期望: 显示 ros2 命令列表
```

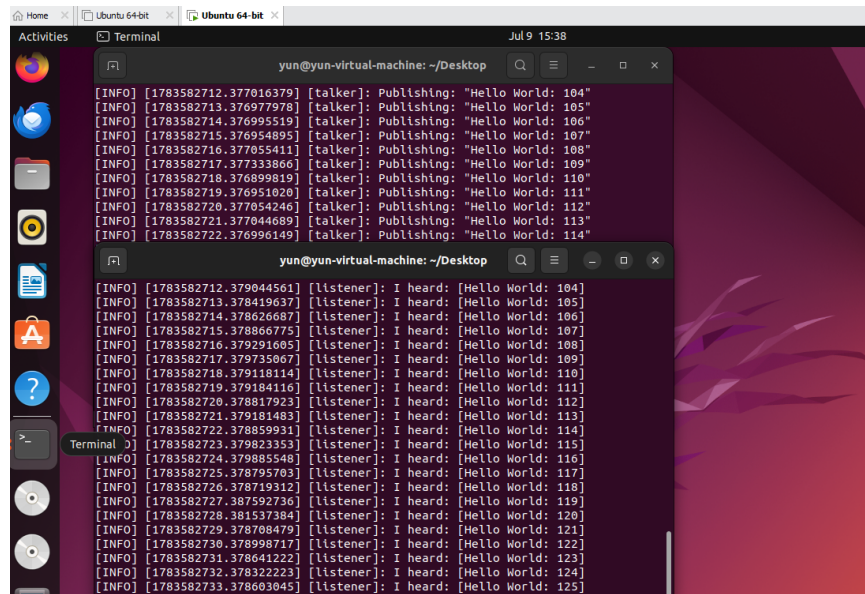
步骤 2: 运行 talker / listener 验证

```
# 终端 1: 启动发布者
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py talker
# 期望: [INFO] Publishing: "Hello World: 0"

# 终端 2: 启动订阅者
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py listener
# 期望: [INFO] I heard: "Hello World: X"

# 终端 3: 查看节点和话题
ros2 node list          # 期望: /talker /listener
ros2 topic echo /chatter # 期望: data: 'Hello World: X'
```

✓ 验证: talker 发布消息, listener 接收消息, ros2 topic echo 能看到实时数据。



```
yun@yun-virtual-machine: ~/Desktop
[INFO] [1783582712.377016379] [talker]: Publishing: "Hello World: 104"
[INFO] [1783582713.376977978] [talker]: Publishing: "Hello World: 105"
[INFO] [1783582714.376995519] [talker]: Publishing: "Hello World: 106"
[INFO] [1783582715.376954895] [talker]: Publishing: "Hello World: 107"
[INFO] [1783582716.377055411] [talker]: Publishing: "Hello World: 108"
[INFO] [1783582717.377333866] [talker]: Publishing: "Hello World: 109"
[INFO] [1783582718.376899819] [talker]: Publishing: "Hello World: 110"
[INFO] [1783582719.376951020] [talker]: Publishing: "Hello World: 111"
[INFO] [1783582720.377054246] [talker]: Publishing: "Hello World: 112"
[INFO] [1783582721.377044689] [talker]: Publishing: "Hello World: 113"
[INFO] [1783582722.376996149] [talker]: Publishing: "Hello World: 114"

yun@yun-virtual-machine: ~/Desktop
[INFO] [1783582712.379044561] [listener]: I heard: [Hello World: 104]
[INFO] [1783582713.378419637] [listener]: I heard: [Hello World: 105]
[INFO] [1783582714.378626687] [listener]: I heard: [Hello World: 106]
[INFO] [1783582715.378666775] [listener]: I heard: [Hello World: 107]
[INFO] [1783582716.379291605] [listener]: I heard: [Hello World: 108]
[INFO] [1783582717.379735067] [listener]: I heard: [Hello World: 109]
[INFO] [1783582718.379118114] [listener]: I heard: [Hello World: 110]
[INFO] [1783582719.379184116] [listener]: I heard: [Hello World: 111]
[INFO] [1783582720.378817923] [listener]: I heard: [Hello World: 112]
[INFO] [1783582721.379181483] [listener]: I heard: [Hello World: 113]
[INFO] [1783582722.378859931] [listener]: I heard: [Hello World: 114]
[INFO] [1783582723.379823353] [listener]: I heard: [Hello World: 115]
[INFO] [1783582724.379885548] [listener]: I heard: [Hello World: 116]
[INFO] [1783582725.378795703] [listener]: I heard: [Hello World: 117]
[INFO] [1783582726.378719312] [listener]: I heard: [Hello World: 118]
[INFO] [1783582727.3787592736] [listener]: I heard: [Hello World: 119]
[INFO] [1783582728.381537384] [listener]: I heard: [Hello World: 120]
[INFO] [1783582729.378708479] [listener]: I heard: [Hello World: 121]
[INFO] [1783582730.378998717] [listener]: I heard: [Hello World: 122]
[INFO] [1783582731.378641222] [listener]: I heard: [Hello World: 123]
[INFO] [1783582732.378322223] [listener]: I heard: [Hello World: 124]
[INFO] [1783582733.378603045] [listener]: I heard: [Hello World: 125]
```

练习 1.2 : 工作空间创建与编译 (约 15 分钟)

目标

创建 ROS 2 工作空间, 掌握 colcon 构建工具链。

步骤

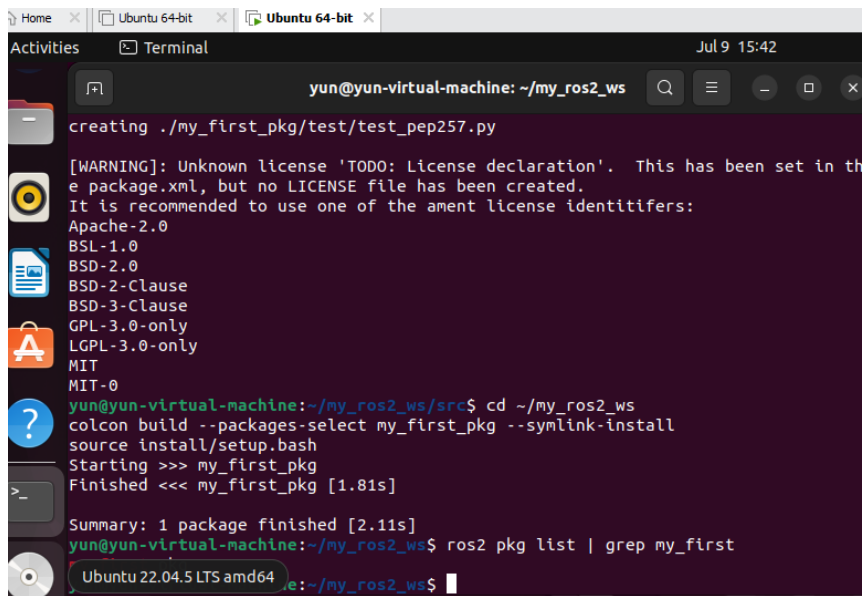
```
# 1. 创建工作空间
mkdir -p ~/my_ros2_ws/src
cd ~/my_ros2_ws

# 2. 编译空工作空间
colcon build --symlink-install

# 3. 创建测试包
cd src
ros2 pkg create my_first_pkg --build-type ament_python \
  --dependencies rclpy std_msgs

# 4. 编译测试包
cd ~/my_ros2_ws
colcon build --packages-select my_first_pkg --symlink-install
source install/setup.bash

# 5. 验证
ros2 pkg list | grep my_first
```

A terminal window on an Ubuntu 22.04 LTS amd64 system. The user is in the directory ~/my_ros2_ws. They create a package named my_first_pkg with a test file test_pep257.py. A warning is shown about an unknown license. The user then builds and installs the package using colcon build. Finally, they run 'ros2 pkg list | grep my_first' to verify the installation.

```
yun@yun-virtual-machine: ~/my_ros2_ws
creating ./my_first_pkg/test/test_pep257.py

[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the
package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
yun@yun-virtual-machine:~/my_ros2_ws/src$ cd ~/my_ros2_ws
colcon build --packages-select my_first_pkg --symlink-install
source install/setup.bash
Starting >>> my_first_pkg
Finished <<< my_first_pkg [1.81s]

Summary: 1 package finished [2.11s]
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 pkg list | grep my_first
Ubuntu 22.04.5 LTS amd64 e:~/my_ros2_ws$
```

✓ 验证: ros2 pkg

list 显示 my_first_pkg。

练习 1.3 : DDS 域 ID 实验 (约 15 分钟)

目标

理解 ROS_DOMAIN_ID 的通信隔离作用。

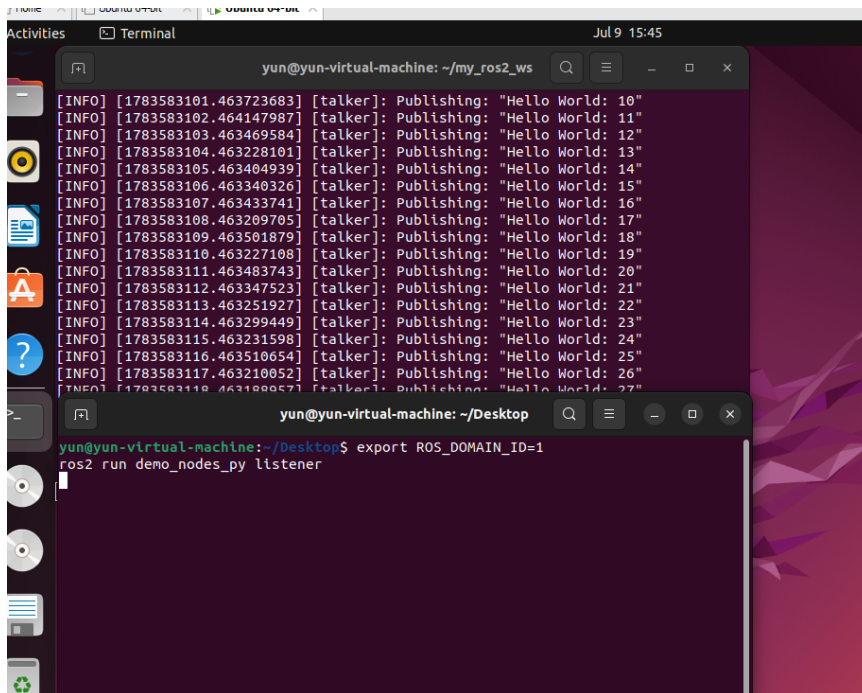
步骤

```
# 终端 1: 域 0 talker
export ROS_DOMAIN_ID=0
ros2 run demo_nodes_py talker

# 终端 2: 域 0 listener → 正常接收
export ROS_DOMAIN_ID=0
ros2 run demo_nodes_py listener
# Ctrl+C 停止
![alt text](images/image-2.png)

# 终端 2: 域 1 listener → 无输出 (域隔离)
export ROS_DOMAIN_ID=1
ros2 run demo_nodes_py listener

# 期望: 无任何输出 (无法跨域通信)
```



✓ 验证：同域通信正

常，跨域通信隔离。

练习 1.4：课程源码包编译与运行（约 15 分钟）

目标

将课程源码包 `ros2_ws/` 复制到用户目录，完成完整编译并运行基础仿真。

步骤

步骤 1：复制课程源码

```
# 将课程源码复制到用户工作空间
cp -r /path/to/course/ros2_ws/src ~/ros2_course_ws/
cd ~/ros2_course_ws
ls src/

# 期望：看到 20 个 ROS 2 包目录
# msgs_demo_interfaces  robot_sim_demo_ros2  topic_demo_py  ...
```

步骤 2：安装系统依赖

```
# 安装 rosdep（如果尚未安装）
sudo apt install python3-rosdep -y
sudo rosdep init          # 仅首次运行
rosdep update
```



```
# 安装所有包的系统依赖
cd ~/ros2_course_ws
rosdep install --from-paths src --ignore-src -r -y
# 期望: [All required rosdeps installed successfully]
```

步骤 3: 编译全部课程包

```
colcon build --symlink-install
# 期望: Summary: 20 packages finished

# 查看编译结果
source install/setup.bash
ros2 pkg list | grep demo
```

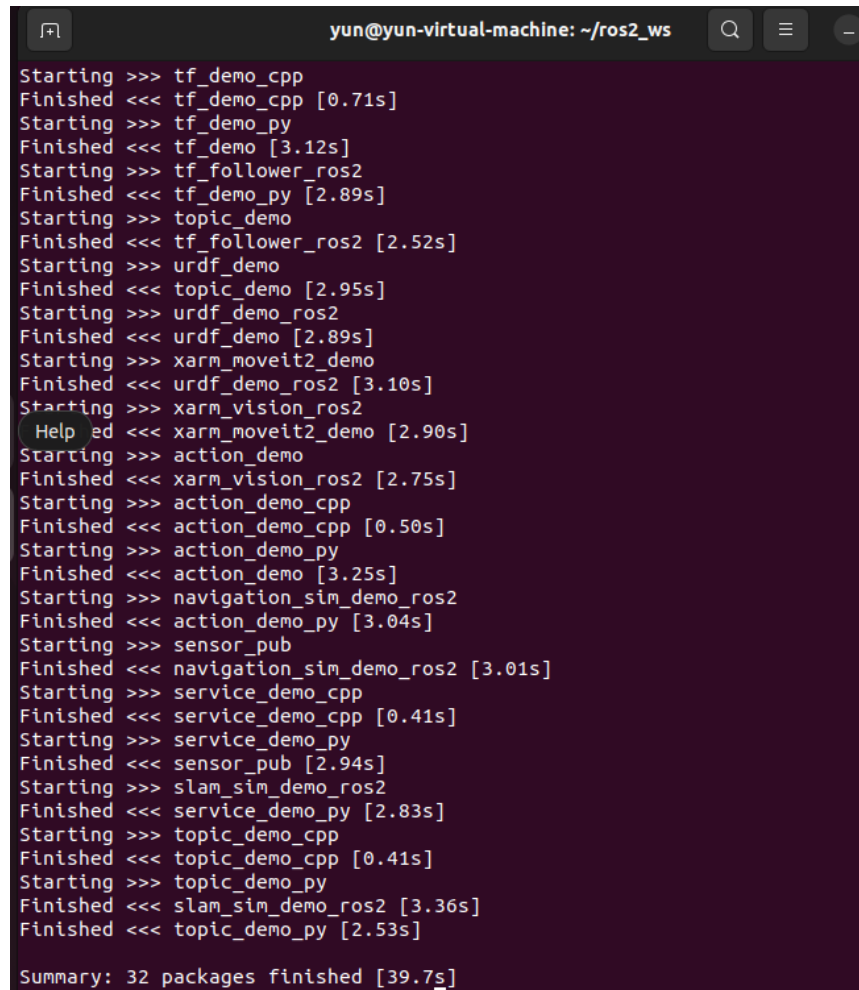
步骤 4: 验证仿真基础节点

```
# 运行 XBot-U 无仿真模式 (纯计算 + TF + RViz)
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
# 期望: RViz 窗口打开, 显示 XBot-U 机器人模型

# 终端输出: [ros2-control-ready:diff_drive_base_controller]
```

✓ 验证:

- 截图 1: colcon build 编译成功 (Summary: 20 packages finished)

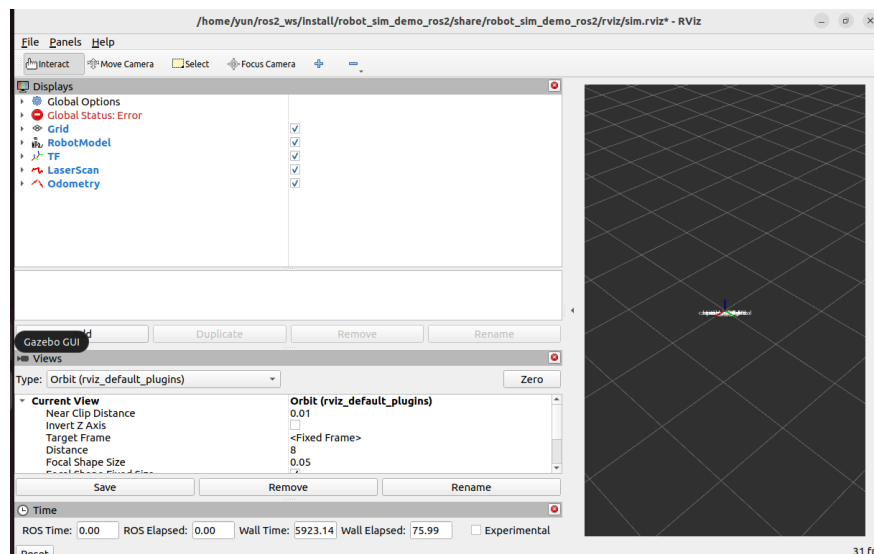


```
yun@yun-virtual-machine: ~/ros2_ws
Starting >>> tf_demo_cpp
Finished <<< tf_demo_cpp [0.71s]
Starting >>> tf_demo_py
Finished <<< tf_demo [3.12s]
Starting >>> tf_follower_ros2
Finished <<< tf_demo_py [2.89s]
Starting >>> topic_demo
Finished <<< tf_follower_ros2 [2.52s]
Starting >>> urdf_demo
Finished <<< topic_demo [2.95s]
Starting >>> urdf_demo_ros2
Finished <<< urdf_demo [2.89s]
Starting >>> xarm_moveit2_demo
Finished <<< urdf_demo_ros2 [3.10s]
Starting >>> xarm_vision_ros2
Help ed <<< xarm_moveit2_demo [2.90s]
Starting >>> action_demo
Finished <<< xarm_vision_ros2 [2.75s]
Starting >>> action_demo_cpp
Finished <<< action_demo_cpp [0.50s]
Starting >>> action_demo_py
Finished <<< action_demo [3.25s]
Starting >>> navigation_sim_demo_ros2
Finished <<< action_demo_py [3.04s]
Starting >>> sensor_pub
Finished <<< navigation_sim_demo_ros2 [3.01s]
Starting >>> service_demo_cpp
Finished <<< service_demo_cpp [0.41s]
Starting >>> service_demo_py
Finished <<< sensor_pub [2.94s]
Starting >>> slam_sim_demo_ros2
Finished <<< service_demo_py [2.83s]
Starting >>> topic_demo_cpp
Finished <<< topic_demo_cpp [0.41s]
Starting >>> topic_demo_py
Finished <<< slam_sim_demo_ros2 [3.36s]
Finished <<< topic_demo_py [2.53s]
Summary: 32 packages finished [39.7s]
```

- 截图 2: `ros2 pkg list | grep demo` 显示课程包列表

```
yun@yun-virtual-machine:~/ros2_ws$ source install/setup.bash
ros2 pkg list | grep demo
action_demo
action_demo_cpp
action_demo_interfaces
action_demo_py
demo_nodes_cpp
demo_nodes_cpp_native
demo_nodes_py
intra_process_demo
logging_demo
msgs_demo_interfaces
name_demo_cpp
navigation_sim_demo_ros2
param_demo
param_demo_cpp
param_demo_py
quality_of_service_demo_cpp
quality_of_service_demo_py
robot_sim_demo_ros2
ros_gz_sim_demos
service_demo
service_demo_cpp
service_demo_interfaces
service_demo_py
sim_demo
slam_sim_demo_ros2
tf_demo
tf_demo_cpp
tf_demo_py
topic_demo
topic_demo_cpp
topic_demo_interfaces
topic_demo_py
urdf_demo
urdf_demo_ros2
xarm_moveit2_demo
```

- 截图 3: RViz 中 XBot-U 机器人模型正常显示



常见问题

问题	原因	解决方法
`rosdep install` 报错	缺少依赖包	`sudo apt update && sudo apt install ros- humble-*`
编译失败 (找不到 gazebo)	未安装 Gazebo	`sudo apt install ros- humble-ros-gz`
RViz 无机器人模型	未 source setup.bash	`cd ~/ros2_course_ws`

source /opt/ros/humble/setup.bash source install/setup.bash ros2 launch
robot_sim_demo_ros2 sim_bringup.launch.py use_rviz:=true` |

练习 1.5：课程仿真启动与使用（约 15 分钟）

目标

启动 Gazebo 仿真环境，在 RViz 中添加激光雷达、机器人模型、摄像头等显示项。

步骤

步骤 1：启动 Gazebo 仿真（含界面）

```
# 加载课程工作空间
source ~/ros2_course_ws/install/setup.bash

# 启动 Gazebo + RViz 仿真
cd ~/ros2_course_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=true
# 期望：Gazebo 窗口 + RViz 窗口同时打开
# Gazebo 中显示 Museum 场景 + XBot-U 机器人
```

步骤 2：在 RViz 中添加激光雷达显示

- 左侧 "Displays" → Add → 选择 "LaserScan"
- 在新增的 LaserScan 项中设置 Topic: /scan
- 期望: 看到绿色激光扫描点组成的轮廓线

步骤 3: 添加机器人模型显示

- Displays → Add → 选择 "RobotModel"
- 期望: 看到 XBot-U 机器人 3D 模型

步骤 4: 添加摄像头显示 (需要启用深度相机)

- Displays → Add → 选择 "Camera"
- 在 Camera 项中设置 Topic: /camera/image_raw
- 期望: 看到摄像头实时画面

步骤 5: 键盘遥控测试

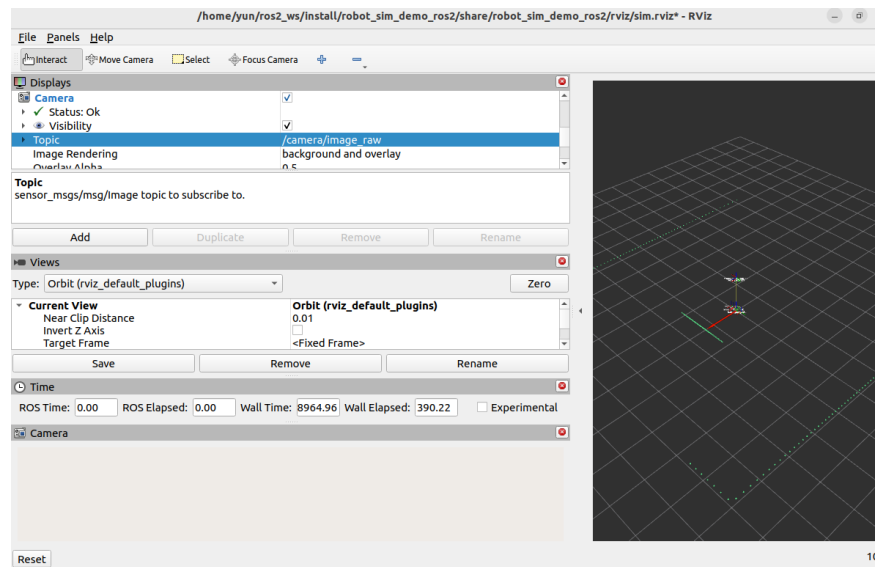
```
# 新开终端
source ~/ros2_course_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
# 按 i 前进, k 停止, j/l 左右转
# 观察 RViz 和 Gazebo 中机器人运动
```

✓ 验证:

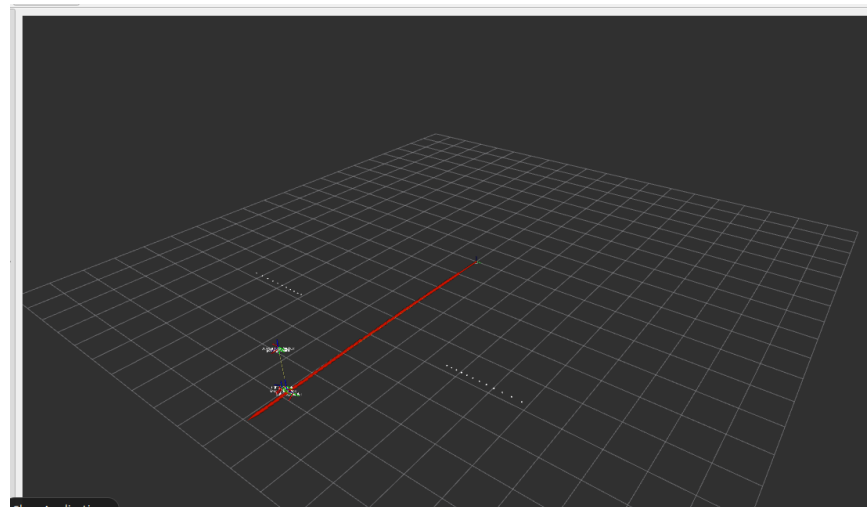
- 截图 1: Gazebo 窗口 (Museum 场景 + XBot-U 机器人)



- 截图 2: RViz 窗口 (RobotModel + LaserScan 正常显示)



- 截图 3: 键盘遥控后机器人位置改变



补充：验证激光雷达与深度相机话题

```
# 终端 1：启动 Gazebo、RViz、激光雷达和深度相机
cd ~/ros2_course_ws
source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=true \
  enable_fake_laser:=true \
  enable_depth_camera:=true

# 终端 2：桥接相机图像和 CameraInfo
```

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run ros_gz_bridge parameter_bridge \
  /rgbd_camera/image@sensor_msgs/msg/Image@gz.msgs.Image \
  /rgbd_camera/camera_info@sensor_msgs/msg/CameraInfo@gz.msgs.CameraInfo

# 终端 3 : 检查传感器话题及发布频率
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 topic list -t | grep -E "scan|rgbd|camera|image|depth|camera_info"
ros2 topic hz /scan
ros2 topic hz /rgbd_camera/image
ros2 topic hz /rgbd_camera/camera_info
```

- 在 RViz 中依次选择 Displays → Add → Image。
- 将 Topic 设置为 /rgbd_camera/image。

练习 1.6 : VS Code + ROS 2 插件编程 (约 15 分钟)

目标

配置 VS Code 开发环境，安装 ROS 扩展，创建、编译、调试 ROS 2 Python 节点。

步骤

步骤 1: 安装 VS Code 和 ROS 插件

```
# 安装 VS Code (如已安装可跳过)
sudo snap install code --classic

# 在 VS Code 扩展市场搜索安装 :
# 1. 使用“ros/ros2 by JaehyunShim”
# 2. "Python" (by Microsoft)
# 3. "Pylance" (by Microsoft)
```

步骤 2: 打开课程工作空间

```
cd ~/ros2_course_ws
code .

# 在 VS Code 中打开工作空间文件夹
```

步骤 3: 创建测试节点

- 在 VS Code 左侧文件树中，右键 src/ → 新建文件夹 hello_ros2
- 在 hello_ros2/ 下创建 hello_node.py:

```

#!/usr/bin/env python3
"""hello_ros2: VS Code 调试示例节点 - LifecyclePublisher + QoS + /cmd_vel"""

import rclpy
from rclpy.lifecycle import LifecycleNode, TransitionCallbackReturn
from geometry_msgs.msg import Twist

from rclpy.qos import (
    QoSProfile,
    HistoryPolicy,
    ReliabilityPolicy,
    DurabilityPolicy,
)

class HelloRos2Node(LifecycleNode):
    def __init__(self):
        super().__init__('hello_ros2_lifecycle')

        self.pub = None
        self.timer = None
        self.count = 0
        self.active = False

        self.get_logger().info('Lifecycle 节点已创建, 等待 configure。')

    def on_configure(self, state):
        self.get_logger().info('on_configure: 配置节点资源。')

        cmd_vel_qos = QoSProfile(
            history=HistoryPolicy.KEEP_LAST,
            depth=10,
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.VOLATILE,
        )
        self.pub = self.create_lifecycle_publisher(
            Twist,
            '/cmd_vel',
            cmd_vel_qos
        )

        self.timer = self.create_timer(0.5, self.timer_callback)

        self.get_logger().info('on_configure: LifecyclePublisher 和定时器创建完成。')

        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state):
        self.get_logger().info('on_activate: 激活节点, 允许发布 /cmd_vel。')

        ret = super().on_activate(state)

        if ret == TransitionCallbackReturn.SUCCESS:
            self.active = True
            self.get_logger().info('on_activate: 节点已激活。')

        return ret

```



```

def on_deactivate(self, state):
    self.get_logger().info('on_deactivate: 停用节点, 停止发布 /cmd_vel。')

    self.active = False

    ret = super().on_deactivate(state)

    return ret

def on_cleanup(self, state):
    self.get_logger().info('on_cleanup: 清理节点资源。')

    self.active = False

    if self.timer is not None:
        self.destroy_timer(self.timer)
        self.timer = None

    if self.pub is not None:
        try:
            self.destroy_lifecycle_publisher(self.pub)
        except AttributeError:
            self.destroy_publisher(self.pub)
        self.pub = None

    self.count = 0

    return TransitionCallbackReturn.SUCCESS

def timer_callback(self):
    if not self.active:
        return

    msg = Twist()
    msg.linear.x = 0.1
    msg.angular.z = 0.0

    self.pub.publish(msg)

    self.count += 1
    self.get_logger().info(
        f'第 {self.count} 次发布 /cmd_vel: '
        f'linear.x={msg.linear.x:.2f}, angular.z={msg.angular.z:.2f}'
    )

def main(args=None):
    rclpy.init(args=args)

    node = HelloRos2Node()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('用户中断, 节点退出。')
    finally:

```

```

        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

#ros2+LifecycleNode+Qos, 可以体现生命周期节点的配置、激活、停用和清理过程。

- 在 on_configure() 阶段创建 /cmd_vel 发布者和定时器;
- 在 on_activate() 阶段激活生命周期发布者, 开始周期发布速度控制消息;
- 在 on_deactivate() 阶段停止发布;
- 在 on_cleanup() 阶段销毁定时器和发布者资源。
- KEEP_LAST + depth=10: 保留最近 10 条消息;
- RELIABLE: 尽量保证控制消息可靠传输;
- VOLATILE: 不保存历史消息, 只向当前在线订阅者发送。

步骤 4: 配置调试 (launch.json)

- 手动创建 .vscode/launch.json:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "ROS: Debug hello_ros2",
      "type": "python",
      "request": "launch",
      "program": "${workspaceFolder}/src/hello_ros2/hello_node.py",
      "cwd": "${workspaceFolder}",
      "env": {
        "PYTHONPATH": "${workspaceFolder}/install/lib/python3.10/site-packages"
      }
    }
  ]
}

```

步骤 5: 断点调试

- 在 self.pub = self.create_lifecycle_publisher(...) ret = super().on_activate(state)self.pub.publish(msg)行左侧单击设置断点 (红点)
- 按 F5 启动调试,新建终端, 依次执行 ros2 lifecycle set /hello_ros2_lifecycle configure ros2 lifecycle set /hello_ros2_lifecycle activate

- 期望：程序在断点处暂停，可查看变量值、单步执行，节点激活后程序周期性进入 `timer_callback`，可以通过 `ros2 topic echo /cmd_vel` 验证话题是否正常发布

✓ 验证：

- 截图 1: VS Code 扩展列表 (ROS + Python 已安装)

Restricted Mode is intended for safe code browsing. Trust

2

EXTENSIONS

...

Search Extensions in Marketplace

6



Pylance

A performant, feature-rich langua...

Microsoft

3849ms



ROS

Software Development Toolset ar...

JaehyunShim



ROS2

Software Development Toolset ar...

JaehyunShim



Python

Python language support with ext...

Microsoft



Python Debugger

Python Debugger extension using ...

Microsoft



Python Environments

Provides a unified python environ...

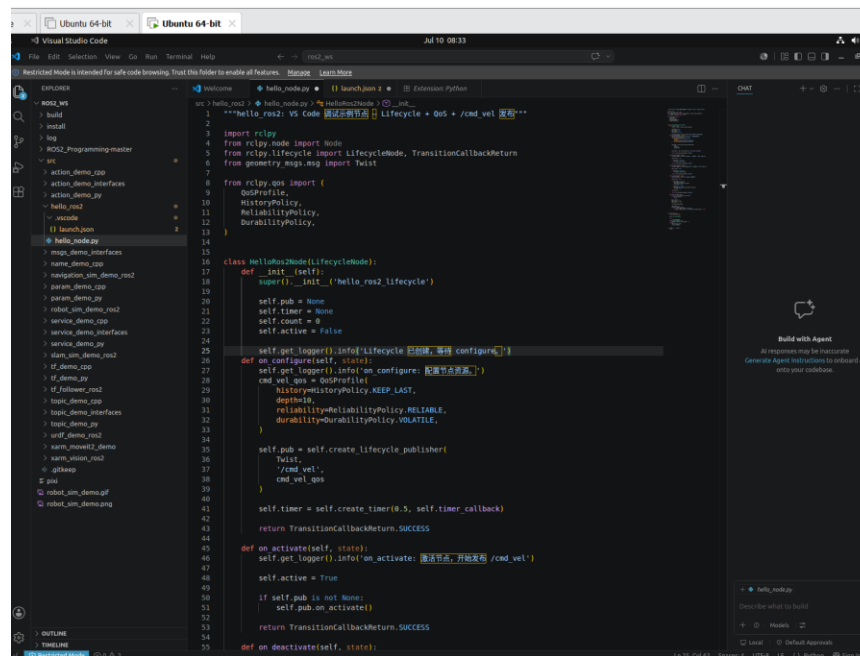
Microsoft

2

RECOMMENDED

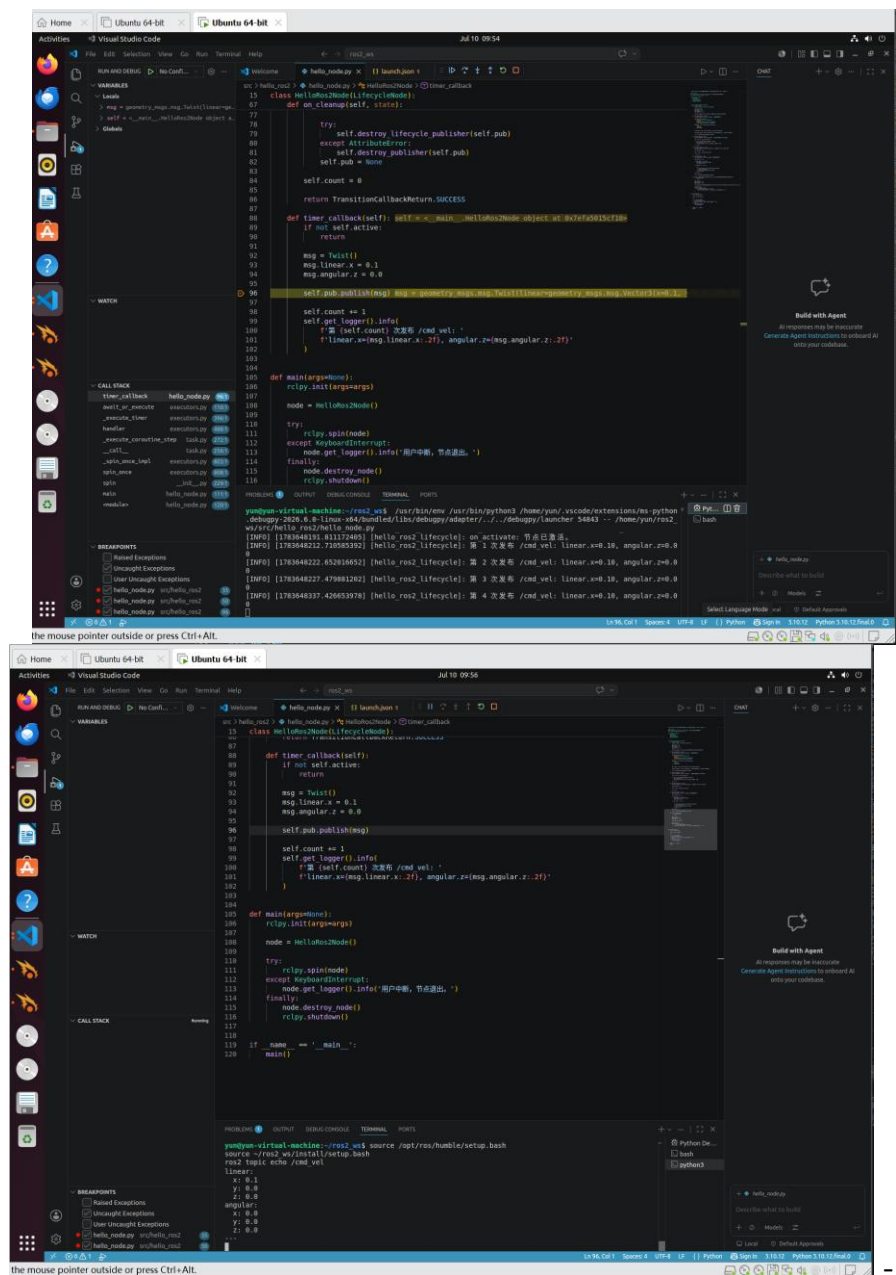
MCP SERVERS

- 截图 2: hello_node.py 代码编辑界面 (含代码补全提示)



```
1  """hello_ros2: VS Code 演示示例节点 Lifecycle + QoS + /cmd_vel 发布"""
2
3  import rcpppy
4  from rcpppy.node import Node
5  from lifecycle import LifecycleNode, TransitionCallbackReturn
6  from geometry_msgs.msg import Twist
7
8  from rcpppy.qos import (
9      QoSProfile,
10     HistoryPolicy,
11     ReliabilityPolicy,
12     DurabilityPolicy,
13 )
14
15 class HelloRos2Node(LifecycleNode):
16     def __init__(self):
17         super().__init__('hello_ros2_lifecycle')
18
19         self.pub = None
20         self.timer = None
21         self.count = 0
22         self.active = False
23
24     def configure(self, state):
25         self.get_logger().info('Lifecycle 已就绪, 等待 configure.')
26         self.get_logger().info('on configure: 配置节点资源。')
27         cmd_vel_qos = QoSProfile(
28             history=HistoryPolicy.KEEP_LAST,
29             depth=10,
30             reliability=ReliabilityPolicy.RELIABLE,
31             durability=DurabilityPolicy.VOLATILE,
32         )
33         self.pub = self.create_lifecycle_publisher(
34             Twist,
35             '/cmd_vel',
36             cmd_vel_qos,
37         )
38
39     self.timer = self.create_timer(0.5, self.timer_callback)
40     return TransitionCallbackReturn.SUCCESS
41
42 def on_activate(self, state):
43     self.get_logger().info('on activate: 激活节点, 开始发布 /cmd_vel')
44     self.active = True
45
46     if self.pub is not None:
47         self.pub.on_activate()
48     return TransitionCallbackReturn.SUCCESS
49
50 def on_deactivate(self, state):
51     pass
```


-



本章实验总结

练习	核心技能	时长
1.1	ROS 2 安装验证	15min
1.2	工作空间 + colcon 构建	15min
1.3	DDS 域 ID 通信隔离	15min
1.4	课程源码包编译与运行	15min

第 2 章 : ROS 2 核心编程基础

课程: ROS2 Python 编程

章节: 第 2 章

课时: 2 课时 (90 分钟)

教学方式: 讲授 + 演示

2.1 创建 Python 包与节点

知识点 2.1.1 : ROS 2 Python 包结构

ROS 2 Python 包使用 ament_python 构建系统，标准目录结构如下：

```
my_robot_pkg/                                # 包根目录
├─ package.xml                               # 包元数据（名称、版本、依赖）
├─ setup.py                                  # 安装脚本
├─ setup.cfg                                  # 安装配置
├─ resource/
│   └─ my_robot_pkg                          # ament index 标记文件
├─ my_robot_pkg/                             # Python 源码
│   ├── __init__.py
│   └─ my_node.py                            # 节点代码
├─ launch/                                   # Launch 文件目录
│   └─ demo.launch.py
└─ test/                                     # 测试代码
    ├── __init__.py
    └─ test_my_node.py
```

图 2-1: ROS 2 Python 包的标准目录结构。

知识点 2.1.2 : package.xml 配置

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_robot_pkg</name>
  <version>0.1.0</version>
  <description>My first ROS 2 Python package</description>
  <maintainer email="student@example.com">Student Name</maintainer>
  <license>Apache-2.0</license>

  <!-- 运行时依赖 -->
  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <!-- 测试依赖 -->
  <test_depend>ament_copyright</test_depend>
```

```

<test_depend>ament_flake8</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

知识点 2.1.3 : setup.py 安装入口

```

from setuptools import find_packages, setup
import os
from glob import glob

package_name = 'my_robot_pkg'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'),
         glob(os.path.join('launch', '*launch.py'))),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student Name',
    maintainer_email='student@example.com',
    description='My first ROS 2 Python package',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'my_node = my_robot_pkg.my_node:main',
        ],
    },
)

```

****注意****: `entry\_points` 是关键配置, 它将 Python 函数注册为 ROS 2 可执行文件。
格式为: `可执行文件名 = 包名.模块名:函数名`

2.2 编写第一个 ROS 2 Python 节点

知识点 2.2.1 : 最小节点模板

```

#!/usr/bin/env python3
import rclpy                                # ROS 2 Python 客户端库
from rclpy.node import Node                 # 节点基类

class MyFirstNode(Node):

```

```

"""我的第一个 ROS 2 Python 节点"""

def __init__(self):
    super().__init__('my_first_node') # 节点名称

    # 设置定时器，每秒调用一次 timer_callback
    self.timer = self.create_timer(1.0, self.timer_callback)

    self.count = 0 # 计数器

    self.get_logger().info('节点已启动!') # 打印日志

def timer_callback(self):
    """定时器回调函数"""

    self.count += 1
    self.get_logger().info(f'Hello ROS 2! Count: {self.count}')

def main(args=None):
    rclpy.init(args=args) # 初始化 ROS 2

    node = MyFirstNode() # 创建节点实例

    try:
        rclpy.spin(node) # 进入事件循环（阻塞）
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node() # 销毁节点
        rclpy.shutdown() # 关闭 ROS 2

if __name__ == '__main__':
    main()

```

程序 2-1：ROS 2 Python 最小节点模板。main 函数遵循“初始化→创建→spin→销毁→关闭”的标准流程。

知识点 2.2.2：节点名称与命名空间

```

# 方式 1：直接在构造函数中设置
newnode = rclpy.create_node('node_name', namespace='my_ns')

# 方式 2：通过命令行参数重映射
# ros2 run my_pkg my_node --ros-args -r __ns:=/new_ns -r __node:=new_name

# 方式 3：在 Node 构造函数中指定
class MyNode(Node):
    def __init__(self):
        super().__init__('node_name', namespace='my_ns')

```

****命名规则****：节点名称只能包含字母、数字和下划线，不能包含空格。完整节点标识 = `{namespace}/{node_name}`，如 `/my_ns/node_name`。

知识点 2.2.3 : 日志系统

ROS 2 提供分级日志系统，优先级从低到高：

日志级别	Python API	用途
DEBUG	<code>`get_logger().debug()`</code>	调试信息（默认不输出）
INFO	<code>`get_logger().info()`</code>	一般信息
WARN	<code>`get_logger().warn()`</code>	警告信息
ERROR	<code>`get_logger().error()`</code>	错误信息
FATAL	<code>`get_logger().fatal()`</code>	致命错误

```
# 设置日志级别
self.get_logger().set_level(rclpy.logging.LoggingSeverity.DEBUG)

# 节流日志（避免高频输出中刷屏）
self.get_logger().info('此消息每秒最多输出一次',
    throttle_duration_sec=1)

# 一次性日志（整个节点生命周期内只输出一次）
self.get_logger().info('此消息只输出一次', once=True)
```

2.3 开发工具链

知识点 2.3.1 : 命令行工具

```
# 构建与运行
colcon build --packages-select my_robot_pkg --symlink-install
source install/setup.bash
ros2 run my_robot_pkg my_node

# 查看节点
ros2 node list                # 列出所有节点
ros2 node info /my_first_node # 查看节点详情

# 查看话题
ros2 topic list                # 列出所有话题
ros2 topic echo /topic_name    # 监听话题消息
ros2 topic info /topic_name    # 查看话题信息
ros2 topic hz /topic_name      # 查看话题频率
```

```
# 查看包信息
ros2 pkg list                                # 列出所有包
ros2 pkg xml my_robot_pkg                    # 查看包 XML
```

知识点 2.3.2 : rqt_graph 可视化

```
# 启动 rqt_graph 可视化节点通信图
rqt_graph
# 或通过 ros2 命令启动
ros2 run rqt_graph rqt_graph
```

图 2-2: rqt_graph 界面示意图。安装命令: `sudo apt install ros-humble-rqt-graph`。

- 椭圆: 节点
- 矩形: 话题
- 连线: 发布/订阅关系
- 勾选 "Nodes/Topics (all)" 查看完整通信图

知识点 2.3.3 : RViz2 可视化

```
# 启动 RViz2
rviz2
# 或指定配置文件
rviz2 -d /path/to/config.rviz
```

图 2-3: RViz2 界面布局。安装命令: `sudo apt install ros-humble-rviz2`。

左侧面板: Displays (显示项配置), 可添加 RobotModel、TF、LaserScan 等显示项。右侧面板: Views (视图控制), 支持 Orbit、TopDownOrtho 等多种视角。

2.4 本章小结

本章核心知识点回顾:

13. ROS 2 Python 包标准结构包含 package.xml (元数据)、setup.py (安装入口) 和源码目录
14. entry_points 配置将 Python 函数映射为 ROS 2 可执行命令
15. 节点遵循"init → create_node → spin → destroy → shutdown"的生命周期

16. 日志系统提供 DEBUG/INFO/WARN/ERROR/FATAL 五个级别，支持节流和一次性输出

17. `ros2 node/cli/topic/pkg` 命令行工具是调试 ROS 2 系统的核心手段

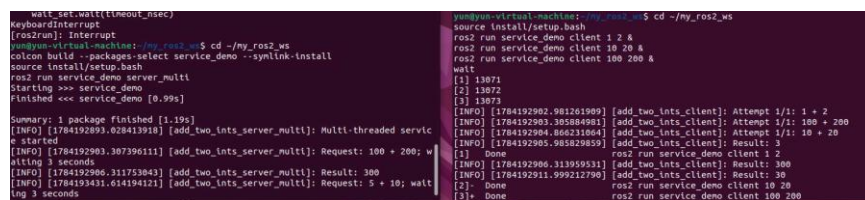
18. `rqt_graph` 可视化节点通信拓扑，`RViz2` 可视化机器人传感器和执行器数据

2.5 练习题

练习 2.1: 创建一个名为 `hello_pkg` 的 ROS 2 Python 包，包含一个名为 `hello_node` 的节点，每秒输出一句问候语。3.1

练习 2.2: 修改练习 2.1 的节点，添加日志级别参数 `--ros-args --log-level DEBUG`，观察不同日志级别的输出差异。

练习 2.3: 运行两个 `hello_node` 节点，使用 `--ros-args -r __node:=hello2` 重命名第二个节点，用 `ros2 node list` 验证。（从 3.2 里改）
`ros2 run hello_pkg hello_node ros2 run hello_pkg hello_node --ros-args -r __node:=hello2 ros2 node list`



```

[ros2run]: Interrupt
KeyboardInterrupt
yanyan@virtual-machine: /my_ros2_ws$ cd ~/my_ros2_ws
colcon build --packages-select service_demo --symlink-install
source install/setup.bash
ros2 run service_demo server_multi
Starting >>> service_demo
Finished <<< service_demo [0.99s]

Summary: 1 package finished [1.19s]
[INFO] [1784192893.028413918] [add_two_ints_server_multi]: Multi-threaded service started
[INFO] [1784192903.387396111] [add_two_ints_server_multi]: Request: 100 + 200; waiting 3 seconds
[INFO] [1784192906.311753043] [add_two_ints_server_multi]: Result: 300
[INFO] [1784193431.614194121] [add_two_ints_server_multi]: Request: 5 + 10; waiting 3 seconds

yanyan@virtual-machine: /my_ros2_ws$ cd ~/my_ros2_ws
source install/setup.bash
ros2 run service_demo client 1 2 &
ros2 run service_demo client 10 20 &
ros2 run service_demo client 100 200 &
wait
[1] 13871
[2] 13872
[3] 13873
[INFO] [1784192902.981201809] [add_two_ints_client]: Attempt 1/1: 1 + 2
[INFO] [1784192903.305804981] [add_two_ints_client]: Attempt 1/1: 10 + 200
[INFO] [1784192904.866231064] [add_two_ints_client]: Attempt 1/1: 10 + 20
[INFO] [1784192905.965029059] [add_two_ints_client]: Result: 3
[1] Done
[INFO] [1784192906.313959531] [add_two_ints_client]: Result: 300
[INFO] [1784192911.999212790] [add_two_ints_client]: Result: 30
[2] Done
[3] Done
ros2 run service_demo client 100 200
```

练习 2.4: 使用 `ros2 node info` 查看 `talker` 节点的发布者和订阅者信息，画出其通信拓扑图。3.3

练习 2.5: 启动 `demo_nodes_py` `talker` 和 `listener`，使用 `rqt_graph` 可视化节点通信关系。3.3

练习 2.6: 安装并启动 `RViz2`，加载默认配置，认识 `RViz2` 的显示面板和视角控制。3.3

实验 2:

第 2 章 实验指导书：Python 节点编程与工具链

****实验课时****: 2 课时 (90 分钟) ****实验平台****: Ubuntu 22.04 + ROS 2 Humble

实验目标

完成本实验后，学员应能够：

19. 独立创建 ROS 2 Python 包和节点
20. 配置 setup.py 中的 entry_points
21. 使用日志系统进行程序调试
22. 使用 ros2 命令行工具查看系统状态
23. 使用 rqt_graph 和 RViz2 可视化工具

练习 3.1：创建并运行 Python 节点（约 30 分钟）

目标

创建一个完整的 Python 节点，通过定时器周期输出日志。

步骤

步骤 1：创建 Python 包

```
cd ~/my_ros2_ws/src
ros2 pkg create hello_pkg --build-type ament_python \
  --dependencies rclpy
```

步骤 2：编写节点代码

在 ~/my_ros2_ws/src/hello_pkg/hello_pkg/ 目录下创建 hello_node.py：

```
#!/usr/bin/env python3
"""hello_node: 第一个 ROS2 Python 节点 - 周期输出问候语和计时"""
import rclpy
from rclpy.node import Node

class HelloNode(Node):
    """周期输出问候语的节点 - 演示定时器和日志功能"""

    def __init__(self):
        # 初始化节点, 名称为 hello_node
        super().__init__('hello_node')
        # 创建 1 Hz 定时器, 回调 timer_callback
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.count = 0                # 计数器
        # 启动日志 (仅输出一次)
        self.get_logger().info('HelloNode 已启动!', once=True)

    def timer_callback(self):
        """定时器回调 - 每 1 秒输出带计数器的日志"""
        self.count += 1
        self.get_logger().info(f'Hello ROS 2! 计数: {self.count}',
                               throttle_duration_sec=1)

def main(args=None):
    rclpy.init(args=args)           # ① 初始化 ROS 2
    node = HelloNode()              # ② 创建节点实例
    try:
        rclpy.spin(node)            # ③ 进入事件循环 (阻塞)
    except KeyboardInterrupt:
        pass                         # ④ 捕获 Ctrl+C 正常退出
    finally:
        node.destroy_node()         # ⑤ 销毁节点释放资源
        rclpy.shutdown()            # ⑥ 关闭 ROS 2

if __name__ == '__main__':
    main()
```

步骤 3: 配置 setup.py

编辑 `~/my_ros2_ws/src/hello_pkg/setup.py`, 在 `entry_points` 中添加:

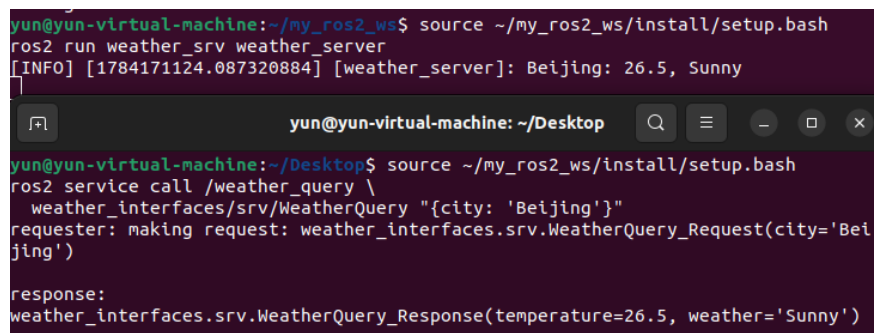
```
entry_points={
    'console_scripts': [
        'hello_node = hello_pkg.hello_node:main',
        # ↑ 可执行命令      ↑ 模块路径:函数名
    ],
},
```

步骤 4: 编译并运行

```
cd ~/my_ros2_ws
colcon build --packages-select hello_pkg --symlink-install
source install/setup.bash
ros2 run hello_pkg hello_node
# 期望输出: [INFO] HelloNode 已启动!
#          [INFO] Hello ROS 2! 计数: 1
#          [INFO] Hello ROS 2! 计数: 2
```

步骤 4: 检查运行结果与图示一致

- 截图 1: colcon build 输出显示编译成功

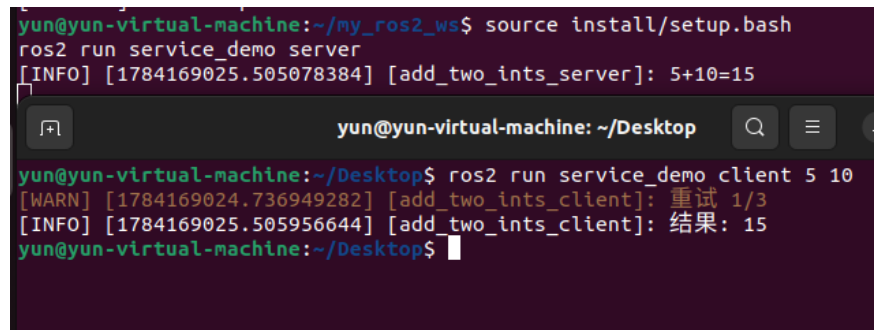


```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run weather_srv weather_server
[INFO] [1784171124.087320884] [weather_server]: Beijing: 26.5, Sunny

yun@yun-virtual-machine:~/Desktop$ source ~/my_ros2_ws/install/setup.bash
ros2 service call /weather_query \
  weather_interfaces/srv/WeatherQuery "{city: 'Beijing'}"
requester: making request: weather_interfaces.srv.WeatherQuery_Request(city='Beijing')

response:
weather_interfaces.srv.WeatherQuery_Response(temperature=26.5, weather='Sunny')
```

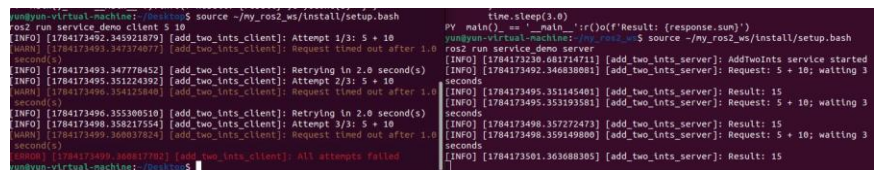
- 截图 2: ros2 run 输出显示周期日志



```
yun@yun-virtual-machine:~/my_ros2_ws$ source install/setup.bash
ros2 run service_demo server
[INFO] [1784169025.505078384] [add_two_ints_server]: 5+10=15

yun@yun-virtual-machine:~/Desktop$ ros2 run service_demo client 5 10
[WARN] [1784169024.736949282] [add_two_ints_client]: 重试 1/3
[INFO] [1784169025.505956644] [add_two_ints_client]: 结果: 15
yun@yun-virtual-machine:~/Desktop$
```

- 截图 3: ros2 node list 显示 /hello_node



```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client 5 10
[INFO] [1784173492.345921879] [add_two_ints_client]: Attempt 1/3: 5 + 10
[WARN] [1784173499.349794077] [add_two_ints_client]: Request timed out after 1.0 seconds
[INFO] [1784173493.347778432] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173495.35224392] [add_two_ints_client]: Attempt 2/3: 5 + 10
[WARN] [1784173496.354125840] [add_two_ints_client]: Request timed out after 1.0 seconds
[INFO] [1784173496.355300510] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173498.358217554] [add_two_ints_client]: Attempt 3/3: 5 + 10
[WARN] [1784173499.360837824] [add_two_ints_client]: Request timed out after 1.0 seconds
[ERROR] [1784173499.360817702] [add_two_ints_client]: All attempts failed
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 node list
add_two_ints_server
add_two_ints_client
hello_node
time.sleep(3.0)
PY main() = '__main__':r()o(f'Result: (response.sum)')
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server
[INFO] [1784173230.681714711] [add_two_ints_server]: AddTwoInts service started
[INFO] [1784173492.346830081] [add_two_ints_server]: Request: 5 + 10; waiting 3 seconds
[INFO] [1784173495.351145401] [add_two_ints_server]: Result: 15
[INFO] [1784173495.353193581] [add_two_ints_server]: Request: 5 + 10; waiting 3 seconds
[INFO] [1784173498.357272473] [add_two_ints_server]: Result: 15
[INFO] [1784173498.359149800] [add_two_ints_server]: Request: 5 + 10; waiting 3 seconds
[INFO] [1784173501.363688305] [add_two_ints_server]: Result: 15
```

参考代码

完整参考代码位于 'lab_code/ch02_lab/hello_pkg/'

练习 3.2：日志系统实验（约 30 分钟）

目标

掌握 ROS 2 日志系统，测试不同日志级别和节流功能。

步骤

步骤 1：创建带日志功能的节点

在 hello_pkg/hello_pkg/ 下创建 logger_demo.py：

```
#!/usr/bin/env python3
"""logger_demo: 演示 ROS2 日志系统的分级输出、节流和一次性输出"""
import rclpy
from rclpy.node import Node

class LoggerDemoNode(Node):
    """展示日志系统全部特性"""

    def __init__(self):
        super().__init__('logger_demo')
        # 设置日志级别为 DEBUG（默认仅输出 INFO 及以上）
        self.get_logger().set_level(rclpy.logging.LoggingSeverity.DEBUG)
        self.timer = self.create_timer(1.0, self.log_all_levels)
        self.counter = 0

    def log_all_levels(self):
        """输出各级别日志"""
        self.counter += 1
        self.get_logger().debug(f'DEBUG 消息 #{self.counter}')
        self.get_logger().info(f'INFO 消息 #{self.counter}')
        self.get_logger().warn(f'WARN 消息 #{self.counter}')

        # 仅输出一次
        if self.counter == 3:
            self.get_logger().error('ERROR: 第 3 次出现异常!', once=True)

        # 节流：每秒最多输出 1 次
        if self.counter >= 5:
            self.get_logger().warn(
                '高频警告—已节流', throttle_duration_sec=2)

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(LoggerDemoNode())
    rclpy.shutdown()
```

```
if __name__ == '__main__':  
    main()
```

步骤 2: 配置 entry_points 并编译

在 setup.py 的 entry_points 中添加:

```
'logger_demo = hello_pkg.logger_demo:main',
```

步骤 3: 运行并观察日志

```
cd ~/my_ros2_ws  
colcon build --packages-select hello_pkg --symlink-install  
source install/setup.bash  
ros2 run hello_pkg logger_demo  
# 观察各级别日志输出  
# Ctrl+C 停止
```

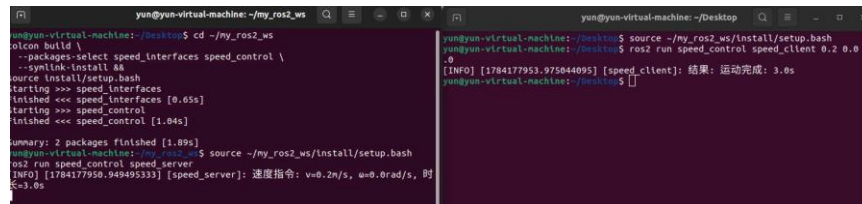
步骤 4: 使用命令行参数修改日志级别 (要先在 setup.py 中注释掉#

self.get_logger().set_level(rclpy.logging.LoggingSeverity.DEBUG))

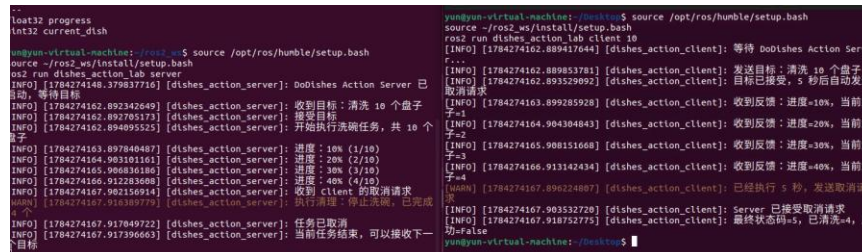
```
# 仅输出 ERROR 级日志  
ros2 run hello_pkg logger_demo --ros-args --log-level ERROR  
# 期望: 仅看到 ERROR 输出  
  
# 指定节点日志级别  
ros2 run hello_pkg logger_demo --ros-args \  
    --log-level logger_demo:=WARN  
# 期望: 仅看到 WARN 及以上输出
```

步骤 4: 检查运行结果与图示一致

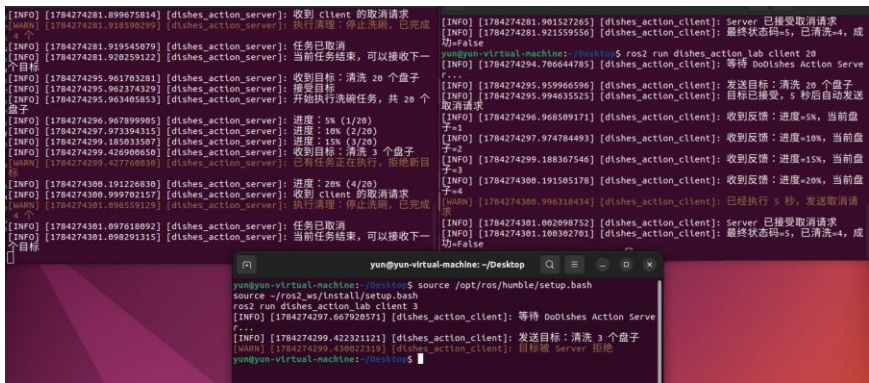
- 截图 1: 默认日志输出 (DEBUG ~ ERROR)



- 截图 2: --log-level ERROR 仅显示错误日志



- 截图 3: rqt_console 输出 (ros2 run rqt_console rqt_console)



参考代码

完整参考代码位于 `'lab_code/ch02_lab/hello_pkg/hello_pkg/logger_demo.py'`

练习 3.3 : 可视化工具使用 (约 30 分钟)

目标

使用 rqt_graph 和 RViz2 查看 ROS 2 系统状态。

步骤

步骤 1: 启动 talker 和 listener 两个节点

```
# 终端 1
ros2 run demo_nodes_py talker

# 终端 2
ros2 run demo_nodes_py listener
```

步骤 2: 启动 rqt_graph

```
# 终端 3
rqt_graph
```

- 在 rqt_graph 界面中, 观察节点 (椭圆) 与话题 (矩形) 的拓扑关系
- 勾选 "Nodes/Topics (all)" 查看完整图
- 截图保存拓扑图

步骤 3: 查看节点信息

```
# 终端 3
ros2 node info /talker

# 期望输出：
# Subscribers: /parameter_events
# Publishers: /chatter: std_msgs/msg/String

ros2 node info /listener
# 期望输出：
# Subscribers: /chatter: std_msgs/msg/String
# Publishers: /parameter_events
```

步骤 4: 启动 RViz2

rviz2

- 左侧 "Displays" → Add → "By topic", 选择话题添加显示
- 理解 RViz2 的 Displays 面板和 Views 面板

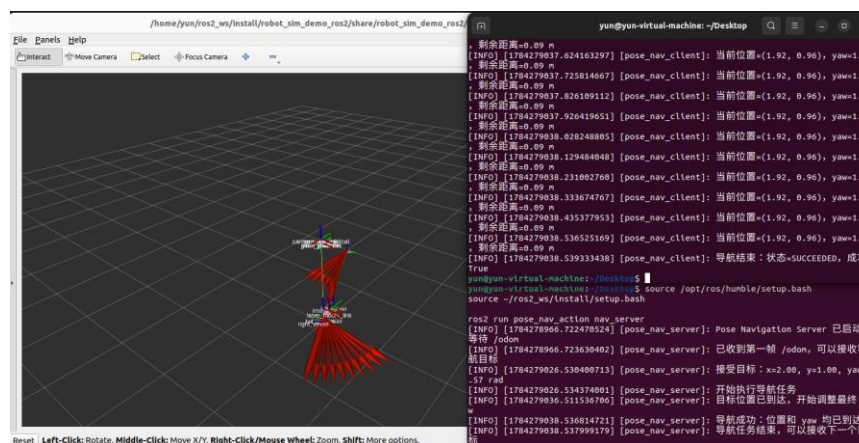
步骤 4: 检查运行结果与图示一致

- 截图 1: rqt_graph 中的节点-话题拓扑图

```
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_server
[INFO] [1784278966.722470524] [pose_nav_server]: Pose Navigation Server 已启动,
等待 /odom
[INFO] [1784278966.723630402] [pose_nav_server]: 已收到第一帧 /odom, 可以接收导
航目标
```

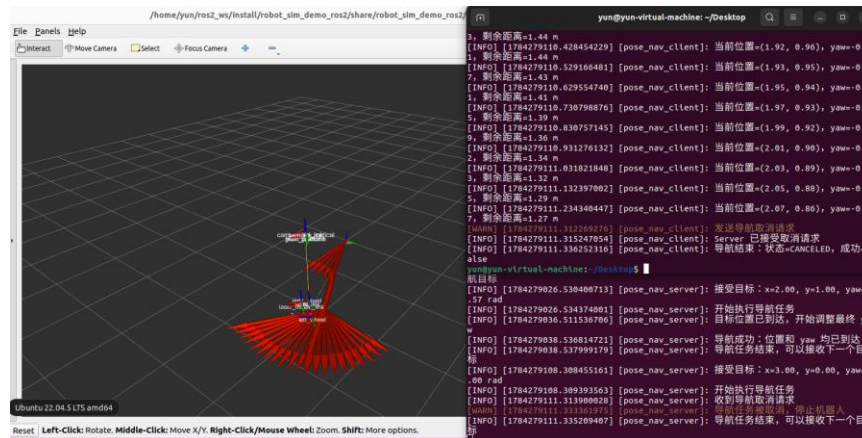
- 截图 2: ros2 node info /talker 输出



```
yun@yun-virtual-machine:~/Desktop$ ros2 node info /talker
[INFO] [1784279037.624163297] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.38
, 剩余距离=0.09 m
[INFO] [1784279037.725814667] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.34
, 剩余距离=0.09 m
[INFO] [1784279037.826109512] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.38
, 剩余距离=0.09 m
[INFO] [1784279037.926419651] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.42
, 剩余距离=0.09 m
[INFO] [1784279038.028248805] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.44
, 剩余距离=0.09 m
[INFO] [1784279038.129484048] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.47
, 剩余距离=0.09 m
[INFO] [1784279038.231092768] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.49
, 剩余距离=0.09 m
[INFO] [1784279038.333074707] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.50
, 剩余距离=0.09 m
[INFO] [1784279038.435377953] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.51
, 剩余距离=0.09 m
[INFO] [1784279038.536525169] [pose_nav_client]: 当前位置=(-1.92, 0.96), yaw=1.52
, 剩余距离=0.09 m
[INFO] [1784279038.539333438] [pose_nav_client]: 导航结束: 状态=SUCCEEDED, 成功:
True
yun@yun-virtual-machine:~/Desktop$
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_server
[INFO] [1784278966.722470524] [pose_nav_server]: Pose Navigation Server 已启动,
等待 /odom
[INFO] [1784278966.723630402] [pose_nav_server]: 已收到第一帧 /odom, 可以接收导
航目标
[INFO] [1784279026.530400713] [pose_nav_server]: 接受目标: x=2.00, y=1.00, yaw=1
.57 rad
[INFO] [1784279026.534376081] [pose_nav_server]: 开始执行导航任务
[INFO] [1784279036.511536706] [pose_nav_server]: 目标位置已到达, 开始调整最终 ya
w
[INFO] [1784279038.536814721] [pose_nav_server]: 导航成功: 位置和目标 yaw 均已到达
[INFO] [1784279038.537995179] [pose_nav_server]: 导航任务结束, 可以接收下一个目
标
```


- 截图 3: RViz2 启动界面



思考题

24. rqt_graph 中哪些是节点？哪些是话题？如何区分？椭圆表述节点，矩形表示话题，用箭头链接，节点 → 话题表示发布，话题 → 节点表示订阅
25. --log-level 参数的默认值是多少？（练习 3.1 步骤 1）默认参数是 info，ROS2 默认会输出 INFO、WARN、ERROR 和 FATAL 级别的日志，不会输出 DEBUG 日志
26. 如果 entry_points 未正确配置，运行时会出现什么错误？（练习 3.1 步骤 3、4）
entry_points 的作用是告诉 ROS2 运行哪个 Python 文件中的哪个函数作为程序入口。执行 ros2 run 时通常会提示 No executable found，如果模块或函数配置错误，还可能出现 ModuleNotFoundError 或 AttributeError

练习 4：节点与仿真交互 — 订阅 /odom 查看机器人位置（约 15 分钟）

目标

编写一个订阅者节点，订阅 XBot-U 仿真的 /odom 话题，实时输出机器人位置和速度。

步骤

步骤 1：启动课程仿真

```
source ~/ros2_course_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py use_rviz:=true
```


步骤 2: 创建 odom_monitor.py

```
#!/usr/bin/env python3
"""odom_monitor: 监听 XBot-U /odom 话题, 实时显示机器人位置"""
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry

class OdomMonitor(Node):
    def __init__(self):
        super().__init__('odom_monitor')
        # 订阅 /odom 话题 (XBot-U 里程计数据)
        self.sub = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10)

    def odom_callback(self, msg):
        # 提取位置 (x, y) 和姿态 (yaw)
        pos = msg.pose.pose.position
        orient = msg.pose.pose.orientation
        # 计算偏航角 (简化取 z * 2)
        yaw = 2.0 * orient.z if orient.z < 1.0 else 0.0
        self.get_logger().info(
            f'XBot-U 位置: x={pos.x:.2f}m, y={pos.y:.2f}m, 航向={yaw:.2f}rad')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(OdomMonitor())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

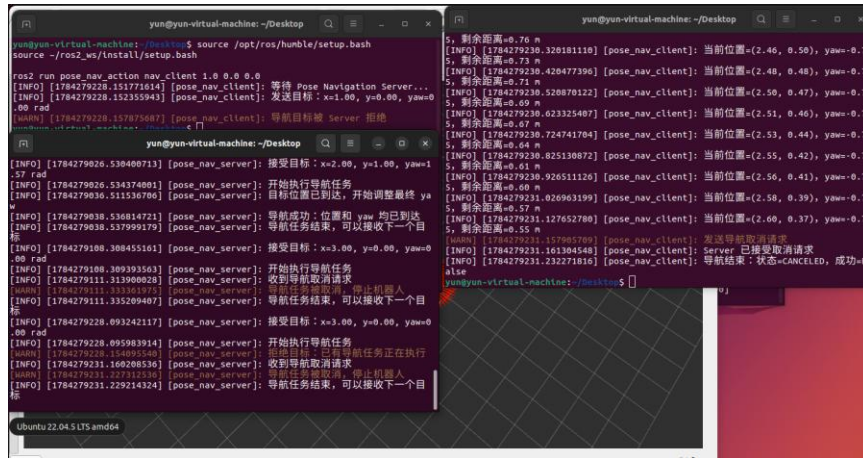
步骤 3: 运行测试

```
# 终端 1: 仿真已启动

# 终端 2: 启动 odom_monitor
cd ~/my_ros2_ws
source /opt/ros/humble/setup.bash
colcon build --packages-select hello_pkg --symlink-install
source install/setup.bash
ros2 run hello_pkg odom_monitor # 需要先添加到 setup.py 'odom_monitor =
hello_pkg.odom_monitor:main'

# 终端 3: 发送控制指令改变机器人位置
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
    "{linear: {x: 0.2}, angular: {z: 0.5}}" -r 10
```

✓ 验证：odom_monitor 终端实时输出机器人位置和航向变化。

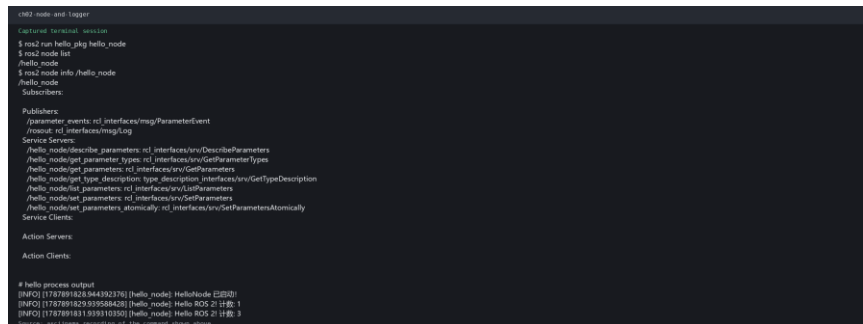


思考题

27. /odom 中的 pose.pose.orientation 使用四元数表示姿态，如何转换为欧拉角？使用 TF2 转换为欧拉角 (Roll、Pitch、Yaw)，其中 Yaw 是航向角
28. 如何利用 /odom 数据计算机人的行驶总距离？每次接收 /odom 消息时，读取当前位置 (x, y)，与上一时刻的位置计算两点间距离 并将每次位移累加，得到机器人的总行驶距离。

实际运行证据

真实运行的 Python 节点、节点列表和节点信息输出：



原始录制：[ch02 nodes.cast](#)。完整证据索引见[实际运行证据](#)。

第 3 章：话题通信 (Topics)

****课程**:** ROS2 Python 编程 ****章节**:** 第 3 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

3.1 发布-订阅模型原理

知识点 3.1.1：话题通信架构

话题通信是 ROS 2 中最基本的通信方式，采用**异步、多对多**的发布-订阅模式：

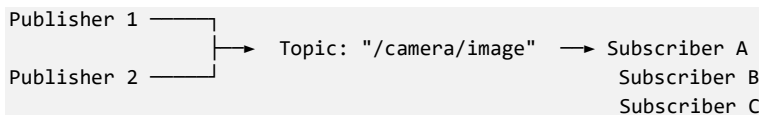


图 3-1：话题通信的多对多发布-订阅模型。发布者与订阅者完全解耦，互不知晓对方的存在。

知识点 3.1.2：Python Publisher API

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String          # 标准消息类型

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        # 创建发布者：话题名 "chatter"，队列深度 10
        self.publisher = self.create_publisher(
            String,                    # 消息类型
            'chatter',                # 话题名称
            10)                       # QoS 队列深度

        # 创建定时器：每 0.5 秒发布一次
        self.timer = self.create_timer(0.5, self.timer_callback)
        self.count = 0

    def timer_callback(self):
        msg = String()                # 创建消息对象
        msg.data = f'Hello ROS 2: {self.count}' # 设置消息内容
        self.publisher.publish(msg)      # 发布消息
        self.get_logger().info(f'发布: "{msg.data}")')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
```

```
rclpy.spin(TalkerNode())
rclpy.shutdown()
```

程序 3-1：Python Publisher 完整示例。create_publisher 的三个核心参数：消息类型、话题名称、QoS 队列深度。

知识点 3.1.3 : Python Subscriber API

```
from std_msgs.msg import String

class ListenerNode(Node):
    def __init__(self):
        super().__init__('listener')
        # 创建订阅者：话题名 "chatter", 队列深度 10
        self.subscription = self.create_subscription(
            String,                    # 消息类型
            'chatter',                # 话题名称
            self.listener_callback,   # 回调函数
            10)                       # QoS 队列深度

    def listener_callback(self, msg):
        """收到消息时自动调用此函数"""
        self.get_logger().info(f'收到: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(ListenerNode())
    rclpy.shutdown()
```

程序 3-2：Python Subscriber 完整示例。create_subscription 的回调函数签名必须为 callback(msg), msg 为消息对象。

3.2 标准消息类型

知识点 3.2.1 : 常用标准消息

ROS 2 提供丰富的标准消息类型，按功能包分类：

std_msgs/	# 基础类型
├ String	# 字符串 (data: string)
├ Int32, Int64	# 整数
├ Float32, Float64	# 浮点数
├ Bool	# 布尔值
├ Empty	# 空消息 (用作信号)
└ Header	# 标准头 (stamp: time, frame_id: string)

```

sensor_msgs/          # 传感器消息
├─ Image              # 图像 (rgb, depth)
├─ LaserScan          # 激光雷达扫描
├─ PointCloud2        # 3D 点云
├─ Imu                # IMU 数据
└─ Joy               # 手柄数据

geometry_msgs/        # 几何消息
├─ Twist              # 速度指令 (linear + angular)
├─ Pose               # 位姿 (position + orientation)
├─ Vector3            # 三维向量
└─ Quaternion         # 四元数

```

知识点 3.2.2 : 查看消息定义

```

# 查看消息类型定义
ros2 interface show std_msgs/msg/String
# 输出: string data

ros2 interface show geometry_msgs/msg/Twist
# 输出:
# Vector3 linear
#   float64 x
#   float64 y
#   float64 z
# Vector3 angular
#   float64 x
#   float64 y
#   float64 z

# 查看消息属性
ros2 interface proto std_msgs/msg/String
# 输出: string data

```

3.3 自定义消息接口

知识点 3.3.1 : 创建自定义 .msg 文件

自定义消息接口需要单独创建一个包（仅支持 CMake 构建）：

```

custom_interfaces/
├─ CMakeLists.txt
├─ package.xml
└─ msg/
    └─ SensorData.msg    # 自定义消息定义文件

# CMakeLists.txt 关键配置
cmake_minimum_required(VERSION 3.8)
project(custom_interfaces)

```

```
find_package(rosidl_default_generators REQUIRED)

# 声明消息文件
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/SensorData.msg"
)

# msg/SensorData.msg 内容
float64 temperature    # 温度 (°C)
float64 humidity       # 湿度 (%)
float64 pressure       # 气压 (hPa)
string device_id       # 设备 ID
```

知识点 3.3.2 : 在 Python 中使用自定义消息

```
# 在 package.xml 中添加依赖
# <exec_depend>custom_interfaces</exec_depend>

from custom_interfaces.msg import SensorData

# 创建并发布自定义消息
msg = SensorData()
msg.temperature = 25.5
msg.humidity = 60.0
msg.pressure = 1013.25
msg.device_id = 'sensor_01'
self.publisher.publish(msg)
```

3.4 QoS 配置实战

知识点 3.4.1 : QoS 兼容性规则

Publisher \ Subscriber	RELIABLE	BEST_EFFORT
RELIABLE	✓	✓
BEST_EFFORT	X	✓

图 3-2: QoS Reliability 兼容性矩阵。Publisher 的可靠级别必须 >= Subscriber 的可靠级别。

知识点 3.4.2 : Python 中配置 QoS

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy, HistoryPolicy

# 方式 1 : 使用预定义配置
from rclpy.qos import qos_profile_sensor_data
self.publisher = self.create_publisher(
    Image, 'camera/image', qos_profile_sensor_data)
```

```
# 方式 2：自定义 QoS
custom_qos = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT, # 传感器数据可丢包
    durability=DurabilityPolicy.VOLATILE, # 不保存历史
    history=HistoryPolicy.KEEP_LAST, # 保留最后 N 条
    depth=5) # 队列深度 5

self.subscription = self.create_subscription(
    LaserScan, 'scan', self.callback, custom_qos)
```

3.5 多线程执行器与回调组

知识点 3.5.1：执行器类型

```
from rclpy.executors import SingleThreadedExecutor, MultiThreadedExecutor

# 单线程执行器（默认）：顺序执行所有回调
executor = SingleThreadedExecutor()
executor.add_node(node)
executor.spin()

# 多线程执行器：并行执行回调
executor = MultiThreadedExecutor(num_threads=4) # 4 个工作线程
executor.add_node(node)
executor.spin()
```

知识点 3.5.2：回调组

```
from rclpy.callback_groups import MutuallyExclusiveCallbackGroup, ReentrantCallbackGroup

# 互斥回调组：组内回调串行执行（默认行为）
group1 = MutuallyExclusiveCallbackGroup()

# 可重入回调组：组内回调可并行执行
group2 = ReentrantCallbackGroup()

# 将订阅者绑定到指定回调组
self.sub = self.create_subscription(
    Image, 'camera', self.callback, 10,
    callback_group=group2)
```

3.6 本章小结

本章核心知识点回顾：

29. 话题通信采用异步多对多的发布-订阅模型，发布者与订阅者完全解耦
30. `create_publisher(msg_type, topic, qos)` 创建发布者，`create_subscription(msg_type, topic, callback, qos)` 创建订阅者
31. ROS 2 提供丰富的标准消息类型 (`std_msgs`, `sensor_msgs`, `geometry_msgs` 等)
32. 自定义消息接口需要在独立的 CMake 包中定义 `.msg` 文件
33. QoS 兼容性取决于可靠性策略: RELIABLE 发布者可与 BEST_EFFORT 订阅者通信, 反之不行
34. 多线程执行器和回调组可实现并行消息处理, 提高吞吐量

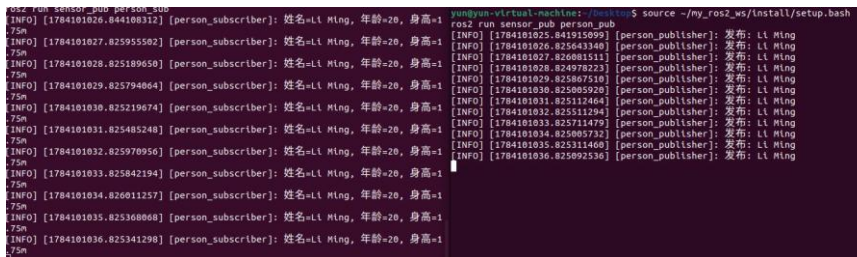
3.7 练习题

练习 3.1: 编写 Publisher 节点发布 Twist 消息控制机器人运动, 包含线速度和角速度设置。

练习 3.2: 编写 Subscriber 节点订阅 `/cmd_vel` 话题, 打印收到的速度指令。

练习 3.3: 创建一个自定义消息接口包, 定义包含姓名、年龄、身高字段的 `Person.msg`。

练习 3.4: 基于 `Person.msg` 编写发布者和订阅者节点。



```
o22:~$ ros2 run sensor_pub person_pub
[INFO] [1784101026.844100312] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101027.825955502] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101028.825189650] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101029.825794064] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101030.825219674] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101031.825485248] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101032.825970956] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101033.825842194] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101034.826011257] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101035.825308088] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
[INFO] [1784101036.825341298] [person_subscriber]: 姓名=Ll Ming, 年龄=20, 身高=1.75m
root@syn-virtual-machine:~/my_ws$ source ~/my_ws/install/setup.bash
ros2 run sensor_pub person_pub
[INFO] [1784101025.841915099] [person_publisher]: 发布: Ll Ming
[INFO] [1784101026.825643340] [person_publisher]: 发布: Ll Ming
[INFO] [1784101027.826081511] [person_publisher]: 发布: Ll Ming
[INFO] [1784101028.824978223] [person_publisher]: 发布: Ll Ming
[INFO] [1784101029.825867510] [person_publisher]: 发布: Ll Ming
[INFO] [1784101030.825085920] [person_publisher]: 发布: Ll Ming
[INFO] [1784101031.825112464] [person_publisher]: 发布: Ll Ming
[INFO] [1784101032.825511294] [person_publisher]: 发布: Ll Ming
[INFO] [1784101033.825711479] [person_publisher]: 发布: Ll Ming
[INFO] [1784101034.825085722] [person_publisher]: 发布: Ll Ming
[INFO] [1784101035.825311400] [person_publisher]: 发布: Ll Ming
[INFO] [1784101036.825092536] [person_publisher]: 发布: Ll Ming
```

创建发布者: `nano`

`~/my_ros2_ws/src/sensor_pub/sensor_pub/person_publisher.py` 输入: `import rclpy`

`from rclpy.node import Node` `from sensor_interfaces.msg import Person`

`class PersonPublisher(Node):` `def __init__(self):` `super().__init__('person_publisher')`

`self.publisher = self.create_publisher(Person, '/person_info', 10)` `self.timer =`

`self.create_timer(1.0, self.publish_person)`


```
def publish_person(self): msg = Person() msg.name = 'Li Ming' msg.age = 20
msg.height = 1.75 self.publisher.publish(msg) self.get_logger().info(f'发布:
{msg.name}')
```

```
def main(args=None): rclpy.init(args=args) rclpy.spin(PersonPublisher())
rclpy.shutdown() 创建订阅者: nano
~/my_ros2_ws/src/sensor_pub/sensor_pub/person_subscriber.py 输入: import rclpy
from rclpy.node import Node from sensor_interfaces.msg import Person

class PersonSubscriber(Node): def __init__(self): super().__init__('person_subscriber')
self.subscription = self.create_subscription( Person, '/person_info', self.callback, 10)

def callback(self, msg): self.get_logger().info( f'姓名={msg.name}, 年龄={msg.age}, 身高
={msg.height:.2f}m')
```

```
def main(args=None): rclpy.init(args=args) rclpy.spin(PersonSubscriber())
rclpy.shutdown() 在 sensor_pub/setup.py 中添加: 'person_pub =
sensor_pub.person_publisher:main', 'person_sub =
sensor_pub.person_subscriber:main', 检查、编译: cd ~/my_ros2_ws/src/sensor_pub
python3 setup.py --name cd ~/my_ros2_ws colcon build --packages-select
sensor_interfaces sensor_pub --symlink-install source install/setup.bash 终端 1:
source ~/my_ros2_ws/install/setup.bash ros2 run sensor_pub person_sub 终端 2:
source ~/my_ros2_ws/install/setup.bash ros2 run sensor_pub person_pub
```

练习 3.5: 测试 RELIABLE 和 BEST_EFFORT 兼容性, 验证不同 QoS 组合下的通信结果。

```
INFO [1784101235.892049375] [qos_publisher]: 发布 #19 yunyun-virtual-machine: /Desktop$ source ~/my_ros2_ws/install/setup.bash
INFO [1784101236.891960410] [qos_publisher]: 发布 #20 yunyun-virtual-machine: /Desktop$ ros2 topic echo /qos_reliable --qos-reliabili
INFO [1784101237.892005865] [qos_publisher]: 发布 #21 ty best_effort
INFO [1784101238.892009102] [qos_publisher]: 发布 #22 data: 'RELIABLE: 34'
INFO [1784101239.892134855] [qos_publisher]: 发布 #23 ---
INFO [1784101240.891972936] [qos_publisher]: 发布 #24 data: 'RELIABLE: 35'
INFO [1784101241.893798268] [qos_publisher]: 发布 #25 ---
INFO [1784101242.891984893] [qos_publisher]: 发布 #26 data: 'RELIABLE: 36'
INFO [1784101243.891965289] [qos_publisher]: 发布 #27 ---
INFO [1784101244.892194192] [qos_publisher]: 发布 #28 data: 'RELIABLE: 37'
INFO [1784101245.891949858] [qos_publisher]: 发布 #29 ---
INFO [1784101246.892083151] [qos_publisher]: 发布 #30 data: 'RELIABLE: 38'
INFO [1784101247.891960596] [qos_publisher]: 发布 #31 ---
INFO [1784101248.891957635] [qos_publisher]: 发布 #32 data: 'RELIABLE: 39'
INFO [1784101249.892203023] [qos_publisher]: 发布 #33 ---
INFO [1784101250.892488389] [qos_publisher]: 发布 #34 data: 'RELIABLE: 40'
INFO [1784101251.892430572] [qos_publisher]: 发布 #35 ---
INFO [1784101252.892967080] [qos_publisher]: 发布 #36 data: 'RELIABLE: 41'
INFO [1784101253.892618632] [qos_publisher]: 发布 #37 ---
INFO [1784101254.892003424] [qos_publisher]: 发布 #38
```

终端 1 ros2 run

topic_demo qos_pub 终端 2 ros2 topic echo /qos_reliable --qos-reliability best_effort

练习 3.6: 使用 MultiThreadedExecutor 同时运行 2 个 Publisher + 2 个 Subscriber, 验证并发性能。创建: nano

~/my_ros2_ws/src/topic_demo/topic_demo/multi_topic_demo.py 输入: import
threading import time

```
import rclpy from rclpy.callback_groups import ReentrantCallbackGroup from  
rclpy.executors import MultiThreadedExecutor, SingleThreadedExecutor from  
rclpy.node import Node from std_msgs.msg import String
```

```
class DemoPublisher(Node): def __init__(self, name, topic, label): super().__init__(name)  
self.publisher = self.create_publisher(String, topic, 10) self.label = label self.count = 0  
self.timer = self.create_timer(1.0, self.publish_message)
```

```
def publish_message(self): msg = String() msg.data = f'{self.label}-{self.count}'  
self.publisher.publish(msg) self.count += 1
```

```
class DemoSubscriber(Node): def __init__(self, name, topic, label):  
super().__init__(name) self.label = label self.group = ReentrantCallbackGroup()  
self.subscription = self.create_subscription( String, topic, self.callback, 10,  
callback_group=self.group)
```

```
def callback(self, msg): thread = threading.get_ident() start = time.monotonic()  
self.get_logger().info( f'{self.label} 开始 thread={thread}, msg={msg.data}')  
time.sleep(0.8) elapsed = time.monotonic() - start self.get_logger().info( f'{self.label}  
结束 thread={thread}, 用时={elapsed:.2f}s')
```

```
def run(use_multi): rclpy.init() nodes = [ DemoPublisher('publisher_a', '/multi_a', 'PUB-  
A'), DemoPublisher('publisher_b', '/multi_b', 'PUB-B'), DemoSubscriber('subscriber_a',  
'/multi_a', 'SUB-A'), DemoSubscriber('subscriber_b', '/multi_b', 'SUB-B'), ] executor =  
( MultiThreadedExecutor(num_threads=4) if use_multi else SingleThreadedExecutor() )  
for node in nodes: executor.add_node(node) try: executor.spin() except
```

KeyboardInterrupt: pass finally: executor.shutdown() for node in nodes:

node.destroy_node() rclpy.shutdown()

def main_single(args=None): run(False)

def main_multi(args=None): run(True) 在 topic_demo/setup.py 添加:

'executor_single = topic_demo.multi_topic_demo:main_single', 'executor_multi =

topic_demo.multi_topic_demo:main_multi', 检查并编译: cd

~/my_ros2_ws/src/topic_demo python3 setup.py --name cd ~/my_ros2_ws colcon

build --packages-select topic_demo --symlink-install source install/setup.bash 先运行

单线程: ros2 run topic_demo executor_single

```
INFO [1784101501.713870653] [subscriber_b]: SUB-B 开始 thread=131709329981888,
msg=PUB-B-1
INFO [1784101502.515810903] [subscriber_b]: SUB-B 结束 thread=131709329981888,
用时=0.80s
INFO [1784101502.516675109] [subscriber_a]: SUB-A 开始 thread=131709329981888,
msg=PUB-A-1
INFO [1784101503.318035697] [subscriber_a]: SUB-A 结束 thread=131709329981888,
用时=0.80s
INFO [1784101503.319010552] [subscriber_b]: SUB-B 开始 thread=131709329981888,
msg=PUB-B-2
INFO [1784101504.119914480] [subscriber_b]: SUB-B 结束 thread=131709329981888,
用时=0.80s
INFO [1784101504.120360902] [subscriber_a]: SUB-A 开始 thread=131709329981888,
msg=PUB-A-2
INFO [1784101504.920949267] [subscriber_a]: SUB-A 结束 thread=131709329981888,
用时=0.80s
INFO [1784101504.923168886] [subscriber_b]: SUB-B 开始 thread=131709329981888,
msg=PUB-B-3
INFO [1784101505.726011986] [subscriber_b]: SUB-B 结束 thread=131709329981888,
用时=0.80s
INFO [1784101505.727008356] [subscriber_a]: SUB-A 开始 thread=131709329981888,
msg=PUB-A-3
```

再运行多线程: ros2

run topic_demo executor_multi

```
[INFO] [1784101546.603255486] [subscriber_b]: SUB-B 开始 thread=139098002019904,
msg=PUB-B-4
[INFO] [1784101547.402921859] [subscriber_a]: SUB-A 结束 thread=139098010412608,
用时=0.80s
[INFO] [1784101547.404937902] [subscriber_b]: SUB-B 结束 thread=139098002019904,
用时=0.80s
[INFO] [1784101547.602297827] [subscriber_a]: SUB-A 开始 thread=139098010412608,
msg=PUB-A-5
[INFO] [1784101547.603523227] [subscriber_b]: SUB-B 开始 thread=139098002019904,
msg=PUB-B-5
[INFO] [1784101548.402917787] [subscriber_a]: SUB-A 结束 thread=139098010412608,
用时=0.80s
[INFO] [1784101548.404740018] [subscriber_b]: SUB-B 结束 thread=139098002019904,
用时=0.80s
[INFO] [1784101548.601712819] [subscriber_a]: SUB-A 开始 thread=139098010412608,
msg=PUB-A-6
[INFO] [1784101548.604789728] [subscriber_b]: SUB-B 开始 thread=139098002019904,
msg=PUB-B-6
[INFO] [1784101549.403087583] [subscriber_a]: SUB-A 结束 thread=139098010412608,
用时=0.80s
[INFO] [1784101549.406099039] [subscriber_b]: SUB-B 结束 thread=139098002019904,
用时=0.80s
```

另开终端验证四个节

点: `ros2 node list` 应包含: `/publisher_a /publisher_b /subscriber_a /subscriber_b`

实验 3:

第 3 章 实验指导书：话题通信编程与实践

****实验课时****: 2 课时 (90 分钟) ****实验平台****: Ubuntu 22.04 + ROS 2 Humble

实验目标

完成本实验后，学员应能够：

- 35. 编写 Publisher 和 Subscriber 节点
- 36. 理解标准消息类型的使用
- 37. 创建并使用自定义消息接口
- 38. 配置测试不同的 QoS 策略

练习 3.1：Publisher 与 Subscriber 通信（约 30 分钟）

目标

实现一个完整的发布-订阅通信链路，发布者周期发送消息，订阅者接收并打印。

步骤

步骤 1：创建 topic_demo 包

```
cd ~/my_ros2_ws/src
ros2 pkg create topic_demo --build-type ament_python \
  --dependencies rclpy std_msgs
```

步骤 2：编写 Publisher 节点

在 ~/my_ros2_ws/src/topic_demo/topic_demo/ 下创建 publisher.py：

```

#!/usr/bin/env python3
"""publisher_demo: 话题发布者 - 周期发布 GPS 坐标数据"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point # 三维点坐标

class GpsPublisher(Node):
    """模拟 GPS 传感器数据发布节点 - 周期发布 Point 消息"""

    def __init__(self):
        super().__init__('gps_publisher')
        # 创建发布者: 发布 Point 消息到 /gps_position 话题
        self.publisher = self.create_publisher(
            Point,          # 消息类型 (三维坐标)
            '/gps_position', # 话题名称
            10)             # 队列深度

        # 创建 1 Hz 定时器
        self.timer = self.create_timer(1.0, self.publish_position)
        self.x = 0.0      # X 坐标初始值

    def publish_position(self):
        """定时器回调 - 构造并发布 Point 消息"""
        msg = Point()
        msg.x = self.x      # 纬度方向
        msg.y = 2 * self.x + 1 # 经度方向 (线性变化)
        msg.z = 0.0        # 高度 (固定)
        self.publisher.publish(msg) # 发布消息
        self.get_logger().info(
            f'发布 GPS 位置: x={msg.x:.2f}, y={msg.y:.2f}')
        self.x += 1.0      # 更新坐标

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(GpsPublisher()) # ① 初始化 ② spin ③ shutdown
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 3: 编写 Subscriber 节点

在 topic_demo/topic_demo/ 下创建 subscriber.py:

```

#!/usr/bin/env python3
"""subscriber_demo: 话题订阅者 - 接收并处理 GPS 位置数据"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point

```

```

class GpsSubscriber(Node):
    """GPS 数据订阅节点 - 监听/gps_position 话题并处理坐标"""

    def __init__(self):
        super().__init__('gps_subscriber')
        # 创建订阅者：监听 Point 消息的 /gps_position 话题
        self.subscription = self.create_subscription(
            Point,          # 消息类型
            '/gps_position', # 话题名称
            self.position_callback, # 回调函数
            10)             # 队列深度

        # 防止未使用的变量被 lint 警告
        self.subscription

    def position_callback(self, msg):
        """收到 GPS 位置消息时的回调函数"""

        # 计算到原点的欧氏距离
        distance = (msg.x**2 + msg.y**2)**0.5
        self.get_logger().info(
            f'收到位置: ({msg.x:.2f}, {msg.y:.2f}), '
            f'距原点: {distance:.2f}m')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(GpsSubscriber())
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 4: 配置 setup.py 并编译

编辑 topic_demo/setup.py 的 entry_points:

```

entry_points={
    'console_scripts': [
        'gps_pub = topic_demo.publisher:main',
        'gps_sub = topic_demo.subscriber:main',
    ],
},
cd ~/my_ros2_ws
colcon build --packages-select topic_demo --symlink-install
source install/setup.bash

```

步骤 5: 运行测试

```

# 终端 1: 启动发布者
ros2 run topic_demo gps_pub

```

```
# 终端 2：启动订阅者
ros2 run topic_demo gps_sub
# 终端 3：监控
ros2 topic echo /gps_position #![alt text](images/image-35.png)
ros2 topic hz /gps_position      # 查看发布频率（期望 ~1Hz）
```

步骤 5：检查运行结果与图示一致

- 截图 1：publisher 发布日志

```
summary: 1 package finished [2.36s]
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 run topic_demo gps_pub
[INFO] [1784013222.065388570] [gps_publisher]: 发布 GPS 位置: x=0.00, y=1.00
[INFO] [1784013223.042061952] [gps_publisher]: 发布 GPS 位置: x=1.00, y=3.00
[INFO] [1784013224.042225858] [gps_publisher]: 发布 GPS 位置: x=2.00, y=5.00
[INFO] [1784013225.042059852] [gps_publisher]: 发布 GPS 位置: x=3.00, y=7.00
[INFO] [1784013226.042224166] [gps_publisher]: 发布 GPS 位置: x=4.00, y=9.00
[INFO] [1784013227.042172433] [gps_publisher]: 发布 GPS 位置: x=5.00, y=11.00
[INFO] [1784013228.042832964] [gps_publisher]: 发布 GPS 位置: x=6.00, y=13.00
```

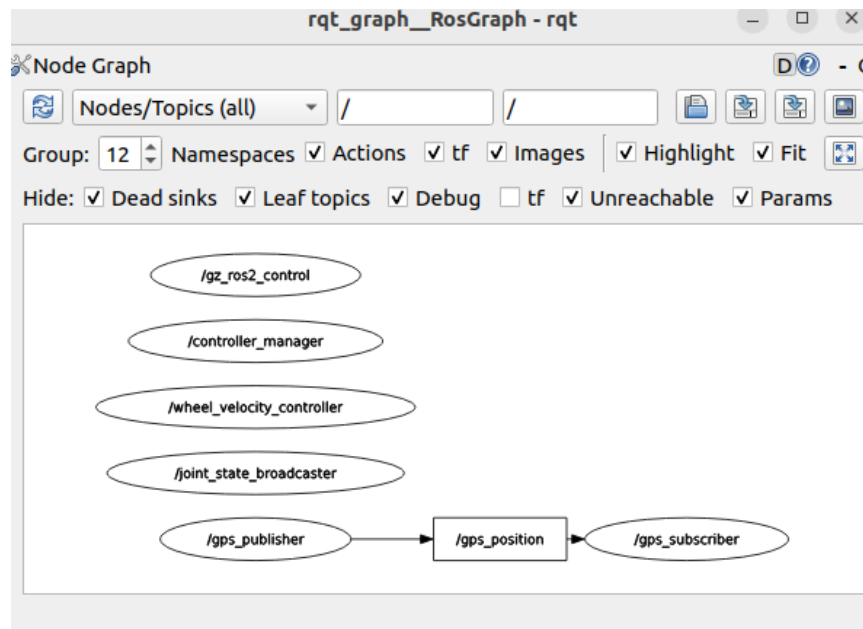
- 截图 2：subscriber 接收日志

```
yun@yun-virtual-machine:~/Desktop$ ros2 run topic_demo gps_sub
[INFO] [1784013261.067748259] [gps_subscriber]: 收到位置: (39.00, 79.00), 距原点
: 88.10m
[INFO] [1784013262.043517927] [gps_subscriber]: 收到位置: (40.00, 81.00), 距原点
: 90.34m
[INFO] [1784013263.042770880] [gps_subscriber]: 收到位置: (41.00, 83.00), 距原点
: 92.57m
[INFO] [1784013264.042259448] [gps_subscriber]: 收到位置: (42.00, 85.00), 距原点
: 94.81m
[INFO] [1784013265.042498721] [gps_subscriber]: 收到位置: (43.00, 87.00), 距原点
: 97.05m
[INFO] [1784013266.042468006] [gps_subscriber]: 收到位置: (44.00, 89.00), 距原点
: 99.28m
[INFO] [1784013267.042840265] [gps_subscriber]: 收到位置: (45.00, 91.00), 距原点
```

- 截图 3：ros2 topic hz /gps_position 输出（频率）

```
^Caverage rate: 1.001
   min: 0.999s max: 1.000s std dev: 0.00072s window: 3
average rate: 1.000
   min: 0.999s max: 1.001s std dev: 0.00089s window: 4
average rate: 1.000
   min: 0.999s max: 1.001s std dev: 0.00083s window: 6
average rate: 1.000
   min: 0.999s max: 1.001s std dev: 0.00090s window: 7
average rate: 1.000
   min: 0.999s max: 1.001s std dev: 0.00093s window: 8
average rate: 1.000
   min: 0.998s max: 1.001s std dev: 0.00110s window: 10
average rate: 1.000
   min: 0.998s max: 1.002s std dev: 0.00130s window: 11
average rate: 1.000
   min: 0.998s max: 1.002s std dev: 0.00128s window: 13
average rate: 1.000
   min: 0.998s max: 1.002s std dev: 0.00128s window: 14
```


- 截图 4: rqt_graph 通信拓扑 运行: `source /opt/ros/humble/setup.bash source ~/my_ros2_ws/install/setup.bash rqt_graph`



参考代码

完整参考代码位于 `'lab_code/ch03_lab/topic_demo/`

练习 3.2 : 自定义消息接口 (约 30 分钟)

目标

创建自定义消息接口包, 定义 `SensorData.msg` 并在 Python 中使用。

步骤

步骤 1: 创建 CMake 消息接口包 (先查有没有, 有的话就不用创建了)

```
cd ~/my_ros2_ws/src
ros2 pkg create sensor_interfaces --build-type ament_cmake \
  --dependencies rosidl_default_generators
```

步骤 2: 定义 `SensorData.msg`

创建 `sensor_interfaces/msg/SensorData.msg`：因为没有 `msg` 文件夹，所以用绝对路径创建

`mkdir -p ~/my_ros2_ws/src/sensor_interfaces/msg` `nano`

`~/my_ros2_ws/src/sensor_interfaces/msg/SensorData.msg`

```
# 传感器数据消息定义
float64 temperature      # 温度 (°C)
float64 humidity         # 湿度 (%)
float64 pressure         # 气压 (hPa)
string device_id        # 传感器 ID
```

步骤 3：配置 `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.8)
project(sensor_interfaces)

find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/SensorData.msg"
)
ament_package()
```

步骤 4：配置 `package.xml`

```
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

步骤 5：编译接口包

```
colcon build --packages-select sensor_interfaces
source install/setup.bash
ros2 interface show sensor_interfaces/msg/SensorData
```

步骤 6：创建使用自定义消息的 Python 包

```
cd ~/my_ros2_ws/src
ros2 pkg create sensor_pub --build-type ament_python \
  --dependencies rclpy sensor_interfaces
```

编写 `sensor_pub/sensor_pub/sensor_publisher.py`：

```
#!/usr/bin/env python3
"""sensor_publisher: 使用自定义 SensorData 消息的发布节点"""
import rclpy
from rclpy.node import Node
from sensor_interfaces.msg import SensorData # 导入自定义消息

class SensorPublisher(Node):
    def __init__(self):
        super().__init__('sensor_publisher')
        self.pub = self.create_publisher(
            SensorData, '/sensor_data', 10) # 使用自定义类型
```

```

        self.timer = self.create_timer(1.0, self.callback)

    def callback(self):
        msg = SensorData()
        msg.temperature = 25.5
        msg.humidity = 60.0
        msg.pressure = 1013.25
        msg.device_id = 'sensor_01'
        self.pub.publish(msg)
        self.get_logger().info(f'发布传感器数据: {msg.device_id}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(SensorPublisher())
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 7: 编译并运行 先在 setup.py 中修改 entry_points={ 'console_scripts':

['sensor_publisher = sensor_pub.sensor_publisher:main',], },

```

source /opt/ros/humble/setup.bash
cd ~/my_ros2_ws

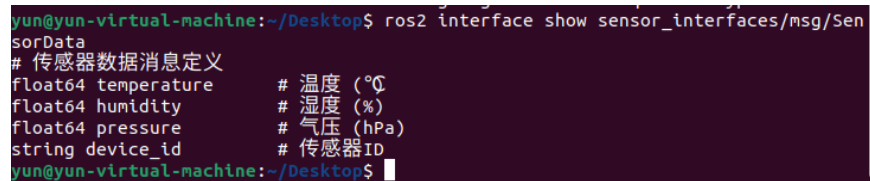
colcon build --packages-select sensor_interfaces sensor_pub --symlink-install
source ~/my_ros2_ws/install/setup.bash

ros2 interface show sensor_interfaces/msg/SensorData
ros2 pkg executables sensor_pub
ros2 run sensor_pub sensor_publisher # 终端 1
ros2 topic echo /sensor_data        # 终端 2

```

步骤 7: 检查运行结果与图示一致

- 截图 1: ros2 interface show sensor_interfaces/msg/SensorData 输出消息定义

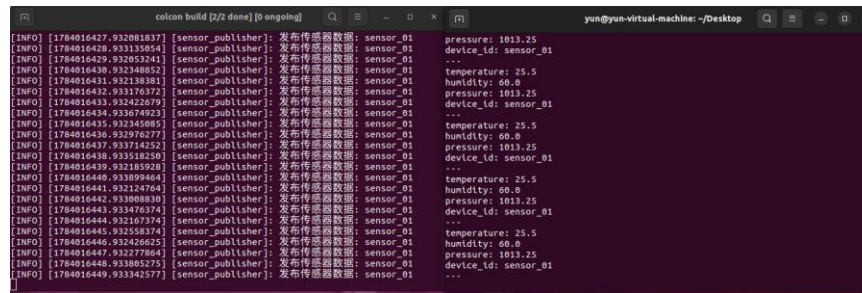


```

yun@yun-virtual-machine:~/Desktop$ ros2 interface show sensor_interfaces/msg/SensorData
# 传感器数据消息定义
float64 temperature      # 温度 (°C)
float64 humidity         # 湿度 (%)
float64 pressure         # 气压 (hPa)
string device_id         # 传感器ID
yun@yun-virtual-machine:~/Desktop$

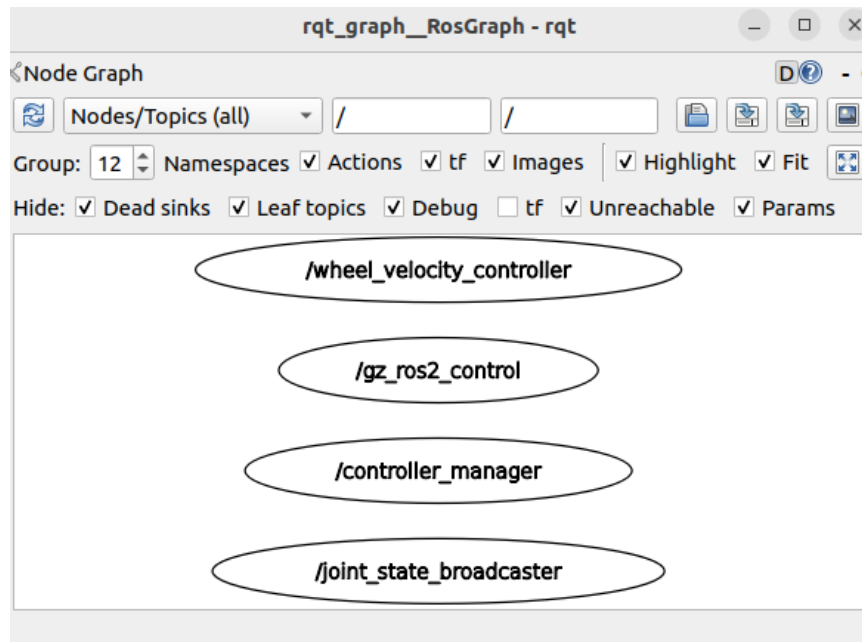
```

- 截图 2：发布者和 echo 终端输出



```
colcon build [2/2 done] [0 ongoing]
yun@yun-virtual-machine: ~/Desktop
[INFO] [1784016427.932081837] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016428.933135854] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016429.932853241] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016430.932348852] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016431.932138381] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016432.933176372] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016433.932422379] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016434.933674923] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016435.932458882] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016436.932976777] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016437.933714252] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016438.933518250] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016439.932185928] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016440.933899464] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016441.932124764] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016442.933808830] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016443.933476374] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016444.932167374] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016445.932558374] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016446.932426625] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016447.932277864] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016448.933803225] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016449.933342577] [sensor_publisher]: 发布传感器数据: sensor_01
pressure: 1013.25
device_id: sensor_01
...
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
...
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
...
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
...
```

- 截图 3：rqt_graph 中的自定义消息话题



参考代码

完整参考代码位于 `'lab_code/ch03_lab/sensor_interfaces/'` 和 `'lab_code/ch03_lab/sensor_pub/'`

练习 3.3 : QoS 策略对比实验（约 30 分钟）

目标

测试不同 QoS 配置下的通信行为差异。

步骤

步骤 1: 编写 QoS 测试发布者

在 topic_demo/topic_demo/ 下创建 qos_publisher.py:

```
#!/usr/bin/env python3
"""qos_publisher: 测试不同 QoS 配置的发布节点"""
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from std_msgs.msg import String

class QosPublisher(Node):
    def __init__(self):
        super().__init__('qos_publisher')
        # 可靠传输发布者
        reliable_qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,
            depth=10)
        self.reliable_pub = self.create_publisher(
            String, '/qos_reliable', reliable_qos)

        # 尽力传输发布者
        best_effort_qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            depth=10)
        self.best_effort_pub = self.create_publisher(
            String, '/qos_best_effort', best_effort_qos)

        # 定时器: 每秒向两个话题各发布一次
        self.timer = self.create_timer(1.0, self.publish)
        self.count = 0

    def publish(self):
        self.count += 1
        rel_msg = String()
        rel_msg.data = f'RELIABLE: {self.count}'
        self.reliable_pub.publish(rel_msg)

        be_msg = String()
        be_msg.data = f'BEST_EFFORT: {self.count}'
        self.best_effort_pub.publish(be_msg)

        self.get_logger().info(f'发布 #{self.count}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(QosPublisher())
    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

步骤 2：测试兼容性

实验	Publisher QoS	Subscriber QoS	预期结果
A	RELIABLE	RELIABLE	✓ 正常通信
B	RELIABLE	BEST_EFFORT	✓ 降级通信
C	BEST_EFFORT	BEST_EFFORT	✓ 正常通信
D	BEST_EFFORT	RELIABLE	✗ 无法通信

```
先在 setup.py 中添加
entry_points={
    'console_scripts': [
        'gps_pub = topic_demo.publisher:main',
        'gps_sub = topic_demo.subscriber:main',
        'qos_pub = topic_demo.qos_publisher:main',
    ],
},
然后编译
source /opt/ros/humble/setup.bash
cd ~/my_ros2_ws
colcon build --packages-select topic_demo --symlink-install
source ~/my_ros2_ws/install/setup.bash
# 终端 1：启动发布者
ros2 run topic_demo qos_pub

# 终端 2：分别测试不同 QoS 订阅
# 实验 A：订阅 /qos_reliable，默认 RELIABLE
ros2 topic echo /qos_reliable

# 实验 D：以 RELIABLE 订阅 BEST_EFFORT 话题 → 应失败
ros2 topic echo /qos_best_effort --qos-reliability reliable

# 实验 C：以 BEST_EFFORT 订阅 BEST_EFFORT 话题 → 应成功
ros2 topic echo /qos_best_effort --qos-reliability best_effort
```

步骤 3：检查运行结果与图示一致

- 截图 1: 实验 A (成功) : /qos_reliable 正常输出

```
yun@yun-virtual-machine: ~/my_ros2_ws/src/topic_demo
---
data: 'RELIABLE: 30'
---
data: 'RELIABLE: 31'
---
data: 'RELIABLE: 32'
---
data: 'RELIABLE: 33'
---
data: 'RELIABLE: 34'
---
data: 'RELIABLE: 35'
---
data: 'RELIABLE: 36'
---
data: 'RELIABLE: 37'
---
data: 'RELIABLE: 38'
---
data: 'RELIABLE: 39'
---
data: 'RELIABLE: 40'
---
```

- 截图 2: 实验 D (失败) : ros2 topic echo 无输出

```
yun@yun-virtual-machine:~/my_ros2_ws/src/topic_demo$ ros2 topic echo /qos_best_effort --qos-reliability reliable
[WARN] [1784017440.126081251] [_ros2cli_20721]: New publisher discovered on topic '/qos_best_effort', offering incompatible QoS. No messages will be received from it. Last incompatible policy: RELIABILITY
```

- 截图 3: 实验 C (成功) : /qos_best_effort 正常输出

```
yun@yun-virtual-machine:~/my_ros2_ws/src/topic_demo$ ros2 topic echo /qos_best_effort --qos-reliability best_effort
data: 'BEST_EFFORT: 127'
---
data: 'BEST_EFFORT: 128'
---
data: 'BEST_EFFORT: 129'
---
data: 'BEST_EFFORT: 130'
---
data: 'BEST_EFFORT: 131'
---
data: 'BEST_EFFORT: 132'
---
data: 'BEST_EFFORT: 133'
---
data: 'BEST_EFFORT: 134'
---
```

参考代码

完整参考代码位于 `lab\_code/ch03\_lab/topic\_demo/topic\_demo/qos\_publisher.py`

本章实验总结

练习	核心技能	关键 API
练习 1	Publisher + Subscriber 编程	`create_publisher()`, `create_subscription()`
练习 2	自定义消息接口定义与使用	`.msg` 定义, `rosidl_generate_interfaces()`
练习 3	QoS 策略配置与兼容性测试	`QoSProfile`, `ReliabilityPolicy`

思考题

39. 如果 Publisher 发布频率 100Hz，Subscriber 处理能力仅 50Hz，使用哪种 QoS 策略处理丢包？KEEP_LAST (depth) ， 队列满就丢掉就消息，保证最新数据，如果可以丢包，可以配合 BEST_EFFORT
40. KEEP_LAST(1) 和 KEEP_ALL 的使用场景分别是什么？KEEP_LAST(1)保存最新的一条消息，用于/cmd_vel 实时状态数据；KEEP_ALL 保存所有消息，用于日志
41. BEST_EFFORT + TRANSIENT_LOCAL 组合能否正常通信？为什么？可以，BEST_EFFORT 属于可靠性策略，TRANSIENT_LOCAL 是持久性策略，是不同的 QoS 维度，不冲突

练习 3.4 : Twist 控制 XBot-U 走方形轨迹（约 15 分钟）

目标

发布 geometry_msgs/Twist 消息到 /cmd_vel，驱动仿真中的 XBot-U 机器人在 Gazebo 中走正方形轨迹。

步骤

步骤 1：启动课程仿真

```
source ~/ros2_course_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
```

步骤 2：创建 twist_square.py

```
#!/usr/bin/env python3
"""twist_square: 发布 Twist 控制 XBot-U 走 1m×1m 正方形轨迹"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class SquareDriver(Node):
    def __init__(self):
        super().__init__('square_driver')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.get_logger().info('方形轨迹控制器就绪 - 5 秒后开始')
        time.sleep(5) # 等待仿真初始化
        self.drive_square()

    def move(self, linear, angular, duration):
        """发布指定速度并持续 duration 秒"""
        msg = Twist()
        msg.linear.x = linear
        msg.angular.z = angular
        end_time = time.time() + duration
        # 高频发布 (10Hz) 维持运动
        while time.time() < end_time and rclpy.ok():
            self.pub.publish(msg)
            time.sleep(0.1) # ~10Hz

    def stop(self):
        self.move(0.0, 0.0, 0.5)

    def drive_square(self):
        """走正方形：直行→转 90°→直行→转 90°→..."""
        self.get_logger().info('开始走正方形轨迹...')
        for i in range(4):
            # 直行 1 米 (速度 0.2m/s × 5 秒)
            self.get_logger().info(f'边 {i+1}: 直行')
            self.move(0.2, 0.0, 5.0)
            # 原地左转 90° (角速度 1.57rad/s × 1 秒 ≈ π/2)
            self.get_logger().info(f'边 {i+1}: 左转 90°')
            self.move(0.0, 1.57, 1.0)
        self.stop()
        self.get_logger().info('正方形轨迹完成!')
```

```
def main(args=None):
    rclpy.init(args=args)
    SquareDriver()
    rclpy.shutdown()

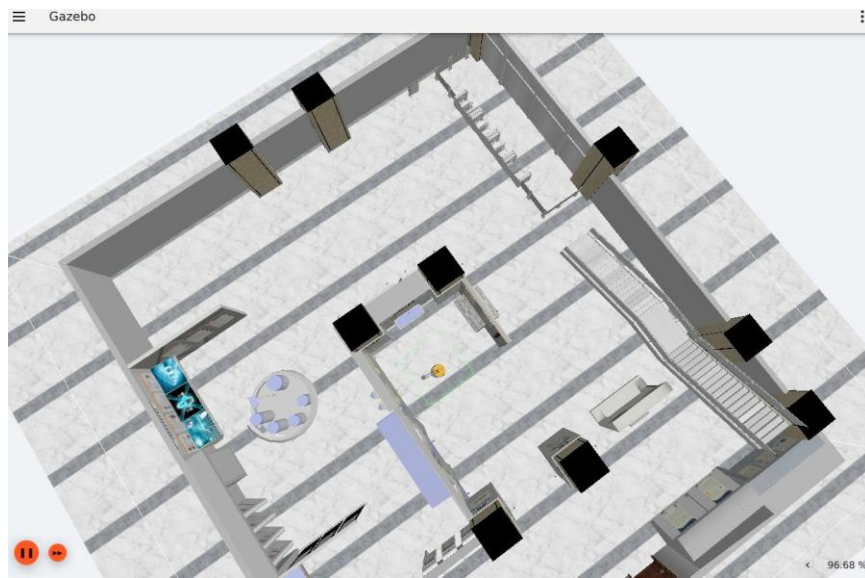
if __name__ == '__main__':
    main()
```

步骤 3：运行测试

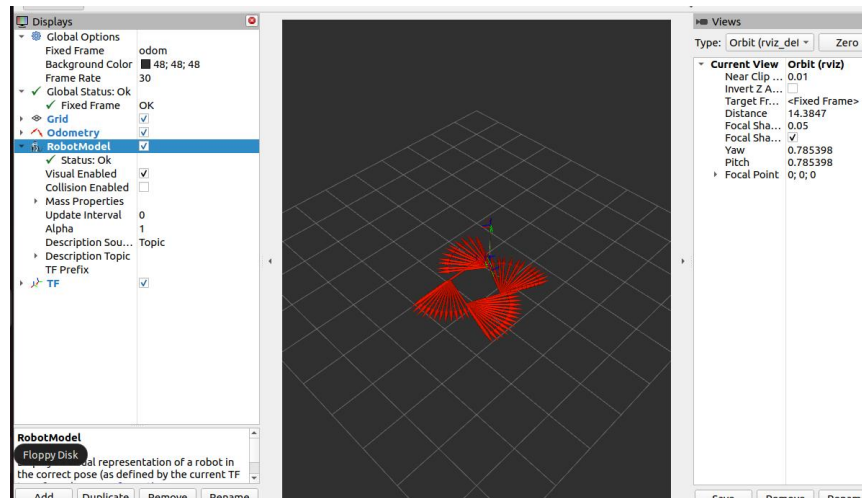
```
# 终端 1：仿真运行中
# 终端 2：启动方形轨迹控制器
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run topic_demo square_driver # 需添加 entry_points
```

✓ 验证：

- 截图 1：Gazebo 中 XBot-U 按正方形轨迹运动



- 截图 2: RViz 中 /odom 轨迹显示正方形图案



- 截图 3: 终端日志输出四边一角的执行进度

```
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 pkg executables topic_demo
ros2 run topic_demo twist_square
topic_demo gps_pub
topic_demo gps_sub
topic_demo qos_pub
topic_demo qos_pub = topic_demo.qos_publisher:mainsquare_driver
topic_demo twist_square
[INFO] [1784018978.173305118] [square_driver]: 方形轨迹控制器就绪 - 5秒后开始
[INFO] [1784018983.178315059] [square_driver]: 开始走正方形轨迹...
[INFO] [1784018983.179384772] [square_driver]: 边 1: 直行
[INFO] [1784018988.207689584] [square_driver]: 边 1: 左转90°
[INFO] [1784018989.211946558] [square_driver]: 边 2: 直行
[INFO] [1784018994.240955366] [square_driver]: 边 2: 左转90°
[INFO] [1784018995.248771277] [square_driver]: 边 3: 直行
[INFO] [1784019000.275947428] [square_driver]: 边 3: 左转90°
[INFO] [1784019001.282192267] [square_driver]: 边 4: 直行
[INFO] [1784019006.305958515] [square_driver]: 边 4: 左转90°
[INFO] [1784019007.814975631] [square_driver]: 正方形轨迹完成 !
yun@yun-virtual-machine:~/my_ros2_ws$
```

参考代码

完整参考代码位于 `lab\_code/ch03\_lab/topic\_demo/topic\_demo/square\_driver.py`

思考题

42. 如何精确控制机器人走 $2m \times 2m$ 的正方形？调整哪些参数？用直线运动+原地旋转 90° 循环四次。要调整线速度、角速度、运动时间、控制频率
43. 如果需要在正方形路径上添加圆角过渡，如何实现？在拐角处加入圆弧轨迹。在接近拐角时同时设置线速度和角速度，使机器人沿圆弧过渡。

实际运行证据

真实运行的话题发布、订阅、自定义消息接口和消息输出：

```
ch03: topic and message
Captured terminal session
$ ros2 topic echo /gps_position --once
x: 1.0
y: 7.0
z: 0.0
---
$ ros2 interface show sensor_interfaces/mag/SensorData
float64 temperature      # 温度 (°C)
float64 humidity         # 湿度 (%)
float64 pressure          # 气压 (hPa)
string device_id         # 传感器ID
$ ros2 topic echo /sensor_data --once
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
---
# gps_subscriber process output
[INFO] [1787891830.904849957]: [gps_subscriber] 收到位置: (1.00, 1.00), 距离点: 3.16m
[INFO] [1787891831.903940213]: [gps_subscriber] 收到位置: (2.00, 3.00), 距离点: 5.39m
[INFO] [1787891832.904679268]: [gps_subscriber] 收到位置: (3.00, 7.00), 距离点: 7.62m
[INFO] [1787891833.904505917]: [gps_subscriber] 收到位置: (4.00, 9.00), 距离点: 9.80m
[INFO] [1787891834.905524399]: [gps_subscriber] 收到位置: (5.00, 11.00), 距离点: 12.08m
[INFO] [1787891835.904906471]: [gps_subscriber] 收到位置: (6.00, 13.00), 距离点: 14.32m
---
# sensor_publisher process output
[INFO] [1787891830.916203208]: [sensor_publisher] 发布传感器数据: sensor_01
[INFO] [1787891831.915352391]: [sensor_publisher] 发布传感器数据: sensor_01
[INFO] [1787891832.915942346]: [sensor_publisher] 发布传感器数据: sensor_01
[INFO] [1787891833.917259152]: [sensor_publisher] 发布传感器数据: sensor_01
[INFO] [1787891834.916073476]: [sensor_publisher] 发布传感器数据: sensor_01
[INFO] [1787891835.916245919]: [sensor_publisher] 发布传感器数据: sensor_01
Source: realtime recording of the camera show demo
```

原始录制：[ch03_topics.cast](#)。完整证据索引见[实际运行证据](#)。

第 4 章：服务通信（Services）

***课程**： ROS2 Python 编程 **章节**： 第 4 章 **课时**： 2 课时 (90 分钟)*

4.1 请求-响应模型

知识点 4.1.1：服务通信架构

服务通信采用 **同步请求-响应** 模式，适用于需要立即获取结果的一次性操作：

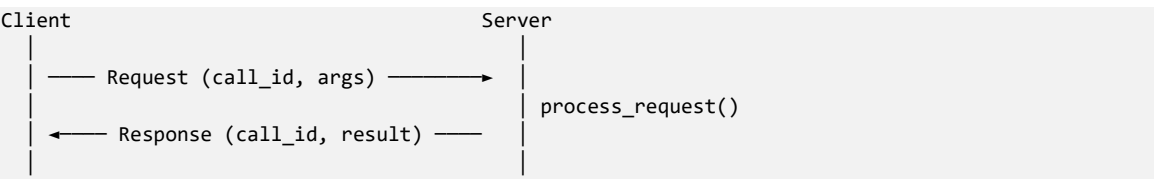


图 4-1：服务通信请求-响应时序图。一个 Server 可并发处理多个 Client 请求。

知识点 4.1.2：服务接口定义 (.srv 文件)

```
# AddTwoInts.srv - 两整数相加服务
int64 a                # 请求：第一个加数
int64 b                # 请求：第二个加数
---                    # 分隔线（上：请求，下：响应）
int64 sum               # 响应：相加结果
string message         # 响应：附加消息
```

分隔线 `---` 上方定义 *****Request***** 字段，下方定义 *****Response***** 字段。

知识点 4.1.3：Python Server API

```
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServer(Node):
    def __init__(self):
        super().__init__('add_two_ints_server')
        # 创建服务：服务名 "add_two_ints", 类型 AddTwoInts
        self.srv = self.create_service(
            AddTwoInts,                # 服务类型
            'add_two_ints',            # 服务名称
            self.handle_add_two_ints)  # 回调函数

    def handle_add_two_ints(self, request, response):
```

```

"""处理服务请求 - request 包含 a,b; response 包含 sum"""
response.sum = request.a + request.b
response.message = f'{request.a} + {request.b} = {response.sum}'
self.get_logger().info(
    f'收到请求: {request.a} + {request.b} = {response.sum}')
return response # 必须返回 response 对象

```

程序 4-1: Service Server 完整示例。

知识点 4.1.4 : Python Client API

```

from example_interfaces.srv import AddTwoInts

class AddTwoIntsClient(Node):
    def __init__(self):
        super().__init__('add_two_ints_client')
        # 创建客户端: 服务名 "add_two_ints", 类型 AddTwoInts
        self.client = self.create_client(
            AddTwoInts, 'add_two_ints')

    def send_request(self, a, b):
        """发送异步服务请求"""
        # 等待服务上线
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待服务上线...')

        # 构造请求
        request = AddTwoInts.Request()
        request.a = a
        request.b = b

        # 异步发送请求
        future = self.client.call_async(request)
        return future

```

程序 4-2: Service Client 异步调用示例。使用 `call_async` 发送异步请求，返回 `future` 对象。

4.2 同步调用 vs 异步调用

知识点 4.2.1 : 同步调用

```

# 同步调用 - 阻塞主线程直到收到响应
req = AddTwoInts.Request()
req.a = 10; req.b = 20
future = self.client.call_async(req) # 发送异步请求
rcipy.spin_until_future_complete( # 同步等待完成
    self, future, timeout_sec=5.0)

```

```
result = future.result()                # 获取结果
self.get_logger().info(f'Result: {result.sum}')
```

知识点 4.2.2 : 异步调用

```
# 异步调用 - 通过回调处理结果, 不阻塞
def send_request_async(self, a, b):
    req = AddTwoInts.Request()
    req.a = a; req.b = b
    future = self.client.call_async(req)
    future.add_done_callback(self.response_callback)

def response_callback(self, future):
    try:
        result = future.result()
        self.get_logger().info(f'Result: {result.sum}')
    except Exception as e:
        self.get_logger().error(f'调用失败: {e}')
```

4.3 服务超时与重试机制

知识点 4.3.1 : 超时处理

```
import time

def call_with_timeout(self, a, b, timeout=5.0):
    req = AddTwoInts.Request()
    req.a = a; req.b = b

    future = self.client.call_async(req)
    start_time = time.time()

    # 带超时的轮询
    while rclpy.ok():
        rclpy.spin_once(self, timeout_sec=0.1)
        if future.done():
            return future.result()
        if time.time() - start_time > timeout:
            self.get_logger().error('服务调用超时!')
            future.cancel()
            return None

    return None
```

程序 4-3: 服务调用超时处理模式。

知识点 4.3.2 : 重试机制

```
def call_with_retry(self, a, b, max_retries=3):
    """带重试的服务调用"""
    for attempt in range(max_retries):
```

```
if self.client.wait_for_service(timeout_sec=2.0):
    req = AddTwoInts.Request()
    req.a = a; req.b = b
    future = self.client.call_async(req)
    rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
    if future.result() is not None:
        return future.result()

self.get_logger().warn(
    f'重试 {attempt + 1}/{max_retries}...')

self.get_logger().error('所有重试均失败!')
return None
```

4.4 本章小结

- 44. 服务通信是同步请求-响应模式，Client 发送 Request，Server 返回 Response
- 45. .srv 文件包含 Request (--- 上方) 和 Response (--- 下方) 两部分
- 46. Server 使用 create_service(type, name, callback)，回调签名为 fn(request, response)
- 47. Client 使用 create_client(type, name)，通过 call_async() 发送请求
- 48. 同步调用使用 spin_until_future_complete()，异步调用使用 add_done_callback()
- 49. 生产环境必须处理超时和重试，提高系统鲁棒性

4.5 练习题

练习 4.1：基于 example_interfaces/srv/AddTwoInts 编写 Server 和 Client 节点，验证加法功能。

练习 4.2：设计一个 .srv 文件 WeatherQuery.srv（输入：城市名 string，输出：温度 float64 + 天气 string），实现查询服务。

练习 4.3：测试服务超时：Client 设置 1 秒超时，Server 处理时间设为 3 秒，观察超时行为。

练习 4.4：编写带重试机制的 Client，当 Server 不在线时自动重试 3 次。

新建 client_wait.py

将 call() 改为:

```
def call(self, a, b, max_retries=3):
    for attempt in range(1, max_retries + 1):
        self.get_logger().info(
            f'第 {attempt}/{max_retries} 次检查服务')

        if self.client.wait_for_service(timeout_sec=2.0):
            request = AddTwoInts.Request()
            request.a = a
            request.b = b

            future = self.client.call_async(request)
            rclpy.spin_until_future_complete(
                self, future, timeout_sec=5.0)

            if future.done():
                try:
                    response = future.result()
                    if response is not None:
                        return response.sum
                except Exception as error:
                    self.get_logger().error(
                        f'服务调用失败: {error}')
            else:
                future.cancel()
                self.get_logger().warning('等待服务响应超时')
        else:
            self.get_logger().warning('Server 尚未上线')

        if attempt < max_retries:
            self.get_logger().info('2 秒后重试')
            time.sleep(2.0)

    self.get_logger().error('3 次尝试均失败')
    return None
```

编译 :

```
cd ~/my_ros2_ws
colcon build --packages-select service_demo --symlink-install
source install/setup.bash
```

先不启动 server, 只启动 client_wait

```
ros2 run service_demo client_wait 5 10
```

```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client wait 5 10
[INFO] [1784195661.913296022] [add_two_ints_client]: 第 1/3 次尝试
[WARN] [1784195663.919019822] [add_two_ints_client]: Server 尚未上线
[INFO] [1784195663.919772880] [add_two_ints_client]: 2 秒后重试
[INFO] [1784195665.923028284] [add_two_ints_client]: 第 2/3 次尝试
[WARN] [1784195667.927213420] [add_two_ints_client]: Server 尚未上线
[INFO] [1784195667.928470839] [add_two_ints_client]: 2 秒后重试
[INFO] [1784195669.931173308] [add_two_ints_client]: 第 3/3 次尝试
[WARN] [1784195671.935947621] [add_two_ints_client]: Server 尚未上线
[ERROR] [1784195671.936881431] [add_two_ints_client]: 三次尝试全部失败
yun@yun-virtual-machine:~/my_ros2_ws$
```

再启动 server

```
source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server
```

```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server
[INFO] [1784195787.234047818] [add_two_ints_server]: AddTwoInts service started
[INFO] [1784195804.449677311] [add_two_ints_server]: Request: 5 + 10; waiting 3
seconds
[INFO] [1784195807.451382440] [add_two_ints_server]: Result: 15

yun@yun-virtual-machine:~/my_ros2_ws$ ros2 run service_demo client wait 5 10
[INFO] [1784195808.427418856] [add_two_ints_client]: 第 1/3 次尝试
[WARN] [1784195808.443065530] [add_two_ints_client]: Server 尚未上线
[INFO] [1784195808.443912047] [add_two_ints_client]: 2 秒后重试
[INFO] [1784195808.447413104] [add_two_ints_client]: 第 2/3 次尝试
[INFO] [1784195807.452460350] [add_two_ints_client]: 调用成功: 5 + 10 = 15
yun@yun-virtual-machine:~/my_ros2_ws$
```

练习 4.5: 测试多个

Client 同时向同一个 Server 发送请求时的并发处理行为。

练习 4.6: 使用 `ros2 service call` 命令行调用服务, 验证 Server 响应。

实验 4:

第 4 章 实验指导书：服务通信编程

****实验课时****: 2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

- 50. 编写 Service Server 和 Client 节点
- 51. 定义自定义 .srv 接口
- 52. 处理服务超时和重试

练习 4.1：AddTwoInts 服务（约 30 分钟）

步骤

1. 创建包

```
cd ~/my_ros2_ws/src
ros2 pkg create service_demo --build-type ament_python \
  --dependencies rclpy example_interfaces
```

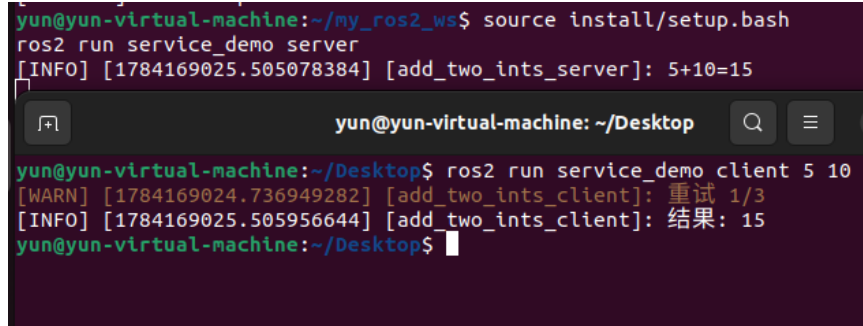
2. 编写 server.py 和 client.py（参考代码见 lab_code/ch04_lab/）

3. 修改 ~/my_ros2_ws/src/service_demo/setup.py

```
entry_points={
    'console_scripts': [
        'server = service_demo.server:main',
        'client = service_demo.client:main',
    ],
},
```

4.编译并运行

```
colcon build --packages-select service_demo
source install/setup.bash
ros2 run service_demo server    # 终端 1
ros2 run service_demo client 5 10 # 终端 2
```



```
yun@yun-virtual-machine:~/my_ros2_ws$ source install/setup.bash
ros2 run service_demo server
[INFO] [1784169025.505078384] [add_two_ints_server]: 5+10=15

yun@yun-virtual-machine:~/Desktop$ ros2 run service_demo client 5 10
[WARN] [1784169024.736949282] [add_two_ints_client]: 重试 1/3
[INFO] [1784169025.505956644] [add_two_ints_client]: 结果: 15
yun@yun-virtual-machine:~/Desktop$
```

参考代码: lab_code/ch04_lab/service_demo/

练习 4.2 : 自定义 .srv 接口 (约 30 分钟)

定义 WeatherQuery.srv (城市 string → 温度 float64 + 天气 string) , 创建 Server 模拟天气查询。

53. 创建两个包

```
cd ~/my_ros2_ws/src
ros2 pkg create weather_interfaces --build-type ament_cmake
ros2 pkg create weather_srv --build-type ament_python \
  --dependencies rclpy weather_interfaces
mkdir -p weather_interfaces/srv
```

创建 weather_interfaces/srv/WeatherQuery.srv: string city

float64 temperature string weather

54. 配置接口包 在 weather_interfaces/CMakeLists.txt 的 ament_package() 前添加:

```
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME} "srv/WeatherQuery.srv" )
ament_export_dependencies(rosidl_default_runtime)
```

在 weather_interfaces/package.xml 的 <export> 前添加:

```
<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

55. 编写 Server 创建 weather_srv/weather_srv/weather_server.py:

```
import rclpy
from rclpy.node import Node
from weather_interfaces.srv import WeatherQuery

class WeatherServer(Node):
    def __init__(self):
        super().__init__('weather_server')
        self.service = self.create_service(
            WeatherQuery, 'weather_query', self.query_weather)

    def query_weather(self, request, response):
        data = {
            'beijing': (26.5, 'Sunny'),
            'shanghai': (24.0, 'Cloudy'),
            'shenzhen': (30.0, 'Rainy'),
        }
        response.temperature, response.weather = data.get(
            request.city.strip().lower(), (0.0, 'Unknown city'))
        self.get_logger().info(
            f'{request.city}: {response.temperature}, {response.weather}')
        return response

def main(args=None):
    rclpy.init(args=args)
    node = WeatherServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

56. 注册可执行程序 修改 weather_srv/setup.py

```
entry_points={
    'console_scripts': [
        'weather_server = weather_srv.weather_server:main',
    ],
},
```

57. 编译和检查接口

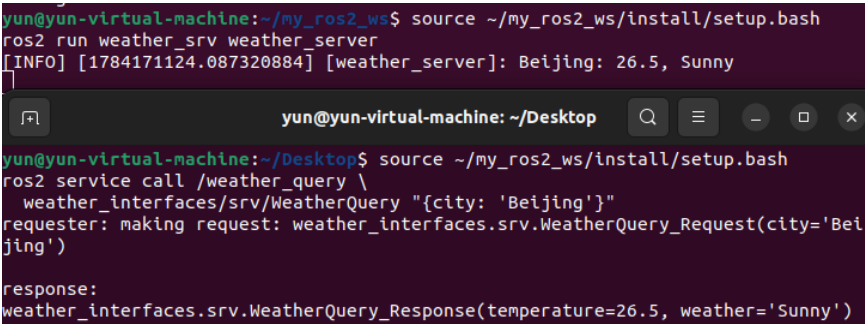
```
cd ~/my_ros2_ws
colcon build --packages-select weather_interfaces weather_srv --symlink-install
source install/setup.bash
ros2 interface show weather_interfaces/srv/WeatherQuery
```

58. 运行测试 终端 1:

```
source ~/my_ros2_ws/install/setup.bash
ros2 run weather_srv weather_server
```

终端 2:

```
source ~/my_ros2_ws/install/setup.bash
ros2 service call /weather_query \
  weather_interfaces/srv/WeatherQuery "{city: 'Beijing'}"
```



```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run weather_srv weather_server
[INFO] [1784171124.087320884] [weather_server]: Beijing: 26.5, Sunny

yun@yun-virtual-machine:~/Desktop$ source ~/my_ros2_ws/install/setup.bash
ros2 service call /weather_query \
  weather_interfaces/srv/WeatherQuery "{city: 'Beijing'}"
requester: making request: weather_interfaces.srv.WeatherQuery_Request(city='Beijing')
response:
weather_interfaces.srv.WeatherQuery_Response(temperature=26.5, weather='Sunny')
```

参考代码: lab_code/ch04_lab/weather_interfaces/ + lab_code/ch04_lab/weather_srv/

练习 4.3 : 超时与重试 (约 30 分钟)

59. Server 设置 3 秒处理延迟

60. Client 设置 1 秒超时 → 观察超时行为

61. 添加重试机制 (最多 3 次, 间隔 2 秒)

62. 修改 Server 编辑:

```
nano ~/my_ros2_ws/src/service_demo/service_demo/server.py
```

增加导入:

```
import time
将 handle_add() 改为:
def handle_add(self, request, response):
    self.get_logger().info(
        f'收到请求 {request.a} + {request.b}, 延迟 3 秒处理')
```

```

time.sleep(3.0)

response.sum = request.a + request.b
self.get_logger().info(f'处理完成, 结果为 {response.sum}')
return response

```

63. 修改 Client 编辑:

```
nano ~/my_ros2_ws/src/service_demo/service_demo/client.py
```

增加导入: import time 将 call() 方法完整替换为:

```

def call(self, a, b, timeout_sec=1.0, max_retries=3,
        retry_interval_sec=2.0):
    if not self.client.wait_for_service(timeout_sec=2.0):
        self.get_logger().error('服务不可用, 请先启动 Server')
        return None

    for attempt in range(1, max_retries + 1):
        request = AddTwoInts.Request()
        request.a = a
        request.b = b

        self.get_logger().info(
            f'第 {attempt}/{max_retries} 次调用服务')

        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(
            self, future, timeout_sec=timeout_sec)

        if future.done():
            response = future.result()
            if response is not None:
                return response.sum

        self.get_logger().warning(
            f'第 {attempt} 次请求超过 {timeout_sec} 秒, 调用超时')

        if attempt < max_retries:
            self.get_logger().info(
                f'{retry_interval_sec} 秒后重试')
            time.sleep(retry_interval_sec)

    return None

```

64. 编译

```

cd ~/my_ros2_ws
colcon build --packages-select service_demo --symlink-install &&
source install/setup.bash

```

终端 1:

```

source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server

```

终端 2:

```
source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client 5 10
```

```
yunyun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
time.sleep(3.0)
PY main() = _main_ :r{o(f'Result: {response.sum}')
ros2 run service_demo client 5 10
yunyun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server
[INFO] [1784173492.345921879] [add_two_ints_client]: Attempt 1/3: 5 + 10
[WARN] [1784173493.347374077] [add_two_ints_client]: Request timed out after 1.0
seconds(s)
[INFO] [1784173493.347778452] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173495.351224392] [add_two_ints_client]: Attempt 2/3: 5 + 10
[WARN] [1784173496.353226848] [add_two_ints_client]: Request timed out after 1.0
seconds(s)
[INFO] [1784173496.353380519] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173498.358217554] [add_two_ints_client]: Attempt 3/3: 5 + 10
[WARN] [1784173499.368037824] [add_two_ints_client]: Request timed out after 1.0
seconds(s)
[ERROR] [1784173499.368037824] [add_two_ints_client]: All attempts failed
yunyun-virtual-machine:~/my_ros2_ws$
[INFO] [1784173501.363688305] [add_two_ints_server]: Result: 15
```

思考题

65. 服务通信适合什么场景？不适合什么场景？服务通信适合低频、短时间、需要明确响应结果的请求操作，例如参数设置、状态查询、启动停止控制。不适合高频数据传输和长时间任务，后者应使用 Topic 或 Action。
66. 如何保证多个 Client 同时调用服务时的安全性？可以通过 Callback Group 控制并发方式、mutex 保证共享数据访问安全、Executor 管理线程调度，以及服务端状态检查避免重复调用

练习 4.4 : Service 控制机器人运动（约 15 分钟）

目标

创建自定义 SpeedControl.srv 服务，设置机器人速度（linear, angular）和运行时间（duration），Service 调用后驱动仿真中的 XBot-U 运动。

步骤

步骤 1: 定义 SpeedControl.srv

```
source /opt/ros/humble/setup.bash
cd ~/my_ros2_ws/src

ros2 pkg create speed_interfaces --build-type ament_cmake
ros2 pkg create speed_control --build-type ament_python \
  --dependencies rclpy geometry_msgs speed_interfaces

mkdir -p speed_interfaces/srv
# speed_interfaces/srv/SpeedControl.srv
float64 linear_x          # 线速度 (m/s)
float64 angular_z         # 角速度 (rad/s)
```



```
float64 duration      # 运行时长 (秒)
---
bool success          # 执行成功
string message        # 结果消息
```

步骤 2: 编写 Service Server

```
#!/usr/bin/env python3
"""speed_server: 速度控制服务 - 设置机器人速度和运行时长"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from speed_interfaces.srv import SpeedControl

class SpeedServer(Node):
    def __init__(self):
        super().__init__('speed_server')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.srv = self.create_service(
            SpeedControl, 'speed_control', self.handle_speed)

    def handle_speed(self, request, response):
        self.get_logger().info(
            f'速度指令: v={request.linear_x}m/s, '
            f'w={request.angular_z}rad/s, 时长={request.duration}s')
        # 发布速度指令
        msg = Twist()
        msg.linear.x = request.linear_x
        msg.angular.z = request.angular_z
        start = time.time()
        while time.time() - start < request.duration and rclpy.ok():
            self.pub.publish(msg)
            time.sleep(0.1)
        # 停止
        msg.linear.x = 0.0
        msg.angular.z = 0.0
        self.pub.publish(msg)
        response.success = True
        response.message = f'运动完成: {request.duration}s'
        return response

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(SpeedServer())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

步骤 3: 编写 Service Client

```
#!/usr/bin/env python3
"""speed_client: 调用速度控制服务"""
import sys
import rclpy
from rclpy.node import Node
from speed_interfaces.srv import SpeedControl

class SpeedClient(Node):
    def __init__(self):
        super().__init__('speed_client')
        self.client = self.create_client(SpeedControl, 'speed_control')

    def call(self, linear, angular, duration):
        req = SpeedControl.Request()
        req.linear_x = linear
        req.angular_z = angular
        req.duration = duration
        self.client.wait_for_service()
        future = self.client.call_async(req)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

def main(args=None):
    rclpy.init(args=args)
    client = SpeedClient()
    v = float(sys.argv[1]) if len(sys.argv) > 1 else 0.2
    w = float(sys.argv[2]) if len(sys.argv) > 2 else 0.0
    t = float(sys.argv[3]) if len(sys.argv) > 3 else 3.0
    result = client.call(v, w, t)
    client.get_logger().info(f'结果: {result.message}')
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

步骤 4: 完善配置

·在 speed_interfaces/CMakeLists.txt 的 ament_package() 前加入:

```
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
    "srv/SpeedControl.srv"
)
ament_export_dependencies(rosidl_default_runtime)
```

·在 speed_interfaces/package.xml 的 <export> 前加入:

```
<build_depend>rosidl_default_generators</build_depend>
```

```
<exec_depend>rosidl_default_runtime</exec_depend>
```

```
<member_of_group>rosidl_interface_packages</member_of_group> ·修改
```

speed_control/setup.py 的入口配置:

```
entry_points={
    'console_scripts': [
        'speed_server = speed_control.speed_server:main',
        'speed_client = speed_control.speed_client:main',
    ],
},
```

步骤 4: 运行测试

```
# 编译
cd ~/my_ros2_ws
colcon build \
    --packages-select speed_interfaces speed_control \
    --symlink-install &&
source install/setup.bash
# 终端 1: 启动仿真
source ~/my_ros2_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
# 终端 2: 启动速度服务
source ~/my_ros2_ws/install/setup.bash
ros2 run speed_control speed_server
# 终端 3: 调用服务 (前进 0.2m/s 持续 3 秒)
source ~/my_ros2_ws/install/setup.bash
ros2 run speed_control speed_client 0.2 0.0 3.0
![alt text](images/image-23.png)
# 旋转测试 (角速度 1.0rad/s 持续 2 秒)
ros2 run speed_control speed_client 0.0 1.0 2.0
![alt text](images/image-24.png)
```

✓ **验证:** Gazebo 中 XBot-U 按指定速度和时长运动, 结束后自动停止。

参考代码

完整参考代码位于 `'lab_code/ch04_lab/speed_control/'`

思考题

67. 服务处理函数中直接 `time.sleep()` 是否会影响其他服务请求? 如何改进? 服务回调中直接使用 `time.sleep()` 会阻塞当前 `Executor` 线程, 在单线程 `Executor` 下会影响其他服务请求和消息处理。改进方法包括使用 `Action` 处理长时间任务、采用异步线程、`Timer` 或 `MultiThreadedExecutor`, 并合理配置 `Callback Group`。
68. 如果同时在 `Server` 中处理话题发布和服务请求, 两者如何协调? `Server` 同时处理 `Topic` 发布和 `Service` 请求时, 需要通过 `Executor` 进行调度。简单场景可以使用单线程 `Executor` 保证安全; 需要并发时使用 `MultiThreadedExecutor`, 并结合 `Callback Group` 和 `mutex` 保护共享数据, 避免服务请求影响实时话题发布。

实际运行证据

真实运行的 AddTwoInts Server、Client 和服务调用结果：

```
ch04_service_demo
Captured terminal session
$ ros2 run service_demo server
$ ros2 run service_demo client 12 -4
[INFO] [1787891830.604317695] [add_two_ints_client]: 结果: 8
$ ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts '{a: 3, b: 4}'
waiting for service to become available...
requester: making request: example_interfaces.srv.AddTwoInts_Request(a=3, b=4)

response:
example_interfaces.srv.AddTwoInts_Response(sum=7)

# service server process output
[INFO] [1787891830.589789976] [add_two_ints_server]: 12+-4=8
[INFO] [1787891831.678804882] [add_two_ints_server]: 3+4=7
Server exit code: 0, output of the command: 0/0/0/0/0/0
```

原始录制：[ch04_service.cast](#)。完整证据索引见[实际运行证据](#)。

第 5 章：动作通信（Actions）

***课程**： ROS2 Python 编程 **章节**： 第 5 章 **课时**： 2 课时 (90 分钟)*

5.1 动作通信架构

知识点 5.1.1：动作 vs 话题 vs 服务

特性	话题 (Topic)	服务 (Service)	动作 (Action)
通信模式	异步多对多	同步一对一	异步+长期任务
时长	持续	短时	长时间 (秒~分钟)
反馈	无	无	有 (进度反馈)
取消	不支持	不支持	支持取消和抢占

知识点 5.1.2：动作通信时序图

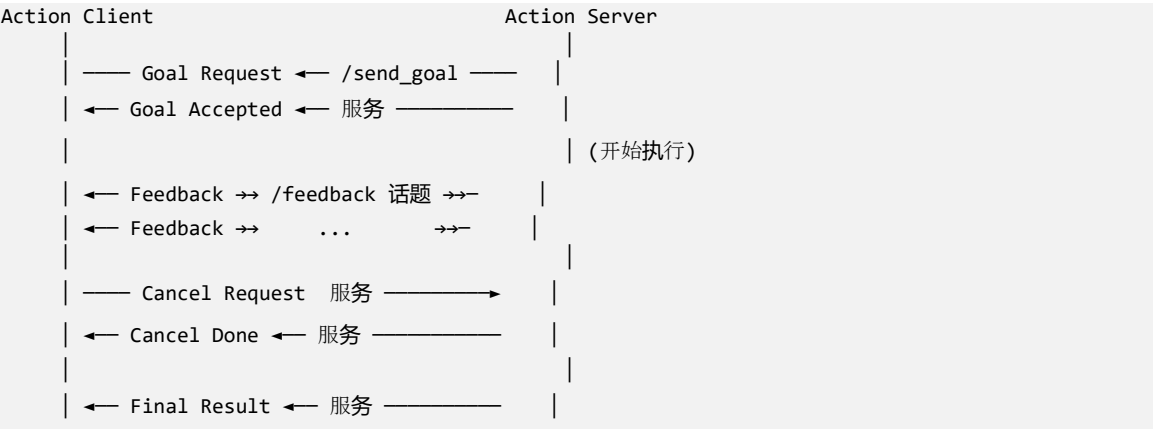


图 5-1：动作通信时序图。动作结合了话题（Feedback 流）和服务（Goal/Result/Cancel）的优势。

5.2 动作接口定义

知识点 5.2.1 : .action 文件结构

```
# DoDishes.action - 洗碗任务定义

# Goal (目标)
uint32 total_dishes      # 需要洗的碗数量
---
# Result (结果)
uint32 cleaned_dishes    # 实际洗干净的碗数量
bool success             # 是否成功
---
# Feedback (反馈)
float32 progress         # 进度百分比 (0.0 ~ 1.0)
uint32 current_dish      # 当前正在洗的碗编号
```

三条 `---` 分隔线分别定义: ***Goal (目标) ***、***Result (结果) ***、***Feedback (反馈) ***。

知识点 5.2.2 : Action Server Python API

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer, GoalResponse, CancelResponse
from action_demo_interfaces.action import DoDishes

class DoDishesServer(Node):
    def __init__(self):
        super().__init__('do_dishes_server')
        self.action_server = ActionServer(
            self,                                # 所属节点
            DoDishes,                             # 动作类型
            'do_dishes',                         # 动作名称
            execute_callback=self.execute,        # 执行回调
            goal_callback=self.goal,             # 目标接收回调
            cancel_callback=self.cancel)         # 取消回调

    def goal(self, goal_request):
        """接收目标时调用 - 决定是否接受"""
        self.get_logger().info(
            f'收到洗碗目标: {goal_request.total_dishes} 个盘子')
        return GoalResponse.ACCEPT            # 或 REJECT

    def cancel(self, goal_handle):
        """收到取消请求时调用"""
        self.get_logger().info('收到取消请求')
        return CancelResponse.ACCEPT
```

```

async def execute(self, goal_handle):
    """执行目标 - 异步函数, 支持 await"""
    total = goal_handle.request.total_dishes
    feedback_msg = DoDishes.Feedback()

    for i in range(1, total + 1):
        # 检查是否被取消
        if goal_handle.is_cancel_requested:
            goal_handle.canceled()
            result = DoDishes.Result()
            result.cleaned_dishes = i - 1
            result.success = False
            return result

        # 模拟任务执行 (1 秒/盘子)
        await asyncio.sleep(1.0)
        feedback_msg.progress = i / total
        feedback_msg.current_dish = i
        goal_handle.publish_feedback(feedback_msg)
        self.get_logger().info(
            f'进度: {feedback_msg.progress:.0%} ({i}/{total})')

    # 返回最终结果
    goal_handle.succeed()
    result = DoDishes.Result()
    result.cleaned_dishes = total
    result.success = True
    return result

```

程序 5-1: Action Server 完整示例。核心方法: execute (异步执行)、goal (接受/拒绝)、cancel (取消)。

知识点 5.2.3 : Action Client Python API

```

from rclpy.action import ActionClient

class DoDishesClient(Node):
    def __init__(self):
        super().__init__('do_dishes_client')
        self.client = ActionClient(self, DoDishes, 'do_dishes')

    def send_goal(self, total_dishes):
        """发送目标到 Action Server"""

        # 等待 Server 上线
        self.client.wait_for_server()
        goal_msg = DoDishes.Goal()
        goal_msg.total_dishes = total_dishes

        # 发送目标, 注册反馈和结果回调
        send_goal_future = self.client.send_goal_async(
            goal_msg,
            feedback_callback=self.feedback_callback)
        send_goal_future.add_done_callback(self.goal_response_callback)

```

```

def goal_response_callback(self, future):
    """目标被接受/拒绝的回调"""
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().info('目标被拒绝')
        return

    self.get_logger().info('目标已接受, 开始执行...')
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def feedback_callback(self, feedback_msg):
    """接收进度反馈的回调"""

    progress = feedback_msg.feedback.progress
    current = feedback_msg.feedback.current_dish

    self.get_logger().info(f'反馈: {progress:.0%} (盘子 {current})')

def result_callback(self, future):
    """获取最终结果的回调"""
    result = future.result().result
    self.get_logger().info(
        f'完成: {result.cleaned_dishes} 个, 成功: {result.success}')

def cancel_goal(self):
    """取消当前目标"""

    self.get_logger().info('请求取消...')
    self.goal_handle.cancel_goal_async()

```

程序 5-2: Action Client 完整示例。通过 `send_goal_async` 发送目标, `feedback_callback` 接收进度。

5.3 取消与抢占机制

知识点 5.3.1 : 取消流程

```

Client 发送取消 → Server.execute() 中检测 is_cancel_requested
├─ True → goal_handle.canceled() → return
└─ False → 继续执行

```

知识点 5.3.2 : 抢占机制

```

# 发送新目标时, 可选择抢占旧目标
# Server 端通过 GoalResponse.REJECT 拒绝旧目标
def goal(self, goal_request):
    # 如果有正在执行的目标, 拒绝新目标
    if self.current_goal is not None:
        self.get_logger().info('已有目标在执行, 拒绝新目标')

```



```
return GoalResponse.REJECT  
return GoalResponse.ACCEPT
```

5.4 本章小结

- 69. 动作通信 = 服务 (Goal/Result/Cancel) + 话题 (Feedback) , 适用于长时间任务
- 70. .action 文件三部分: Goal / Result / Feedback
- 71. ActionServer 核心方法: execute (异步) 、 goal、 cancel
- 72. ActionClient 通过 send_goal_async 发送目标, feedback_callback 接收进度
- 73. 支持取消 (cancel_goal_async) 和抢占 (GoalResponse.REJECT)

5.5 练习题

练习 5.1: 基于 action_demo_interfaces/action/DoDishes 编写完整的 Action Server 和 Client。

练习 5.2: 设计自定义 Action Tracking.action (Goal: 目标 ID, speed; Result: 成功与否; Feedback: 当前位置、距离) 。

练习 5.3: 在 Client 中实现中途取消: 发送目标后等待 3 秒, 然后发送取消请求。

练习 5.4: 测试抢占行为: Server 执行中收到新目标时的处理策略。

练习 5.5: 使用 ros2 action 命令行工具查看动作列表、发送目标和获取结果。

实验 5:

第 5 章 实验指导书：动作通信编程

****实验课时****: 2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

74. 编写 Action Server 和 Client

75. 处理进度反馈

76. 实现取消和抢占

练习 3.1：DoDishes 动作（约 30 分钟）

使用现有的 action_demo_interfaces 包，编写洗碗任务 Action Server/Client。

创建接口包

```
ros2 pkg create --build-type ament_cmake \  
  --license Apache-2.0 action_demo_interfaces \  
mkdir -p action_demo_interfaces/action \  
nano action_demo_interfaces/action/DoDishes.action
```

粘贴： uint32 total_dishes

uint32 cleaned_dishes bool success

float32 progress uint32 current_dish

配置 Cmake

```
cmake_minimum_required(VERSION 3.8)
project(action_demo_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/DoDishes.action"
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

配置 action_demo_interfaces/package.xml

```
<?xml version="1.0"?> <package format="3">
<name>action_demo_interfaces</name> <version>0.1.0</version>
<description>DoDishes Action interface</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_cmake</buildtool_depend>
<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>

<member_of_group>rosidl_interface_packages</member_of_group>

<export> <build_type>ament_cmake</build_type> </export> </package>
```

编译

```
colcon build --packages-select action_demo_interfaces
source install/setup.bash
```

创建 action_demo

参考代码：lab_code/ch05_lab/action_demo/ 修改 dishes_server.py

```
import asyncio 改为 import time
async def execute(self, goal_handle): 改为 def execute(self, goal_handle):
await asyncio.sleep(1.0) 改为 time.sleep(1.0)
```

编译

```
cd ~/my_ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install \
  --packages-select action_demo_interfaces action_demo
source ~/my_ros2_ws/install/setup.bash
ros2 pkg prefix action_demo_interfaces
ros2 interface show action_demo_interfaces/action/DoDishes
```

运行

终端一

```
source /opt/ros/humble/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run action_demo server
```

终端二

```
source /opt/ros/humble/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run action_demo client
```

```
yun@yun-virtual-machine:~$ ros2 run action_demo client
[INFO] [1784258265.434948968] [do_dishes_client]: 目标已接受
[INFO] [1784258266.424985427] [do_dishes_client]: 反馈: 20% (盘子 1)
[INFO] [1784258267.429480243] [do_dishes_client]: 反馈: 40% (盘子 2)
[INFO] [1784258268.432489243] [do_dishes_client]: 反馈: 60% (盘子 3)
[INFO] [1784258269.436372423] [do_dishes_client]: 反馈: 80% (盘子 4)
[INFO] [1784258270.438397418] [do_dishes_client]: 反馈: 100% (盘子 5)
[INFO] [1784258270.440880313] [do_dishes_client]: 完成: 5个, 成功:true

yun@yun-virtual-machine:~/my_ros2_ws$ ros2 run action_demo server
[INFO] [1784258265.428877453] [do_dishes_server]: 收到目标: 5 个盘子
[INFO] [1784258266.425689945] [do_dishes_server]: 进度: 20% (1/5)
[INFO] [1784258267.429154929] [do_dishes_server]: 进度: 40% (2/5)
[INFO] [1784258268.431949678] [do_dishes_server]: 进度: 60% (3/5)
[INFO] [1784258269.434617866] [do_dishes_server]: 进度: 80% (4/5)
[INFO] [1784258270.437780888] [do_dishes_server]: 进度: 100% (5/5)
```

练习 3.2 : 自定义 Tracking 动作 (约 30 分钟)

定义 Tracking.action (Goal: 目标坐标 Point; Result: 成功标志 bool; Feedback: 当前位置 float64 + 距离 float64), 模拟 XBot-U 的路径跟踪。

创建接口包 :

```
ros2 pkg create --build-type ament_cmake \
  --license Apache-2.0 tracking_interfaces
mkdir -p tracking_interfaces/action
# 创建 Tracking.action
nano tracking_interfaces/action/Tracking.action
```

geometry_msgs/Point target

bool success

float64 current_position float64 distance

编写 CMakeLists.txt

```
nano tracking_interfaces/CMakeLists.txt

cmake_minimum_required(VERSION 3.8)
project(tracking_interfaces)

find_package(ament_cmake REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Tracking.action"
  DEPENDENCIES geometry_msgs
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

77. 编写 package.xml

```
nano tracking_interfaces/package.xml

<?xml version="1.0"?> <package format="3"> <name>tracking_interfaces</name>
<version>0.1.0</version> <description>Tracking Action interface</description>
<maintainer email="student@example.com">Student</maintainer>
<license>Apache-2.0</license>

<buildtool_depend>ament_cmake</buildtool_depend>
<build_depend>rosidl_default_generators</build_depend>

<depend>geometry_msgs</depend>

<exec_depend>rosidl_default_runtime</exec_depend>

<member_of_group>rosidl_interface_packages</member_of_group>
```

```
<export> <build_type>ament_cmake</build_type> </export> </package>
```

编译

```
source /opt/ros/humble/setup.bash
colcon build --packages-select tracking_interfaces
source install/setup.bash
```

创建 Tracking Python 包

```
cd ~/my_ros2_ws/src
#创建包：
ros2 pkg create tracking_server --build-type ament_python --license Apache-2.0 --dependencies
rclpy tracking_interfaces

nano ~/ros2_course_ws/src/tracking_server/tracking_server/server.py

"""Tracking Action Server : 模拟机器人沿直线移动。"""

import math
import time

import rclpy
from rclpy.action import ActionServer, GoalResponse
from rclpy.node import Node

from tracking_interfaces.action import Tracking

class TrackingServer(Node):

    def __init__(self):
        super().__init__('tracking_server')

        self.action_server = ActionServer(
            self,
            Tracking,
            'tracking',
            execute_callback=self.execute_callback,
            goal_callback=self.goal_callback,
        )

        self.get_logger().info('Tracking Action Server 已启动')

    def goal_callback(self, goal_request):
        target = goal_request.target

        self.get_logger().info(
            f'收到目标: x={target.x:.2f}, '
            f'y={target.y:.2f}, z={target.z:.2f}'
        )

        values = (target.x, target.y, target.z)
```

```

        if not all(math.isfinite(value) for value in values):
            self.get_logger().warning('目标坐标无效, 拒绝目标')
            return GoalResponse.REJECT

        return GoalResponse.ACCEPT

def execute_callback(self, goal_handle):
    target = goal_handle.request.target

    total_distance = math.sqrt(
        target.x ** 2 +
        target.y ** 2 +
        target.z ** 2
    )

    current_position = 0.0
    speed = 0.25
    period = 0.1

    feedback = Tracking.Feedback()

    while current_position < total_distance:
        time.sleep(period)

        current_position += speed * period
        current_position = min(current_position, total_distance)

        feedback.current_position = current_position
        feedback.distance = total_distance - current_position

        goal_handle.publish_feedback(feedback)

        self.get_logger().info(
            f'已移动: {feedback.current_position:.2f} m, '
            f'剩余: {feedback.distance:.2f} m'
        )

    goal_handle.succeed()

    result = Tracking.Result()
    result.success = True

    self.get_logger().info('已经到达目标')
    return result

def main(args=None):
    rclpy.init(args=args)

    node = TrackingServer()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()

```

```

        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

nano ~/my_ros2_ws/src/tracking_server/tracking_server/client.py
"""Tracking Action Client : 发送目标坐标并接收反馈。"""

import sys

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node

from tracking_interfaces.action import Tracking

class TrackingClient(Node):

    def __init__(self):
        super().__init__('tracking_client')

        self.action_client = ActionClient(
            self,
            Tracking,
            'tracking'
        )

    def send_goal(self, x, y, z):
        self.get_logger().info('等待 Tracking Server...')

        if not self.action_client.wait_for_server(timeout_sec=10.0):
            self.get_logger().error('没有找到 Tracking Server')
            rclpy.shutdown()
            return

        goal = Tracking.Goal()
        goal.target.x = x
        goal.target.y = y
        goal.target.z = z

        self.get_logger().info(
            f'发送目标: ({x:.2f}, {y:.2f}, {z:.2f})'
        )

        future = self.action_client.send_goal_async(
            goal,
            feedback_callback=self.feedback_callback
        )

        future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()

        if not goal_handle.accepted:

```



```

        self.get_logger().warning('目标被 Server 拒绝')
        rclpy.shutdown()
        return

    self.get_logger().info('目标已接受')

    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def feedback_callback(self, feedback_message):
    feedback = feedback_message.feedback

    self.get_logger().info(
        f'反馈: 已移动={feedback.current_position:.2f} m, '
        f'剩余={feedback.distance:.2f} m'
    )

def result_callback(self, future):
    response = future.result()
    result = response.result

    self.get_logger().info(
        f'任务结束: 状态码={response.status}, '
        f'成功={result.success}'
    )

    rclpy.shutdown()

def main(args=None):
    if len(sys.argv) not in (3, 4):
        print(
            '用法: ros2 run tracking_server client '
            '<x> <y> [z]'
        )
        print(
            '示例: ros2 run tracking_server client '
            '2.0 1.0'
        )
        return

    try:
        x = float(sys.argv[1])
        y = float(sys.argv[2])
        z = float(sys.argv[3]) if len(sys.argv) == 4 else 0.0
    except ValueError:
        print('错误: x、y、z 必须是数字')
        return

    rclpy.init(args=args)

    node = TrackingClient()
    node.send_goal(x, y, z)

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:

```

```

        pass
    finally:
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

修改 setup.py

```

nano ~/my_ros2_ws/src/tracking_server/setup.py
from setuptools import find_packages, setup

package_name = 'tracking_server'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name]
        ),
        (
            'share/' + package_name,
            ['package.xml']
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='Tracking Action Server and Client',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'server = tracking_server.server:main',
            'client = tracking_server.client:main',
        ],
    },
)

```

修改 tracking_server/package.xml

```

nano ~/ros2_course_ws/src/tracking_server/package.xml

```

```

<?xml version="1.0"?> <package format="3"> <name>tracking_server</name>
<version>0.1.0</version> <description>Tracking Action Server and
Client</description> <maintainer

```

```
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>rcpp</exec_depend>

<exec_depend>tracking_interfaces</exec_depend>

<export> <build_type>ament_python</build_type> </export> </package>
```

编译

```
source /opt/ros/humble/setup.bash
source install/setup.bash
colcon build --symlink-install \
  --packages-select tracking_interfaces tracking_server
source install/setup.bash
ros2 pkg executables tracking_server
```

应显示: tracking_server client tracking_server server

运行

终端 1:

```
source /opt/ros/humble/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run tracking_server server
```

终端 2:

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run tracking_server client 2.0 1.0
```

参考代码: lab_code/ch05_lab/tracking_interfaces/ + lab_code/ch05_lab/tracking_server/

```
INFO [1784206658.563785551] [tracking_server]: 已移动: 1.72 m, 剩余: 0.51 m
INFO [1784206658.665913001] [tracking_server]: 已移动: 1.75 m, 剩余: 0.49 m
INFO [1784206658.766906427] [tracking_server]: 已移动: 1.77 m, 剩余: 0.46 m
INFO [1784206658.868038059] [tracking_server]: 已移动: 1.80 m, 剩余: 0.44 m
INFO [1784206658.978022968] [tracking_server]: 已移动: 1.82 m, 剩余: 0.41 m
INFO [1784206659.071636570] [tracking_server]: 已移动: 1.85 m, 剩余: 0.39 m
INFO [1784206659.173364983] [tracking_server]: 已移动: 1.87 m, 剩余: 0.36 m
INFO [1784206659.275321796] [tracking_server]: 已移动: 1.90 m, 剩余: 0.34 m
INFO [1784206659.376839935] [tracking_server]: 已移动: 1.92 m, 剩余: 0.31 m
INFO [1784206659.478025746] [tracking_server]: 已移动: 1.95 m, 剩余: 0.29 m
INFO [1784206659.579103542] [tracking_server]: 已移动: 1.97 m, 剩余: 0.26 m
INFO [1784206659.680731738] [tracking_server]: 已移动: 2.00 m, 剩余: 0.24 m
INFO [1784206659.782784302] [tracking_server]: 已移动: 2.02 m, 剩余: 0.21 m
INFO [1784206659.884739153] [tracking_server]: 已移动: 2.05 m, 剩余: 0.19 m
INFO [1784206659.986560510] [tracking_server]: 已移动: 2.07 m, 剩余: 0.16 m
INFO [1784206660.087983033] [tracking_server]: 已移动: 2.10 m, 剩余: 0.14 m
INFO [1784206660.190035872] [tracking_server]: 已移动: 2.12 m, 剩余: 0.11 m
INFO [1784206660.291157788] [tracking_server]: 已移动: 2.15 m, 剩余: 0.09 m
INFO [1784206660.392066313] [tracking_server]: 已移动: 2.17 m, 剩余: 0.06 m
INFO [1784206660.494411018] [tracking_server]: 已移动: 2.20 m, 剩余: 0.04 m
INFO [1784206660.598535242] [tracking_server]: 已移动: 2.22 m, 剩余: 0.01 m
INFO [1784206660.700647415] [tracking_server]: 已移动: 2.24 m, 剩余: 0.00 m
INFO [1784206660.701576749] [tracking_server]: 已经达到目标
[INFO] [1784206632.599750136] [tracking_client]: 反馈: 已移动+2.00 m, 剩余=0.24 m
[INFO] [1784206632.701509659] [tracking_client]: 反馈: 已移动+2.02 m, 剩余=0.21 m
[INFO] [1784206632.803483510] [tracking_client]: 反馈: 已移动+2.05 m, 剩余=0.19 m
[INFO] [1784206632.905585138] [tracking_client]: 反馈: 已移动+2.07 m, 剩余=0.16 m
[INFO] [1784206633.006555098] [tracking_client]: 反馈: 已移动+2.10 m, 剩余=0.14 m
[INFO] [1784206633.108451846] [tracking_client]: 反馈: 已移动+2.12 m, 剩余=0.11 m
[INFO] [1784206633.210297773] [tracking_client]: 反馈: 已移动+2.15 m, 剩余=0.09 m
[INFO] [1784206633.314768509] [tracking_client]: 反馈: 已移动+2.17 m, 剩余=0.06 m
[INFO] [1784206633.415545277] [tracking_client]: 反馈: 已移动+2.20 m, 剩余=0.04 m
[INFO] [1784206633.517167842] [tracking_client]: 反馈: 已移动+2.22 m, 剩余=0.01 m
[INFO] [1784206633.61796239] [tracking_client]: 反馈: 已移动+2.24 m, 剩余=0.00 m
[INFO] [1784206633.623509310] [tracking_client]: 任务结束: 状态码=4, 成功=True
```

改成用 gazebo 仿真

保留 Tracking.action 和 Client, 改 Server

```
cd ~/my_ros2_ws
colcon build --symlink-install \
  --packages-select tracking_interfaces tracking_server
```

```
nano ~/my_ros2_ws/src/tracking_server/package.xml
```

```
<exec_depend>geometry_msgs</exec_depend>
```

```
<exec_depend>nav_msgs</exec_depend>
```

```
cd ~/my_ros2_ws
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
colcon build --symlink-install \
  --packages-select tracking_interfaces tracking_server
source ~/my_ros2_ws/install/setup.bash
```

#终端 1 启动仿真：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

LIBGL_DRIR3_DISABLE=1 QT_QPA_PLATFORM=xcb \
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=true \
  use_rviz:=true \
  enable_fake_laser:=false \
  world:=~/ros2_ws/install/robot_sim_demo_ros2/share/robot_sim_demo_ros2/worlds/ground_test.sdf
```

#终端 2 启动 Gazebo Tracking Server：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run tracking_server server_gazebo
```

#终端 3 发送目标：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run tracking_server client 2.0 1.0
```

练习 3.3：取消与抢占机制（约 30 分钟）

78. Client 发送目标后 5 秒自动取消

79. Server 检测 is_cancel_requested 并执行清理

80. 抢占：发送新目标时拒绝或覆盖旧目标

创建新包

```
source /opt/ros/humble/setup.bash
cd ~/ros2_ws/src

ros2 pkg create --build-type ament_cmake \
  --license Apache-2.0 dishes_action_interfaces

ros2 pkg create --build-type ament_python \
  --license Apache-2.0 dishes_action_lab \
  --dependencies rclpy dishes_action_interfaces

mkdir -p dishes_action_interfaces/action
```

修改接口

```
nano ~/ros2_ws/src/dishes_action_interfaces/action/DoDishes.action
```

```
uint32 total_dishes
```

```
uint32 cleaned_dishes bool success
```

```
float32 progress uint32 current_dish
```

接口 CMakeList

```
nano ~/ros2_ws/src/dishes_action_interfaces/CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.8) project(dishes_action_interfaces)
```

```
find_package(ament_cmake REQUIRED) find_package(rosidl_default_generators
REQUIRED)
```

```
rosidl_generate_interfaces(${PROJECT_NAME} "action/DoDishes.action" )
```

```
ament_export_dependencies(rosidl_default_runtime) ament_package()
```

接口 package.xml

```
nano ~/ros2_ws/src/dishes_action_interfaces/package.xml
```

```

<?xml version="1.0"?> <package format="3">
<name>dishes_action_interfaces</name> <version>0.1.0</version>
<description>DoDishes Action interface</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_cmake</buildtool_depend>

<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>

<member_of_group>rosidl_interface_packages</member_of_group>

<export> <build_type>ament_cmake</build_type> </export> </package>

```

创建 python 文件

```

nano ~/ros2_ws/src/dishes_action_lab/dishes_action_lab/dishes_server.py
"""DoDishes Action Server with cancellation and goal rejection."""

import threading
import time

import rclpy
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from dishes_action_interfaces.action import DoDishes

class DishesActionServer(Node):
    def __init__(self):
        super().__init__('dishes_action_server')

        self._goal_lock = threading.Lock()
        self._goal_active = False
        self._callback_group = ReentrantCallbackGroup()

        self._action_server = ActionServer(
            self,
            DoDishes,
            'do_dishes_lab',
            execute_callback=self.execute_callback,

```

```

        goal_callback=self.goal_callback,
        cancel_callback=self.cancel_callback,
        callback_group=self._callback_group,
    )

    self.get_logger().info(
        'DoDishes Action Server 已启动, 等待目标'
    )

def goal_callback(self, goal_request):
    total = goal_request.total_dishes

    self.get_logger().info(
        f'收到目标: 清洗 {total} 个盘子'
    )

    if total == 0:
        self.get_logger().warning(
            '盘子数量不能为 0, 拒绝目标'
        )
        return GoalResponse.REJECT

    with self._goal_lock:
        if self._goal_active:
            self.get_logger().warning(
                '已有任务正在执行, 拒绝新目标'
            )
            return GoalResponse.REJECT

        self._goal_active = True

    self.get_logger().info('接受目标')
    return GoalResponse.ACCEPT

def cancel_callback(self, goal_handle):
    self.get_logger().info('收到 Client 的取消请求')
    return CancelResponse.ACCEPT

def execute_callback(self, goal_handle):
    total = goal_handle.request.total_dishes
    cleaned = 0
    feedback = DoDishes.Feedback()

    self.get_logger().info(
        f'开始执行洗碗任务, 共 {total} 个盘子'
    )

    try:
        for current_dish in range(1, total + 1):
            # 模拟每个盘子清洗 1 秒, 每 0.1 秒检查一次取消。
            for _ in range(10):
                time.sleep(0.1)

                if goal_handle.is_cancel_requested:
                    break

```

```

        if goal_handle.is_cancel_requested:
            break

        cleaned = current_dish

        feedback.progress = current_dish / total
        feedback.current_dish = current_dish
        goal_handle.publish_feedback(feedback)

        self.get_logger().info(
            f'进度: {feedback.progress:.0%} '
            f'({current_dish}/{total})'
        )

    if goal_handle.is_cancel_requested:
        self.get_logger().warning(
            f'执行清理: 停止洗碗, 已完成 {cleaned} 个'
        )

        goal_handle.canceled()

        result = DoDishes.Result()
        result.cleaned_dishes = cleaned
        result.success = False

        self.get_logger().info('任务已取消')
        return result

    goal_handle.succeed()

    result = DoDishes.Result()
    result.cleaned_dishes = cleaned
    result.success = True

    self.get_logger().info(
        f'任务成功, 共清洗 {cleaned} 个盘子'
    )
    return result

finally:
    with self._goal_lock:
        self._goal_active = False

    self.get_logger().info(
        '当前任务结束, 可以接收下一个目标'
    )

def main(args=None):
    rclpy.init(args=args)

    node = DishesActionServer()
    executor = MultiThreadedExecutor(num_threads=2)
    executor.add_node(node)

    try:
        executor.spin()

```



```

except KeyboardInterrupt:
    pass
finally:
    executor.shutdown()
    node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

nano ~/ros2_ws/src/dishes_action_lab/dishes_action_lab/dishes_client.py
"""DoDishes Client that automatically cancels after five seconds."""

import sys

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node

from dishes_action_interfaces.action import DoDishes

class DishesActionClient(Node):

    def __init__(self):
        super().__init__('dishes_action_client')

        self._action_client = ActionClient(
            self,
            DoDishes,
            'do_dishes_lab',
        )

        self._goal_handle = None
        self._cancel_timer = None
        self._finished = False

    def send_goal(self, total_dishes):
        self.get_logger().info(
            '等待 DoDishes Action Server...'
        )

        if not self._action_client.wait_for_server(
            timeout_sec=10.0
        ):
            self.get_logger().error(
                '10 秒内没有找到 Action Server'
            )
            return False

        goal = DoDishes.Goal()
        goal.total_dishes = total_dishes

        self.get_logger().info(
            f'发送目标: 清洗 {total_dishes} 个盘子'
        )

```

```

        send_future = self._action_client.send_goal_async(
            goal,
            feedback_callback=self.feedback_callback,
        )
        send_future.add_done_callback(
            self.goal_response_callback
        )

        return True

def goal_response_callback(self, future):
    self._goal_handle = future.result()

    if not self._goal_handle.accepted:
        self.get_logger().warning(
            '目标被 Server 拒绝'
        )
        self._finished = True
        rclpy.shutdown()
        return

    self.get_logger().info(
        '目标已接受, 5 秒后自动发送取消请求'
    )

    result_future = self._goal_handle.get_result_async()
    result_future.add_done_callback(
        self.result_callback
    )

    self._cancel_timer = self.create_timer(
        5.0,
        self.cancel_goal,
    )

def cancel_goal(self):
    # create_timer 默认周期执行, 所以第一次触发后立即停止。
    self._cancel_timer.cancel()

    if self._finished:
        return

    if self._goal_handle is None:
        return

    self.get_logger().warning(
        '已经执行 5 秒, 发送取消请求'
    )

    cancel_future = self._goal_handle.cancel_goal_async()
    cancel_future.add_done_callback(
        self.cancel_response_callback
    )

def cancel_response_callback(self, future):
    response = future.result()

```

```

        if response.goals_canceling:
            self.get_logger().info(
                'Server 已接受取消请求'
            )
        else:
            self.get_logger().warning(
                '取消请求未被接受, 任务可能已经结束'
            )

    def feedback_callback(self, feedback_message):
        feedback = feedback_message.feedback

        self.get_logger().info(
            f'收到反馈: 进度={feedback.progress:.0%}, '
            f'当前盘子={feedback.current_dish}'
        )

    def result_callback(self, future):
        self._finished = True

        if self._cancel_timer is not None:
            self._cancel_timer.cancel()

        response = future.result()
        result = response.result

        self.get_logger().info(
            f'最终状态码={response.status}, '
            f'已清洗={result.cleaned_dishes}, '
            f'成功={result.success}'
        )

    rclpy.shutdown()

def main(args=None):
    total_dishes = 10

    if len(sys.argv) > 1:
        try:
            total_dishes = int(sys.argv[1])
        except ValueError:
            print('错误: 盘子数量必须是整数')
            return

    if total_dishes <= 0:
        print('错误: 盘子数量必须大于 0')
        return

    rclpy.init(args=args)
    node = DishesActionClient()

    try:
        if node.send_goal(total_dishes):
            rclpy.spin(node)
    except KeyboardInterrupt:

```

```

        pass
    finally:
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

完善配置

```
nano ~/ros2_ws/src/dishes_action_lab/package.xml
```

```

<?xml version="1.0"?> <package format="3"> <name>dishes_action_lab</name>
<version>0.1.0</version> <description>DoDishes cancellation and preemption
exercise</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

```

```
<buildtool_depend>ament_python</buildtool_depend>
```

```
<exec_depend>rclpy</exec_depend>
```

```
<exec_depend>dishes_action_interfaces</exec_depend>
```

```
<export> <build_type>ament_python</build_type> </export> </package>
```

setup.cfg 可以复制之前的

```

nano ~/ros2_ws/src/dishes_action_lab/setup.py
from setuptools import find_packages, setup

package_name = 'dishes_action_lab'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        (
            'share/' + package_name,
            ['package.xml'],
        ),
    ],
)

```

```

    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='DoDishes cancellation and preemption exercise',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'server = dishes_action_lab.dishes_server:main',
            'client = dishes_action_lab.dishes_client:main',
        ],
    },
)

```

编译

打开新终端

```

cd ~/ros2_ws
source /opt/ros/humble/setup.bash

colcon build --symlink-install \
    --packages-select dishes_action_interfaces dishes_action_lab

source ~/ros2_ws/install/setup.bash

```

自动取消

终端 1:

```

bash
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab server

```

终端 2:

```

bash
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab client 10

```

拒绝新目标

```

#现在终端 2 发一个较长的目标
ros2 run dishes_action_lab client 20
#5 秒内终端 3 发送第 2 个目标
source /opt/ros/humble/setup.bash

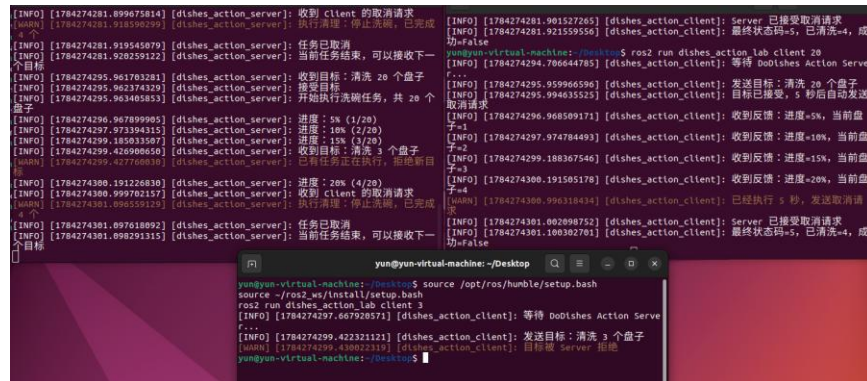
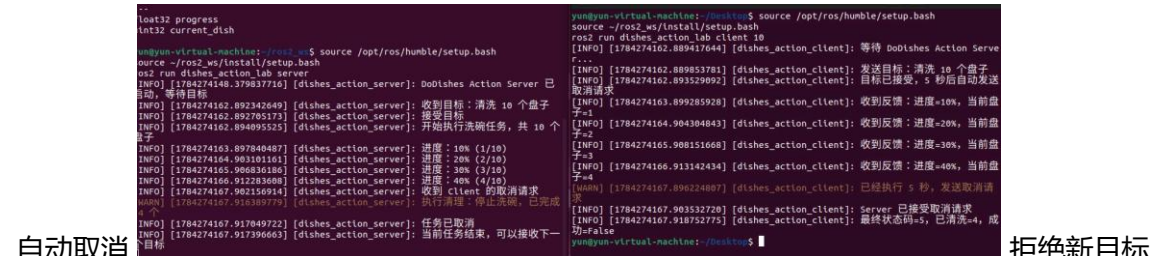
```

```
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab client 3
```

思考题

81. 动作通信为什么需要 `async/await`?

82. Feedback 话题的 QoS 应如何配置?



练习 4 : Action 导航任务 — 发送目标坐标驱动 XBot-U (约 15 分钟)

目标

基于 `action_demo_interfaces/action/MoveBase`, 编写 Action Client 发送导航目标 (x, y, yaw), Server 通过发布 `/cmd_vel` 控制 XBot-U 向目标运动, 实时反馈当前距离。

步骤

创建包

```
source /opt/ros/humble/setup.bash
cd ~/ros2_ws/src

ros2 pkg create --build-type ament_cmake \
```

```
--license Apache-2.0 pose_nav_interfaces

ros2 pkg create --build-type ament_python \
  --license Apache-2.0 pose_nav_action \
  --dependencies rclpy action_msgs geometry_msgs nav_msgs \
  pose_nav_interfaces

mkdir -p pose_nav_interfaces/action
```

创建并配置接口

```
nano ~/ros2_ws/src/pose_nav_interfaces/action/MoveBase.action
```

geometry_msgs/PoseStamped target_pose

bool success

geometry_msgs/Pose feedback_pose float64 distance_remaining

```
nano ~/ros2_ws/src/pose_nav_interfaces/CMakeLists.txt

cmake_minimum_required(VERSION 3.8)
project(pose_nav_interfaces)

find_package(ament_cmake REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/MoveBase.action"
  DEPENDENCIES geometry_msgs
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()

nano ~/ros2_ws/src/pose_nav_interfaces/package.xml
```

```
<?xml version="1.0"?> <package format="3"> <name>pose_nav_interfaces</name>
<version>0.1.0</version> <description>Pose navigation Action
interface</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_cmake</buildtool_depend>
```

```

<build_depend>roslint_default_generators</build_depend>

<exec_depend>roslint_default_runtime</exec_depend>

<depend>geometry_msgs</depend>

<member_of_group>roslint_interface_packages</member_of_group>

<export> <build_type>ament_cmake</build_type> </export> </package>

```

编写导航 Action Server

```

nano ~/ros2_ws/src/pose_nav_action/pose_nav_action/nav_server.py
#!/usr/bin/env python3
"""nav_action_server: 接收导航目标, 发布 /cmd_vel 驱动机器人到目标"""
import math
import threading
import time

import rclpy
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from pose_nav_interfaces.action import MoveBase

def clamp(value, minimum, maximum):
    return max(minimum, min(value, maximum))

def normalize_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

def quaternion_norm(quaternion):
    return math.sqrt(
        quaternion.x ** 2
        + quaternion.y ** 2
        + quaternion.z ** 2
        + quaternion.w ** 2
    )

def yaw_from_quaternion(quaternion):
    norm = quaternion_norm(quaternion)

    if norm < 1e-9:
        return 0.0

```



```

x = quaternion.x / norm
y = quaternion.y / norm
z = quaternion.z / norm
w = quaternion.w / norm

sin_yaw = 2.0 * (w * z + x * y)
cos_yaw = 1.0 - 2.0 * (y * y + z * z)

return math.atan2(sin_yaw, cos_yaw)

```

```

class PoseNavServer(Node):

```

```

    def __init__(self):
        super().__init__('pose_nav_server')

        self._state_lock = threading.Lock()
        self._goal_lock = threading.Lock()
        self._shutdown_event = threading.Event()

        self._odom_received = False
        self._last_odom_time = None
        self._current_x = 0.0
        self._current_y = 0.0
        self._current_z = 0.0
        self._current_yaw = 0.0
        self._goal_active = False

        self._control_period = 0.1
        self._position_tolerance = 0.10
        self._reacquire_distance = 0.16
        self._yaw_tolerance = math.radians(3.0)

        self._linear_gain = 0.8
        self._angular_gain = 1.8
        self._max_linear_speed = 0.25
        self._max_angular_speed = 1.0
        self._drive_heading_limit = math.radians(20.0)

        self._odom_timeout = 1.0
        self._goal_timeout = 120.0

        self._callback_group = ReentrantCallbackGroup()

        self._cmd_pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            10,
        )

        self._odom_sub = self.create_subscription(
            Odometry,
            '/odom',
            self.odom_callback,
            10,
            callback_group=self._callback_group,
        )

        self._action_server = ActionServer(

```

```

        self,
        MoveBase,
        'move_base_lab',
        execute_callback=self.execute_callback,
        goal_callback=self.goal_callback,
        cancel_callback=self.cancel_callback,
        callback_group=self._callback_group,
    )

    self.get_logger().info(
        'Pose Navigation Server 已启动, 等待 /odom'
    )

def odom_callback(self, message):
    pose = message.pose.pose

    with self._state_lock:
        self._current_x = pose.position.x
        self._current_y = pose.position.y
        self._current_z = pose.position.z
        self._current_yaw = yaw_from_quaternion(
            pose.orientation
        )
        self._last_odom_time = time.monotonic()
        first_message = not self._odom_received
        self._odom_received = True

    if first_message:
        self.get_logger().info(
            '已收到第一帧 /odom, 可以接收导航目标'
        )

def goal_callback(self, goal_request):
    target = goal_request.target_pose
    pose = target.pose

    if target.header.frame_id != 'odom':
        self.get_logger().warning(
            f'拒绝目标: 坐标系必须是 odom, '
            f'收到的是 "{target.header.frame_id}"'
        )
        return GoalResponse.REJECT

    values = (
        pose.position.x,
        pose.position.y,
        pose.position.z,
        pose.orientation.x,
        pose.orientation.y,
        pose.orientation.z,
        pose.orientation.w,
    )

    if not all(math.isfinite(value) for value in values):
        self.get_logger().warning(
            '拒绝目标: 位置或姿态包含无效数值'
        )
        return GoalResponse.REJECT

```

```

    if quaternion_norm(pose.orientation) < 1e-9:
        self.get_logger().warning(
            '拒绝目标：四元数长度为 0'
        )
        return GoalResponse.REJECT

    with self._state_lock:
        odom_ready = (
            self._odom_received
            and self._last_odom_time is not None
            and time.monotonic() - self._last_odom_time
            <= self._odom_timeout
        )

    if not odom_ready:
        self.get_logger().warning(
            '拒绝目标：尚未收到有效 /odom'
        )
        return GoalResponse.REJECT

    with self._goal_lock:
        if self._goal_active:
            self.get_logger().warning(
                '拒绝目标：已有导航任务正在执行'
            )
            return GoalResponse.REJECT

        self._goal_active = True

    target_yaw = yaw_from_quaternion(
        pose.orientation
    )

    self.get_logger().info(
        f'接受目标：x={pose.position.x:.2f}, '
        f'y={pose.position.y:.2f}, '
        f'yaw={target_yaw:.2f} rad'
    )

    return GoalResponse.ACCEPT

def cancel_callback(self, goal_handle):
    self.get_logger().info('收到导航取消请求')
    return CancelResponse.ACCEPT

def pose_snapshot(self):
    with self._state_lock:
        return (
            self._current_x,
            self._current_y,
            self._current_z,
            self._current_yaw,
            self._last_odom_time,
        )

def publish_feedback(

```

```

        self,
        goal_handle,
        x,
        y,
        z,
        yaw,
        distance,
    ):
        feedback = MoveBase.Feedback()

        feedback.feedback_pose.position.x = x
        feedback.feedback_pose.position.y = y
        feedback.feedback_pose.position.z = z

        feedback.feedback_pose.orientation.x = 0.0
        feedback.feedback_pose.orientation.y = 0.0
        feedback.feedback_pose.orientation.z = math.sin(
            yaw / 2.0
        )
        feedback.feedback_pose.orientation.w = math.cos(
            yaw / 2.0
        )

        feedback.distance_remaining = distance
        goal_handle.publish_feedback(feedback)

    def stop_robot(self):
        self._cmd_pub.publish(Twist())

    def request_shutdown(self):
        self._shutdown_event.set()
        self.stop_robot()

    def execute_callback(self, goal_handle):
        target = goal_handle.request.target_pose.pose
        target_x = target.position.x
        target_y = target.position.y
        target_yaw = yaw_from_quaternion(
            target.orientation
        )

        result = MoveBase.Result()
        result.success = False

        position_reached = False
        start_time = time.monotonic()

        self.get_logger().info('开始执行导航任务')

        try:
            while (
                rclpy.ok()
                and not self._shutdown_event.is_set()
            ):
                now = time.monotonic()

                if goal_handle.is_cancel_requested:
                    self.get_logger().warning(
                        '导航任务被取消, 停止机器人'
                    )

```

```

        )
        goal_handle.canceled()
        return result

    if now - start_time > self._goal_timeout:
        self.get_logger().error(
            '导航任务超时, 停止机器人'
        )
        goal_handle.abort()
        return result

    (
        current_x,
        current_y,
        current_z,
        current_yaw,
        last_odom_time,
    ) = self.pose_snapshot()

    if (
        last_odom_time is None
        or now - last_odom_time
        > self._odom_timeout
    ):
        self.get_logger().error(
            '/odom 已超时, 停止机器人'
        )
        goal_handle.abort()
        return result

    dx = target_x - current_x
    dy = target_y - current_y
    distance = math.hypot(dx, dy)

    self.publish_feedback(
        goal_handle,
        current_x,
        current_y,
        current_z,
        current_yaw,
        distance,
    )

    command = Twist()

    if not position_reached:
        if distance <= self._position_tolerance:
            position_reached = True
            self.stop_robot()

            self.get_logger().info(
                '目标位置已到达, 开始调整最终 yaw'
            )

            time.sleep(self._control_period)
            continue

    desired_heading = math.atan2(dy, dx)

```

```

        heading_error = normalize_angle(
            desired_heading - current_yaw
        )

        command.angular.z = clamp(
            self._angular_gain * heading_error,
            -self._max_angular_speed,
            self._max_angular_speed,
        )

        if (
            abs(heading_error)
            <= self._drive_heading_limit
        ):
            command.linear.x = min(
                self._max_linear_speed,
                self._linear_gain * distance,
            )
        else:
            command.linear.x = 0.0

    else:
        if distance > self._reacquire_distance:
            position_reached = False

            self.get_logger().warning(
                '调整姿态时偏离目标位置, '
                '重新进入位置控制'
            )
            continue

        yaw_error = normalize_angle(
            target_yaw - current_yaw
        )

        if abs(yaw_error) <= self._yaw_tolerance:
            self.stop_robot()
            goal_handle.succeed()
            result.success = True

            self.get_logger().info(
                '导航成功: 位置和 yaw 均已到达'
            )
            return result

        command.linear.x = 0.0
        command.angular.z = clamp(
            self._angular_gain * yaw_error,
            -self._max_angular_speed,
            self._max_angular_speed,
        )

        self._cmd_pub.publish(command)
        time.sleep(self._control_period)

    if rclpy.ok():
        goal_handle.abort()

```

```

        return result

    finally:
        self.stop_robot()

    with self._goal_lock:
        self._goal_active = False

    self.get_logger().info(
        '导航任务结束，可以接收下一个目标'
    )

def main(args=None):
    rclpy.init(args=args)

    node = PoseNavServer()
    executor = MultiThreadedExecutor(num_threads=3)
    executor.add_node(node)

    try:
        executor.spin()
    except KeyboardInterrupt:
        pass
    finally:
        node.request_shutdown()
        executor.shutdown()
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

编写导航 Action Client

```

nano ~/ros2_ws/src/pose_nav_action/pose_nav_action/nav_client.py
#!/usr/bin/env python3
"""nav_action_client: 发送导航目标并接收进度反馈"""
import math
import sys

import rclpy
from action_msgs.msg import GoalStatus
from rclpy.action import ActionClient
from rclpy.node import Node

from pose_nav_interfaces.action import MoveBase

def normalize_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

```

```

def yaw_from_quaternion(quaternion):
    sin_yaw = 2.0 * (
        quaternion.w * quaternion.z
        + quaternion.x * quaternion.y
    )
    cos_yaw = 1.0 - 2.0 * (
        quaternion.y ** 2
        + quaternion.z ** 2
    )
    return math.atan2(sin_yaw, cos_yaw)

class PoseNavClient(Node):

    def __init__(self, cancel_after=None):
        super().__init__('pose_nav_client')

        self._action_client = ActionClient(
            self,
            MoveBase,
            'move_base_lab',
        )

        self._cancel_after = cancel_after
        self._cancel_timer = None
        self._goal_handle = None
        self._finished = False

    def send_goal(self, x, y, yaw):
        self.get_logger().info(
            '等待 Pose Navigation Server...'
        )

        if not self._action_client.wait_for_server(
            timeout_sec=10.0
        ):
            self.get_logger().error(
                '10 秒内没有找到 Action Server'
            )
            return False

        yaw = normalize_angle(yaw)

        goal = MoveBase.Goal()
        goal.target_pose.header.frame_id = 'odom'
        goal.target_pose.header.stamp = (
            self.get_clock().now().to_msg()
        )

        goal.target_pose.pose.position.x = x
        goal.target_pose.pose.position.y = y
        goal.target_pose.pose.position.z = 0.0

        goal.target_pose.pose.orientation.x = 0.0
        goal.target_pose.pose.orientation.y = 0.0
        goal.target_pose.pose.orientation.z = math.sin(
            yaw / 2.0
        )
        goal.target_pose.pose.orientation.w = math.cos(
            yaw / 2.0

```



```

    )

    self.get_logger().info(
        f'发送目标: x={x:.2f}, y={y:.2f}, '
        f'yaw={yaw:.2f} rad'
    )

    send_future = self._action_client.send_goal_async(
        goal,
        feedback_callback=self.feedback_callback,
    )
    send_future.add_done_callback(
        self.goal_response_callback
    )

    return True

def goal_response_callback(self, future):
    self._goal_handle = future.result()

    if not self._goal_handle.accepted:
        self.get_logger().warning(
            '导航目标被 Server 拒绝'
        )
        self._finished = True
        rclpy.shutdown()
        return

    self.get_logger().info('导航目标已接受')

    result_future = self._goal_handle.get_result_async()
    result_future.add_done_callback(
        self.result_callback
    )

    if self._cancel_after is not None:
        self.get_logger().info(
            f'{self._cancel_after:.1f} 秒后自动取消'
        )
        self._cancel_timer = self.create_timer(
            self._cancel_after,
            self.cancel_goal,
        )

def cancel_goal(self):
    if self._cancel_timer is not None:
        self._cancel_timer.cancel()

    if self._finished or self._goal_handle is None:
        return

    self.get_logger().warning('发送导航取消请求')

    cancel_future = self._goal_handle.cancel_goal_async()
    cancel_future.add_done_callback(
        self.cancel_response_callback
    )

```

```

def cancel_response_callback(self, future):
    response = future.result()

    if response.goals_canceling:
        self.get_logger().info(
            'Server 已接受取消请求'
        )
    else:
        self.get_logger().warning(
            '取消请求未被接受, 任务可能已经结束'
        )

def feedback_callback(self, feedback_message):
    feedback = feedback_message.feedback
    pose = feedback.feedback_pose
    yaw = yaw_from_quaternion(pose.orientation)

    self.get_logger().info(
        f'当前位置=({pose.position.x:.2f}, '
        f'{pose.position.y:.2f}), '
        f'yaw={yaw:.2f}, '
        f'剩余距离={feedback.distance_remaining:.2f} m'
    )

def result_callback(self, future):
    self._finished = True

    if self._cancel_timer is not None:
        self._cancel_timer.cancel()

    response = future.result()
    result = response.result

    status_names = {
        GoalStatus.STATUS_SUCCEEDED: 'SUCCEEDED',
        GoalStatus.STATUS_CANCELED: 'CANCELED',
        GoalStatus.STATUS_ABORTED: 'ABORTED',
    }

    status_name = status_names.get(
        response.status,
        f'UNKNOWN({response.status})',
    )

    self.get_logger().info(
        f'导航结束: 状态={status_name}, '
        f'成功={result.success}'
    )

    rclpy.shutdown()

def main(args=None):
    if len(sys.argv) not in (4, 5):
        print(
            '用法: ros2 run pose_nav_action nav_client '
            '<x> <y> <yaw_rad> [cancel_after_sec]'
        )

```

```

    )
    print(
        '示例 : ros2 run pose_nav_action nav_client '
        '2.0 1.0 1.57'
    )
    print(
        '取消测试 : ros2 run pose_nav_action nav_client '
        '3.0 0.0 0.0 3.0'
    )
    return

try:
    x = float(sys.argv[1])
    y = float(sys.argv[2])
    yaw = float(sys.argv[3])

    cancel_after = (
        float(sys.argv[4])
        if len(sys.argv) == 5
        else None
    )
except ValueError:
    print('错误 : x、y、yaw 和取消时间必须是数字')
    return

values = [x, y, yaw]

if cancel_after is not None:
    values.append(cancel_after)

if not all(math.isfinite(value) for value in values):
    print('错误 : 参数必须是有限数值')
    return

if cancel_after is not None and cancel_after <= 0.0:
    print('错误 : 取消时间必须大于 0')
    return

rclpy.init(args=args)
node = PoseNavClient(cancel_after=cancel_after)

try:
    if node.send_goal(x, y, yaw):
        rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Python package.xml

```
nano ~/ros2_ws/src/pose_nav_action/package.xml
```

```
<?xml version="1.0"?> <package format="3"> <name>pose_nav_action</name>
<version>0.1.0</version> <description>Pose navigation Action Server and
Client</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>roscpp</exec_depend> <exec_depend>action_msgs</exec_depend>
<exec_depend>geometry_msgs</exec_depend>
<exec_depend>nav_msgs</exec_depend>
<exec_depend>pose_nav_interfaces</exec_depend>

<export> <build_type>ament_python</build_type> </export> </package>
```

setup.py

```
nano ~/ros2_ws/src/pose_nav_action/setup.py
```

```
from setuptools import find_packages, setup

package_name = 'pose_nav_action'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        (
            'share/' + package_name,
            ['package.xml'],
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='Pose navigation Action Server and Client',
    license='Apache-2.0',
```

```

    entry_points={
        'console_scripts': [
            'nav_server = pose_nav_action.nav_server:main',
            'nav_client = pose_nav_action.nav_client:main',
        ],
    },
)

```

setup.cfg

```

nano ~/ros2_ws/src/pose_nav_action/setup.cfg

[develop]
script_dir=$base/lib/pose_nav_action
[install]
install_scripts=$base/lib/pose_nav_action

```

编译与运行

打开新终端：

```

cd ~/ros2_ws
source /opt/ros/humble/setup.bash

colcon build --symlink-install \
  --packages-select pose_nav_interfaces pose_nav_action

source ~/ros2_ws/install/setup.bash
ros2 interface show pose_nav_interfaces/action/MoveBase

```

终端 1：

```

source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

LIBGL_DRI3_DISABLE=1 QT_QPA_PLATFORM=xcb \
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=true \
  use_rviz:=true \
  enable_fake_laser:=false \

world:=$HOME/ros2_ws/install/robot_sim_demo_ros2/share/robot_sim_demo_ros2/worlds/ground_test.sdf

```

确认仿真提供正确话题：

```

ros2 topic info /cmd_vel
ros2 topic echo /odom --once

```

/cmd_vel 应使用：

```

geometry_msgs/msg/Twist

```

/odom 应使用：

```
nav_msgs/msg/Odometry
```

启动 Server

终端 2:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_server
```

```
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

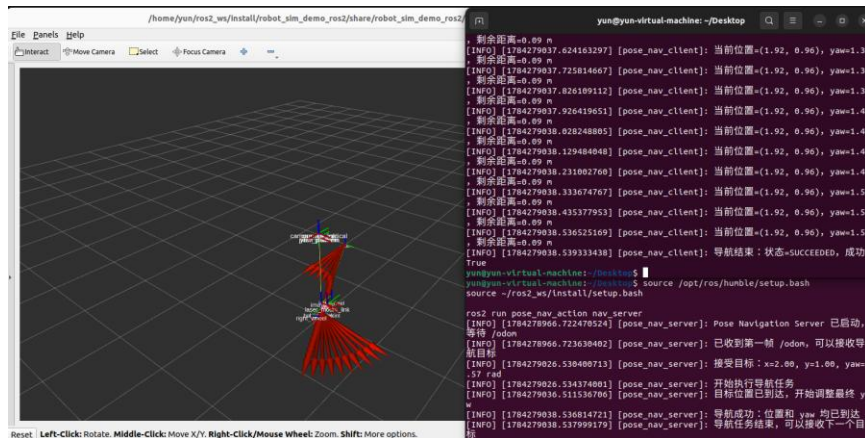
ros2 run pose_nav_action nav_server
[INFO] [1784278966.722470524] [pose_nav_server]: Pose Navigation Server 已启动,
等待 /odom
[INFO] [1784278966.723630402] [pose_nav_server]: 已收到第一帧 /odom, 可以接收导
航目标
```

发送完整位姿目标

终端 3:

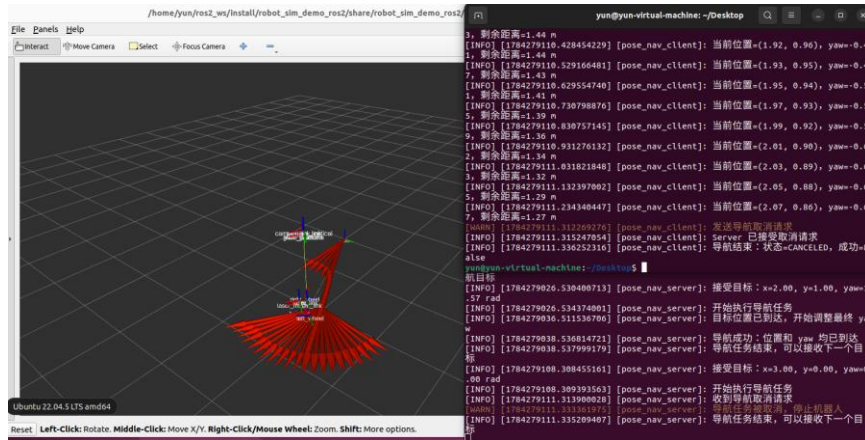
```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_client 2.0 1.0 1.57
```



自动取消测试

```
ros2 run pose_nav_action nav_client 3.0 0.0 0.0 3.0
```



忙碌拒绝测试

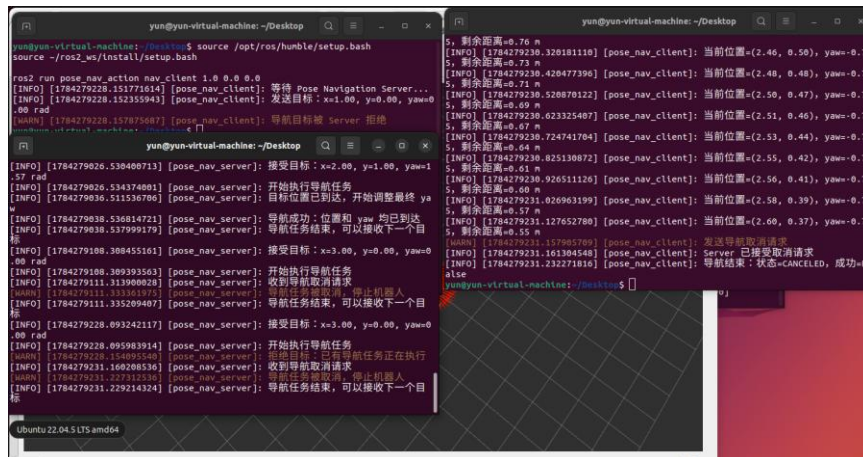
终端 3 先发送较远目标：

```
ros2 run pose_nav_action nav_client 3.0 0.0 0.0 3.0
```

目标尚未结束时，在终端 4 发送第二个目标：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

```
ros2 run pose_nav_action nav_client 1.0 0.0 0.0
```



Action 检查

Server 运行时执行：

```
ros2 action list -t
ros2 action info /move_base_lab
```

```
yun@yun-virtual-machine:~/Desktop$ ros2 action list -t
ros2 action info /move_base_lab
/move_base_lab [pose_nav_interfaces/action/MoveBase]
Action: /move_base_lab
Action clients: 0
Action servers: 1
    /pose_nav_server
yun@yun-virtual-machine:~/Desktop$
```

思考题

83. Action 导航与 Nav2 的区别是什么？什么场景用 Action 更合适？Action 是一种通信机制，适用于执行时间较长、需要实时反馈、支持取消并返回最终结果的任务；Nav2 是导航框架，内部使用 Action（如 NavigateToPose）实现导航功能。Action 更适合导航、机械臂运动、自动充电等长时间任务。



84. 如何在 execute 中同时处理 /odom 回调和 feedback 发布？在节点中订阅 /odom，通过回调实时更新机器人位姿；在 execute() 中循环读取最新位姿，计算任务进度并调用 publish_feedback() 发布反馈，达到目标后返回 Result。如果存在并发访问共享数据，应使用互斥锁等机制保证线程安全。

实际运行证据

真实运行的 DoDishes Action Server、Client 反馈进度和完成结果：

```
ch05 action demo
Captured terminal session
$ ros2 run action_demo_server --ros-args -p stop_duration_sec:=0.15
$ ros2 run action_demo_client
[INFO] [1787891963.509208196] [do_dishes_client]: 目标已设置
[INFO] [1787891963.544025843] [do_dishes_client]: 进度: 20%
[INFO] [1787891963.796076536] [do_dishes_client]: 进度: 40%
[INFO] [1787891963.947159465] [do_dishes_client]: 进度: 60%
[INFO] [1787891964.609197896] [do_dishes_client]: 进度: 80%
[INFO] [1787891964.255003568] [do_dishes_client]: 进度: 100%
[INFO] [1787891964.452444555] [do_dishes_client]: 完成: 目标数字位数=10
$ ros2 action info /do_dishes
Action: /do_dishes
Action clients: 1
Action servers: 1
    /do_dishes_server
# action server process output
[INFO] [1787891963.849781947] [do_dishes_server]: 收到目标: 10000, 2
[INFO] [1787891963.843620202] [do_dishes_server]: 进度: 20%
[INFO] [1787891963.792323212] [do_dishes_server]: 进度: 40%
[INFO] [1787891963.846353636] [do_dishes_server]: 进度: 60%
[INFO] [1787891964.096233676] [do_dishes_server]: 进度: 80%
[INFO] [1787891964.449746729] [do_dishes_server]: 进度: 100%
Source: actions_recording of the current robot state
```

原始录制：[ch05 action.cast](#)。完整证据索引见[实际运行证据](#)。

第 6 章：参数系统与 Launch 文件

****课程**:** ROS2 Python 编程 ****章节**:** 第 6 章 ****课时**:** 2 课时 (90 分钟)

6.1 参数系统

知识点 6.1.1：参数声明与获取

```
import rclpy
from rclpy.node import Node

class ParamDemoNode(Node):
    def __init__(self):
        super().__init__('param_demo')

        # 声明参数：名称、默认值、描述
        self.declare_parameter('robot_name', 'xbot')
        self.declare_parameter('max_speed', 2.0)
        self.declare_parameter('enable_logging', True)
        self.declare_parameter('sensor_list', ['lidar', 'camera'])

        # 获取参数值
        name = self.get_parameter('robot_name').value
        speed = self.get_parameter('max_speed').value
        self.get_logger().info(
            f'机器人: {name}, 最大速度: {speed}m/s')

    def read_param_dynamic(self):
        """运行时动态读取参数"""

        # 方式 1：直接读取
        speed = self.get_parameter('max_speed').get_parameter_value().double_value

        # 方式 2：通过 property 语法
        name = self.get_parameter('robot_name')._value

        return speed
```

知识点 6.1.2：参数回调与动态重配置

```
class DynamicParamNode(Node):
    def __init__(self):
        super().__init__('dynamic_param')
        self.declare_parameter('speed', 1.0)

        # 注册参数变化回调
        self.add_on_set_parameters_callback(self.param_callback)

    def param_callback(self, params):
        """参数变化时自动调用 - 验证并应用新值"""
```

```

from rclpy.parameter import Parameter
for param in params:
    if param.name == 'speed':
        if param.value < 0 or param.value > 10.0:
            self.get_logger().error(
                f'速度必须在 [0.0, 10.0] 范围内, '
                f'收到: {param.value}')
            return SetParametersResult(successful=False)
        self.get_logger().info(
            f'速度已更新: {self.current_speed} → {param.value}')
        self.current_speed = param.value

return SetParametersResult(successful=True)

```

程序 6-1：参数动态回调模式。回调返回 SetParametersResult 告知验证结果。

知识点 6.1.3 : YAML 参数文件

```

# config/robot_params.yaml
param_demo:
  ros__parameters:
    robot_name: 'xbot'
    max_speed: 2.0
    enable_logging: true
    sensor_list: ['lidar', 'camera', 'imu']

# 通过 Launch 文件加载 YAML 参数
# 或命令行直接加载
ros2 run my_pkg param_demo \
  --ros-args --params-file config/robot_params.yaml

```

6.2 Launch 文件系统

知识点 6.2.1 : Python Launch 文件基础结构

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # 启动 talker 节点
        Node(
            package='demo_nodes_py',      # 包名
            executable='talker',           # 可执行文件
            name='my_talker',              # 节点名（可选重映射）
            output='screen',               # 输出到屏幕
        ),
        # 启动 listener 节点
        Node(
            package='demo_nodes_py',
            executable='listener',

```

```

        name='my_listener',
        output='screen',
    ),
])

```

程序 6-2: Python Launch 文件最小示例。

知识点 6.2.2 : 高级 Launch 功能

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, LogInfo
from launch.substitutions import LaunchConfiguration
from launch.conditions import IfCondition
from launch_ros.actions import Node

def generate_launch_description():
    # 声明可配置参数
    use_rviz = LaunchConfiguration('use_rviz', default='false')
    robot_speed = LaunchConfiguration('robot_speed', default='1.0')

    return LaunchDescription([
        # 声明命令行参数
        DeclareLaunchArgument('use_rviz', default_value='false',
                               description='是否启动 RViz'),
        DeclareLaunchArgument('robot_speed', default_value='1.0',
                               description='机器人最大速度'),

        # 条件启动: 仅当 use_rviz=true 时启动 RViz
        Node(
            package='rviz2',
            executable='rviz2',
            condition=IfCondition(use_rviz),
        ),

        # 启动节点并传入参数
        Node(
            package='my_pkg',
            executable='controller',
            name='robot_controller',
            parameters=[{'max_speed': robot_speed}],
            output='screen',
        ),

        # 启动信息日志
        LogInfo(msg=['启动完成, 速度=', robot_speed]),
    ])

```

知识点 6.2.3 : IncludeLaunchDescription 组合启动

```

from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

# 引用另一个 Launch 文件
def generate_launch_description():
    return LaunchDescription([

```

```
# 先启动机器人仿真
IncludeLaunchDescription(
    PythonLaunchDescriptionSource([
        get_package_share_directory('robot_sim_demo_ros2'),
        '/launch/sim_bringup.launch.py'
    ]),
    launch_arguments={'use_gazebo': 'true'}.items(),
),
# 再启动导航
Node(
    package='my_pkg',
    executable='navigator',
),
])
```

6.3 本章小结

- 85. 参数声明 `declare_parameter(name, default)`，获取 `get_parameter(name).value`
- 86. 参数回调 `add_on_set_parameters_callback()` 实现动态重配置和验证
- 87. YAML 文件存储参数 → 通过 `--params-file` 或 Launch 加载
- 88. Python Launch 文件使用 `Node()` 启动节点，`LaunchConfiguration()` 传递参数
- 89. `IfCondition` 实现条件启动，`IncludeLaunchDescription` 组合多个 Launch

6.4 练习题

- 练习 6.1：**编写节点 `param_demo`，声明 `name`、`speed`、`mode` 三个参数，每秒输出参数值。
- 练习 6.2：**实现参数回调验证：`speed` 必须在 0.0~10.0 范围内，`mode` 只能是 `"auto"/"manual"/"hybrid"`。
- 练习 6.3：**编写 YAML 参数文件，通过 `--params-file` 加载覆盖默认参数。
- 练习 6.4：**编写 Python Launch 文件，同时启动 `talker`、`listener` 两个节点。
- 练习 6.5：**在 Launch 中添加条件启动参数 `use_rviz`，控制 RViz 是否启动。
- 练习 6.6：**使用 `ros2 param list/get/set` 命令行操作节点参数。

实验 6:

第 6 章 实验指导书：参数系统与 Launch 文件

****实验课时****: 2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

- 90. 声明、读取、设置参数
- 91. 实现参数动态回调查证
- 92. 编写 YAML 参数文件和 Python Launch 文件
- 93. 条件启动与组合启动

练习 3.1：参数 CRUD 操作（约 30 分钟）

创建 param_demo 节点，声明 robot_name、max_speed、sensor_list 三个参数，每秒输出参数值。使用 ros2 param 命令行动态修改参数。

参考代码：lab_code/ch06_lab/param_demo/

创建 param_demo 功能包

```
cd ~/ros2_ws/src
ros2 pkg create param_demo \
  --build-type ament_python \
  --license Apache-2.0 \
  --dependencies rclpy rcl_interfaces geometry_msgs demo_nodes_py launch launch_ros
ament_index_python rviz2
```

创建 Launch 和 YAML 配置目录：

```
cd ~/ros2_ws/src/param_demo
mkdir -p launch config
```

配置功能包公共文件

package.xml

文件路径:

```
nano ~/ros2_ws/src/param_demo/package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>param_demo</name>
  <version>0.1.0</version>
  <description>ROS 2 parameter and launch laboratory</description>

  <maintainer email="student@example.com">Student</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>rcl_interfaces</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>demo_nodes_py</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <exec_depend>ament_index_python</exec_depend>
  <exec_depend>rviz2</exec_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

setup.py

```
nano ~/ros2_ws/src/param_demo/setup.py
import os
from glob import glob

from setuptools import find_packages, setup

package_name = 'param_demo'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        ('share/' + package_name, ['package.xml']),
        (
            os.path.join('share', package_name, 'launch'),
```

```

        glob('launch/*.launch.py'),
    ),
    (
        os.path.join('share', package_name, 'config'),
        glob('config/*.yaml'),
    ),
],
install_requires=['setuptools'],
zip_safe=True,
maintainer='Student',
maintainer_email='student@example.com',
description='ROS 2 parameter and launch laboratory',
license='Apache-2.0',
entry_points={
    'console_scripts': [
        'param_node = param_demo.param_demo:main',
        'speed_ctrl = param_demo.speed_controller:main',
    ],
},
)

```

setup.cfg

```

nano ~/ros2_ws/src/param_demo/setup.cfg

[develop]
script_dir=$base/lib/param_demo

[install]
install_scripts=$base/lib/param_demo

```

空文件

```

~/ros2_ws/src/param_demo/resource/param_demo
~/ros2_ws/src/param_demo/param_demo/__init__.py

```

编写 param_demo.py

```
~/ros2_ws/src/param_demo/param_demo/param_demo.py
```

完整代码：

```

#!/usr/bin/env python3
"""ROS 2 参数声明、读取、动态修改和验证示例。"""

import rclpy
from rcl_interfaces.msg import SetParametersResult
from rclpy.node import Node

class ParamDemoNode(Node):

    def __init__(self):
        super().__init__('param_demo')

        # 练习 3.1 要求的参数。
        self.declare_parameter('robot_name', 'xbot')
        self.declare_parameter('max_speed', 2.0)
        self.declare_parameter('sensor_list', ['lidar', 'camera'])

```



```

# 练习 3.2 使用的参数。
self.declare_parameter('mode', 'auto')
self.declare_parameter('enable_debug', False)

# 参数正式写入前，先调用 validate_parameters() 进行验证。
self.add_on_set_parameters_callback(self.validate_parameters)

# 每 1 秒执行一次 print_parameters()。
self.timer = self.create_timer(1.0, self.print_parameters)

self.get_logger().info('param_demo 节点已经启动')

def validate_parameters(self, params):
    """检查准备写入的新参数，非法参数将被拒绝。"""
    for param in params:
        if param.name == 'max_speed':
            if param.value < 0.0 or param.value > 10.0:
                return SetParametersResult(
                    successful=False,
                    reason='max_speed 必须在 [0.0, 10.0] 范围内',
                )

            elif param.name == 'mode':
                allowed_modes = ('auto', 'manual', 'hybrid')
                if param.value not in allowed_modes:
                    return SetParametersResult(
                        successful=False,
                        reason='mode 必须是 auto、manual 或 hybrid',
                    )

    return SetParametersResult(successful=True)

def print_parameters(self):
    """每秒读取并输出一当前参数。"""
    robot_name = self.get_parameter('robot_name').value
    max_speed = self.get_parameter('max_speed').value
    sensor_list = self.get_parameter('sensor_list').value
    mode = self.get_parameter('mode').value
    enable_debug = self.get_parameter('enable_debug').value

    self.get_logger().info(
        f'robot_name={robot_name} | '
        f'max_speed={max_speed:.2f} m/s | '
        f'sensor_list={sensor_list} | '
        f'mode={mode} | '
        f'enable_debug={enable_debug}'
    )

def main(args=None):
    rclpy.init(args=args)
    node = ParamDemoNode()

    try:
        rclpy.spin(node)

```

```
except KeyboardInterrupt:
    pass
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

编译功能包

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build --packages-select param_demo --symlink-install
source ~/ros2_ws/install/setup.bash
```

运行参数节点

终端 1

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node
```

终端 2

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

检查节点

```
ros2 node list
```

列出全部参数

```
ros2 param list /param_demo
```

读取字符串参数

```
ros2 param get /param_demo robot_name
```

读取浮点参数

```
ros2 param get /param_demo max_speed
```

读取字符串数组

```
ros2 param get /param_demo sensor_list
```

查看参数说明和类型

```
ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
```

```

yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
yun@yun-virtual-machine:~/ros2_ws$ ros2 node list
/param_demo
yun@yun-virtual-machine:~/ros2_ws$ ros2 param list /param_demo
  enable_debug
  max_speed
  mode
  robot_name
  sensor_list
  use_sim_time
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo robot_name
String value is: xbot
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 2.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo sensor_list
String values are: ['lidar', 'camera']
yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
Parameter name: max_speed
  Type: double
  Constraints:
Parameter name: sensor_list
  Type: string array
  Constraints:
yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
Parameter name: max_speed
  Type: double
  Constraints:
Parameter name: sensor_list
  Type: string array
  Constraints:
yun@yun-virtual-machine:~/ros2_ws$ █

```

动态修改参数

修改机器人名称

```
ros2 param set /param_demo robot_name xbot_u
```

```

s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugString value is: xbot
INFO] [1784523135.329208502] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugDouble value is: 2.0
INFO] [1784523136.328827883] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo sensor_list
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugString values are: ['lidar', 'camera']
INFO] [1784523137.327985741] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugros2 param describe /param_demo sensor_list
INFO] [1784523138.328956026] [param_demo]: robot_name=xbot_u | Parameter name: max_speed
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug  Type: double
INFO] [1784523139.328613816] [param_demo]: robot_name=xbot_u | Constraints:
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugParameter name: sensor_list
INFO] [1784523140.327637251] [param_demo]: robot_name=xbot_u | Type: string array
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug Constraints:
INFO] [1784523141.328843326] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugros2 param describe /param_demo sensor_list
INFO] [1784523142.328494323] [param_demo]: robot_name=xbot_u | Parameter name: max_speed
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug  Type: double
INFO] [1784523144.328712530] [param_demo]: robot_name=xbot_u | Constraints:
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugParameter name: sensor_list
INFO] [1784523145.328815502] [param_demo]: robot_name=xbot_u | Type: string array
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug Constraints:
INFO] [1784523146.327649231] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo robot_nam
s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugSet parameter successful
yun@yun-virtual-machine:~/ros2_ws$ █

```

修改最大速度

```
ros2 param set /param_demo max_speed 4.5
```

```
ros2 param get /param_demo max_speed
```

```

yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo robot_name xbot_u
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 4.5
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 4.5
yun@yun-virtual-machine:~/ros2_ws$ █

```

修改传感器数组

```
ros2 param set /param_demo sensor_list "['lidar', 'camera', 'imu']"
```

```
ros2 param get /param_demo sensor_list
```

修改布尔参数

```
ros2 param set /param_demo enable_debug true
```

关闭调试:

```
ros2 param set /param_demo enable_debug false
```

```
yun@yun-virtual-machine: ~/ros2_ws
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523339.328717137] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523340.328729590] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523341.328540946] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523342.328338157] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523343.327720230] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523345.327937533] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523347.328361231] [param_demo]: robot_name=xbot_u | max_speed=4.50 m

yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
Parameter name: max_speed
  Type: double
  Constraints:
Parameter name: sensor_list
  Type: string array
  Constraints:
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo robot_name xbot_u
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 4.5
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 4.5
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo sensor_list "['lidar', 'camera', 'imu']"
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo sensor_list
String values are: ['lidar', 'camera', 'imu']
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo enable_debug true
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo enable_debug false
```

导出当前参数

```
ros2 param dump /param_demo
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param dump /param_demo
/param_demo:
  ros__parameters:
    enable_debug: false
    max_speed: 4.5
    mode: auto
    robot_name: xbot_u
    sensor_list:
      - lidar
      - camera
      - imu
    use_sim_time: false
```

练习 3.2 : 参数回调查证 (约 30 分钟)

实现 `add_on_set_parameters_callback`: 验证 `max_speed` 范围 `[0.0, 10.0]` 和 `mode` 值 `{"auto","manual","hybrid"}`。

```
self.add_on_set_parameters_callback(self.validate_parameters)
```

对应验证函数为:

```
def validate_parameters(self, params):
    for param in params:
        if param.name == 'max_speed':
            if param.value < 0.0 or param.value > 10.0:
                return SetParametersResult(
                    successful=False,
                    reason='max_speed 必须在 [0.0, 10.0] 范围内',
                )

            elif param.name == 'mode':
                allowed_modes = ('auto', 'manual', 'hybrid')
                if param.value not in allowed_modes:
                    return SetParametersResult(
                        successful=False,
                        reason='mode 必须是 auto、manual 或 hybrid',
                    )

    return SetParametersResult(successful=True)
```

终端 1

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node
```

终端 2

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

测试 `max_speed`

设置合法值

```
ros2 param set /param_demo max_speed 8.0
ros2 param get /param_demo max_speed
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 8.0
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 8.0
```

测试小于下限的值

```
ros2 param set /param_demo max_speed -1.0
ros2 param get /param_demo max_speed
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed -1.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 8.0
```

测试大于上限的值

```
ros2 param set /param_demo max_speed 15.0
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed -1.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 8.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$
```

测试边界值

```
ros2 param set /param_demo max_speed 0.0
ros2 param set /param_demo max_speed 10.0
ros2 param set /param_demo max_speed 10.1
```

```
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 0.0
ros2 param set /param_demo max_speed 10.0
ros2 param set /param_demo max_speed 10.1
Set parameter successful
Set parameter successful
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$
```

17. 测试 mode

测试三个合法模式

```
ros2 param set /param_demo mode auto
ros2 param set /param_demo mode manual
ros2 param set /param_demo mode hybrid
```

测试非法模式

```
ros2 param set /param_demo mode sport
ros2 param get /param_demo mode
ros2 param set /param_demo mode AUTO
```



```
Double value is: 8.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 0.0
ros2 param set /param_demo max_speed 10.0
ros2 param set /param_demo max_speed 10.1
Set parameter successful
Set parameter successful
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo mode auto
ros2 param set /param_demo mode manual
ros2 param set /param_demo mode hybrid
Set parameter successful
Set parameter successful
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo mode sport
ros2 param get /param_demo mode
Setting parameter failed: mode 必须是 auto、manual 或 hybrid
String value is: hybrid
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo mode AUTO
Setting parameter failed: mode 必须是 auto、manual 或 hybrid
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$
```

YAML 参数文件实验

创建 params.yaml

文件路径:

```
~/ros2_ws/src/param_demo/config/params.yaml
```

完整内容:

```
param_demo:
  ros__parameters:
    robot_name: 'xbot_u'
    max_speed: 3.5
    sensor_list: ['lidar', 'camera', 'imu']
    mode: 'hybrid'
    enable_debug: true
```

通过命令行加载 YAML

先停止之前运行的 /param_demo 节点, 避免出现同名节点 终端 1 中执行:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node --ros-args \
  --params-file ~/ros2_ws/src/param_demo/config/params.yaml
```

```
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node --ros-args \
  --params-file ~/ros2_ws/src/param_demo/config/params.yaml
[INFO] [1784528333.364383935] [param_demo]: param_demo 节点已经启动
[INFO] [1784528334.353611193] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528335.353164515] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528336.353659063] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528337.353804357] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528338.354212533] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528339.353668639] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
```

启动后的输出应包含：

向运行中的节点加载 YAML

保持节点运行，终端 2 执行：

```
ros2 param load /param_demo \
  ~/ros2_ws/src/param_demo/config/params.yaml #####
ros2 param get /param_demo robot_name
ros2 param get /param_demo max_speed
ros2 param get /param_demo sensor_list
ros2 param get /param_demo mode
ros2 param get /param_demo enable_debug
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param load /param_demo \
  ~/ros2_ws/src/param_demo/config/params.yaml
Param file does not contain parameters for /param_demo, only for nodes: dict_ke
ys(['param_demo'])
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo robot_name
ros2 param get /param_demo max_speed
ros2 param get /param_demo sensor_list
ros2 param get /param_demo mode
ros2 param get /param_demo enable_debug
String value is: xbot_u
Double value is: 3.5
String values are: ['lidar', 'camera', 'imu']
String value is: hybrid
Boolean value is: True
```

练习 3.3 : Launch 文件实战（约 30 分钟）

94. 编写 Python Launch 文件，同时启动 talker + listener

95. 添加 use_rviz 启动参数控制 RViz 是否启动

96. 组合仿真 Launch：IncludeLaunchDescription 引用 XBot-U 仿真 + 自主导航节点

提示： 参考 `ros2_ws/src/navigation_sim_demo_ros2/launch/nav2_demo.launch.py`

启动 talker、listener 和 RViz

```
~/ros2_ws/src/param_demo/launch/demo.launch.py
#!/usr/bin/env python3

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
```



```

from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    use_rviz = LaunchConfiguration('use_rviz')

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_rviz',
            default_value='false',
            description='是否启动 RViz2',
        ),

        Node(
            package='demo_nodes_py',
            executable='talker',
            name='my_talker',
            output='screen',
        ),

        Node(
            package='demo_nodes_py',
            executable='listener',
            name='my_listener',
            output='screen',
        ),

        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
            output='screen',
            condition=IfCondition(use_rviz),
        ),
    ])

```

使用 Launch 加载 params.yaml

```

~/ros2_ws/src/param_demo/launch/param_with_yaml.launch.py
#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    default_params_file = os.path.join(
        get_package_share_directory('param_demo'),
        'config',
        'params.yaml',
    )

```

```

params_file = LaunchConfiguration('params_file')

return LaunchDescription([
    DeclareLaunchArgument(
        'params_file',
        default_value=default_params_file,
        description='param_demo 节点使用的 YAML 参数文件',
    ),

    Node(
        package='param_demo',
        executable='param_node',
        name='param_demo',
        output='screen',
        parameters=[params_file],
    ),
])

```

编写组合仿真与导航 Launch

```

~/ros2_ws/src/param_demo/launch/combined_sim_nav.launch.py
#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    IncludeLaunchDescription,
)
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    use_rviz = LaunchConfiguration('use_rviz')
    use_gazebo = LaunchConfiguration('use_gazebo')
    gz_headless = LaunchConfiguration('gz_headless')
    use_sim_time = LaunchConfiguration('use_sim_time')

    navigation_launch_path = os.path.join(
        get_package_share_directory('navigation_sim_demo_ros2'),
        'launch',
        'nav2_demo.launch.py',
    )

    navigation_stack = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(navigation_launch_path),
        launch_arguments={
            'use_rviz': use_rviz,
            'use_gazebo': use_gazebo,
            'gz_headless': gz_headless,
            'use_sim_time': use_sim_time,
        }.items(),
    )

```

```

return LaunchDescription([
    DeclareLaunchArgument(
        'use_rviz',
        default_value='true',
        description='是否启动 RViz2',
    ),
    DeclareLaunchArgument(
        'use_gazebo',
        default_value='true',
        description='是否启动 Gazebo Sim',
    ),
    DeclareLaunchArgument(
        'gz_headless',
        default_value='false',
        description='是否以无界面模式运行 Gazebo',
    ),
    DeclareLaunchArgument(
        'use_sim_time',
        default_value='true',
        description='是否使用仿真时钟',
    ),
    navigation_stack,
])

```

重新编译 Launch 文件

在终端 A 中执行：

```

cd ~/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --base-paths src/param_demo \
  --packages-select param_demo \
  --symlink-install
source ~/ros2_ws/install/setup.bash
ls ~/ros2_ws/install/param_demo/share/param_demo/launch

```

应看到：

```

combined_sim_nav.launch.py
demo.launch.py
param_with_yaml.launch.py

```

测试 talker 和 listener

先确保之前的测试节点已经停止，然后打开终端 1：

```

source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch param_demo demo.launch.py

```

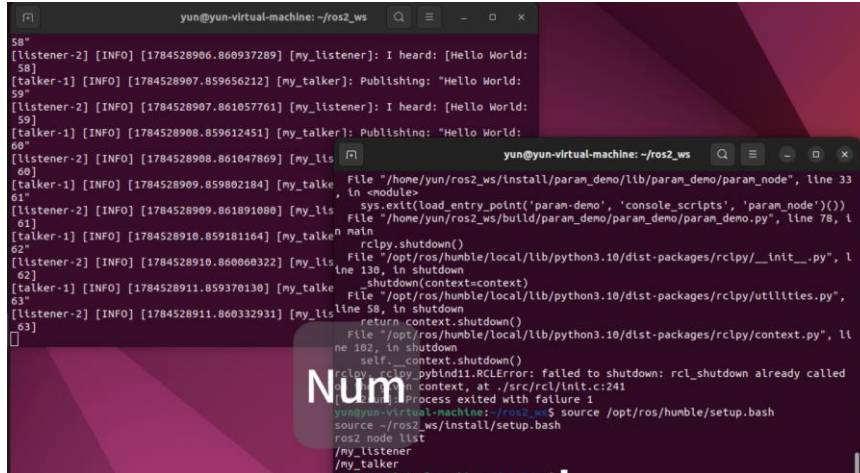
因为 use_rviz 默认是 false，此时只会启动 talker 和 listener。

终端应持续出现类似输出：

```
[my_talker]: Publishing: "Hello World: 1"  
[my_listener]: I heard: [Hello World: 1]
```

在终端 2 中检查节点：

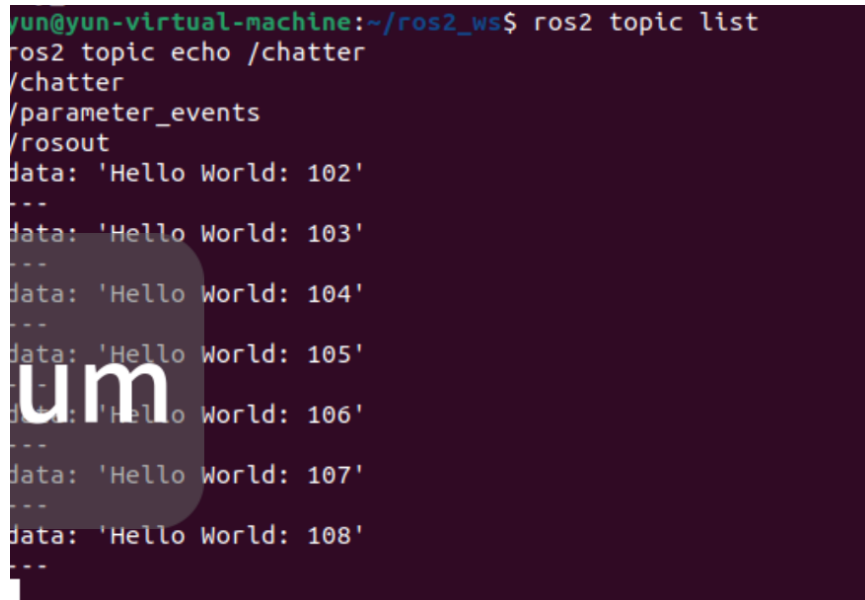
```
source /opt/ros/humble/setup.bash  
source ~/ros2_ws/install/setup.bash  
ros2 node list
```



```
yun@yun-virtual-machine: ~/ros2_ws  
58"  
[listener-2] [INFO] [1784528906.860937289] [my_listener]: I heard: [Hello World:  
58]  
[talker-1] [INFO] [1784528907.859656212] [my_talker]: Publishing: "Hello World:  
59"  
[listener-2] [INFO] [1784528907.861057761] [my_listener]: I heard: [Hello World:  
59]  
[talker-1] [INFO] [1784528908.859612451] [my_talker]: Publishing: "Hello World:  
60"  
[listener-2] [INFO] [1784528908.861047869] [my_lis  
60]  
[talker-1] [INFO] [1784528909.859802184] [my_talke  
61"  
[listener-2] [INFO] [1784528909.861891080] [my_lis  
61]  
[talker-1] [INFO] [1784528910.859181164] [my_talke  
62"  
[listener-2] [INFO] [1784528910.86060322] [my_lis  
62]  
[talker-1] [INFO] [1784528911.859370130] [my_talke  
63"  
[listener-2] [INFO] [1784528911.860332931] [my_lis  
63]  
[  
File "/home/yun/ros2_ws/install/param_deno/lib/param_deno/param_node", line 33  
, in <module>  
sys.exit(load_entry_point('param-demo', 'console_scripts', 'param_node'))()  
File "/home/yun/ros2_ws/build/param_deno/param_deno/param_deno.py", line 78, i  
main  
rclpy.shutdown()  
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", l  
line 130, in shutdown  
shutdown(context=context)  
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/utilities.py",  
line 58, in shutdown  
return context.shutdown()  
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/context.py", li  
ne 102, in shutdown  
self._context.shutdown()  
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/pybind11.RCLError: failed to shutdown: rcl_shutdown already called  
in context, at ./src/rcl/init.c:241  
Process exited with failure 1  
yun@yun-virtual-machine: ~/ros2_ws$ source /opt/ros/humble/setup.bash  
source ~/ros2_ws/install/setup.bash  
ros2 node list  
/my_listener  
/my_talker
```

应看到：

```
ros2 topic list  
ros2 topic echo /chatter
```



```
yun@yun-virtual-machine: ~/ros2_ws$ ros2 topic list  
ros2 topic echo /chatter  
/chatter  
/parameter_events  
/rosout  
data: 'Hello World: 102'  
---  
data: 'Hello World: 103'  
---  
data: 'Hello World: 104'  
---  
data: 'Hello World: 105'  
---  
data: 'Hello World: 106'  
---  
data: 'Hello World: 107'  
---  
data: 'Hello World: 108'  
---
```

测试 use_rviz 条件启动

不启动 RViz

终端 1:

```
ros2 launch param_demo demo.launch.py use_rviz:=false
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 launch param_demo demo.launch.py use_rviz:=false
[INFO] [launch]: All log files can be found below /home/yun/.ros/log/2026-07-20-14-30-29-729748-yun-virtual-machine-19994
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [talker-1]: process started with pid [19995]
[INFO] [listener-2]: process started with pid [19997]
[talker-1] [INFO] [1784529031.031730556] [my_talker]: Publishing: "Hello World: 0"
[listener-2] [INFO] [1784529031.048279324] [my_listener]: I heard: [Hello World: 0]
[talker-1] [INFO] [1784529032.016248676] [my_talker]: Publishing: "Hello World: 1"
[listener-2] [INFO] [1784529032.018316200] [my_listener]: I heard: [Hello World: 1]
[talker-1] [INFO] [1784529033.015772355] [my_talker]: Publishing: "Hello World: 2"
```

终端 2:

```
ros2 node list
```

启动 RViz

```
ros2 launch param_demo demo.launch.py use_rviz:=true
```

```
ros2 node list
```

查看 Launch 参数说明:

```
ros2 launch param_demo demo.launch.py --show-args
```

```
5]
[talker-1] [INFO] [1784529364.146397914] [my_talker]: Publishing: "Hello World: 219"
[listener-2] [INFO] [1784529364.147084803] [my_listener]: I heard: [Hello World: 219]
[listener-2] [INFO] [1784529364.253070240] [my_listener]: I heard: [Hello World: 0]
[talker-1] [INFO] [1784529365.146553222] [my_talker]: Publishing: "Hello World: 220"
[listener-2] [INFO] [1784529365.147542848] [my_listener]: I heard: [Hello World: 220]
[listener-2] [INFO] [1784529365.253608670] [my_listener]: I heard: [Hello World: 7]
[talker-1] [INFO] [1784529366.146948044] [my_talker]: Publishing: "Hello World: 221"
[listener-2] [INFO] [1784529366.148513097] [my_listener]: I heard: [Hello World: 221]
[listener-2] [INFO] [1784529366.148513097] [my_listener]: I heard: [Hello World: 222]
[talker-1] [INFO] [1784529367.148862220] [my_talker]: Publishing: "Hello World: 222"
[listener-2] [INFO] [1784529367.253291399] [my_listener]: I heard: [Hello World: 222]
[listener-2] [INFO] [1784529367.253291399] [my_listener]: I heard: [Hello World: 217]
[talker-1] [INFO] [1784529367.253300666] [my_talker]: Publishing: "Hello World: 4"
```

测试 Launch 加载 YAML

先停止前一个 Launch，然后在终端 1 中执行:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch param_demo param_with_yaml.launch.py
```

```
[param_node-1] [INFO] [1784529444.947081287] [param_demo]: param_demo 节点已经启动
[param_node-1] [INFO] [1784529445.937667922] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529446.937689967] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529447.937864815] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529448.937758057] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529449.937642769] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529450.938095463] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529451.938142080] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
```

测试组合 Launch

检查依赖包

执行：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 pkg prefix robot_sim_demo_ros2
ros2 pkg prefix navigation_sim_demo_ros2
```

启动组合系统

```
LIBGL_ALWAYS_SOFTWARE=1 \
ros2 launch param_demo combined_sim_nav.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=true \
  use_sim_time:=true
```

正常启动顺序为：

97. nav2_demo.launch.py 包含并启动 XBot-U Gazebo 仿真。
98. 启动 Nav2 节点并加载地图。
99. 启动 RViz2。
100. 导航生命周期节点激活后输出 Nav2 stack is active。

不需要 RViz 时执行：

```
ros2 launch param_demo combined_sim_nav.launch.py use_rviz:=false
```

在终端 2 中可以检查：

```
ros2 node list
ros2 topic list
ros2 service list
ros2 action list
```

思考题

101. LaunchConfiguration 和 DeclareLaunchArgument 的区别？
102. 什么是 GroupAction(scoped=True)，何时使用？

练习 4：动态参数调速 — 运行时修改机器人速度（约 15 分钟）

目标

创建 speed_controller 节点，持续发布 /cmd_vel 控制机器人运动，通过动态参数实时调整线速度和角速度。

步骤

编写 speed_controller.py

```
~/ros2_ws/src/param_demo/param_demo/speed_controller.py
#!/usr/bin/env python3
"""使用动态参数控制机器人的线速度和角速度。"""

import rclpy
from geometry_msgs.msg import Twist
from rcl_interfaces.msg import SetParametersResult
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node

class SpeedController(Node):

    def __init__(self):
        super().__init__('speed_controller')

        self.declare_parameter('linear_speed', 0.2)
        self.declare_parameter('angular_speed', 0.0)
        self.declare_parameter('enable_control', True)

        self.add_on_set_parameters_callback(
            self.validate_parameters
        )
```

```

        self.publisher = self.create_publisher(
            Twist,
            '/cmd_vel',
            10,
        )

        # 0.1 秒执行一次，即 10 Hz。
        self.timer = self.create_timer(0.1, self.control_loop)

        self.get_logger().info('speed_controller 已启动')

def validate_parameters(self, params):
    """在参数写入前检查速度范围。"""
    for param in params:
        if param.name == 'linear_speed':
            if abs(param.value) > 1.0:
                return SetParametersResult(
                    successful=False,
                    reason='线速度范围必须是 [-1.0, 1.0] m/s',
                )

            elif param.name == 'angular_speed':
                if abs(param.value) > 2.0:
                    return SetParametersResult(
                        successful=False,
                        reason='角速度范围必须是 [-2.0, 2.0] rad/s',
                    )

    return SetParametersResult(successful=True)

def control_loop(self):
    """读取实时参数并发布速度。"""
    enabled = self.get_parameter('enable_control').value

    message = Twist()

    if enabled:
        message.linear.x = self.get_parameter(
            'linear_speed'
        ).value
        message.angular.z = self.get_parameter(
            'angular_speed'
        ).value

    # 禁用控制时 message 保持全零，保证机器人停止。
    self.publisher.publish(message)

    self.get_logger().info(
        f'enable={enabled}, '
        f'v={message.linear.x:.2f} m/s, '
        f'w={message.angular.z:.2f} rad/s',
        throttle_duration_sec=2.0,
    )

def main(args=None):
    rclpy.init(args=args)

```



```

node = SpeedController()

try:
    rclpy.spin(node)
except (KeyboardInterrupt, ExternalShutdownException):
    pass
finally:
    # Humble 收到 Ctrl+C 后可能已经关闭 context。
    # 只有 context 仍有效时才能继续发布消息。
    if rclpy.ok():
        node.publisher.publish(Twist())
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

创建初始速度 YAML

```

~/ros2_ws/src/param_demo/config/speed_params.yaml

speed_controller:
  ros__parameters:
    linear_speed: 0.2
    angular_speed: 0.0
    enable_control: true

```

创建 speed.launch.py

```

~/ros2_ws/src/param_demo/launch/speed.launch.py

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    params_file = os.path.join(
        get_package_share_directory('param_demo'),
        'config',
        'speed_params.yaml',
    )

    return LaunchDescription([
        Node(
            package='param_demo',
            executable='speed_ctrl',
            name='speed_controller',
            output='screen',
            parameters=[params_file],
        ),
    ])

```

编译速度控制节点

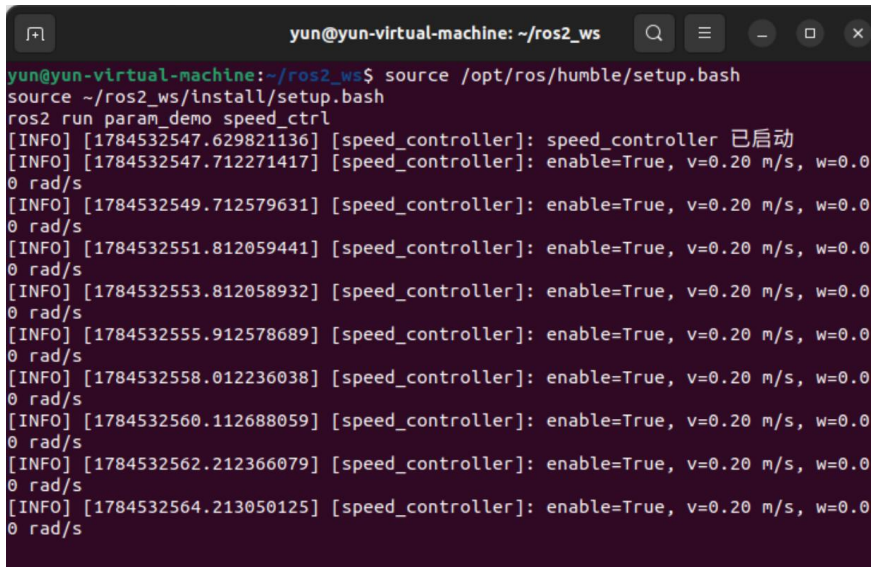
```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --base-paths src/param_demo \
  --packages-select param_demo \
  --symlink-install
source ~/ros2_ws/install/setup.bash
ros2 pkg executables param_demo
```

启动 Gazebo 仿真

```
source ~/ros2_ws/install/setup.bash
ros2 pkg prefix robot_sim_demo_ros2
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
```

启动速度控制节点

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo speed_ctrl
```



```
yun@yun-virtual-machine: ~/ros2_ws
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo speed_ctrl
[INFO] [1784532547.629821136] [speed_controller]: speed_controller 已启动
[INFO] [1784532547.712271417] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532549.712579631] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532551.812059441] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532553.812058932] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532555.912578689] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532558.012236038] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532560.112688059] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532562.212366079] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532564.213050125] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
```

也可以使用 YAML

Launch 启动:

```
ros2 launch param_demo speed.launch.py
```

检查节点、参数和话题

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

检查节点:

```
ros2 node list
ros2 param list /speed_controller
```

读取初始值:

```
ros2 param get /speed_controller linear_speed
ros2 param get /speed_controller angular_speed
ros2 param get /speed_controller enable_control
```

检查 /cmd_vel:

```
ros2 topic info /cmd_vel
```

```
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
yun@yun-virtual-machine:~/ros2_ws$ ros2 node list
/controller_manager
/fake_laser
/gazebo_interface_bridge
/gz_ros2_control
/joint_state_broadcaster
/robot_state_publisher
/speed_controller
/twist_mux
/wheel_velocity_controller
yun@yun-virtual-machine:~/ros2_ws$ ros2 param list /speed_controller
  angular_speed
  enable_control
  linear_speed
  use_sim_time
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /speed_controller linear_speed
0.2
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /speed_controller angular_speed
0.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /speed_controller enable_control
True
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic info /cmd_vel
Type: geometry_msgs/msg/Twist
Publisher count: 2
Subscription count: 1
yun@yun-virtual-machine:~/ros2_ws$
```

动态修改速度

加速直行

```
ros2 param set /speed_controller linear_speed 0.5
```

```
[speed_ctrl-1] [INFO] [1784533119.675168628] [speed] Double value is: 0.0
0.50 m/s, w=0.00 rad/s
[speed_ctrl-1] [INFO] [1784533121.675247713] [speed] Boolean value is: True
0.50 m/s, w=0.00 rad/s
[speed_ctrl-1] [INFO] [1784533123.775269602] [speed] Type: geometry_msgs/msg/Twist
0.50 m/s, w=0.00 rad/s
[speed_ctrl-1] [INFO] [1784533125.775945865] [speed] Publisher count: 2
0.50 m/s, w=0.00 rad/s
[speed_ctrl-1] [INFO] [1784533125.775945865] [speed] Subscription count: 1
0.50 m/s, w=0.00 rad/s
[speed_ctrl-1] [INFO] [1784533125.775945865] [speed] Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$
```

一边前进一边左转

```
ros2 param set /speed_controller angular_speed 1.0
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /speed_controller angular_speed 1.0
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$
[speed_ctrl-1] [INFO] [1784533175.375757236] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533177.475163436] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533179.475574480] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533181.575182529] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533183.575777272] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533185.675854106] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533187.775686288] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
```

一边前进一边右转

```
ros2 param set /speed_controller angular_speed -1.0
```

停止转动，继续直行

```
ros2 param set /speed_controller angular_speed 0.0
```

倒车

```
ros2 param set /speed_controller linear_speed -0.3
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /speed_controller linear_speed -0.3
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$
[speed_ctrl-1] [INFO] [1784533238.975428728] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533240.975757510] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533243.075948030] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533245.076771676] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533247.176779499] [speed_controller]: enable=True, v=-0.30 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533249.275161402] [speed_controller]: enable=True, v=-0.30 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533251.276622234] [speed_controller]: enable=True, v=-0.30 m/s, w=1.00 rad/s
```

#负线速度表示后退。

完全停止

同时把两个速度设置为零：

```
ros2 param set /speed_controller linear_speed 0.0
ros2 param set /speed_controller angular_speed 0.0
```

暂停和恢复控制

暂停控制：

```
ros2 param set /speed_controller enable_control false
```

在禁用状态下修改速度：

```
ros2 param set /speed_controller linear_speed 0.4
ros2 param set /speed_controller angular_speed 0.5
```

参数可以修改成功，但节点仍发布零速度，机器人不会运动。

恢复控制：

```
ros2 param set /speed_controller enable_control true
```

恢复后，节点会立即使用刚才保存的 0.4 和 0.5，机器人开始沿弧线运动。

实验结束前再次完全停止：

```
ros2 param set /speed_controller linear_speed 0.0
ros2 param set /speed_controller angular_speed 0.0
```

测试超限参数

线速度超限

```
ros2 param set /speed_controller linear_speed 3.0
```

负线速度超限

```
ros2 param set /speed_controller linear_speed -1.5
```

角速度超限

```
ros2 param set /speed_controller angular_speed 3.0
```

负方向超限：

```
ros2 param set /speed_controller angular_speed -2.5
```

测试边界值

```
ros2 param set /speed_controller linear_speed 1.0
ros2 param set /speed_controller linear_speed -1.0
ros2 param set /speed_controller angular_speed 2.0
ros2 param set /speed_controller angular_speed -2.0
```

完成测试后立即停止：

```
ros2 param set /speed_controller linear_speed 0.0
ros2 param set /speed_controller angular_speed 0.0
```

/cmd_vel 数据

在终端 3 中执行：

```
ros2 topic echo /cmd_vel
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic echo /cmd_vel
linear:
  x: -0.3
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0
---
linear:
  x: -0.3
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0
---
```

检查发布频率:

```
ros2 topic hz /cmd_vel
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic hz /cmd_vel
average rate: 12.954
  min: 0.000s max: 0.102s std dev: 0.04170s window: 13
average rate: 11.399
  min: 0.000s max: 0.102s std dev: 0.03776s window: 25
average rate: 10.727
  min: 0.000s max: 0.102s std dev: 0.02470s window: 35
average rate: 10.501
  min: 0.000s max: 0.102s std dev: 0.02279s window: 45
average rate: 10.476
  min: 0.000s max: 0.102s std dev: 0.02339s window: 55
```

查看详细通信关系:

```
ros2 topic info /cmd_vel --verbose
```

```
Node namespace: /
Topic type: geometry_msgs/msg/Twist
Endpoint type: PUBLISHER
GID: 01.0f.72.95.3c.5a.4d.42.00.00.00.00.00.11.03.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: twist_mux
Node namespace: /
Topic type: geometry_msgs/msg/Twist
Endpoint type: PUBLISHER
GID: 01.0f.72.95.59.59.ec.c9.00.00.00.00.00.15.03.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 1

Node name: gazebo_interface_bridge
Node namespace: /
Topic type: geometry_msgs/msg/Twist
Endpoint type: SUBSCRIPTION
GID: 01.0f.72.95.57.59.15.b9.00.00.00.00.00.15.04.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
```

实际运行证据

真实运行的参数列表、合法参数更新和非法边界值拒绝输出：

```
ch06 parameter demo
Captured terminal session
$ ros2 run param_demo param_node
$ ros2 param list /param_demo
enable_debug
max_speed
mode
robot_name
start_type_description_service
use_sim_time
$ ros2 param set /param_demo max_speed 3.5
Set parameter successfully
$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
# param_node process output
[INFO] [1767891830.201962150]: [param_demo]: Robot: xbot | Speed: 2.0m/s | Mode: auto | Debug: False
[INFO] [1767891831.195777417]: [param_demo]: Robot: xbot | Speed: 2.0m/s | Mode: auto | Debug: False
[INFO] [1767891832.195399555]: [param_demo]: Robot: xbot | Speed: 3.5m/s | Mode: auto | Debug: False
Source: ros2demo_recording at the rosidemos_demo_demo
```

原始录制：[ch06_parameters.cast](#)。完整证据索引见[实际运行证据](#)。

第 7 章：TF2 坐标变换系统

***课程**： ROS2 Python 编程 **章节**： 第 7 章 **课时**： 2 课时 (90 分钟)*

7.1 坐标系树 (Frame Tree) 设计原理

知识点 7.1.1：TF 坐标系树

```
# TF2 将坐标系组织为树状结构，确保无环单向依赖
# 典型 XBot-U 坐标系树：
#   map → odom → base_footprint → base_link
#                                   | laser_frame
#                                   | camera_frame
#                                   | imu_link
#
```

TF2 要求坐标系之间形成有向无环图 (DAG)，各坐标系只能有一个父系，保证变换查找唯一路径。

知识点 7.1.2：TF2 核心 API 模块

```
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
from tf2_ros.transform_broadcaster import TransformBroadcaster
from tf2_ros.static_transform_broadcaster import StaticTransformBroadcaster
```

7.2 静态变换与动态变换

知识点 7.2.1：静态变换广播器

```
import rclpy
from rclpy.node import Node
from tf2_ros import StaticTransformBroadcaster
from geometry_msgs.msg import TransformStamped

class StaticTFPublisher(Node):
    def __init__(self):
        super().__init__('static_tf_pub')
        self.broadcaster = StaticTransformBroadcaster(self)

        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'base_link'
        t.child_frame_id = 'laser_frame'
        t.transform.translation.x = 0.2
        t.transform.translation.z = 0.1
```



```
t.transform.rotation.w = 1.0

self.broadcaster.sendTransform(t)
```

程序 7-1：静态变换只发送一次，适用于固定不变的传感器安装位置。

知识点 7.2.2：动态变换广播器

```
class DynamicTFPublisher(Node):
    def __init__(self):
        super().__init__('dynamic_tf_pub')
        self.broadcaster = TransformBroadcaster(self)
        self.timer = self.create_timer(0.1, self.publish_tf)

    def publish_tf(self):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'
        t.transform.translation.x = math.sin(self.count * 0.1)
        t.transform.translation.y = math.cos(self.count * 0.1)
        t.transform.rotation.w = 1.0
        self.broadcaster.sendTransform(t)
```

程序 7-2：动态变换持续广播（如 odom→base_link），100Hz 更新。

7.3 tf2_ros.Buffer 与 lookup_transform()

知识点 7.3.1：监听 TF 变换

```
from tf2_ros import Buffer, TransformListener

class TFListener(Node):
    def __init__(self):
        super().__init__('tf_listener')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(1.0, self.lookup_tf)

    def lookup_tf(self):
        try:
            now = rclpy.time.Time()
            transform = self.tf_buffer.lookup_transform(
                'base_link',          # 目标坐标系
                'laser_frame',        # 源坐标系
                now,                   # 时间戳
                timeout=rclpy.duration.Duration(seconds=1.0))

            t = transform.transform.translation
            self.get_logger().info(
                f'Laser→Base: x={t.x:.3f}, y={t.y:.3f}, z={t.z:.3f}')
```

```
except Exception as e:
    self.get_logger().warn(f'TF 查询失败: {e}')
```

程序 7-3: `lookup_transform(target, source, time)` 查询两个坐标系之间的变换关系。

知识点 7.3.2 : TF 坐标点变换

```
from tf2_geometry_msgs import do_transform_point
from geometry_msgs.msg import PointStamped

# 将激光雷达坐标系下的点转换到 base_link
point_in_laser = PointStamped()
point_in_laser.header.frame_id = 'laser_frame'
point_in_laser.point.x = 1.0

transform = self.tf_buffer.lookup_transform(
    'base_link', 'laser_frame', rclpy.time.Time())
point_in_base = do_transform_point(point_in_laser, transform)
```

7.4 时间同步与插值

知识点 7.4.1 : `waitForTransform` 与时间同步

```
def lookup_tf_with_wait(self):
    # 等待直到指定变换可用
    if self.tf_buffer.can_transform(
        'base_link', 'laser_frame', rclpy.time.Time(),
        timeout=rclpy.duration.Duration(seconds=2.0)):

        transform = self.tf_buffer.lookup_transform(
            'base_link', 'laser_frame', rclpy.time.Time())

    # 或使用 waitForTransform (已废弃, 建议用 can_transform 的 timeout)
```

程序 7-4: `can_transform` 等待变换就绪, 避免因时序问题导致查询失败。

知识点 7.4.2 : TF 插值机制

TF2 自动在两帧变换之间线性插值。调用 `lookup_transform` 时指定 `time=Time()` (最新) 或指定历史时间戳获取该时刻的变换 (需有足够缓存数据)。

```
# 获取最新变换
t = self.tf_buffer.lookup_transform('base_link', 'laser', rclpy.time.Time())

# 获取 2 秒前的变换 (如果 buffer 缓存足够)
past = rclpy.time.Time(seconds=2.0)
t = self.tf_buffer.lookup_transform('base_link', 'laser', past)
```

7.5 TF2 调试工具

知识点 7.5.1 : 命令行工具

```
# 实时查看两坐标系变换
ros2 run tf2_ros tf2_echo base_link laser_frame

# 监视所有 TF 变换
ros2 run tf2_ros tf2_monitor

# 生成坐标系树 PDF
ros2 run tf2_tools view_frames
# 输出 frames.pdf 可用 evince/draw.io 打开

# 静态变换发布
ros2 run tf2_ros static_transform_publisher \
  --x 0.2 --z 0.1 --yaw 0.0 \
  --frame-id base_link --child-frame-id laser_frame
```

7.6 本章小结

- 103. TF2 坐标系统呈树状结构 (DAG), 每子系只有一个父系
- 104. StaticTransformBroadcaster 发送固定变换, TransformBroadcaster 发送动态变换
- 105. Buffer.lookup_transform(target, source, time) 查询变换关系
- 106. can_transform + timeout 实现时间同步等待
- 107. TF2 自动插值支持历史时间戳查询
- 108. tf2_echo / tf2_monitor / view_frames 三大调试工具

7.7 练习题

练习 7.1: 编写静态 TF 广播器, 发布 base_link → laser_frame 的固定变换 (x=0.3, z=0.15) 。

练习 7.2: 编写动态 TF 广播器, 以 10Hz 频率发布 odom → base_link 的圆形运动轨迹变换。

练习 7.3: 编写 TF 监听器, 每秒查询 laser_frame 在 base_link 中的位姿并输出。

练习 7.4: 在监听器中实现点坐标变换: 将 laser_frame 下的 (1.0, 0.0, 0.0) 点转换到 base_link 系下。

练习 7.5: 使用 can_transform 实现带超时的安全查询, 超时后重试最多 3 次。

练习 7.6: 使用 tf2_echo, tf2_monitor, view_frames 调试完整的 TF 树, 导出 frames.pdf。

实验 7:

第 7 章 实验指导书：TF2 坐标变换系统

****实验课时****: 2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

- 109. 掌握 TF2 广播与监听的核心 API
- 110. 实现多传感器坐标系对齐
- 111. 熟练使用 TF2 调试工具

练习 7.1：TF 广播和监听基础（约 30 分钟）

任务

- 112. 编写 `tf_broadcaster.py`: 以 20Hz 频率发布 `odom` $\rightarrow$ `base_link` 动态变换（圆形轨迹运动），同时发布 `base_link` $\rightarrow$ `laser_frame` 静态变换。
- 113. 编写 `tf_listener.py`: 监听并每秒输出 `laser_frame` 相对于 `base_link` 的位姿。

步骤

- 114. 创建包 `tf_demo`, 参考 `lab_code/ch07_lab/tf_demo/`
- 115. 运行广播器: `ros2 run tf_demo tf_broadcaster`
- 116. 运行监听器: `ros2 run tf_demo tf_listener`
- 117. 验证: `ros2 run tf2_ros tf2_echo base_link laser_frame`

参考代码

`tf_broadcaster.py` 核心片段:

```

import math
from geometry_msgs.msg import TransformStamped
from tf2_ros import TransformBroadcaster, StaticTransformBroadcaster

class TFBroadcaster(Node):
    def __init__(self):
        super().__init__('tf_broadcaster')
        self.dynamic_broadcaster = TransformBroadcaster(self)
        self.static_broadcaster = StaticTransformBroadcaster(self)

        # 发送静态变换 (base_link → laser_frame)
        static_tf = TransformStamped()
        static_tf.header.frame_id = 'base_link'
        static_tf.child_frame_id = 'laser_frame'
        static_tf.transform.translation.x = 0.2
        static_tf.transform.translation.z = 0.1
        static_tf.transform.rotation.w = 1.0
        self.static_broadcaster.sendTransform(static_tf)

        self.timer = self.create_timer(0.05, self.publish_dynamic_tf)
        self.angle = 0.0

    def publish_dynamic_tf(self):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'
        t.transform.translation.x = 1.0 * math.cos(self.angle)
        t.transform.translation.y = 1.0 * math.sin(self.angle)
        t.transform.rotation.z = math.sin(self.angle / 2)
        t.transform.rotation.w = math.cos(self.angle / 2)
        self.dynamic_broadcaster.sendTransform(t)
        self.angle += 0.05

```

练习 7.2：多传感器坐标对齐（约 30 分钟）

任务

为 XBot-U 扩展多传感器 TF 树：在 base_link 下添加 camera_frame、imu_link、left_wheel 和 right_wheel 四个子系，实现激光雷达点到相机坐标系的对齐变换。

步骤

118. 在 tf_broadcaster.py 中扩展静态变换广播：
119. base_link → camera_frame: (x=0.15, z=0.25)
120. base_link → imu_link: (x=0.0, z=0.05)
121. base_link → left_wheel: (y=0.15, z=-0.05)
122. base_link → right_wheel: (y=-0.15, z=-0.05)

123. 在 `tf_listener.py` 中实现激光→相机坐标转换：

```
from tf2_geometry_msgs import do_transform_point
from geometry_msgs.msg import PointStamped

# 激光点 (1.0, 0.5, 0.0) → 相机坐标系
pt = PointStamped()
pt.header.frame_id = 'laser_frame'
pt.header.stamp = self.get_clock().now().to_msg()
pt.point.x = 1.0; pt.point.y = 0.5

laser_to_camera = self.tf_buffer.lookup_transform(
    'camera_frame', 'laser_frame', rclpy.time.Time())
pt_in_camera = do_transform_point(pt, laser_to_camera)
```

124. 运行 `view_frames` 验证完整的 TF 树结构。

练习 7.3 : TF 调试工具使用 (约 30 分钟)

任务

使用 TF2 三大调试工具诊断和验证坐标系系统。

步骤

125. **tf2_echo**: 实时查看任意两坐标系间变换

```
ros2 run tf2_ros tf2_echo base_link laser_frame
ros2 run tf2_ros tf2_echo odom base_link
ros2 run tf2_ros tf2_echo camera_frame laser_frame
```

126. **tf2_monitor**: 监视所有 TF 发布者，查看帧率和延迟

```
ros2 run tf2_ros tf2_monitor
```

127. **view_frames**: 生成 PDF 坐标树并分析

```
ros2 run tf2_tools view_frames
# 查看生成的 frames.pdf
```

思考题

128. 如何判断 TF 树中是否存在环路？

129. 如果 `lookup_transform` 频繁报 `ExtrapolationException`，说明什么？如何修复？

130. 为什么要使用 `can_transform` 而不是直接 `lookup_transform`？

练习 4 : TF 仿真查询 — 查询 XBot-U 传感器坐标系（约 15 分钟）

目标

启动仿真后，使用 TF2 查询 LiDAR 和 Camera 相对于 base_link 的位置，计算传感器安装偏移。

步骤

步骤 1: 启动仿真

```
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
```

步骤 2: 编写 tf_lookup.py

```
#!/usr/bin/env python3
"""tf_lookup: 查询 XBot-U 各坐标系相对位置"""
import rclpy
from rclpy.node import Node
from tf2_ros import Buffer, TransformListener

class TfLookup(Node):
    def __init__(self):
        super().__init__('tf_lookup')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(2.0, self.lookup_tf)

    def lookup_tf(self):
        try:
            # 查询 laser → base_link 变换
            t = self.tf_buffer.lookup_transform(
                'base_link', 'laser', rclpy.time.Time()
            )
            self.get_logger().info(
                f'LiDAR 位置: x={t.transform.translation.x:.3f}, '
                f'y={t.transform.translation.y:.3f}, '
                f'z={t.transform.translation.z:.3f}')
        except Exception as e:
            self.get_logger().warn(f'TF 查询失败: {e}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(TfLookup())
    rclpy.shutdown()
```

步骤 3: 运行


```
ros2 run tf_demo tf_lookup
```

✓ 验证：终端输出 LiDAR 和 Camera 相对 base_link 的位置坐标。用 `ros2 run tf2_tools view_frames` 查看完整 TF 树。

思考题

131. TF 树中 base_footprint 和 base_link 的区别是什么？
132. 如果 TF 查询超时，如何处理？

实际运行证据

真实运行的 TF broadcaster、listener 和 tf2_echo 输出：

```
ch07 tf_demo
tf2_echo process output
- Rotations in Quaternion (type) [0.000, 0.000, 0.000, 1.000]
- Rotations in RPY (radian) [0.000, -0.000, 0.000]
- Rotations in RPY (degree) [0.000, -0.000, 0.000]
- Matrix:
  1.000 0.000 0.000 0.200
  0.000 1.000 0.000 0.000
  0.000 0.000 1.000 0.100
  0.000 0.000 0.000 1.000
At time 0.0
- Translation: [0.200, 0.000, 0.100]
- Rotations in Quaternion (type) [0.000, 0.000, 0.000, 1.000]
- Rotations in RPY (radian) [0.000, -0.000, 0.000]
- Rotations in RPY (degree) [0.000, -0.000, 0.000]
- Matrix:
  1.000 0.000 0.000 0.200
  0.000 1.000 0.000 0.000
  0.000 0.000 1.000 0.100
  0.000 0.000 0.000 1.000
[INFO] [1787891832.545019950] [tf_broadcaster]: 已发送 5 个静态 TF 变换

# tf listener process output
[INFO] [1787891833.943198177] [tf_listener]: Laser-Basel pose=[0.200, 0.000, 0.100] quat=[0.00, 0.00, 0.00, 1.00]
[INFO] [1787891833.943404794] [tf_listener]: 激光计 (1.0,0.5) -> 相机 (1.000, 0.500, -0.150)
[INFO] [1787891834.943209910] [tf_listener]: Laser-Basel pose=[0.200, 0.000, 0.100] quat=[0.00, 0.00, 0.00, 1.00]
[INFO] [1787891834.944035197] [tf_listener]: 激光计 (1.0,0.5) -> 相机 (1.050, 0.500, -0.150)
[INFO] [1787891835.943171180] [tf_listener]: Laser-Basel pose=[0.200, 0.000, 0.100] quat=[0.00, 0.00, 0.00, 1.00]
[INFO] [1787891835.944777255] [tf_listener]: 激光计 (1.0,0.5) -> 相机 (1.050, 0.500, -0.150)
[INFO] [1787891836.943555555] [tf_listener]: Laser-Basel pose=[0.200, 0.000, 0.100] quat=[0.00, 0.00, 0.00, 1.00]
[INFO] [1787891836.944662477] [tf_listener]: 激光计 (1.0,0.5) -> 相机 (1.050, 0.500, -0.150)
Source: see below recording of the console when above
```

原始录制：[ch07_tf.cast](#)。完整证据索引见[实际运行证据](#)。

第 8 章：URDF/Xacro 机器人建模

***课程**： ROS2 Python 编程 **章节**： 第 8 章 **课时**： 2 课时 (90 分钟)*

8.1 URDF 语法基础

知识点 8.1.1：URDF 概述与文件结构

```
<?xml version="1.0"?>
<robot name="my_robot">
  <!-- 连杆定义 -->
  <link name="base_link">
    <visual> ... </visual>
    <collision> ... </collision>
    <inertial> ... </inertial>
  </link>

  <!-- 关节定义 -->
  <joint name="base_to_wheel" type="continuous">
    <parent link="base_link"/>
    <child link="wheel"/>
    <origin xyz="0 0 -0.1" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>
</robot>
```

程序 8-1：URDF 文件最小结构。每个 <link> 描述刚体，每个 <joint> 描述连接关系。

知识点 8.1.2：link 元素详解

子元素	用途	示例
`<visual>`	可视化几何体 (mesh/box/cylinder/sphere)	`<cylinder radius="0.1" length="0.2"/>`
`<collision>`	碰撞检测几何体 (通常与 visual 一致)	`<box size="0.4 0.3 0.15"/>`
`<inertial>`	惯性参数 (mass + inertia)	`<mass value="1.0"/>`

```
<link name="base_link">
  <visual>
    <geometry><box size="0.4 0.3 0.15"/></geometry>
    <material name="blue"><color rgba="0 0 0.8 1"/></material>
  </visual>
  <collision>
```

```

    <geometry><box size="0.4 0.3 0.15"/></geometry>
  </collision>
  <inertial>
    <mass value="5.0"/>
    <inertia ixx="0.1" ixy="0" ixz="0" iyy="0.1" iyz="0" izz="0.1"/>
  </inertial>
</link>

```

知识点 8.1.3 : joint 类型

类型	运动自由度	典型应用
`revolute`	绕轴旋转 (有限范围)	机械臂关节
`continuous`	无限旋转	轮子
`prismatic`	沿轴平移	升降机构
`fixed`	无自由度	传感器固定连接
`planar`	平面运动	地面移动

8.2 XACRO 宏与参数化

知识点 8.2.1 : XACRO 基础宏

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="xbot">

  <!-- 参数化属性 -->
  <xacro:property name="wheel_radius" value="0.05"/>
  <xacro:property name="wheel_width" value="0.03"/>

  <!-- 宏定义 -->
  <xacro:macro name="wheel" params="name prefix reflect">
    <joint name="${name}_joint" type="continuous">
      <parent link="base_link"/>
      <child link="${prefix}_wheel"/>
      <origin xyz="0 ${reflect*wheel_base/2} -0.1" rpy="0 0 0"/>
      <axis xyz="0 1 0"/>
    </joint>
    <link name="${prefix}_wheel">
      <visual>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
        </geometry>
      </visual>
    </link>
  </xacro:macro>

```

```

<!-- 宏实例化 -->
<xacro:wheel name="left" prefix="left" reflect="1"/>
<xacro:wheel name="right" prefix="right" reflect="-1"/>
</robot>

```

程序 8-2: XACRO 宏减少重复代码, 参数化实现车轮批量生成。

知识点 8.2.2 : mesh 文件引用

```

<link name="lidar_link">
  <visual>
    <geometry>
      <mesh filename="package://robot_description/meshes/lidar.stl"/>
    </geometry>
    <origin xyz="0 0 0.02" rpy="0 0 0"/>
  </visual>
</link>

```

Mesh 文件通常放在 meshes/ 目录, 使用 package://<pkg_name>/ 路径前缀。

8.3 robot_state_publisher + joint_state_publisher

知识点 8.3.1 : TF 自动发布机器人状态

```

# launch/display.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    urdf_path = os.path.join(
        get_package_share_directory('urdf_demo'),
        'urdf', 'simple_robot.xacro')

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': Command(['xacro ', urdf_path])}],
        ),
        Node(
            package='joint_state_publisher',
            executable='joint_state_publisher',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            arguments=['-d', os.path.join(
                get_package_share_directory('urdf_demo'),
                'rviz', 'display.rviz')],
        ),
    ])

```

程序 8-3: robot_state_publisher 解析 URDF 并发布 TF, joint_state_publisher 提供关节状态 GUI。

8.4 RViz2 RobotModel 显示

知识点 8.4.1 : RViz2 配置

- 133. 添加 RobotModel 插件, 设置 Description Topic: /robot_description
 - 134. 添加 TF 插件查看坐标系
 - 135. 设置 Fixed Frame 为 base_link 或 odom
 - 136. 保存配置为 .rviz 文件供 Launch 加载
-

8.5 本章小结

- 137. URDF = link (刚体) + joint (连接), 包含 visual/collision/inertial
 - 138. joint 类型: revolute / continuous / prismatic / fixed
 - 139. XACRO = 宏 + 参数变量, 消除重复, 支持条件分支
 - 140. robot_state_publisher 解析 URDF → 发布 TF
 - 141. joint_state_publisher 提供关节值 GUI 滑块
 - 142. RViz2 RobotModel 插件可视化完整机器人模型
-

8.6 练习题

练习 8.1: 编写一个包含 base_link 和 lidar_link 的 URDF, 通过 fixed joint 连接。

练习 8.2: 为 base_link 添加 collision 和 inertial 元素 (mass=10kg, box inertia) 。

练习 8.3: 使用 XACRO macro 参数化生成 4 个麦克纳姆轮 (前左/前右/后左/后右) 。

练习 8.4: 编写 `display.launch.py`, 启动 `robot_state_publisher` + `joint_state_publisher` + `rviz2`。

练习 8.5: 在 RViz2 中调出 RobotModel 和 TF 显示, 截图完整的机器人模型。

练习 8.6: 使用 XACRO `<xacro:if>` 条件判断: 当 `use_lidar=true` 时才加载 `lidar_link`。

实验 8:

第 8 章 实验指导书：URDF/Xacro 机器人建模

****实验课时****: 2 课时 (90 分钟) / XBot-U 模型

实验目标

143. 编写基础 URDF 描述机器人结构
144. 使用 XACRO 宏实现参数化建模
145. RViz2 可视化完整机器人模型

练习 8.1：基础 URDF 编写（约 30 分钟）

任务

编写一个简单差速机器人 URDF，包含：

- base_link: 底盘长方体 (0.4×0.3×0.15), 蓝色, mass=5kg
- left_wheel_link / right_wheel_link: 两个轮子 (cylinder, r=0.05, l=0.03)
- left_wheel_joint / right_wheel_joint: continuous 类型
- caster_wheel_link: 万向轮 (sphere, r=0.02)

步骤

146. 在 lab_code/ch08_lab/urdf_demo/urdf/ 下创建 simple_robot.urdf
147. 运行 xacro 解析验证: xacro urdf/simple_robot.urdf
148. 使用 check_urdf 工具验证语法: check_urdf <(xacro urdf/simple_robot.urdf)

提示

```
<geometry>
  <cylinder radius="0.05" length="0.03"/>
</geometry>
<!-- 圆柱体默认沿 Z 轴，需要 rpy 旋转到 Y 轴 -->
<origin xyz="0 0.15 0" rpy="1.5708 0 0"/>
```

练习 8.2 : XACRO 参数化建模 (约 30 分钟)

任务

将练习 8.1 的 URDF 改写为 XACRO 文件 `simple_robot.xacro`:

- 149. 定义 `<xacro:property>`: `wheel_radius`, `wheel_width`, `chassis_length`, `chassis_width`, `chassis_height`
- 150. 编写 `<xacro:macro name="wheel" params="name reflect">` 消除重复代码
- 151. 实例化左右轮: `<xacro:wheel name="left" reflect="1"/>`

步骤

- 152. 编写 `simple_robot.xacro`, 参考
`lab_code/ch08_lab/urdf_demo/urdf/simple_robot.xacro`
- 153. 运行 `ros2 launch urdf_demo display.launch.py` 启动 RViz2
- 154. 修改参数值 (如 `wheel_radius`) , 观察模型变化

练习 8.3 : RViz2 可视化机器人模型 (约 30 分钟)

任务

配置 RViz2 完整显示机器人模型。

步骤

- 155. 在 RViz2 中添加 `RobotModel` 插件
- 156. 设置 `Description Topic` 为 `/robot_description`
- 157. 添加 `TF` 插件显示坐标系

158. 将 Fixed Frame 设为 base_link
159. 运行 joint_state_publisher_gui, 拖动滑块测试关节运动
160. 保存 RViz 配置到 urdf_demo/rviz/display.rviz

思考题

161. robot_state_publisher 和 joint_state_publisher 的分工有何不同?
162. 如果 RViz 显示 "No transform from [left_wheel] to [base_link]", 可能是什么原因?
163. XACRO 宏中的 *origin 参数传递 block 时如何实现默认值?

练习 4 : XACRO 自定义标记 — 为 XBot-U 添加视觉标记 (约 15 分钟)

目标

修改 XBot-U 的 xacro 文件, 在 base_link 上方添加一个红色球体标记, 并在 RViz 中可视化。

步骤

步骤 1: 编辑 xacro 文件 在 base_link 的 xacro 定义中添加 marker link 和 joint:

```
<link name="custom_marker">
  <visual>
    <geometry>
      <sphere radius="0.03"/>
    </geometry>
    <material name="red_marker">
      <color rgba="1.0 0.0 0.0 1.0"/>
    </material>
  </visual>
</link>

<joint name="marker_joint" type="fixed">
  <parent link="base_link"/>
  <child link="custom_marker"/>
  <origin xyz="0 0 0.3" rpy="0 0 0"/>
</joint>
```

步骤 2: 启动 RViz 验证

```
ros2 launch urdf_demo display.launch.py
```

在 RViz 中添加 RobotModel, 确认红色球体出现在机器人顶部。

步骤 3: 修改 marker 位置和颜色 改变 `<origin xyz="..." />` 和 `<color rgba="..." />` 参数, 重新启动观察变化。

✓ **验证:** RViz 中 XBot-U 模型顶部显示自定义红色球体标记, 位置正确。

思考题

164. 如何在 xacro 宏中定义一个参数化的 marker, 支持不同颜色和大小?
165. `<visual>` 和 `<collision>` 有什么区别? 为什么 marker 可以省略碰撞?

第 9 章 : Gazebo 仿真

***课程**: ROS2 Python 编程 **章节**: 第 9 章 **课时**: 2 课时 (90 分钟)*

9.1 Gazebo 仿真环境概述

知识点 9.1.1 : Gazebo 版本选择

特性	Gazebo Classic (11)	Gazebo Sim (Ignition)
ROS2 支持	ros_gz_bridge	原生 ros_gz
发行版搭配	Humble + Classic	Iron+ 推荐 Ignition
稳定性	成熟稳定	积极开发中

本章以 Gazebo Classic + ROS2 Humble 为主进行讲解。

知识点 9.1.2 : URDF → SDF 转换桥

```
# Gazebo 内部使用 SDF 格式, ROS 2 自动通过 URDF→SDF 桥接
# 启动时在 launch 文件中指定 spawn 参数即可
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py use_gazebo:=true
```

Gazebo 通过 libgazebo_ros_init.so 插件加载机器人模型。

9.2 Spawn 机器人模型

知识点 9.2.1 : Gazebo 启动 Launch 文件

```
# launch/sim_bringup.launch.py
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
from launch.substitutions import LaunchConfiguration
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_path = get_package_share_directory('sim_demo')
    world_path = os.path.join(pkg_path, 'worlds', 'empty.world')
```

```

use_gazebo = LaunchConfiguration('use_gazebo', default='true')

return LaunchDescription([
    DeclareLaunchArgument('use_gazebo', default_value='true',
                           description='是否启动 Gazebo'),

    # 启动 Gazebo
    IncludeLaunchDescription(
        PythonLaunchDescriptionSource([
            get_package_share_directory('gazebo_ros'),
            '/launch/gazebo.launch.py'
        ]),
        launch_arguments={ 'world': world_path }.items(),
    ),

    # Spawn 机器人模型
    Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=[
            '-entity', 'xbot',
            '-topic', 'robot_description',
            '-x', '0.0', '-y', '0.0', '-z', '0.1',
        ],
        output='screen',
    ),
])

```

程序 9-1: 最小 Gazebo 启动文件, 通过 spawn_entity 加载机器人。

9.3 传感器插件配置

知识点 9.3.1 : LiDAR 传感器插件

```

<!-- 在 URDF/XACRO 中添加 Gazebo 插件 -->
<gazebo reference="lidar_link">
  <sensor type="ray" name="lidar_sensor">
    <pose>0 0 0 0 0 0</pose>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <resolution>1</resolution>
          <min_angle>-3.14159</min_angle>
          <max_angle>3.14159</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.12</min>
        <max>12.0</max>
        <resolution>0.01</resolution>
      </range>
    </ray>
  </gazebo>

```

```

<plugin name="lidar_plugin" filename="libgazebo_ros_ray_sensor.so">
  <ros>
    <namespace>/xbot</namespace>
    <argument>~/out:=scan</argument>
  </ros>
  <output_type>sensor_msgs/LaserScan</output_type>
</plugin>
</sensor>
</gazebo>

```

程序 9-2: LiDAR 插件发布 /scan 话题。

知识点 9.3.2 : Camera 传感器插件

```

<gazebo reference="camera_link">
  <sensor type="camera" name="camera_sensor">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image><width>640</width><height>480</height></image>
    </camera>
    <plugin name="camera_plugin" filename="libgazebo_ros_camera.so">
      <ros><namespace>/xbot</namespace></ros>
    </plugin>
  </sensor>
</gazebo>

```

知识点 9.3.3 : IMU 传感器插件

```

<gazebo reference="imu_link">
  <sensor type="imu" name="imu_sensor">
    <plugin name="imu_plugin" filename="libgazebo_ros_imu_sensor.so">
      <ros><namespace>/xbot</namespace></ros>
    </plugin>
  </sensor>
</gazebo>

```

9.4 ros2_control 差速驱动

知识点 9.4.1 : ros2_control 架构

硬件接口层 : GazeboSystem (模拟编码器+电机)

↓

控制器管理层 : diff_drive_controller (速度→左右轮速)

↓

ROS2 命令接口 : /cmd_vel → 速度指令

知识点 9.4.2 : Gazebo ros2_control 插件

```

<gazebo>
  <plugin name="diff_drive" filename="libgazebo_ros2_control.so">
    <ros><namespace>/xbot</namespace></ros>
  </plugin>
</gazebo>

```

```
</plugin>
</gazebo>
```

配套配置 YAML 文件定义控制器参数：

```
controller_manager:
  ros__parameters:
    update_rate: 50

diff_drive_controller:
  type: diff_drive_controller/DiffDriveController
  ros__parameters:
    left_wheel_names: ["left_wheel_joint"]
    right_wheel_names: ["right_wheel_joint"]
    wheel_separation: 0.3
    wheel_radius: 0.05
    cmd_vel_timeout: 1.0
    publish_odom: true
```

程序 9-3：diff_drive_controller 接收 /cmd_vel，计算左右轮速写入硬件接口。

9.5 本章小结

- 166. Gazebo Classic 通过 ros_gz_bridge 与 ROS2 通信
- 167. URDF → SDF 自动桥接，通过 spawn_entity.py 加载
- 168. LiDAR (gazebo_ros_ray_sensor) / Camera (gazebo_ros_camera) / IMU 插件
- 169. ros2_control diff_drive_controller 实现差速运动控制
- 170. 所有插件通过 YAML 参数文件配置

9.6 练习题

练习 9.1：编写 sim_bringup.launch.py，启动 Gazebo + 空世界 + spawn 机器人。

练习 9.2：在 URDF 中添加 LiDAR 的 <gazebo> 插件配置，发布 /scan 话题。

练习 9.3：在 URDF 中添加 Camera 插件，验证 /camera/image_raw 话题。

练习 9.4：编写 diff_drive_controller YAML 配置文件，配置 wheel_separation 和 wheel_radius。

练习 9.5: 发布 `/cmd_vel` 指令控制机器人运动: `ros2 topic pub /cmd_vel geometry_msgs/Twist "{linear: {x: 0.2}, angular: {z: 0.5}}"`

练习 9.6: 在 RViz2 中订阅 `/scan` 和 `/camera/image_raw`, 观察传感器数据。

实验 9:

第 9 章 实验指导书 : Gazebo 仿真

****实验课时****: 2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

- 171. 搭建 Gazebo 仿真环境并 spawn 机器人模型
- 172. 配置 LiDAR + Camera 传感器插件
- 173. 使用 ros2_control 差速驱动控制机器人运动

练习 9.1 : Gazebo 环境搭建与 Spawn 机器人 (约 30 分钟)

任务

编写 `sim_bringup.launch.py`, 启动 Gazebo + 空世界 + spawn XBot-U 模型。

步骤

- 174. 创建包 `sim_demo`, 参考 `lab_code/ch09_lab/sim_demo/`
- 175. 编写 Launch 文件启动 Gazebo:

```
IncludeLaunchDescription(  
  PythonLaunchDescriptionSource([  
    get_package_share_directory('gazebo_ros'),  
    '/launch/gazebo.launch.py'  
  ]),  
)
```

- 176. 添加 `spawn_entity` 节点加载 `robot_description`
- 177. 运行: `ros2 launch sim_demo sim_bringup.launch.py`
- 178. 使用 `gz topic -l` 验证模型已加载

验证

```
gz topic -l          # 列出 Gazebo 话题
ros2 topic list       # 列出 ROS2 话题
ros2 run gazebo_ros gazebo_ros_state # 查看模型状态
```

练习 9.2：传感器插件配置（约 30 分钟）

任务

在 XBot-U 的 URDF/XACRO 中添加 LiDAR 和 Camera 的 Gazebo 传感器插件。

步骤

179. 在 URDF <robot> 标签内添加 <gazebo> 元素：

LiDAR 插件：

```
<gazebo reference="lidar_link">
  <sensor type="ray" name="lidar">
    <pose>0 0 0 0 0 0</pose>
    <visualize>true</visualize>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <min_angle>-1.570796</min_angle>
          <max_angle>1.570796</max_angle>
        </horizontal>
      </scan>
      <range><min>0.12</min><max>12.0</max></range>
    </ray>
    <plugin name="lidar_plugin" filename="libgazebo_ros_ray_sensor.so">
      <ros><namespace>/xbot</namespace></ros>
      <output_type>sensor_msgs/LaserScan</output_type>
    </plugin>
  </sensor>
</gazebo>
```

Camera 插件：

```
<gazebo reference="camera_link">
  <sensor type="camera" name="camera">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image><width>640</width><height>480</height></image>
      <clip><near>0.05</near><far>8.0</far></clip>
    </camera>
    <plugin name="camera_plugin" filename="libgazebo_ros_camera.so">
      <ros><namespace>/xbot</namespace></ros>
    </plugin>
  </sensor>
</gazebo>
```

```
</plugin>
</sensor>
</gazebo>
```

180. 重新启动仿真，验证话题：

```
ros2 topic echo /xbot/scan
ros2 run rqt_image_view rqt_image_view /xbot/image_raw
```

练习 9.3 : ros2_control 差速驱动控制（约 30 分钟）

任务

配置 `diff_drive_controller` 并通过 `/cmd_vel` 控制机器人运动。

步骤

181. 在 URDF 中添加 `ros2_control` Gazebo 插件：

```
<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <ros><namespace>/xbot</namespace></ros>
  </plugin>
</gazebo>
```

182. 编写控制器配置 YAML (`config/diff_drive.yaml`)：

```
controller_manager:
  ros__parameters:
    update_rate: 50
    use_sim_time: true

    diff_drive_controller:
      type: diff_drive_controller/DiffDriveController

diff_drive_controller:
  ros__parameters:
    left_wheel_names: ["left_wheel_joint"]
    right_wheel_names: ["right_wheel_joint"]
    wheel_separation: 0.3
    wheel_radius: 0.05
    cmd_vel_timeout: 1.0
    publish_odom: true
    odom_frame_id: odom
    base_frame_id: base_link
```

183. 启动 `controller_manager` 并加载控制器：

```
ros2 run controller_manager spawner diff_drive_controller
```

184. 发布速度指令控制运动：

```
# 前进
ros2 topic pub /diff_drive_controller/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.3}, angular: {z: 0.0}}" -1

# 左转
ros2 topic pub /diff_drive_controller/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0}, angular: {z: 0.8}}" -1
```

思考题

185. diff_drive_controller 如何将 cmd_vel 的线速度和角速度分解为左右轮速？
186. 为什么需要 cmd_vel_timeout 参数？如果设为 0 会怎样？
187. publish_odom: true 产生的话题和 TF 分别是什么？

练习 4 : Gazebo 传感器数据订阅 — 处理 /scan 和 /camera (约 15 分钟)

目标

订阅仿真中的 LiDAR 和 Camera 数据，处理并输出传感器数据摘要。

步骤

步骤 1: 启动仿真

```
ros2 launch sim_demo sim_bringup.launch.py
```

步骤 2: 编写 sensor_reader.py

```
#!/usr/bin/env python3
"""sensor_reader: 订阅 /scan 和 /camera 并输出摘要"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan, Image

class SensorReader(Node):
    def __init__(self):
        super().__init__('sensor_reader')
        self.scan_sub = self.create_subscription(
            LaserScan, '/xbot/scan', self.scan_cb, 10)
        self.img_sub = self.create_subscription(
            Image, '/xbot/image_raw', self.img_cb, 10)

    def scan_cb(self, msg: LaserScan):
        ranges = [r for r in msg.ranges if msg.range_min < r < msg.range_max]
        if ranges:
            self.get_logger().info(
                f'LiDAR: {msg.angle_max - msg.angle_min:.2f}rad, '
```

```

        f'{len(msg.ranges)} 点, '
        f'最近: {min(ranges):.2f}m, 最远: {max(ranges):.2f}m')

def img_cb(self, msg: Image):
    self.get_logger().info(
        f'Camera: {msg.width}x{msg.height}, '
        f'编码: {msg.encoding}', throttle_duration_sec=2.0)

def main():
    rclpy.init()
    rclpy.spin(SensorReader())
    rclpy.shutdown()

```

步骤 3: 运行

```
ros2 run sim_demo sensor_reader
```

✓ **验证:** 终端持续输出 LiDAR 扫描角度、点数和距离范围, 以及 Camera 图像尺寸。

思考题

188. LaserScan 消息中 ranges 数组的长度由哪个字段决定?

189. 如何判断 LiDAR 是否被遮挡 (大量 NaN 值) ?

实际运行证据

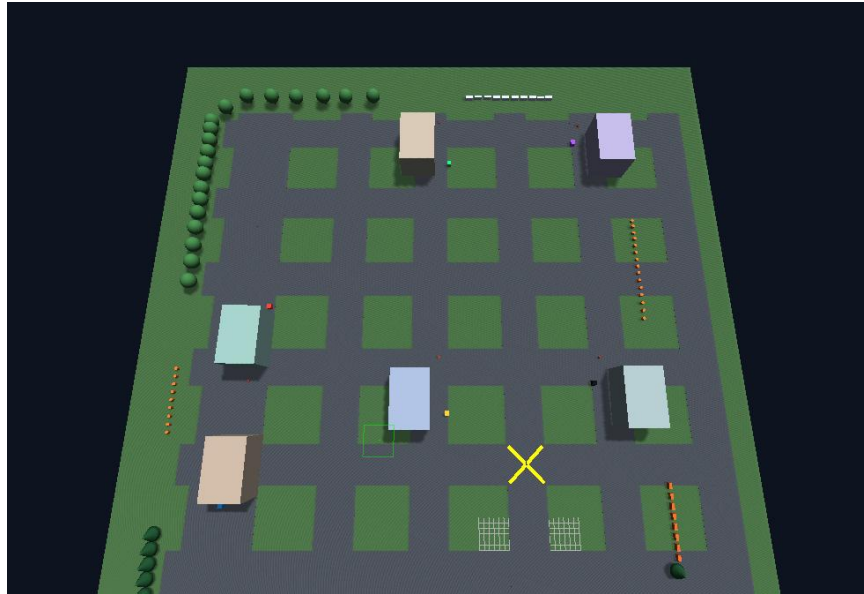
Gazebo headless 会话真实输出了 /clock、/scan、/odom 和实体创建结果:

```

ch09-gazebo-harmonic
Captured terminal session
$ ros2 launch sim_demo sim_bringup.launch.py gui:=false rviz:=false drive:=false gz_partition:=runtime_capture_ch09
$ ros2 topic echo /clock --once --no-lost-messages
clock:
  sec: 286
  nanosec: 121000000
---
$ ros2 topic info /scan
Type: sensor_msgs/msg/LaserScan
Publisher count: 1
Subscription count: 0
$ ros2 topic echo /scan --once --field header --no-lost-messages
stamp:
  sec: 151
  nanosec: 625000000
frame_id: laser_link
---
$ ros2 topic echo /odom --once --field pose.pose --no-lost-messages
position:
  x: -2.523473846739742e-18
  y: -4.47503763558946e-35
  z: 0.0
orientation:
  x: 0.0
  y: 0.0
  z: 1.7003945535765475e-17
  w: 1.0
---
# simulation process output
[robot_state_publisher-5] [INFO] [1787892336.744803216] [gazebo2_robot_state_publisher]: Robot initialized
[parameter_bridge-2] [INFO] [1787892337.797406067] [gazebo2_bridge]: Creating GZ->ROS Bridge: [/clock (gz.msgs.Clock)
/clock (rosgraph_msgs/msg/Clock)] (Lazy 0)
[parameter_bridge-2] [INFO] [1787892337.800526004] [gazebo2_bridge]: Creating GZ->ROS Bridge: [/scan (gz.msgs.LaserScan)
-> /scan (sensor_msgs/msg/LaserScan)] (Lazy 0)
[parameter_bridge-2] [INFO] [1787892337.809300498] [gazebo2_bridge]: Creating GZ->ROS Bridge: [/odom (gz.msgs.Odometry)
-> /odom (nav_msgs/msg/Odometry)] (Lazy 0)
[parameter_bridge-2] [INFO] [1787892337.816828452] [gazebo2_bridge]: Creating GZ->ROS Bridge: [/tf (gz.msgs.Pose_V) ->
tf (tf2_msgs/msg/TFMessage)] (Lazy 0)
[create-6] [INFO] [1787892340.285381133] [ros_gz_sim]: Entity creation successful.
Source: asciinema recording of the command shown above.

```

Campus PUCRS 场景由 Gazebo GUI Screenshot 插件生成：



原始终端录制：[ch09_gazebo_headless.cast](#) · [Campus headless](#)。完整证据索引见[实际运行证据](#)。

Part 2: SLAM 与自主导航（智能网联汽车环境感知与决策）

第 10 章 SLAM 建图与定位

10.1 知识要点

10.1.1 SLAM 基本原理

SLAM (Simultaneous Localization and Mapping) 是机器人同时进行自身定位和环境地图构建的核心技术。

贝叶斯滤波框架:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t-1})$$

其中 x 为位姿, m 为地图, z 为观测, u 为控制输入。

10.1.2 GraphSLAM 图优化

GraphSLAM 将 SLAM 问题建模为图优化问题:

- **节点 (Vertex):** 机器人位姿 x_t 和路标点 l_j
- **边 (Edge):** 里程计约束和观测约束
- **优化目标:** 最小化所有约束误差

```
# GraphSLAM 图构建伪代码
def build_graph(poses, odometry, observations):
    graph = g2o.SparseOptimizer()
    # 添加位姿顶点
    for i, pose in enumerate(poses):
        v = g2o.VertexSE2()
        v.set_estimate(pose)
        graph.add_vertex(v)
    # 添加里程计边 (连续位姿间约束)
    for i in range(len(poses) - 1):
        edge = g2o.EdgeSE2()
        edge.set_measurement(odometry[i])
        edge.set_vertex(0, graph.vertex(i))
        edge.set_vertex(1, graph.vertex(i + 1))
        edge.set_information(np.eye(3) * info_odom)
        graph.add_edge(edge)
    # 添加观测边 (位姿-路标点约束)
    for obs in observations:
        edge = g2o.EdgeSE2PointXY()
        edge.set_measurement(obs.measurement)
        graph.add_edge(edge)
    graph.optimize()
    return graph
```

10.1.3 粒子滤波 SLAM (FastSLAM)

FastSLAM 使用 Rao-Blackwellized 粒子滤波，将 SLAM 分解为：

- 粒子滤波器估计机器人路径
- 每个粒子维护独立的 EKF 地图

```
import numpy as np

class FastSLAMParticle:
    """FastSLAM 单个粒子实现"""
    def __init__(self, num_landmarks: int):
        self.pose = np.zeros(3)          # x, y, theta
        self.weight = 1.0
        # 每个路标点的 EKF: mean[2], cov[2,2]
        self.landmarks_mean = np.zeros((num_landmarks, 2))
        self.landmarks_cov = np.zeros((num_landmarks, 2, 2))
        self.landmarks_seen = [False] * num_landmarks

    def predict(self, control: np.ndarray, noise: np.ndarray):
        """运动模型预测"""
        self.pose[0] += control[0] * np.cos(self.pose[2]) + np.random.normal(0, noise[0])
        self.pose[1] += control[0] * np.sin(self.pose[2]) + np.random.normal(0, noise[1])
        self.pose[2] += control[1] + np.random.normal(0, noise[2])

    def update(self, z: np.ndarray, j: int, sensor_noise: np.ndarray):
        """EKF 更新单个路标点"""
        if not self.landmarks_seen[j]:
            # 初始化新路标点
            self.landmarks_mean[j] = self.pose[:2] + z
            self.landmarks_cov[j] = np.diag(sensor_noise[:2])
            self.landmarks_seen[j] = True
        else:
            # EKF 更新已知路标点
            H = np.eye(2)
            S = H @ self.landmarks_cov[j] @ H.T + np.diag(sensor_noise[:2])
            K = self.landmarks_cov[j] @ H.T @ np.linalg.inv(S)
            self.landmarks_mean[j] += K @ (z - self.landmarks_mean[j] + self.pose[:2])
            self.landmarks_cov[j] = (np.eye(2) - K @ H) @ self.landmarks_cov[j]

class FastSLAM:
    """FastSLAM 2.0 粒子滤波 SLAM"""
    def __init__(self, num_particles: int = 50, num_landmarks: int = 100):
        self.particles = [FastSLAMParticle(num_landmarks) for _ in range(num_particles)]
        self.num_particles = num_particles

    def resample(self):
        """低方差重采样"""
        weights = np.array([p.weight for p in self.particles])
        weights /= weights.sum()
        indices = np.random.choice(
            self.num_particles, self.num_particles, p=weights
```



```

    )
    self.particles = [self.particles[i] for i in indices]
    for p in self.particles:
        p.weight = 1.0 / self.num_particles

```

10.1.4 slam_toolbox 在线/离线建图

slam_toolbox 是 ROS 2 主流 2D SLAM 方案:

在线建图 (Online Async):

```

import rclpy
from rclpy.node import Node
from slam_toolbox.srv import SaveMap
from nav_msgs.msg import OccupancyGrid
import subprocess

class OnlineSlamManager(Node):
    """在线 SLAM 建图管理器"""
    def __init__(self):
        super().__init__('slam_manager')
        # 订阅地图更新
        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10
        )
        # 保存地图服务客户端
        self.save_client = self.create_client(SaveMap, '/slam_toolbox/save_map')
        self.get_logger().info('SLAM 管理器已就绪')

    def map_callback(self, msg: OccupancyGrid):
        """接收实时建图结果"""
        resolution = msg.info.resolution
        width = msg.info.width
        height = msg.info.height
        free = sum(1 for c in msg.data if c == 0)
        occupied = sum(1 for c in msg.data if c == 100)
        self.get_logger().info(
            f'地图: {width}x{height}, 分辨率={resolution:.3f}m, '
            f'空闲={free}, 占据={occupied}'
        )

    async def save_map(self, name: str = 'my_map'):
        """保存建图结果"""
        # 通过 ROS 2 service 保存地图
        req = SaveMap.Request()
        req.name.data = name
        future = self.save_client.call_async(req)
        await future
        self.get_logger().info(f'地图已保存: {name}')

def spawn_slam_toolbox_online():
    """启动 slam_toolbox 在线建图 Lifecycle 节点"""

```

```

cmd = [
    'ros2', 'launch', 'slam_toolbox',
    'online_async_launch.py',
    'params_file:=config/mapper_params_online_async.yaml',
    'use_sim_time:=true'
]
subprocess.Popen(cmd)

```

离线建图 (手册建图, 读取 rosbag):

```

# 录制数据包
ros2 bag record -o my_mapping_bag /scan /tf /tf_static /odom

# 离线建图
ros2 launch slam_toolbox offline_launch.py \
    bag_filename:=my_mapping_bag use_sim_time:=true

```

10.1.5 AMCL 蒙特卡洛定位

AMCL (Adaptive Monte Carlo Localization) 使用粒子滤波器进行全局定位:

```

import numpy as np
from geometry_msgs.msg import PoseWithCovarianceStamped
from nav2_msgs.action import NavigateToPose

class AMCLManager(Node):
    """AMCL 定位管理器"""
    def __init__(self):
        super().__init__('amcl_manager')
        # 初始位姿发布者
        self.pose_pub = self.create_publisher(
            PoseWithCovarianceStamped, '/initialpose', 10
        )
        # 当前估计位姿
        self.current_pose = None

    def set_initial_pose(self, x: float, y: float, yaw: float):
        """设置 AMCL 初始位姿估计"""
        msg = PoseWithCovarianceStamped()
        msg.header.frame_id = 'map'
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.pose.pose.position.x = x
        msg.pose.pose.position.y = y
        qz = np.sin(yaw / 2.0)
        qw = np.cos(yaw / 2.0)
        msg.pose.pose.orientation.z = qz
        msg.pose.pose.orientation.w = qw
        # 协方差: 表示不确定性
        msg.pose.covariance[0] = 0.25 # x 方差
        msg.pose.covariance[7] = 0.25 # y 方差
        msg.pose.covariance[35] = 0.05 # yaw 方差
        self.pose_pub.publish(msg)
        self.get_logger().info(f'初始位姿已设置: ({x:.2f}, {y:.2f}, {yaw:.2f})')

```

```
def launch_amcl(use_sim_time: bool = True):
    """启动 AMCL 定位节点"""
    import subprocess
    cmd = [
        'ros2', 'launch', 'nav2_bringup',
        'localization_launch.py',
        'map:=src/maps/my_map.yaml',
        'params_file:=config/nav2_params.yaml',
        f'use_sim_time:={str(use_sim_time).lower()}'
    ]
    subprocess.Popen(cmd)
```

AMCL 粒子收敛示意:

初始分布 (全局均匀): 收敛后 (聚类):

```
o o o o o                      . . . . .
o o o o o                      . . o . .
o o o o o                      . o o o .
o o o o o                      . . o . .
o o o o o                      . . . . .
```

-->

10.1.6 map_server 地图保存与加载

```
from nav2_msgs.srv import LoadMap, SaveMap
from nav_msgs.msg import OccupancyGrid
import yaml

class MapServer(Node):
    """地图服务器 Python 接口"""
    def __init__(self):
        super().__init__('map_server_manager')
        self.load_client = self.create_client(
            LoadMap, '/map_server/load_map'
        )
        self.save_client = self.create_client(
            SaveMap, '/map_server/save_map'
        )
        self.current_map: OccupancyGrid = None

    async def load_map(self, yaml_path: str):
        """加载地图"""
        req = LoadMap.Request()
        req.map_url = yaml_path
        future = self.load_client.call_async(req)
        result = await future
        self.get_logger().info(f'地图已加载: {yaml_path} (result={result.result})')
        return result

    async def save_map(self, map_name: str):
        """保存当前地图"""
        req = SaveMap.Request()
        req.map_topic = 'map'
        req.map_url = f'src/maps/{map_name}'
        future = self.save_client.call_async(req)
        await future
        self.get_logger().info(f'地图已保存: {map_name}')
```

```
def read_map_yaml(self, yaml_path: str) -> dict:
    """读取地图 YAML 元数据"""
    with open(yaml_path, 'r') as f:
        return yaml.safe_load(f)
```

10.1.7 Cartographer 多传感器 SLAM

Cartographer 支持激光 + IMU + 里程计多传感器融合，采用子图 (Submap) 和回环检测机制。

Cartographer 核心流程:

传感器数据 → 局部 SLAM (构建子图 submap)

↓

全局 SLAM (回环检测 + 后端优化)

↓

输出全局一致地图

```
import subprocess
import yaml

def generate_cartographer_config(
    num_lasers: int = 1,
    provide_odom: bool = True,
    use_imu: bool = False,
    tracking_frame: str = 'base_link',
    published_frame: str = 'odom',
    map_frame: str = 'map'
) -> str:
    """动态生成 Cartographer 配置文件"""
    config = {
        'options': {
            'use_sim_time': True,
            'map_frame': map_frame,
            'tracking_frame': tracking_frame,
            'published_frame': published_frame,
            'odom_frame': 'odom',
            'provide_odom_frame': provide_odom,
            'use_odometry': provide_odom,
            'use_nav_sat': False,
            'num_range_sensors': num_lasers,
            'use_landmarks': False,
            'num_laser_scans': num_lasers,
            'num_point_clouds': 0,
        },
        'trajectory_builder_options': {
            'use_imu_data': use_imu,
            'min_range': 0.15,
            'max_range': 25.0,
            'voxel_filter_size': 0.05,
        },
        'pose_graph_options': {
            'optimize_every_n_nodes': 90,
            'constraint_builder_options': {
```

```

        'sampling_ratio': 0.3,
        'max_constraint_distance': 15.0,
        'ceres_scan_matcher_options': {
            'occupied_space_weight': 20.0,
            'translation_weight': 10.0,
            'rotation_weight': 1.0,
        },
        'fast_correlative_scan_matcher_options': {
            'linear_search_window': 7.0,
            'angular_search_window': 0.5236,
            'branch_and_bound_depth': 7,
        },
    },
    'matcher_translation_weight': 5e2,
    'matcher_rotation_weight': 1.6e3,
    'optimization_problem_options': {
        'huber_scale': 1e1,
    },
},
}
config_yaml = yaml.dump(config, default_flow_style=False, allow_unicode=True)
return config_yaml

def launch_cartographer(rviz: bool = True):
    """启动 Cartographer 建图"""
    cmd = [
        'ros2', 'launch', 'turtlebot3_cartographer',
        'cartographer.launch.py'
    ]
    if not rviz:
        cmd.append('open_rviz:=false')
    subprocess.Popen(cmd)

```

10.1.8 slam_sim_demo 仿真建图演示

参考 `ros2_ws/src/slam_sim_demo_ros2/` 包结构，使用 teleop 控制机器人在 Gazebo 仿真环境中建图：

```

# 从 slam_sim_demo_ros2 包核心建图流程
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
import numpy as np

class AutoExplore(Node):
    """自动探索建图节点 - 基于激光避障的边界探索"""
    def __init__(self):
        super().__init__('auto_explore')
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10
        )
        self.front_dist = float('inf')
        self.timer = self.create_timer(0.1, self.control_loop)

```

```

def scan_callback(self, msg: LaserScan):
    """分析激光数据"""
    ranges = np.array(msg.ranges)
    # 前方 30 度扇形区域
    idx = len(ranges) // 2
    half_window = len(ranges) // 12
    front_sector = ranges[idx - half_window: idx + half_window]
    self.front_dist = np.min(front_sector[np.isfinite(front_sector)])

def control_loop(self):
    """基于激光的避障巡航"""
    cmd = Twist()
    if self.front_dist < 0.8:
        cmd.angular.z = 0.8 # 障碍物过近, 旋转
        cmd.linear.x = 0.0
    elif self.front_dist < 1.5:
        cmd.linear.x = 0.15
        cmd.angular.z = 0.3 # 慢速避障
    else:
        cmd.linear.x = 0.3 # 前进探索
        cmd.angular.z = 0.0
    self.cmd_pub.publish(cmd)

def main():
    rclpy.init()
    node = AutoExplore()
    rclpy.spin(node)
    rclpy.shutdown()

```

10.2 练习题

- 原理题:** 简述 GraphSLAM 图优化模型中顶点和边的物理含义, 并说明为什么回环检测对 SLAM 至关重要。
- 编程题:** 编写一个 ROS 2 节点, 订阅 `/scan` 话题并统计激光雷达的测量极值 (最近/最远有效距离), 将结果以 ROS 2 日志形式输出。
- 配置题:** 编写 `slam_toolbox` 在线建图的 YAML 配置文件, 要求设置地图分辨率为 0.05m, 更新频率 5Hz, 使用 `scan_matching` 方法, 并调整回环检测阈值。
- 分析题:** AMCL 定位中, 粒子数增加会带来什么影响? 说明 KLD 采样如何自适应调整粒子数量。

5. 操作题: 描述从 Gazebo 仿真到保存最终地图的完整 ROS 2 命令行流程, 包括启动仿真、建图、teleop 控制、保存地图及验证。

6. 设计题: 某仓库分 3 层楼, 每层约 500m², 需要全自主建图。设计方案包括传感器选型、SLAM 算法选择、多楼层地图配准方法, 并给出 ROS 2 Launch 文件框架。

实验 10:

第 10 章 实验指导书：SLAM 建图与定位

****实验平台**:** Ubuntu 22.04 + ROS 2 Humble + Gazebo 仿真 ****预计时间**:** 2 课时 (90 分钟)

实验准备：创建 slam_lab 功能包

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 pkg prefix robot_sim_demo_ros2
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py --show-args
```

安装全部依赖

```
source /opt/ros/humble/setup.bash
sudo apt update
sudo apt install -y \
  python3-colcon-common-extensions \
  python3-rosdep \
  python3-setuptools \
  ros-humble-slam-toolbox \
  ros-humble-navigation2 \
  ros-humble-nav2-bringup \
  ros-humble-nav2-map-server \
  ros-humble-rosbag2 \
  ros-humble-cartographer \
  ros-humble-cartographer-ros \
  ros-humble-teleop-twist-keyboard \
  ros-humble-tf2-geometry-msgs \
  ros-humble-tf2-tools
sudo rosdep init 2>/dev/null || true
rosdep update
```

创建目录

```
source /opt/ros/humble/setup.bash
mkdir -p ~/ros2_course_ws/src
cd ~/ros2_course_ws/src

ros2 pkg create slam_lab \
  --build-type ament_python \
  --license Apache-2.0

cd ~/ros2_course_ws/src/slam_lab
mkdir -p launch config/cartographer
```



```
touch resource/slam_lab slam_lab/__init__.py
mkdir -p ~/maps ~/bags ~/lab10
```

最终目录必须是：

```
slam_lab/
├─ package.xml
├─ setup.py
├─ setup.cfg
├─ resource/
│   └─ slam_lab
├─ slam_lab/
│   ├── __init__.py
│   ├── slam_monitor.py
│   ├── initial_pose_setter.py
│   ├── amcl_evaluator.py
│   └─ cartographer_state_saver.py
├─ launch/
│   ├── online_mapping.launch.py
│   ├── amcl_localization.launch.py
│   └─ cartographer_mapping.launch.py
└─ config/
    ├── mapper_params_online_async.yaml
    ├── nav2_localization.yaml
    └─ cartographer/
        ├── xbot_2d.lua
        └─ Cartographer 官方基础 Lua 文件
```

package.xml

文件路径：~/ros2_course_ws/src/slam_lab/package.xml

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>slam_lab</name>
  <version>0.1.0</version>
  <description>SLAM, AMCL and Cartographer laboratory helpers</description>

  <maintainer email="student@example.com">Student</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>ament_index_python</exec_depend>
  <exec_depend>cartographer_ros</exec_depend>
  <exec_depend>cartographer_ros_msgs</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <exec_depend>nav2_bringup</exec_depend>
  <exec_depend>nav2_map_server</exec_depend>
  <exec_depend>nav_msgs</exec_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>rviz2</exec_depend>
  <exec_depend>slam_toolbox</exec_depend>
  <exec_depend>tf2_geometry_msgs</exec_depend>
```

```

<exec_depend>tf2_ros</exec_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

setup.py

文件路径: ~/ros2_course_ws/src/slam_lab/setup.py

```

import os
from glob import glob

from setuptools import find_packages, setup

package_name = 'slam_lab'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        ('share/' + package_name, ['package.xml']),
        (
            os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py'),
        ),
        (
            os.path.join('share', package_name, 'config'),
            glob('config/*.yaml'),
        ),
        (
            os.path.join('share', package_name, 'config', 'cartographer'),
            glob('config/cartographer/*.lua'),
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='SLAM, AMCL and Cartographer laboratory helpers',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'slam_monitor = slam_lab.slam_monitor:main',
            'set_initial_pose = slam_lab.initial_pose_setter:main',
            'amcl_evaluator = slam_lab.amcl_evaluator:main',
            'save_cartographer_state = slam_lab.cartographer_state_saver:main',
        ],
    },
)

```

setup.cfg

文件路径: ~/ros2_course_ws/src/slam_lab/setup.cfg

```
[develop]
script_dir=$base/lib/slam_lab

[install]
install_scripts=$base/lib/slam_lab
```

两个空文件

```
~/ros2_course_ws/src/slam_lab/resource/slam_lab
~/ros2_course_ws/src/slam_lab/slam_lab/__init__.py
```

slam_monitor.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/slam_monitor.py

```
#!/usr/bin/env python3
"""Print basic statistics for the current slam_toolbox map."""

import rclpy
from nav_msgs.msg import OccupancyGrid
from rclpy.node import Node
from rclpy.qos import DurabilityPolicy, QoSProfile, ReliabilityPolicy

class SlamMonitor(Node):
    def __init__(self):
        super().__init__('slam_monitor')

        map_qos = QoSProfile(
            depth=1,
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.TRANSIENT_LOCAL,
        )
        self.map_sub = self.create_subscription(
            OccupancyGrid,
            '/map',
            self.map_callback,
            map_qos,
        )
        self.map_count = 0
        self.get_logger().info('SLAM map monitor started')

    def map_callback(self, msg: OccupancyGrid):
        self.map_count += 1
        occupied = sum(value >= 65 for value in msg.data)
        unknown = sum(value < 0 for value in msg.data)
        total = len(msg.data)
        known = total - unknown
        free = known - occupied
        known_ratio = known / total * 100.0 if total else 0.0

        self.get_logger().info(
            f'#{self.map_count}: {msg.info.width}x{msg.info.height}, '
            f'{msg.info.resolution:.3f} m/cell, '
            f'occupied={occupied}, free={free}, unknown={unknown}, '
```

```

        f'known={known_ratio:.1f}%'
    )

def main(args=None):
    rclpy.init(args=args)
    node = SlamMonitor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

initial_pose_setter.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/initial_pose_setter.py

```

#!/usr/bin/env python3
"""Publish an AMCL initial pose from x, y and yaw command-line values."""

import argparse
import math
import sys
import time

import rclpy
from geometry_msgs.msg import PoseWithCovarianceStamped
from rclpy.node import Node
from rclpy.utilities import remove_ros_args

def make_message(node: Node, x: float, y: float, yaw: float):
    message = PoseWithCovarianceStamped()
    message.header.frame_id = 'map'
    message.header.stamp = node.get_clock().now().to_msg()
    message.pose.pose.position.x = x
    message.pose.pose.position.y = y
    message.pose.pose.orientation.z = math.sin(yaw / 2.0)
    message.pose.pose.orientation.w = math.cos(yaw / 2.0)
    message.pose.covariance[0] = 0.25
    message.pose.covariance[7] = 0.25
    message.pose.covariance[35] = 0.0685
    return message

def parse_arguments(raw_args):
    parser = argparse.ArgumentParser()
    parser.add_argument('x', type=float)
    parser.add_argument('y', type=float)
    parser.add_argument('yaw_deg', type=float)
    return parser.parse_args(remove_ros_args(raw_args)[1:])

```

```

def main():
    raw_args = sys.argv
    parsed = parse_arguments(raw_args)
    rclpy.init(args=raw_args)

    node = Node('initial_pose_setter')
    publisher = node.create_publisher(
        PoseWithCovarianceStamped,
        '/initialpose',
        10,
    )

    deadline = time.monotonic() + 3.0
    while publisher.get_subscription_count() == 0 and time.monotonic() < deadline:
        rclpy.spin_once(node, timeout_sec=0.1)

    if publisher.get_subscription_count() == 0:
        node.get_logger().warn(
            'No /initialpose subscriber found; check whether AMCL is active'
        )

    yaw = math.radians(parsed.yaw_deg)
    for _ in range(3):
        publisher.publish(make_message(node, parsed.x, parsed.y, yaw))
        rclpy.spin_once(node, timeout_sec=0.2)

    node.get_logger().info(
        f'Published initial pose: x={parsed.x:.2f}, y={parsed.y:.2f}, '
        f'yaw={parsed.yaw_deg:.1f} deg'
    )
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

amcl_evaluator.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/amcl_evaluator.py

```

#!/usr/bin/env python3
"""Compare AMCL and simulator ground-truth poses in one TF frame."""

import math

import rclpy
import tf2_geometry_msgs # Registers geometry message conversions with tf2.
from geometry_msgs.msg import PoseStamped, PoseWithCovarianceStamped
from nav_msgs.msg import Odometry
from rclpy.duration import Duration
from rclpy.node import Node
from tf2_ros import Buffer, TransformException, TransformListener

def yaw_from_pose(pose):
    q = pose.orientation

```

```

return math.atan2(
    2.0 * (q.w * q.z + q.x * q.y),
    1.0 - 2.0 * (q.y * q.y + q.z * q.z),
)

```

```

class AmclEvaluator(Node):
    def __init__(self):
        super().__init__('amcl_evaluator')
        self.declare_parameter('ground_truth_topic', '/ground_truth/odom')
        self.declare_parameter('target_frame', 'map')

        ground_truth_topic = self.get_parameter('ground_truth_topic').value
        self.target_frame = self.get_parameter('target_frame').value
        self.amcl_message = None
        self.ground_truth_message = None

        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.amcl_sub = self.create_subscription(
            PoseWithCovarianceStamped,
            '/amcl_pose',
            self.amcl_callback,
            10,
        )
        self.ground_truth_sub = self.create_subscription(
            Odometry,
            ground_truth_topic,
            self.ground_truth_callback,
            10,
        )
        self.timer = self.create_timer(1.0, self.evaluate)
        self.get_logger().info(
            f'Comparing /amcl_pose with {ground_truth_topic} in '
            f'{self.target_frame}'
        )

    def amcl_callback(self, msg):
        self.amcl_message = msg

    def ground_truth_callback(self, msg):
        self.ground_truth_message = msg

    def transform_pose(self, header, pose):
        stamped = PoseStamped()
        stamped.header = header
        stamped.pose = pose
        return self.tf_buffer.transform(
            stamped,
            self.target_frame,
            timeout=Duration(seconds=0.2),
        )

    def evaluate(self):
        if self.amcl_message is None or self.ground_truth_message is None:
            return

        try:
            amcl_pose = self.transform_pose(
                self.amcl_message.header,

```

```

        self.amcl_message.pose.pose,
    )
    ground_truth_pose = self.transform_pose(
        self.ground_truth_message.header,
        self.ground_truth_message.pose.pose,
    )
    except TransformException as error:
        self.get_logger().warn(f'Cannot transform poses: {error}')
        return

    dx = amcl_pose.pose.position.x - ground_truth_pose.pose.position.x
    dy = amcl_pose.pose.position.y - ground_truth_pose.pose.position.y
    position_error = math.hypot(dx, dy)

    yaw_error = yaw_from_pose(amcl_pose.pose) - yaw_from_pose(
        ground_truth_pose.pose
    )
    yaw_error = math.atan2(math.sin(yaw_error), math.cos(yaw_error))

    self.get_logger().info(
        f'position_error={position_error:.3f} m, '
        f'yaw_error={math.degrees(abs(yaw_error)):.2f} deg'
    )

def main(args=None):
    rclpy.init(args=args)
    node = AmclEvaluator()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

cartographer_state_saver.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/cartographer_state_saver.py

```

#!/usr/bin/env python3
"""Finish one Cartographer trajectory and save its pbstream state."""

import argparse
from pathlib import Path
import sys

from cartographer_ros_msgs.srv import FinishTrajectory, WriteState
import rclpy
from rclpy.node import Node
from rclpy.utilities import remove_ros_args

class CartographerStateSaver(Node):

```

```

def __init__(self):
    super().__init__('cartographer_state_saver')
    self.finish_client = self.create_client(
        FinishTrajectory,
        '/finish_trajectory',
    )
    self.write_client = self.create_client(WriteState, '/write_state')

def wait_for_services(self):
    if not self.finish_client.wait_for_service(timeout_sec=10.0):
        self.get_logger().error('/finish_trajectory is unavailable')
        return False
    if not self.write_client.wait_for_service(timeout_sec=10.0):
        self.get_logger().error('/write_state is unavailable')
        return False
    return True

def finish_trajectory(self, trajectory_id):
    request = FinishTrajectory.Request()
    request.trajectory_id = trajectory_id
    future = self.finish_client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=30.0)
    if not future.done() or future.result() is None:
        self.get_logger().error('Failed to finish trajectory')
        return False
    status = getattr(future.result(), 'status', None)
    if status is not None and status.code != 0:
        self.get_logger().error(f'Finish failed: {status.message}')
        return False
    return True

def write_state(self, filename):
    request = WriteState.Request()
    request.filename = str(filename)
    request.include_unfinished_submaps = True
    future = self.write_client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=30.0)
    if not future.done() or future.result() is None:
        self.get_logger().error('Failed to write Cartographer state')
        return False
    status = getattr(future.result(), 'status', None)
    if status is not None and status.code != 0:
        self.get_logger().error(f'Write failed: {status.message}')
        return False
    return True

def parse_arguments(raw_args):
    parser = argparse.ArgumentParser()
    parser.add_argument('trajectory_id', type=int)
    parser.add_argument('output_file')
    return parser.parse_args(remove_ros_args(raw_args)[1:])

def main():
    raw_args = sys.argv
    parsed = parse_arguments(raw_args)
    output_file = Path(parsed.output_file).expanduser().resolve()
    output_file.parent.mkdir(parents=True, exist_ok=True)

```



```

rclpy.init(args=raw_args)
node = CartographerStateSaver()
try:
    if not node.wait_for_services():
        return
    if not node.finish_trajectory(parsed.trajectory_id):
        return
    if node.write_state(output_file):
        node.get_logger().info(f'Saved Cartographer state: {output_file}')
finally:
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

online_mapping.launch.py

文件路径: ~/ros2_course_ws/src/slam_lab/launch/online_mapping.launch.py

```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    package_share = get_package_share_directory('slam_lab')
    slam_toolbox_share = get_package_share_directory('slam_toolbox')

    use_sim_time = LaunchConfiguration('use_sim_time')
    params_file = LaunchConfiguration('params_file')
    start_monitor = LaunchConfiguration('start_monitor')

    slam_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                slam_toolbox_share,
                'launch',
                'online_async_launch.py',
            )
        ),
        launch_arguments={
            'slam_params_file': params_file,
            'use_sim_time': use_sim_time,
        }.items(),
    )

    monitor_node = Node(
        package='slam_lab',

```

```

        executable='slam_monitor',
        name='slam_monitor',
        output='screen',
        parameters=[{'use_sim_time': use_sim_time}],
        condition=IfCondition(start_monitor),
    )

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='true',
            description='Use the Gazebo /clock topic',
        ),
        DeclareLaunchArgument(
            'params_file',
            default_value=os.path.join(
                package_share,
                'config',
                'mapper_params_online_async.yaml',
            ),
            description='Absolute path to the slam_toolbox parameter file',
        ),
        DeclareLaunchArgument(
            'start_monitor',
            default_value='true',
            description='Start the map statistics node',
        ),
        slam_launch,
        monitor_node,
    ])

```

amcl_localization.launch.py

文件路径: ~/ros2_course_ws/src/slam_lab/launch/amcl_localization.launch.py

```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    package_share = get_package_share_directory('slam_lab')
    nav2_share = get_package_share_directory('nav2_bringup')

    map_file = LaunchConfiguration('map')
    params_file = LaunchConfiguration('params_file')
    use_sim_time = LaunchConfiguration('use_sim_time')
    autostart = LaunchConfiguration('autostart')
    use_rviz = LaunchConfiguration('use_rviz')

    localization = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(

```

```

        os.path.join(nav2_share, 'launch', 'localization_launch.py')
    ),
    launch_arguments={
        'map': map_file,
        'params_file': params_file,
        'use_sim_time': use_sim_time,
        'autostart': autostart,
    }.items(),
)

rviz = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(nav2_share, 'launch', 'rviz_launch.py')
    ),
    launch_arguments={'use_sim_time': use_sim_time}.items(),
    condition=IfCondition(use_rviz),
)

return LaunchDescription([
    DeclareLaunchArgument(
        'map',
        default_value=os.path.expanduser('~/.maps/lab10_map.yaml'),
        description='Absolute path to the saved map YAML file',
    ),
    DeclareLaunchArgument(
        'params_file',
        default_value=os.path.join(
            package_share,
            'config',
            'nav2_localization.yaml',
        ),
        description='Absolute path to the Nav2 localization parameters',
    ),
    DeclareLaunchArgument('use_sim_time', default_value='true'),
    DeclareLaunchArgument('autostart', default_value='true'),
    DeclareLaunchArgument('use_rviz', default_value='true'),
    localization,
    rviz,
])

```

cartographer_mapping.launch.py

文件路径: ~/ros2_course_ws/src/slam_lab/launch/cartographer_mapping.launch.py

```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    package_share = get_package_share_directory('slam_lab')

    configuration_directory = LaunchConfiguration('configuration_directory')

```

```

configuration_basename = LaunchConfiguration('configuration_basename')
scan_topic = LaunchConfiguration('scan_topic')
odom_topic = LaunchConfiguration('odom_topic')
resolution = LaunchConfiguration('resolution')
use_sim_time = LaunchConfiguration('use_sim_time')

cartographer_node = Node(
    package='cartographer_ros',
    executable='cartographer_node',
    name='cartographer_node',
    output='screen',
    arguments=[
        '-configuration_directory',
        configuration_directory,
        '-configuration_basename',
        configuration_basename,
    ],
    parameters=[{'use_sim_time': use_sim_time}],
    remappings=[
        ('scan', scan_topic),
        ('odom', odom_topic),
    ],
)

occupancy_grid_node = Node(
    package='cartographer_ros',
    executable='cartographer_occupancy_grid_node',
    name='cartographer_occupancy_grid_node',
    output='screen',
    arguments=[
        '-resolution',
        resolution,
        '-publish_period_sec',
        '1.0',
    ],
    parameters=[{'use_sim_time': use_sim_time}],
)

return LaunchDescription([
    DeclareLaunchArgument(
        'configuration_directory',
        default_value=os.path.join(
            package_share,
            'config',
            'cartographer',
        ),
    ),
    DeclareLaunchArgument(
        'configuration_basename',
        default_value='xbot_2d.lua',
    ),
    DeclareLaunchArgument('scan_topic', default_value='/scan'),
    DeclareLaunchArgument('odom_topic', default_value='/odom'),
    DeclareLaunchArgument('resolution', default_value='0.05'),
    DeclareLaunchArgument('use_sim_time', default_value='true'),
    cartographer_node,
    occupancy_grid_node,
])

```

mapper_params_online_async.yaml

文件路径: ~/ros2_course_ws/src/slam_lab/config/mapper_params_online_async.yaml

```
slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    odom_frame: odom
    map_frame: map
    base_frame: base_link
    scan_topic: /scan
    use_map_saver: true
    mode: mapping

    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.02
    map_update_interval: 5.0
    resolution: 0.05
    max_laser_range: 12.0
    minimum_time_interval: 0.5
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
    stack_size_to_use: 40000000
    enable_interactive_mode: true

    use_scan_matching: true
    use_scan_barycenter: true
    minimum_travel_distance: 0.2
    minimum_travel_heading: 0.2
    scan_buffer_size: 10
    scan_buffer_maximum_scan_distance: 10.0
    link_match_minimum_response_fine: 0.1
    link_scan_maximum_distance: 1.5
    loop_search_maximum_distance: 3.0
    do_loop_closing: true
    loop_match_minimum_chain_size: 10
    loop_match_maximum_variance_coarse: 3.0
    loop_match_minimum_response_coarse: 0.35
    loop_match_minimum_response_fine: 0.45

    correlation_search_space_dimension: 0.5
    correlation_search_space_resolution: 0.01
    correlation_search_space_smear_deviation: 0.1

    loop_search_space_dimension: 8.0
    loop_search_space_resolution: 0.05
    loop_search_space_smear_deviation: 0.03

    distance_variance_penalty: 0.5
    angle_variance_penalty: 1.0
    fine_search_angle_offset: 0.00349
    coarse_search_angle_offset: 0.349
```

```
coarse_angle_resolution: 0.0349
minimum_angle_penalty: 0.9
minimum_distance_penalty: 0.5
use_response_expansion: true
```

nav2_localization.yaml

文件路径: ~/ros2_course_ws/src/slam_lab/config/nav2_localization.yaml

```
amcl:
  ros__parameters:
    use_sim_time: true
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    base_frame_id: base_link
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: map
    lambda_short: 0.1
    laser_likelihood_max_dist: 2.0
    laser_max_range: 12.0
    laser_min_range: 0.15
    laser_model_type: likelihood_field
    max_beams: 60
    max_particles: 2000
    min_particles: 500
    odom_frame_id: odom
    pf_err: 0.05
    pf_z: 0.99
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
    robot_model_type: nav2_amcl::DifferentialMotionModel
    save_pose_rate: 0.5
    sigma_hit: 0.2
    tf_broadcast: true
    transform_tolerance: 1.0
    update_min_a: 0.2
    update_min_d: 0.25
    z_hit: 0.5
    z_max: 0.05
    z_rand: 0.5
    z_short: 0.05
    scan_topic: scan
    set_initial_pose: false
    always_reset_initial_pose: false
    first_map_only: false

map_server:
  ros__parameters:
    use_sim_time: true
    yaml_filename: ''
    topic_name: map
    frame_id: map
```

```
lifecycle_manager_localization:
  ros__parameters:
    use_sim_time: true
    autostart: true
    node_names: [map_server, amcl]
    bond_timeout: 4.0
```

xbot_2d.lua

文件路径: ~/ros2_course_ws/src/slam_lab/config/cartographer/xbot_2d.lua

Cartographer 的 map_builder.lua、trajectory_builder.lua 及其依赖文件属于已安装的 Cartographer 软件包，不是本实验重新编写的代码。先复制官方基础文件，再创建下面这个完整的机器人配置：

```
cd ~/ros2_course_ws/src/slam_lab
CARTO_CONFIG="$(ros2 pkg prefix cartographer_ros)/share/cartographer_ros/configuration_files"
cp "$CARTO_CONFIG"/*.lua config/cartographer/
nano config/cartographer/xbot_2d.lua

include "map_builder.lua"
include "trajectory_builder.lua"

options = {
  map_builder = MAP_BUILDER,
  trajectory_builder = TRAJECTORY_BUILDER,

  map_frame = "map",
  tracking_frame = "base_link",
  published_frame = "odom",
  odom_frame = "odom",
  provide_odom_frame = false,
  publish_frame_projected_to_2d = true,

  use_odometry = true,
  use_nav_sat = false,
  use_landmarks = false,

  num_laser_scans = 1,
  num_multi_echo_laser_scans = 0,
  num_subdivisions_per_laser_scan = 1,
  num_point_clouds = 0,

  lookup_transform_timeout_sec = 0.2,
  submap_publish_period_sec = 0.3,
  pose_publish_period_sec = 0.005,
  trajectory_publish_period_sec = 0.03,

  rangefinder_sampling_ratio = 1.0,
  odometry_sampling_ratio = 1.0,
  fixed_frame_pose_sampling_ratio = 1.0,
  imu_sampling_ratio = 1.0,
  landmarks_sampling_ratio = 1.0,
}

MAP_BUILDER.use_trajectory_builder_2d = true
```

```

TRAJECTORY_BUILDER_2D.use_imu_data = false
TRAJECTORY_BUILDER_2D.min_range = 0.15
TRAJECTORY_BUILDER_2D.max_range = 12.0
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 5.0
TRAJECTORY_BUILDER_2D.num_accumulated_range_data = 1

POSE_GRAPH.optimize_every_n_nodes = 90
POSE_GRAPH.constraint_builder.sampling_ratio = 0.3
POSE_GRAPH.constraint_builder.min_score = 0.55

return options

```

编译完整功能包

Cartographer 官方 Lua 基础文件必须已经复制到 config/cartographer/, 否则 include 无法解析。然后编译:

```

cd ~/ros2_course_ws
source /opt/ros/humble/setup.bash
rosdep install --from-paths src/slam_lab --ignore-src -r -y
colcon build --packages-select slam_lab --symlink-install
source install/setup.bash

ros2 pkg prefix slam_lab
ros2 pkg executables slam_lab
ros2 launch slam_lab online_mapping.launch.py --show-args
ros2 launch slam_lab amcl_localization.launch.py --show-args
ros2 launch slam_lab cartographer_mapping.launch.py --show-args

```

每个新终端都执行:

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

```

练习 1 : slam_toolbox 在线建图 (约 30 分钟)

启动仿真

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=false

```

检查传感器和 TF

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 topic list | grep -E '^(scan|odom|clock|tf|tf_static)$'
ros2 topic type /scan
ros2 topic echo /scan --once

```



```
ros2 topic echo /odom --once
ros2 run tf2_ros tf2_echo odom base_link
```

```
yun@yun-virtual-machine: ~/Desktop
[ign gazebo-1] [INFO] [1784609544.370422058] [controller_manager]: Loading controller 'joint_state_broadcaster'
[ign gazebo-1] [INFO] [1784609544.432883256] [controller_manager]: Loading controller 'wheel_velocity_controller'
[ign gazebo-1] [INFO] [1784609544.548033252] [controller_manager]: Configuring controller 'joint_state_broadcaster'
[ign gazebo-1] [INFO] [1784609544.548185286] [joint_state_broadcaster]: 'joints' or 'interfaces' parameter is empty. All available state interfaces will be published
[ign gazebo-1] [INFO] [1784609544.570419705] [controller_manager]: Configuring controller 'wheel_velocity_controller'
[ign gazebo-1] [INFO] [1784609544.572502885] [wheel_velocity_controller]: configuration successful

yun@yun-virtual-machine: ~/Desktop
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 108.784000000
- Translation: [0.000, -0.000, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.000, 1.000]
- Rotation: in RPY (radian) [0.000, 0.000, -0.000]
- Rotation: in RPY (degree) [0.000, 0.000, -0.000]
- Matrix:
1.000 0.000 0.000 0.000
-0.000 1.000 -0.000 -0.000
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 109.770000000
- Translation: [0.000, -0.000, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.000, 1.000]
- Rotation: in RPY (radian) [0.000, 0.000, -0.000]
- Rotation: in RPY (degree) [0.000, 0.000, -0.000]
- Matrix:
1.000 0.000 0.000 0.000
-0.000 1.000 -0.000 -0.000
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
^C[INFO] [1784609654.736145978] [rclcpp]: signal_handler(SIGINT/SIGTERM)
```

```
ros2 topic echo /scan --once --field header
ros2 run tf2_tools view_frames
```

```
yun@yun-virtual-machine:~/Desktop$ ros2 topic echo /scan --once --field header
ros2 run tf2_tools view_frames
stamp:
  sec: 180
  nanosec: 490000000
frame_id: laser
...
[INFO] [1784609725.656760144] [view_frames]: Listening to tf data for 5.0 second
S...
[INFO] [1784609730.679145214] [view_frames]: Generating graph in frames.pdf file
...
[INFO] [1784609730.682087760] [view_frames]: Result:tf2_msgs.srv.FrameGraph_Response(frame_yaml="yaw_platform: \n parent: 'base_link'\n broadcaster: 'default_authority'\n rate: 29.612\n most_recent_transform: 186.134000\n oldest_transform: 181.136000\n buffer_length: 4.998\nbase_link: \n parent: 'base_footprint'\n broadcaster: 'default_authority'\n rate: 10000.000\n most_recent_transform: 0.000000\n oldest_transform: 0.000000\n buffer_length: 0.000\nleft_wheel: \n parent: 'base_link'\n broadcaster: 'default_authority'\n rate: 29.612\n most_recent_transform: 186.134000\n oldest_transform: 181.136000\n buffer_length: 4.998\nright_wheel: \n parent: 'base_link'\n broadcaster: 'default_authority'\n rate: 29.612\n most_recent_transform: 186.134000\n oldest_transform: 181.136000\n buffer_length: 4.998")
```

slam_toolbox 参数

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
```

```
SLAM_LAB_SHARE="$(ros2 pkg prefix slam_lab)/share/slam_lab"
test -f "$SLAM_LAB_SHARE/config/mapper_params_online_async.yaml1"
```

```
grep -nE 'odom_frame|map_frame|base_frame|scan_topic|mode|resolution|max_laser_range|use_sim_time' \
"$SLAM_LAB_SHARE/config/mapper_params_online_async.yaml"
```

启动在线异步建图

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch slam_lab online_mapping.launch.py \
  use_sim_time:=true \
  start_monitor:=true
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 node list | grep slam
ros2 topic info /map --verbose
ros2 topic echo /map --once --field info
ros2 service list | grep slam_toolbox
```

启动 RViz2

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

rviz2 --ros-args -p use_sim_time:=true
```

在 RViz2 中设置：

190. Fixed Frame 设为 map;
191. 添加 Map, Topic 设为 /map;
192. 添加 LaserScan, Topic 设为 /scan;
193. 添加 TF 和 RobotModel;
194. 确认地图、激光扫描和机器人位姿重合。

配置完成后在 RViz2 中保存为 ~/lab10/slam_view.rviz, 下次使用：

```
rviz2 -d ~/lab10/slam_view.rviz --ros-args -p use_sim_time:=true
```

键盘遥控并探索环境

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args \
  -p speed:=0.3 \
  -p turn:=0.8 \
```

检查并保存地图

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

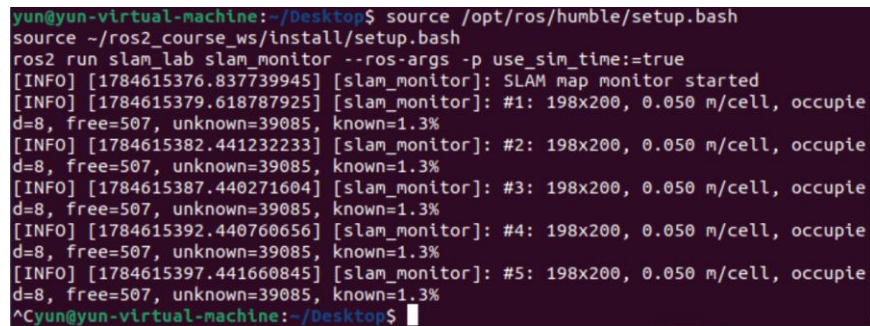
ros2 run nav2_map_server map_saver_cli \
  -t /map \
  -f "$HOME/maps/lab10_map" \
  --ros-args \
  -p use_sim_time:=true \
  -p save_map_timeout:=20.0

test -s "$HOME/maps/lab10_map.pgm" && \
test -s "$HOME/maps/lab10_map.yaml" && \
echo '地图文件保存成功'

ls -lh "$HOME/maps/lab10_map.pgm" "$HOME/maps/lab10_map.yaml"
```

可运行的地图监控脚本

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run slam_lab slam_monitor --ros-args -p use_sim_time:=true
```



```
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run slam_lab slam_monitor --ros-args -p use_sim_time:=true
[INFO] [1784615376.837739945] [slam_monitor]: SLAM map monitor started
[INFO] [1784615379.618787925] [slam_monitor]: #1: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615382.441232233] [slam_monitor]: #2: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615387.440271604] [slam_monitor]: #3: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615392.440760656] [slam_monitor]: #4: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615397.441660845] [slam_monitor]: #5: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
^Cyun@yun-virtual-machine:~/Desktop$
```

练习 2：加载地图并使用 AMCL 定位（约 30 分钟）

停止建图节点并确认地图文件

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

test -s "$HOME/maps/lab10_map.yaml"
test -s "$HOME/maps/lab10_map.pgm"
grep -E '^[[:space:]]*(image|resolution|origin|occupied_thresh|free_thresh):' \
  "$HOME/maps/lab10_map.yaml"
```

启动仿真

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
```

```
use_gazebo:=true \
gz_headless:=false \
use_rviz:=false
```

启动 AMCL 和地图服务器

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch slam_lab amcl_localization.launch.py \
  map:=$HOME/maps/lab10_map.yaml \
  use_sim_time:=true \
  use_rviz:=false \
  autostart:=true
```

检查生命周期状态：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 lifecycle get /map_server
ros2 lifecycle get /amcl
ros2 topic info /map --verbose
ros2 topic echo /map --once --field info
```

```
ros2 lifecycle get /amcl
ros2 topic info /map --verbose
ros2 topic echo /map --once --field info
active [3]
active [3]
Type: nav_msgs/msg/OccupancyGrid
Publisher count: 1

Node name: map_server
Node namespace: /
Topic type: nav_msgs/msg/OccupancyGrid
Endpoint type: PUBLISHER
UID: 0a..ef.72.95.50.2c.0e.03.0a.0a.00.00.00.20.03.00.00.00.00.00.00.00
QoS: rmapfisea...
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: TRANSIENT_LOCAL
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Subscription count: 1
Node name: amcl
Node namespace: /
Topic type: nav_msgs/msg/OccupancyGrid
Endpoint type: SUBSCRIBER
UID: 01.1f.y2.95a50.2d102c0ea.00.00.00.00.00.00.00.00.00.00.00.00.00.00.00
QoS: rprofine:1...
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: TRANSIENT_LOCAL
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite
```

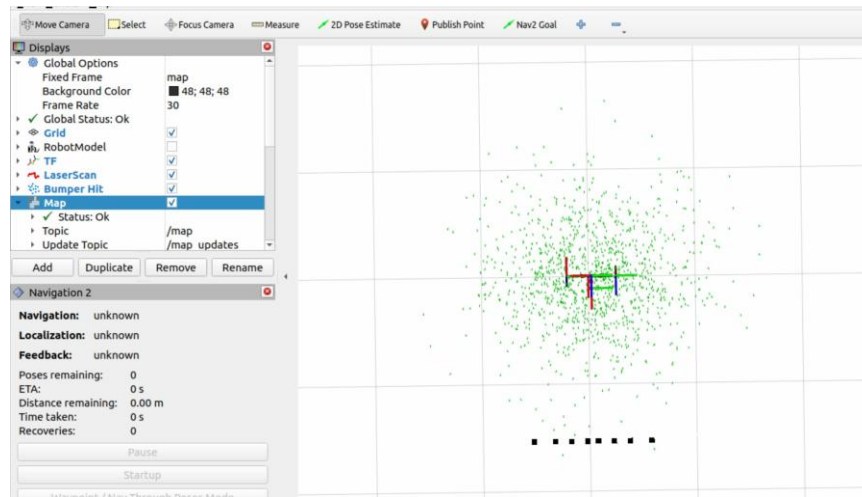
启动 RViz2 并设置初始位姿

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

rviz2 -d /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz \
  --ros-args -p use_sim_time:=true
```

在 RViz2 中:

195. Fixed Frame 设为 map;
196. 确认 Map 的 Topic 是 /map;
197. 添加或确认 LaserScan 的 Topic 是 /scan;
198. 点击 2D Pose Estimate, 在地图中点击机器人位置并拖动箭头指定朝向;
199. 观察粒子云是否逐渐收敛到机器人附近。



```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

# 可以按地图实际位置修改三个数值, yaw 单位为度
ros2 run slam_lab set_initial_pose 0.0 0.0 0.0 \
  --ros-args -p use_sim_time:=true
```

验证 AMCL 输出

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 topic echo /amcl_pose --once
ros2 topic echo /particle_cloud --once
ros2 run tf2_ros tf2_echo map base_link
```

```
yun@yun-virtual-machine: ~/Desktop
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 2836.636000000
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 2837.622000000
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 2838.608000000
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
```

Nav2 AMCL 的粒子话

题是 /particle_cloud。移动机器人后可观察频率：

```
ros2 topic Hz /amcl_pose
```

进行移动和全局重定位测试

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -p speed:=0.2 -p turn:=0.6
```

先在地图中移动到不同区域，再停止机器人并观察 /amcl_pose 是否稳定。要测试全局重定位，先在 RViz2 中给出明显错误的初始位姿，然后执行：

```
ros2 service list | grep reinitialize
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty "{}"

yun@yun-virtual-machine:~/Desktop$ ros2 service list | grep reinitialize
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty "{}"
/reinitialize_global_localization
(requester: making request: std_srvs.srv.Empty_Request())
{
  response:
    std_srvs.srv.Empty_Response()
```

练习 3 : Cartographer 2D 建图和地图导出（扩展，约 30 分钟）

使用 `museum.sdf` 世界文件模拟“两层”地图的两次独立建图流程。

检查 Cartographer 和楼层资源

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 pkg prefix cartographer_ros
ros2 pkg executables cartographer_ros | grep -E 'cartographer_node|occupancy_grid|pbstream'

WORLD_DIR="$(ros2 pkg prefix robot_sim_demo_ros2)/share/robot_sim_demo_ros2/worlds"
ls -lah "$WORLD_DIR"
```

使用现有的 museum.sdf 作为建筑世界：

```
FLOOR_WORLD="$WORLD_DIR/museum.sdf"
test -f "$FLOOR_WORLD" && echo '博物馆世界文件已找到'
```

Shell 变量只在当前终端有效。后续打开新终端时，需要重新设置 WORLD_DIR 和 FLOOR_WORLD。

创建完整 Lua 配置

完整的 xbot_2d.lua 已在实验准备部分给出，并安装在 slam_lab 中。Cartographer 官方基础 Lua 文件也必须位于同一目录。检查：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

CARTO_LAB_CONFIG="$(ros2 pkg prefix slam_lab)/share/slam_lab/config/cartographer"
test -f "$CARTO_LAB_CONFIG/xbot_2d.lua"
test -f "$CARTO_LAB_CONFIG/map_builder.lua"
test -f "$CARTO_LAB_CONFIG/trajectory_builder.lua"
ls -l "$CARTO_LAB_CONFIG" | head
```

若任何 test 失败，回到“xbot_2d.lua”和“编译完整功能包”步骤重新复制基础 Lua 并编译。配置中的 provide_odom_frame = false 是因为 XBot-U 仿真已经提供 odom -> base_link；若实际机器人没有这条 TF，应按实际 TF 架构修改。

启动第一轮 Cartographer

先停止练习 1 或练习 2 的 SLAM/AMCL 节点。打开终端 1 启动仿真：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

WORLD_DIR="$(ros2 pkg prefix robot_sim_demo_ros2)/share/robot_sim_demo_ros2/worlds"
FLOOR_WORLD="$WORLD_DIR/museum.sdf"

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  world:="$FLOOR_WORLD" \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=false
```

打开终端 2，同时启动 Cartographer 和栅格地图节点：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch slam_lab cartographer_mapping.launch.py \
  scan_topic:=/scan \
  odom_topic:=/odom \
  resolution:=0.05 \
  use_sim_time:=true
```

打开终端 3 遥控和终端 4 RViz2：

```
# 终端 3
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -p speed:=0.2 -p turn:=0.6

# 终端 4
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
rviz2 --ros-args -p use_sim_time:=true
```

RViz2 的 Fixed Frame 设为 map，添加 /map、/scan 和 TF。确认 Cartographer 的地图在机器人移动时更新。

保存并导出第一轮地图

先查看当前轨迹编号。通常新启动的 Cartographer 轨迹编号为 0，但应以 /submap_list 实际输出为准：

```
ros2 topic echo /submap_list --once
ros2 service list | grep -E 'finish_trajectory|write_state'
```

建图完成后，在终端 5 运行实验准备部分完整给出的状态保存节点：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
mkdir -p "$HOME/maps"

ros2 run slam_lab save_cartographer_state \
  0 "$HOME/maps/floor1.pbstream"

test -s "$HOME/maps/floor1.pbstream" && echo 'floor1 状态保存成功'
```

该节点依次调用 ROS 2 的 /finish_trajectory 和 /write_state 服务，并检查返回状态。旧版的 rosservice call 是 ROS 1 命令，在 ROS 2 Humble 中不可用。

停止 Cartographer 节点后，将状态转换为 Nav2 可读取的地图：


```
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename "$HOME/maps/floor1.pbstream" \
  -map_filestem "$HOME/maps/floor1_map" \
  -resolution 0.05

ls -lh "$HOME/maps/floor1_map.pgm" "$HOME/maps/floor1_map.yaml"
```

3.5 第二轮地图

第二轮必须使用全新的仿真和 Cartographer 进程。先停止第一轮 Gazebo、Cartographer、occupancy grid 和 teleop，再执行：

```
# 终端 1：重新启动仿真（同一世界，模拟第二层）
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

WORLD_DIR="$(ros2 pkg prefix robot_sim_demo_ros2)/share/robot_sim_demo_ros2/worlds"
FLOOR_WORLD="$WORLD_DIR/museum.sdf"

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  world:="$FLOOR_WORLD" \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=false
```

使用与第一轮相同的 Cartographer 和遥控命令，完成探索后保存：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run slam_lab save_cartographer_state \
  0 "$HOME/maps/floor2.pbstream"

ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename "$HOME/maps/floor2.pbstream" \
  -map_filestem "$HOME/maps/floor2_map" \
  -resolution 0.05

ls -lh "$HOME/maps/floor2_map.pgm" "$HOME/maps/floor2_map.yaml"
```

练习 4：rosbag 回放建图（约 15 分钟）

录制建图数据

停止所有 SLAM、AMCL 和 Cartographer 节点。打开终端 1 重新启动仿真：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
```

```
gz_headless:=false \  
use_rviz:=false
```

打开终端 2，先确认传感器和控制话题已经就绪：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
ros2 topic list | grep -E '^(scan|odom|tf|tf_static|cmd_vel_teleop)$'  
ros2 topic info /scan --verbose  
ros2 topic info /cmd_vel_teleop --verbose
```

在终端 2 开始录制：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
mkdir -p "$HOME/bags"  
BAG_DIR="$HOME/bags/lab10_$(date +%Y%m%d_%H%M%S)"  
printf '%s\n' "$BAG_DIR" | tee "$HOME/bags/latest_bag.txt"  
  
ros2 bag record -o "$BAG_DIR" \  
  /scan /odom /tf /tf_static
```

看到 All requested topics are subscribed 后保持终端 2 运行。打开终端 3 遥控机器人覆盖环境：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
ros2 run teleop_twist_keyboard teleop_twist_keyboard \  
  --ros-args \  
  -p speed:=0.3 \  
  -p turn:=0.8 \  
  -r cmd_vel:=/cmd_vel_teleop
```

/cmd_vel_teleop 会经过 twist_mux -> /cmd_vel -> gazebo_interface_bridge 转换为底层轮速命令。

等待出现 Recording stopped 后检查包内容：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
BAG_DIR="$(cat "$HOME/bags/latest_bag.txt")"  
test -d "$BAG_DIR"  
ros2 bag info "$BAG_DIR"
```

```

[1m] [10:00:00.000000] [roscpp_core] Recording stopped
yun@yun-virtual-machine:~/Desktop$ ros2 bag info "$BAG_DIR"

Files:          lab10_20260722_102831_0.db3
Bag size:       5.4 MiB
Storage id:     sqlite3
Duration:       81.863940324s
Start:          Jul 22 2026 10:28:31.445687000 (1784687311.445687000)
End:            Jul 22 2026 10:29:53.309627324 (1784687393.309627324)
Messages:       9401
Topic information: Topic: /odom | Type: nav_msgs/msg/Odometry | Count: 2350 | Se
rialization Format: cdr
                  Topic: /scan | Type: sensor_msgs/msg/LaserScan | Count: 2350
                  | Serialization Format: cdr
                  Topic: /tf | Type: tf2_msgs/msg/TFMessage | Count: 4700 | Ser
ialization Format: cdr
                  Topic: /tf_static | Type: tf2_msgs/msg/TFMessage | Count: 1 |
                  Serialization Format: cdr

```

回放数据并建图

停止 Gazebo，避免回放期间同时存在两套 /scan、/odom 和 TF 发布者。确认 Gazebo、fake_laser 和控制器节点已经退出，再打开终端 1 启动同步建图：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 node list | grep -E 'gazebo|fake_laser|controller_manager'
ros2 topic info /scan --verbose
ros2 topic info /odom --verbose

```

节点过滤命令应无输出，/scan 和 /odom 应不存在或显示 Publisher count: 0。然后启动同步建图：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

SLAM_PARAMS="$(ros2 pkg prefix slam_lab)/share/slam_lab/config/mapper_params_online_async.yaml"
ros2 launch slam_toolbox online_sync_launch.py \
  slam_params_file:="$SLAM_PARAMS" \
  use_sim_time:=true

```

看到 /slam_toolbox 节点后，打开终端 2 回放：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

BAG_DIR="$(cat "$HOME/bags/latest_bag.txt")"
test -d "$BAG_DIR"
ros2 bag play "$BAG_DIR" --clock --rate 0.5

```

打开终端 3 运行 RViz2，固定坐标系设为 map，观察地图是否随着回放生成：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

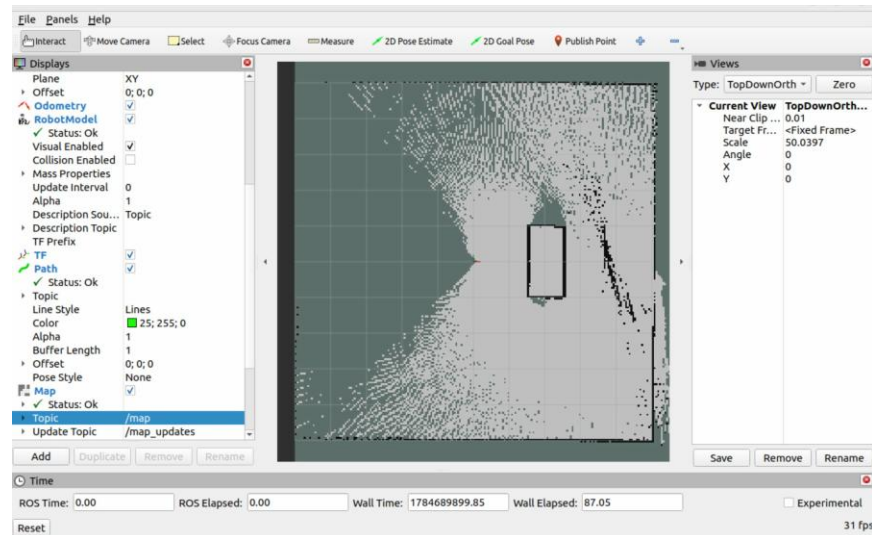
rviz2 --ros-args -p use_sim_time:=true

```

如果练习 1 已经保存 RViz 配置，也可以运行：

```
rviz2 -d "$HOME/lab10/slam_view.rviz" \  
--ros-args -p use_sim_time:=true
```

RViz2 的 Fixed Frame 设为 map，添加 /map、/scan 和 TF。当前 VM 可能输出 GLSL 着色器错误，但只要日志持续出现 Trying to create a map 且地图尺寸更新，就不影响建图结果。



回放过程中打开终端 4 验证数据来源、地图和 TF：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
ros2 node list | grep -E 'rosbag2_player|slam_toolbox'  
ros2 topic info /clock --verbose  
ros2 topic info /scan --verbose  
ros2 topic info /odom --verbose  
ros2 topic info /map --verbose  
ros2 topic echo /map --once --field info  
ros2 run tf2_ros tf2_echo map base_link
```

看到 map -> base_link 连续输出后停止 tf2_echo。/clock、/scan 和 /odom 应各有一个来自 rosbag2_player 的发布者，/map 应由 slam_toolbox 发布。/tf 同时包含 rosbag 回放的里程计 TF 和 slam_toolbox 发布的 map -> odom，因此可能有两个发布者，这是正常现象。偶尔出现一条 Message Filter dropping message ... queue is full 通常是启动瞬间的缓存抖动；如果持续出现，停止回放并把 --rate 0.5 降到 --rate 0.2。

回放结束后保存地图：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
mkdir -p "$HOME/maps"  
ros2 topic info /map --verbose
```

```
ros2 run nav2_map_server map_saver_cli \  
-t /map \  
-f "$HOME/maps/lab10_replayed_map" \  
--ros-args \  
-p use_sim_time:=true \  
-p save_map_timeout:=20.0  
  
test -s "$HOME/maps/lab10_replayed_map.pgm" && \  
test -s "$HOME/maps/lab10_replayed_map.yaml" && \  
echo '回放地图保存成功'  
  
ls -lh "$HOME/maps/lab10_replayed_map.pgm" \  
"$HOME/maps/lab10_replayed_map.yaml"
```

第 11 章 Nav2 自主导航

11.1 知识要点

11.1.1 Nav2 行为树架构

Nav2 使用行为树 (Behavior Tree) 作为任务编排的核心, 替代传统的有限状态机 (FSM)。

行为树节点类型:

Sequence (→): 顺序执行子节点, 任一失败则失败
Fallback (?): 尝试子节点, 任一成功则成功
ReactiveSequence: 顺序执行且可被更高优先级中断
Parallel (⇨): 并行执行多个子节点
Decorator: 装饰节点, 修改子节点行为
Action: 叶子节点, 执行具体任务 (导航/等待)
Condition: 叶子节点, 检查条件 (电池/障碍物)

Nav2 默认行为树结构:

```
<BehaviorTree>
  <PipelineSequence name="NavigateToPose">
    <RateController hz="10.0">
      <RecoveryNode>
        <ComputePathToPose goal="{goal}" planner_id="GridBased"/>
        <ClearEntireCostmap service_name="global_costmap/clear_entirely"/>
      </RecoveryNode>
    </RateController>
    <RateController hz="10.0">
      <RecoveryNode>
        <FollowPath controller_id="FollowPath"/>
        <Spin/>
        <Wait wait_duration="1.0"/>
        <ClearEntireCostmap service_name="local_costmap/clear_entirely"/>
      </RecoveryNode>
    </RateController>
  </PipelineSequence>
</BehaviorTree>

import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose
from nav2_msgs.srv import LoadMap, ClearEntireCostmap
from geometry_msgs.msg import PoseStamped
import math

class Nav2Client(Node):
    """Nav2 Python 客户端 — 使用 Action API"""
    def __init__(self):
```

```

super().__init__('nav2_client')
self.navigate_client = ActionClient(self, NavigateToPose, 'navigate_to_pose')
self.clear_costmap_client = self.create_client(
    ClearEntireCostmap, '/local_costmap/clear_entirely_costmap'
)
self.get_logger().info('Nav2 客户端已就绪')

def goto_pose(self, x: float, y: float, yaw: float = 0.0):
    """
    发送导航目标
    参数:
        x: 目标 x 坐标 (map 坐标系)
        y: 目标 y 坐标
        yaw: 目标朝向 (弧度)
    """
    if not self.navigate_client.wait_for_server(timeout_sec=5.0):
        self.get_logger().error('导航服务器不可用')
        return None

    goal_msg = NavigateToPose.Goal()
    goal_msg.pose.header.frame_id = 'map'
    goal_msg.pose.header.stamp = self.get_clock().now().to_msg()
    goal_msg.pose.pose.position.x = x
    goal_msg.pose.pose.position.y = y
    # yaw → quaternion
    goal_msg.pose.pose.orientation.z = math.sin(yaw / 2.0)
    goal_msg.pose.pose.orientation.w = math.cos(yaw / 2.0)
    goal_msg.behavior_tree = '' # 使用默认行为树

    future = self.navigate_client.send_goal_async(
        goal_msg, feedback_callback=self.feedback_callback
    )
    future.add_done_callback(self.goal_response_callback)
    return future

def goal_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().error('导航目标被拒绝')
        return
    self.get_logger().info('导航目标已接受')
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def result_callback(self, future):
    result = future.result().result
    status = future.result().status
    if status == 4: # SUCCEEDED
        self.get_logger().info(f'导航完成! 耗时: {result.total_elapsed_time.sec}s')
    else:
        self.get_logger().error(f'导航失败, 状态码: {status}')

def feedback_callback(self, feedback_msg):
    feedback = feedback_msg.feedback
    remaining = feedback.distance_remaining

```

```
self.get_logger().debug(f'剩余距离: {remaining:.2f} m')

async def cancel_navigation(self):
    """取消当前导航任务"""
    if self.navigate_client.server_is_ready():
        self.navigate_client._cancel_goal_async()
```

11.1.2 全局规划器

全局规划器根据静态地图计算从当前位置到目标的最优路径。

规划器	算法	特点	适用场景
NavFn	导航函数/梯度下降	平滑路径, 经典算法	简单环境
SmacPlannerHybrid	Hybrid-A*	Dubin 约束, 可反向	非全向机器人
SmacPlannerLattice	状态格	满足动力学约束	阿克曼转向
ThetaStar	Theta*	任意角度路径	全向机器人

```
import numpy as np
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
from geometry_msgs.msg import PoseStamped

def demonstrate_planners():
    """
    演示不同全局规划器切换
    NavigateToPose 的 behavior_tree 参数可以指定不同规划器
    """
    navigator = BasicNavigator()

    # 等待 Nav2 激活
    navigator.waitUntilNav2Active()

    # 目标位姿
    goal = PoseStamped()
    goal.header.frame_id = 'map'
    goal.pose.position.x = 5.0
    goal.pose.position.y = 3.0
    goal.pose.orientation.w = 1.0

    # 规划器可以通过参数服务器动态切换（不推荐运行时变）
    # 推荐在 launch 文件中配置
    # params.yaml:
    #   planner_server:
    #     ros__parameters:
    #       planner_plugin: "nav2_smac_planner/SmacPlannerHybrid"

    navigator.goToPose(goal)
    while not navigator.isTaskComplete():
```



```

feedback = navigator.getFeedback()
print(f'剩余: {feedback.distance_remaining:.2f}m')

result = navigator.getResult()
if result == TaskResult.SUCCEEDED:
    print('导航成功!')

```

自定义 Planner 接口 (Python):

```

# 通过继承 Planner 基类实现自定义全局规划器
import rclpy
from rclpy.node import Node
from nav2_core import GlobalPlanner
from nav_msgs.msg import Path

class SimpleStraightLinePlanner(Node, GlobalPlanner):
    """自定义直线全局规划器 (仅供教学)"""
    def __init__(self):
        super().__init__('straight_line_planner')

    def configure(self, parent_node, name, costmap, global_frame, ...):
        self.global_frame = global_frame
        self.costmap = costmap

    def createPlan(self, start: PoseStamped, goal: PoseStamped) -> Path:
        path = Path()
        path.header.frame_id = self.global_frame
        # 简单直线插值
        num_points = 50
        for i in range(num_points + 1):
            t = i / num_points
            pose = PoseStamped()
            pose.pose.position.x = start.pose.position.x * (1 - t) + goal.pose.position.x * t
            pose.pose.position.y = start.pose.position.y * (1 - t) + goal.pose.position.y * t
            pose.pose.orientation = start.pose.orientation
            path.poses.append(pose)
        return path

```

11.1.3 局部控制器

局部控制器负责执行全局规划的路径, 实现实时避障和速度控制。

控制器	算法	特点	适用场景
DWB	动态窗口逼近	评分机制选择最优速度	通用轮式机器人
MPPI	模型预测路径积分	采样+并行前向模拟	高性能动态环境
RPP	调节纯追踪	纯追踪+自适应前瞻	阿克曼/差分
Regulated Pure	RPP	速度自适应缩放	简单可靠场景

Pursuit			
---------	--	--	--

```
# DWB 控制器评分机制 (概念代码)
class DWBCritic:
    """DWB 控制器的核心 - 评分器 (Critic)"""
    def score_trajectory(self, trajectory):
        """
        对每条候选轨迹打分
        返回: 综合分数 (越低越好)
        """
        scores = {}
        # 1. 路径对齐评分
        scores['path_align'] = self.path_align_score(trajectory)
        # 2. 目标距离评分
        scores['goal_dist'] = self.goal_dist_score(trajectory)
        # 3. 障碍物距离评分 (安全评分)
        scores['obstacle'] = self.obstacle_cost_score(trajectory)
        # 4. 速度偏好评分
        scores['vel_pref'] = self.vel_pref_score(trajectory)
        # 5. 摆动评分 (抑制摇摆)
        scores['oscillation'] = self.oscillation_score(trajectory)
        # 加权求和
        return sum(scores.values())

    def obstacle_cost_score(self, trajectory):
        """障碍物代价值 - 越大越危险"""
        min_dist = min(trajectory.dist_to_obstacles)
        return 1.0 / (min_dist + 0.01) * 10.0

# 控制器参数调优示例 (nav2_params.yaml)
controller_params = {
    'FollowPath': {
        'plugin': 'dwb_core::DWBLocalPlanner',
        'debug_trajectory_details': True,
        'min_vel_x': 0.0,
        'max_vel_x': 0.5,
        'min_vel_y': 0.0,
        'max_vel_y': 0.0,          # 差速机器人, 无侧向运动
        'max_vel_theta': 1.0,
        'min_speed_xy': 0.0,
        'max_speed_xy': 0.5,
        'min_speed_theta': 0.0,
        'acc_lim_x': 2.5,
        'acc_lim_theta': 3.2,
        'decel_lim_x': -2.5,
        'decel_lim_theta': -3.2,
        'goal_checker_plugin': 'goal_checker',
        'critics': [
            'RotateToGoal',

```

```

        'Oscillation',
        'GoalAlign',
        'PathAlign',
        'PathDist',
        'GoalDist'
    ],
    # 各评分器权重
    'PathAlign.scale': 32.0,
    'GoalDist.scale': 24.0,
    'PathDist.scale': 32.0,
    'GoalAlign.scale': 24.0,
    'RotateToGoal.scale': 32.0,
    'Oscillation.scale': 10.0,
}
}
}

```

11.1.4 代价地图分层配置

Costmap2D 采用分层架构, 每一层维护不同的代价信息:

Master Costmap (融合层)	
Static Layer	障碍物地图
Obstacle Layer	传感器更新的动态障碍物
Inflation Layer	障碍物膨胀/安全距离
Range Layer	距离传感器直接更新
Voxel Layer	3D 体素感知
Keepout Layer	禁区 (Filter Mask)
Speed Layer	速度限制区域

代价地图 YAML 配置示例 (以代码形式展示层次结构)

```

costmap_config = {
  'global_costmap': {
    'ros__parameters': {
      'update_frequency': 1.0,          # 全局代价地图更新频率低
      'publish_frequency': 1.0,
      'global_frame': 'map',
      'robot_base_frame': 'base_link',
      'robot_radius': 0.22,
      'resolution': 0.05,
      'track_unknown_space': True,
      'plugins': ['static_layer', 'obstacle_layer', 'inflation_layer'],
      'static_layer': {
        'plugin': 'nav2_costmap_2d::StaticLayer',
        'map_subscribe_transient_local': True,
      },
    },
    'obstacle_layer': {
      'plugin': 'nav2_costmap_2d::ObstacleLayer',
      'enabled': True,
      'observation_sources': 'scan',
      'scan': {
        'topic': '/scan',
        'max_obstacle_height': 2.0,

```

```

        'clearing': True,
        'marking': True,
        'data_type': 'LaserScan',
    },
},
'inflation_layer': {
    'plugin': 'nav2_costmap_2d::InflationLayer',
    'cost_scaling_factor': 3.0,
    'inflation_radius': 0.55,      # 膨胀半径
},
}
},
'local_costmap': {
    'ros__parameters': {
        'update_frequency': 5.0,      # 局部代价地图更新频率高
        'publish_frequency': 2.0,
        'global_frame': 'odom',
        'robot_base_frame': 'base_link',
        'rolling_window': True,        # 滚动窗口
        'width': 3,                    # 3m 窗口
        'height': 3,
        'resolution': 0.05,
        'robot_radius': 0.22,
        'plugins': ['voxel_layer', 'inflation_layer'],
        'voxel_layer': {
            'plugin': 'nav2_costmap_2d::VoxelLayer',
            'enabled': True,
            'publish_voxel_map': True,
            'origin_z': 0.0,
            'z_resolution': 0.05,
            'z_voxels': 16,
            'max_obstacle_height': 2.0,
            'mark_threshold': 0,
            'observation_sources': 'scan',
            'scan': {
                'topic': '/scan',
                'max_obstacle_height': 2.0,
                'clearing': True,
                'marking': True,
                'data_type': 'LaserScan',
            },
        },
    },
    'inflation_layer': {
        'plugin': 'nav2_costmap_2d::InflationLayer',
        'cost_scaling_factor': 3.0,
        'inflation_radius': 0.55,
    },
}
}
}

```

11.1.5 恢复行为

当机器人无法正常导航时, Nav2 会触发恢复行为:

```

NavigateToPose
└─ RecoveryNode (Fallback 风格)

```

```

└─ ComputePathToPose (成功返回 SUCCESS)
└─ RecoverySequence
    └─ Spin (原地旋转 360°)
    └─ Wait (等待 1s)
    └─ BackUp (后退 0.3m)
    └─ ClearCostmap (清除代价地图)

from rclpy.node import Node
from nav2_core import Recovery

class CustomRecovery(Node, Recovery):
    """自定义恢复行为 — 螺旋搜索"""
    def __init__(self):
        super().__init__('spiral_recovery')

    def configure(self, parent_node, name, tf_buffer, costmap_ros, global_costmap, local_costmap):
        self.parent_node = parent_node
        self.cmd_vel_pub = parent_node.create_publisher(Twist, '/cmd_vel', 10)

    def run(self):
        """执行螺旋搜索恢复"""
        self.get_logger().info('执行螺旋搜索恢复...')
        twist = Twist()
        twist.linear.x = 0.1
        twist.angular.z = 0.5 # 逐渐扩大搜索半径
        self.cmd_vel_pub.publish(twist)
        rclpy.sleep(0.5)
        return True

```

11.1.6 Simple Commander API (Python)

Nav2 Simple Commander 是 Python 封装的高层 API, 简化导航调用:

```

#!/usr/bin/env python3
"""
Nav2 Simple Commander 完整示例
参考: nav_goal_runner.py
"""
import rclpy
from rclpy.node import Node
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
from geometry_msgs.msg import PoseStamped, PoseWithCovarianceStamped
import math

class NavGoalRunner(Node):
    """
    多点导航任务执行器
    支持: goToPose, followWaypoints, cancelTask
    """
    def __init__(self):
        super().__init__('nav_goal_runner')
        self.navigators = BasicNavigator()
        self.get_logger().info('导航执行器已启动')

    def set_initial_pose(self, x: float = 0.0, y: float = 0.0, yaw: float = 0.0):

```

```

    """设置初始位姿估计"""
    pose = PoseWithCovarianceStamped()
    pose.header.frame_id = 'map'
    pose.header.stamp = self.get_clock().now().to_msg()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.orientation.z = math.sin(yaw / 2.0)
    pose.pose.orientation.w = math.cos(yaw / 2.0)
    pose.pose.covariance[0] = 0.25
    pose.pose.covariance[7] = 0.25
    pose.pose.covariance[35] = 0.068
    self.navigator.setInitialPose(pose)
    self.get_logger().info('初始位姿已设置')

def wait_for_nav2(self):
    """等待 Nav2 激活"""
    self.get_logger().info('等待 Nav2 就绪...')
    self.navigator.waitUntilNav2Active()
    self.get_logger().info('Nav2 已就绪!')

def create_pose(self, x: float, y: float, yaw: float = 0.0) -> PoseStamped:
    """创建目标位姿消息"""
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.header.stamp = self.navigator.get_clock().now().to_msg()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.orientation.z = math.sin(yaw / 2.0)
    pose.pose.orientation.w = math.cos(yaw / 2.0)
    return pose

def run_single_goal(self, x: float, y: float, yaw: float):
    """执行单点导航"""
    goal = self.create_pose(x, y, yaw)
    self.get_logger().info(f'导航至: ({x:.2f}, {y:.2f}, ∠{yaw:.2f})')
    self.navigator.goToPose(goal)

    while not self.navigator.isTaskComplete():
        feedback = self.navigator.getFeedback()
        if feedback:
            self.get_logger().info(
                f'剩余距离: {feedback.distance_remaining:.2f}m, '
                f'预计时间: {feedback.estimated_time_remaining:.1f}s'
            )

    result = self.navigator.getResult()
    if result == TaskResult.SUCCEEDED:
        self.get_logger().info('目标已到达!')
    elif result == TaskResult.CANCELED:
        self.get_logger().warn('导航已取消')
    elif result == TaskResult.FAILED:
        self.get_logger().error('导航失败')

def run_waypoints(self, waypoints: list):

```

```

"""
执行航点序列导航
参数:
    waypoints: [(x1, y1, yaw1), (x2, y2, yaw2), ...]
"""
poses = [self.create_pose(x, y, yaw) for x, y, yaw in waypoints]
self.get_logger().info(f'开始航点导航: {len(poses)} 个目标')
self.navigator.followWaypoints(poses)

i = 0
while not self.navigator.isTaskComplete():
    i += 1
    feedback = self.navigator.getFeedback()
    if feedback and i % 10 == 0:
        self.get_logger().info(
            f'正在前往 #{feedback.current_waypoint}/{len(poses)}, '
            f'剩余: {feedback.distance_remaining:.2f}m'
        )

    result = self.navigator.getResult()
    if result == TaskResult.SUCCEEDED:
        self.get_logger().info('所有航点已到达!')
    else:
        self.get_logger().error(f'航点导航中断: {result}')

def patrol_loop(self, waypoints: list, loops: int = 3):
    """
    巡逻循环 — 在航点间反复导航
    参数:
        waypoints: 巡逻点列表
        loops: 循环次数
    """
    for loop in range(loops):
        self.get_logger().info(f'--- 第 {loop + 1}/{loops} 轮巡逻 ---')
        for wp in waypoints:
            self.run_single_goal(*wp)
            if self.navigator.getResult() != TaskResult.SUCCEEDED:
                return

def main():
    rclpy.init()
    runner = NavGoalRunner()

    # 设置初始位姿
    runner.set_initial_pose(0.0, 0.0, 0.0)
    runner.wait_for_nav2()

    # 示例: 单点导航
    runner.run_single_goal(3.0, 2.0, math.radians(90))

    # 示例: 航点导航
    waypoints = [
        (2.0, 0.0, 0.0),
        (3.0, 1.5, math.radians(90)),

```

```

        (2.0, 3.0, math.radians(180)),
        (0.0, 2.0, math.radians(-90)),
    ]
    runner.run_waypoints(waypoints)

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

11.1.7 导航系统完整启动流程

```

# 1. 启动仿真环境
ros2 launch xbot_sim xbot_gazebo.launch.py world:=office

# 2. 启动 Nav2 Bringup (包含所有导航组件)
ros2 launch nav2_bringup navigation_launch.py \
    map:=src/maps/office_map.yaml \
    params_file:=config/nav2_params.yaml \
    use_sim_time:=true

# 3. (可选) 启动 SLAM (无地图时的 mapping 模式)
ros2 launch nav2_bringup slam_launch.py \
    params_file:=config/mapper_params_online_async.yaml

```

11.1.8 navigation_sim_demo 参考代码

```

# 参考 ros2_ws/src/navigation_sim_demo_ros2/ 包
# 典型导航包结构:
navigation_sim_demo_ros2/
├── launch/
│   ├── nav2_demo.launch.py          # 主启动文件
│   └── navigation_custom.launch.py  # 自定义导航启动
├── config/
│   └── nav2_params.yaml              # Nav2 完整参数
├── nav_goal_runner.py                # Python 导航执行器
├── patrol_demo.py                    # 巡逻演示
└── dynamic_obstacle_spawner.py      # 动态障碍物生成

```

11.2 练习题

- 架构题:** 绘制 Nav2 行为树结构图, 说明 RecoveryNode 的工作机制,并解释为什么使用 Fallback 节点作为恢复树根节点。
- 配置题:** 为差速轮式机器人配置 DWB 控制器参数: 最大线速度 0.5 m/s, 最大角速度 1.0 rad/s, 膨胀半径 0.55m, 并写出来调节 PathAlign 和 GoalDist 权重对导航行为的影响。

3. 编程题: 使用 `nav2_simple_commander` 编写一个 "8 字形" 巡逻任务: 机器人轮流经过 4 个目标点, 每次到达后原地旋转 360° , 共执行 3 轮。

4. 分析题: 比较全局代价地图和局部代价地图在更新频率、坐标系、大小和插件配置上的差异, 说明为什么需要两种代价地图。

5. 设计题: 设计一个自定义恢复行为: "后退并重新规划" — 当导航失败时, 机器人先后退 0.5m 再尝试重新规划路径。写出 XML 行为树片段和 Python 实现框架。

6. 综合题: 某仓库场景下机器人需要在狭窄通道中导航, 经常出现路径规划失败。设计方案改进包括: 代价地图分辨率调整、规划器替换为 Hybrid-A*、增加 Keepout Filter 避免特定区域, 并说明各参数调整的预期效果。

实验 11:

第 11 章 实验手册: Nav2 自主导航

环境说明

本实验使用 XBot-U 机器人 + Gazebo Sim (Fortress) 仿真, 预建地图位于 `/home/yun/maps/`:

地图文件	尺寸	分辨率	原点
<code>`lab10_e2e_replayed_map.yaml`</code>	202×202	0.05 m/pix	<code>`[-5.05, -5.04, 0]`</code>
<code>`lab10_map.yaml`</code>	198×200	0.05 m/pix	<code>`[-4.89, -5, 0]`</code>

控制链:

```
/cmd_vel_teleop (teleop) ┐
                        │ ─ /twist_mux → /cmd_vel → GazeboInterfaceBridge
/cmd_vel_nav (Nav2) ───┘                → /wheel_velocity_controller/commands
```

准备工作

构建包

```
cd ~/ros2_course_ws
colcon build --packages-select navigation_lab
source install/setup.bash
```

启动仿真 + Nav2

```
# 终端 1: Gazebo 仿真
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  fake_laser_map_yaml:=/home/yun/maps/lab10_e2e_replayed_map.yaml

# 终端 2: Nav2 导航栈
ros2 launch navigation_lab nav_bringup.launch.py \
  map:=/home/yun/maps/lab10_e2e_replayed_map.yaml \
  use_composition:=False autostart:=true
```

```
# 终端 3: RViz2 可视化 (Nav2 默认视图, 含规划路径/代价地图)
rviz2 -d /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz
```

练习 1: goToPose 单点导航 (~15 分钟)

目标

使用 Nav2 Simple Commander API 实现单目标点自主导航, 理解 NavigateToPose action 的反馈结构。

知识点

NavigateToPose 的 action 反馈包含以下字段:

字段	类型	说明
`distance_remaining`	float32	到目标的剩余距离 (米)
`navigation_time`	Duration	已导航时间
`estimated_time_remaining`	Duration	预计剩余时间

代码

源码位置: navigation_lab/navigation_lab/go_to_pose_demo.py

```
import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    goals = [
        (3.0, 0.0, 0.0),
        (3.0, 2.0, math.radians(90)),
        (0.0, 2.0, math.radians(180)),
        (0.0, 0.0, math.radians(-90)),
    ]

    for i, (x, y, yaw) in enumerate(goals):
        nav.get_logger().info(f'--- Goal {i+1}/{len(goals)} ---')
```

```

goal = make_pose_stamped('map', x, y, yaw)
nav.goToPose(goal)

while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    if feedback:
        nav.get_logger().info(
            f' distance_remaining={feedback.distance_remaining:.2f}m, '
            f'estimated_time={feedback.estimated_time_remaining.sec}s'
        )

    result = nav.getResult()
    if result == TaskResult.SUCCEEDED:
        nav.get_logger().info(f'Goal {i+1} reached!')
    elif result == TaskResult.CANCELED:
        nav.get_logger().warn(f'Goal {i+1} canceled')
    elif result == TaskResult.FAILED:
        nav.get_logger().error(f'Goal {i+1} failed')
    else:
        nav.get_logger().error(f'Goal {i+1} unknown result')

nav.get_logger().info('All goals completed!')
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```

# 确认仿真和 Nav2 已启动后
ros2 run navigation_lab go_to_pose_demo

```

观察

- 终端输出: distance_remaining 逐渐减小, estimated_time_remaining 动态更新
- RViz2 (nav2_default_view.rviz) 已预置以下显示:
- **Map** (/map) — 静态地图
- **Global Costmap** (/global_costmap/costmap) — 全局代价地图
- **Local Costmap** (/local_costmap/costmap) — 局部代价地图
- **Path** (/plan) — 全局规划路径 (绿色)
- **Local Plan** (/local_plan) — 局部规划轨迹 (蓝色)
- **Amcl Particle Swarm** (/particle_cloud) — AMCL 粒子云
- **Navigation 2 面板** — 提供 "2D Pose Estimate" 和 "2D Nav Goal" 工具
- 如果 RViz2 中未自动显示数据, 检查 Fixed Frame 是否为 map, 以及 Nav2 是否已正常启动

验收标准

- [] 机器人依次到达 4 个目标点
- [] 终端正确显示 distance_remaining 和 estimated_time_remaining
- [] 理解 goToPose 是异步调用, 需轮询 isTaskComplete() + getFeedback()

练习 2: followWaypoints 多点导航 (~15 分钟)

目标

掌握 FollowWaypoints action 的航点序列导航, 理解其反馈机制 (与 NavigateToPose 不同)。

知识点

- FollowWaypoints 的反馈仅包含 current_waypoint (uint32) — **没有** distance_remaining
- 机器人依次导航到序列中的每个航点
- 默认 waypoint_follower 到达每个航点后停留 200ms

代码

源码位置: navigation_lab/navigation_lab/follow_waypoints_demo.py

```
import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    waypoints = [
        make_pose_stamped('map', 2.0, 0.0, 0.0),
        make_pose_stamped('map', 2.0, 2.0, math.radians(90)),
        make_pose_stamped('map', 0.0, 2.0, math.radians(180)),
        make_pose_stamped('map', 0.0, 0.0, math.radians(-90)),
    ]

    nav.followWaypoints(waypoints)
```

```

while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    if feedback:
        nav.get_logger().info(
            f'current_waypoint={feedback.current_waypoint}/{len(waypoints)}'
        )

    result = nav.getResult()
    if result == TaskResult.SUCCEEDED:
        nav.get_logger().info('All waypoints reached!')
    elif result == TaskResult.CANCELED:
        nav.get_logger().warn('Waypoint task canceled')
    elif result == TaskResult.FAILED:
        nav.get_logger().error('Waypoint task failed')

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```
ros2 run navigation_lab follow_waypoints_demo
```

观察

- 终端输出 `current_waypoint` 递增: 1/4 → 2/4 → 3/4 → 4/4 → SUCCEEDED
- RViz2 中观察机器人按顺序通过所有航点

验收标准

- [] 机器人按顺序访问所有 4 个航点
- [] 终端正确显示 `current_waypoint` 序号
- [] 将 `stop_on_failure` 设为 `true` 再测试 (修改 `config/navigation_lab.yaml` 后重启 Nav2)

练习 3: 恢复行为与动态障碍 (~20 分钟)

目标

了解 Nav2 的恢复行为 (Spin, BackUp, ClearCostmap) 触发条件, 并通过合成 LaserScan 模拟动态障碍物。

3.1 恢复行为 Demo

源码位置: `navigation_lab/navigation_lab/recovery_demo.py`

```

import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    nav.get_logger().info('Navigating toward obstacles to trigger recovery...')
    goal = make_pose_stamped('map', 4.0, 0.0, 0.0)
    nav.goToPose(goal)

    while not nav.isTaskComplete():
        feedback = nav.getFeedback()
        if feedback:
            nav.get_logger().info(
                f'distance_remaining={feedback.distance_remaining:.2f}m, '
                f'navigation_time={feedback.navigation_time.sec}s, '
                f'estimated_time_remaining={feedback.estimated_time_remaining.sec}s'
            )

        result = nav.getResult()
        if result == TaskResult.SUCCEEDED:
            nav.get_logger().info('Goal reached!')
        elif result == TaskResult.CANCELED:
            nav.get_logger().warn('Canceled')
        elif result == TaskResult.FAILED:
            nav.get_logger().error('Failed - recovery behaviors were triggered')

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```
ros2 run navigation_lab recovery_demo
```

观察

- 当机器人在障碍物前卡住时, 自动触发 Spin (原地旋转) 或 BackUp (后退)
- 终端输出 `number_of_recoveries` 字段 (在 `NavigateToPose` feedback 中)
- RViz2 中观察代价地图清除和重新规划

3.2 合成激光扫描注入 (模拟动态障碍物)

由于 Gazebo Sim 的 `fake_laser` 基于预计算 2D 占用, 无法响应新生成的实体, 故使用合成 `LaserScan` 消息模拟动态障碍物。

源码位置: `navigation_lab/navigation_lab/scan_injector.py`

```
import math

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan

class ScanInjector(Node):
    def __init__(self):
        super().__init__('scan_injector')
        self.pub = self.create_publisher(LaserScan, '/scan', 10)
        self.timer = self.create_timer(0.1, self.publish_scan)
        self.angle = 0.0

    def publish_scan(self):
        self.angle += 0.02
        scan = LaserScan()
        scan.header.frame_id = 'laser'
        scan.header.stamp = self.get_clock().now().to_msg()
        scan.angle_min = -math.pi
        scan.angle_max = math.pi
        scan.angle_increment = math.pi / 180.0
        scan.time_increment = 1.0 / 360.0 / 180.0
        scan.scan_time = 0.1
        scan.range_min = 0.1
        scan.range_max = 3.5

        obstacle_x = 2.0 + 0.8 * math.sin(self.angle * 0.5)
        obstacle_y = 0.0 + 0.8 * math.cos(self.angle * 0.5)

        num_readings = int((scan.angle_max - scan.angle_min) / scan.angle_increment)
        ranges = []
        for i in range(num_readings):
            theta = scan.angle_min + i * scan.angle_increment
            dx = obstacle_x - 0.0
            dy = obstacle_y - 0.0
            dist = math.sqrt(dx * dx + dy * dy)
            angle_to_obs = math.atan2(dy, dx)
            angle_diff = abs(theta - angle_to_obs)
            if angle_diff < 0.05 and dist < 3.0 and dist > 0.1:
                ranges.append(dist)
            else:
                ranges.append(3.5)
        scan.ranges = ranges
        scan.intensities = [0.0] * num_readings
        self.pub.publish(scan)

def main():
    rclpy.init()
```



```
node = ScanInjector()
rclpy.spin(node)
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

运行

```
# 终端 4: 启动 scan 注入器
ros2 run navigation_lab scan_injector
```

观察

- 合成障碍物以圆形轨迹在 (2.0, 0.0) 附近移动
- RViz2: 局部代价地图上观察到移动的障碍物
- 机器人尝试绕过障碍物

验收标准

- [] 恢复 demo 显示 distance_remaining, navigation_time, estimated_time_remaining 全部正确
- [] 机器人卡住时触发 Spin/BackUp 行为
- [] scan_injector 发布的消息被 Nav2 接收, 代价地图中显示动态障碍物

练习 4: 导航过程监控 (~15 分钟)

目标

编写一个监控节点, 实时监听导航过程中的关键话题, 理解 Nav2 各模块间的数据流。

代码

源码位置: navigation_lab/navigation_lab/nav_monitor.py

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Path, Odometry
from geometry_msgs.msg import Twist
from nav2_msgs.msg import Costmap

class NavMonitor(Node):
    def __init__(self):
```

```

    super().__init__('nav_monitor')
    self.create_subscription(Path, '/plan', self.plan_cb, 10)
    self.create_subscription(Path, '/local_plan', self.local_plan_cb, 10)
    self.create_subscription(Twist, '/cmd_vel', self.cmd_cb, 10)
    self.create_subscription(Odometry, '/odom', self.odom_cb, 10)
    self.plan_count = 0
    self.last_plan_key = None

    def plan_cb(self, msg):
        if not msg.poses:
            return
        key = (msg.poses[0].pose.position.x, msg.poses[0].pose.position.y,
              msg.poses[-1].pose.position.x, msg.poses[-1].pose.position.y)
        if key != self.last_plan_key:
            self.plan_count += 1
            self.get_logger().info(f'New global plan #{self.plan_count}')
            self.last_plan_key = key

    def local_plan_cb(self, msg):
        self.get_logger().info(f'Local plan: {len(msg.poses)} poses')

    def cmd_cb(self, msg):
        self.get_logger().info(
            f'Cmd vel: linear={msg.linear.x:.2f} angular={msg.angular.z:.2f}'
        )

    def odom_cb(self, msg):
        pass

def main():
    rclpy.init()
    node = NavMonitor()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```

# 与任一导航 demo 同时运行
ros2 run navigation_lab nav_monitor

```

观察

- 全局规划重规划次数 — 障碍物触发新规划时计数递增
- /local_plan 的轨迹长度变化 — 反映避障机动
- /cmd_vel 速度指令 — 观察减速转向行为

验收标准

- [] 终端显示全局路径重规划事件

- [] 终端显示局部路径姿态数量
- [] 终端同步显示速度指令变化

练习 5: 多圈巡逻导航 (~15 分钟)

目标

使用 `followWaypoints` 实现多圈巡逻任务, 结合反馈判断每圈完成状态。

代码

源码位置: `navigation_lab/navigation_lab/waypoint_patrol.py`

```
import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    patrol_points = [
        make_pose_stamped('map', 2.0, 0.0, 0.0),
        make_pose_stamped('map', 2.0, 2.0, math.radians(90)),
        make_pose_stamped('map', 0.0, 2.0, math.radians(180)),
        make_pose_stamped('map', 0.0, 0.0, math.radians(-90)),
    ]

    loops = 2

    for lap in range(loops):
        nav.get_logger().info(f'=== Patrol lap {lap+1}/{loops} ===')

        nav.followWaypoints(patrol_points)

        while not nav.isTaskComplete():
            feedback = nav.getFeedback()
            if feedback:
                nav.get_logger().info(
                    f'current_waypoint={feedback.current_waypoint}/{len(patrol_points)}'
                )

        result = nav.getResult()
        if result == TaskResult.SUCCEEDED:
```

```

        nav.get_logger().info(f'Lap {lap+1} complete!')
    else:
        nav.get_logger().error(f'Lap {lap+1} interrupted')
        break

    nav.get_logger().info('Patrol finished!')
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```
ros2 run navigation_lab waypoint_patrol
```

观察

- 机器人完成 2 圈巡逻 (每圈 4 个航点)
- 终端显示 `current_waypoint` 从 `1/4` $\rightarrow$ `4/4`, 然后开始下一圈
- 如果某圈失败, 自动停止 (通过 `break`)

验收标准

- [] 机器人完成指定圈数的巡逻
- [] 每圈之间终端正确显示圈数信息
- [] 理解 `followWaypoints` 是阻塞式调用 — 必须在 `isTaskComplete()` 后检查结果

练习 6: 综合挑战 — 自主巡逻 + 动态避障 (~20 分钟)

目标

结合 `scan_injector` 和 `waypoint_patrol`, 观察导航系统在动态障碍物环境中的表现。

步骤

```

# 终端 1: Gazebo 仿真 (已运行)
# 终端 2: Nav2 (已运行)
# 终端 3: 巡逻程序
ros2 run navigation_lab waypoint_patrol
# 终端 4: 启动后等待几秒, 再启动 scan_injector
ros2 run navigation_lab scan_injector

```

观察

- 200. 无干扰时: 机器人正常巡逻
- 201. 注入合成障碍物后机器人绕行:
- 202. `/plan` 全局路径重规划
- 203. `/local_plan` 局部轨迹调整
- 204. `/cmd_vel` 速度变化 (减速 + 转向)
- 205. 停止 `scan_injector` 后: 机器人恢复原路径

思考题

- `followWaypoints` 和循环 `goToPose` 的区别?
- 如果某个航点不可达, `stop_on_failure` 两种设置 (`true/false`) 的行为差异?
- 为什么 Gazebo 中 `spawn` 实体不显示在 `LaserScan` 中? (提示: `fake_laser` 的原理)

验收标准

- [] 动态障碍物注入后, 巡逻路径改变
- [] 障碍物移除后, 机器人恢复原计划
- [] 终端显示全局路径重规划和避障机动事件

常用调试命令

```
# 查看所有话题
ros2 topic list

# 查看导航 action 反馈 (实时)
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose "{}"

# 查看行为树状态
ros2 param get /bt_navigator default_nav_to_pose_bt_xml

# 查看生命周期节点状态
ros2 service call /bt_navigator/get_state lifecycle_msgs/srv/GetState "{}"

# 手动清除代价地图
ros2 service call /global_costmap/clear_entirely_global_costmap nav2_msgs/srv/ClearEntireCostmap "{}"

# 查看机器人位姿
```

```
ros2 topic echo /amcl_pose

# 查看速度指令
ros2 topic echo /cmd_vel
```

常见问题

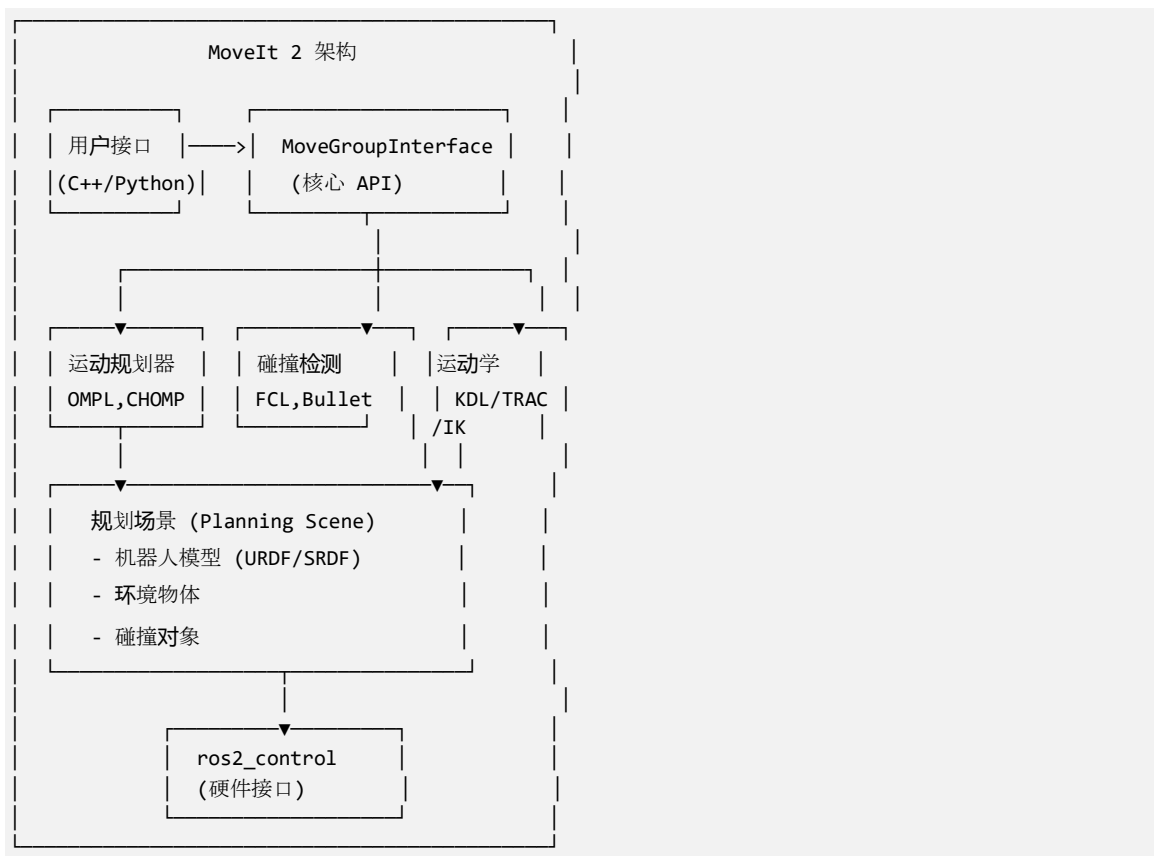
问题	原因	解决
<code>`waitUntilNav2Active()`</code> 卡住	未发布初始位姿	确保 <code>`setInitialPose()`</code> 在 <code>`waitUntilNav2Active()`</code> 之前调用, 或在 RViz 中用 2D Pose Estimate
机器人不移动	lifecycle 节点未激活	检查 launch 是否传了 <code>`autostart:=true`</code>
<code>`followWaypoints`</code> 反馈无 <code>`distance_remaining`</code>	<code>`FollowWaypoints`</code> action 只提供 <code>`current_waypoint`</code>	改用 <code>`NavigateToPose`</code> 的 feedback 获取距离
Gazebo 中实体不影响导航	<code>`fake_laser`</code> 不响应 Gazebo 实体	使用 scan_injector 合成 LaserScan 消息
导航目标被拒绝	目标点在障碍物内或不可达	检查 RViz2 代价地图, 确认目标点在 free 区域

第 12 章 MoveIt 2 机械臂编程

12.1 知识要点

12.1.1 MoveIt 2 架构概览

MoveIt 2 是 ROS 2 最主流的机械臂运动规划框架。



核心概念:

- **MoveGroup:** 机械臂/末端执行器的抽象接口
- **Planning Scene:** 完整的环境模型 (机器人 + 障碍物 + 物体)
- **Motion Planner:** 运动规划算法库
- **Kinematics Solver:** 正/逆运动学求解器

12.1.2 MoveItPy Python API

MoveItPy 是 MoveIt 2 的 Python 封装:

```

#!/usr/bin/env python3
"""
MoveItPy 基础示例 - 机械臂运动控制
需要: pip install moveit, ros-humble-moveit-py
"""
import rclpy
from rclpy.node import Node

# MoveItPy 核心导入
from moveit.core.robot_state import RobotState
from moveit.planning import (
    MoveItPy,
    PlanningComponent,
    SinglePlanParameters,
    MultiPipelinePlanRequestParameters,
)

class MoveItPyArmController(Node):
    """基于 MoveItPy 的机械臂控制器"""
    def __init__(self):
        super().__init__('arm_controller')

        # 初始化 MoveItPy (加载配置)
        self.moveit = MoveItPy(node_name='moveit_py')
        # 获取 PlanningComponent (对应 MoveGroup)
        self.arm = PlanningComponent(
            self.moveit, 'panda_arm', 'panda_hand'
        )
        self.get_logger().info('MoveItPy 初始化完成')

    def plan_joint_goal(self, joint_values: list):
        """
        规划到指定关节角
        参数:
            joint_values: [j1, j2, j3, j4, j5, j6, j7] 关节角 (弧度)
        """
        # 创建关节目标
        self.arm.set_start_state_to_current_state()
        joint_goal = {f'panda_joint{i+1}': joint_values[i] for i in range(7)}

        # 规划
        plan_result = self.arm.plan(
            target=joint_goal,
            planner_id='RRTConnectkConfigDefault',
        )

        if plan_result:
            self.get_logger().info(f'规划成功! 轨迹点数: {len(plan_result.trajectory)}')
            return plan_result
        else:
            self.get_logger().error('规划失败!')
            return None

    def plan_pose_goal(self, x: float, y: float, z: float,

```



```

        qx: float, qy: float, qz: float, qw: float):
    """
    规划到指定位姿（逆运动学）
    参数:
        x, y, z: 位置 (m)
        qx, qy, qz, qw: 姿态四元数
    """
    from geometry_msgs.msg import PoseStamped

    pose = PoseStamped()
    pose.header.frame_id = 'panda_link0'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.x = qx
    pose.pose.orientation.y = qy
    pose.pose.orientation.z = qz
    pose.pose.orientation.w = qw

    self.arm.set_start_state_to_current_state()
    plan_result = self.arm.plan(
        target=pose.pose,
        planner_id='RRTConnectkConfigDefault',
        constraints=None,
        planning_time=5.0,
    )

    if plan_result:
        self.get_logger().info(
            f'位姿规划成功! 时间: {plan_result.planning_time:.3f}s'
        )
    return plan_result

def plan_cartesian_path(self, waypoints: list, step: float = 0.01,
                        jump_threshold: float = 0.0):
    """
    笛卡尔空间路径规划（直线路径）
    参数:
        waypoints: 路径点列表 [(x0,y0,z0,q0), ...]
        step: 插值步长 (m)
        jump_threshold: 关节空间跳变阈值, 0 表示禁用
    返回: 轨迹或 None
    """
    self.arm.set_start_state_to_current_state()
    plan_result = self.arm.compute_cartesian_path(
        waypoints=waypoints,
        step=step,
        jump_threshold=jump_threshold,
    )
    return plan_result

def execute(self, plan_result):
    """执行规划结果"""
    if plan_result:
        self.arm.execute(plan_result)

```

```

        self.get_logger().info('轨迹执行完成!')

def stop(self):
    """停止机械臂运动"""
    self.arm.stop()

def go_home(self):
    """回到 Home 位姿"""
    home_joints = [0.0, -0.785, 0.0, -2.356, 0.0, 1.571, 0.785]
    plan = self.plan_joint_goal(home_joints)
    if plan:
        self.execute(plan)

def pick_and_place_demo(self):
    """
    Pick and Place 完整流程
    1. 移动到预抓取位置 (pre-grasp)
    2. 下降到抓取位置 (grasp)
    3. 抓取物体
    4. 提升 (lift)
    5. 移动到放置位置 (place)
    6. 释放物体
    """
    # 1. 预抓取位置 (目标上方 10cm)
    pre_grasp = self.plan_pose_goal(
        0.5, 0.0, 0.3, # x, y, z
        0.0, 1.0, 0.0, 0.0 # 夹爪朝下
    )
    if pre_grasp:
        self.execute(pre_grasp)

    # 2. 笛卡尔下降到抓取位置
    import numpy as np
    cartesian_waypoints = []
    for z in np.arange(0.3, 0.1, -0.01):
        cartesian_waypoints.append([0.5, 0.0, z, 0.0, 1.0, 0.0, 0.0])

    if self.move_to_waypoints:
        cart_path = self.plan_cartesian_path(cartesian_waypoints)
        if cart_path:
            self.execute(cart_path)

    # 3. 抓取物体 (闭合夹爪)
    self.attach_object('target_object')

    # 4. 笛卡尔提升
    cartesian_lift = []
    for z in np.arange(0.1, 0.3, 0.01):
        cartesian_lift.append([0.5, 0.0, z, 0.0, 1.0, 0.0, 0.0])
    lift_path = self.plan_cartesian_path(cartesian_lift)
    if lift_path:
        self.execute(lift_path)

    # 5. 移动到放置位置

```

```

        place_pose = self.plan_pose_goal(
            -0.3, -0.5, 0.3,
            0.0, 1.0, 0.0, 0.0
        )
        if place_pose:
            self.execute(place_pose)

# 6. 释放物体
self.detach_object('target_object')

# 7. 回到 Home
self.go_home()

```

12.1.3 运动规划器

OMPL (Open Motion Planning Library):

```

class OMPLPlannerConfig:
    """
    OMPL 规划器配置说明
    MoveIt 2 默认使用 OMPL 作为规划后端
    """
    # 常用 OMPL 规划器
    planners = {
        'RRTConnect':      '双向 RRT 连接, 快速, 非最优',
        'RRTstar':         'RRT* 渐进最优, 较慢',
        'PRM':             '概率路图, 适合多次查询',
        'PRMstar':         'PRM* 渐进最优',
        'EST':             '扩展空间树',
        'SBL':             '基于采样的均衡树',
        'KPIECE':          '动力学空间下的投影扩展',
        'TRRT':            '过渡态 RRT, 适合狭窄通道',
        'BiTRRT':          '双向 TRRT',
    }

    # 配置示例 (ompl_planning.yaml)
    default_config = """
    RRTConnectkConfigDefault:
      type: geometric::RRTConnect
      range: 0.0 # 0 = 无限制
    """

# STOMP (Stochastic Trajectory Optimization for Motion Planning)
# 基于优化的规划器, 通过随机噪声优化轨迹平滑性和避障
class STOMPPlanner:
    """
    STOMP: 随机轨迹优化
    - 不需要显式碰撞检测 (代价函数内处理)
    - 适合平滑的关节空间轨迹
    """
    pass

```

```
# CHOMP (Covariant Hamiltonian Optimization for Motion Planning)
class CHOMPPanner:
    """
    CHOMP: 协变哈密顿优化
    - 基于梯度的轨迹优化
    - 同时优化平滑性和障碍物距离
    """
    pass
```

12.1.4 碰撞检测与规划场景

```
from moveit.core.planning_scene import PlanningScene
from shape_msgs.msg import SolidPrimitive
from geometry_msgs.msg import Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject

class SceneManager:
    """规划场景管理 - 添加/移除碰撞物体"""
    def __init__(self, moveit_instance):
        self.moveit = moveit_instance
        self.scene = PlanningScene(moveit_instance)

    def add_box(self, name: str, size: tuple, pose: tuple,
                frame_id: str = 'world'):
        """
        添加碰撞盒子
        参数:
            name: 物体名称
            size: (x, y, z) 尺寸 (m)
            pose: (x, y, z, qx, qy, qz, qw) 位姿
        """
        obj = CollisionObject()
        obj.id = name
        obj.header.frame_id = frame_id

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [float(s) for s in size]
        obj.primitives.append(primitive)

        obj_pose = Pose()
        obj_pose.position.x = pose[0]
        obj_pose.position.y = pose[1]
        obj_pose.position.z = pose[2]
        obj_pose.orientation.x = pose[3]
        obj_pose.orientation.y = pose[4]
        obj_pose.orientation.z = pose[5]
        obj_pose.orientation.w = pose[6]
        obj.primitive_poses.append(obj_pose)
        obj.operation = CollisionObject.ADD

        self.scene.apply_collision_object(obj)
        print(f'碰撞物体 {name} 已添加')

    def remove_object(self, name: str):
```

```

        """移除碰撞物体"""
        obj = CollisionObject()
        obj.id = name
        obj.operation = CollisionObject.REMOVE
        self.scene.apply_collision_object(obj)

def attach_to_end_effector(self, obj_name: str):
    """将物体附着在末端执行器上 (模拟抓取)"""
    obj = AttachedCollisionObject()
    obj.object.id = obj_name
    obj.link_name = 'panda_hand' # 末端 link
    obj.object.operation = CollisionObject.ADD
    self.scene.apply_attached_collision_object(obj)

def detach_from_end_effector(self, obj_name: str):
    """从末端执行器分离物体"""
    obj = AttachedCollisionObject()
    obj.object.id = obj_name
    obj.object.operation = CollisionObject.REMOVE
    self.scene.apply_attached_collision_object(obj)

def check_self_collision(self) -> bool:
    """检测自碰撞"""
    current_state = self.moveit.get_robot_state()
    collision_result = self.scene.check_self_collision(current_state)
    return collision_result.collision

def check_collision_with_objects(self) -> bool:
    """检测与场景物体的碰撞"""
    current_state = self.moveit.get_robot_state()
    collision_result = self.scene.check_collision(
        current_state, verbose=True
    )
    if collision_result.collision:
        for contact in collision_result.contacts:
            print(f'碰撞: {contact.body_1} <-> {contact.body_2}')
    return collision_result.collision

def add_table_and_shelf(self):
    """添加典型场景: 桌子 + 货架"""
    # 地面桌子 (80cm x 60cm)
    self.add_box(
        'table',
        (0.8, 0.6, 0.05),
        (0.5, 0.0, 0.35, 0.0, 0.0, 0.0, 1.0)
    )
    # 货架隔板
    self.add_box(
        'shelf_board',
        (0.8, 0.02, 0.6),
        (0.5, 0.3, 0.3, 0.0, 0.0, 0.0, 1.0)
    )

```

12.1.5 运动学求解器

```
import numpy as np
from ikpy.chain import Chain
from ikpy.link import OriginLink, URDFLink

class IKSolverDemo:
    """
    正运动学 (FK) / 逆运动学 (IK) 求解示例
    使用 ikpy 库 (教学参考)
    """
    def __init__(self):
        # 构建 7-DOF Panda 手臂链
        self.chain = Chain(name='panda', links=[
            OriginLink(),
            URDFLink(name='joint1', origin_translation=[0, 0, 0.333],
                    origin_orientation=[0, 0, 0], rotation=[0, 0, 1]),
            URDFLink(name='joint2', origin_translation=[0, 0, 0],
                    origin_orientation=[0, 0, 0], rotation=[0, 1, 0]),
            URDFLink(name='joint3', origin_translation=[0, 0, 0.316],
                    origin_orientation=[0, 0, 0], rotation=[0, 0, 1]),
            URDFLink(name='joint4', origin_translation=[0.0825, 0, 0],
                    origin_orientation=[0, 0, 0], rotation=[0, 1, 0]),
            URDFLink(name='joint5', origin_translation=[-0.0825, 0, 0.384],
                    origin_orientation=[0, 0, 0], rotation=[0, 0, 1]),
            URDFLink(name='joint6', origin_translation=[0, 0, 0],
                    origin_orientation=[0, 0, 0], rotation=[0, 1, 0]),
            URDFLink(name='joint7', origin_translation=[0.088, 0, 0],
                    origin_orientation=[0, 0, 0], rotation=[0, 0, 1]),
        ])

    def forward_kinematics(self, joints: list) -> np.ndarray:
        """
        正运动学: 关节角 → 末端位姿
        参数:
            joints: 7 个关节角 (弧度)
        返回:
            (4,4) 齐次变换矩阵
        """
        fk = self.chain.forward_kinematics(joints)
        return fk

    def inverse_kinematics(self, target_position: list,
                          target_orientation: list = None,
                          initial_joints: list = None) -> np.ndarray:
        """
        逆运动学: 目标位姿 → 关节角
        参数:
            target_position: [x, y, z] 目标位置
            target_orientation: [rx, ry, rz] 目标姿态 (轴角)
            initial_joints: 初始猜测关节角
        返回:
            7 个关节角 (弧度)
        """
```

```

"""
if initial_joints is None:
    initial_joints = [0.0] * 7
ik = self.chain.inverse_kinematics(
    target_position,
    target_orientation,
    initial_position=initial_joints
)
return ik

def test_reachable(self, x: float, y: float, z: float):
    """测试目标位置是否在可达空间内"""
    try:
        ik = self.inverse_kinematics([x, y, z])
        fk = self.forward_kinematics(ik.tolist())
        pos_err = np.linalg.norm(fk[:3, 3] - np.array([x, y, z]))
        return True, pos_err, ik
    except Exception:
        return False, float('inf'), None

```

12.1.6 ros2_control 接口集成

```

#!/usr/bin/env python3
"""
ros2_control 控制器切换与管理
"""
import rclpy
from rclpy.node import Node
from controller_manager_msgs.srv import (
    SwitchController, ListControllers, LoadController
)
from controller_manager_msgs.msg import ControllerState

class ControllerManager(Node):
    """ros2_control 控制器管理器"""
    def __init__(self):
        super().__init__('controller_manager_client')
        self.switch_srv = self.create_client(
            SwitchController, '/controller_manager/switch_controller'
        )
        self.list_srv = self.create_client(
            ListControllers, '/controller_manager/list_controllers'
        )

    async def list_controllers(self) -> list:
        """列出所有控制器及状态"""
        future = self.list_srv.call_async(ListControllers.Request())
        result = await future
        for ctrl in result.controller:
            self.get_logger().info(
                f'{ctrl.name}: {ctrl.state} (类型: {ctrl.type})'
            )
        return result.controller

    async def switch_to_position_controller(self):
        """
        切换到位置控制器 (轨迹执行用)

```

```

        MoveIt 需要 joint_trajectory_controller
        """

        req = SwitchController.Request()
        req.activate_controllers = ['joint_trajectory_controller']
        req.deactivate_controllers = ['joint_state_broadcaster']
        req.strictness = SwitchController.Request.STRICT
        future = self.switch_srv.call_async(req)
        result = await future

        self.get_logger().info(f'控制器切换结果: {result.ok}')

def launch_control_setup():
    """
    MoveIt 2 + ros2_control 完整启动流程
    """
    commands = """
    # 1. 启动机器人描述和硬件接口
    ros2 launch robot_description display.launch.py use_fake_hardware:=true

    # 2. 启动 MoveIt 2
    ros2 launch moveit2 moveit.launch.py

    # 3. 启动控制器管理
    ros2 run controller_manager spawner joint_state_broadcaster
    ros2 run controller_manager spawner joint_trajectory_controller

    # 4. 启动 Python 运动控制节点
    python3 arm_control_demo.py
    """
    print(commands)

```

12.1.7 抓取规划

```

import numpy as np

class GraspPlanner:
    """
    抓取规划器 - 生成候选抓取位姿
    参考: moveit_msgs/Grasp 消息定义
    """
    def __init__(self, obj_center: np.ndarray, obj_size: np.ndarray):
        """
        参数:
            obj_center: 物体中心 [x, y, z]
            obj_size: 物体尺寸 [sx, sy, sz]
        """
        self.obj_center = obj_center
        self.obj_size = obj_size

    def generate_top_grasps(self, n: int = 8) -> list:
        """
        生成顶部抓取候选
        在物体上方生成 n 个均匀分布方向的抓取姿态

```



```

"""
grasps = []
for i in range(n):
    angle = 2 * np.pi * i / n
    # 抓取位姿: 在物体上方, 夹爪朝下
    grasp = {
        'position': [
            self.obj_center[0],
            self.obj_center[1],
            self.obj_center[2] + self.obj_size[2] / 2 + 0.05,
        ],
        'orientation': self._grasp_orientation(angle),
        'pre_grasp_approach': [
            0, 0, -0.05 # 沿 -Z 接近
        ],
        'post_grasp_retreat': [
            0, 0, 0.05 # 沿 +Z 撤回
        ],
    }
    grasps.append(grasp)
return grasps

def generate_side_grasps(self, n: int = 8) -> list:
    """
    生成侧面抓取候选
    """
    grasps = []
    for i in range(n):
        angle = 2 * np.pi * i / n
        dx = np.cos(angle) * (self.obj_size[0] / 2 + 0.1)
        dy = np.sin(angle) * (self.obj_size[1] / 2 + 0.1)
        grasp = {
            'position': [
                self.obj_center[0] + dx,
                self.obj_center[1] + dy,
                self.obj_center[2],
            ],
            'orientation': self._side_grasp_orientation(angle),
            'pre_grasp_approach': [
                -np.cos(angle) * 0.1, -np.sin(angle) * 0.1, 0
            ],
        }
        grasps.append(grasp)
    return grasps

def _grasp_orientation(self, yaw: float) -> list:
    """生成夹爪朝向四元数"""
    import math
    return [0.0, math.cos(yaw), 0.0, math.sin(yaw)]

def _side_grasp_orientation(self, angle: float) -> list:
    """生成侧面抓取朝向"""
    import math
    half = math.pi / 4
    return [0.0, math.sin(half), 0.0, math.cos(half)]

```

12.1.8 MoveIt 2 配置与启动

```
# moveit_controller.yaml (控制器配置)
controller_names:
  - joint_trajectory_controller

joint_trajectory_controller:
  type: "FollowJointTrajectory"
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5
    - joint6
    - joint7
  action_monitor_rate: 20
  constraints:
    goal_time: 0.6
    stopped_velocity_tolerance: 0.05
    joint1: {trajectory: 0.1, goal: 0.1}
    joint2: {trajectory: 0.1, goal: 0.1}
    joint3: {trajectory: 0.1, goal: 0.1}
```

12.2 练习题

- 原理解析题:** 说明 MoveIt 2 中 Planning Scene、MoveGroup 和 Motion Planner 三者之间的关系。为什么需要维护一个 Planning Scene 而不是每次规划时直接读取传感器数据?
- 运动学题:** 给定 Panda 机械臂的 DH 参数, 手工推导正运动学 (FK) 表达式, 并编写 Python 代码实现 7-DOF 机械臂的正运动学计算。
- 编程题:** 使用 MoveItPy 编写一个 "画正方形" 程序: 末端执行器在水平面上绘制边长 0.2m 的正方形 (笛卡尔空间轨迹), 要求 4 条边各用 20 个路径点。
- 碰撞题:** 在规划场景中添加 3 个随机位置/尺寸的障碍物盒子, 编写程序验证机械臂从 A 点到 B 点的规划能否正确避开所有障碍物, 并输出碰撞检测结果。
- 设计题:** 设计一个 "传送带分拣" 场景: 传送带上有红色和蓝色两种物体, 机械臂需要将红色物体分拣到左侧箱子、蓝色物体分拣到右侧箱子。写出完整的程序框架 (包括视觉感知接口定义)。

6. 综合题: 比较 OMPL-RRTConnect、STOMP 和 CHOMP 三种规划器的优缺点和适用场景。
针对焊接任务 (需要平滑轨迹且靠近工件) 应选择哪种规划器? 说明理由。

实验 12:

第 12 章 实验手册: MoveIt 2 机械臂编程

环境说明

本实验使用 XBot-U 手臂 + Gazebo 仿真, 预装 moveit_py 和 ros-humble-moveit。

练习 1: MoveIt2 配置加载与单轴运动 (~30 分钟)

目标

加载 MoveIt 2 配置, 掌握 PlanningComponent 规划与执行流程, 完成单关节运动。

步骤

1.1 启动 MoveIt 2 仿真环境

```
# 终端 1: 启动 Gazebo + 机械臂仿真
ros2 launch xbot_sim xbot_arm_gazebo.launch.py

# 终端 2: 启动 MoveIt 2 (含 ros2_control)
ros2 launch xbot_moveit_config moveit.launch.py \
  use_sim_time:=true use_fake_hardware:=false

# 终端 3: 启动 RViz
rviz2 -d src/courseware/rviz/moveit_view.rviz
```

1.2 编写基础运动控制节点

```
#!/usr/bin/env python3
"""练习 1: MoveIt2 单轴运动与状态获取"""
import rclpy
from rclpy.node import Node
import numpy as np
import time

class BasicArmMover(Node):
    """基础机械臂运动控制器 (使用 MoveItPy)"""
```

```

def __init__(self):
    super().__init__('basic_arm_mover')

    # 初始化 MoveItPy
    from moveit.planning import MoveItPy, PlanningComponent
    self.moveit = MoveItPy(node_name='moveit_py')
    self.arm = PlanningComponent(
        self.moveit, 'xbot_arm', 'xbot_gripper'
    )
    self.get_logger().info('MoveItPy 初始化完成')

def get_current_joints(self) -> dict:
    """获取当前关节角"""
    state = self.moveit.get_robot_state()
    joint_names = [
        'arm_joint1', 'arm_joint2', 'arm_joint3',
        'arm_joint4', 'arm_joint5', 'arm_joint6'
    ]
    joints = {}
    for name in joint_names:
        joints[name] = state.get_joint_positions(name)
    return joints

def get_current_ee_pose(self) -> list:
    """获取当前末端执行器位姿"""
    state = self.moveit.get_robot_state()
    ee_pose = state.get_pose('ee_link')
    p = ee_pose.position
    o = ee_pose.orientation
    return [p.x, p.y, p.z, o.x, o.y, o.z, o.w]

def move_single_joint(self, joint_name: str, target_rad: float,
                      speed: float = 1.0):
    """
    控制单个关节运动
    参数:
        joint_name: 关节名称
        target_rad: 目标角度 (弧度)
        speed: 速度因子 (1.0 = 全速)
    """
    current = self.get_current_joints()
    if joint_name not in current:
        self.get_logger().error(f'未知关节: {joint_name}')
        return False

    # 设置目标 (仅改变指定关节)
    target = dict(current)
    target[joint_name] = target_rad

    self.get_logger().info(
        f'{joint_name}: {current[joint_name]:.3f} → {target_rad:.3f}'
    )

    # 规划
    self.arm.set_start_state_to_current_state()
    plan = self.arm.plan(

```

```

        target=target,
        planner_id='RRTConnectkConfigDefault',
        planning_time=2.0,
        max_velocity_scaling_factor=speed,
    )

    if plan:
        self.arm.execute(plan)
        self.get_logger().info(f'{joint_name} 运动完成!')
        return True
    else:
        self.get_logger().error(f'{joint_name} 规划失败!')
        return False

def scan_joint_range(self, joint_name: str, start: float,
                    end: float, steps: int = 5):
    """
    扫描关节运动范围（检验关节限制）
    参数:
        joint_name: 关节名称
        start, end: 扫描范围（弧度）
        steps: 步数
    """
    self.get_logger().info(f'扫描关节 {joint_name} [{start:.2f}, {end:.2f}]')
    angles = np.linspace(start, end, steps)
    for i, angle in enumerate(angles):
        success = self.move_single_joint(joint_name, angle, speed=0.5)
        if not success:
            self.get_logger().warn(f'角度 {angle:.2f} 规划失败!')
            break
        time.sleep(1.5)

def go_home(self):
    """回到 Home 位姿"""
    home = {
        'arm_joint1': 0.0,
        'arm_joint2': -0.5,
        'arm_joint3': 0.0,
        'arm_joint4': -1.5,
        'arm_joint5': 0.0,
        'arm_joint6': 1.0,
    }
    self.arm.set_start_state_to_current_state()
    plan = self.arm.plan(target=home, planner_id='RRTConnectkConfigDefault')
    if plan:
        self.arm.execute(plan)
        self.get_logger().info('已回到 Home 位姿')

def run_demo(self):
    """演示：逐个关节运动"""
    self.get_logger().info('=== 单关节运动演示 ===')

    # 回到 Home
    self.go_home()
    time.sleep(2.0)

```

```

# 逐个关节运动
self.move_single_joint('arm_joint1', 1.0)
time.sleep(1.5)
self.move_single_joint('arm_joint1', -1.0)
time.sleep(1.5)
self.move_single_joint('arm_joint2', -1.0)
time.sleep(1.5)
self.move_single_joint('arm_joint3', 1.0)
time.sleep(1.5)
self.move_single_joint('arm_joint4', -2.0)

# 回到 Home
self.go_home()
ee_pose = self.get_current_ee_pose()
self.get_logger().info(
    f'末端位姿: ({ee_pose[0]:.3f}, {ee_pose[1]:.3f}, {ee_pose[2]:.3f})'
)

def main():
    rclpy.init()
    node = BasicArmMover()
    node.run_demo()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

验收标准

- [] MoveItPy 成功加载配置
- [] 各关节可独立规划并执行
- [] 能获取当前关节角和末端位姿
- [] Home 位姿正确恢复

练习 2: 笛卡尔路径规划 (~30 分钟)

目标

掌握笛卡尔空间直线路径规划, 使用末端执行器画出指定形状。

步骤

2.1 编写笛卡尔路径程序

```
#!/usr/bin/env python3
"""练习 2: 笛卡尔路径规划 - 画正方形/圆形"""
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
import numpy as np
import time
import copy

class CartesianPathPlanner(Node):
    """笛卡尔路径规划器"""
    def __init__(self):
        super().__init__('cartesian_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'xbot_arm', 'xbot_gripper')
        self.get_logger().info('笛卡尔规划器已就绪')

    def get_current_pose(self) -> Pose:
        """获取当前末端位姿"""
        state = self.moveit.get_robot_state()
        ee_pose = state.get_pose('ee_link')
        pose = Pose()
        pose.position = copy.deepcopy(ee_pose.position)
        pose.orientation = copy.deepcopy(ee_pose.orientation)
        return pose

    def compute_cartesian_path(self, waypoints: list[Pose],
                               step: float = 0.01,
                               jump_threshold: float = 2.0):
        """
        计算笛卡尔路径
        参数:
            waypoints: 目标位姿序列
            step: 插值分辨率 (m)
            jump_threshold: 关节跳变检测阈值
        """
        result = self.arm.compute_cartesian_path(
            waypoints=waypoints,
            step=step,
            jump_threshold=jump_threshold,
        )
        if result:
            # 检查路径完成度 (0~1)
            fraction = result.fraction
            self.get_logger().info(f'笛卡尔路径完成度: {fraction*100:.1f}%')
            return result if fraction > 0.99 else None
        return None
```



```

def draw_square(self, side_length: float = 0.15,
                 plane_height: float = 0.3):
    """
    在水平面上画正方形
    参数:
        side_length: 边长 (m)
        plane_height: 绘图平面高度 (m)
    """
    self.get_logger().info(f'绘制 {side_length}m 正方形, 高度 {plane_height}m')

    # 移动到起点
    start_pose = self.get_current_pose()
    start_pose.position.x -= side_length / 2
    start_pose.position.y -= side_length / 2
    start_pose.position.z = plane_height

    plan = self.arm.plan(target=start_pose)
    if plan:
        self.arm.execute(plan)
    else:
        self.get_logger().error('无法到达起点')
        return

    # 正方形 4 条边
    edges = [
        ( side_length,  0, 0),      # 沿 X+
        ( 0,  side_length, 0),     # 沿 Y+
        (-side_length,  0, 0),     # 沿 X-
        ( 0, -side_length, 0),     # 沿 Y-
    ]

    for edge_idx, (dx, dy, dz) in enumerate(edges):
        self.get_logger().info(f'绘制边 {edge_idx+1}/4')

        edge_waypoints = []
        current = self.get_current_pose()
        n_steps = 20 # 每边 20 个路径点

        for i in range(1, n_steps + 1):
            t = i / n_steps
            wp = Pose()
            wp.position.x = current.position.x + dx * t
            wp.position.y = current.position.y + dy * t
            wp.position.z = current.position.z + dz * t
            wp.orientation = copy.deepcopy(current.orientation)
            edge_waypoints.append(wp)

        result = self.compute_cartesian_path(edge_waypoints)
        if result:
            self.arm.execute(result)
            self.get_logger().info(f'边 {edge_idx+1} 完成 ({n_steps} 个路径点)')
        else:
            self.get_logger().error(f'边 {edge_idx+1} 路径不可达')
            break

```

```

        self.get_logger().info('正方形绘制完成!')

def draw_circle(self, radius: float = 0.1, plane_height: float = 0.3,
                n_points: int = 72):
    """
    在水平面上画圆
    参数:
        radius: 半径 (m)
        plane_height: 绘图平面高度
        n_points: 路径点数
    """
    self.get_logger().info(f'绘制半径 {radius}m 圆, {n_points} 个点')

    # 移动到圆起点
    start = self.get_current_pose()
    start.position.x += radius
    start.position.z = plane_height

    plan = self.arm.plan(target=start)
    if plan:
        self.arm.execute(plan)
    else:
        return

    # 圆形路径点
    waypoints = []
    center = self.get_current_pose()
    cx, cy = center.position.x, center.position.y

    for i in range(1, n_points + 1):
        angle = 2 * np.pi * i / n_points
        wp = Pose()
        wp.position.x = cx + radius * np.cos(angle)
        wp.position.y = cy + radius * np.sin(angle)
        wp.position.z = plane_height
        wp.orientation = copy.deepcopy(center.orientation)
        waypoints.append(wp)

    result = self.compute_cartesian_path(waypoints)
    if result:
        self.arm.execute(result)
        self.get_logger().info('圆形绘制完成!')

def run_demo(self):
    """演示: 画正方形 + 圆形"""
    # 先回到安全位置
    self.arm.set_start_state_to_current_state()
    home = {'arm_joint1': 0.0, 'arm_joint2': -0.5, 'arm_joint3': 0.0,
            'arm_joint4': -1.5, 'arm_joint5': 0.0, 'arm_joint6': 1.0}
    plan = self.arm.plan(target=home)
    if plan:
        self.arm.execute(plan)
    time.sleep(2.0)

    # 画正方形
    self.draw_square(side_length=0.15, plane_height=0.25)

```

```

        time.sleep(1.0)

        # 画圆
        self.draw_circle(radius=0.08, plane_height=0.25)
        time.sleep(1.0)

        # 回到 Home
        plan = self.arm.plan(target=home)
        if plan:
            self.arm.execute(plan)

def main():
    rclpy.init()
    node = CartesianPathPlanner()
    node.run_demo()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

验收标准

- [] 正方形 4 边连续画出, 无明显中断
- [] 圆形 72 个路径点完整执行
- [] 路径完成度 fraction > 99%
- [] 机械臂运动平稳, 无关节跳变

练习 3: Pick & Place 抓取任务 (~30 分钟)

目标

实现完整的 Pick & Place 流程: 检测物体 → 规划抓取 → 抓取 → 移动到目标 → 放置。

步骤

3.1 编写 Pick & Place 完整流程

```

#!/usr/bin/env python3
"""练习 3: Pick & Place 抓取放置任务"""
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive

```

```

import numpy as np
import copy
import time

class PickPlaceRunner(Node):
    """Pick & Place 任务执行器"""
    def __init__(self):
        super().__init__('pick_place_runner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'xbot_arm', 'xbot_gripper')
        self.gripper = PlanningComponent(
            self.moveit, 'xbot_arm', 'xbot_gripper'
        )
        self.scene = self.moveit.get_planning_scene_monitor()

        # 物体参数
        self.object_name = 'target_cube'
        self.object_size = (0.04, 0.04, 0.04) # 4cm 正方体
        self.get_logger().info('Pick & Place 执行器已就绪')

    def add_box_to_scene(self, name: str, size: tuple,
                        x: float, y: float, z: float):
        """在规划场景中添加碰撞物体"""
        obj = CollisionObject()
        obj.id = name
        obj.header.frame_id = 'world'

        box = SolidPrimitive()
        box.type = SolidPrimitive.BOX
        box.dimensions = [float(s) for s in size]
        obj.primitives.append(box)

        obj_pose = Pose()
        obj_pose.position.x = x
        obj_pose.position.y = y
        obj_pose.position.z = z
        obj_pose.orientation.w = 1.0
        obj.primitive_poses.append(obj_pose)
        obj.operation = CollisionObject.ADD

        self.scene.apply_collision_object(obj)
        self.get_logger().info(f'添加物体: {name} ({x:.2f}, {y:.2f}, {z:.2f})')

    def remove_object(self, name: str):
        """移除物体"""
        obj = CollisionObject()
        obj.id = name
        obj.operation = CollisionObject.REMOVE
        self.scene.apply_collision_object(obj)

    def attach_object(self, obj_name: str, link: str = 'gripper_link'):
        """将物体附着到夹爪 (模拟抓取)"""
        obj = AttachedCollisionObject()
        obj.object.id = obj_name
        obj.link_name = link
        obj.object.operation = CollisionObject.ADD

```

```

        self.scene.apply_attached_collision_object(obj)
        self.get_logger().info(f'物体 {obj_name} 已附着到 {link}')

def detach_object(self, obj_name: str, link: str = 'gripper_link'):
    """从夹爪分离物体 (模拟放置)"""
    obj = AttachedCollisionObject()
    obj.object.id = obj_name
    obj.link_name = link
    obj.object.operation = CollisionObject.REMOVE
    self.scene.apply_attached_collision_object(obj)
    self.get_logger().info(f'物体 {obj_name} 已分离')

def open_gripper(self):
    """打开夹爪"""

    self.get_logger().info('夹爪打开')
    target = {'gripper_joint': 0.035} # 全开 0.035m
    plan = self.arm.plan(target=target)
    if plan:
        self.arm.execute(plan)

def close_gripper(self):
    """关闭夹爪"""

    self.get_logger().info('夹爪关闭')
    target = {'gripper_joint': 0.0} # 闭合
    plan = self.arm.plan(target=target)
    if plan:
        self.arm.execute(plan)

def pick(self, obj_x: float, obj_y: float, obj_z: float):
    """
    抓取物体
    流程:
        1. 打开夹爪
        2. 移动到预抓取位姿 (物体上方)
        3. 笛卡尔下降到抓取位姿
        4. 关闭夹爪 (抓取)
        5. 附着物体到夹爪
        6. 笛卡尔提升
    """
    self.get_logger().info(f'开始抓取物体 at ({obj_x:.2f}, {obj_y:.2f}, {obj_z:.2f})')

    # 1. 打开夹爪
    self.open_gripper()
    time.sleep(1.0)

    # 2. 移动到预抓取位姿 (物体上方 8cm)
    pre_grasp = Pose()
    pre_grasp.position.x = obj_x
    pre_grasp.position.y = obj_y
    pre_grasp.position.z = obj_z + 0.08 # 物体上方
    pre_grasp.orientation.x = 0.0
    pre_grasp.orientation.y = 1.0
    pre_grasp.orientation.z = 0.0

```

```

pre_grasp.orientation.w = 0.0 # 夹爪朝下

self.arm.set_start_state_to_current_state()
plan = self.arm.plan(target=pre_grasp)
if plan:
    self.arm.execute(plan)
    self.get_logger().info('到达预抓取位姿')
else:
    self.get_logger().error('预抓取位姿不可达!')
    return False

time.sleep(1.0)

# 3. 笛卡尔下降到抓取位姿
grasp_waypoints = []
for z in np.arange(obj_z + 0.08, obj_z + 0.01, -0.005):
    wp = Pose()
    wp.position.x = obj_x
    wp.position.y = obj_y
    wp.position.z = z
    wp.orientation = copy.deepcopy(pre_grasp.orientation)
    grasp_waypoints.append(wp)

result = self.arm.compute_cartesian_path(
    waypoints=grasp_waypoints, step=0.005, jump_threshold=0.0
)
if result and result.fraction > 0.95:
    self.arm.execute(result)
    self.get_logger().info('到达抓取位姿')
else:
    self.get_logger().error('抓取路径不可达!')
    return False

# 4. 关闭夹爪
self.close_gripper()
time.sleep(1.0)

# 5. 附着物体
self.attach_object(self.object_name)
time.sleep(0.5)

# 6. 笛卡尔提升
lift_waypoints = []
for z in np.arange(obj_z + 0.01, obj_z + 0.15, 0.005):
    wp = Pose()
    wp.position.x = obj_x
    wp.position.y = obj_y
    wp.position.z = z
    wp.orientation = copy.deepcopy(pre_grasp.orientation)
    lift_waypoints.append(wp)

result = self.arm.compute_cartesian_path(
    waypoints=lift_waypoints, step=0.005, jump_threshold=0.0
)
if result:
    self.arm.execute(result)
    self.get_logger().info('物体已提升')

```

```

        return True
    return False

def place(self, target_x: float, target_y: float, target_z: float):
    """
    放置物体
    流程：
        1. 移动到放置预位置
        2. 笛卡尔下降到放置位姿
        3. 打开夹爪（释放）
        4. 分离物体
        5. 笛卡尔提升
    """
    self.get_logger().info(f'开始放置物体 at ({target_x:.2f}, {target_y:.2f}, {target_z:.2f})')

    # 1. 移动到放置预位置（上方 8cm）
    pre_place = Pose()
    pre_place.position.x = target_x
    pre_place.position.y = target_y
    pre_place.position.z = target_z + 0.08
    pre_place.orientation.x = 0.0
    pre_place.orientation.y = 1.0
    pre_place.orientation.z = 0.0
    pre_place.orientation.w = 0.0

    self.arm.set_start_state_to_current_state()
    plan = self.arm.plan(target=pre_place)
    if plan:
        self.arm.execute(plan)
        self.get_logger().info('到达放置预位姿')
    else:
        self.get_logger().error('放置预位姿不可达!')
        return False

    time.sleep(1.0)

    # 2. 下降到放置位姿
    place_waypoints = []
    for z in np.arange(target_z + 0.08, target_z + 0.02, -0.005):
        wp = Pose()
        wp.position.x = target_x
        wp.position.y = target_y
        wp.position.z = z
        wp.orientation = copy.deepcopy(pre_place.orientation)
        place_waypoints.append(wp)

    result = self.arm.compute_cartesian_path(
        waypoints=place_waypoints, step=0.005, jump_threshold=0.0
    )
    if result:
        self.arm.execute(result)
        self.get_logger().info('到达放置位姿')
    else:
        self.get_logger().error('放置路径不可达!')
        return False

```

```

# 3. 打开夹爪
self.open_gripper()
time.sleep(1.0)

# 4. 分离物体 + 将物体添加回场景 (在放置位置)
self.detach_object(self.object_name)
self.add_box_to_scene(
    self.object_name + '_placed',
    self.object_size,
    target_x, target_y, target_z
)
time.sleep(0.5)

# 5. 笛卡尔提升
retreat_waypoints = []
for z in np.arange(target_z + 0.02, target_z + 0.15, 0.005):
    wp = Pose()
    wp.position.x = target_x
    wp.position.y = target_y
    wp.position.z = z
    wp.orientation = copy.deepcopy(pre_place.orientation)
    retreat_waypoints.append(wp)

result = self.arm.compute_cartesian_path(
    waypoints=retreat_waypoints, step=0.005, jump_threshold=0.0
)
if result:
    self.arm.execute(result)
    self.get_logger().info('放置完成, 机械臂已撤回')
return True

def go_home(self):
    """回到 Home 位姿"""
    home = {
        'arm_joint1': 0.0, 'arm_joint2': -0.5, 'arm_joint3': 0.0,
        'arm_joint4': -1.5, 'arm_joint5': 0.0, 'arm_joint6': 1.0,
    }
    self.arm.set_start_state_to_current_state()
    plan = self.arm.plan(target=home)
    if plan:
        self.arm.execute(plan)

def run_demo(self):
    """执行演示"""

    # 初始化: 添加桌子 + 物体到场景
    self.add_box_to_scene('table', (0.8, 0.6, 0.05), 0.4, 0.0, 0.375)
    self.add_box_to_scene(
        self.object_name, self.object_size, 0.5, -0.1, 0.42
    )

    # 回到 Home
    self.go_home()
    time.sleep(2.0)

    # Pick: 抓取物体
    success = self.pick(0.5, -0.1, 0.43)
    if not success:

```



```

        self.get_logger().error('Pick 失败!')
        return

    time.sleep(1.5)

    # Place: 放置到新位置
    success = self.place(0.3, 0.2, 0.42)
    if not success:
        self.get_logger().error('Place 失败!')
        return

    # 回到 Home
    self.go_home()
    self.get_logger().info('Pick & Place 任务完成!')

def main():
    rclpy.init()
    node = PickPlaceRunner()
    node.run_demo()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

验收标准

- ☐ 成功抓取物体 (夹爪闭合到位)
- ☐ 物体在规划场景中随夹爪移动 (附着/分离正确)
- ☐ 成功放置到目标位置
- ☐ 机械臂回到 Home (无自碰撞)
- ☐ 笛卡尔路径完成度 > 95%

练习 4 : MoveItPy 单关节运动规划 (约 15 分钟)

目标

使用 moveit_py Python API 规划单个关节的运动, 控制 XBot-U 机械臂完成指定角度旋转。

步骤

步骤 1: 编写 single_joint_planner.py

```
#!/usr/bin/env python3
"""练习 4: MoveItPy 单关节规划"""
import rclpy
import time

def main():
    rclpy.init()
    from moveit.planning import MoveItPy, PlanningComponent

    moveit = MoveItPy(node_name='moveit_py')
    arm = PlanningComponent(moveit, 'xbot_arm', 'xbot_gripper')

    print('规划 arm_joint1 旋转到 1.0 rad...')
    arm.set_start_state_to_current_state()
    plan = arm.plan(
        target={'arm_joint1': 1.0},
        planner_id='RRTConnectkConfigDefault',
        planning_time=2.0,
    )
    if plan:
        arm.execute(plan)
        print('运动完成!')
        time.sleep(1.0)

    print('规划 arm_joint1 回到 0.0 rad...')
    arm.set_start_state_to_current_state()
    plan = arm.plan(
        target={'arm_joint1': 0.0},
        planner_id='RRTConnectkConfigDefault',
    )
    if plan:
        arm.execute(plan)
        print('回零完成!')

    rclpy.shutdown()
```

步骤 2: 启动 MoveIt 并运行

```
ros2 launch xbot_moveit_config moveit.launch.py use_sim_time:=true
python3 single_joint_planner.py
```

✓ **验证:** 机械臂 arm_joint1 旋转到 1.0 rad 后再回到 0.0 rad, Gazebo/RViz 中可见运动。

思考题

206. PlanningComponent.plan() 返回的 plan 对象包含哪些信息?

207. 如果目标关节角超出限位, 规划会怎样?


```

"""非极大值抑制 (简化实现)"""
mask = confidences > conf_thresh
boxes = boxes[mask]
confidences = confidences[mask]
class_ids = class_ids[mask]

indices = []
sorted_idx = np.argsort(confidences)[::-1]

while len(sorted_idx) > 0:
    best = sorted_idx[0]
    indices.append(best)
    if len(sorted_idx) == 1:
        break
    ious = compute_iou(boxes[best], boxes[sorted_idx[1:]])
    sorted_idx = sorted_idx[1:][ious < iou_thresh]

return [(class_ids[i], confidences[i], *boxes[i]) for i in indices]

def compute_iou(box, boxes):
    """计算 IoU (Intersection over Union)"""
    x1 = np.maximum(box[0], boxes[:, 0])
    y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2])
    y2 = np.minimum(box[3], boxes[:, 3])
    inter = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)
    area_box = (box[2] - box[0]) * (box[3] - box[1])
    area_boxes = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
    union = area_box + area_boxes - inter
    return inter / (union + 1e-6)

```

13.1.2 模型导出与 ONNX/TensorRT 推理

```

# 导出 YOLOv8 到 ONNX
from ultralytics import YOLO

def export_yolo_for_ros2():
    """导出 YOLO 模型为 ONNX 格式 (ROS 2 部署用)"""
    model = YOLO('yolov8n.pt') # 加载预训练模型
    model.export(
        format='onnx',
        imgsz=640,
        opset=12,
        half=False, # FP32 (ROS 2 标准部署)
        simplify=True,
        dynamic=False,
    )
    # 生成: yolov8n.onnx
    print('模型已导出: yolov8n.onnx')

# ONNX Runtime 推理节点
import onnxruntime as ort
import cv2

class ONNXYoloDetector:

```

```

"""基于 ONNX Runtime 的 YOLO 推理器"""
def __init__(self, model_path: str, conf_thresh: float = 0.5,
              nms_thresh: float = 0.45):
    self.session = ort.InferenceSession(model_path)
    self.input_name = self.session.get_inputs()[0].name
    self.conf_thresh = conf_thresh
    self.nms_thresh = nms_thresh
    # COCO 类别名称
    self.class_names = [
        'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
        'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
        'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
        'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
        'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee',
        'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
        'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
        'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
        'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
        'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
        'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
        'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven',
        'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase',
        'scissors', 'teddy bear', 'hair drier', 'toothbrush',
    ]

def preprocess(self, img: np.ndarray, input_size: int = 640):
    """图像预处理: resize + normalize"""
    original_h, original_w = img.shape[:2]
    img_resized = cv2.resize(img, (input_size, input_size))
    img_normalized = img_resized.astype(np.float32) / 255.0
    img_transposed = np.transpose(img_normalized, (2, 0, 1))
    img_batch = np.expand_dims(img_transposed, axis=0)
    return img_batch, original_w, original_h

def postprocess(self, output: np.ndarray, orig_w: int, orig_h: int):
    """后处理: 解析检测结果 (带坐标缩放)"""
    detections = []
    output = output[0].T # [8400, 84]

    for row in output:
        class_scores = row[4:84]
        class_id = np.argmax(class_scores)
        confidence = class_scores[class_id]

        if confidence < self.conf_thresh:
            continue

        cx, cy, w, h = row[:4]
        # 归一化坐标 → 原始图像坐标
        x1 = int((cx - w / 2) * orig_w / 640)
        y1 = int((cy - h / 2) * orig_h / 640)
        x2 = int((cx + w / 2) * orig_w / 640)
        y2 = int((cy + h / 2) * orig_h / 640)

        detections.append((class_id, confidence, x1, y1, x2, y2))

    return detections

```

```
def detect(self, image: np.ndarray) -> list:
    """执行检测"""
    input_tensor, ow, oh = self.preprocess(image)
    outputs = self.session.run(None, {self.input_name: input_tensor})
    return self.postprocess(outputs[0], ow, oh)
```

13.1.3 cv_bridge 图像转换

cv_bridge 是 ROS 图像消息 (sensor_msgs/Image) 与 OpenCV cv::Mat 之间的桥梁:

```
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import cv2
import numpy as np

class ImageBridge:
    """ROS 2 图像 ↔ OpenCV 转换器"""
    def __init__(self):
        self.bridge = CvBridge()

    def imgmsg_to_cv2(self, msg: Image, encoding: str = 'bgr8') -> np.ndarray:
        """
        ROS Image 消息 → OpenCV 图像
        参数:
            msg: sensor_msgs/Image
            encoding: 目标编码 (bgr8, rgb8, mono8)
        返回:
            OpenCV 图像 (numpy 数组)
        """
        try:
            cv_image = self.bridge.imgmsg_to_cv2(
                msg, desired_encoding=encoding
            )
            return cv_image
        except CvBridgeError as e:
            print(f'cv_bridge 转换失败: {e}')
            return None

    def cv2_to_imgmsg(self, cv_image: np.ndarray,
                     encoding: str = 'bgr8') -> Image:
        """
        OpenCV 图像 → ROS Image 消息
        参数:
            cv_image: OpenCV 图像
            encoding: 源编码
        返回:
            sensor_msgs/Image 消息
        """
        try:
            msg = self.bridge.cv2_to_imgmsg(cv_image, encoding=encoding)
            return msg
        except CvBridgeError as e:
            print(f'cv_bridge 转换失败: {e}')
```

```

        return None

    def compress_image(self, cv_image: np.ndarray, quality: int = 80):
        """JPEG 压缩 (用于网络传输优化)"""
        import base64
        ret, jpeg = cv2.imencode('.jpg', cv_image,
                                [cv2.IMWRITE_JPEG_QUALITY, quality])
        return jpeg.tobytes()

```

13.1.4 ROS 2 YOLO 推理节点

```

#!/usr/bin/env python3
"""
完整的 ROS 2 YOLO 推理节点
Camera → YOLO ONNX → /detections 话题
"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose
from cv_bridge import CvBridge
import onnxruntime as ort
import numpy as np
import cv2

class YoloDetectorNode(Node):
    """YOLOv8 ONNX 推理节点"""
    def __init__(self):
        super().__init__('yolo_detector')

        # 声明参数
        self.declare_parameter('model_path', 'yolov8n.onnx')
        self.declare_parameter('conf_threshold', 0.5)
        self.declare_parameter('nms_threshold', 0.45)
        self.declare_parameter('input_size', 640)
        self.declare_parameter('image_topic', '/camera/image_raw')

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_threshold').value
        self.nms_thresh = self.get_parameter('nms_threshold').value
        self.input_size = self.get_parameter('input_size').value

        # 加载 ONNX 模型
        self.get_logger().info(f'加载模型: {model_path}')
        self.session = ort.InferenceSession(model_path)
        self.input_name = self.session.get_inputs()[0].name
        self.bridge = CvBridge()
        self.get_logger().info('YOLO 模型已加载')

        # 订阅相机图像
        self.image_sub = self.create_subscription(
            Image,
            self.get_parameter('image_topic').value,
            self.image_callback,
            10

```

```

    )
    # 发布检测结果
    self.detection_pub = self.create_publisher(
        Detection2DArray, '/detections', 10
    )
    # 发布标注图像
    self.annotated_pub = self.create_publisher(
        Image, '/detections/annotated', 10
    )

    # COCO 类别名称 (80 类)
    self.class_names = [ # 完整 80 类
        'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
        'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
        'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
        'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
        'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee',
        'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
        'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
        'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
        'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
        'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
        'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
        'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven',
        'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase',
        'scissors', 'teddy bear', 'hair drier', 'toothbrush',
    ]

    # 帧率统计
    self.frame_count = 0
    self.timer = self.create_timer(5.0, self.report_fps)

    def report_fps(self):
        """报告推理帧率"""
        fps = self.frame_count / 5.0
        self.get_logger().info(f'推理 FPS: {fps:.1f}')
        self.frame_count = 0

    def image_callback(self, msg: Image):
        """图像回调: 推理 + 发布结果"""
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        except Exception as e:
            self.get_logger().error(f'cv_bridge 转换失败: {e}')
            return

        # 执行推理
        detections = self.detect(cv_image)
        self.frame_count += 1

        # 发布 Detection2DArray
        det_msg = self.build_detection_msg(detections, msg.header)
        self.detection_pub.publish(det_msg)

        # 发布标注图像

```



```

        annotated = self.draw_detections(cv_image, detections)
        ann_msg = self.bridge.cv2_to_imgmsg(annotated, encoding='bgr8')
        ann_msg.header = msg.header
        self.annotated_pub.publish(ann_msg)

def detect(self, image: np.ndarray) -> list:
    """YOLO 推理"""
    h, w = image.shape[:2]
    # 预处理
    img_resized = cv2.resize(image, (self.input_size, self.input_size))
    img_norm = img_resized.astype(np.float32) / 255.0
    img_chw = np.transpose(img_norm, (2, 0, 1))
    img_batch = np.expand_dims(img_chw, axis=0)

    # 推理
    outputs = self.session.run(None, {self.input_name: img_batch})
    output = outputs[0][0].T # [8400, 84]

    # 解析
    detections = []
    for row in output:
        scores = row[4:84]
        class_id = np.argmax(scores)
        conf = scores[class_id]
        if conf < self.conf_thresh:
            continue

        cx, cy, bw, bh = row[:4]
        x1 = int((cx - bw / 2) * w / self.input_size)
        y1 = int((cy - bh / 2) * h / self.input_size)
        x2 = int((cx + bw / 2) * w / self.input_size)
        y2 = int((cy + bh / 2) * h / self.input_size)

        detections.append((class_id, float(conf), x1, y1, x2, y2))

    return detections

def build_detection_msg(self, detections: list,
                        header) -> Detection2DArray:
    """构建 ROS 2 标准检测消息"""
    msg = Detection2DArray()
    msg.header = header

    for class_id, conf, x1, y1, x2, y2 in detections:
        det = Detection2D()
        # bbox 中心 + 尺寸
        det.bbox.center.position.x = (x1 + x2) / 2.0
        det.bbox.center.position.y = (y1 + y2) / 2.0
        det.bbox.size_x = float(x2 - x1)
        det.bbox.size_y = float(y2 - y1)
        # 检测结果
        hyp = ObjectHypothesisWithPose()
        hyp.hypothesis.class_id = str(class_id)
        hyp.hypothesis.score = conf
        det.results.append(hyp)
        msg.detections.append(det)

    return msg

```

```

def draw_detections(self, image: np.ndarray,
                    detections: list) -> np.ndarray:
    """在图像上绘制检测框"""
    result = image.copy()
    colors = [
        (0, 255, 0), (255, 0, 0), (0, 0, 255), (255, 255, 0),
        (255, 0, 255), (0, 255, 255), (128, 0, 128), (128, 128, 0),
    ]

    for class_id, conf, x1, y1, x2, y2 in detections:
        color = colors[class_id % len(colors)]
        name = self.class_names[class_id] if class_id < len(self.class_names) else
f'cls_{class_id}'
        label = f'{name} {conf:.2f}'

        cv2.rectangle(result, (x1, y1), (x2, y2), color, 2)
        cv2.putText(result, label, (x1, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

    return result

def main():
    rclpy.init()
    node = YoloDetectorNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

13.1.5 检测结果 3D 定位

通过深度相机将 2D bbox 投影到 3D 空间:

```

import numpy as np

def bbox_to_3d_position(bbox: tuple, depth_image: np.ndarray,
                       camera_info) -> tuple:
    """
    将 2D bbox + 深度图投影为 3D 位置
    参数:
        bbox: (x1, y1, x2, y2) 检测框
        depth_image: 深度图 (米)
        camera_info: CameraInfo (含内参)
    返回:
        (x, y, z) 物体在 camera_frame 中的 3D 位置
    """
    x1, y1, x2, y2 = bbox

    # bbox 中心点
    cx_bbox = (x1 + x2) // 2
    cy_bbox = (y1 + y2) // 2

```

```

# 提取 bbox 区域的深度值 (取中位数过滤噪声)
depth_roi = depth_image[y1:y2, x1:x2]
valid_depths = depth_roi[depth_roi > 0]
if len(valid_depths) == 0:
    return None
depth = np.median(valid_depths)

# 像素 → 3D (使用相机内参)
fx = camera_info.k[0] # [fx, 0, cx]
fy = camera_info.k[4] # [0, fy, cy]
cx_cam = camera_info.k[2]
cy_cam = camera_info.k[5]

x = (cx_bbox - cx_cam) * depth / fx
y = (cy_bbox - cy_cam) * depth / fy
z = depth

return (x, y, z)

```

13.1.6 目标跟随控制

基于 YOLO 检测结果的机器人跟随:

```

class YoloFollower(Node):
    """基于 YOLO 检测的目标跟随控制器"""
    def __init__(self):
        super().__init__('yolo_follower')
        self.detection_sub = self.create_subscription(
            Detection2DArray, '/detections', self.detection_cb, 10
        )
        self.depth_sub = self.create_subscription(
            Image, '/depth/image_raw', self.depth_cb, 10
        )
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.bridge = CvBridge()

        self.target_class = 'person' # 跟随目标类别
        self.target_distance = 1.5 # 期望跟随距离 (m)
        self.max_linear = 0.5 # 最大线速度
        self.max_angular = 1.0 # 最大角速度

        self.current_detection = None
        self.current_depth = None

    def detection_cb(self, msg: Detection2DArray):
        """筛选目标检测"""
        for det in msg.detections:
            class_id = det.results[0].hypothesis.class_id
            if class_id == self.target_class:
                self.current_detection = det
                return
        self.current_detection = None

    def depth_cb(self, msg: Image):
        self.current_depth = self.bridge.imgmsg_to_cv2(

```

```

        msg, desired_encoding='passthrough'
    )

def control_loop(self):
    """PID 跟随控制"""
    if self.current_detection is None:
        self.publish_velocity(0.0, 0.0)
        return

    # 目标偏离像中心的角偏差
    image_width = 640 # 假设图像宽度
    bbox_center = self.current_detection.bbox.center.position.x
    angular_error = (bbox_center - image_width / 2) / (image_width / 2)

    # 距离误差 (从 bbox 大小估算, 简化为固定值)
    distance_error = self.bbox_size_to_distance(
        self.current_detection.bbox.size_x,
        self.current_detection.bbox.size_y
    ) - self.target_distance

    # PID 控制
    linear = min(max(distance_error * 0.5, -self.max_linear), self.max_linear)
    angular = min(max(-angular_error * 2.0, -self.max_angular), self.max_angular)

    self.publish_velocity(linear, angular)

def bbox_size_to_distance(self, w: float, h: float) -> float:
    """粗略距离估计 (基于 bbox 大小)"""
    return 1.0 / (max(w, h) + 0.01) * 100

def publish_velocity(self, linear: float, angular: float):
    cmd = Twist()
    cmd.linear.x = linear
    cmd.angular.z = angular
    self.cmd_pub.publish(cmd)

```

13.1.7 自定义检测消息

```

# 自定义检测接口定义
# 文件: yolo_detector_interfaces/msg/YoloDetection.msg

# std_msgs/Header header
# uint16 class_id
# string class_name
# float32 confidence
# uint16 x1
# uint16 y1
# uint16 x2
# uint16 y2
# float32 center_x
# float32 center_y
# float32 width
# float32 height

# 自定义服务定义
# YoloDetectorInterfaces/srv/SetClasses.srv

```

```
# string[] classes    # 要检测的类别
# ---
# bool success
# string message
```

13.1.8 完整启动流程

```
# 1. 启动相机 (仿真或真实)
ros2 launch usb_cam camera.launch.py

# 2. 启动 YOLO 推理节点
python3 yolo_detector_node.py --ros-args -p model_path:=yolov8n.onnx

# 3. 启动跟随控制节点
python3 yolo_follower.py

# 4. 查看结果
ros2 topic echo /detections
rviz2 -d src/courseware/rviz/yolo_detection.rviz
```

13.2 练习题

- 1. 原理解析题:** 解释 YOLO 的 "单阶段" 检测含义, 与 Faster R-CNN 的两阶段方法对比优势和劣势。
- 2. 模型题:** 编写脚本将 YOLOv8 模型导出为 ONNX 格式, 并在 Python 中用 cv2.dnn 或 onnxruntime 加载并推理, 输出一张测试图片的检测结果。
- 3. ROS 2 节点题:** 编写一个完整的 ROS 2 节点, 订阅 /camera/image_raw, 使用 ONNX Runtime 进行 YOLO 推理, 将检测结果发布为自定义 Detection2DArray 消息。
- 4. 3D 定位题:** 将 YOLO 检测框中心点结合深度图 (来自 /depth/image_raw) 投影到 3D 空间, 发布为 geometry_msgs/PointStamped 话题。
- 5. 控制题:** 实现基于 YOLO 检测的 "人跟随" 控制器: 当检测到 person 类别时, 计算角偏差和距离误差, 通过 PID 控制 cmd_vel 使机器人保持在目标后方 1.5 米。
- 6. 综合题:** 设计一个安保巡逻场景: 机器人按预设航点巡逻, 途中使用 YOLO 检测 "异常物体" (如背包、行李箱), 发现后发布警告并在 rviz 中高亮标记。写出完整系统架构和 ROS 2 多节点通信方案。

实验 13:

第 13 章 实验手册: YOLO + ROS 2 目标检测

环境说明

本实验使用 XBot-U 仿真环境 + USB 相机或 Gazebo 模拟相机, 预装 ultralytics 和 onnxruntime。

练习 1: YOLO 模型导出与 ROS 2 推理节点 (~30 分钟)

目标

导出 YOLO 模型为 ONNX 格式, 编写 ROS 2 节点实现实时目标检测推理。

步骤

1.1 导出 YOLOv8 ONNX 模型

```
# 安装 ultralytics
pip install ultralytics onnxruntime

# 导出 ONNX 模型
python3 -c "
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.export(format='onnx', opset=12, simplify=True, imgsz=640)
print('模型已导出: yolov8n.onnx')
"

# 验证模型
python3 -c "
import onnx
model = onnx.load('yolov8n.onnx')
onnx.checker.check_model(model)
print('ONNX 模型验证通过')
"
```

1.2 启动相机 (仿真或真实)

```
# 选项 A: Gazebo 仿真相机
ros2 launch xbot_sim xbot_gazebo_camera.launch.py

# 选项 B: 真实 USB 相机
ros2 run usb_cam usb_cam_node_exe --ros-args -p video_device:=/dev/video0

# 验证图像流
ros2 run rqt_image_view rqt_image_view /camera/image_raw
```

1.3 编写基础 YOLO ROS 2 推理节点

```
#!/usr/bin/env python3
"""练习 1: YOLO ONNX ROS 2 推理节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose
from cv_bridge import CvBridge
import onnxruntime as ort
import numpy as np
import cv2
import time

class SimpleYoloNode(Node):
    """基础 YOLO 推理 ROS 2 节点"""
    def __init__(self):
        super().__init__('simple_yolo')

        self.declare_parameter('model_path', 'yolov8n.onnx')
        self.declare_parameter('conf_thresh', 0.5)

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_thresh').value

        # 加载 ONNX 模型
        self.get_logger().info(f'加载模型: {model_path}')
        self.session = ort.InferenceSession(
            model_path,
            providers=['CPUExecutionProvider']
        )
        self.input_name = self.session.get_inputs()[0].name
        self.input_size = 640
        self.bridge = CvBridge()

        # 类别映射 (COCO 80 类)
        self.classes = [
            'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
            'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
            'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
            'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
            'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee',
            'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
```



```

        'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
        'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
        'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
        'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
        'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
        'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven',
        'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase',
        'scissors', 'teddy bear', 'hair drier', 'toothbrush',
    ]

    # 订阅 / 发布
    self.image_sub = self.create_subscription(
        Image, '/camera/image_raw', self.image_cb, 10
    )
    self.det_pub = self.create_publisher(
        Detection2DArray, '/yolo/detections', 10
    )
    self.image_pub = self.create_publisher(
        Image, '/yolo/annotated', 10
    )

    # 帧率统计
    self.last_time = time.time()
    self.frame_count = 0

    self.get_logger().info('YOLO 推理节点已就绪')

def image_cb(self, msg: Image):
    """处理每一帧图像"""

    # 计时开始
    t0 = time.time()

    # 转换
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception as e:
        self.get_logger().error(f'图像转换失败: {e}')
        return

    # 推理
    h, w = cv_image.shape[:2]
    detections = self.infer(cv_image, w, h)
    infer_time = time.time() - t0

    # 发布检测结果
    det_msg = self.build_msg(detections, msg.header, w, h)
    self.det_pub.publish(det_msg)

    # 发布标注图像
    annotated = self.draw_boxes(cv_image, detections)
    ann_msg = self.bridge.cv2_to_imgmsg(annotated, 'bgr8')
    ann_msg.header = msg.header
    self.image_pub.publish(ann_msg)

    # 帧率统计

```

```

        self.frame_count += 1
        elapsed = time.time() - self.last_time
        if elapsed > 5.0:
            fps = self.frame_count / elapsed
            self.get_logger().info(
                f'FPS: {fps:.1f} | 推理: {infer_time*1000:.0f}ms | '
                f'检测: {len(detections)} 个'
            )
            self.frame_count = 0
            self.last_time = time.time()

def infer(self, image: np.ndarray, orig_w: int, orig_h: int) -> list:
    """YOLO ONNX 推理"""

    # 预处理
    img = cv2.resize(image, (self.input_size, self.input_size))
    img = img.astype(np.float32) / 255.0
    img = np.transpose(img, (2, 0, 1))
    img = np.expand_dims(img, axis=0)

    # ONNX 推理
    outputs = self.session.run(None, {self.input_name: img})
    output = outputs[0][0].T

    # 解析结果
    detections = []
    for row in output:
        scores = row[4:84]
        class_id = np.argmax(scores)
        conf = scores[class_id]
        if conf < self.conf_thresh:
            continue

        cx, cy, bw, bh = row[:4]
        x1 = int((cx - bw / 2) * orig_w / self.input_size)
        y1 = int((cy - bh / 2) * orig_h / self.input_size)
        x2 = int((cx + bw / 2) * orig_w / self.input_size)
        y2 = int((cy + bh / 2) * orig_h / self.input_size)

        detections.append({
            'class_id': int(class_id),
            'class_name': self.classes[class_id] if class_id < len(self.classes) else
'unknown',
            'confidence': float(conf),
            'bbox': (x1, y1, x2, y2),
            'center': ((x1 + x2) / 2, (y1 + y2) / 2),
        })

    return detections

def build_msg(self, detections: list, header, img_w: int, img_h: int) -> Detection2DArray:
    """构建标准检测消息"""
    msg = Detection2DArray()
    msg.header = header

    for det in detections:
        d = Detection2D()
        x1, y1, x2, y2 = det['bbox']
        d.bbox.center.position.x = float((x1 + x2) / 2)

```

```

        d.bbox.center.position.y = float((y1 + y2) / 2)
        d.bbox.size_x = float(x2 - x1)
        d.bbox.size_y = float(y2 - y1)

        hyp = ObjectHypothesisWithPose()
        hyp.hypothesis.class_id = str(det['class_id'])
        hyp.hypothesis.score = det['confidence']
        d.results.append(hyp)
        msg.detections.append(d)

    return msg

def draw_boxes(self, image: np.ndarray, detections: list) -> np.ndarray:
    """绘制检测框"""
    result = image.copy()
    colors = [
        (0, 255, 0), (255, 0, 0), (0, 0, 255),
        (255, 255, 0), (255, 0, 255), (0, 255, 255),
    ]
    for det in detections:
        x1, y1, x2, y2 = det['bbox']
        color = colors[det['class_id'] % len(colors)]
        label = f"{det['class_name']} {det['confidence']:.2f}"
        cv2.rectangle(result, (x1, y1), (x2, y2), color, 2)
        cv2.putText(result, label, (x1, max(y1 - 5, 10)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
    return result

def main():
    rclpy.init()
    node = SimpleYoloNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

1.4 运行与验证

```

# 运行 YOLO 节点
python3 lab13_exercise1.py

# 查看检测结果
ros2 topic echo /yolo/detections --field detections --once

# 可视化标注图像
ros2 run rqt_image_view rqt_image_view /yolo/annotated

```

验收标准

- [] ONNX 模型成功加载, 推理无报错
- [] 实时输出检测数量和类别
- [] /yolo/annotated 话题可显示标注框

- [] FPS > 10 (CPU) 或 > 30 (GPU)

练习 2: 检测结果可视化与话题发布 (~30 分钟)

目标

发布检测结果为 ROS 2 标准消息和 Marker, 实现多下游节点订阅。

步骤

2.1 增强版检测可视化节点

```
#!/usr/bin/env python3
"""练习 2: 检测结果可视化与多话题发布"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from vision_msgs.msg import Detection2DArray, Detection2D
from visualization_msgs.msg import Marker, MarkerArray
from geometry_msgs.msg import Point
from cv_bridge import CvBridge
import onnxruntime as ort
import numpy as np
import cv2
import json

class VisualizedYoloNode(Node):
    """增强版 YOLO: 多话题发布 + Marker 可视化"""
    def __init__(self):
        super().__init__('visualized_yolo')

        self.declare_parameter('model_path', 'yolov8n.onnx')
        self.declare_parameter('conf_thresh', 0.5)
        self.declare_parameter('classes_of_interest', ['person', 'bottle', 'chair'])

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_thresh').value
        self.interest_classes = self.get_parameter('classes_of_interest').value

        self.session = ort.InferenceSession(model_path, providers=['CPUExecutionProvider'])
        self.input_name = self.session.get_inputs()[0].name
        self.input_size = 640
        self.bridge = CvBridge()

        self.classes = ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
                        'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
                        'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
                        'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
                        'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee',
                        'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
```

```

        'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
        'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
        'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
        'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
        'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
        'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven',
        'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase',
        'scissors', 'teddy bear', 'hair drier', 'toothbrush']

# 订阅
self.image_sub = self.create_subscription(Image, '/camera/image_raw', self.image_cb, 10)
self.camera_info_sub = self.create_subscription(CameraInfo, '/camera/camera_info',
self.camera_info_cb, 10)
self.camera_info = None

# 发布
self.det_pub = self.create_publisher(Detection2DArray, '/yolo/detections', 10)
self.image_pub = self.create_publisher(Image, '/yolo/annotated', 10)
self.marker_pub = self.create_publisher(MarkerArray, '/yolo/markers', 10)
self.text_pub = self.create_publisher(Marker, '/yolo/text_markers', 10)

# 每个类别单独的话题
self.per_class_pubs = {}
for cls in self.interest_classes:
    self.per_class_pubs[cls] = self.create_publisher(
        Detection2DArray, f'/yolo/{cls}', 10
    )

self.get_logger().info('可视化 YOLO 节点已就绪')

def camera_info_cb(self, msg: CameraInfo):
    self.camera_info = msg

def image_cb(self, msg: Image):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    h, w = cv_image.shape[:2]
    detections = self.infer(cv_image, w, h)

# 发布全景检测
det_msg = self.build_det_msg(detections, msg.header)
self.det_pub.publish(det_msg)

# 发布标注图像
ann = self.draw_full_annotation(cv_image, detections)
self.image_pub.publish(self.bridge.cv2_to_imgmsg(ann, 'bgr8'))

# 发布 3D Marker
self.publish_markers(detections, msg.header, w, h)

# 按类别分发
self.publish_per_class(detections, msg.header)

```

```

def publish_markers(self, detections: list, header, img_w: int, img_h: int):
    """发布 3D Marker 数组 (在 rviz 中显示)"""
    marker_array = MarkerArray()
    text_marker = Marker()
    text_marker.header = header
    text_marker.type = Marker.TEXT_VIEW_FACING
    text_marker.scale.z = 0.15
    text_marker.color.r = 1.0
    text_marker.color.g = 1.0
    text_marker.color.b = 0.0
    text_marker.color.a = 1.0
    text_marker.ns = 'yolo_detections'

    for i, det in enumerate(detections):
        # 为每个检测创建球形 Marker (简化表示)
        m = Marker()
        m.header = header
        m.ns = 'yolo_objects'
        m.id = i
        m.type = Marker.CUBE
        m.action = Marker.ADD

        # 近似 3D 位置 (假设在相机前方固定距离)
        m.pose.position.x = 2.0
        m.pose.position.y = (det['bbox'][0] / img_w - 0.5) * 3.0
        m.pose.position.z = (0.5 - det['bbox'][1] / img_h) * 2.0
        m.scale.x = 0.2
        m.scale.y = 0.2
        m.scale.z = 0.2
        m.color.r = 0.0
        m.color.g = 1.0
        m.color.b = 0.0
        m.color.a = 0.8
        marker_array.markers.append(m)

        # 组合文字标记
        text_lines = f"{det['class_name']}\n{det['confidence']:.2f}"
        text_marker.text += f"[{i}] {text_lines}\n"

    self.marker_pub.publish(marker_array)
    self.text_pub.publish(text_marker)

def publish_per_class(self, detections: list, header):
    """按类别分组发布"""
    class_groups = {}
    for det in detections:
        cls = det['class_name']
        if cls not in class_groups:
            class_groups[cls] = []
        class_groups[cls].append(det)

    for cls, dets in class_groups.items():
        if cls in self.per_class_pubs:
            msg = self.build_det_msg(dets, header)
            self.per_class_pubs[cls].publish(msg)

def draw_full_annotation(self, image: np.ndarray, detections: list) -> np.ndarray:
    """绘制完整标注 (含统计信息)"""

```

```

result = image.copy()
colors = {
    'person': (0, 255, 0),
    'bottle': (255, 0, 0),
    'chair': (0, 0, 255),
}
# 绘制检测框
for det in detections:
    x1, y1, x2, y2 = det['bbox']
    color = colors.get(det['class_name'], (128, 128, 128))
    cv2.rectangle(result, (x1, y1), (x2, y2), color, 2)
    label = f"{det['class_name']} {det['confidence']:.2f}"
    cv2.putText(result, label, (x1, max(y1 - 5, 15)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, color, 1)

# 顶部统计面板
h, w = result.shape[:2]
panel = np.zeros((120, w, 3), dtype=np.uint8)
panel[:] = (50, 50, 50)

cv2.putText(panel, f'Detections: {len(detections)}', (10, 25),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

class_counts = {}
for det in detections:
    name = det['class_name']
    class_counts[name] = class_counts.get(name, 0) + 1

y = 55
for name, count in sorted(class_counts.items(), key=lambda x: -x[1])[0:6]:
    cv2.putText(panel, f' {name}: {count}', (10, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1)
    y += 22

return np.vstack([panel, result])

def infer(self, image: np.ndarray, orig_w: int, orig_h: int) -> list:
    img = cv2.resize(image, (self.input_size, self.input_size))
    img = img.astype(np.float32) / 255.0
    img = np.transpose(img, (2, 0, 1))
    img = np.expand_dims(img, axis=0)
    outputs = self.session.run(None, {self.input_name: img})
    output = outputs[0][0].T

    detections = []
    for row in output:
        scores = row[4:84]
        class_id = np.argmax(scores)
        conf = scores[class_id]
        if conf < self.conf_thresh:
            continue
        cx, cy, bw, bh = row[:4]
        x1 = int((cx - bw / 2) * orig_w / self.input_size)
        y1 = int((cy - bh / 2) * orig_h / self.input_size)
        x2 = int((cx + bw / 2) * orig_w / self.input_size)
        y2 = int((cy + bh / 2) * orig_h / self.input_size)
        detections.append({
            'class_id': int(class_id),
            'class_name': self.classes[class_id] if class_id < len(self.classes) else

```

```

'unknown',
        'confidence': float(conf),
        'bbox': (x1, y1, x2, y2),
    })
    return detections

def build_det_msg(self, detections: list, header) -> Detection2DArray:
    msg = Detection2DArray()
    msg.header = header
    for det in detections:
        d = Detection2D()
        x1, y1, x2, y2 = det['bbox']
        d.bbox.center.position.x = float((x1 + x2) / 2)
        d.bbox.center.position.y = float((y1 + y2) / 2)
        d.bbox.size_x = float(x2 - x1)
        d.bbox.size_y = float(y2 - y1)
        d.results.append(ObjectHypothesisWithPose(
            hypothesis=ObjectHypothesisWithPose.Hypothesis(
                class_id=str(det['class_id']),
                score=det['confidence']
            )
        ))
        msg.detections.append(d)
    return msg

def main():
    rclpy.init()
    node = VisualizedYoloNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

验收标准

- [] 按类别分组发布正确
- [] Marker 在 rviz 中正常显示
- [] 统计面板叠加在图像上
- [] 每个类别话题均可独立订阅

练习 3: 目标跟随机器人控制 (~30 分钟)

目标

基于 YOLO 检测实现机器人对特定类别 (person) 的视觉跟随。

步骤

3.1 编写视觉跟随控制节点

```
#!/usr/bin/env python3
"""练习 3: YOLO 目标跟随控制器"""
import rclpy
from rclpy.node import Node
from vision_msgs.msg import Detection2DArray
from geometry_msgs.msg import Twist
import numpy as np

class YoloTracker(Node):
    """基于 YOLO 检测的视觉跟随控制器"""
    def __init__(self):
        super().__init__('yolo_tracker')

        self.declare_parameter('target_class', 'person')
        self.declare_parameter('image_width', 640)
        self.declare_parameter('image_height', 480)
        self.declare_parameter('desired_bbox_width', 200)
        self.declare_parameter('search_rotation_speed', 0.8)
        self.declare_parameter('max_linear_speed', 0.4)
        self.declare_parameter('max_angular_speed', 1.5)

        self.target_class = self.get_parameter('target_class').value
        self.img_w = self.get_parameter('image_width').value
        self.img_h = self.get_parameter('image_height').value
        self.desired_width = self.get_parameter('desired_bbox_width').value
        self.search_omega = self.get_parameter('search_rotation_speed').value
        self.max_v = self.get_parameter('max_linear_speed').value
        self.max_w = self.get_parameter('max_angular_speed').value

        # 订阅 YOLO 检测
        self.det_sub = self.create_subscription(
            Detection2DArray, '/yolo/detections', self.det_cb, 10
        )
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)

        self.current_detections = []
        self.lost_counter = 0
        self.lost_threshold = 30 # 丢失帧数阈值

        # 控制循环 (20 Hz)
        self.timer = self.create_timer(0.05, self.control_loop)
        self.get_logger().info(f'跟随控制器已启动, 目标类别: {self.target_class}')

    def det_cb(self, msg: Detection2DArray):
        self.current_detections = msg.detections

    def find_target(self):
        """从检测结果中找到最大置信度的目标"""
        best = None
        best_conf = 0.0
```

```

for det in self.current_detections:
    if len(det.results) == 0:
        continue
    class_id = det.results[0].hypothesis.class_id
    score = det.results[0].hypothesis.score
    if class_id == self.target_class and score > best_conf:
        best = det
        best_conf = score
return best

def control_loop(self):
    """PID 视觉跟随控制"""
    target = self.find_target()
    cmd = Twist()

    if target is None:
        self.lost_counter += 1
        if self.lost_counter > self.lost_threshold:
            # 目标丢失: 旋转搜索
            cmd.angular.z = self.search_omega
            self.get_logger().info('搜索目标...', throttle_duration_sec=2.0)
        else:
            cmd.linear.x = 0.0
            cmd.angular.z = 0.0
    else:
        self.lost_counter = 0

        # 目标中心像素坐标
        cx = target.bbox.center.position.x
        cy = target.bbox.center.position.y
        bbox_w = target.bbox.size_x

        # 1. 角速度控制 (水平对齐)
        center_error = (cx - self.img_w / 2.0) / (self.img_w / 2.0)
        cmd.angular.z = -np.clip(center_error * 2.0, -self.max_w, self.max_w)

        # 2. 线速度控制 (距离调节, 基于 bbox 宽度)
        size_error = (self.desired_width - bbox_w) / self.desired_width
        cmd.linear.x = np.clip(size_error * self.max_v, -self.max_v, self.max_v)

        # 3. 停止条件: 距离合适且已对齐
        if abs(center_error) < 0.05 and abs(size_error) < 0.1:
            cmd.linear.x = 0.0
            cmd.angular.z = 0.0
            self.get_logger().info('目标已对齐!', throttle_duration_sec=2.0)

        self.get_logger().info(
            f'追踪: bbox=({cx:.0f},{cy:.0f}) w={bbox_w:.0f} '
            f'err=({center_error:.2f},{size_error:.2f}) '
            f'cmd=({cmd.linear.x:.2f},{cmd.angular.z:.2f})',
            throttle_duration_sec=1.0
        )

    self.cmd_pub.publish(cmd)

def stop(self):

```

```

        """安全停止"""
        cmd = Twist()
        self.cmd_pub.publish(cmd)

def main():
    rclpy.init()
    node = YoloTracker()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.stop()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

3.2 完整测试流程

```

# 终端 1: 启动仿真 (含机器人和相机)
ros2 launch xbot_sim xbot_full.launch.py world:=open_arena

# 终端 2: 启动 YOLO 推理
python3 lab13_exercise2.py

# 终端 3: 启动跟随控制
python3 lab13_exercise3.py

# 终端 4: 在 Gazebo 中放置 person 模型并移动
# (手动拖拽 human_model 或使用模型生成脚本)

```

3.3 添加运动模型生成脚本

```

#!/usr/bin/env python3
"""辅助: Gazebo 中生成移动的行人模型"""
import rclpy
from rclpy.node import Node
from gazebo_msgs.srv import SpawnEntity
from geometry_msgs.msg import Pose, Twist

class HumanSpawner(Node):
    """生成移动行人用于测试跟随"""
    def __init__(self):
        super().__init__('human_spawner')
        self.spawn_client = self.create_client(SpawnEntity, '/spawn_entity')

    def spawn_walking_human(self, name: str, x: float, y: float):
        """生成行走的行人模型"""
        sdf = f"""<?xml version="1.0"?>
<sdf version="1.6">
  <model name="{name}">
    <static>false</static>
    <link name="link">
      <collision name="collision">

```

```

        <geometry><box><size>0.3 0.3 1.7</size></box></geometry>
    </collision>
    <visual name="visual">
        <geometry><box><size>0.3 0.3 1.7</size></box></geometry>
        <material><ambient>1 0 0 1</ambient></material>
    </visual>
</link>
</model>
</sdf>"""

req = SpawnEntity.Request()
req.name = name
req.xml = sdf
req.initial_pose = Pose()
req.initial_pose.position.x = x
req.initial_pose.position.y = y
req.initial_pose.position.z = 0.85

future = self.spawn_client.call_async(req)
rclpy.spin_until_future_complete(self, future)
self.get_logger().info(f'行人 {name} 已生成 at ({x:.1f}, {y:.1f})')

def main():
    rclpy.init()
    spawner = HumanSpawner()
    spawner.spawn_walking_human('human1', 2.0, 2.0)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

验收标准

- ☐ 机器人成功锁定目标并开始跟随
- ☐ 目标移动时机器人相应调整线速度和角速度
- ☐ 目标丢失后机器人进入搜索模式
- ☐ 接近目标时自动减速停止
- ☐ 可通过 Ctrl+C 安全停止

练习 4 : YOLO 物体检测 → 视觉跟随控制（约 15 分钟）

目标

利用 YOLO 检测 Gazebo 中的物体，计算 bbox 中心偏移，生成 cmd_vel 指令实现视觉跟随。

步骤

步骤 1: 编写 visual_follow.py

```
#!/usr/bin/env python3
"""练习 4: YOLO 检测→bbox 偏移→cmd_vel 视觉跟随"""
import rclpy
from rclpy.node import Node
from vision_msgs.msg import Detection2DArray
from geometry_msgs.msg import Twist

class VisualFollow(Node):
    def __init__(self):
        super().__init__('visual_follow')
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.det_sub = self.create_subscription(
            Detection2DArray, '/yolo/detections', self.det_cb, 10)
        self.timer = self.create_timer(0.1, self.control_loop)
        self.target_det = None

    def det_cb(self, msg):
        for det in msg.detections:
            if det.results and det.results[0].hypothesis.score > 0.5:
                self.target_det = det
                return
        self.target_det = None

    def control_loop(self):
        cmd = Twist()
        if self.target_det is None:
            cmd.angular.z = 0.5
        else:
            cx = self.target_det.bbox.center.position.x
            err = (cx - 320) / 320
            cmd.angular.z = -err * 1.5
            if abs(err) < 0.05:
                cmd.linear.x = 0.2
        self.cmd_pub.publish(cmd)

def main():
    rclpy.init()
    rclpy.spin(VisualFollow())
    rclpy.shutdown()
```

步骤 2: 运行

```
# 终端 1: 启动 YOLO 推理
python3 lab13_exercise1.py
# 终端 2: 启动视觉跟随
python3 visual_follow.py
```

✓ **验证:** 在相机视野中移动物体，机器人自动转向跟踪目标。目标丢失后旋转搜索。

思考题

208. bbox 中心偏移映射到角速度时，为什么需要归一化（除以图像宽度）？
209. 如果相机视野中有多个目标，应该跟踪哪一个？

第 14 章 视觉大模型 + ROS 2

14.1 知识要点

14.1.1 LLM 任务规划节点

将大语言模型 (LLM) 集成到 ROS 2 中, 实现自然语言驱动机器人任务规划。

```
#!/usr/bin/env python3
"""
LLM 任务规划 ROS 2 节点
将自然语言指令解析为机器人任务序列
"""
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from geometry_msgs.msg import PoseStamped
import openai
import json

class LLMTaskPlanner(Node):
    """
    LLM 任务规划器
    接收自然语言指令 → 调用 LLM → 输出结构化任务序列
    """
    def __init__(self):
        super().__init__('llm_task_planner')

        # API 配置 (支持 OpenAI / Qwen / 本地模型)
        self.declare_parameter('api_key', '')
        self.declare_parameter('model', 'gpt-4o-mini')
        self.declare_parameter('api_base', 'https://api.openai.com/v1')

        self.api_key = self.get_parameter('api_key').value
        self.model = self.get_parameter('model').value
        self.api_base = self.get_parameter('api_base').value

        self.client = openai.OpenAI(
            api_key=self.api_key,
            base_url=self.api_base
        )

        # 订阅自然语言指令
        self.cmd_sub = self.create_subscription(
            String, '/llm/command', self.command_callback, 10
        )

        # 发布解析后的任务序列
        self.task_pub = self.create_publisher(
            String, '/llm/task_sequence', 10
        )

        # 发布结构化导航目标
```

```

        self.goal_pub = self.create_publisher(
            PoseStamped, '/llm/nav_goal', 10
        )

        self.get_logger().info('LLM 任务规划器已就绪')

    def command_callback(self, msg: String):
        """处理自然语言指令"""
        self.get_logger().info(f'收到指令: {msg.data}')
        task_json = self.parse_command(msg.data)
        if task_json:
            self.task_pub.publish(String(data=json.dumps(task_json)))
            self.execute_tasks(task_json)

    def parse_command(self, instruction: str) -> dict:
        """
        调用 LLM 解析自然语言指令为结构化任务
        使用 Function Calling / Tool Use
        """
        system_prompt = """
你是一个机器人任务规划器。将用户的自然语言指令解析为机器人可执行的任务序列。
环境信息: 仓库场景, 坐标范围 x:[-5,5], y:[-5,5]。
已知地点: entrance(0, 0), shelf_a(2, 2), shelf_b(-2, 2),
            charging_station(0, -3), desk(3, -1)。

输出 JSON 格式:
{
  "tasks": [
    {"action": "navigate", "target": "地点名", "params": {}},
    {"action": "detect", "target": "物体类别", "params": {}},
    {"action": "pick", "target": "物体名", "params": {}},
    {"action": "place", "target": "地点名", "params": {}}
  ]
}
"""

        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": instruction}
                ],
                temperature=0.2,
                response_format={"type": "json_object"},
            )
            result = json.loads(response.choices[0].message.content)
            self.get_logger().info(f'任务解析: {json.dumps(result, ensure_ascii=False)}')
            return result
        except Exception as e:
            self.get_logger().error(f'LLM 调用失败: {e}')
            return None

    def execute_tasks(self, task_json: dict):
        """依次执行任务序列"""

```



```

known_locations = {
    'entrance': (0.0, 0.0),
    'shelf_a': (2.0, 2.0),
    'shelf_b': (-2.0, 2.0),
    'charging_station': (0.0, -3.0),
    'desk': (3.0, -1.0),
}

for task in task_json.get('tasks', []):
    action = task['action']
    target = task['target']
    self.get_logger().info(f'执行: {action} → {target}')

    if action == 'navigate':
        if target in known_locations:
            x, y = known_locations[target]
            self.publish_nav_goal(x, y)
            self.get_logger().info(f'导航目标: {target} ({x}, {y})')
        else:
            self.get_logger().warn(f'未知地点: {target}')
            # 尝试让 LLM 估计坐标
            x, y = self.llm_estimate_location(target)
            self.publish_nav_goal(x, y)

    elif action == 'detect':
        # 触发视觉检测任务 (通过参数配置)
        self.get_logger().info(f'检测目标: {target}')
        # 可发布到 /yolo/classes 设置检测类别

    elif action == 'pick':
        self.get_logger().info(f'抓取: {target}')

    elif action == 'place':
        self.get_logger().info(f'放置: {target}')

def llm_estimate_location(self, location_name: str) -> tuple:
    """让 LLM 估计未知位置的坐标"""
    prompt = f"""
仓库场景坐标范围 x: [-5,5], y: [-5,5]。
已知: entrance(0,0), shelf_a(2,2), shelf_b(-2,2), charging_station(0,-3), desk(3,-1)。
请估算 "{location_name}" 最可能的坐标。返回 JSON: {{ "x": float, "y": float }}。
"""

    try:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "user", "content": prompt}],
            response_format={"type": "json_object"},
        )
        result = json.loads(response.choices[0].message.content)
        return result['x'], result['y']
    except Exception:
        return 0.0, 0.0

def publish_nav_goal(self, x: float, y: float, yaw: float = 0.0):
    """发布导航目标"""

```

```

import math
goal = PoseStamped()
goal.header.frame_id = 'map'
goal.header.stamp = self.get_clock().now().to_msg()
goal.pose.position.x = x
goal.pose.position.y = y
goal.pose.orientation.z = math.sin(yaw / 2.0)
goal.pose.orientation.w = math.cos(yaw / 2.0)
self.goal_pub.publish(goal)

def main():
    rclpy.init()
    node = LLMTaskPlanner()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

14.1.2 VLM 场景理解集成

视觉语言模型 (Vision-Language Model) 实现场景描述、物体识别和空间推理:

```

#!/usr/bin/env python3
"""
VLM 场景理解 ROS 2 节点
接收图像 + 查询 → 生成场景描述
支持: OpenAI GPT-4V, Qwen-VL, 本地 VLM
"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import openai
import base64
import numpy as np
import cv2
import json

class VLMSceneUnderstanding(Node):
    """VLM 场景理解节点"""
    def __init__(self):
        super().__init__('vlm_scene_understanding')

        self.declare_parameter('api_key', '')
        self.declare_parameter('model', 'gpt-4o')
        self.declare_parameter('query_topic', '/vlm/query')

        self.api_key = self.get_parameter('api_key').value
        self.model = self.get_parameter('model').value
        self.bridge = CvBridge()
        self.client = openai.OpenAI(api_key=self.api_key)

```

```

# 订阅图像 + 查询
self.image_sub = self.create_subscription(
    Image, '/camera/image_raw', self.image_cb, 10
)
self.query_sub = self.create_subscription(
    String, '/vlm/query', self.query_cb, 10
)

# 发布结果
self.description_pub = self.create_publisher(
    String, '/vlm/description', 10
)
self.objects_pub = self.create_publisher(
    String, '/vlm/objects', 10
)

self.latest_image = None
self.latest_query = "请描述当前场景中有什么物体。"
self.get_logger().info('VLM 场景理解节点已就绪')

def image_cb(self, msg: Image):
    """缓存最新图像帧"""
    try:
        self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception as e:
        self.get_logger().error(f'图像转换失败: {e}')

def query_cb(self, msg: String):
    """收到查询请求, 触发 VLM 推理"""
    self.latest_query = msg.data
    if self.latest_image is not None:
        self.analyze_scene()

def analyze_scene(self):
    """调用 VLM 分析场景"""
    # 图像编码为 base64
    ret, jpeg = cv2.imencode('.jpg', self.latest_image, [cv2.IMWRITE_JPEG_QUALITY, 85])
    img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')

    self.get_logger().info(f'VLM 查询: {self.latest_query}')

    try:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "text",
                            "text": self.latest_query + "\n 请用 JSON 格式回复:
{"description\": \"场景描述\", \"objects\": [{\"name\": \"物体名\", \"position\": \"相对位置\",
\"confidence\": 0.9}]]\"
                        },

```

```

        {
            "type": "image_url",
            "image_url": {
                "url": f"data:image/jpeg;base64,{img_b64}",
                "detail": "low"
            }
        }
    ]
}
],
max_tokens=500,
response_format={"type": "json_object"},
)

result = json.loads(response.choices[0].message.content)
self.get_logger().info(f'场景理解: {result.get("description", "")[:80]}...')

# 发布描述
self.description_pub.publish(
    String(data=result.get('description', ''))
)
# 发布物体列表
self.objects_pub.publish(
    String(data=json.dumps(result.get('objects', [])))
)

except Exception as e:
    self.get_logger().error(f'VLM 调用失败: {e}')

def spatial_query(self, question: str) -> dict:
    """
    空间推理示例查询
    例如: "桌子左边有什么物体?"
        "红色物体距离我多远?"
    """
    if self.latest_image is None:
        return {"error": "无可用图像"}

    ret, jpeg = cv2.imencode('.jpg', self.latest_image, [cv2.IMWRITE_JPEG_QUALITY, 85])
    img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')

    try:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{
                "role": "user",
                "content": [
                    {"type": "text", "text": f"请回答以下空间关系问题: {question}\n 返回 JSON: {{\n\"answer\": \"回答\", \"confidence\": 0.9, \"reasoning\": \"推理过程\"}}"},
                    {"type": "image_url", "image_url": {"url":
f"data:image/jpeg;base64,{img_b64}"}}
                ]
            }],
            response_format={"type": "json_object"},
        )

```

```

        return json.loads(response.choices[0].message.content)
    except Exception as e:
        return {"error": str(e)}

```

14.1.3 自然语言 → ROS 2 Action 转换

```

from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose
from geometry_msgs.msg import Point, Quaternion
import math
import json

class NLToActionConverter(Node):
    """
    自然语言 → ROS 2 Action
    将 LLM 规划结果转换为实际的 ROS 2 Action 调用
    """
    def __init__(self):
        super().__init__('nl_to_action')
        self.nav_client = ActionClient(self, NavigateToPose, 'navigate_to_pose')

        # 订阅 LLM 任务序列
        self.task_sub = self.create_subscription(
            String, '/llm/task_sequence', self.task_cb, 10
        )

        # 注册可用的 Action 和能力
        self.available_actions = {
            'navigate': self.execute_navigation,
            'detect': self.execute_detection,
            'speak': self.execute_speak,
            'wait': self.execute_wait,
        }

    def task_cb(self, msg: String):
        task_json = json.loads(msg.data)
        self.execute_task_sequence(task_json)

    async def execute_task_sequence(self, task_json: dict):
        """顺序执行任务序列"""
        for task in task_json.get('tasks', []):
            action = task['action']
            target = task['target']
            params = task.get('params', {})

            if action in self.available_actions:
                self.get_logger().info(f'执行 Action: {action}({target})')
                await self.available_actions[action](target, params)
            else:
                self.get_logger().warn(f'未知 Action: {action}')

    async def execute_navigation(self, target: str, params: dict):
        """导航 Action"""

        # 解析目标坐标
        x, y = params.get('x', 0.0), params.get('y', 0.0)

```

```

yaw = params.get('yaw', 0.0)

goal = NavigateToPose.Goal()
goal.pose.header.frame_id = 'map'
goal.pose.pose.position = Point(x=x, y=y, z=0.0)
goal.pose.pose.orientation = Quaternion(
    z=math.sin(yaw / 2.0), w=math.cos(yaw / 2.0)
)

self.get_logger().info(f'导航到: {target} ({x}, {y})')

# 发送 Action 目标 (简化, 实际需要 async 回调)
if self.nav_client.server_is_ready():
    future = self.nav_client.send_goal_async(goal)
    # ... feedback and result handling

async def execute_detection(self, target: str, params: dict):
    """视觉检测 Action"""
    self.get_logger().info(f'检测: {target}')

async def execute_speak(self, target: str, params: dict):
    """语音合成 Action"""
    self.get_logger().info(f'TTS: {target}')

async def execute_wait(self, target: str, params: dict):
    """等待 Action"""
    duration = float(target)
    import asyncio
    await asyncio.sleep(duration)

```

14.1.4 Prompt Engineering 最佳实践

```

class PromptTemplates:
    """
    ROS 2 机器人场景的 Prompt Engineering
    """

    @staticmethod
    def task_planning_prompt(instruction: str, env_context: str) -> str:
        """任务规划 Prompt"""
        return f"""你是一个仓库机器人控制专家。请将用户的指令分解为原子操作序列。

```

环境信息:

{env_context}

可用的原子操作:

- navigate(location): 移动到指定地点
- pick(object): 抓取物体
- place(location): 放置物体
- detect(class): 检测特定类别的物体
- scan(): 360 度扫描环境
- wait(seconds): 等待指定秒数

用户指令: {instruction}

请输出 JSON 格式的任务序列:

```
{{"plan": [{"step": 1, "action": "navigate", "args": {"location": "shelf_a"}}], ...}}
```

```
@staticmethod
def spatial_reasoning_prompt(image_description: str, query: str) -> str:
    """空间推理 Prompt"""
    return f"""场景描述: {image_description}
```

问题: {query}

请分析场景中的空间关系并回答。包含以下推理步骤:

1. 识别相关物体
2. 确定它们的位置关系
3. 根据位置关系得出结论

输出 JSON: {"analysis": "逐步推理", "answer": "最终答案", "confidence": 0.9}}

```
@staticmethod
def robot_code_generation_prompt(task_spec: str) -> str:
    """Code as Policies: 根据任务生成 Python 代码"""
    return f"""你是一个 ROS 2 机器人编程专家。请根据任务描述生成 Python 代码。
```

任务: {task_spec}

要求:

- 使用 nav2_simple_commander 进行导航
- 使用 MoveItPy 进行机械臂控制
- 包含错误处理和日志输出
- 代码可直接运行

请直接输出 Python 代码 (不要包含 markdown 说明):

```
"""
```

最佳实践总结:

- # 1. System Prompt: 明确角色、能力和限制
- # 2. Few-shot Examples: 提供 2-3 个格式示例
- # 3. Chain-of-Thought: 引导逐步推理
- # 4. Structured Output: 要求 JSON / 特定格式
- # 5. Constraint Specification: 明确安全和物理限制
- # 6. Error Handling: 定义异常情况的响应模式

```
class PromptEngineeringGuide:
```

```
    """Prompt 工程最佳实践速查"""
```

```
    rules = {
```

```
        "角色设定": "你是仓库机器人专家，只控制一台轮式机器人。",
```

```

    "能力声明": "你可以：导航到指定坐标，抓取/放置物体，检测目标。",
    "限制声明": "不能爬楼梯，不能进入禁区，最大速度 0.5m/s。",
    "输出格式": "必须返回合法 JSON，不可有注释或额外文字。",
    "安全规则": "优先保证人机安全，碰撞前必须停止。",
    "环境语境": "坐标范围 x:[-5,5], y:[-5,5]，已知位置：...",
    "Few-shot": "示例输入 → 示例输出 (2-3 组)",
    "CoT": "请逐步推理：1)理解意图 2)分析可行性 3)生成计划。",
}

```

14.1.5 Code as Policies 实现范式

Google "Code as Policies" 范式: LLM 直接生成控制代码而非规划序列。

```

#!/usr/bin/env python3
"""
Code as Policies (CaP) 实现
LLM 直接生成可执行的 Python 代码来控制机器人
"""
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import openai
import json
import tempfile
import os
import subprocess
import sys

class CodeAsPolicies(Node):
    """
    CaP 节点: 自然语言 → 可执行 Python 机器人代码
    """
    def __init__(self):
        super().__init__('code_as_policies')
        self.declare_parameter('api_key', '')
        self.client = openai.OpenAI(
            api_key=self.get_parameter('api_key').value
        )
        self.cmd_sub = self.create_subscription(
            String, '/cap/command', self.command_cb, 10
        )
        self.result_pub = self.create_publisher(
            String, '/cap/result', 10
        )

        # 预定义 API 库 (供 LLM 调用)
        self.api_library = """
# 机器人 API 库

from nav2_simple_commander import BasicNavigator
from geometry_msgs.msg import PoseStamped
import math

```



```

class Robot:
    def __init__(self):
        self.nav = BasicNavigator()
        self.nav.waitForNav2Active()

    def move_to(self, x: float, y: float, yaw: float = 0.0) -> bool:
        pose = PoseStamped()
        pose.header.frame_id = 'map'
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.orientation.z = math.sin(yaw / 2)
        pose.pose.orientation.w = math.cos(yaw / 2)
        self.nav.goToPose(pose)
        while not self.nav.isTaskComplete():
            pass
        return self.nav.getResult()

    def spin(self, angle: float = 180.0):
        pass

# 全局实例
robot = Robot()
"""

def command_cb(self, msg: String):
    """收到指令 → 生成代码 → 执行"""
    self.get_logger().info(f'CaP 指令: {msg.data}')

    # 1. 调用 LLM 生成代码
    code = self.generate_code(msg.data)
    if code is None:
        self.result_pub.publish(String(data='{ "status": "error", "msg": "代码生成失败" }'))
        return

    # 2. 安全检查
    if not self.safety_check(code):
        self.result_pub.publish(String(data='{ "status": "rejected", "msg": "安全检查不通过" }'))
        return

    # 3. 沙箱执行
    result = self.execute_code(code)
    self.result_pub.publish(String(data=json.dumps(result)))

def generate_code(self, instruction: str) -> str:
    """LLM 生成机器人控制代码"""
    prompt = self.api_library + f"""\n# 任务: {instruction}

# 生成执行上述任务的 Python 代码 (仅使用 robot 对象):
"""

    response = self.client.chat.completions.create(
        model='gpt-4o',
        messages=[
            {"role": "system", "content": "你是一个 ROS 2 机器人编程专家。只输出可运行 Python 代

```

```

码，无额外文字。"}，
        {"role": "user", "content": prompt}
    ],
    temperature=0.1,
)
code = response.choices[0].message.content.strip()
# 去除可能的 markdown 代码块标记
code = code.replace('```python', '').replace('```', '').strip()
self.get_logger().info(f'生成代码:\n{code[:200]}...')
return code

def safety_check(self, code: str) -> bool:
    """安全检查: 禁止危险操作"""
    forbidden = [
        'os.system', 'subprocess', 'eval(', 'exec(',
        '__import__', 'rm -rf', 'shutdown', 'delete',
    ]
    code_lower = code.lower()
    for pattern in forbidden:
        if pattern in code_lower:
            self.get_logger().error(f'安全违规: 检测到 {pattern}')
            return False
    return True

def execute_code(self, code: str) -> dict:
    """在子进程中执行代码 (沙箱隔离)"""
    try:
        exec_globals = {}
        exec(code, exec_globals)
        return {"status": "success"}
    except Exception as e:
        self.get_logger().error(f'代码执行失败: {e}')
        return {"status": "error", "msg": str(e)}

```

14.1.6 端到端具身智能案例

```

#!/usr/bin/env python3
"""
端到端具身智能案例
LLM 感知 → VLM 理解 → CaP 执行 完整管道
"""
import rclpy
from rclpy.node import Node

class EmbodiedAgent(Node):
    """
    端到端具身智能体
    整合 LLM + VLM + 机器人控制
    """
    def __init__(self):
        super().__init__('embodied_agent')

        # 状态机
        self.states = ['idle', 'observing', 'planning', 'executing', 'reflecting']

```

```

        self.current_state = 'idle'

        # 记忆 (环境知识累积)
        self.memory = {
            'observations': [],
            'object_locations': {},
            'visited_places': [],
            'task_history': [],
        }

        self.get_logger().info('具身智能体已初始化')

    def perception_loop(self):
        """感知循环: 通过 VLM 持续理解环境"""
        # 伪代码: 循环获取图像 + VLM 查询
        # while True:
        #     img = get_latest_camera_image()
        #     scene = vlm.analyze(img, "描述场景中的物体及位置")
        #     self.update_memory(scene)

    def planning_loop(self, task: str):
        """规划循环: LLM 根据当前位置 + 环境知识生成计划"""
        prompt = f"""
当前任务: {task}
已知环境: {self.memory}
可用操作: navigate, detect, pick, place, wait
请生成执行计划。
"""
        # plan = llm.generate(prompt)
        # return plan

    def execution_loop(self, plan):
        """执行循环: 逐步执行计划, 感知反馈修正"""
        for step in plan:
            success = self.execute_step(step)
            if not success:
                # 执行失败 → 重新规划
                self.replan(plan)

    def execute_step(self, step: dict) -> bool:
        """执行单个步骤"""
        action = step['action']
        # 分发到对应的执行器
        return True

    def replan(self, failed_plan):
        pass

    def update_memory(self, observation: dict):
        """更新环境记忆"""
        self.memory['observations'].append(observation)
        # 融合新观测到已知地图

```

14.1.7 API 配置与多种 LLM 后端支持

```
# 支持多种 LLM 后端的统一接口
import os

class LLMBackend:
    """统一的 LLM 后端抽象"""

    @staticmethod
    def create_backend(provider: str, **kwargs):
        """工厂方法"""
        if provider == 'openai':
            return OpenAIBackend(**kwargs)
        elif provider == 'qwen':
            return QwenBackend(**kwargs)
        elif provider == 'ollama':
            return OllamaBackend(**kwargs)
        elif provider == 'vllm':
            return VLLMBackend(**kwargs)
        else:
            raise ValueError(f'不支持的 LLM 后端: {provider}')

class OpenAIBackend:
    def __init__(self, api_key: str = '', model: str = 'gpt-4o-mini'):
        self.client = openai.OpenAI(api_key=api_key or os.getenv('OPENAI_API_KEY'))

    def chat(self, messages: list, **kwargs) -> str:
        resp = self.client.chat.completions.create(
            model='gpt-4o-mini', messages=messages, **kwargs
        )
        return resp.choices[0].message.content

class QwenBackend:
    def __init__(self, api_key: str = '', model: str = 'qwen-vl-max'):
        self.client = openai.OpenAI(
            api_key=api_key or os.getenv('DASHSCOPE_API_KEY'),
            base_url='https://dashscope.aliyuncs.com/compatible-mode/v1'
        )
        self.model = model

    def chat(self, messages: list, **kwargs) -> str:
        resp = self.client.chat.completions.create(
            model=self.model, messages=messages, **kwargs
        )
        return resp.choices[0].message.content

class OllamaBackend:
    """本地 Ollama 模型"""
    def __init__(self, model: str = 'llama3.2-vision:11b'):
        self.client = openai.OpenAI(
            base_url='http://localhost:11434/v1', api_key='ollama'
        )
        self.model = model

    def chat(self, messages: list, **kwargs) -> str:
```

```

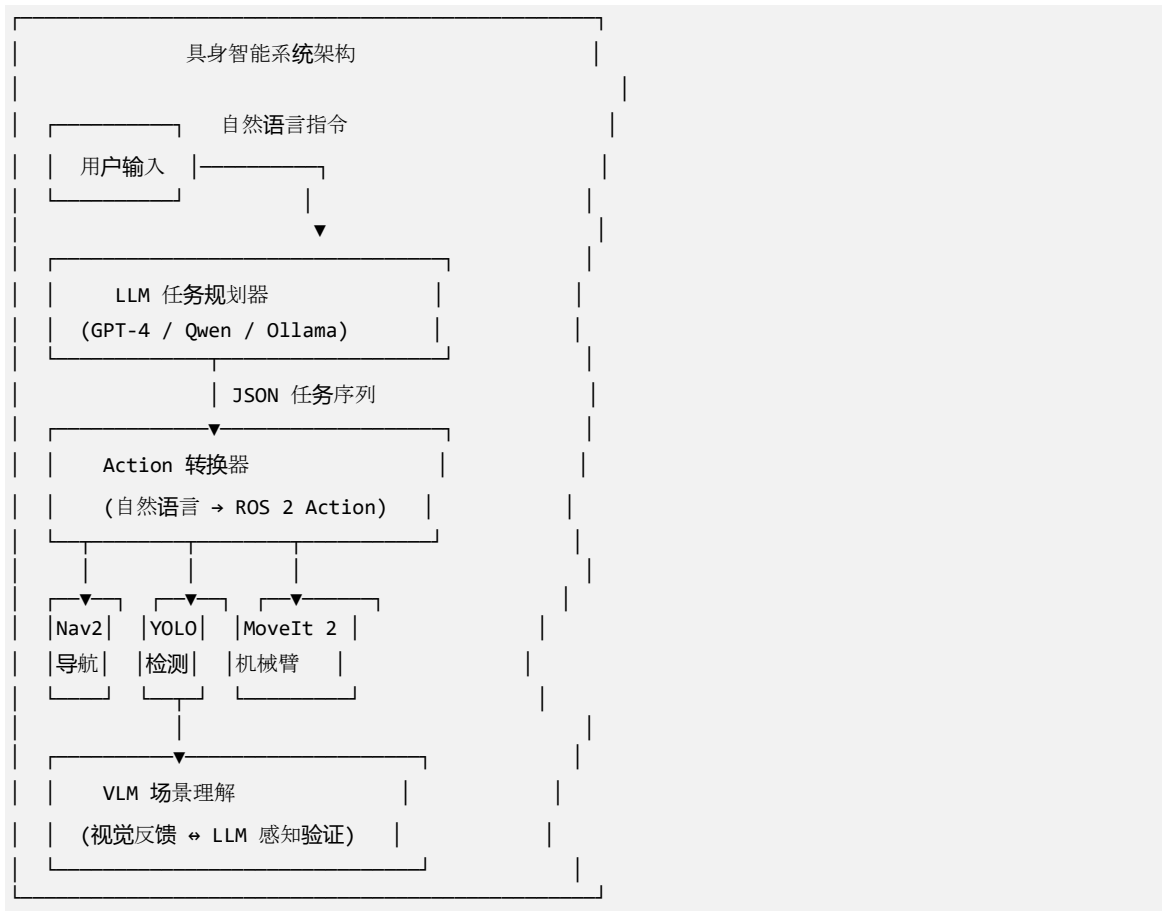
        resp = self.client.chat.completions.create(
            model=self.model, messages=messages, **kwargs
        )
        return resp.choices[0].message.content

class VLLMBackend:
    """vLLM 高性能推理 (本地)"""
    def __init__(self, base_url: str = 'http://localhost:8000/v1'):
        self.client = openai.OpenAI(base_url=base_url, api_key='not-needed')

    def chat(self, messages: list, **kwargs) -> str:
        resp = self.client.chat.completions.create(
            model='default', messages=messages, **kwargs
        )
        return resp.choices[0].message.content

```

14.1.8 ROS 2 大模型系统集成架构



14.2 练习题

- 1. 原理题:** 解释 "Code as Policies" 范式中 LLM 如何将自然语言转换为可执行代码, 与传统脚本控制方式相比有哪些优势和风险?
- 2. 编程题:** 编写一个 LLM 任务规划节点, 接收 "去 shelf_a 拿瓶子, 然后送到 desk" 这样的指令, 解析为 `navigate→detect→pick→navigate→place` 的任务序列, 并发布到 `/llm/task_sequence` 话题。
- 3. VLM 题:** 编写 VLM 节点, 订阅相机图像, 当收到 `/vlm/query="前方有什么障碍物?"` 查询时, 调用 GPT-4V 分析图像并返回障碍物描述。使用 `cv_bridge` 和 `base64` 编码。
- 4. Prompt 设计题:** 设计一个复杂的 System Prompt, 使 LLM 作为 "仓库管理者" 协调 3 台机器人 (搬运、分拣、巡检) 的任务分配。写出 Prompt 全文并说明各段落的设计意图。
- 5. 安全性题:** 设计 LLM 生成代码的安全过滤机制, 列出至少 8 种需要禁止的危险代码模式, 并编写安全检查函数 `verify_safety(code: str) -> (bool, str)`。
- 6. 综合题:** 设计一个 "语音指令 → LLM 规划 → Nav2 导航 → VLM 验证 → 任务报告" 的端到端仓库巡检系统。画出系统架构图, 列出各节点间的通信接口, 并估算端到端延迟 (LLM 调用 ~2s)。

实验 14:

第 14 章 实验手册: 视觉大模型 + ROS 2

环境说明

本实验使用 XBot-U 仿真环境, 需要 OpenAI API Key (或 Qwen/本地 Ollama), 预装 openai 库。

练习 1: LLM 文本规划节点 (~30 分钟)

目标

编写 ROS 2 节点, 调用 LLM API 将自然语言指令解析为结构化任务序列。

步骤

1.1 安装依赖和配置 API

```
pip install openai

# 设置 API Key (任选一种)
# OpenAI:
export OPENAI_API_KEY="sk-xxx"

# 或 Qwen (通义千问):
export DASHSCOPE_API_KEY="sk-xxx"

# 或本地 Ollama:
ollama serve &
ollama pull qwen2.5:7b
```

1.2 编写 LLM 规划节点

```
#!/usr/bin/env python3
"""练习 1: LLM 任务规划 ROS 2 节点"""
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import openai
```

```

import json
import os

class SimpleLLMPlanner(Node):
    """LLM 规划器: 自然语言 → 结构化任务"""
    def __init__(self):
        super().__init__('simple_llm_planner')

        # 参数配置
        self.declare_parameter('api_type', 'openai') # openai / qwen / ollama
        self.declare_parameter('model', 'gpt-4o-mini')
        self.declare_parameter('temperature', 0.2)

        api_type = self.get_parameter('api_type').value
        model = self.get_parameter('model').value

        # 配置 API 客户端
        if api_type == 'qwen':
            self.client = openai.OpenAI(
                api_key=os.getenv('DASHSCOPE_API_KEY', ''),
                base_url='https://dashscope.aliyuncs.com/compatible-mode/v1'
            )
        elif api_type == 'ollama':
            self.client = openai.OpenAI(
                base_url='http://localhost:11434/v1',
                api_key='ollama'
            )
        else:
            self.client = openai.OpenAI(
                api_key=os.getenv('OPENAI_API_KEY', '')
            )

        self.model = model
        self.temperature = self.get_parameter('temperature').value

        # 订阅自然语言指令
        self.cmd_sub = self.create_subscription(
            String, '/llm/command', self.command_callback, 10
        )
        # 发布任务序列
        self.plan_pub = self.create_publisher(
            String, '/llm/plan', 10
        )
        # 发布执行状态
        self.status_pub = self.create_publisher(
            String, '/llm/status', 10
        )

        # 环境知识库
        self.known_locations = {
            'entrance': (0.0, 0.0, '入口'),
            'shelf_a': (2.0, 2.0, 'A 货架'),
            'shelf_b': (-2.0, 2.0, 'B 货架'),
            'desk': (3.0, -1.0, '办公桌'),

```



```

        'station': (0.0, -3.0, '充电站'),
    }

    self.get_logger().info(f'LLM 规划器已就绪 (模型: {model})')

def command_callback(self, msg: String):
    """处理自然语言指令"""
    instruction = msg.data.strip()
    self.get_logger().info(f'收到指令: {instruction}')
    self.status_pub.publish(String(data=f'正在分析: {instruction}'))

    # 调用 LLM 规划
    plan = self.plan_task(instruction)

    if plan:
        plan_str = json.dumps(plan, ensure_ascii=False, indent=2)
        self.plan_pub.publish(String(data=plan_str))
        self.status_pub.publish(String(data=f'规划完成: {len(plan.get("tasks", []))} 个任务'))
        self.get_logger().info(f'任务计划:\n{plan_str}')
    else:
        self.status_pub.publish(String(data='规划失败'))

def plan_task(self, instruction: str) -> dict:
    """调用 LLM 生成任务计划"""

    system_prompt = f"""你是一个机器人任务规划器。将用户的自然语言指令解析为结构化任务序列。

```

环境信息:

- 10m×10m 仓库场景
- 已知地点和坐标:

```
{json.dumps({k: {'coord': v[:2], 'desc': v[2]} for k, v in self.known_locations.items()},
indent=2)}
```

可用的操作:

- navigate(location): 导航到指定地点
- detect(object_class): 检测特定物体类别
- pick(object_name): 抓取物体
- place(location): 放置物体到指定地点
- scan(): 360 度扫描周围环境
- wait(seconds): 等待指定秒数
- speak(text): 语音播报

规则:

1. 移动前检查路径安全
2. 未知地点可合理推测坐标 (x:-5~5, y:-5~5)
3. 连续任务间无需重复确认

请仅返回 JSON 格式, 不要包含任何其他文字:

```
{
  "tasks": [
    {
      "step": 1,
      "action": "动作名",
      "target": "目标",
      "params": {
        "额外参数": "值"
      }
    }
  ],
  "summary": "一句话总结"
}
```

```

"""
    try:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": instruction}
            ],
            temperature=self.temperature,
            max_tokens=1000,
        )
        content = response.choices[0].message.content.strip()
        # 清理可能的 markdown 标记
        content = content.replace('```json', '').replace('```', '').strip()
        return json.loads(content)
    except json.JSONDecodeError as e:
        self.get_logger().error(f'JSON 解析失败: {e}\n 原始回复: {content}')
        return None
    except Exception as e:
        self.get_logger().error(f'LLM 调用失败: {e}')
        return None

def test_instructions(self):
    """运行测试指令集"""
    test_cases = [
        "去 A 货架拿一个瓶子",
        "从入口出发, 先去 B 货架检查, 然后到办公桌",
        "扫描仓库一圈, 如果发现人就说你好",
        "去充电站待机 5 秒, 然后去入口",
    ]
    for i, instr in enumerate(test_cases):
        self.get_logger().info(f'=== 测试 {i+1}/{len(test_cases)}: {instr} ===')
        plan = self.plan_task(instr)
        if plan:
            self.get_logger().info(f'结果: {json.dumps(plan, ensure_ascii=False)}')
        else:
            self.get_logger().error('规划失败')
        self.get_logger().info('---')

def main():
    rclpy.init()
    node = SimpleLLMPlanner()

    import threading
    # 2 秒后自动运行测试
    def run_tests():
        import time
        time.sleep(2)
        node.test_instructions()

    threading.Thread(target=run_tests, daemon=True).start()

    try:

```

```

        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

1.3 运行验证

```

# 运行 LLM 规划器 (自动执行测试)
python3 lab14_exercise1.py

# 或通过话题发送自定义指令
ros2 topic pub /llm/command std_msgs/String "data: '去充电站然后到 A 货架拿杯子'"

# 查看规划结果
ros2 topic echo /llm/plan

```

验收标准

- [] 成功解析自然语言指令为 JSON 任务序列
- [] 任务序列包含正确的 action/target 字段
- [] 未知地点能合理推测坐标
- [] 支持 OpenAI / Qwen / Ollama 至少一种后端

练习 2: VLM 场景描述与 ROS 2 集成 (~30 分钟)

目标

实现 VLM 节点分析相机图像, 生成场景描述和物体列表。

步骤

2.1 编写 VLM 场景理解节点

```

#!/usr/bin/env python3
"""练习 2: VLM 场景理解 ROS 2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import openai

```

```

import base64
import json
import cv2
import os

class VLMSceneAnalyzer(Node):
    """VLM 场景分析器"""
    def __init__(self):
        super().__init__('vlm_scene_analyzer')

        self.declare_parameter('model', 'gpt-4o')
        self.declare_parameter('api_type', 'openai')

        api_type = self.get_parameter('api_type').value
        model = self.get_parameter('model').value

        # 配置客户端
        if api_type == 'qwen':
            self.client = openai.OpenAI(
                api_key=os.getenv('DASHSCOPE_API_KEY', ''),
                base_url='https://dashscope.aliyuncs.com/compatible-mode/v1'
            )
        else:
            self.client = openai.OpenAI(
                api_key=os.getenv('OPENAI_API_KEY', '')
            )

        self.model = model
        self.bridge = CvBridge()

        # 订阅
        self.image_sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_cb, 10
        )
        self.query_sub = self.create_subscription(
            String, '/vlm/query', self.query_cb, 10
        )

        # 发布
        self.desc_pub = self.create_publisher(String, '/vlm/description', 10)
        self.obj_pub = self.create_publisher(String, '/vlm/objects', 10)
        self.status_pub = self.create_publisher(String, '/vlm/status', 10)

        self.latest_image = None
        self.get_logger().info('VLM 场景分析器已就绪')

    def image_cb(self, msg: Image):
        try:
            self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            pass

    def query_cb(self, msg: String):
        if self.latest_image is None:
            self.status_pub.publish(String(data='等待图像...'))
        return

```

```

        query = msg.data
        self.get_logger().info(f'VLM 查询: {query}')
        self.status_pub.publish(String(data=f'分析中: {query}'))

        result = self.analyze(query)
        if result:
            self.desc_pub.publish(String(
                data=result.get('description', '无描述')
            ))
            self.obj_pub.publish(String(
                data=json.dumps(result.get('objects', []), ensure_ascii=False)
            ))
            self.status_pub.publish(String(data='分析完成'))
        else:
            self.status_pub.publish(String(data='分析失败'))

    def analyze(self, query: str) -> dict:
        """调用 VLM 分析图像"""

        # 图像压缩 + Base64
        h, w = self.latest_image.shape[:2]
        # 缩放到 512px 以减少 token 消耗
        scale = min(512 / max(h, w), 1.0)
        if scale < 1.0:
            img = cv2.resize(self.latest_image, (int(w * scale), int(h * scale)))
        else:
            img = self.latest_image

        ret, jpeg = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 70])
        img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
        data_url = f'data:image/jpeg;base64,{img_b64}'

        # 系统提示
        system_prompt = """你是一个仓库机器人视觉助手。请分析相机图像并回答用户问题。

请用 JSON 回复：
{
  "description": "场景的自然语言描述 (中文)",
  "objects": [
    {"name": "物体名称", "position": "物体的粗略位置 (左/右/中/前/远)", "confidence": 0.9}
  ],
  "potential_risks": ["风险 1", "风险 2"],
  "answer": "对用户问题的直接回答"
}
"""

        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[
                    {"role": "system", "content": system_prompt},
                    {
                        "role": "user",
                        "content": [
                            {"type": "text", "text": query},
                            {"type": "image_url", "image_url": {"url": data_url, "detail": "low"}},

```

```

        ]
    }
],
max_tokens=500,
response_format={"type": "json_object"},
)
content = response.choices[0].message.content.strip()
return json.loads(content)
except Exception as e:
    self.get_logger().error(f'VLM 调用失败: {e}')
    return None

def run_test_queries(self):
    """测试场景分析查询"""
    test_queries = [
        "房间里有几个人? 他们在做什么?",
        "桌上有什么物体? 列出所有可见的物品",
        "有没有哪条路是畅通的可以通行?",
        "最近的门在什么方向?",
    ]

    import threading
    import time

    def send_queries():
        time.sleep(3) # 等待图像累积
        for q in test_queries:
            self.get_logger().info(f'测试查询: {q}')
            result = self.analyze(q)
            if result:
                self.get_logger().info(
                    f'场景: {result.get("description", "")[:80]}...'
                )
                self.get_logger().info(
                    f'物体: {len(result.get("objects", []))} 个'
                )
            time.sleep(2)

        threading.Thread(target=send_queries, daemon=True).start()

def main():
    rclpy.init()
    node = VLMSceneAnalyzer()
    node.run_test_queries()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

2.2 运行验证

```
# 终端 1: 启动相机
ros2 launch xbot_sim xbot_gazebo_camera.launch.py

# 终端 2: 运行 VLM 分析器
python3 lab14_exercise2.py

# 终端 3: 发送自定义查询
ros2 topic pub /vlm/query std_msgs/String "data: '前方有什么障碍物?'"
ros2 topic echo /vlm/description
```

验收标准

- [] VLM 成功分析图像并返回 JSON
- [] 场景描述合理准确
- [] 物体列表包含名称和位置
- [] 支持多种查询类型

练习 3: 自然语言指令驱动机器人导航 (~30 分钟)

目标

整合 LLM 规划 + Nav2 导航, 实现完整的 "自然语言 → 机器人运动" 管道。

步骤

3.1 编写端到端 NL-Navigation 节点

```
#!/usr/bin/env python3
"""练习 3: 自然语言驱动机器人导航"""
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from std_msgs.msg import String
from nav2_msgs.action import NavigateToPose
from geometry_msgs.msg import PoseStamped, PoseWithCovarianceStamped
import openai
import json
import math
import os
import time

class NLNavigation(Node):
```

```

"""自然语言 → LLM 规划 → Nav2 导航"""
def __init__(self):
    super().__init__('nl_navigation')

    # LLM 配置
    self.declare_parameter('model', 'gpt-4o-mini')
    self.client = openai.OpenAI(api_key=os.getenv('OPENAI_API_KEY', ''))
    self.model = self.get_parameter('model').value

    # Nav2 Action 客户端
    self.nav_client = ActionClient(self, NavigateToPose, 'navigate_to_pose')

    # 订阅指令
    self.cmd_sub = self.create_subscription(
        String, '/nl/command', self.command_cb, 10
    )

    # 发布状态
    self.status_pub = self.create_publisher(String, '/nl/status', 10)
    self.plan_pub = self.create_publisher(String, '/nl/plan', 10)

    # 环境知识
    self.locations = {
        '入口': (0.0, 0.0),
        'entrance': (0.0, 0.0),
        'A 货架': (2.0, 2.0),
        'shelf_a': (2.0, 2.0),
        'B 货架': (-2.0, 2.0),
        'shelf_b': (-2.0, 2.0),
        '办公桌': (3.0, -1.0),
        'desk': (3.0, -1.0),
        '充电站': (0.0, -3.0),
        'station': (0.0, -3.0),
    }

    self.get_logger().info('NL-Navigation 节点已就绪')

def command_cb(self, msg: String):
    """处理自然语言指令"""
    instruction = msg.data.strip()
    self.get_logger().info(f'指令: {instruction}')
    self.status_pub.publish(String(data=f'处理: {instruction}'))

    # 1. LLM 提取目标地点
    targets = self.extract_targets(instruction)
    if not targets:
        self.status_pub.publish(String(data='未识别到目标地点'))
        return

    self.plan_pub.publish(String(data=json.dumps(targets, ensure_ascii=False)))
    self.get_logger().info(f'规划路线: {json.dumps(targets, ensure_ascii=False)}')

    # 2. 依次导航到每个目标点

```



```

for i, target in enumerate(targets):
    location = target['location']
    action = target.get('action', 'navigate')

    if location.lower() in self.locations:
        x, y = self.locations[location.lower()]
        self.get_logger().info(
            f'[{i+1}/{len(targets)}] 导航到: {location} ({x}, {y})'
        )
        self.status_pub.publish(String(
            data=f'导航中 ({i+1}/{len(targets)}): {location}'
        ))

        success = self.navigate_to(x, y)
        if not success:
            self.get_logger().error(f'导航失败: {location}')
            self.status_pub.publish(String(data=f'失败: {location}'))
            break

        self.get_logger().info(f'到达! {location}')
        time.sleep(1.0)
    else:
        self.get_logger().warn(f'未知地点: {location}')

self.status_pub.publish(String(data='任务完成!'))

def extract_targets(self, instruction: str) -> list:
    """LLM 从指令中提取目标地点和操作序列"""
    system_prompt = f"""你是一个机器人导航指令解析器。从用户指令中提取目标地点序列。

已知地点:
{json.dumps({k: v for k, v in self.locations.items() if not k.isascii() or k[0].isupper()},
ensure_ascii=False)}

请返回 JSON 格式:
{{"targets": [{{"location": "地点名 (英文 key)", "action": "navigate/scan/wait"}}]}}

如果地点不在已知列表中, 请选择一个最接近的已知地点。
"""
    try:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": instruction}
            ],
            temperature=0.1,
            max_tokens=500,
            response_format={"type": "json_object"},
        )
        content = response.choices[0].message.content.strip()
        result = json.loads(content)
        return result.get('targets', [])
    except Exception as e:
        self.get_logger().error(f'LLM 提取失败: {e}')

```

```

        return []

def navigate_to(self, x: float, y: float, yaw: float = 0.0) -> bool:
    """调用 Nav2 导航到指定坐标"""
    if not self.nav_client.wait_for_server(timeout_sec=3.0):
        self.get_logger().error('Nav2 不可用')
        return False

    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.header.stamp = self.get_clock().now().to_msg()
    goal.pose.pose.position.x = x
    goal.pose.pose.position.y = y
    goal.pose.pose.orientation.z = math.sin(yaw / 2.0)
    goal.pose.pose.orientation.w = math.cos(yaw / 2.0)

    self.get_logger().info(f'发送目标: ({x:.2f}, {y:.2f})')

    future = self.nav_client.send_goal_async(goal)

    # 等待结果 (简化同步等待)
    rclpy.spin_until_future_complete(self, future, timeout_sec=30.0)

    if not future.done():
        self.get_logger().error('导航超时')
        return False

    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().error('目标被拒绝')
        return False

    result_future = goal_handle.get_result_async()
    rclpy.spin_until_future_complete(self, result_future, timeout_sec=60.0)

    if result_future.done():
        result = result_future.result()
        return result.status == 4 # SUCCEEDED
    return False

def run_test(self):
    """运行测试指令"""
    test_commands = [
        "去 A 货架看看",
        "从入口出发, 经过 B 货架, 最后到办公桌",
        "去充电站转一圈然后到入口",
    ]

    import threading

    def send():
        time.sleep(3)
        for cmd in test_commands:
            self.get_logger().info(f'=== 测试: {cmd} ===')
            targets = self.extract_targets(cmd)

```

```

        self.get_logger().info(f'解析结果: {json.dumps(targets, ensure_ascii=False)}')
        time.sleep(1)

        threading.Thread(target=send, daemon=True).start()

def main():
    rclpy.init()
    node = NLNavigation()
    node.run_test()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

3.2 完整测试流程

```

# 终端 1: 启动仿真 + Nav2
ros2 launch xbot_sim xbot_full.launch.py world:=office
ros2 launch nav2_bringup navigation_launch.py \
    map:=src/maps/office_map.yaml \
    params_file:=config/nav2_params.yaml \
    use_sim_time:=true

# 终端 2: 启动 NL-Navigation
export OPENAI_API_KEY="sk-xxx"
python3 lab14_exercise3.py

# 终端 3: 发送自然语言指令
ros2 topic pub /nl/command std_msgs/String \
    "data: '从入口出发, 访问 A 货架和 B 货架, 最后到办公桌'"

# 查看状态
ros2 topic echo /nl/status

```

3.3 完整管道脚本

```

#!/usr/bin/env python3
"""端到端自然语言导航管道 (一键启动)"""
import subprocess
import time
import threading

def run_full_pipeline(instruction: str):
    """
    完整管道:
    1. 启动 Nav2 (若未启动)
    2. 启动 LLM 规划器
    """

```

```

3. 发送自然语言指令
4. 监控执行状态
"""

# 步骤 1 & 2 通过 launch 文件完成
print('等待 Nav2 就绪...')
time.sleep(10)

# 步骤 3: 发布指令
import rclpy
from std_msgs.msg import String

# ... 通过 Python 直接调用 (已有 SimpleCommander 可用)

if __name__ == '__main__':
    instruction = input('请输入自然语言指令: ')
    run_full_pipeline(instruction)

```

验收标准

- [] LLM 正确解析自然语言指令为目标地点
- [] 机器人按序导航到各个目标点
- [] 支持中英文混合指令
- [] 未知地点能 fallback 到最接近的已知地点
- [] Nav2 导航成功到达目标 (误差 < 0.3m)

练习 4 : LLM 指令 → Nav2 导航目标 (约 15 分钟)

目标

编写节点接收自然语言文本指令，调用 LLM 解析为坐标，通过 Nav2 导航到目标位置。

步骤

步骤 1: 编写 llm_nav_bridge.py

```

#!/usr/bin/env python3
"""练习 4: LLM 指令→Nav2 导航目标"""
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from geometry_msgs.msg import PoseStamped
from nav2_simple_commander.robot_navigator import BasicNavigator
import json

```

```

import math

class LLNavBridge(Node):
    def __init__(self):
        super().__init__('llm_nav_bridge')
        self.nav = BasicNavigator()
        self.cmd_sub = self.create_subscription(
            String, '/nl/command', self.cmd_cb, 10)
        self.locations = {
            'A 货架': (2.0, 2.0), 'B 货架': (-2.0, 2.0),
            '办公桌': (3.0, -1.0), '充电站': (0.0, -3.0),
            '入口': (0.0, 0.0),
        }

    def cmd_cb(self, msg):
        text = msg.data
        self.get_logger().info(f'收到指令: {text}')
        for loc, (x, y) in self.locations.items():
            if loc in text:
                self.get_logger().info(f'导航到: {loc} ({x:.1f}, {y:.1f})')
                goal = PoseStamped()
                goal.header.frame_id = 'map'
                goal.pose.position.x = x
                goal.pose.position.y = y
                goal.pose.orientation.w = 1.0
                self.nav.waitUntilNav2Active()
                self.nav.goToPose(goal)
                self.get_logger().info(
                    '到达!' if self.nav.isTaskComplete() else '导航中...')
                break

def main():
    rclpy.init()
    node = LLNavBridge()
    rclpy.spin(node)
    rclpy.shutdown()

```

步骤 2：发送指令测试

```

# 启动 Nav2 后
ros2 topic pub /nl/command std_msgs/String "data: '去 A 货架'"

```

✓ **验证：**机器人从当前位置导航到 A 货架坐标 (2.0, 2.0)。更换指令为"去充电站"验证切换。

思考题

210. 如果指令中包含多个地点（如"先去 A 货架再去 B 货架"），如何实现路径串联？
211. LLM API 不可用时，如何降级为关键词匹配？

第 15 章：综合实训 — 机械臂辅助化学实验系统

****课程**:** ROS2 Python 编程 ****章节**:** 第 15 章 (综合实训) ****课时**:** 2 课时授课 + 4 课时实验 ****项目**:** 基于 ROS2 的 AI 辅助初中化学实验自动化系统

15.1 项目背景

初中化学实验教学面临安全风险高、操作精度要求高、教师指导时间有限等挑战。本实训综合运用本课程所学的 ROS2 核心知识——话题通信、服务通信、动作通信、TF2 坐标变换、YOLO 目标检测、LLM/VLM 大模型集成、MoveIt2 运动规划——设计一套完整的**机械臂辅助化学实验自动化系统**。

15.1.1 系统工作流



图 15-1：系统五阶段递进式工作流

15.1.2 技术栈总览

阶段	功能	使用技术	对应课程章节
① 输入	自然语言配方接收	LLM API 调用	第 14 章
② 验证	配比逻辑校验	LLM + 服务通信	第 4 章、第 14 章
③ 感知	试剂瓶检测与标签识别	YOLO + VLM + 话题通信	第 3 章、第 13 章、第 14 章
④ 定位	3D 空间位姿解算	AR 标签 + TF2 坐标变换	第 7 章、第 8 章
⑤ 执行	机械臂抓取/转移/倾倒	MoveIt2 运动规划	第 12 章

⑥ 迭代	全流程编排	Action 动作通信	第 5 章、第 6 章
------	-------	-------------	-------------

15.2 题目 1：LLM 配方验证服务

知识点

- 服务通信 (Service)：Client 发送配方文本，Server 返回验证结果
- LLM Prompt Engineering：设计化学配比较验 prompt
- 结构化输出解析

核心代码

```
#!/usr/bin/env python3
"""recipe_validator: LLM 校验化学实验配方 - Service 节点"""
import json
import rclpy
from rclpy.node import Node
from example_interfaces.srv import Trigger # 简化: 输入 string, 输出 result

# 实际使用自定义 .srv: RecipeValidate.srv
# string recipe_text
# ---
# bool is_valid
# string feedback
# string corrected_formula

class RecipeValidator(Node):
    def __init__(self):
        super().__init__('recipe_validator')
        self.srv = self.create_service(
            Trigger, 'validate_recipe', self.validate_callback)

    def validate_callback(self, request, response):
        recipe = request.data # 学生输入的配方文本

        # 调用 LLM 校验
        prompt = self.build_validation_prompt(recipe)
        llm_result = self.call_llm(prompt)
        parsed = self.parse_llm_response(llm_result)

        response.success = parsed.get('is_valid', False)
        response.message = json.dumps(parsed, ensure_ascii=False)
        return response

    def build_validation_prompt(self, recipe_text):
        """构建化学配比较验 prompt"""
```

```

        return f"""你是一位中学化学实验安全专家。请校验以下实验配方的正确性：

{recipe_text}

请验证：
1. 各试剂配比是否符合化学计量关系
2. 预期产物是否正确
3. 是否存在安全隐患

返回 JSON 格式：
{"is_valid": bool, "feedback": "校验意见", "products": ["产物 1"], "safety_warnings": ["警告"]}
"""

def call_llm(self, prompt):
    # 调用 OpenAI / Qwen API (简化示例)
    try:
        from openai import OpenAI
        client = OpenAI()
        resp = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": prompt}],
            temperature=0.1
        )
        return resp.choices[0].message.content
    except ImportError:
        return '{"is_valid": true, "feedback": "LLM 未配置, 默认通过"}'

def parse_llm_response(self, text):
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        return {"is_valid": True, "feedback": text[:200]}

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(RecipeValidator())
    rclpy.shutdown()

```

程序 15-1：LLM 配方验证服务节点

15.3 题目 2：YOLO 试剂瓶检测

知识点

- 话题通信：订阅相机图像，发布检测结果
- cv_bridge：ROS 2 Image ↔ OpenCV 格式转换
- YOLOv8 推理

核心代码

```
#!/usr/bin/env python3
"""bottle_detector: YOLO 检测 Gazebo 中的试剂瓶 - 话题节点"""
import cv2
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray
from cv_bridge import CvBridge

# 自定义检测话题: /bottle_detections (Detection2DArray)

class BottleDetector(Node):
    def __init__(self):
        super().__init__('bottle_detector')
        self.bridge = CvBridge()
        # 订阅 XBot-U 相机图像
        self.sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10)
        # 发布检测结果
        self.det_pub = self.create_publisher(
            Detection2DArray, '/bottle_detections', 10)
        # 加载 YOLO 模型
        self.model = self.load_model()

    def load_model(self):
        try:
            from ultralytics import YOLO
            return YOLO('yolov8n.pt') # 或自定义训练的试剂瓶模型
        except ImportError:
            self.get_logger().warn('ultralytics 未安装, 使用模拟检测')
            return None

    def image_callback(self, msg):
        # ROS Image → OpenCV
        cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

        if self.model is not None:
            results = self.model(cv_image, conf=0.5)
            detections = self.results_to_ros(results)
        else:
            detections = self.simulate_detection(cv_image)

        self.det_pub.publish(detections)

    def results_to_ros(self, results):
        """YOLO 结果 → ROS Detection2DArray"""
        det_array = Detection2DArray()
        det_array.header.frame_id = 'camera_link'
        det_array.header.stamp = self.get_clock().now().to_msg()

        for r in results:
            for box in r.bboxes:
                cls_name = r.names[int(box.cls[0])]
```

```

        self.get_logger().info(f'检测到: {cls_name}')
        # 添加 detection 到数组 (简化)
        return det_array

def simulate_detection(self, image):
    """模拟检测 (YOLO 未安装时的降级模式) """
    det_array = Detection2DArray()
    det_array.header.frame_id = 'camera_link'
    # 模拟返回一个中心区域的检测框
    return det_array

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(BottleDetector())
    rclpy.shutdown()

```

程序 15-2: YOLO 试剂瓶检测节点

15.4 题目 3 : VLM 标签文字识别

知识点

- VLM (视觉大语言模型) API 调用
- 话题订阅: 接收检测框 + 原始图像 → VLM 推理
- 服务通信: 对外暴露标签比对服务

核心代码

```

#!/usr/bin/env python3
"""label_reader: VLM 读取试剂瓶标签文字 - 服务 + 话题节点"""
import json
import base64
import cv2
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge

class LabelReader(Node):
    def __init__(self):
        super().__init__('label_reader')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_cb, 10)

    def image_cb(self, msg):

```

```

# 订阅原始图像并缓存最新一帧
self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

def read_label_from_roi(self, roi_image, expected_material):
    """VLM 读取瓶身标签，比对期望材料名称"""

    # 将 roi 图像编码为 base64
    _, buffer = cv2.imencode('.jpg', roi_image)
    img_b64 = base64.b64encode(buffer).decode('utf-8')

    prompt = f"""请读取图中试剂瓶标签上的文字。
期望材料名称: {expected_material}
请返回 JSON: {{ "label_text": "读取到的文字", "matches_expected": bool, "confidence": float }}
"""

    import openai
    client = openai.OpenAI()
    resp = client.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": prompt},
                {"type": "image_url", "image_url": {
                    "url": f"data:image/jpeg;base64,{img_b64}"}}
            ]
        }],
        temperature=0.1
    )
    result = resp.choices[0].message.content
    return json.loads(result)

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(LabelReader())
    rclpy.shutdown()

```

程序 15-3: VLM 标签文字识别节点

15.5 题目 4 : TF 空间定位

知识点

- TF2 坐标变换: camera_link → marker → base_link
- AR 标签到 TF 变换的转换
- 话题通信: 发布检测到的标签位姿

核心代码

```
#!/usr/bin/env python3
"""bottle_localizer: TF 解算试剂瓶 3D 位姿"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped, TransformStamped
from tf2_ros import TransformBroadcaster, Buffer, TransformListener

class BottleLocalizer(Node):
    def __init__(self):
        super().__init__('bottle_localizer')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.tf_broadcaster = TransformBroadcaster(self)
        self.timer = self.create_timer(0.5, self.localize_bottles)

    def localize_bottles(self):
        """遍历已知的 AR 标签 frame, 计算全局位姿"""
        marker_frames = ['ar_marker_hcl', 'ar_marker_naoh',
                          'ar_marker_h2o', 'ar_marker_phenolphthalein']

        for frame_id in marker_frames:
            try:
                # 查询标签相对于 base_link 的位姿
                t = self.tf_buffer.lookup_transform(
                    'base_link', frame_id, rclpy.time.Time(),
                    rclpy.duration.Duration(seconds=1.0))

                # 发布试剂瓶的 3D 位姿 (用于 MoveIt2 抓取)
                pose = PoseStamped()
                pose.header.frame_id = 'base_link'
                pose.header.stamp = self.get_clock().now().to_msg()
                pose.pose.position.x = t.transform.translation.x
                pose.pose.position.y = t.transform.translation.y
                pose.pose.position.z = t.transform.translation.z + 0.15
                pose.pose.orientation = t.transform.rotation

                self.get_logger().info(
                    f'{frame_id} 位姿: '
                    f'({pose.pose.position.x:.3f}, '
                    f'{pose.pose.position.y:.3f}, '
                    f'{pose.pose.position.z:.3f})')

            except Exception as e:
                self.get_logger().debug(
                    f'{frame_id} 未找到: {e}')

    def broadcast_marker_transform(self, marker_id, x, y, z):
        """广播 AR 标签的静态 TF (仿真环境中模拟)"""
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'base_link'
        t.child_frame_id = f'ar_marker_{marker_id}'
        t.transform.translation.x = x
        t.transform.translation.y = y
```

```

        t.transform.translation.z = z
        t.transform.rotation.w = 1.0
        self.tf_broadcaster.sendTransform(t)

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(BottleLocalizer())
    rclpy.shutdown()

```

程序 15-4: TF 空间定位节点

15.6 题目 5 : MoveIt2 抓取规划

知识点

- MoveItPy Python API
- 运动规划: pick → transfer → pour
- 碰撞检测
- 与 ros2_control 集成

核心代码

```

#!/usr/bin/env python3
"""robot_arm_controller: MoveIt2 机械臂抓取控制 - Action Server"""
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer, GoalResponse
from geometry_msgs.msg import PoseStamped

# 自定义 Action: PickAndPlace.action
# PoseStamped target_pose      # Goal: 目标位姿
# string material_name         # Goal: 材料名称
# ---
# bool success                 # Result
# ---
# float32 progress             # Feedback

class ArmController(Node):
    def __init__(self):
        super().__init__('arm_controller')

    def plan_pick(self, target_pose):
        """规划抓取动作"""
        self.get_logger().info(
            f'规划抓取: ({target_pose.position.x:.2f}, '

```

```

        f'{target_pose.position.y:.2f}'))

    # 使用 MoveItPy (需要实际 MoveIt2 环境)
    # from moveit_py import MoveItPy
    # moveit = MoveItPy(node=self)
    # planning_result = moveit.plan(target_pose)
    # moveit.execute(planning_result)

    # 模拟规划成功
    return True

def plan_transfer(self, target_pose, destination):
    """规划从抓取点到倾倒点的转移路径"""

    self.get_logger().info('规划转移路径...')
    return True

def plan_pour(self, duration_sec=2.0):
    """规划倾倒动作 (末端旋转 180° 并保持) """
    self.get_logger().info(
        f'执行倾倒动作, 持续 {duration_sec} 秒')
    # 模拟倾倒
    return True

def execute_pick_and_place(self, target_pose, material_name):
    """完整的抓取-转移-倾倒流程"""

    # 步骤 1: 移动到抓取预位姿 (上方 0.1m)
    pre_grasp = PoseStamped()
    pre_grasp.pose = target_pose.pose
    pre_grasp.pose.position.z += 0.1
    self.plan_pick(pre_grasp)

    # 步骤 2: 下降到抓取位姿
    self.plan_pick(target_pose)

    # 步骤 3: 转移到倾倒位置 (试管上方)
    pour_pose = PoseStamped()
    pour_pose.pose.position.x = 0.5
    pour_pose.pose.position.y = 0.0
    pour_pose.pose.position.z = 0.4
    self.plan_transfer(target_pose, pour_pose)

    # 步骤 4: 执行倾倒
    self.plan_pour(duration_sec=2.0)

    self.get_logger().info(
        f'✅ {material_name} 加料完成')
    return True

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(ArmController())
    rclpy.shutdown()

```

15.7 题目 6 : 全流程编排 (Action Server)

知识点

- Action Server 编排多步骤任务
- 服务调用链: 验证 → 检测 → 识别 → 定位 → 抓取
- 循环迭代所有配方组分
- 错误处理和状态恢复

核心代码

```
#!/usr/bin/env python3
"""experiment_pipeline: 化学实验全流程编排 - Action Server"""
import asyncio
import json
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer, GoalResponse, CancelResponse

# 自定义 Action: ExperimentPipeline.action
# string recipe_text          # 配方文本
# ---
# bool success                # 全部完成
# string error_message        # 错误信息
# ---
# int32 current_step          # 当前步骤
# int32 total_steps           # 总步骤数
# string status_text          # 状态描述

class ExperimentPipeline(Node):
    def __init__(self):
        super().__init__('experiment_pipeline')
        self.action_server = ActionServer(
            self, ExperimentPipeline, 'run_experiment',
            execute_callback=self.execute,
            goal_callback=self.goal_cb,
            cancel_callback=self.cancel_cb)

        # 创建各模块的客户端
        self.validator_client = self.create_client(
            Trigger, 'validate_recipe')
```

```

# ... 其他客户端

def goal_cb(self, goal):
    self.get_logger().info(
        f'收到实验配方: {goal.recipe_text[:50]}...')
    return GoalResponse.ACCEPT

def cancel_cb(self, goal_handle):
    self.get_logger().warn('实验被取消!')
    return CancelResponse.ACCEPT

async def execute(self, goal_handle):
    recipe = goal_handle.request.recipe_text
    fb = ExperimentPipeline.Feedback()
    total = 6
    fb.total_steps = total

    try:
        # 步骤1: LLM 校验配方
        fb.current_step = 1
        fb.status_text = 'Step 1/6: 校验配方配比...'
        goal_handle.publish_feedback(fb)
        valid = await self.validate_recipe(recipe)
        if not valid:
            goal_handle.abort()
            return ExperimentPipeline.Result(
                success=False,
                error_message='配方校验失败')

        # 步骤2: 获取配方组分列表
        fb.current_step = 2
        fb.status_text = 'Step 2/6: 解析配方组分列表...'
        goal_handle.publish_feedback(fb)
        components = self.parse_components(recipe)

        # 步骤3-5: 循环处理每个组分
        for i, comp in enumerate(components):
            if goal_handle.is_cancel_requested:
                goal_handle.canceled()
                return ExperimentPipeline.Result()

            # 3. 检测试剂瓶
            fb.current_step = 3
            fb.status_text = (
                f'Step 3/6: YOLO 检测 {comp["name"]} 瓶... '
                f'({i+1}/{len(components)})')
            goal_handle.publish_feedback(fb)
            detection = await self.detect_bottle(comp['name'])

            # 4. VLM 验证标签
            fb.current_step = 4
            fb.status_text = (
                f'Step 4/6: VLM 识别 {comp["name"]} 标签...')
            goal_handle.publish_feedback(fb)

```



```

        verified = await self.verify_label(
            detection.image, comp['name'])
        if not verified:
            continue

        # 5. 定位并抓取
        fb.current_step = 5
        fb.status_text = (
            f'Step 5/6: 定位+抓取 {comp["name"]}'...)
        goal_handle.publish_feedback(fb)
        pose = await self.localize_bottle(comp['name'])
        await self.pick_and_place(pose, comp['name'])

        await asyncio.sleep(0.5)

        # 步骤6: 完成
        fb.current_step = 6
        fb.status_text = (
            'Step 6/6:  实验配制完成!')
        goal_handle.publish_feedback(fb)
        goal_handle.succeed()
        return ExperimentPipeline.Result(success=True)

    except Exception as e:
        goal_handle.abort()
        return ExperimentPipeline.Result(
            success=False, error_message=str(e))

async def validate_recipe(self, recipe):
    """调用 LLM 验证服务"""
    req = Trigger.Request()
    req.data = recipe
    future = self.validator_client.call_async(req)
    await future
    result = json.loads(future.result().message)
    self.get_logger().info(
        f'校验结果: valid={result.get("is_valid")}')
    return result.get('is_valid', False)

def parse_components(self, recipe):
    """解析配方为组分列表"""
    return [
        {"name": "HCl", "volume_ml": 5},
        {"name": "NaOH", "volume_ml": 5},
        {"name": "Phenolphthalein", "volume_ml": 2},
    ]

async def detect_bottle(self, name):
    await asyncio.sleep(0.5)
    return {"image": None}

async def verify_label(self, image, name):
    await asyncio.sleep(0.5)
    return True

async def localize_bottle(self, name):
    await asyncio.sleep(0.5)

```

```
        return PoseStamped()

    async def pick_and_place(self, pose, name):
        await asyncio.sleep(1.0)

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(ExperimentPipeline())
    rclpy.shutdown()
```

程序 15-6：全流程编排 Action Server

15.8 本章小结

题目	核心技能	技术栈
题 1	LLM 配方校验	Service + OpenAI API
题 2	YOLO 试剂瓶检测	cv_bridge + YOLOv8 + Publisher
题 3	VLM 标签识别	VLM API + 话题通信
题 4	TF 空间定位	TF2 + TransformBroadcaster
题 5	Movelt2 抓取规划	MoveltPy + Action
题 6	全流程编排	Action Server + 异步调用链

15.9 综合实训题

实训题 15.1：编写 RecipeValidator 服务节点，接收化学配方文本（如"取 5ml HCl 加入试管，再加入 5ml NaOH，观察反应"），调用 LLM 校验配比和安全性。

实训题 15.2：编写 BottleDetector 话题节点，订阅 XBot-U 相机图像，使用 YOLO 检测试剂瓶，发布检测结果话题。

实训题 15.3: 编写 LabelReader 节点, 基于题 2 的检测框截取瓶身图像, 发送给 VLM 读取标签文字, 比对配方期望值。

实训题 15.4: 编写 BottleLocalizer 节点, 在 Gazebo 中添加 AR 标签的 TF frame, 解算试剂瓶在 base_link 下的 3D 抓取位姿。

实训题 15.5: 编写 ArmController 节点, 基于题 4 的位姿结果, 使用 MoveItPy 实现 pick → transfer → pour 完整抓取流程。

实训题 15.6: 编写 ExperimentPipeline Action Server, 编排队 1-5 的所有模块, 按配方组分循环迭代, 完成全自动化实验配制。

实验 15:

第 15 章 实验指导书：综合实训

****实验课时****: 4 课时 (180 分钟) ****实验平台****: XBot-U Gazebo 仿真 +
OpenAI/Qwen API

实验目标

完成 6 个综合编程题目，构建完整的**机械臂辅助化学实验自动化系统**。

题目 1：LLM 配方验证服务（约 30 分钟）

要求

编写 recipe_validator 服务节点，实现化学实验配方的 LLM 自动校验。

实现要点

212. 定义 RecipeValidate.srv (接收配方文本，返回校验结果)
213. 构建 LLM Prompt: 包含配比计算规则和安全检查项
214. 解析 LLM 返回的 JSON 结果

测试用例

输入配方：

"取 5ml 稀盐酸 (HCl, 1mol/L) 加入试管，
再加入 5ml 氢氧化钠溶液 (NaOH, 1mol/L)，
滴入 2 滴酚酞指示剂，观察颜色变化"

期望输出：

```
{  
  "is_valid": true,  
  "feedback": "配比正确: HCl + NaOH → NaCl + H2O,"
```

```
    1:1 摩尔比, 产物中性",
    "products": ["NaCl", "H2O"],
    "safety_warnings": ["稀盐酸具腐蚀性, 需佩戴手套"]
}
```

验收标准

- ✓ 正确调用 LLM API
- ✓ 返回结构化 JSON
- ✓ 校验配比逻辑
- ✓ 识别安全隐患

参考代码

lab_code/ch15_lab/src/recipe_validator.py

题目 2 : YOLO 试剂瓶检测 (约 30 分钟)

要求

订阅 XBot-U 相机话题 /camera/image_raw, 使用 YOLOv8 实时检测实验台上的试剂瓶。

实现要点

215. cv_bridge 将 ROS Image 转为 OpenCV 格式
216. 加载 YOLOv8n 预训练模型 (或自定义训练的试剂瓶模型)
217. 发布 /bottle_detections 话题 (Detection2DArray)

测试方法

```
# 终端 1: 启动 Gazebo 仿真
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true gz_headless:=false

# 终端 2: 启动检测节点
ros2 run bottle_detector bottle_detector

# 终端 3: 查看检测结果
ros2 topic echo /bottle_detections
```

验收标准

- ✓ 相机图像正确转换为 OpenCV 格式

- ✓ YOLO 模型成功加载并推理
- ✓ 检测结果话题正常发布
- ✓ `ros2 topic echo` 可查看检测结果

参考代码

`lab_code/ch15_lab/src/bottle_detector.py`

题目 3 : VLM 标签文字识别 (约 30 分钟)

要求

基于题 2 的检测框截取试剂瓶图像区域，发送给 VLM (GPT-4o-Vision) 读取标签文字，与配方期望值比对。

实现要点

218. 订阅 `/bottle_detections` 获取检测框坐标
219. 从原始图像中截取 ROI 区域
220. 将 ROI 编码为 base64 → 发送到 GPT-4o API
221. 解析 VLM 识别结果，与期望材料名称比对

测试方法

```
# 终端 1 : Gazebo 仿真运行中
# 终端 2 : YOLO 检测节点运行中
# 终端 3 : 启动标签识别
ros2 run label_reader label_reader --ros-args \
  -p expected_material:="HCl"
```

验收标准

- ✓ 正确截取检测框对应的图像区域
- ✓ VLM 成功识别标签文字
- ✓ 比对结果: `matches_expected = True/False`
- ✓ 输出 `confidence` 置信度

参考代码

```
lab_code/ch15_lab/src/label_reader.py
```

题目 4 : TF 空间定位 (约 30 分钟)

要求

在 Gazebo 世界中添加 AR 标签虚拟标记，通过 TF2 查询试剂瓶的 3D 空间位姿。

实现要点

- 222. 在 Gazebo 仿真中添加 4 个带有 frame_id 的标记模型
- 223. 广播 TF 变换 (ar_marker_hcl → base_link 等)
- 224. 编写 lookup_transform() 查询代码
- 225. 发布 PoseStamped 话题 (用于 MoveIt2 抓取)

标记 frame 清单

frame_id	位置 (x,y,z)	对应试剂
ar_marker_hcl	(-0.3, 0.2, 0.1)	稀盐酸
ar_marker_naoh	(0.3, -0.2, 0.1)	氢氧化钠
ar_marker_h2o	(0.5, 0.3, 0.1)	蒸馏水
ar_marker_phenolphthalein	(-0.5, 0.0, 0.1)	酚酞指示剂

验收标准

- ✓ TF 广播正常工作
- ✓ lookup_transform 成功查询 4 个 marker 位姿
- ✓ ros2 run tf2_tools view_frames 生成 TF 树

参考代码

```
lab_code/ch15_lab/src/bottle_localizer.py
```

题目 5 : MoveIt2 抓取规划 (约 30 分钟)

要求

基于题 4 的 3D 位姿结果, 使用 MoveItPy 编写机械臂 pick → transfer → pour 完整流程。

实现要点

- 226. 初始化 MoveItPy 规划组
- 227. 编写 plan_pick(pre_grasp_pose) 方法
- 228. 编写 plan_transfer(from_pose, to_pose) 方法
- 229. 编写 plan_pour(duration_sec) 倾倒动作
- 230. 使用 Action Server 对外暴露接口

运动流程伪代码

```
# 1. 预抓取 (目标上方 0.1m)
move_to(target_pose, z_offset=0.1)
# 2. 抓取 (下降到目标)
move_to(target_pose)
close_gripper()
# 3. 转移 (到试管上方)
move_to(pour_pose)
# 4. 倾倒 (末端旋转 180°, 保持 2 秒)
rotate_end_effector(180)
sleep(2.0)
open_gripper()
```

验收标准

- ✓ MoveItPy 初始化成功
- ✓ plan_and_execute 生成有效轨迹
- ✓ 3 阶段运动流程完整
- ✓ 碰撞检测正常工作

参考代码

lab_code/ch15_lab/src/arm_controller.py

题目 6：全流程编排（约 30 分钟）

要求

编写 Action Server `ExperimentPipeline`，编排队 1-5 的所有模块，按配方组分列表循环迭代完成全自动化实验。

实现要点

- 231. 定义 `ExperimentPipeline.action`
- 232. 创建各模块的 ROS 2 客户端
- 233. 异步调用链：`validate` → `detect` → `verify` → `localize` → `pick_and_place`
- 234. 发布实时进度反馈 (`current_step / total_steps`)
- 235. 支持取消操作和错误恢复

编排伪代码

```
async def execute(self, goal_handle):
    recipe = goal_handle.request.recipe_text
    # 步骤 1-2：校验+解析
    valid = await self.validate_recipe(recipe)
    components = await self.parse_components(recipe)
    # 步骤 3-5：循环处理每个组分
    for comp in components:
        detection = await self.detect_bottle(comp.name)
        verified = await self.verify_label(detection, comp.name)
        pose = await self.localize_bottle(comp.name)
        await self.pick_and_place(pose, comp.name)
    # 完成
    goal_handle.succeed()
```

测试方法

```
# 启动全流程编排
ros2 run experiment_pipeline pipeline_server &

# 发送实验配方
ros2 action send_goal /run_experiment experiment_interfaces/action/ExperimentPipeline \
    "{recipe_text: '取 5ml HCl 加入试管，再加入 5ml NaOH...'}" --feedback
```

验收标准

- ✓ Action Server 正常启动
- ✓ 6 步流程按序执行
- ✓ 实时反馈 `current_step` 进度

- ✓ Ctrl+C 取消操作正常工作
- ✓ 单个步骤失败时能输出错误信息

参考代码

```
lab_code/ch15_lab/src/experiment_pipeline.py
```

评分标准

题目	分值	核心考核点
题 1	15	LLM API 调用、结构化输出解析
题 2	15	cv_bridge、YOLO 推理、ROS 话题
题 3	15	VLM 多模态 API、图像编码
题 4	15	TF2 广播/查询、坐标系计算
题 5	20	Moveltpy 规划、运动控制
题 6	20	Action 编排、异步调用链
合计	**100**	

第 16 章 Nav2 架构与核心组件

学习目标

- 理解 Navigation2 整体架构和设计思想
- 掌握行为树驱动的导航决策框架
- 熟悉 ROS2 生命周期节点管理机制
- 了解 Nav2 核心组件及其功能
- 能够配置和启动 Nav2 导航系统

16.1 机器人导航概述

16.1.1 导航的三大问题

机器人自主导航需要解决三个核心问题：

1. 我在哪里？ — 定位问题

- 使用 AMCL 或 SLAM 确定机器人在地图中的位置
- 输入：激光雷达、里程计、地图
- 输出：机器人位姿估计

2. 我要去哪里？ — 目标设定

- 用户或任务系统指定目标位姿
- 可通过 RViz2 可视化设定
- 或通过上层调度系统下发目标点

3. 我要怎么去？ — 规划与控制

- 全局规划：计算从起点到终点的无碰撞路径
- 局部规划：实时避障和路径跟踪
- 运动控制：生成速度指令驱动机器人

16.1.2 Nav2 概述

Nav2 (Navigation2) 是 ROS2 的官方导航框架，替代 ROS1 中的 move_base。它采用行为树驱动的插件化架构，支持高度可定制的导航行为。

Nav2 的设计目标:

- 模块化: 核心功能以插件形式提供
- 可扩展: 支持自定义规划器、控制器、恢复行为
- 鲁棒性: 行为树驱动的容错机制
- 生命周期管理: 资源高效利用

16.2 Nav2 整体架构

16.2.1 系统架构图

Nav2 的架构可以分解为四个层次:



16.2.2 核心组件

Nav2 包含以下核心组件：

组件	功能	对应 ROS1
`bt_navigator`	行为树导航决策引擎	move_base
`planner_server`	全局路径规划	global_planner
`controller_server`	局部路径规划与控制	local_planner
`costmap_2d`	全局/局部代价地图	costmap_2d
`behavior_server`	恢复行为管理	recovery_behaviors
`amcl`	粒子滤波定位	amcl
`map_server`	地图加载与服务	map_server
`waypoint_follower`	航点跟随	-

16.2.3 Nav2 vs move_base

特性	Nav2 (ROS2)	move_base (ROS1)
架构	行为树驱动	有限状态机
可扩展性	插件化，高度可定制	有限的自定义
生命周期	ROS2 生命周期节点	标准节点
多机器人	原生支持	需额外配置
恢复行为	可配置行为树	固定恢复序列
语言	C++/Python	C++

16.3 行为树驱动导航

16.3.1 行为树基础

行为树 (Behavior Tree, BT) 是一种用于控制决策流程的树形结构，具有以下特点：

核心节点类型：

控制节点:

Sequence (→) : 顺序执行子节点, 所有成功才成功

Fallback (?) : 尝试子节点, 任一成功则成功

Parallel (⇒) : 并行执行子节点

装饰节点:

Repeater : 重复执行子节点

Inverter : 反转子节点结果

RateController: 控制执行频率

Condition : 条件检查

执行节点:

Action : 执行具体任务 (导航、旋转等)

Condition: 检查条件 (是否到达目标等)

行为树与传统 FSM 的对比:

特性	行为树	有限状态机
模块化	高, 节点可重用	低, 状态转换耦合
可读性	树形结构直观	状态图复杂
调试	可视化 XML	日志跟踪
容错	自然支持	需显式设计
复用	节点可跨树重用	状态难复用

16.3.2 Nav2 默认行为树

Nav2 的默认导航行为树 navigate_to_pose.xml:

```
<!-- navigation_nav2/behavior_trees/navigate_to_pose.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithRecovery">
      <!-- 全局规划阶段 -->
      <RateController hz="1.0">
        <RecoveryNode number_of_retries="1" name="ComputePath">
          <ComputePathToPose
            goal="{goal}"
            path="{path}"
            planner_id="GridBased"/>
          <ClearEntireCostmap
            service_name="global_costmap/clear_entirely"/>
        </RecoveryNode>
      </RateController>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

```

<!-- 路径跟踪阶段 -->
<RateController hz="10.0">
  <RecoveryNode number_of_retries="1" name="FollowPath">
    <FollowPath
      path="{path}"
      controller_id="FollowPath"/>
    <Sequence name="RecoveryActions">
      <Spin
        spin_dist="1.57"/>
      <Wait
        wait_duration="1.0"/>
      <BackUp
        backup_dist="0.15"/>
      <ClearEntireCostmap
        service_name="local_costmap/clear_entirely"/>
    </Sequence>
  </RecoveryNode>
</RateController>
</PipelineSequence>
</BehaviorTree>
</root>

```

16.3.3 行为树的工作原理

行为树执行流程：

1. 接收到导航目标 {goal}
2. 进入 PipelineSequence
3. 第一阶段：RateController(1Hz)
 - ├─ ComputePathToPose：计算全局路径
 - | ─ 成功 → 进入第二阶段
 - | ─ 失败 → RecoveryNode
 - ├─ ClearEntireCostmap：清除代价地图
 - └─ 重试 ComputePathToPose
 - └─ 继续第一阶段
4. 第二阶段：RateController(10Hz)
 - ├─ FollowPath：执行路径跟踪
 - | ─ 成功 → 导航完成
 - | ─ 失败 → RecoveryNode
 - ├─ Spin：原地旋转
 - ├─ Wait：等待
 - ├─ BackUp：后退
 - ├─ ClearEntireCostmap：清除代价地图
 - └─ 重试 FollowPath
 - └─ 继续第二阶段

16.3.4 自定义行为树

用户可以根据需求创建自定义行为树：

```

<!-- custom_navigation.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="CustomNavigate">
      <!-- 步骤 1: 检查电池电量 -->
      <BatteryCheck
        min_battery_level="0.2"
        battery_topic="/battery_status"/>

      <!-- 步骤 2: 计算路径 -->
      <ComputePathToPose
        goal="{goal}"
        path="{path}"/>

      <!-- 步骤 3: 使用 MPPI 控制器跟踪路径 -->
      <FollowPath
        path="{path}"
        controller_id="MPPI"/>

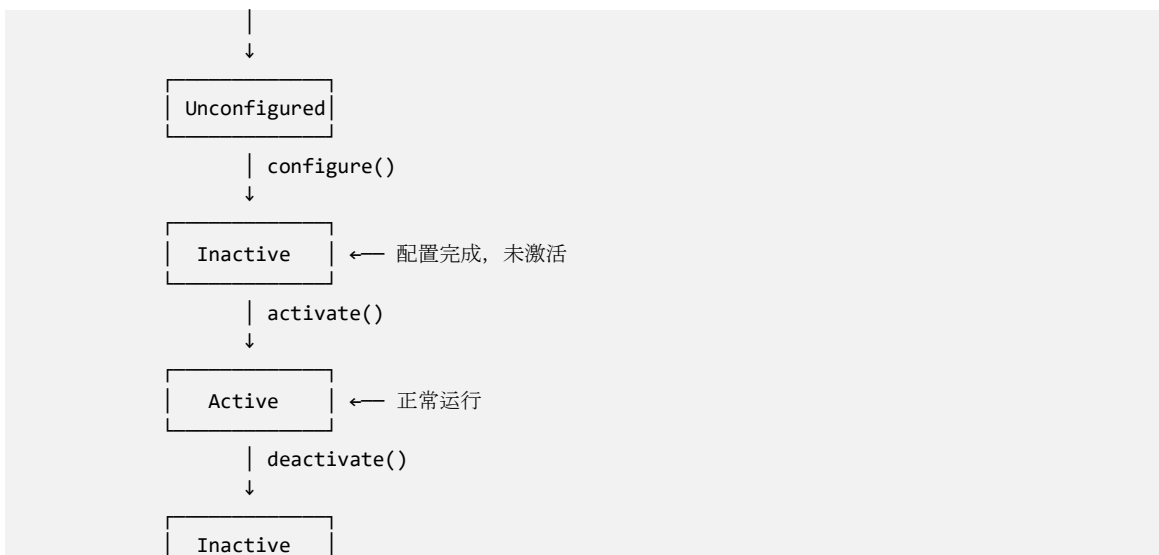
      <!-- 步骤 4: 到达后执行额外动作 -->
      <Sequence name="PostArrival">
        <Wait wait_duration="2.0"/>
        <Spin spin_dist="6.2832"/>
      </Sequence>
    </Sequence>
  </BehaviorTree>
</root>

```

16.4 生命周期节点管理

16.4.1 生命周期节点概念

Nav2 使用 ROS2 的**生命周期节点 (Lifecycle Node)** 管理所有导航组件。生命周期节点具有明确的状态转换过程：





状态说明:

- **Unconfigured:** 初始状态, 节点已创建但未初始化
- **Inactive:** 配置完成, 可以发布/订阅但不处理数据
- **Active:** 正常运行, 执行核心功能
- **Finalized:** 节点已销毁

16.4.2 生命周期管理服务

```
# 查看当前生命周期状态
ros2 service call /bt_navigator/get_state \
  lifecycle_msgs/srv/GetState

# 配置节点 (Unconfigured → Inactive)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 1}}" # 1=TRANSITION_CONFIGURE

# 激活节点 (Inactive → Active)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 3}}" # 3=TRANSITION_ACTIVATE

# 停用节点 (Active → Inactive)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 4}}" # 4=TRANSITION_DEACTIVATE

# 清理节点 (Inactive → Unconfigured)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 2}}" # 2=TRANSITION_CLEANUP
```

16.4.3 生命周期管理器

Nav2 使用 `nav2_lifecycle_manager` 统一管理所有导航组件的生命周期:

```
# nav2_params.yaml
lifecycle_manager:
```

```

lifecycle_manager:
  ros__parameters:
    autostart: true          # 自动启动
    node_names:             # 管理的节点列表
      - bt_navigator
      - planner_server
      - controller_server
      - behavior_server
      - local_costmap
      - global_costmap
      - amcl
      - map_server

# 使用生命周期管理器一键启动
ros2 launch nav2_bringup navigation_launch.py \
  map:=./maps/my_map.yaml \
  use_sim_time:=true

# 生命周期管理器自动执行:
# 1. 创建所有节点 → Unconfigured
# 2. 配置所有节点 → Inactive
# 3. 激活所有节点 → Active

```

16.4.4 自定义生命周期节点

```

import rclpy
from rclpy.lifecycle import LifecycleNode, State, TransitionCallbackReturn

class Nav2Component(LifecycleNode):
    """自定义 Nav2 生命周期组件"""
    def __init__(self):
        super().__init__('custom_nav_component')

    def on_configure(self, state: State) -> TransitionCallbackReturn:
        """配置阶段：分配资源、读取参数"""
        self.get_logger().info('正在配置...')

        # 读取参数
        self.declare_parameter('param1', 'default')
        self.param1 = self.get_parameter('param1').value

        # 创建发布器和订阅器（但不激活）
        self.publisher = self.create_lifecycle_publisher(
            PoseStamped, '/custom_pose', 10)
        self.subscriber = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)

        self.get_logger().info('配置完成')
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state: State) -> TransitionCallbackReturn:
        """激活阶段：开始处理数据"""
        self.get_logger().info('正在激活...')

```

```

# 激活发布者
self.publisher.on_activate()

self.get_logger().info('已激活')
return TransitionCallbackReturn.SUCCESS

def on_deactivate(self, state: State) -> TransitionCallbackReturn:
    """停用阶段：暂停处理"""
    self.get_logger().info('正在停用...')
    self.publisher.on_deactivate()
    return TransitionCallbackReturn.SUCCESS

def on_cleanup(self, state: State) -> TransitionCallbackReturn:
    """清理阶段：释放资源"""
    self.get_logger().info('正在清理...')
    self.publisher = None
    self.subscriber = None
    return TransitionCallbackReturn.SUCCESS

def on_shutdown(self, state: State) -> TransitionCallbackReturn:
    """关闭阶段"""
    self.get_logger().info('正在关闭...')
    return TransitionCallbackReturn.SUCCESS

def scan_callback(self, msg):
    """激光数据回调（仅在 Active 状态处理）"""
    pass

```

16.5 核心组件详解

16.5.1 BT Navigator

BT Navigator 是 Nav2 的行为树执行引擎，负责：

- 加载和执行行为树 XML
- 管理导航任务的生命周期
- 提供 NavigateToPose 和 FollowWaypoints 的 Action 接口

```

import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose
import math

class BTNavigatorClient(Node):
    """BT Navigator Action 客户端"""
    def __init__(self):
        super().__init__('bt_navigator_client')

        self.action_client = ActionClient(
            self, NavigateToPose, 'navigate_to_pose')

```

```

def send_goal(self, x: float, y: float, yaw: float,
              behavior_tree: str = ''):
    """发送导航目标"""
    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.header.stamp = self.get_clock().now().to_msg()
    goal.pose.pose.position.x = x
    goal.pose.pose.position.y = y

    qz = math.sin(yaw / 2.0)
    qw = math.cos(yaw / 2.0)
    goal.pose.pose.orientation.z = qz
    goal.pose.pose.orientation.w = qw

    if behavior_tree:
        goal.behavior_tree = behavior_tree

    self.action_client.wait_for_server()
    self.send_goal_future = self.action_client.send_goal_async(
        goal, feedback_callback=self.feedback_callback)
    self.send_goal_future.add_done_callback(
        self.goal_response_callback)

def goal_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().error('目标被拒绝')
        return

    self.get_logger().info('目标已接受')
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def result_callback(self, future):
    result = future.result().result
    status = future.result().status
    if status == 4: # SUCCEEDED
        self.get_logger().info(f'导航成功!')
    else:
        self.get_logger().error(f'导航失败, status={status}')

def feedback_callback(self, feedback_msg):
    feedback = feedback_msg.feedback
    if feedback.distance_remaining > 0:
        self.get_logger().info(
            f'剩余距离: {feedback.distance_remaining:.2f}m',
            throttle_duration_sec=1.0)

```

16.5.2 Planner Server

全局规划器负责在代价地图上搜索从起点到终点的无碰撞路径:

```

import rclpy
from rclpy.node import Node
from nav_msgs.msg import Path
from geometry_msgs.msg import PoseStamped

```

```

class GlobalPlannerClient(Node):
    """全局规划器客户端"""
    def __init__(self):
        super().__init__('global_planner_client')

        self.path_pub = self.create_publisher(Path, '/plan', 10)
        self.planner_action_client = ActionClient(
            self, ComputePathToPose, '/compute_path_to_pose')

    def request_plan(self, start: PoseStamped, goal: PoseStamped,
                    planner_id: str = 'GridBased'):
        """请求全局路径规划"""
        goal_msg = ComputePathToPose.Goal()
        goal_msg.start = start
        goal_msg.goal = goal
        goal_msg.planner_id = planner_id

        self.planner_action_client.wait_for_server()
        future = self.planner_action_client.send_goal_async(goal_msg)
        future.add_done_callback(self.plan_response_callback)

    def plan_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().error('规划请求被拒绝')
            return
        result_future = goal_handle.get_result_async()
        result_future.add_done_callback(self.plan_result_callback)

    def plan_result_callback(self, future):
        result = future.result().result
        if result.path.poses:
            self.get_logger().info(
                f'路径规划成功: {len(result.path.poses)}个路径点')

```

16.5.3 Controller Server

局部控制器负责跟踪全局路径并避开动态障碍物：

- 支持多种控制器插件（DWB, MPPI, Regulated Pure Pursuit）
- 实时处理激光传感器数据
- 输出 `cmd_vel` 速度指令

16.5.4 Behavior Server

恢复行为服务器管理导航失败时的恢复策略：

- Spin：原地旋转
- BackUp：后退
- Drive on Heading：沿当前方向前进

- Wait: 等待
- Clear Costmap: 清除代价地图

16.5.5 Costmap Layers

代价地图层负责将环境建模为带代价值的栅格：

图层	插件	功能
静态层	StaticLayer	加载预建地图
障碍图层	ObstacleLayer	传感器实时障碍物
膨胀层	InflationLayer	障碍物膨胀
体素层	VoxelLayer	3D 障碍物感知

16.6 Nav2 启动与配置

16.6.1 启动文件

```
# 使用预建地图启动导航
ros2 launch nav2_bringup navigation_launch.py \
  map:=./maps/office_map.yaml \
  params_file:=./config/nav2_params.yaml \
  use_sim_time:=true

# 自动启动（包含定位）
ros2 launch nav2_bringup bringup_launch.py \
  map:=./maps/office_map.yaml \
  use_sim_time:=true

# 仅定位模式
ros2 launch nav2_bringup localization_launch.py \
  map:=./maps/office_map.yaml \
  use_sim_time:=true
```

16.6.2 参数文件结构

```
# nav2_params.yaml 整体结构
bt_navigator:
  ros__parameters:
    ... # 行为树配置

planner_server:
  ros__parameters:
    ... # 规划器配置
```

```

controller_server:
  ros__parameters:
    ... # 控制器配置

local_costmap:
  local_costmap:
    ros__parameters:
      ... # 局部代价地图

global_costmap:
  global_costmap:
    ros__parameters:
      ... # 全局代价地图

amcl:
  ros__parameters:
    ... # AMCL 定位配置

map_server:
  ros__parameters:
    ... # 地图服务配置

behavior_server:
  ros__parameters:
    ... # 恢复行为配置

```

16.6.3 Simple Commander API

Nav2 提供了 Python 的 Simple Commander API，简化导航调用：

```

#!/usr/bin/env python3
import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator
from geometry_msgs.msg import PoseStamped
import math

def main():
    rclpy.init()
    navigator = BasicNavigator()

    # 等待 Nav2 就绪
    navigator.waitUntilNav2Active()

    # 创建目标位姿
    goal = PoseStamped()
    goal.header.frame_id = 'map'
    goal.header.stamp = navigator.get_clock().now().to_msg()
    goal.pose.position.x = 3.0
    goal.pose.position.y = 2.0
    goal.pose.orientation.w = 1.0

    # 发送导航目标
    navigator.goToPose(goal)

    # 监控执行过程

```

```
while not navigator.isTaskComplete():
    feedback = navigator.getFeedback()
    if feedback:
        print(f'剩余距离: {feedback.distance_remaining:.2f}m')

# 检查结果
result = navigator.getResult()
if result == TaskResult.SUCCEEDED:
    print('导航成功!')
else:
    print(f'导航失败: {result}')

rclpy.shutdown()
```

课后练习

236. **架构题:** 绘制 Nav2 的系统架构图, 说明各组件之间的数据流和依赖关系。
237. **配置题:** 编写 Nav2 的 nav2_params.yaml 配置文件, 包括行为树导航器、全局规划器、局部控制器和代价地图的完整配置。
238. **编程题:** 使用 nav2_simple_commander 编写一个多点导航程序, 机器人依次访问 5 个目标点, 每到达一个点后暂停 3 秒钟。
239. **分析题:** 分析 Nav2 使用生命周期节点管理的优势, 说明生命周期状态转换机制如何提高系统的鲁棒性和资源利用效率。
240. **设计题:** 设计一个自定义的导航行为树, 包含以下功能: 电池检查、前往充电桩、充电等待、继续任务。写出对应的 XML 行为树文件。

实验 16:

第 16 章 实验：机械臂 URDF 建模与状态发布

****对应理论章节**:** 第 25 章《ROS2 机械臂建模 (URDF/Xacro)》 ****实验课时**:** 2
课时 ****实验代码**:** `src/lab\_code/ch16\_lab/` (`arm\_urdf` 三自由度臂
URDF/Xacro + `arm\_state\_publisher/` 状态发布者)

实验目标

- 掌握 URDF 语法和 link/joint 配置方法
- 学会创建 3 自由度机械臂的 URDF 模型
- 掌握 Xacro 的宏定义和数学计算功能
- 能够在 RViz2 中可视化自定义机械臂模型
- 学会用关节状态发布者驱动自建模型运动

实验环境

- ROS 2 Jazzy
- URDF/Xacro 工具链
- RViz2 可视化工具
- joint_state_publisher_gui

参考代码说明

src/lab_code/ch16_lab/ 下的两个目录为**纯文件示例 (非 colcon 功能包)**，无需构建，可直接阅读源码学习，也可将文件复制到自己的功能包中运行：

```
src/lab_code/ch16_lab/
├── arm_urdf/                # 三自由度机械臂模型
│   ├── urdf/three_dof_arm.urdf  # URDF 版模型
│   ├── urdf/three_dof_arm.xacro  # Xacro 宏定义版模型
│   └── launch/display.launch.py  # RViz 显示 launch 文件
└── arm_state_publisher/
    └── state_publisher.py        # 向 /joint_states 发布关节角度
```

- `three_dof_arm.xacro` 定义了底座 + 3 个旋转关节 (`joint1` 绕 Z 轴, `joint2/joint3` 绕 Y 轴) 与两个夹爪指关节 (`finger1_joint/finger2_joint`) , 使用 `xacro:property` 常量 (`M_PI`、阻尼、摩擦) 与惯性矩阵宏 (`cylinder_inertial`、`box_inertial`) , 与本实验 16.5 节练习的写法一致。
- `state_publisher.py` 以正弦/往复规律发布 `joint1~joint3` 与 `finger1_joint`、`finger2_joint` 的关节角 (夹爪周期性开合) , 配合 `robot_state_publisher` 可在 RViz2 中观察模型运动, 关节命名与 `three_dof_arm` 模型一致。

参考运行方式:

```
# 终端 1: 加载 Xacro 并启动 robot_state_publisher
ros2 run robot_state_publisher robot_state_publisher --ros-args \
  -p robot_description:="$(xacro src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.xacro)"

# 终端 2: 运行状态发布者
python3 src/lab_code/ch16_lab/arm_state_publisher/state_publisher.py

# 终端 3: 启动 RViz2, Fixed Frame 设为 base_link, 添加 RobotModel 显示
rviz2
```

实验步骤

16.1 创建功能包

```
cd ~/ros2_arm_ws/src
ros2 pkg create my_arm_description --build-type ament_python --dependencies rclpy
cd my_arm_description
mkdir urdf launch rviz
```

16.2 创建 3 自由度机械臂 URDF

创建 `urdf/simple_arm.urdf` (可对照参考实现

`src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.urdf`) :

```
<?xml version="1.0"?>
<robot name="simple_arm">
  <!-- 底座 -->
  <link name="base_link">
    <visual>
      <origin rpy="0 0 0" xyz="0 0 0.05"/>
      <geometry><cylinder length="0.1" radius="0.08"/></geometry>
      <material name="gray">
        <color rgba="0.5 0.5 0.5 1"/>
      </material>
    </visual>
    <collision>
```

```

        <origin rpy="0 0 0" xyz="0 0 0.05"/>
        <geometry><cylinder length="0.1" radius="0.08"/></geometry>
    </collision>
    <inertial>
        <mass value="0.5"/>
        <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节1: 绕Z轴旋转 (腰部) -->
<joint name="joint1" type="revolute">
    <parent link="base_link"/>
    <child link="link1"/>
    <origin rpy="0 0 0" xyz="0 0 0.1"/>
    <axis xyz="0 0 1"/>
    <limit effort="100" velocity="1.0" lower="-3.14" upper="3.14"/>
</joint>

<!-- 连杆1 -->
<link name="link1">
    <visual>
        <geometry><box size="0.08 0.06 0.25"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
        <material name="blue">
            <color rgba="0 0 1 1"/>
        </material>
    </visual>
    <collision>
        <geometry><box size="0.08 0.06 0.25"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
    </collision>
    <inertial>
        <mass value="0.3"/>
        <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节2: 绕Y轴旋转 (肩部) -->
<joint name="joint2" type="revolute">
    <parent link="link1"/>
    <child link="link2"/>
    <origin rpy="0 0 0" xyz="0 0 0.25"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆2 -->
<link name="link2">
    <visual>
        <geometry><cylinder length="0.3" radius="0.025"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.15"/>
        <material name="red">
            <color rgba="1 0 0 1"/>
        </material>
    </visual>
    <collision>
        <geometry><cylinder length="0.3" radius="0.025"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.15"/>
    </collision>

```

```

        </collision>
        <inertial>
            <mass value="0.2"/>
            <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
        </inertial>
    </link>

    <!-- 关节3: 绕Y轴旋转 (肘部) -->
    <joint name="joint3" type="revolute">
        <parent link="link2"/>
        <child link="link3"/>
        <origin rpy="0 0 0" xyz="0 0 0.3"/>
        <axis xyz="0 1 0"/>
        <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
    </joint>

    <!-- 连杆3 (末端) -->
    <link name="link3">
        <visual>
            <geometry><box size="0.03 0.03 0.15"/></geometry>
            <origin rpy="0 0 0" xyz="0 0 0.075"/>
            <material name="green">
                <color rgba="0 1 0 1"/>
            </material>
        </visual>
        <collision>
            <geometry><box size="0.03 0.03 0.15"/></geometry>
            <origin rpy="0 0 0" xyz="0 0 0.075"/>
        </collision>
        <inertial>
            <mass value="0.1"/>
            <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
        </inertial>
    </link>
</robot>

```

16.3 检查 URDF 语法

```

sudo apt install liburdfdom-tools -y
check_urdf urdf/simple_arm.urdf

# 生成模型结构图
urdf_to_graphviz urdf/simple_arm.urdf
# 查看生成的 simple_arm.pdf

```

16.4 创建 launch 文件显示模型

创建 launch/display_urdf.launch.py:

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_path = get_package_share_directory('my_arm_description')

```

```

urdf_path = os.path.join(pkg_path, 'urdf', 'simple_arm.urdf')

with open(urdf_path, 'r') as f:
    robot_desc = f.read()

return LaunchDescription([
    Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        parameters=[{'robot_description': robot_desc}],
    ),
    Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        name='joint_state_publisher_gui',
    ),
    Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
    ),
])

```

在 RViz2 中查看模型:

```

cd ~/ros2_arm_ws
colcon build --packages-select my_arm_description
source install/setup.bash
ros2 launch my_arm_description display_urdf.launch.py

```

在 RViz2 中:

- 设置 Fixed Frame 为 base_link
- 添加 RobotModel 显示插件
- 添加 TF 显示插件
- 拖动 joint1/joint2/joint3 滑块观察运动

16.5 创建 Xacro 版本模型

创建 urdf/materials.xacro:

```

<?xml version="1.0"?>
<robot name="materials" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <material name="gray"><color rgba="0.5 0.5 0.5 1"/></material>
  <material name="blue"><color rgba="0 0 1 1"/></material>
  <material name="red"><color rgba="1 0 0 1"/></material>
  <material name="green"><color rgba="0 1 0 1"/></material>
  <material name="orange"><color rgba="1 0.5 0 1"/></material>
</robot>

```

创建 urdf/macros.xacro:

```

<?xml version="1.0"?>
<robot name="macros" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:macro name="inertia_box" params="m w h d">
    <inertial>
      <mass value="{m}" />
      <inertia ixx="{m*(h*h + d*d)/12}" ixy="0" ixz="0"
        iyy="{m*(w*w + d*d)/12}" iyz="0"
        izz="{m*(w*w + h*h)/12}" />
    </inertial>
  </xacro:macro>

  <xacro:macro name="inertia_cylinder" params="m r l">
    <inertial>
      <mass value="{m}" />
      <inertia ixx="{m*(3*r*r + l*l)/12}" ixy="0" ixz="0"
        iyy="{m*(3*r*r + l*l)/12}" iyz="0"
        izz="{m*r*r/2}" />
    </inertial>
  </xacro:macro>
</robot>

```

创建 urdf/simple_arm.xacro (可对照参考实现

src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.xacro, 后者在相同结构上增加了夹爪指关节) :

```

<?xml version="1.0"?>
<robot name="simple_arm" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:property name="PI" value="3.1415926" />
  <xacro:property name="damping" value="50.0" />
  <xacro:property name="friction" value="1.0" />

  <xacro:include filename="$(find my_arm_description)/urdf/materials.xacro" />
  <xacro:include filename="$(find my_arm_description)/urdf/macros.xacro" />

  <link name="base_link">
    <visual>
      <origin rpy="0 0 0" xyz="0 0 0.05" />
      <geometry><cylinder length="0.1" radius="0.08" /></geometry>
      <material name="gray" />
    </visual>
    <collision>
      <origin rpy="0 0 0" xyz="0 0 0.05" />
      <geometry><cylinder length="0.1" radius="0.08" /></geometry>
    </collision>
    <inertia_cylinder m="0.5" r="0.08" l="0.1" />
  </link>

  <joint name="joint1" type="revolute">
    <parent link="base_link" /><child link="link1" />
    <origin rpy="0 0 0" xyz="0 0 0.1" />
    <axis xyz="0 0 1" />
    <limit effort="100" velocity="1.0" lower="{-PI}" upper="{PI}" />
    <dynamics damping="{damping}" friction="{friction}" />
  </joint>

  <link name="link1">
    <visual>

```

```

    <geometry><box size="0.08 0.06 0.25"/></geometry>
    <origin rpy="0 0 0" xyz="0 0 0.125"/>
    <material name="blue"/>
  </visual>
  <collision>
    <geometry><box size="0.08 0.06 0.25"/></geometry>
    <origin rpy="0 0 0" xyz="0 0 0.125"/>
  </collision>
  <inertia_box m="0.3" w="0.08" h="0.06" d="0.25"/>
</link>

<joint name="joint2" type="revolute">
  <parent link="link1"/><child link="link2"/>
  <origin rpy="0 0 0" xyz="0 0 0.25"/>
  <axis xyz="0 1 0"/>
  <limit effort="100" velocity="1.0" lower="-PI*2/3" upper="PI*2/3"/>
  <dynamics damping="${damping}" friction="${friction}"/>
</joint>

<link name="link2">
  <visual>
    <geometry><cylinder length="0.3" radius="0.025"/></geometry>
    <origin rpy="0 0 0" xyz="0 0 0.15"/>
    <material name="red"/>
  </visual>
  <collision>
    <geometry><cylinder length="0.3" radius="0.025"/></geometry>
    <origin rpy="0 0 0" xyz="0 0 0.15"/>
  </collision>
  <inertia_cylinder m="0.2" r="0.025" l="0.3"/>
</link>

<joint name="joint3" type="revolute">
  <parent link="link2"/><child link="link3"/>
  <origin rpy="0 0 0" xyz="0 0 0.3"/>
  <axis xyz="0 1 0"/>
  <limit effort="100" velocity="1.0" lower="-PI*2/3" upper="PI*2/3"/>
  <dynamics damping="${damping}" friction="${friction}"/>
</joint>

<link name="link3">
  <visual>
    <geometry><box size="0.03 0.03 0.15"/></geometry>
    <origin rpy="0 0 0" xyz="0 0 0.075"/>
    <material name="green"/>
  </visual>
  <collision>
    <geometry><box size="0.03 0.03 0.15"/></geometry>
    <origin rpy="0 0 0" xyz="0 0 0.075"/>
  </collision>
  <inertia_box m="0.1" w="0.03" h="0.03" d="0.15"/>
</link>
</robot>

```

16.6 测试 Xacro 转换

```

cd ~/ros2_arm_ws
source install/setup.bash

```

```
# 转换 Xacro 为 URDF
xacro src/my_arm_description/urdf/simple_arm.xacro > /tmp/test_arm.urdf
check_urdf /tmp/test_arm.urdf

# 验证转换后的 URDF
diff <(check_urdf /tmp/test_arm.urdf 2>&1) <(echo "")
```

16.7 添加夹爪

扩展模型，在 link3 末端添加夹爪。创建 urdf/simple_arm_gripper.xacro:

```
<?xml version="1.0"?>
<robot name="simple_arm_gripper" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:include filename="$(find my_arm_description)/urdf/simple_arm.xacro"/>

  <joint name="gripper_joint" type="fixed">
    <parent link="link3"/><child link="gripper_base"/>
    <origin rpy="0 0 0" xyz="0 0 0.15"/>
  </joint>

  <link name="gripper_base">
    <visual>
      <geometry><box size="0.06 0.04 0.02"/></geometry>
      <material name="orange"/>
    </visual>
  </link>

  <joint name="finger1_joint" type="prismatic">
    <parent link="gripper_base"/><child link="finger1"/>
    <origin rpy="0 0 0" xyz="0 -0.02 0"/>
    <axis xyz="0 1 0"/>
    <limit effort="10" velocity="0.5" lower="0" upper="0.03"/>
  </joint>

  <link name="finger1">
    <visual>
      <geometry><box size="0.02 0.04 0.05"/></geometry>
      <origin rpy="0 0 0" xyz="0 0.04 0.025"/>
      <material name="gray"/>
    </visual>
  </link>

  <joint name="finger2_joint" type="prismatic">
    <parent link="gripper_base"/><child link="finger2"/>
    <origin rpy="0 0 0" xyz="0 0.02 0"/>
    <axis xyz="0 -1 0"/>
    <limit effort="10" velocity="0.5" lower="0" upper="0.03"/>
  </joint>

  <link name="finger2">
    <visual>
      <geometry><box size="0.02 0.04 0.05"/></geometry>
      <origin rpy="0 0 0" xyz="0 0.04 0.025"/>
      <material name="gray"/>
    </visual>
  </link>
</robot>
```


创建 `launch/display_xacro.launch.py`:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_path = get_package_share_directory('my_arm_description')
    xacro_path = os.path.join(pkg_path, 'urdf', 'simple_arm_gripper.xacro')

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            parameters=[{'robot_description': Command(['xacro ', xacro_path])}],
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
            name='joint_state_publisher_gui',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
        ),
    ])
])
```

编译并查看完整模型:

```
cd ~/ros2_arm_ws
colcon build --packages-select my_arm_description
source install/setup.bash
ros2 launch my_arm_description display_xacro.launch.py
```

在 RViz2 中拖动所有滑块, 包括 `finger1_joint` 和 `finger2_joint`, 观察夹爪开合。

也可以运行参考代码的状态发布者驱动该模型 (关节命名已对齐 `three_dof_arm`) :

```
python3 src/lab_code/ch16_lab/arm_state_publisher/state_publisher.py
```

16.8 保存 RViz2 配置

在 RViz2 中完成显示配置后:

- 调整好视角 (按 `Ctrl+0` 重置视角)
- 按 `Ctrl+S` 保存配置到 `my_arm_description/rviz/display.rviz`
- 下次启动时可以在 `launch` 文件中指定该配置

16.9 模型参数修改练习

修改以下参数，观察效果变化：

```
# 修改连杆长度
# 在 simple_arm.xacro 中找到 link1 的 box size, 将 z 方向的 0.25 改为 0.35

# 修改关节限位
# 将 joint2 的 lower 从 -2.09 改为 -1.57, upper 从 2.09 改为 1.57

# 添加新的连杆
# 在 link3 后添加 joint4 和 link4, 增加一个自由度
```

实验结果与分析

- 通过 URDF 成功创建了 3 自由度机械臂模型，在 RViz2 中完整显示
- Xacro 通过宏定义和数学表达式简化了模型描述，减少了重复代码
- check_urdf 工具确保模型语法正确，urdf_to_graphviz 生成了直观的结构图
- 夹爪使用 prismatic 关节实现了平移开合运动
- 参考代码 arm_state_publisher/state_publisher.py 驱动 three_dof_arm 模型实现了关节正弦运动与夹爪周期开合，验证了“模型+状态发布者”组合的正确性

思考题

241. URDF 中 revolute 和 prismatic 关节有什么区别？各适合什么应用场景？
242. Xacro 的宏定义有哪些优势？在复杂机器人建模中如何组织模块化 Xacro 文件？
243. inertial 参数对仿真有什么影响？如果全部设为 0 会怎样？
244. 如何为机械臂添加更多的自由度？需要注意哪些约束？

第 17 章 全局代价地图

学习目标

- 理解代价地图(Costmap2D)的原理和工作机制
- 掌握静态层、障碍物层、膨胀层的配置方法
- 熟悉代价地图的分层架构和插件机制
- 能够根据应用场景调优代价地图参数
- 掌握全局与局部代价地图的区别和配合

17.1 代价地图基本原理

17.1.1 什么是代价地图

代价地图 (Costmap) 是导航系统中将物理空间建模为带代价值栅格的核心数据结构。每个栅格 (cell) 包含一个代价值，表示该位置是否适合机器人通过。

代价值的含义：

值范围	标签	含义
0	Free (空闲)	无障碍物，可自由通行
1-127	Inscribed (刻划)	靠近障碍物的区域，代价值随距离增加
128-253	Obstacle (障碍物)	障碍物膨胀区域，不宜通行
254	Lethal (致命)	障碍物占据，不可通行
255	Unknown (未知)	未探索区域

代价地图的作用：

- 为路径规划提供环境模型
- 编码障碍物信息和安全距离
- 支持动态障碍物实时更新

- 分层架构支持多种信息来源

17.1.2 Costmap2D 数据结构

```
import numpy as np
from nav_msgs.msg import OccupancyGrid

class Costmap2D:
    """代价地图数据结构"""
    def __init__(self, width: float, height: float,
                 resolution: float = 0.05):
        self.resolution = resolution
        self.width = int(width / resolution)
        self.height = int(height / resolution)
        self.origin_x = -width / 2.0
        self.origin_y = -height / 2.0

        # 主代价数组 (0-255)
        self.costmap = np.zeros((self.height, self.width), dtype=np.uint8)

        # 未知区域初始化为 255
        self.costmap[:] = 255

    def world_to_grid(self, wx: float, wy: float) -> tuple:
        """世界坐标转栅格坐标"""
        gx = int((wx - self.origin_x) / self.resolution)
        gy = int((wy - self.origin_y) / self.resolution)
        return gx, gy

    def grid_to_world(self, gx: int, gy: int) -> tuple:
        """栅格坐标转世界坐标"""
        wx = gx * self.resolution + self.origin_x
        wy = gy * self.resolution + self.origin_y
        return wx, wy

    def set_cost(self, wx: float, wy: float, cost: int):
        """设置某点的代价值"""
        gx, gy = self.world_to_grid(wx, wy)
        if 0 <= gx < self.width and 0 <= gy < self.height:
            self.costmap[gy, gx] = np.clip(cost, 0, 255)

    def get_cost(self, wx: float, wy: float) -> int:
        """获取某点的代价值"""
        gx, gy = self.world_to_grid(wx, wy)
        if 0 <= gx < self.width and 0 <= gy < self.height:
            return int(self.costmap[gy, gx])
        return 255

    def to_occupancy_grid(self) -> OccupancyGrid:
        """转换为 ROS OccupancyGrid 消息"""
        msg = OccupancyGrid()
        msg.header.frame_id = 'map'
        msg.info.resolution = self.resolution
        msg.info.width = self.width
        msg.info.height = self.height
```

```

msg.info.origin.position.x = self.origin_x
msg.info.origin.position.y = self.origin_y

# 转换代价为-1,0,100 格式
occ = np.zeros_like(self.costmap, dtype=np.int8)
occ[self.costmap == 0] = 0      # 空闲
occ[self.costmap == 255] = -1   # 未知
occ[self.costmap >= 254] = 100  # 障碍物
occ[(0 < self.costmap) & (self.costmap < 254)] = 0

msg.data = occ.flatten().tolist()
return msg

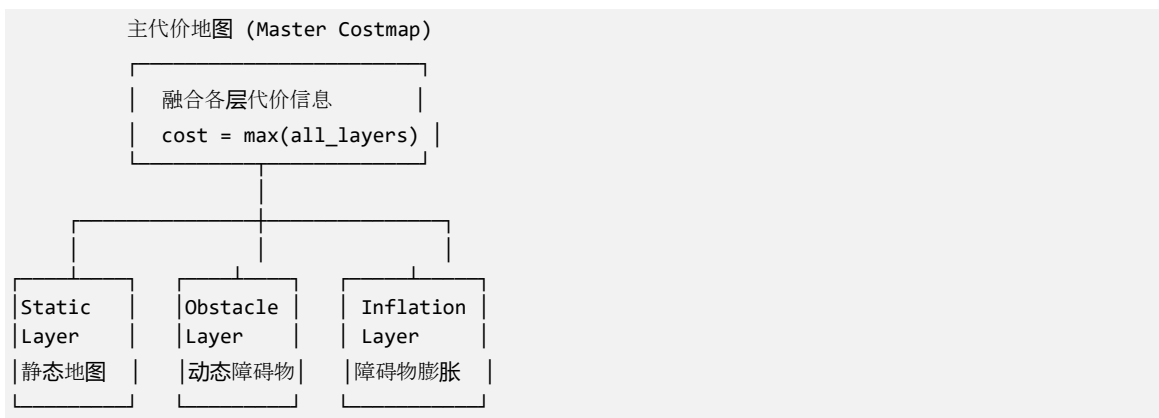
def reset(self):
    """重置代价地图"""
    self.costmap[:] = 255

```

17.2 分层代价地图架构

17.2.1 分层设计思想

Nav2 将代价地图设计为分层架构，每一层负责维护不同的代价信息，最终融合为一张主代价地图：



各层功能：

图层	插件名称	功能	更新方式
静态层	StaticLayer	加载预建栅格地图	静态加载
障碍物层	ObstacleLayer	传感器实时障碍物检测	实时更新
膨胀层	InflationLayer	障碍物膨胀/安全距	计算生成

		离	
体素层	VoxelLayer	3D 体素障碍物感知	实时更新
范围层	RangeLayer	距离传感器更新	实时更新
禁飞层	KeepoutLayer	禁止进入区域	配置加载

17.2.2 全局代价地图 vs 局部代价地图

特性	全局代价地图 (Global Costmap)	局部代价地图 (Local Costmap)
坐标系	map	odom
范围	整个已知地图	机器人周围滚动窗口
更新频率	低 (1-2 Hz)	高 (5-10 Hz)
分辨率	较高 (0.05m)	较高 (0.05m)
用途	全局路径规划	局部路径规划和避障
图层配置	static+obstacle+inflation	obstacle+inflation (无 static)

17.2.3 代价地图 YAML 配置

```
# global_costmap 配置
global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0          # 更新频率 (Hz)
      publish_frequency: 1.0         # 发布频率 (Hz)
      global_frame: map               # 全局坐标系
      robot_base_frame: base_link     # 机器人基座坐标系
      robot_radius: 0.22              # 机器人半径 (用于足迹)
      resolution: 0.05               # 地图分辨率 (m/像素)
      track_unknown_space: true        # 跟踪未知区域
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: true # 使用 transient_local QoS
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: true
```

```

        observation_sources: "scan"
        scan:
            topic: /scan
            max_obstacle_height: 2.0
            clearing: true
            marking: true
            data_type: "LaserScan"
        inflation_layer:
            plugin: "nav2_costmap_2d::InflationLayer"
            cost_scaling_factor: 3.0      # 代价衰减因子
            inflation_radius: 0.55      # 膨胀半径 (m)

# local_costmap 配置
local_costmap:
    local_costmap:
        ros__parameters:
            update_frequency: 5.0
            publish_frequency: 2.0

            global_frame: odom          # 使用里程计坐标系
            robot_base_frame: base_link
            robot_radius: 0.22
            resolution: 0.05

            rolling_window: true        # 滚动窗口模式
            width: 4                    # 窗口宽度 (m)
            height: 4                   # 窗口高度 (m)
            track_unknown_space: true
            plugins: ["voxel_layer", "inflation_layer"]
            voxel_layer:
                plugin: "nav2_costmap_2d::VoxelLayer"
                enabled: true
                publish_voxel_map: true
                origin_z: 0.0
                z_resolution: 0.05
                z_voxels: 16
                max_obstacle_height: 2.0
                observation_sources: "scan"
            scan:
                topic: /scan
                max_obstacle_height: 2.0
                clearing: true
                marking: true
                data_type: "LaserScan"
            inflation_layer:
                plugin: "nav2_costmap_2d::InflationLayer"
                cost_scaling_factor: 3.0
                inflation_radius: 0.55

```

17.3 静态层配置

17.3.1 静态层作用

静态层 (StaticLayer) 负责加载预先构建的栅格地图，将其代价信息传递给主代价地图。

静态层的输入：

- map 话题上的 OccupancyGrid 消息
- 或通过 map_server 加载的 YAML/PGM 地图文件

静态层的工作流程：

1. 订阅 /map 话题
2. 将地图数据转换为 costmap 格式
3. 标记已知区域为对应代价
4. 标记未知区域为 255
5. 持续监听地图更新

17.3.2 地图文件格式

```
# office_map.yaml
image: office_map.pgm          # 地图图像文件
resolution: 0.050000           # 地图分辨率 (m/像素)
origin: [-10.0, -10.0, 0.0]    # 地图原点 (x, y, yaw)
negate: 0                      # 是否反转像素值
occupied_thresh: 0.65          # 占据阈值 (>0.65 视为障碍物)
free_thresh: 0.196             # 空闲阈值 (<0.196 视为空闲)
```

17.3.3 地图加载机制

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
from nav2_msgs.srv import LoadMap
import yaml

class MapLoader(Node):
    """地图加载管理器"""
    def __init__(self):
        super().__init__('map_loader')

        self.map_pub = self.create_publisher(
            OccupancyGrid, '/map', 10,
            qos_profile=rclpy.qos.QoSProfile(
                depth=1,
                durability=rclpy.qos.DurabilityPolicy.TRANSIENT_LOCAL
            ))

        self.load_map_service = self.create_service(
            LoadMap, '/map_server/load_map', self.load_map_callback)

    def load_map_from_yaml(self, yaml_path: str) -> OccupancyGrid:
        """从 YAML 文件加载地图"""
        with open(yaml_path, 'r') as f:
            meta = yaml.safe_load(f)
```



```

# 读取 PGM 图像
import cv2
image = cv2.imread(meta['image'], cv2.IMREAD_GRAYSCALE)

height, width = image.shape
resolution = meta['resolution']
origin = meta['origin']

# 转换 PGM 到 OccupancyGrid
msg = OccupancyGrid()
msg.header.frame_id = 'map'
msg.info.resolution = resolution
msg.info.width = width
msg.info.height = height
msg.info.origin.position.x = origin[0]
msg.info.origin.position.y = origin[1]

# 像素值转换 (0=白=空闲, 255=黑=障碍物)
occupied_thresh = meta.get('occupied_thresh', 0.65)
free_thresh = meta.get('free_thresh', 0.196)
negate = meta.get('negate', 0)

data = []
for row in image:
    for pixel in row:
        if negate:
            pixel = 255 - pixel

        normalized = pixel / 255.0
        if normalized > occupied_thresh:
            data.append(100) # 占据
        elif normalized < free_thresh:
            data.append(0) # 空闲
        else:
            data.append(-1) # 未知

msg.data = data
return msg

def load_map_callback(self, request, response):
    """服务回调：加载地图"""
    try:
        occupancy_grid = self.load_map_from_yaml(request.map_url)
        self.map_pub.publish(occupancy_grid)
        self.get_logger().info(f'地图已加载: {request.map_url}')
        response.result = LoadMap.Response().RESULT_SUCCESS
    except Exception as e:
        self.get_logger().error(f'地图加载失败: {e}')
        response.result = LoadMap.Response().RESULT_UNDEFINED_FAILURE
    return response

```

17.4 障碍物层配置

17.4.1 障碍物层作用

障碍物层 (ObstacleLayer) 使用传感器数据实时检测动态障碍物并更新代价地图。

障碍物层的工作流程：

1. 订阅传感器话题 (LaserScan/PointCloud2)
2. 将传感器数据转换到地图坐标系
3. 标记传感器点到代价地图 (marking)
4. 清除传感器路径上的代价 (clearing)
5. 障碍物标记为 254 (lethal)

17.4.2 多传感器源配置

```
# 多传感器障碍物层配置
obstacle_layer:
  plugin: "nav2_costmap_2d::ObstacleLayer"
  enabled: true
  observation_sources: "scan front_sonar rear_sonar"

  scan:
    topic: /scan
    sensor_frame: laser
    max_obstacle_height: 2.0
    min_obstacle_height: 0.0
    clearing: true
    marking: true
    data_type: "LaserScan"
    raytrace_max_range: 3.0
    raytrace_min_range: 0.1

  front_sonar:
    topic: /sonar/front
    sensor_frame: front_sonar_link
    clearing: true
    marking: true
    data_type: "Range"
    min_range: 0.1
    max_range: 2.0

  rear_sonar:
    topic: /sonar/rear
    sensor_frame: rear_sonar_link
    clearing: true
    marking: true
    data_type: "Range"
    min_range: 0.1
    max_range: 1.5
```

17.4.3 传感器数据处理

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan, PointCloud2
import numpy as np
import sensor_msgs_py.point_cloud2 as pc2

class ObstacleLayerProcessor(Node):
    """障碍物层数据处理节点"""
    def __init__(self):
        super().__init__('obstacle_layer_processor')

        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cloud_sub = self.create_subscription(
            PointCloud2, '/point_cloud', self.cloud_callback, 10)

        self.obstacle_map = np.zeros((200, 200), dtype=np.uint8)
        self.resolution = 0.05

    def scan_callback(self, msg: LaserScan):
        """处理激光扫描数据"""
        ranges = np.array(msg.ranges)
        angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))

        # 过滤无效点
        valid = (ranges > msg.range_min) & (ranges < msg.range_max)
        valid_ranges = ranges[valid]
        valid_angles = angles[valid]

        # 转换为笛卡尔坐标
        x = valid_ranges * np.cos(valid_angles)
        y = valid_ranges * np.sin(valid_angles)
        points = np.column_stack((x, y))

        # 标记障碍物 (marking)
        for point in points:
            gx = int(point[0] / self.resolution)
            gy = int(point[1] / self.resolution)
            if 0 <= gx < 200 and 0 <= gy < 200:
                self.obstacle_map[gy, gx] = 254 # lethal

        # 清除传感器路径 (clearing)
        for angle, distance in zip(valid_angles, valid_ranges):
            for d in np.arange(0.1, distance, 0.1):
                px = d * np.cos(angle)
                py = d * np.sin(angle)
                gx = int(px / self.resolution)
                gy = int(py / self.resolution)
                if 0 <= gx < 200 and 0 <= gy < 200:
                    # 如果栅格在路径上非障碍物, 设为 0
                    if self.obstacle_map[gy, gx] != 254:
                        self.obstacle_map[gy, gx] = 0

    def cloud_callback(self, msg: PointCloud2):
```

```

"""处理点云数据（3D 障碍物投影到 2D）"""
points = pc2.read_points(msg, field_names=("x", "y", "z"),
                          skip_nans=True)

for point in points:
    x, y, z = point
    # 只考虑一定高度范围内的障碍物
    if -0.1 < z < 2.0:
        # 投影到 2D 代价地图
        gx = int(x / self.resolution)
        gy = int(y / self.resolution)
        if 0 <= gx < 200 and 0 <= gy < 200:
            self.obstacle_map[gy, gx] = 254

```

17.5 膨胀层配置

17.5.1 膨胀层作用

膨胀层 (InflationLayer) 在障碍物周围生成代价值梯度，使规划器能够生成远离障碍物的安全路径。

膨胀的数学公式：

```

if distance < inscribed_radius:
    cost = 254 (lethal)
elif distance < inflation_radius:
    cost = exp(-scaling_factor * (distance - inscribed_radius)) * 253
else:
    cost = 0

```

17.5.2 膨胀参数详解

```

inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  inflation_radius: 0.55          # 膨胀半径 (m)
  cost_scaling_factor: 3.0        # 代价衰减因子

```

参数说明：

`inflation_radius`: 膨胀半径，决定障碍物的影响范围

- 大半径 → 路径远离障碍物（安全但可能绕路）
- 小半径 → 路径贴近障碍物（高效但有碰撞风险）

`cost_scaling_factor`: 代价衰减速度

- 高因子 → 衰减快，只有近距离有高代价

- 低因子 → 衰减慢，影响范围大

```
import numpy as np

class InflationLayer:
    """膨胀层实现"""
    def __init__(self, inflation_radius: float = 0.55,
                  cost_scaling_factor: float = 3.0,
                  resolution: float = 0.05):
        self.inflation_radius = inflation_radius
        self.cost_scaling_factor = cost_scaling_factor
        self.resolution = resolution

        # 预计算膨胀核
        self.inflation_kernel = self._compute_inflation_kernel()

    def _compute_inflation_kernel(self) -> np.ndarray:
        """预计算膨胀核（用于快速膨胀）"""
        radius_cells = int(np.ceil(self.inflation_radius / self.resolution))
        size = 2 * radius_cells + 1
        kernel = np.zeros((size, size), dtype=np.float32)

        # 刻划半径（等于机器人半径）
        inscribed_radius = 0.22 # 机器人半径

        center = radius_cells
        for dy in range(-radius_cells, radius_cells + 1):
            for dx in range(-radius_cells, radius_cells + 1):
                distance = np.sqrt(dx**2 + dy**2) * self.resolution

                if distance < inscribed_radius:
                    kernel[center + dy, center + dx] = 254 # lethal
                elif distance < self.inflation_radius:
                    # 指数衰减
                    kernel[center + dy, center + dx] = (
                        np.exp(-self.cost_scaling_factor *
                              (distance - inscribed_radius)) * 253
                    )
                else:
                    kernel[center + dy, center + dx] = 0

        return kernel

    def inflate(self, costmap: np.ndarray) -> np.ndarray:
        """对代价地图执行膨胀"""
        inflated = costmap.copy()

        # 找到所有障碍物栅格
        obstacle_cells = np.where(costmap >= 254)

        kh, kw = self.inflation_kernel.shape
        kh2, kw2 = kh // 2, kw // 2

        for y, x in zip(*obstacle_cells):
            # 确定 kernel 覆盖范围
            y_start = max(0, y - kh2)
```

```

        y_end = min(costmap.shape[0], y + kh2 + 1)
        x_start = max(0, x - kw2)
        x_end = min(costmap.shape[1], x + kw2 + 1)

        ky_start = kh2 - (y - y_start)
        ky_end = ky_start + (y_end - y_start)
        kx_start = kw2 - (x - x_start)
        kx_end = kx_start + (x_end - x_start)

        # 膨胀操作：取各层最大值
        region = inflated[y_start:y_end, x_start:x_end]
        kernel_region = self.inflation_kernel[
            ky_start:ky_end, kx_start:kx_end
        ]

        mask = kernel_region > region
        region[mask] = kernel_region[mask]

    return inflated

def compute_cost(self, distance: float) -> float:
    """计算给定距离的代价值"""
    inscribed_radius = 0.22

    if distance < inscribed_radius:
        return 254.0
    elif distance < self.inflation_radius:
        cost = (np.exp(-self.cost_scaling_factor *
                      (distance - inscribed_radius)) * 253)
        return cost
    else:
        return 0.0

```

17.5.3 膨胀效果调优

膨胀参数对路径规划的影响：

```

def visualize_inflation_effects():
    """展示不同膨胀参数的视觉效果"""
    import matplotlib.pyplot as plt

    inflation = InflationLayer(0.55, 3.0)
    distances = np.linspace(0, 1.0, 100)

    # 不同 scaling_factor 的对比
    factors = [1.0, 3.0, 5.0, 10.0]

    plt.figure(figsize=(10, 6))
    for factor in factors:
        costs = [
            InflationLayer(0.55, factor).compute_cost(d)
            for d in distances
        ]
        plt.plot(distances, costs, label=f'scaling={factor}')

    plt.xlabel('距离障碍物距离 (m)')

```

```
plt.ylabel('代价值')
plt.title('膨胀代价曲线')
plt.legend()
plt.grid(True)
plt.show()
```

17.6 代价地图参数调优

17.6.1 分辨率选择

```
# 不同场景的推荐分辨率
# 室内小场景 (50-500m²)
resolution: 0.05 # 精细

# 中型环境 (500-2000m²)
resolution: 0.05 # 平衡精度和计算量

# 大型环境 (>2000m²)
resolution: 0.1 # 降低计算负担
```

17.6.2 更新频率调优

```
# 全局代价地图 (低频)
global_costmap:
  update_frequency: 1.0 # 1Hz 更新
  publish_frequency: 1.0 # 1Hz 发布

# 局部代价地图 (高频)
local_costmap:
  update_frequency: 5.0 # 5Hz 更新
  publish_frequency: 2.0 # 2Hz 发布
```

17.6.3 膨胀半径选择

```
def recommend_inflation_radius(robot_radius: float,
                               environment_type: str) -> float:
    """
    根据机器人尺寸和环境推荐膨胀半径
    """
    base_radius = robot_radius

    if environment_type == 'narrow_corridor':
        # 狭窄走廊：尽量靠近障碍物
        multiplier = 1.5
    elif environment_type == 'open_space':
        # 开阔空间：保持安全距离
        multiplier = 3.0
    elif environment_type == 'warehouse':
        # 仓库：标准设置
```

```

        multiplier = 2.0
    elif environment_type == 'office':
        # 办公室：兼顾安全和通过性
        multiplier = 2.5
    else:
        multiplier = 2.0

    return base_radius * multiplier

```

17.6.4 滚动窗口配置

局部代价地图的滚动窗口大小影响导航性能：

```

# 差速机器人
local_costmap:
    width: 3.0    # 窗口宽度
    height: 3.0   # 窗口高度

# 阿克曼转向机器人（需要更大的前瞻距离）
local_costmap:
    width: 5.0
    height: 5.0

# 高速机器人
local_costmap:
    width: 8.0
    height: 8.0

```

17.6.5 多图层融合策略

```

class MasterCostmap:
    """主代价地图：融合各层代价"""
    def __init__(self):
        self.layers = {}
        self.master = None

    def add_layer(self, name: str, layer: np.ndarray):
        """添加代价图层"""
        self.layers[name] = layer

    def update_master(self) -> np.ndarray:
        """融合所有图层到主代价地图

        融合策略：
        1. 取所有层的最大值
        2. 静态层优先级最高（255 覆盖一切）
        3. 障碍物层覆盖膨胀层
        """
        if not self.layers:
            return None

        # 初始化为所有层中最大的
        self.master = np.zeros_like(

```



```

        next(iter(self.layers.values())), dtype=np.uint8
    )

    for name, layer in self.layers.items():
        # 逐层取最大值融合
        mask = layer > self.master
        self.master[mask] = layer[mask]

    return self.master

def get_cost_at_pose(self, x: float, y: float,
                    robot_radius: float) -> int:
    """查询机器人占用区域的最高代价"""

    # 检查机器人足迹覆盖的栅格
    gx = int(x / 0.05)
    gy = int(y / 0.05)
    radius_cells = int(robot_radius / 0.05)

    max_cost = 0
    for dy in range(-radius_cells, radius_cells + 1):
        for dx in range(-radius_cells, radius_cells + 1):
            if dx*dx + dy*dy <= radius_cells*radius_cells:
                cx, cy = gx + dx, gy + dy
                if (0 <= cx < self.master.shape[1] and
                    0 <= cy < self.master.shape[0]):
                    cost = self.master[cy, cx]
                    max_cost = max(max_cost, cost)

    return max_cost

```

17.7 实战与调试

17.7.1 代价地图监控

```

# 查看代价地图话题
ros2 topic echo /global_costmap/costmap --once
ros2 topic echo /local_costmap/costmap --once

# 查看代价地图更新频率
ros2 topic hz /global_costmap/costmap
ros2 topic hz /local_costmap/costmap

# 在 RViz2 中可视化代价地图
rviz2
# 添加 → Map → Topic: /global_costmap/costmap
# 添加 → Map → Topic: /local_costmap/costmap

```

17.7.2 代价地图调试工具

```

import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
import numpy as np

```

```

class CostmapDebugger(Node):
    """代价地图调试工具"""
    def __init__(self):
        super().__init__('costmap_debugger')

        self.global_costmap_sub = self.create_subscription(
            OccupancyGrid, '/global_costmap/costmap',
            self.global_callback, 10)
        self.local_costmap_sub = self.create_subscription(
            OccupancyGrid, '/local_costmap/costmap',
            self.local_callback, 10)

    def analyze_costmap(self, msg: OccupancyGrid, name: str):
        """分析代价地图统计数据"""
        data = np.array(msg.data).reshape(
            msg.info.height, msg.info.width)

        total = data.size

        lethal = np.sum(data == 100)
        inscribed = np.sum((data > 0) & (data < 100))
        free = np.sum(data == 0)
        unknown = np.sum(data == -1)

        self.get_logger().info(
            f'{name}分析: '
            f'致命={lethal/total*100:.1f}%, '
            f'膨胀={inscribed/total*100:.1f}%, '
            f'空闲={free/total*100:.1f}%, '
            f'未知={unknown/total*100:.1f}%'
        )

    def global_callback(self, msg):
        self.analyze_costmap(msg, '全局代价地图')

    def local_callback(self, msg):
        self.analyze_costmap(msg, '局部代价地图')

```

17.7.3 手动清除代价地图

```

# 清除全局代价地图
ros2 service call /global_costmap/clear_entirely \
  nav2_msgs/srv/ClearEntireCostmap "{}"

# 清除局部代价地图
ros2 service call /local_costmap/clear_entirely \
  nav2_msgs/srv/ClearEntireCostmap "{}"

# 清除指定区域
ros2 service call /global_costmap/clear_around_local_costmap \
  nav2_msgs/srv/ClearCostmapAroundRobot "{}"

```

17.7.4 常见问题与解决方案

问题 1：代价地图未更新

- 检查传感器话题是否发布
- 检查 TF 变换是否正确
- 检查生命周期节点状态

问题 2：路径穿过障碍物

- 膨胀半径过小
- `cost_scaling_factor` 过大（衰减太快）
- 障碍物层未正确标记

问题 3：计算性能下降

- 减小代价地图尺寸
- 降低更新频率
- 增大分辨率（如从 0.05 改为 0.1）

课后练习

245. **原理题:** 解释代价地图中各代价值(0, 1-127, 128-253, 254, 255)的含义, 说明膨胀层的代价计算公式。
246. **配置题:** 为一个半径 0.3m 的差速机器人配置全局和局部代价地图, 要求: 全局地图分辨率 0.05m, 膨胀半径 0.6m; 局部地图为 3m 滚动窗口, 更新频率 5Hz。
247. **编程题:** 实现一个简单的代价地图膨胀层, 给定障碍物位置和膨胀半径, 计算每个栅格的代价值。
248. **分析题:** 比较全局代价地图和局部代价地图在坐标系、更新频率、大小和插件配置上的差异, 分析两者配合工作的机制。
249. **操作题:** 通过 ROS2 命令行工具查看当前代价地图的状态, 分析代价分布情况, 识别潜在导航问题。

250. **设计题:** 某机器人需要在包含玻璃墙的展览馆导航。玻璃墙对激光雷达透明但物理存在。设计代价地图方案，包括如何标记玻璃墙位置、配置传感器融合和膨胀参数以保证导航安全。

实验 17:

第 17 章 实验：MoveIt2 基础与运动学规划

****对应理论章节**:** 第 26 章《MoveIt2 配置与基础规划》、第 27 章《Python 关节空间规划》 ****实验课时**:** 4 课时 ****实验代码**:**
``src/lab_code/ch17_lab/moveit_fk_ik_lab/``

实验目标

- 掌握 MoveIt2 Setup Assistant 的使用方法
- 学会生成机械臂 MoveIt2 配置功能包
- 理解正运动学和逆运动学求解原理
- 能够使用 MotionPlanning 插件进行运动规划
- 掌握 MoveItPy 的基本编程方法
- 学会使用 PlanningComponent 进行关节空间规划
- 实现机械臂的多目标关节运动序列
- 掌握正运动学和逆运动学两种目标设置方式

实验环境

- ROS 2 Jazzy + MoveIt2
- xarm_description 功能包
- RViz2 + MotionPlanning 插件
- MoveIt2 Setup Assistant
- moveit_py

MoveIt 前置依赖（统一说明）：本章及后续 MoveIt 相关实验（第 18 章、第 21 章）开始前，需先启动 xArm 仿真与 MoveIt：

```
# 方式 1: 纯 MoveIt + RViz 仿真（不含 Gazebo）
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
```

```
# 方式 2: 含 Gazebo 的完整仿真 (本章参考代码 README 使用的命令)
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

默认使用上述 `xarm_ros2_arm_only` 启动命令。17.1 节的 MoveIt Setup Assistant 流程仅作为自行生成配置包的可选练习；生成后请使用实际包名及其生成的 launch 文件。

参考代码说明

`src/lab_code/ch17_lab/moveit_fk_ik_lab/` (ament_python 包) 提供以下程序，覆盖本章 FK/IK 规划练习：

程序	功能
<code>`fk_demo.py`</code>	正运动学 (FK) 演示：关节空间目标 + 夹爪开合 + Home 归位
<code>`ik_demo.py`</code>	逆运动学 (IK) 演示：设置末端目标位姿 (位置+姿态)，由 MoveIt2 解算 IK 并运动
<code>`fk_ik_exercise.py`</code>	FK/IK 综合练习 (含 TODO 填空)
<code>`rectangle_exercise.py`</code>	笛卡尔路径矩形轨迹规划练习

构建：

```
cd <工作区根目录>
colcon build --symlink-install --packages-select moveit_fk_ik_lab
source install/setup.bash
```

17.1 MoveIt2 配置与基础规划

17.1.1 安装 MoveIt2

```
sudo apt install ros-jazzy-moveit ros-jazzy-moveit-py
sudo apt install ros-jazzy-moveit-setup-assistant

# 验证安装
ros2 pkg list | grep moveit
```

17.1.2 准备机械臂模型

确保已准备与课程配置兼容的 XBot Arm `xarm_description` 2.0.0：

```
source /path/to/xarm_description_workspace/install/setup.bash

# 找到 URDF 文件路径
ros2 pkg prefix xarm_description
ls $(ros2 pkg prefix xarm_description)/share/xarm_description/urdf/
```

17.1.3 启动 Setup Assistant

```
# 新建终端
ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

17.1.4 配置流程

创建新配置

- 点击 "Create New MoveIt Configuration Package"
- 点击 "Browse", 选择 URDF 文件路径: \$(ros2 pkg prefix xarm_description)/share/xarm_description/urdf/arm.urdf.xacro
- 点击 "Load Files"

生成自碰撞矩阵

- 点击左侧 "Self-Collisions"
- 保持默认采样密度 10000
- 点击 "Generate Collision Matrix"
- 查看生成的免检碰撞对

添加虚拟关节

- 点击 "Virtual Joints"
- 添加虚拟关节: xarm_virtual_joint
- Type: fixed
- Parent Frame: world
- Child Frame: base_link

创建规划组

- 点击 "Planning Groups" → "Add Group"
- 创建 arm 组:
- Group Name: xarm_group
- Kinematic Solver: kdl_kinematics_plugin/KDLKinematicsPlugin

- IK Solver: KDL
- Planner: RRTConnectkConfigDefault
- Add Joints: 选择 arm_1_joint 到 arm_6_joint
- 创建 gripper 组:
- Group Name: gripper_group
- Kinematic Solver: kdl_kinematics_plugin/KDLKinematicsPlugin
- Add Joints: 选择 gripper_1_joint, gripper_2_joint

添加预设位姿

- 点击 "Robot Poses"
- 添加 Home 位姿:
- Pose Name: home
- Group: xarm_group
- Joint Values: 全部设为 0
- 添加 vertical 位姿:
- Pose Name: vertical
- arm_1_joint: 0, arm_2_joint: -1.57, arm_3_joint: 0, arm_4_joint: -1.57, arm_5_joint: 0, arm_6_joint: 0

添加末端执行器

- 点击 "End Effectors"
- Name: gripper
- End Effector Group: gripper_group
- Parent Link: gripper_centor_link

配置被动关节

- 点击 "Passive Joints"
- 确认没有需要设为被动的关节

生成配置包

- 点击 "Configuration Files"

- 输出目录: `~/ros2_arm_ws/src/<你的配置包名>`
- 点击 "Generate Package"

17.1.5 编译配置包

```
cd ~/ros2_arm_ws
colcon build --packages-select <你的配置包名>
source install/setup.bash
```

17.1.6 启动 MoveIt2 演示

```
# 默认使用课程提供的 xArm 纯 MoveIt + RViz 仿真（见"实验环境"前置说明）
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py

# 可选：使用自行生成的配置包（将占位名称替换为实际包名）
ros2 launch <你的配置包名> demo.launch.py
```

17.1.7 配置 MotionPlanning 插件

在 RViz2 中:

- 在左侧 Displays 面板确认已添加 MotionPlanning 插件
- 设置 Global Options → Fixed Frame: `base_link`
- MotionPlanning → Planning Group: `xarm_group`
- 勾选 "Query Start State" 和 "Query Goal State"
- 设置 "Robot Description": `robot_description`

17.1.8 正运动学规划

251. 在 MotionPlanning 面板中:
252. Start State: 选择 `<current>`, 点击 "Update"
253. Goal State: 选择 `home`, 点击 "Update"
254. 点击 "Plan"
255. 观察规划的绿色轨迹
256. 点击 "Execute" 执行
257. 尝试预设位姿:
258. Goal State 选择 `vertical`
259. 点击 "Plan and Execute"

17.1.9 随机位姿测试

260. Goal State: 选择 random valid
261. 点击 "Update"
262. 点击 "Plan"
263. 观察随机生成的关节值

17.1.10 鼠标拖拽交互

- 在 3D 视图中，拖动机械臂末端的浅蓝色圆球
- 旋转红绿蓝圆环改变末端姿态
- 点击 "Plan" 查看逆运动学求解结果
- 点击 "Execute" 执行运动

17.1.11 逆运动学验证

创建 Python 脚本 ik_test.py:

```
import rclpy
from rclpy.node import Node
from moveit_msgs.srv import GetPositionIK
from geometry_msgs.msg import PoseStamped
from std_msgs.msg import Header

def main():
    rclpy.init()
    node = Node('ik_test_client')
    client = node.create_client(GetPositionIK, '/compute_ik')
    while not client.wait_for_service(timeout_sec=1.0):
        node.get_logger().info('等待 IK 服务...')

    request = GetPositionIK.Request()
    request.ik_request.group_name = 'xarm_group'
    request.ik_request.robot_state.joint_state.name = [
        'arm_1_joint', 'arm_2_joint', 'arm_3_joint',
        'arm_4_joint', 'arm_5_joint', 'arm_6_joint'
    ]
    request.ik_request.robot_state.joint_state.position = [0.0]*6

    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.x = 0.3
    pose.pose.position.y = 0.0
    pose.pose.position.z = 0.4
    pose.pose.orientation.w = 1.0
    request.ik_request.pose_stamped = pose

    future = client.call_async(request)
    rclpy.spin_until_future_complete(node, future)
```

```

result = future.result()

if result.error_code.val == result.error_code.SUCCESS:
    joints = result.solution.joint_state.position
    names = result.solution.joint_state.name
    for name, pos in zip(names, joints):
        print(f'{name}: {pos:.4f}')
else:
    print(f'IK 求解失败: error_code={result.error_code.val}')

node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行:

```

# 在 MoveIt2 演示环境运行的同时执行
python3 ik_test.py

```

17.1.12 正运动学验证

创建 fk_test.py:

```

import rclpy
from rclpy.node import Node
from moveit_msgs.srv import GetPositionFK
from sensor_msgs.msg import JointState

def main():
    rclpy.init()
    node = Node('fk_test_client')
    client = node.create_client(GetPositionFK, '/compute_fk')
    while not client.wait_for_service(timeout_sec=1.0):
        node.get_logger().info('等待 FK 服务...')

    request = GetPositionFK.Request()
    request.header.frame_id = 'base_link'
    request.fk_link_names = ['gripper_center_link']
    request.robot_state.joint_state.name = [
        'arm_1_joint', 'arm_2_joint', 'arm_3_joint',
        'arm_4_joint', 'arm_5_joint', 'arm_6_joint'
    ]
    request.robot_state.joint_state.position = [0.5, -0.3, 0.2, 0.0, 0.0, 0.0]

    future = client.call_async(request)
    rclpy.spin_until_future_complete(node, future)
    result = future.result()

    if result.error_code.val == result.error_code.SUCCESS:
        for pose in result.pose_stamped:
            p = pose.pose.position
            print(f'末端位置: x={p.x:.4f}, y={p.y:.4f}, z={p.z:.4f}')
    else:
        print(f'FK 求解失败: error_code={result.error_code.val}')

```

```

node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.1.13 使用命令行工具

```

# 安装 moveit_commander
sudo apt install ros-jazzy-moveit-commander

# 启动交互式命令行
ros2 run moveit_commander moveit_commander_cmdline.py

# 在命令行中:
use xarm_group
current
rec c
goal = c
goal[0] = 0.5
go goal

```

17.1.14 测试规划场景

添加障碍物测试避障:

264. 在 MotionPlanning → Scene Objects 面板

265. 点击 "Import From Text"

266. 输入障碍物描述:

```

<object>
  <id>box</id>
  <pose>0.3 0 0.2 0 0 0 1</pose>
  <size>0.1 0.3 0.4</size>
</object>

```

267. 将目标设到障碍物后方，观察避障路径

17.2 Python 关节空间规划

17.2.1 运行参考代码 moveit_fk_ik_lab

先按"实验环境"中的前置说明启动 xArm 仿真与 MoveIt (默认使用 `ros2 launch`

`xarm_ros2_arm_only arm_only_move_group.launch.py`; 需要 Gazebo 时使用 `ros2 launch`

`xarm_ros2_arm_only arm_only.launch.py`)，再运行实验程序:

```

ros2 run moveit_fk_ik_lab fk_demo
ros2 run moveit_fk_ik_lab ik_demo

```

```
ros2 run moveit_fk_ik_lab fk_ik_exercise
ros2 run moveit_fk_ik_lab rectangle_exercise
```

程序依次输出各目标规划与执行日志（如 arm joints: [...], pose x=0.30）, RViz/Gazebo 中机械臂按规划运动并最终回到 Home。

`fk\_ik\_exercise.py` TODO 练习 fk_ik_exercise.py TODO 练习**——打开脚本，完成以下 TODO:

- 268. 设置末端执行器目标位姿（位置 x=0.3, y=-0.3, z=0.3, 姿态 rpy=0,0,- $\pi/4$ ）
- 269. 设置机械臂当前状态为初始状态
- 270. 规划并执行运动到目标位姿
- 271. 设置六关节角度目标值 [-0.9, -1.0, 0.2, 0.9, -0.76, 1.5] 并规划执行
- 272. 使用命名目标 Home 回到初始位置

`rectangle\_exercise.py` TODO 练习 rectangle_exercise.py TODO 练习**——补全第三个和第四个矩形顶点坐标，完成笛卡尔路径规划。

17.2.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch17_moveit_py --build-type ament_python --dependencies rclpy moveit
cd ch17_moveit_py
mkdir -p ch17_moveit_py
```

17.2.3 编写基础关节空间规划程序

创建 ch17_moveit_py/joint_space_planner.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class JointSpacePlanner(Node):
    def __init__(self):
        super().__init__('joint_space_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.get_logger().info('MoveItPy 关节空间规划器已启动')

    def plan_and_execute(self, target_joints):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(target_joints)
        plan_result = self.arm.plan()
```

```

        if plan_result:
            self.arm.execute(plan_result.trajectory)
            self.get_logger().info(f'执行成功: {target_joints}')
            return True
        else:
            self.get_logger().warn('规划失败')
            return False

    def run_joint_sequence(self):
        self.get_logger().info('=== 关节空间运动序列 ===')

        # 回到 Home
        home = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
        self.plan_and_execute(home)
        time.sleep(1.0)

        # 序列 1
        pose1 = [0.5, -0.3, 0.2, 0.0, 0.0, 0.0]
        self.plan_and_execute(pose1)
        time.sleep(1.0)

        # 序列 2
        pose2 = [-0.3, -0.8, 0.5, -0.2, 0.0, 0.0]
        self.plan_and_execute(pose2)
        time.sleep(1.0)

        # 序列 3
        pose3 = [0.0, -1.2, 1.0, -0.5, 0.3, 0.0]
        self.plan_and_execute(pose3)
        time.sleep(1.0)

        # 回到 Home
        self.plan_and_execute(home)
        self.get_logger().info('关节运动序列完成')

def main(args=None):
    rclpy.init(args=args)
    node = JointSpacePlanner()
    node.run_joint_sequence()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.4 运行关节空间规划

```

cd ~/ros2_arm_ws
colcon build --packages-select ch17_moveit_py
source install/setup.bash

# 在 MoveIt2 演示环境运行的同时
ros2 run ch17_moveit_py joint_space_planner

```

17.2.5 编写逆运动学规划程序

创建 ch17_moveit_py/ik_planner.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import PoseStamped
from tf_transformations import quaternion_from_euler
import time
import math

class IKPlanner(Node):
    def __init__(self):
        super().__init__('ik_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )

        self.reference_frame = 'base_link'
        self.end_effector_link = 'gripper_cenotor_link'

        self.arm.set_pose_reference_frame(self.reference_frame)
        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.02)
        self.arm.set_max_acceleration_scaling_factor(0.5)
        self.arm.set_max_velocity_scaling_factor(0.5)

        self.get_logger().info('IK 规划器已启动')

    def set_pose_target(self, x, y, z, roll=0, pitch=0, yaw=0):
        target_pose = PoseStamped()
        target_pose.header.frame_id = self.reference_frame
        target_pose.pose.position.x = x
        target_pose.pose.position.y = y
        target_pose.pose.position.z = z
        q = quaternion_from_euler(roll, pitch, yaw)
        target_pose.pose.orientation.x = q[0]
        target_pose.pose.orientation.y = q[1]
        target_pose.pose.orientation.z = q[2]
        target_pose.pose.orientation.w = q[3]

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose.pose, self.end_effector_link)
        plan_result = self.arm.plan()
        if plan_result:
            self.arm.execute(plan_result.trajectory)
            self.get_logger().info(f'到达目标位姿: ({x:.3f}, {y:.3f}, {z:.3f})')
            return True
        else:
            self.get_logger().warn(f'IK 规划失败: ({x:.3f}, {y:.3f}, {z:.3f})')
            return False

    def run_ik_sequence(self):
        self.get_logger().info('=== IK 规划序列 ===')
```

```

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        time.sleep(1.0)

        # 目标1: 前方
        self.set_pose_target(0.3, 0.0, 0.4, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 目标2: 右侧
        self.set_pose_target(0.2, 0.25, 0.35, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 目标3: 左侧
        self.set_pose_target(0.2, -0.25, 0.35, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 目标4: 上方
        self.set_pose_target(0.2, 0.0, 0.5, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

        self.get_logger().info('IK 规划序列完成')

def main(args=None):
    rclpy.init(args=args)
    node = IKPlanner()
    node.run_ik_sequence()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行 IK 规划:

```
ros2 run ch17_moveit_py ik_planner
```

17.2.6 编写带速度控制的规划程序

创建 `ch17_moveit_py/velocity_controlled_planner.py`:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class VelocityControlledPlanner(Node):
    def __init__(self):

```



```

    super().__init__('velocity_controlled_planner')
    self.moveit = MoveItPy(node_name='moveit_py')
    self.arm = PlanningComponent(
        self.moveit, 'xarm_group', 'gripper_cenror_link'
    )

def move_joints(self, joints, vel_factor=0.5, acc_factor=0.5):
    self.arm.set_start_state_to_current_state()
    self.arm.set_joint_value_target(joints)
    self.arm.set_max_velocity_scaling_factor(vel_factor)
    self.arm.set_max_acceleration_scaling_factor(acc_factor)
    plan_result = self.arm.plan()
    if plan_result:
        duration = plan_result.trajectory.joint_trajectory.points[-1].time_from_start.sec
        self.get_logger().info(
            f'速度: {vel_factor}, 加速度: {acc_factor}, 轨迹时长: {duration}s'
        )
        self.arm.execute(plan_result.trajectory)
        return True
    return False

def run_velocity_demo(self):
    target = [0.5, -0.5, 0.3, -0.1, 0.0, 0.0]
    self.get_logger().info('=== 速度对比演示 ===')

    # 低速
    self.get_logger().info('低速运动 (vel=0.2)')
    self.move_joints(target, 0.2, 0.2)
    time.sleep(1.0)

    # 回到 home
    self.move_joints([0.0]*6)
    time.sleep(1.0)

    # 中速
    self.get_logger().info('中速运动 (vel=0.5)')
    self.move_joints(target, 0.5, 0.5)
    time.sleep(1.0)

    # 回到 home
    self.move_joints([0.0]*6)
    time.sleep(1.0)

    # 高速
    self.get_logger().info('高速运动 (vel=1.0)')
    self.move_joints(target, 1.0, 1.0)

    self.get_logger().info('速度演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = VelocityControlledPlanner()
    node.run_velocity_demo()
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':  
    main()
```

17.2.7 编写手爪控制程序

创建 ch17_moveit_py/gripper_controller.py:

```
import rclpy  
from rclpy.node import Node  
from moveit.planning import MoveItPy, PlanningComponent  
import time  
  
class GripperController(Node):  
    def __init__(self):  
        super().__init__('gripper_controller')  
        self.moveit = MoveItPy(node_name='moveit_py')  
        self.gripper = PlanningComponent(  
            self.moveit, 'gripper_group', 'gripper_cenotor_link'  
        )  
  
    def set_gripper(self, position):  
        self.gripper.set_start_state_to_current_state()  
        self.gripper.set_joint_value_target(position)  
        plan_result = self.gripper.plan()  
        if plan_result:  
            self.gripper.execute(plan_result.trajectory)  
            self.get_logger().info(f'手爪位置: {position}')  
            return True  
        return False  
  
    def run_gripper_demo(self):  
        self.get_logger().info('=== 手爪控制演示 ===')  
  
        open_pos = [0.65, 0.65]  
        close_pos = [0.0, 0.0]  
  
        self.get_logger().info('张开手爪')  
        self.set_gripper(open_pos)  
        time.sleep(1.0)  
  
        self.get_logger().info('闭合手爪')  
        self.set_gripper(close_pos)  
        time.sleep(1.0)  
  
        self.get_logger().info('半开手爪')  
        half_pos = [0.3, 0.3]  
        self.set_gripper(half_pos)  
        time.sleep(1.0)  
  
        self.get_logger().info('回到张开')  
        self.set_gripper(open_pos)  
  
        self.get_logger().info('手爪控制演示完成')  
  
def main(args=None):  
    rclpy.init(args=args)  
    node = GripperController()
```

```

node.run_gripper_demo()
node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.8 编写完整运动序列程序

创建 `ch17_moveit_py/complete_sequence.py`:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class CompleteSequence(Node):
    def __init__(self):
        super().__init__('complete_sequence')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenror_link'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper_group', 'gripper_cenror_link'
        )
        self.get_logger().info('完整运动序列已启动')

    def move_arm(self, joints):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joints)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

    def move_gripper(self, pos):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target(pos)
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def run(self):
        self.get_logger().info('开始序列: Home → 准备 → 抓取 → 提起 → 放置 → Home')

        home = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
        prepare = [0.5, -0.8, 0.6, -0.3, 0.0, 0.0]
        grasp = [0.5, -0.3, 0.2, -0.1, 0.0, 0.0]
        lift = [0.5, -0.8, 0.6, -0.3, 0.0, 0.0]
        place = [-0.3, -0.8, 0.5, -0.2, 0.0, 0.0]

        self.move_arm(home)
        time.sleep(1)

        self.get_logger().info('[1/5] 移动到准备位姿')
        self.move_arm(prepare)
        time.sleep(1)

```

```

        self.get_logger().info('[2/5] 张开手爪')
        self.move_gripper([0.65, 0.65])
        time.sleep(1)

        self.get_logger().info('[3/5] 下降到抓取位姿')
        self.move_arm(grasp)
        time.sleep(1)

        self.get_logger().info('[4/5] 闭合手爪')
        self.move_gripper([0.0, 0.0])
        time.sleep(1)

        self.get_logger().info('[5/5] 提起物体')
        self.move_arm(lift)
        time.sleep(1)

        self.get_logger().info('移动到放置位置')
        self.move_arm(place)
        time.sleep(1)

        self.get_logger().info('张开手爪释放物体')
        self.move_gripper([0.65, 0.65])
        time.sleep(1)

        self.move_arm(home)
        self.get_logger().info('序列完成!')

def main(args=None):
    rclpy.init(args=args)
    node = CompleteSequence()
    node.run()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.9 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'joint_space_planner = ch17_moveit_py.joint_space_planner:main',
        'ik_planner = ch17_moveit_py.ik_planner:main',
        'velocity_controlled_planner = ch17_moveit_py.velocity_controlled_planner:main',
        'gripper_controller = ch17_moveit_py.gripper_controller:main',
        'complete_sequence = ch17_moveit_py.complete_sequence:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch17_moveit_py
source install/setup.bash

# MoveIt2 演示环境启动后，分别执行：
ros2 run ch17_moveit_py joint_space_planner
ros2 run ch17_moveit_py ik_planner

```

```
ros2 run ch17_moveit_py velocity_controlled_planner
ros2 run ch17_moveit_py gripper_controller
ros2 run ch17_moveit_py complete_sequence
```

17.2.10 获取当前状态信息

创建 ch17_moveit_py/state_monitor.py:

```
import rclpy
from rclpy.node import Node
from moveit.Planning import MoveItPy, PlanningComponent

class StateMonitor(Node):
    def __init__(self):
        super().__init__('state_monitor')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.timer = self.create_timer(2.0, self.print_state)

    def print_state(self):
        state = self.moveit.get_robot_state()
        joint_names = [
            'arm_1_joint', 'arm_2_joint', 'arm_3_joint',
            'arm_4_joint', 'arm_5_joint', 'arm_6_joint'
        ]
        for name in joint_names:
            pos = state.get_joint_positions(name)
            self.get_logger().info(f'{name}: {pos:.4f}')

def main(args=None):
    rclpy.init(args=args)
    node = StateMonitor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

实验结果与分析

- Setup Assistant 成功生成了 xArm 的 MoveIt2 配置包
- MotionPlanning 插件支持正运动学和逆运动学两种规划方式
- IK 服务成功将末端位姿转换为关节角度
- FK 服务从关节角度计算出末端位姿
- 规划场景中的障碍物会影响路径规划，产生绕行轨迹
- MoveItPy 提供了丰富的 Python API 实现机械臂运动规划
- set_joint_value_target 实现关节空间控制, set_pose_target 调用逆运动学求解
- 速度/加速度缩放因子可以有效控制机械臂运动速率
- 完整的运动序列可以组合多个规划步骤完成复杂任务

- 参考代码 `moveit_fk_ik_lab` 的 `fk_demo/ik_demo` 与自实现程序行为一致, `fk_ik_exercise/rectangle_exercise` 提供了填空练习用于自测

思考题

273. MoveIt2 中 KDL 求解器和 TRAC-IK 求解器有什么区别? 如何切换?
274. Self-Collision 矩阵在什么场景下需要重新生成?
275. 在逆运动学求解中, 如何指定求解精度和最大迭代次数?
276. 虚拟关节(Virtual Joint)的作用是什么? 在什么场景下需要使用?
277. `plan()`方法返回的 `plan_result` 中包含哪些信息? 如何获取轨迹执行时间?
278. MoveItPy 中如何设置不同的规划器(planner)? RRTConnect 和 RRT 有什么区别?
279. 当 IK 求解失败时, 如何调整参数提高求解成功率?
280. 如何实现多个机械臂组件的协同运动?

第 18 章 全局路径规划

学习目标

- 理解 Dijkstra 算法和 A*算法的原理与实现
- 掌握 Nav2 Smac Planner 的工作原理
- 熟悉全局路径规划的代价函数设计
- 能够配置和切换不同的全局规划器
- 掌握路径优化和平滑技术

18.1 全局路径规划概述

18.1.1 问题定义

全局路径规划是在已知环境地图中，寻找从起点到终点的无碰撞最优路径。

输入：

- 环境地图（代价地图）
- 起点位姿 (x_start, y_start, θ_{start})
- 终点位姿 (x_goal, y_goal, θ_{goal})

输出：

- 路径：一系列连续的位姿点 $\{(x_1, y_1, \theta_1), \dots, (x_n, y_n, \theta_n)\}$

优化目标：

```
path* = argmin f(path)
```

其中 $f(\text{path})$ 通常为路径总长度，也可包含其他约束（如转弯代价、安全距离）。

18.1.2 路径规划的评价指标

指标	含义	重要性
路径长度	从起点到终点的总距离	高
计算时间	规划所需时间	高（实时性）

平滑度	路径曲率变化	中
安全距离	距离障碍物的最近距离	高
完备性	是否存在可行解	高

18.2 Dijkstra 算法

18.2.1 算法原理

Dijkstra 算法是一种广度优先的最短路径搜索算法，它从起点开始，逐步扩展，保证找到最短路径。

核心思想：

- 维护从起点到每个节点的最短距离 $G(n)$
- 每次选择 $G(n)$ 最小的节点进行扩展
- 直到扩展到目标点

伪代码：

```

1. 初始化：
   - open_list = {start}
   - closed_list = {}
   -  $g(\text{start}) = 0$ ,  $g(\text{其他}) = \infty$ 

2. while open_list 不为空：
   3. 从 open_list 中取出  $g$  值最小的节点  $n$ 
   4. if  $n == \text{goal}$ : 回溯路径，返回成功
   5. 将  $n$  移入 closed_list
   6. 遍历  $n$  的相邻节点  $m$ :
       7. if  $m$  in closed_list: continue
       8. if  $m$  是障碍物: continue
       9.  $\text{new\_g} = g(n) + \text{cost}(n, m)$ 
       10. if  $\text{new\_g} < g(m)$ :
           11.  $g(m) = \text{new\_g}$ 
           12.  $\text{parent}(m) = n$ 
           13. if  $m$  not in open_list: 加入 open_list

3. return 失败 (无可行路径)

```

18.2.2 Python 实现

```

import numpy as np
import heapq

```



```

from enum import Enum

class CellState(Enum):
    FREE = 0
    OBSTACLE = 1
    START = 2
    GOAL = 3
    PATH = 4

class DijkstraPlanner:
    """Dijkstra 全局路径规划器"""
    def __init__(self, costmap: np.ndarray, resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution
        self.height, self.width = costmap.shape

        # 8 邻域连接
        self.neighbors = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),      (0, 1),
            (1, -1), (1, 0), (1, 1)
        ]
        self.movement_costs = [
            np.sqrt(2), 1.0, np.sqrt(2),
            1.0,      1.0,
            np.sqrt(2), 1.0, np.sqrt(2)
        ]

    def world_to_grid(self, wx: float, wy: float) -> tuple:
        """世界坐标转栅格坐标"""
        # 假设地图原点在 (0, 0)
        return int(wy / self.resolution), int(wx / self.resolution)

    def grid_to_world(self, row: int, col: int) -> tuple:
        """栅格坐标转世界坐标"""
        return col * self.resolution, row * self.resolution

    def plan(self, start_world: tuple, goal_world: tuple) -> list:
        """
        执行 Dijkstra 路径规划

        参数:
            start_world: (x, y) 起点世界坐标
            goal_world: (x, y) 终点世界坐标

        返回:
            path: [(x1,y1), (x2,y2), ...] 世界坐标路径点
        """
        start = self.world_to_grid(*start_world)
        goal = self.world_to_grid(*goal_world)

        if not self._is_valid(start) or not self._is_valid(goal):
            print("起点或终点无效")
            return []

```

```

# 初始化
g_score = np.full((self.height, self.width), np.inf)
g_score[start] = 0

parent = {}

# 优先队列: (g_score, (row, col))
open_heap = [(0, start)]
closed_set = set()

while open_heap:
    # 取出 g 值最小的节点
    current_g, current = heapq.heappop(open_heap)

    if current in closed_set:
        continue

    # 到达目标
    if current == goal:
        return self._reconstruct_path(parent, start, goal)

    closed_set.add(current)

    # 扩展邻居
    for i, (dr, dc) in enumerate(self.neighbors):
        neighbor = (current[0] + dr, current[1] + dc)

        if neighbor in closed_set:
            continue

        if not self._is_valid(neighbor):
            continue

        # 代价 = 移动代价 + 地图代价
        movement_cost = self.movement_costs[i]
        map_cost = self._get_map_cost(neighbor)
        step_cost = movement_cost * (1.0 + map_cost / 50.0)

        tentative_g = current_g + step_cost

        if tentative_g < g_score[neighbor]:
            g_score[neighbor] = tentative_g
            parent[neighbor] = current
            heapq.heappush(open_heap, (tentative_g, neighbor))

    return [] # 无可行路径

def _is_valid(self, cell: tuple) -> bool:
    """检查栅格是否有效"""
    r, c = cell
    if not (0 <= r < self.height and 0 <= c < self.width):
        return False
    return self.costmap[r, c] < 254 # 非障碍物

def _get_map_cost(self, cell: tuple) -> float:
    """获取地图代价值"""

```

```

        r, c = cell
        cost = self.costmap[r, c]
        if cost >= 254:
            return float('inf')
        return float(cost)

    def _reconstruct_path(self, parent: dict, start: tuple,
                          goal: tuple) -> list:
        """回溯重建路径"""
        path_grid = []
        current = goal

        while current != start:
            path_grid.append(current)
            current = parent.get(current)
            if current is None:
                return []
        path_grid.append(start)
        path_grid.reverse()

        # 转换为世界坐标
        path_world = [self.grid_to_world(r, c) for r, c in path_grid]
        return path_world

```

18.2.3 Dijkstra 的特点

优点：

- 保证找到最短路径
- 算法完备（有解必能找到）
- 实现简单

缺点：

- 搜索范围大，效率低
- 不考虑目标位置，向所有方向等距离扩展
- 在大地图上计算时间过长

18.3 A*算法

18.3.1 算法原理

A*算法在 Dijkstra 的基础上加入启发式函数，引导搜索方向，提高效率。

评价函数：

$$F(n) = G(n) + H(n)$$

- $G(n)$: 起点到当前节点 n 的实际代价

- $H(n)$: 当前节点 n 到目标点的启发式估计代价

启发式函数的选择:

```
def manhattan_distance(a: tuple, b: tuple) -> float:
    """曼哈顿距离（四邻域连接时使用）"""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean_distance(a: tuple, b: tuple) -> float:
    """欧氏距离（八邻域连接时使用）"""
    return np.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)

def chebyshev_distance(a: tuple, b: tuple) -> float:
    """切比雪夫距离"""
    return max(abs(a[0] - b[0]), abs(a[1] - b[1]))

def diagonal_distance(a: tuple, b: tuple, d1: float = 1.0,
                      d2: float = np.sqrt(2)) -> float:
    """对角线距离（考虑对角线移动）"""
    dx = abs(a[0] - b[0])
    dy = abs(a[1] - b[1])
    return d1 * (dx + dy) + (d2 - 2 * d1) * min(dx, dy)
```

18.3.2 Python 实现

```
import heapq
import numpy as np

class AStarPlanner:
    """A*路径规划器"""
    def __init__(self, costmap: np.ndarray, resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution
        self.height, self.width = costmap.shape

        # 8 邻域
        self.neighbors = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),          (0, 1),
            (1, -1),  (1, 0),  (1, 1)
        ]
        self.movement_costs = [
            np.sqrt(2), 1.0, np.sqrt(2),
            1.0,          1.0,
            np.sqrt(2), 1.0, np.sqrt(2)
        ]

    def plan(self, start_world: tuple, goal_world: tuple) -> list:
        """执行 A*路径规划"""
        start = self._world_to_grid(*start_world)
        goal = self._world_to_grid(*goal_world)

        if not self._is_valid(start) or not self._is_valid(goal):
            return []
```

```

g_score = {start: 0.0}
f_score = {start: self._heuristic(start, goal)}

open_heap = [(f_score[start], start)]
open_set = {start}
closed_set = set()
parent = {}

while open_heap:
    _, current = heapq.heappop(open_heap)

    if current == goal:
        return self._reconstruct_path(parent, start, goal)

    open_set.discard(current)
    closed_set.add(current)

    for i, (dr, dc) in enumerate(self.neighbors):
        neighbor = (current[0] + dr, current[1] + dc)

        if neighbor in closed_set:
            continue

        if not self._is_valid(neighbor):
            continue

        move_cost = self.movement_costs[i]
        map_cost = self._get_map_cost(neighbor)
        step_cost = move_cost * (1.0 + map_cost / 50.0)

        tentative_g = g_score[current] + step_cost

        if neighbor not in g_score or tentative_g < g_score[neighbor]:
            parent[neighbor] = current
            g_score[neighbor] = tentative_g
            f_score[neighbor] = tentative_g + self._heuristic(neighbor, goal)

            if neighbor not in open_set:
                heapq.heappush(open_heap, (f_score[neighbor], neighbor))
                open_set.add(neighbor)

    return []

def _heuristic(self, cell: tuple, goal: tuple) -> float:
    """启发式函数：对角线距离"""
    dr = abs(cell[0] - goal[0])
    dc = abs(cell[1] - goal[1])
    return (1.0 * (dr + dc) + (np.sqrt(2) - 2 * 1.0) * min(dr, dc))

def _world_to_grid(self, wx: float, wy: float) -> tuple:
    return int(wy / self.resolution), int(wx / self.resolution)

def _grid_to_world(self, row: int, col: int) -> tuple:
    return col * self.resolution, row * self.resolution

def _is_valid(self, cell: tuple) -> bool:
    r, c = cell
    if not (0 <= r < self.height and 0 <= c < self.width):

```

```

        return False
    return self.costmap[r, c] < 254

def _get_map_cost(self, cell: tuple) -> float:
    r, c = cell
    cost = self.costmap[r, c]
    return float('inf') if cost >= 254 else float(cost)

def _reconstruct_path(self, parent: dict, start: tuple,
                      goal: tuple) -> list:
    path_grid = []
    current = goal
    while current != start:
        path_grid.append(current)
        current = parent.get(current)
        if current is None:
            return []
    path_grid.append(start)
    path_grid.reverse()

    return [self.grid_to_world(r, c) for r, c in path_grid]

```

18.3.3 A* vs Dijkstra

```

def compare_algorithms(costmap: np.ndarray,
                       start: tuple, goal: tuple):
    """对比 Dijkstra 和 A*的性能"""
    import time

    # Dijkstra
    dijkstra = DijkstraPlanner(costmap)
    t1 = time.time()
    path_d = dijkstra.plan(start, goal)
    t_d = time.time() - t1

    # A*
    astar = AStarPlanner(costmap)
    t1 = time.time()
    path_a = astar.plan(start, goal)
    t_a = time.time() - t1

    print(f"Dijkstra: {len(path_d)}点, {t_d*1000:.1f}ms")
    print(f"A*: {len(path_a)}点, {t_a*1000:.1f}ms")

    if path_d and path_a:
        len_d = sum(np.linalg.norm(np.array(path_d[i+1]) - np.array(path_d[i]))
                    for i in range(len(path_d)-1))
        len_a = sum(np.linalg.norm(np.array(path_a[i+1]) - np.array(path_a[i]))
                    for i in range(len(path_a)-1))
        print(f"Dijkstra 路径长度: {len_d:.2f}m")
        print(f"A*路径长度: {len_a:.2f}m")

```

18.4 Nav2 Smac Planner

18.4.1 Smac Planner 概述

Smac Planner 是 Nav2 内置的高级规划器，支持多种规划算法：

规划器	算法	特点	适用场景
NavfnPlanner	导航函数	经典算法，路径平滑	简单环境
SmacPlanner2D	A*	标准 A*在 2D 代价地图	通用场景
SmacPlannerHybrid	Hybrid-A*	带运动学约束	非全向机器人
SmacPlannerLattice	状态格	满足动力学约束	阿克曼转向
ThetaStarPlanner	Theta*	任意角度路径	全向机器人

18.4.2 Hybrid-A*算法

Hybrid-A 在标准 A 基础上加入运动学约束，适用于非全向移动机器人。

与 A*的区别：

- 状态空间从 (x, y) 扩展到 (x, y, θ)
- 考虑最小转弯半径
- 支持前进和后退
- 使用 Reeds-Shepp 曲线连接

```
import numpy as np
import heapq

class HybridAStarNode:
    """Hybrid-A*节点"""
    def __init__(self, x: float, y: float, theta: float):
        self.x = x
        self.y = y
        self.theta = theta
        self.g = float('inf')
        self.f = float('inf')
        self.parent = None

    def __lt__(self, other):
        return self.f < other.f

class HybridAStarPlanner:
    """Hybrid-A*路径规划器（简化版）"""
    def __init__(self, costmap: np.ndarray,
                 min_turning_radius: float = 0.5,
                 resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution
        self.min_turning_radius = min_turning_radius
```

```

        # 运动控制采样 (速度+转角)
        self.motion_primitives = self._generate_motion_primitives()

def _generate_motion_primitives(self) -> list:
    """生成运动基元"""
    primitives = []

    # 前进
    for steer in [-0.4, -0.2, 0.0, 0.2, 0.4]:
        primitives.append({
            'v': 0.3,          # 前进速度
            'steer': steer,    # 转角
            'dt': 0.5,        # 时间步长
            'gear': 1         # 前进
        })

    # 后退
    for steer in [-0.2, 0.0, 0.2]:
        primitives.append({
            'v': -0.2,
            'steer': steer,
            'dt': 0.3,
            'gear': -1
        })

    return primitives

def _simulate_motion(self, x: float, y: float, theta: float,
                    primitive: dict) -> tuple:
    """模拟运动基元"""
    v = primitive['v']
    steer = primitive['steer']
    dt = primitive['dt']

    # 自行车模型
    new_x = x + v * np.cos(theta) * dt
    new_y = y + v * np.sin(theta) * dt
    new_theta = theta + v * np.tan(steer) / 0.5 * dt

    return new_x, new_y, new_theta

def _is_collision_free(self, x: float, y: float) -> bool:
    """检查位姿是否无碰撞"""
    gx = int(y / self.resolution)
    gy = int(x / self.resolution)

    if not (0 <= gx < self.costmap.shape[0] and
            0 <= gy < self.costmap.shape[1]):
        return False

    # 检查足迹范围内是否有障碍物
    robot_radius_cells = int(0.22 / self.resolution)
    for dr in range(-robot_radius_cells, robot_radius_cells + 1):
        for dc in range(-robot_radius_cells, robot_radius_cells + 1):

```



```

        if dr*dr + dc*dc <= robot_radius_cells*robot_radius_cells:
            nr, nc = gx + dr, gy + dc
            if (0 <= nr < self.costmap.shape[0] and
                0 <= nc < self.costmap.shape[1]):
                if self.costmap[nr, nc] >= 254:
                    return False
        return True

def plan(self, start: tuple, goal: tuple) -> list:
    """
    执行 Hybrid-A*规划

    参数:
        start: (x, y, theta)
        goal: (x, y, theta)
    """
    start_node = HybridAStarNode(*start)
    goal_node = HybridAStarNode(*goal)

    # 离散化分辨率
    xy_resolution = 0.2 # 比代价地图粗
    theta_resolution = np.radians(15)

    # 离散化状态空间
    def discretize(x, y, theta):
        dx = int(x / xy_resolution)
        dy = int(y / xy_resolution)
        dt = int(theta / theta_resolution)
        return (dx, dy, dt)

    start_node.g = 0.0
    start_node.f = self._heuristic(start, goal)

    open_heap = [(start_node.f, id(start_node), start_node)]
    open_dict = {discretize(*start): start_node}
    closed_set = set()

    while open_heap:
        _, _, current = heapq.heappop(open_heap)
        current_key = discretize(current.x, current.y, current.theta)

        if current_key in closed_set:
            continue

        # 检查是否到达目标
        dist_to_goal = np.sqrt((current.x - goal[0])**2 +
                                (current.y - goal[1])**2)
        if dist_to_goal < 0.5:
            # 使用 Reeds-Shepp 曲线连接
            return self._reconstruct_path(current)

        closed_set.add(current_key)

        # 扩展运动基元
        for primitive in self.motion_primitives:
            new_x, new_y, new_theta = self._simulate_motion(
                current.x, current.y, current.theta, primitive
            )
            new_key = discretize(new_x, new_y, new_theta)
            if new_key not in closed_set:
                new_node = HybridAStarNode(new_x, new_y, new_theta)
                new_node.g = current.g + primitive.g
                new_node.f = new_node.g + self._heuristic(new_x, new_y, new_theta)
                open_heap.append(new_node)
                open_dict[new_key] = new_node
    
```

```

    )

    # 碰撞检测
    if not self._is_collision_free(new_x, new_y):
        continue

    new_key = discretize(new_x, new_y, new_theta)

    if new_key in closed_set:
        continue

    # 计算代价
    motion_cost = abs(primitive['v']) * primitive['dt']
    steering_penalty = abs(primitive['steer']) * 0.5
    step_cost = motion_cost + steering_penalty

    new_g = current.g + step_cost

    if new_key not in open_dict or new_g < open_dict[new_key].g:
        new_node = HybridAStarNode(new_x, new_y, new_theta)
        new_node.g = new_g
        new_node.f = new_g + self._heuristic(
            (new_x, new_y), (goal[0], goal[1])
        )
        new_node.parent = current

        open_dict[new_key] = new_node
        heapq.heappush(open_heap, (new_node.f, id(new_node), new_node))

    return []

def _heuristic(self, a: tuple, b: tuple) -> float:
    """启发式函数：欧氏距离"""
    return np.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)

def _reconstruct_path(self, node: HybridAStarNode) -> list:
    """重建路径"""
    path = []
    current = node
    while current:
        path.append((current.x, current.y, current.theta))
        current = current.parent
    path.reverse()
    return path

```

18.4.3 Nav2 规划器配置

```

# 规划器配置
planner_server:
  ros__parameters:
    planner_plugins: ["GridBased"]

  GridBased:
    plugin: "nav2_navfn_planner/NavfnPlanner"
    tolerance: 0.5          # 目标容忍距离
    use_astar: true         # 使用 A*加速

```

```

allow_unknown: false          # 不允许未知区域

# Smac Planner Hybrid 可选配置
# GridBased:
#   plugin: "nav2_smac_planner/SmacPlannerHybrid"
#   tolerance: 0.5
#   max_planning_time: 2.0      # 最大规划时间(s)
#   minimum_turning_radius: 0.2 # 最小转弯半径(m)
#   reverse_penalty: 2.0        # 后退惩罚
#   change_penalty: 0.5         # 方向变化惩罚
#   non_straight_penalty: 1.1   # 非直线惩罚

```

18.4.4 规划器切换

```

# 查看当前使用的规划器
ros2 param get /planner_server planner_plugins

# 动态切换规划器
ros2 param set /planner_server planner_plugins "['SmacPlanner']"

# 重新配置后需要重新规划

```

18.5 路径优化

18.5.1 路径平滑

路径平滑去除路径中的锐角转折：

```

import numpy as np
from scipy import interpolate

class PathSmoother:
    """路径平滑器"""
    def __init__(self, weight_data: float = 0.5,
                  weight_smooth: float = 0.3,
                  weight_length: float = 0.2):
        self.weight_data = weight_data
        self.weight_smooth = weight_smooth
        self.weight_length = weight_length

    def smooth(self, path: np.ndarray,
               num_iterations: int = 100) -> np.ndarray:
        """
        梯度下降路径平滑

        参数:
            path: (N, 2) 原始路径点
            num_iterations: 迭代次数
        """
        new_path = path.copy()
        n = len(path)

```

```

for _ in range(num_iterations):
    for i in range(1, n - 1):
        # 保持接近原始点
        d_data = self.weight_data * (path[i] - new_path[i])

        # 平滑项 (与邻居的差距)
        d_smooth = self.weight_smooth * (
            new_path[i-1] + new_path[i+1] - 2 * new_path[i]
        )

        # 长度约束 (均匀间距)
        d_length = self.weight_length * (
            new_path[i-1] - 2 * new_path[i] + new_path[i+1]
        )

        new_path[i] += d_data + d_smooth + d_length

    return new_path

def b_spline_smooth(self, path: np.ndarray,
                    num_points: int = 100) -> np.ndarray:
    """B 样条平滑"""
    n = len(path)
    t = np.linspace(0, 1, n)
    t_new = np.linspace(0, 1, num_points)

    # B 样条插值
    tck, _ = interpolate.splprep([path[:, 0], path[:, 1]], s=0)
    smooth_points = interpolate.splev(t_new, tck)

    return np.column_stack(smooth_points)

```

18.5.2 路径修剪

```

class PathOptimizer:
    """路径优化器"""
    def __init__(self):
        pass

    def remove_redundant_points(self, path: np.ndarray,
                               epsilon: float = 0.1) -> np.ndarray:
        """
        移除冗余路径点 (Douglas-Peucker 算法)
        """
        if len(path) < 3:
            return path

        # 找最远点
        start, end = path[0], path[-1]
        line_vec = end - start
        line_len = np.linalg.norm(line_vec)

        if line_len == 0:
            return np.array([start, end])

        line_unit = line_vec / line_len

```

```

max_dist = 0
max_idx = 0

for i in range(1, len(path) - 1):
    vec = path[i] - start
    proj_len = np.dot(vec, line_unit)
    proj_point = start + proj_len * line_unit
    dist = np.linalg.norm(path[i] - proj_point)

    if dist > max_dist:
        max_dist = dist
        max_idx = i

if max_dist > epsilon:
    # 递归
    left = self.remove_redundant_points(path[:max_idx + 1], epsilon)
    right = self.remove_redundant_points(path[max_idx:], epsilon)
    return np.vstack((left[:-1], right))
else:
    return np.array([start, end])

def interpolate_path(self, path: np.ndarray,
                    step_size: float = 0.1) -> np.ndarray:
    """路径插值 (均匀间距) """
    if len(path) < 2:
        return path

    new_path = [path[0]]
    remaining = 0.0

    for i in range(len(path) - 1):
        segment = path[i+1] - path[i]
        seg_len = np.linalg.norm(segment)

        if seg_len == 0:
            continue

        seg_unit = segment / seg_len
        pos = path[i]

        while remaining < seg_len:
            pos = pos + seg_unit * step_size
            new_path.append(pos.copy())
            remaining += step_size

        remaining -= seg_len

    new_path.append(path[-1])
    return np.array(new_path)

```

18.5.3 代价感知路径优化

```

class CostAwarePathOptimizer:
    """代价感知路径优化"""

    def __init__(self, costmap: np.ndarray, resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution

```

```

def optimize_path(self, path: np.ndarray,
                  safety_margin: float = 0.3) -> np.ndarray:
    """
    调整路径点，使其远离高代价区域
    """
    optimized = path.copy()

    for iteration in range(50):
        max_change = 0

        for i in range(1, len(path) - 1):
            # 获取当前点的代价
            cost = self._get_cost(path[i])

            # 如果处于高代价区域，向外推
            if cost > 50:
                # 计算法线方向
                prev_vec = path[i] - path[i-1]
                next_vec = path[i+1] - path[i]

                # 法线 = 垂直路径方向
                tangent = (next_vec + prev_vec) / 2
                if np.linalg.norm(tangent) > 0:
                    normal = np.array([-tangent[1], tangent[0]])
                    normal = normal / np.linalg.norm(normal)
                else:
                    normal = np.zeros(2)

                # 朝低代价方向移动
                push_force = min(cost / 100.0, 0.5) * 0.05
                optimized[i] += normal * push_force

                change = np.linalg.norm(optimized[i] - path[i])
                max_change = max(max_change, change)

        if max_change < 0.001:
            break

    return optimized

def _get_cost(self, point: np.ndarray) -> float:
    """获取栅格代价值"""
    gx = int(point[1] / self.resolution)
    gy = int(point[0] / self.resolution)

    if (0 <= gx < self.costmap.shape[0] and
        0 <= gy < self.costmap.shape[1]):
        return float(self.costmap[gx, gy])
    return 255

```

18.6 实战案例

18.6.1 路径规划在 Nav2 中的完整流程

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Path, OccupancyGrid
from geometry_msgs.msg import PoseStamped
import numpy as np

class GlobalPathPlannerDemo(Node):
    """全局路径规划演示节点"""
    def __init__(self):
        super().__init__('global_planner_demo')

        self.costmap_sub = self.create_subscription(
            OccupancyGrid, '/global_costmap/costmap',
            self.costmap_callback, 10)
        self.path_pub = self.create_publisher(Path, '/planned_path', 10)

        self.costmap = None
        self.planner = None

    def costmap_callback(self, msg: OccupancyGrid):
        """接收代价地图并执行规划"""
        if self.costmap is None:
            h, w = msg.info.height, msg.info.width
            data = np.array(msg.data).reshape(h, w)
            self.costmap = data
            self.planner = AStarPlanner(self.costmap, msg.info.resolution)

            # 示例规划
            start = (0.0, 0.0)
            goal = (3.0, 2.0)

            path = self.planner.plan(start, goal)

            if path:
                self.publish_path(path, msg.header.frame_id)
                self.get_logger().info(f'路径规划成功: {len(path)}点')

    def publish_path(self, path_world: list, frame_id: str):
        """发布规划路径"""
        msg = Path()
        msg.header.frame_id = frame_id
        msg.header.stamp = self.get_clock().now().to_msg()

        for x, y in path_world:
            pose = PoseStamped()
            pose.header = msg.header
            pose.pose.position.x = x
            pose.pose.position.y = y
            pose.pose.orientation.w = 1.0
            msg.poses.append(pose)
```

```
self.path_pub.publish(msg)
```

18.6.2 命令行调试

```
# 查看规划路径
ros2 topic echo /plan --once

# 查看代价地图
ros2 topic echo /global_costmap/costmap --once

# 查看当前规划器参数
ros2 param describe /planner_server GridBased.plugin

# 设置规划容忍度
ros2 param set /planner_server GridBased.tolerance 0.3
```

18.6.3 常见问题

问题 1：路径规划超时

- 降低代价地图分辨率
- 减小搜索范围
- 使用更高效的规划器

问题 2：路径穿过障碍物

- 膨胀半径过小
- 检查代价地图是否正确更新
- 增加规划容忍度

问题 3：路径不平滑

- 使用路径平滑器后处理
- 使用 Hybrid-A*规划器
- 降低代价地图分辨率

课后练习

281. **原理题:** 比较 Dijkstra 算法和 A*算法的异同, 说明启发式函数 $H(n)$ 如何影响搜索效率和路径质量。

282. **编程题:** 实现 A* 路径规划算法, 在给定代价地图上搜索从起点到终点的最短路径, 并可视化路径。
283. **分析题:** 分析 Hybrid-A 与标准 A 的区别, 说明为什么非全向机器人需要 Hybrid-A* 规划器。
284. **配置题:** 在 Nav2 中配置 Smac Planner Hybrid, 调整最小转弯半径、后退惩罚等参数以适应差速机器人。
285. **操作题:** 在仿真环境中使用不同的全局规划器进行导航, 对比各规划器生成路径的平滑度和计算时间。
286. **设计题:** 设计一个三层路径规划框架: 上层 (全局搜索) 使用 A 在中分辨率地图规划, 中层使用 Hybrid-A 细化路径, 下层使用路径平滑器后处理。

实验 18:

第 18 章 实验：MoveIt2 规划场景、避障与抓取放置

****对应理论章节****: 第 28 章《笛卡尔路径与避障》、第 29 章《抓取与放置实验》 ****实验课时****: 4 课时 ****实验代码****: `src/lab\_code/ch18\_lab/moveit\_pick\_place\_lab/`

实验目标

- 掌握笛卡尔空间直线路径规划方法
- 学会使用 `plan_cartesian_path` 进行连续路径规划
- 理解碰撞检测与避障原理
- 能够在规划场景中添加障碍物并验证避障效果
- 掌握 MoveItPy 中物体的 `attach` 和 `detach` 操作
- 实现完整的 Pick-and-Place 流程
- 理解规划场景中的碰撞物体管理
- 学会使用 `PlanningSceneMonitor` 进行场景同步

实验环境

- ROS 2 Jazzy + MoveIt2
- RViz2 + MotionPlanning 插件

MoveIt 前置依赖（统一说明）：本章实验开始前，需先启动 xArm 仿真与 MoveIt：

```
# 方式 1: 纯 MoveIt + RViz 仿真 (不含 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py

# 方式 2: 含 Gazebo 的完整仿真 (本章参考代码 README 使用的命令)
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

默认使用上述 `xarm_ros2_arm_only` 启动命令。若按第 17 章 17.1 节自行生成配置包，请使用该包的名称及其生成的 `launch` 文件。

参考代码说明

src/lab_code/ch18_lab/moveit_pick_place_lab/ (ament_python 包) 提供以下程序：

程序	功能
`beeline_demo.py`	笛卡尔直线路径规划。沿三角形路径的三个顶点做笛卡尔直线运动
`obstacles_demo.py`	障碍物避障演示。在规划场景中添加桌子、球体和长方体，观察机械臂避障运动
`attach_object_demo.py`	物体附着与分离。将工具附着到末端执行器，执行抓取放置
`pick_place_demo.py`	抓取-放置 (Pick & Place) 全流程综合演示
`target_publisher.py`	3D 目标位姿发布者。发布正弦运动的目标点 (`PoseStamped` 到 `/target_pose`) , 可在 RViz 中可视化

构建：

```
cd <工作区根目录>
colcon build --symlink-install --packages-select moveit_pick_place_lab
source install/setup.bash
```

18.1 笛卡尔路径与避障

18.1.1 运行参考代码

启动 xArm 仿真（见前置说明）后，另开终端运行：

```
ros2 run moveit_pick_place_lab beeline_demo      # 直线路径规划
ros2 run moveit_pick_place_lab obstacles_demo     # 避障演示（按 Enter 逐步执行）
ros2 run moveit_pick_place_lab target_publisher   # 3D 目标点发布
```

target_publisher.py 支持参数调整：

```
python3 target_publisher.py --ros-args \
  -p rate:=20 \
  -p speed:=1.5 \
  -p target_frame:=base_link
```

可在 RViz 中添加 Marker 和 PoseStamped 显示查看动态移动的目标球体。

18.1.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch18_cartesian --build-type ament_python --dependencies rclpy moveit
cd ch18_cartesian
mkdir -p ch18_cartesian
```

18.1.3 编写直线路径规划程序

创建 ch18_cartesian/line_path.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from copy import deepcopy
import time

class LinePathPlanner(Node):
    def __init__(self):
        super().__init__('line_path_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenror_link'
        )
        self.reference_frame = 'base_link'
        self.end_effector = 'gripper_cenror_link'
        self.arm.set_pose_reference_frame(self.reference_frame)
        self.get_logger().info('直线路径规划器已启动')

    def get_current_pose(self):
        return self.arm.get_current_pose(self.end_effector).pose

    def move_to_pose(self, target_pose):
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose, self.end_effector)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

    def plan_cartesian_line(self, waypoints):
        self.arm.set_start_state_to_current_state()
        fraction = 0.0
        max_tries = 100
        attempts = 0

        while fraction < 1.0 and attempts < max_tries:
            plan_result = self.arm.plan_cartesian_path(
                waypoints, 0.01, 0.0, True
            )
            if plan_result:
                fraction = plan_result.fraction
                self.get_logger().info(
```

```

        f'尝试 {attempts+1}, 覆盖率: {fraction*100:.1f}%'
    )
    attempts += 1

    if fraction >= 1.0:
        self.get_logger().info('笛卡尔路径规划成功!')
        self.arm.execute(plan_result.trajectory)
        return True
    else:
        self.get_logger().error(f'规划失败, 最大覆盖率: {fraction*100:.1f}%')
        return False

def run_straight_line(self):
    self.get_logger().info('=== 直线路径演示 ===')

    start_pose = Pose()
    start_pose.position.x = 0.3
    start_pose.position.y = 0.0
    start_pose.position.z = 0.45
    start_pose.orientation.w = 1.0

    self.get_logger().info('移动到起始点')
    if not self.move_to_pose(start_pose):
        self.get_logger().error('起始点不可达')
        return
    time.sleep(1)

    waypoints = []
    current = self.get_current_pose()

    wpose = deepcopy(current)
    wpose.position.z += 0.1
    waypoints.append(deepcopy(wpose))

    wpose.position.y += 0.15
    waypoints.append(deepcopy(wpose))

    wpose.position.z -= 0.1
    waypoints.append(deepcopy(wpose))

    wpose.position.y -= 0.15
    waypoints.append(deepcopy(wpose))

    self.get_logger().info('规划直线路径...')
    self.plan_cartesian_line(waypoints)

def main(args=None):
    rclpy.init(args=args)
    node = LinePathPlanner()
    node.run_straight_line()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.1.4 编写方形路径规划程序

创建 ch18_cartesian/square_path.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from copy import deepcopy
import time

class SquarePathPlanner(Node):
    def __init__(self):
        super().__init__('square_path_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.arm.set_pose_reference_frame('base_link')
        self.end_effector = 'gripper_cenotor_link'

    def run_square(self, side_length=0.12, height=0.35):
        self.get_logger().info(f'画方形: 边长{side_length}m, 高度{height}m')

        start_pose = Pose()
        start_pose.position.x = 0.3
        start_pose.position.y = -side_length/2
        start_pose.position.z = height
        start_pose.orientation.w = 1.0

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(start_pose, self.end_effector)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            time.sleep(1)

        waypoints = []
        current = self.arm.get_current_pose(self.end_effector).pose

        corners = [
            (0, side_length, 0),
            (side_length, 0, 0),
            (0, -side_length, 0),
            (-side_length, 0, 0),
        ]

        temp = deepcopy(current)
        for dx, dy, dz in corners:
            temp.position.x += dx
            temp.position.y += dy
            temp.position.z += dz
            waypoints.append(deepcopy(temp))
            for _ in range(10):
                interp = deepcopy(temp)
                waypoints.append(interp)

        self.arm.set_start_state_to_current_state()
```

```

        plan_result = self.arm.plan_cartesian_path(
            waypoints, 0.005, 0.0, True
        )
        if plan_result and plan_result.fraction >= 1.0:
            self.get_logger().info('方形路径规划成功!')
            self.arm.execute(plan_result.trajectory)
        else:
            fraction = plan_result.fraction if plan_result else 0
            self.get_logger().error(f'方形路径失败: {fraction*100:.1f}%')

        time.sleep(1)
        home = [0.0]*6
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(home)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

def main(args=None):
    rclpy.init(args=args)
    node = SquarePathPlanner()
    node.run_square()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.1.5 编写避障规划程序

创建 ch18_cartesian/obstacle_avoidance.py (参考实现:

src/lab_code/ch18_lab/moveit_pick_place_lab/obstacles_demo.py) :

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import PoseStamped
from moveit_msgs.msg import CollisionObject, ObjectColor, PlanningScene
from shape_msgs.msg import SolidPrimitive
from tf_transformations import quaternion_from_euler
import time
import math

class ObstacleAvoidance(Node):
    def __init__(self):
        super().__init__('obstacle_avoidance')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.planning_scene_monitor = self.moveit.get_planning_scene_monitor()
        self.scene_pub = self.create_publisher(PlanningScene, 'planning_scene', 10)
        time.sleep(1)

        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.arm.set_pose_reference_frame('base_link')
        self.arm.set_goal_position_tolerance(0.02)
        self.arm.set_goal_orientation_tolerance(0.03)

```

```

def add_box(self, object_id, x, y, z, size, frame_id='base_link'):
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD
    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = size
    co.primitives = [primitive]
    pose = PoseStamped()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def add_cylinder(self, object_id, x, y, z, height, radius, frame_id='base_link'):
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD
    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.CYLINDER
    primitive.dimensions = [height, radius]
    co.primitives = [primitive]
    pose = PoseStamped()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def remove_object(self, object_id):
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    self.planning_scene_monitor.process_collision_object(co)

def wait_for_object(self, obstacle_name, timeout=4):
    start = self.get_clock().now()
    while (self.get_clock().now() - start).nanoseconds / 1e9 < timeout:
        known = self.planning_scene_monitor.get_known_object_names()
        if obstacle_name in known:
            return True
        time.sleep(0.1)
    return False

def run(self):
    self.get_logger().info('=== 避障规划演示 ===')
    input("按 Enter 添加桌子...")
    self.add_box('table', 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    self.wait_for_object('table')

    input("按 Enter 添加障碍物 1 (圆柱)...")
    self.add_cylinder('obstacle1', 0.3, 0.0, 0.15, 0.3, 0.05)
    self.wait_for_object('obstacle1')

```



```

input("按 Enter 添加障碍物 2 (方块)...")
self.add_box('obstacle2', 0.3, -0.2, 0.15, [0.15, 0.05, 0.3])
self.wait_for_object('obstacle2')

input("按 Enter 添加障碍物 3 (方块)...")
self.add_box('obstacle3', 0.3, 0.2, 0.15, [0.15, 0.05, 0.3])
self.wait_for_object('obstacle3')

input("按 Enter 改变障碍物颜色...")
colors = [
    (1.0, 0.0, 0.0, 1.0),
    (0.0, 1.0, 0.0, 1.0),
    (0.0, 0.0, 1.0, 1.0),
]
p = PlanningScene()
p.is_diff = True
for i, obj_id in enumerate(['table', 'obstacle1', 'obstacle2']):
    oc = ObjectColor()
    oc.id = obj_id
    oc.color.r, oc.color.g, oc.color.b, oc.color.a = colors[i]
    p.object_colors.append(oc)
self.scene_pub.publish(p)
time.sleep(1)

input("按 Enter 执行避障规划...")
self.arm.set_start_state_to_current_state()

target_pose = PoseStamped()
target_pose.header.frame_id = 'base_link'
target_pose.pose.position.x = 0.25
target_pose.pose.position.y = -0.4
target_pose.pose.position.z = 0.25
q = quaternion_from_euler(0, 0, -math.pi/4)
target_pose.pose.orientation.x = q[0]
target_pose.pose.orientation.y = q[1]
target_pose.pose.orientation.z = q[2]
target_pose.pose.orientation.w = q[3]

self.arm.set_pose_target(target_pose.pose, 'gripper_cenor_link')
plan_result = self.arm.plan()
if plan_result:
    self.arm.execute(plan_result.trajectory)
    self.get_logger().info('避障规划成功! 路径已避开障碍物')
else:
    self.get_logger().error('避障规划失败')

input("按 Enter 移除所有障碍物...")
for obj_id in ['table', 'obstacle1', 'obstacle2', 'obstacle3']:
    self.remove_object(obj_id)
time.sleep(1)

self.get_logger().info('避障演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleAvoidance()

```

```

node.run()
node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.1.6 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'line_path = ch18_cartesian.line_path:main',
        'square_path = ch18_cartesian.square_path:main',
        'obstacle_avoidance = ch18_cartesian.obstacle_avoidance:main',
    ],
},
},
cd ~/ros2_arm_ws
colcon build --packages-select ch18_cartesian
source install/setup.bash

# 直线路径
ros2 run ch18_cartesian line_path

# 方形路径
ros2 run ch18_cartesian square_path

# 避障规划（交互式，按 Enter 逐步执行）
ros2 run ch18_cartesian obstacle_avoidance

```

18.1.7 对比笛卡尔与非笛卡尔模式

创建 ch18_cartesian/comparison.py:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from copy import deepcopy
import time

class Comparison(Node):
    def __init__(self):
        super().__init__('comparison')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.arm.set_pose_reference_frame('base_link')
        self.end_effector = 'gripper_cenotor_link'

    def move_via_poses(self, waypoints, cartesian=False):
        if cartesian:
            self.arm.set_start_state_to_current_state()
            plan = self.arm.plan_cartesian_path(waypoints, 0.01, 0.0, True)
            if plan and plan.fraction >= 1.0:
                self.arm.execute(plan.trajectory)
            return

```

```

else:
    for wp in waypoints:
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(wp, self.end_effector)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        time.sleep(1)

def run(self):
    start_pose = Pose()
    start_pose.position.x = 0.3
    start_pose.position.y = 0.0
    start_pose.position.z = 0.4
    start_pose.orientation.w = 1.0

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(start_pose, self.end_effector)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    current = self.arm.get_current_pose(self.end_effector).pose
    waypoints = []
    w = deepcopy(current)
    w.position.x += 0.1; w.position.y += 0.1; w.position.z -= 0.05
    waypoints.append(deepcopy(w))
    w.position.x -= 0.2; w.position.y -= 0.1; w.position.z += 0.1
    waypoints.append(deepcopy(w))
    w.position.x += 0.1
    waypoints.append(deepcopy(w))

    self.get_logger().info('=== 非笛卡尔模式 (逐点运动) ===')
    self.move_via_poses(waypoints, cartesian=False)
    time.sleep(1)

    self.arm.set_start_state_to_current_state()
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    self.get_logger().info('=== 笛卡尔模式 (直线路径) ===')
    self.move_via_poses(waypoints, cartesian=True)

def main(args=None):
    rclpy.init(args=args)
    node = Comparison()
    node.run()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.2 抓取与放置

18.2.1 运行参考代码

启动 xArm 仿真（见前置说明）后，另开终端运行：

```
ros2 run moveit_pick_place_lab target_publisher    # 先发布目标
ros2 run moveit_pick_place_lab pick_place_demo    # 抓取-放置全流程
ros2 run moveit_pick_place_lab attach_object_demo # 物体附着/分离
```

pick_place_demo 依次完成接近→抓取→提升→移动→放置→归位，终端输出各阶段规划结果；

RViz/Gazebo 中可见目标物体被拾起与放下。

18.2.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch18_pick_place --build-type ament_python --dependencies rclpy moveit
cd ch18_pick_place
mkdir -p ch18_pick_place
```

18.2.3 编写基础的 Pick-and-Place 程序

创建 ch18_pick_place/pick_place_demo.py:

```
import rclpy
from rclpy.node import Node
from moveit.Planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import PoseStamped, Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from tf_transformations import quaternion_from_euler
from copy import deepcopy
import time
import math

class PickPlaceDemo(Node):
    ARM_GROUP = 'xarm_group'
    GRIPPER_GROUP = 'gripper_group'
    GRIPPER_FRAME = 'gripper_cenotr_link'
    GRIPPER_OPEN = [0.65, 0.65]
    GRIPPER_CLOSE = [0.0, 0.0]
    REFERENCE_FRAME = 'base_link'

    def __init__(self):
        super().__init__('pick_place_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, self.ARM_GROUP, self.GRIPPER_FRAME
        )
        self.gripper = PlanningComponent(
            self.moveit, self.GRIPPER_GROUP, self.GRIPPER_FRAME
        )
        self.arm.set_pose_reference_frame(self.REFERENCE_FRAME)
```

```

        time.sleep(1)
        self.get_logger().info('Pick and Place Demo 已启动')

    def add_collision_box(self, object_id, x, y, z, size):
        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = self.REFERENCE_FRAME
        co.operation = CollisionObject.ADD
        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = size
        co.primitives = [primitive]
        pose = Pose()
        pose.position.x = x
        pose.position.y = y
        pose.position.z = z
        pose.orientation.w = 1.0
        co.primitive_poses = [pose]
        self.psm.process_collision_object(co)

    def remove_object(self, object_id):
        co = CollisionObject()
        co.id = object_id
        co.operation = CollisionObject.REMOVE
        self.psm.process_collision_object(co)

    def attach_object(self, object_id):
        aco = AttachedCollisionObject()
        aco.link_name = self.GRIPPER_FRAME
        aco.object.id = object_id
        aco.object.operation = CollisionObject.ADD
        aco.touch_links = ['gripper_1_link', 'gripper_2_link']
        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'附着物体: {object_id}')

    def detach_object(self, object_id):
        aco = AttachedCollisionObject()
        aco.link_name = self.GRIPPER_FRAME
        aco.object.id = object_id
        aco.object.operation = CollisionObject.REMOVE
        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'分离物体: {object_id}')

    def move_gripper(self, position):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target(position)
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def move_arm_to_joints(self, joints):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joints)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

```

```

def move_arm_to_pose(self, target_pose):
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose, self.GRIPPER_FRAME)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
        return True
    return False

def run_pick_and_place(self):
    self.get_logger().info('=== Pick and Place 演示 ===')

    table_id = 'table'
    target_id = 'target_cube'

    self.get_logger().info('添加桌子和目标物体到规划场景')
    self.add_collision_box(table_id, 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    time.sleep(0.5)

    cube_size = [0.04, 0.04, 0.04]
    cube_x, cube_y, cube_z = 0.35, 0.0, 0.04
    self.add_collision_box(target_id, cube_x, cube_y, cube_z, cube_size)
    time.sleep(0.5)

    home = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    pre_grasp = [0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
    grasp = [0.3, -0.2, 0.1, 0.0, 0.0, 0.0]
    lift = [0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
    pre_place = [-0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
    place = [-0.3, -0.2, 0.1, 0.0, 0.0, 0.0]

    self.move_arm_to_joints(home)
    time.sleep(1)

    self.get_logger().info('[1] 移动到手爪预张开位置')
    self.move_gripper(self.GRIPPER_OPEN)
    time.sleep(0.5)

    self.get_logger().info('[2] 移动到预抓取位姿')
    self.move_arm_to_joints(pre_grasp)
    time.sleep(1)

    self.get_logger().info('[3] 抓取物体')
    self.remove_object(target_id)
    self.move_arm_to_joints(grasp)
    time.sleep(1)

    self.get_logger().info('[4] 闭合手爪')
    self.move_gripper(self.GRIPPER_CLOSE)
    time.sleep(0.5)
    self.attach_object(target_id)
    time.sleep(0.5)

    self.get_logger().info('[5] 提起物体')
    self.move_arm_to_joints(lift)
    time.sleep(1)

```

```

        self.get_logger().info('[6] 移动到放置区域')
        self.move_arm_to_joints(pre_place)
        time.sleep(1)

        self.get_logger().info('[7] 放置物体')
        self.move_arm_to_joints(place)
        time.sleep(1)

        self.get_logger().info('[8] 张开手爪释放物体')
        self.detach_object(target_id)
        place_x, place_y, place_z = -0.3, -0.2, 0.04
        self.add_collision_box(
            target_id + '_placed', place_x, place_y, place_z, cube_size
        )
        time.sleep(0.5)
        self.move_gripper(self.GRIPPER_OPEN)
        time.sleep(1)

        self.get_logger().info('[9] 回到 Home')
        self.move_arm_to_joints(home)

        self.get_logger().info('清理场景')
        self.remove_object(table_id)
        self.remove_object(target_id + '_placed')

        self.get_logger().info('Pick and Place 完成!')

def main(args=None):
    rclpy.init(args=args)
    node = PickPlaceDemo()
    node.run_pick_and_place()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.2.4 编写带笛卡尔路径的抓取程序

创建 `ch18_pick_place/cartesian_pick_place.py`:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from copy import deepcopy
import time
import numpy as np

class CartesianPickPlace(Node):
    def __init__(self):
        super().__init__('cartesian_pick_place')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(

```

```

        self.moveit, 'xarm_group', 'gripper_centor_link'
    )
    self.gripper = PlanningComponent(
        self.moveit, 'gripper_group', 'gripper_centor_link'
    )
    self.arm.set_pose_reference_frame('base_link')

def add_box(self, obj_id, x, y, z, size):
    co = CollisionObject()
    co.id = obj_id
    co.header.frame_id = 'base_link'
    co.operation = CollisionObject.ADD
    p = SolidPrimitive()
    p.type = SolidPrimitive.BOX
    p.dimensions = size
    co.primitives = [p]
    pose = Pose()
    pose.position.x = x; pose.position.y = y; pose.position.z = z
    pose.orientation.w = 1.0
    co.primitive_poses = [pose]
    self.psm.process_collision_object(co)

def remove_obj(self, obj_id):
    co = CollisionObject()
    co.id = obj_id
    co.operation = CollisionObject.REMOVE
    self.psm.process_collision_object(co)

def attach(self, obj_id):
    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_centor_link'
    aco.object.id = obj_id
    aco.object.operation = CollisionObject.ADD
    aco.touch_links = ['gripper_1_link', 'gripper_2_link']
    self.psm.process_attached_collision_object(aco)

def detach(self, obj_id):
    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_centor_link'
    aco.object.id = obj_id
    aco.object.operation = CollisionObject.REMOVE
    self.psm.process_attached_collision_object(aco)

def run_cartesian_pick_place(self):
    self.get_logger().info('=== 笛卡尔路径 Pick and Place ===')

    target_id = 'target'
    table_id = 'table'

    self.add_box(table_id, 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    self.add_box(target_id, 0.35, 0.0, 0.04, [0.04, 0.04, 0.04])

    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan: self.gripper.execute(plan.trajectory)
    time.sleep(0.5)

    self.arm.set_start_state_to_current_state()

```



```

home_pose = Pose()
home_pose.position.x = 0.3
home_pose.position.y = 0.0
home_pose.position.z = 0.4
home_pose.orientation.w = 1.0
self.arm.set_pose_target(home_pose, 'gripper_cenior_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(0.5)

self.remove_obj(target_id)

self.get_logger().info('笛卡尔下降抓取')
current = self.arm.get_current_pose('gripper_cenior_link').pose
waypoints = []
for i in range(20):
    wp = deepcopy(current)
    wp.position.z -= 0.01 * i
    waypoints.append(wp)

self.arm.set_start_state_to_current_state()
result = self.arm.plan_cartesian_path(waypoints, 0.005, 0.0, True)
if result and result.fraction >= 1.0:
    self.arm.execute(result.trajectory)
time.sleep(0.5)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
self.attach(target_id)
time.sleep(0.5)

self.get_logger().info('笛卡尔提升')
current = self.arm.get_current_pose('gripper_cenior_link').pose
lift_waypoints = []
for i in range(15):
    wp = deepcopy(current)
    wp.position.z += 0.01 * i
    lift_waypoints.append(wp)

self.arm.set_start_state_to_current_state()
result = self.arm.plan_cartesian_path(lift_waypoints, 0.005, 0.0, True)
if result and result.fraction >= 1.0:
    self.arm.execute(result.trajectory)
time.sleep(0.5)

self.get_logger().info('移动到放置区域')
place_pose = Pose()
place_pose.position.x = 0.2
place_pose.position.y = -0.25
place_pose.position.z = 0.3
place_pose.orientation.w = 1.0

self.arm.set_start_state_to_current_state()
plan = self.arm.plan()
self.arm.set_pose_target(place_pose, 'gripper_cenior_link')
plan = self.arm.plan()

```

```

        if plan: self.arm.execute(plan.trajectory)
        time.sleep(0.5)

        self.get_logger().info('笛卡尔下降放置')
        current = self.arm.get_current_pose('gripper_cenor_link').pose
        place_down = []
        for i in range(18):
            wp = deepcopy(current)
            wp.position.z -= 0.01 * i
            place_down.append(wp)

        self.arm.set_start_state_to_current_state()
        result = self.arm.plan_cartesian_path(place_down, 0.005, 0.0, True)
        if result and result.fraction >= 1.0:
            self.arm.execute(result.trajectory)
        time.sleep(0.5)

        self.detach(target_id)
        self.add_box(target_id + '_placed', 0.2, -0.25, 0.04, [0.04]*3)
        time.sleep(0.5)

        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.65, 0.65])
        plan = self.gripper.plan()
        if plan: self.gripper.execute(plan.trajectory)
        time.sleep(0.5)

        self.arm.set_start_state_to_current_state()
        home_joints = [0.0]*6
        self.arm.set_joint_value_target(home_joints)
        plan = self.arm.plan()
        if plan: self.arm.execute(plan.trajectory)

        self.remove_obj(table_id)
        self.remove_obj(target_id + '_placed')
        self.get_logger().info('笛卡尔路径 Pick and Place 完成!')

def main(args=None):
    rclpy.init(args=args)
    node = CartesianPickPlace()
    node.run_cartesian_pick_place()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.2.5 编写抓取放置服务端

创建 ch18_pick_place/pick_place_server.py:

```

import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive

```

```

import time

class PickPlaceServer(Node):
    def __init__(self):
        super().__init__('pick_place_server')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenot_link'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper_group', 'gripper_cenot_link'
        )
        self.srv = self.create_service(Trigger, '/pick_place', self.callback)
        self.get_logger().info('Pick Place Server ready. Call /pick_place')

    def callback(self, request, response):
        self.get_logger().info('触发 Pick and Place')
        try:
            self.execute_pick_place()
            response.success = True
            response.message = 'Pick and Place succeeded'
        except Exception as e:
            response.success = False
            response.message = str(e)
        return response

    def execute_pick_place(self):
        home = [0.0]*6
        pre = [0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
        pick = [0.3, -0.2, 0.1, 0.0, 0.0, 0.0]
        place = [-0.3, -0.2, 0.1, 0.0, 0.0, 0.0]

        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(home)
        plan = self.arm.plan()
        if plan: self.arm.execute(plan.trajectory)
        time.sleep(0.5)

        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.65]*2)
        plan = self.gripper.plan()
        if plan: self.gripper.execute(plan.trajectory)
        time.sleep(0.5)

        for target in [pre, pick, place, home]:
            self.arm.set_start_state_to_current_state()
            self.arm.set_joint_value_target(target)
            plan = self.arm.plan()
            if plan: self.arm.execute(plan.trajectory)
            time.sleep(0.5)

def main(args=None):
    rclpy.init(args=args)
    node = PickPlaceServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':  
    main()
```

18.2.6 配置 setup.py 并编译运行

```
entry_points={  
    'console_scripts': [  
        'pick_place_demo = ch18_pick_place.pick_place_demo:main',  
        'cartesian_pick_place = ch18_pick_place.cartesian_pick_place:main',  
        'pick_place_server = ch18_pick_place.pick_place_server:main',  
    ],  
},  
cd ~/ros2_arm_ws  
colcon build --packages-select ch18_pick_place  
source install/setup.bash  
  
# 在 MoveIt2 演示环境运行的同时  
ros2 run ch18_pick_place pick_place_demo  
# 或  
ros2 run ch18_pick_place cartesian_pick_place  
# 或通过服务触发  
ros2 run ch18_pick_place pick_place_server  
ros2 service call /pick_place std_srvs/srv/Trigger
```

18.2.7 观察附着的效果

在 RViz2 中:

- 抓取前: 场景中桌面上有一个绿色方块
- 抓取时: 方块从场景中消失(被 remove)
- 闭合手爪后: 方块附着到 gripper_centor_link, 随机机械臂运动
- 放置后: 方块重新出现在放置位置, 从末端分离

18.2.8 修改物体尺寸和位置

修改代码中的立方体尺寸和位置参数, 重新运行:

```
# 修改为目标物体在不同位置  
cube_x, cube_y, cube_z = 0.4, 0.1, 0.04  
# 或不同尺寸  
cube_size = [0.06, 0.06, 0.06]
```

实验结果与分析

- 笛卡尔路径规划使得机械臂末端沿直线轨迹运动, 轨迹在 RViz2 中显示为直线
- 非笛卡尔模式下, 各路径点之间是关节空间的插值, 末端走曲线而非直线
- 避障规划中, 添加障碍物后 MoveIt2 自动生成绕行路径

- eef_step 参数影响笛卡尔路径规划的精度和成功率
- AttachedCollisionObject 使得物体能跟随末端执行器移动
- 抓取流程: 预抓取→下降→闭合→attach→提升→移动→下降→detach→提升
- 完整的 Pick-and-Place 需要协调手臂和手爪的配合运动
- 规划场景中的物体管理对碰撞检测至关重要
- 参考代码 moveit_pick_place_lab 的
beeline_demo/obstacles_demo/pick_place_demo/attach_object_demo 与本章自实现程序——
对应, 可用于对照验证

思考题

287. plan_cartesian_path 中的 eef_step 参数如何影响路径规划?
288. 在障碍物密集的环境中, 如何提高避障规划的成功率?
289. 笛卡尔路径规划失败时, 如何通过降采样或分段规划来解决问题?
290. 怎样在 RViz2 中实时显示末端执行器的运动轨迹?
291. attach 和 detach 操作对碰撞检测有什么影响?
292. 如果抓取前后物体的位姿没有正确更新, 会有什么问题?
293. 如何实现多物体的连续抓取和放置?
294. 实际机器人抓取中, 如何通过力反馈确定抓取是否成功?

第 19 章 局部路径规划

学习目标

- 理解 DWA(动态窗口法)算法的原理和实现
- 掌握 Regulated Pure Pursuit 控制器的工作机制
- 熟悉局部路径规划的轨迹评价方法
- 能够在 Nav2 中配置 DWB 和 RPP 控制器
- 掌握局部规划参数调优方法

19.1 局部路径规划概述

19.1.1 问题定义

局部路径规划是在全局路径的引导下，根据传感器实时信息生成安全的速度指令，实现动态避障和路径跟踪。

与全局规划的区别：

特性	全局路径规划	局部路径规划
规划范围	整个地图	机器人周围（数米）
信息源	已知地图	实时传感器
更新频率	低 (1-2 Hz)	高 (10-50 Hz)
规划目标	最短路径	路径跟踪+避障
输出	路径点序列	速度指令 (v, ω)

局部规划的核心挑战：

- 实时性：需要在毫秒级生成控制指令
- 安全性：必须避开动态障碍物
- 平滑性：速度变化不能太剧烈
- 可达性：满足机器人运动学约束

19.1.2 Nav2 局部控制器

Nav2 支持多种局部控制器插件：

控制器	算法	特点	适用场景
DWB	动态窗口法 + 评分	功能最全，可定制	通用场景
Regulated Pure Pursuit	调节纯追踪	简单可靠	差速/阿克曼
MPPI	模型预测路径积分	采样+并行优化	高端动态场景
TEB	时间弹性带	考虑时间最优	复杂环境

19.2 DWA 动态窗口法

19.2.1 DWA 算法原理

DWA (Dynamic Window Approach) 在速度空间中采样，预测每条轨迹的未来状态，通过评分选择最优速度指令。

算法流程：

1. 生成速度采样窗口
 - 考虑最大/最小速度限制
 - 考虑加速度限制（动态窗口）
 - 考虑安全停止距离
2. 对每组速度（ v, ω ）模拟轨迹
 - 使用运动模型前向模拟
 - 预测未来 N 步的位姿
3. 评估每条轨迹
 - 方向评分（与目标对齐）
 - 障碍物评分（最小距离）
 - 速度评分（鼓励高速）
 - 路径评分（与全局路径对齐）
4. 选择最优速度
 - 加权求和各评分
 - 选择综合得分最高的速度

19.2.2 Python 实现

```
import numpy as np
from enum import Enum

class DWAPlaner:
    """DWA 动态窗口法规划器"""
    def __init__(self, config: dict = None):
        # 运动学约束
        self.max_speed = config.get('max_speed', 0.5)          # m/s
        self.min_speed = config.get('min_speed', 0.0)
        self.max_yaw_rate = config.get('max_yaw_rate', 1.0)    # rad/s
        self.max_accel = config.get('max_accel', 2.0)          # m/s²
        self.max_delta_yaw_rate = config.get('max_delta_yaw_rate', 3.0) # rad/s²

        # 仿真参数
        self.dt = config.get('dt', 0.1)                        # 时间步长(s)
        self.predict_time = config.get('predict_time', 2.0)    # 预测时长(s)

        # 速度分辨率
        self.v_resolution = config.get('v_resolution', 0.05)
        self.w_resolution = config.get('w_resolution', 0.1)

        # 评分权重
        self.alpha_heading = config.get('alpha_heading', 1.0)  # 目标方向
        self.alpha_dist = config.get('alpha_dist', 0.5)        # 障碍物距离
        self.alpha_velocity = config.get('alpha_velocity', 0.3) # 速度
        self.alpha_path = config.get('alpha_path', 2.0)        # 路径跟踪

    def plan(self, state: np.ndarray, goal: np.ndarray,
             obstacles: np.ndarray, global_path: np.ndarray = None) -> np.ndarray:
        """
        执行 DWA 规划

        参数:
            state: [x, y, theta, v, omega] 当前状态
            goal: [x, y] 局部目标点
            obstacles: (N, 2) 障碍物坐标
            global_path: (M, 2) 全局路径

        返回:
            [v, omega] 最优速度指令
        """
        # 1. 生成动态窗口
        v_min, v_max, w_min, w_max = self._calc_dynamic_window(state)

        # 2. 速度空间采样
        best_traj = None
        best_score = -float('inf')
        best_cmd = np.array([0.0, 0.0])

        for v in np.arange(v_min, v_max, self.v_resolution):
```



```

        for w in np.arange(w_min, w_max, self.w_resolution):
            # 3. 轨迹预测
            traj = self._predict_trajectory(state, v, w)

            # 4. 评分
            score = self._evaluate_trajectory(
                traj, goal, obstacles, global_path, v, w
            )

            if score > best_score:
                best_score = score
                best_cmd = np.array([v, w])
                best_traj = traj

    return best_cmd, best_traj

```

```

def _calc_dynamic_window(self, state: np.ndarray) -> tuple:
    """

```

计算动态窗口

考虑:

1. 速度极限: $[v_{\min}, v_{\max}]$, $[w_{\min}, w_{\max}]$
2. 加速度极限: 在 dt 内可达到的速度范围
3. 安全停止: 能及时停下的速度

"""

```

v = state[3]
w = state[4]

```

运动学极限

```

vs = [self.min_speed, self.max_speed,
      -self.max_yaw_rate, self.max_yaw_rate]

```

加速度极限

```

vd = [v - self.max_accel * self.dt,
      v + self.max_accel * self.dt,
      w - self.max_delta_yaw_rate * self.dt,
      w + self.max_delta_yaw_rate * self.dt]

```

取交集

```

v_min = max(vs[0], vd[0])
v_max = min(vs[1], vd[1])
w_min = max(vs[2], vd[2])
w_max = min(vs[3], vd[3])

```

```

return v_min, v_max, w_min, w_max

```

```

def _predict_trajectory(self, state: np.ndarray, v: float,
                       w: float) -> np.ndarray:
    """

```

前向模拟轨迹

参数:

state: $[x, y, \theta, v, \omega]$

v: 线速度

w: 角速度

```

    返回:
        traj: (N, 3) 轨迹点 [x, y, theta]
    """
    x, y, theta = state[0], state[1], state[2]
    traj = [(x, y, theta)]

    time = 0
    while time < self.predict_time:
        x += v * np.cos(theta) * self.dt
        y += v * np.sin(theta) * self.dt
        theta += w * self.dt
        time += self.dt
        traj.append((x, y, theta))

    return np.array(traj)

def _evaluate_trajectory(self, traj: np.ndarray, goal: np.ndarray,
                        obstacles: np.ndarray,
                        global_path: np.ndarray,
                        v: float, w: float) -> float:
    """
    轨迹评分

    评分项:
    1. 目标方向评分: 轨迹终点与目标方向的对齐程度
    2. 障碍物距离评分: 轨迹与最近障碍物的距离
    3. 速度评分: 鼓励高速运动
    4. 路径跟踪评分: 与全局路径的贴合度
    """
    final_x, final_y, final_theta = traj[-1]

    # 1. 目标方向评分
    angle_to_goal = np.arctan2(
        goal[1] - final_y, goal[0] - final_x
    )
    heading_diff = abs(final_theta - angle_to_goal)
    heading_diff = min(heading_diff, 2 * np.pi - heading_diff)
    heading_score = 1.0 - heading_diff / np.pi

    # 2. 障碍物距离评分
    min_obstacle_dist = float('inf')
    for traj_point in traj:
        for obs in obstacles:
            dist = np.linalg.norm(traj_point[:2] - obs)
            min_obstacle_dist = min(min_obstacle_dist, dist)

    if min_obstacle_dist < 0.2: # 碰撞
        return -float('inf')

    obstacle_score = min(1.0, min_obstacle_dist / 0.5)

    # 3. 速度评分
    speed_score = v / self.max_speed

```

```

# 4. 路径跟踪评分
path_score = 0
if global_path is not None and len(global_path) > 0:
    for traj_point in traj:
        min_dist = min(
            np.linalg.norm(traj_point[:2] - gp)
            for gp in global_path
        )
        path_score += 1.0 / (1.0 + min_dist)
    path_score /= len(traj)

# 加权综合评分
total_score = (
    self.alpha_heading * heading_score +
    self.alpha_dist * obstacle_score +
    self.alpha_velocity * speed_score +
    self.alpha_path * path_score
)

return total_score

```

19.2.3 DWA 评分机制详解

```

class DWAScorer:
    """DWA 评分器 - 各评分项详解"""
    def __init__(self):
        pass

    def heading_score(self, robot_pose, goal, trajectory):
        """
        目标方向评分

        评估轨迹终点方向与目标方向的偏差
        """
        # 轨迹终点
        end_pose = trajectory[-1]
        theta = end_pose[2]

        # 到目标的向量
        dx = goal[0] - end_pose[0]
        dy = goal[1] - end_pose[1]
        target_angle = np.arctan2(dy, dx)

        # 角度差
        error = abs(theta - target_angle)
        error = min(error, 2*np.pi - error)

        # 归一化
        return 1.0 - error / np.pi

    def obstacle_score(self, trajectory, obstacles,
                       robot_radius=0.22):
        """
        障碍物距离评分

```

```

    评估轨迹中所有点到障碍物的最小距离
    """
    min_dist = float('inf')

    for point in trajectory:
        px, py = point[0], point[1]
        for ox, oy in obstacles:
            dist = np.sqrt((px-ox)**2 + (py-oy)**2) - robot_radius
            min_dist = min(min_dist, dist)

    if min_dist < 0:
        # 碰撞：给予极低分数
        return -float('inf')
    elif min_dist < 0.2:
        # 危险区域：线性比例
        return min_dist / 0.2
    else:
        # 安全区域
        return 1.0

def velocity_score(self, v, omega, max_v, max_w):
    """
    速度评分

    鼓励机器人以较高速度行驶
    """
    v_score = abs(v) / max_v
    w_penalty = abs(omega) / max_w

    # 鼓励直线高速，惩罚过度旋转
    return v_score * (1.0 - 0.3 * w_penalty)

def path_alignment_score(self, trajectory, global_path):
    """
    路径对齐评分

    评估轨迹与全局路径的贴合程度
    """
    if global_path is None or len(global_path) == 0:
        return 0.5

    total_dist = 0
    for point in trajectory:
        px, py = point[0], point[1]
        dists = [np.sqrt((px-gx)**2 + (py-gy)**2)
                  for gx, gy, *_ in global_path]
        total_dist += min(dists) if dists else float('inf')

    avg_dist = total_dist / len(trajectory)
    return 1.0 / (1.0 + avg_dist)

```

19.3 DWB 控制器

19.3.1 DWB 与 DWA 的关系

DWB (Dynamic Window Based) 是 Nav2 中 DWA 的升级实现, 增加了更多可配置的评分器 (Critic) :

DWB 的核心组件:

- **Critics (评分器)** : 评估轨迹的各项指标
- **Goal Checker (目标检查器)** : 判断是否到达目标
- **Oscillation Handler (振荡处理器)** : 防止前进-后退振荡

19.3.2 DWB 参数配置

```
# DWB 控制器配置
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      debug_trajectory_details: True

    # 速度限制
    min_vel_x: 0.0
    max_vel_x: 0.5
    min_vel_y: 0.0
    max_vel_y: 0.0          # 差速机器人, 无侧向运动
    max_vel_theta: 1.0
    min_speed_xy: 0.0
    max_speed_xy: 0.5
    min_speed_theta: 0.0

    # 加速度限制
    acc_lim_x: 2.5
    acc_lim_theta: 3.2
    decel_lim_x: -2.5
    decel_lim_theta: -3.2

    # 仿真参数
    vx_samples: 20          # 线速度采样数
    vtheta_samples: 40      # 角速度采样数
    sim_time: 1.5           # 仿真时长(s)
    sim_granularity: 0.025  # 仿真步长(m)

    # 评分器列表
    critics: [
      "RotateToGoal",
      "Oscillation",
```

```

    "BaseObstacle",
    "GoalAlign",
    "PathAlign",
    "PathDist",
    "GoalDist"
]

# 各评分器权重
RotateToGoal.scale: 32.0
Oscillation.scale: 10.0
BaseObstacle.scale: 32.0
GoalAlign.scale: 24.0
PathAlign.scale: 32.0
PathDist.scale: 32.0
GoalDist.scale: 24.0

# 目标检查器
goal_checker_plugin: "goal_checker"
goal_checker:
    plugin: "dwb_core::SimpleGoalChecker"
    xy_goal_tolerance: 0.1      # 位置容忍度(m)
    yaw_goal_tolerance: 0.1    # 角度容忍度(rad)
    stateful: True

```

19.3.3 评分器详解

```

class DWBCritics:
    """DWB 评分器集合"""

    @staticmethod
    def BaseObstacle(trajectory, costmap):
        """
        基础障碍物评分

        评估轨迹上各栅格的代价地图值

        碰撞=最大惩罚
        """
        total_cost = 0
        for point in trajectory:
            gx = int(point[1] / 0.05)
            gy = int(point[0] / 0.05)
            if 0 <= gx < costmap.shape[0] and 0 <= gy < costmap.shape[1]:
                cost = costmap[gx, gy]
                if cost >= 254: # 致命障碍
                    return float('inf')
                total_cost += cost
        return total_cost / len(trajectory)

    @staticmethod
    def PathAlign(trajectory, global_path):
        """
        路径对齐评分

        轨迹与全局路径的对齐程度
        """

```

```

        if len(global_path) < 2:
            return float('inf')

        total_dist = 0
        for point in trajectory:
            px, py = point[0], point[1]
            # 找到全局路径上最近的点
            min_dist = min(np.sqrt((px-gx)**2 + (py-gy)**2)
                           for gx, gy in global_path)
            total_dist += min_dist

        return total_dist / len(trajectory)

    @staticmethod
    def GoalDist(trajectory, goal):
        """
        目标距离评分

        轨迹终点到目标点的距离
        """
        end_pt = trajectory[-1]
        dist = np.sqrt((end_pt[0]-goal[0])**2 + (end_pt[1]-goal[1])**2)
        return dist

    @staticmethod
    def RotateToGoal(robot_pose, goal):
        """
        转向目标评分

        机器人当前朝向与目标方向的偏差
        """
        dx = goal[0] - robot_pose[0]
        dy = goal[1] - robot_pose[1]
        target_angle = np.arctan2(dy, dx)

        angle_diff = abs(robot_pose[2] - target_angle)
        angle_diff = min(angle_diff, 2*np.pi - angle_diff)
        return angle_diff

    @staticmethod
    def Oscillation(history):
        """
        振荡检测评分

        检测机器人是否在前进-后退间来回摆动
        """
        if len(history) < 5:
            return 0

        # 检查最近的速度指令是否有方向交替
        recent_v = [cmd[0] for cmd in history[-5:]]
        sign_changes = sum(1 for i in range(len(recent_v)-1)
                           if recent_v[i]*recent_v[i+1] < 0)

        # 方向变换过多则惩罚
        return sign_changes * 100.0

```

19.4 Regulated Pure Pursuit

19.4.1 Pure Pursuit 基本原理

Pure Pursuit 是一种基于几何追踪的路径跟踪算法，通过追踪前方的一个目标点来引导机器人。

算法原理：

1. 在全局路径上找到一个前瞻点 (lookahead point)
 - 距离机器人当前位置为 `lookahead_distance`
2. 计算机器人到前瞻点的转角
 - 使用圆弧拟合，圆弧经过机器人和前瞻点
3. 输出速度指令
 - 线速度：根据曲率自适应调节
 - 角速度：圆弧曲率

曲率计算公式：

$$\gamma = 2 * \sin(\alpha) / L$$

其中：

- α ：机器人朝向与前向量之间的夹角
- L ：前瞻距离
- γ ：曲率

控制量：

- $\omega = v * \gamma$ (角速度 = 线速度 $\times$ 曲率)

19.4.2 Regulated Pure Pursuit 实现

```
import numpy as np

class RegulatedPurePursuit:
    """调节纯追踪控制器"""
    def __init__(self, config: dict = None):
        # 基本参数
        self.desired_linear_vel = config.get('desired_linear_vel', 0.3)
        self.max_linear_vel = config.get('max_linear_vel', 0.5)
        self.min_linear_vel = config.get('min_linear_vel', 0.05)
        self.max_angular_vel = config.get('max_angular_vel', 1.0)

        # 前瞻参数
        self.lookahead_dist = config.get('lookahead_dist', 1.0)
        self.min_lookahead_dist = config.get('min_lookahead_dist', 0.3)
        self.max_lookahead_dist = config.get('max_lookahead_dist', 2.0)
        self.use_velocity_scaled_lookahead = config.get(
            'use_velocity_scaled_lookahead', True)

        # 调节参数
```



```

self.use_regulated_linear_velocity = config.get(
    'use_regulated_linear_velocity', True)
self.use_cost_regulated_linear_velocity = config.get(
    'use_cost_regulated_linear_velocity', True)
self.cost_scaling_dist = config.get('cost_scaling_dist', 0.3)
self.cost_scaling_gain = config.get('cost_scaling_gain', 1.0)

# 曲率调节
self.max_allowed_curvature = config.get('max_allowed_curvature', 0.5)
self.curvature_weight = config.get('curvature_weight', 0.3)

def compute_velocity_commands(self, robot_pose: np.ndarray,
                             global_path: np.ndarray,
                             costmap: np.ndarray = None) -> np.ndarray:
    """
    计算速度指令

    参数:
        robot_pose: [x, y, theta] 机器人当前位姿
        global_path: (N, 2) 全局路径点
        costmap: 代价地图 (可选, 用于速度调节)

    返回:
        [linear_vel, angular_vel]
    """
    if len(global_path) < 2:
        return np.array([0.0, 0.0])

    # 1. 寻找前瞻点
    lookahead_point, lookahead_idx = self._find_lookahead_point(
        robot_pose, global_path
    )

    if lookahead_point is None:
        return np.array([0.0, 0.0])

    # 2. 计算曲率
    curvature = self._compute_curvature(robot_pose, lookahead_point)

    # 3. 计算线速度
    linear_vel = self._compute_linear_velocity(
        robot_pose, lookahead_point, curvature, costmap
    )

    # 4. 计算角速度
    angular_vel = linear_vel * curvature

    # 限幅
    angular_vel = np.clip(angular_vel, -self.max_angular_vel,
                          self.max_angular_vel)
    linear_vel = np.clip(linear_vel, self.min_linear_vel,
                         self.max_linear_vel)

    return np.array([linear_vel, angular_vel])

def _find_lookahead_point(self, robot_pose: np.ndarray,

```

```

        global_path: np.ndarray) -> tuple:

    """寻找前瞻点"""

    # 计算机人到路径上各点的距离
    dx = global_path[:, 0] - robot_pose[0]
    dy = global_path[:, 1] - robot_pose[1]
    distances = np.sqrt(dx**2 + dy**2)

    # 计算前瞻距离
    if self.use_velocity_scaled_lookahead:
        # 速度自适应前瞻：速度越快，前瞻越远
        lookahead = (self.lookahead_dist +
                     self.desired_linear_vel * 0.5)
        lookahead = np.clip(lookahead, self.min_lookahead_dist,
                           self.max_lookahead_dist)
    else:
        lookahead = self.lookahead_dist

    # 找第一个超过前瞻距离的点
    for i, dist in enumerate(distances):
        if dist >= lookahead:
            return global_path[i], i

    # 如果都小于前瞻距离，返回最后一个点
    return global_path[-1], len(global_path) - 1

def _compute_curvature(self, robot_pose: np.ndarray,
                      lookahead_point: np.ndarray) -> float:
    """计算曲率"""

    dx = lookahead_point[0] - robot_pose[0]
    dy = lookahead_point[1] - robot_pose[1]

    # 机器人到前瞻点的距离
    L = np.sqrt(dx**2 + dy**2)
    if L < 0.001:
        return 0.0

    # 机器人朝向与目标方向的夹角
    alpha = np.arctan2(dy, dx) - robot_pose[2]
    alpha = np.arctan2(np.sin(alpha), np.cos(alpha))

    # 曲率 = 2 * sin(alpha) / L
    curvature = 2.0 * np.sin(alpha) / L

    # 限幅
    return np.clip(curvature, -self.max_allowed_curvature,
                  self.max_allowed_curvature)

def _compute_linear_velocity(self, robot_pose: np.ndarray,
                            lookahead_point: np.ndarray,
                            curvature: float,
                            costmap: np.ndarray = None) -> float:
    """计算线速度（带调节机制）"""

    velocity = self.desired_linear_vel

    if self.use_regulated_linear_velocity:

```

```

        # 曲率调节：曲率大则减速
        curvature_factor = 1.0 - abs(curvature) * self.curvature_weight
        velocity *= max(curvature_factor, 0.3)

    if self.use_cost_regulated_linear_velocity and costmap is not None:
        # 代价调节：靠近障碍物则减速
        gx = int(robot_pose[1] / 0.05)
        gy = int(robot_pose[0] / 0.05)

        if 0 <= gx < costmap.shape[0] and 0 <= gy < costmap.shape[1]:
            cost = costmap[gx, gy]
            if cost > 0:
                cost_factor = 1.0 - (cost / 254.0) * self.cost_scaling_gain
                velocity *= max(cost_factor, 0.1)

    return max(velocity, self.min_linear_vel)

```

19.4.3 RPP 配置

```

# Regulated Pure Pursuit 参数配置
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "nav2_regulated_pure_pursuit_controller::\
        RegulatedPurePursuitController"

    # 速度参数
    desired_linear_vel: 0.3
    max_linear_vel: 0.5
    min_linear_vel: 0.05
    max_angular_vel: 1.0

    # 前瞻参数
    lookahead_dist: 1.0
    min_lookahead_dist: 0.3
    max_lookahead_dist: 2.0
    use_velocity_scaled_lookahead: true

    # 调节参数
    use_regulated_linear_velocity: true
    use_cost_regulated_linear_velocity: true
    cost_scaling_dist: 0.3
    cost_scaling_gain: 1.0

    # 曲率限制
    max_allowed_curvature: 0.5
    curvature_weight: 0.3

    # 碰撞防护
    use_collision_detection: true
    collision_check_distance: 0.5
    max_linear_deceleration: -0.5

```

19.4.4 DWB vs RPP 对比

```
def compare_controllers():
    """对比 DWB 和 RPP 控制器的特点"""

    comparison = """


| 特性    | DWB        | RPP        |
|-------|------------|------------|
| 算法基础  | 动态窗口法+多评分器 | 纯追踪+速度调节   |
| 计算复杂度 | 高（评分器多）    | 低（几何计算）    |
| 参数数量  | 多（大量评分器参数） | 少（简单直观）    |
| 平滑性   | 好          | 好          |
| 狭窄通道  | 好          | 中（需要调前瞻参数） |
| 动态环境  | 好（实时避障）    | 中（被动反应）    |
| 调试难度  | 难（参数多）     | 易（参数少）     |
| 适用场景  | 通用         | 简单可靠场景     |


    """

    print(comparison)
```

19.5 MPPI 控制器

19.5.1 MPPI 原理

MPPI (Model Predictive Path Integral) 是一种基于采样的模型预测控制方法：

核心思想：

- 295. 对控制序列进行多次采样（加入噪声）
- 296. 前向模拟每条采样序列的未来状态
- 297. 根据代价函数计算每条序列的权重
- 298. 加权平均生成最优控制序列
- 299. 执行第一个控制指令，重复

19.5.2 MPPI 优势

- 可处理非线性动力学
- 自然支持并行计算（GPU 加速）
- 对模型误差鲁棒
- 适合复杂动态环境

```

controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "nav2_mppi_controller::MPPIController"

      # 采样参数
      samples: 1000          # 采样数
      time_steps: 56         # 预测步数
      dt: 0.1                # 时间步长

      # 噪声参数
      vx_std: 0.2            # 线速度噪声标准差
      vy_std: 0.0            # 侧向速度噪声
      wz_std: 0.4            # 角速度噪声标准差

      # 代价权重
      path_cost_weight: 5.0  # 路径跟踪代价
      goal_cost_weight: 5.0  # 目标到达代价
      obstacle_cost_weight: 10.0 # 避障代价
      smoothness_cost_weight: 1.0 # 平滑性代价

```

19.6 局部规划参数调优

19.6.1 速度参数调优

```

def recommend_speed_params(robot_type: str,
                           environment: str) -> dict:
    """根据场景推荐速度参数"""
    params = {}

    if robot_type == 'differential':
        # 差速机器人
        params['max_linear_vel'] = 0.5
        params['max_angular_vel'] = 1.0
        params['acc_lim_x'] = 2.5
    elif robot_type == 'ackermann':
        # 阿克曼机器人
        params['max_linear_vel'] = 0.8
        params['max_angular_vel'] = 0.5
        params['acc_lim_x'] = 1.5

    if environment == 'narrow':
        # 狭窄环境：低速高转向
        params['max_linear_vel'] *= 0.6
        params['acc_lim_x'] *= 0.7
    elif environment == 'open':
        # 开阔环境：高速
        params['max_linear_vel'] *= 1.2

    return params

```

19.6.2 前瞻距离调优

前瞻距离是 RPP 最关键参数：

```
def compute_optimal_lookahead(linear_vel: float,
                              control_frequency: float,
                              robot_length: float) -> float:
    """
    计算最优前瞻距离

    原则：
    - 速度越快，前瞻越远
    - 控制频率越高，前瞻可适当缩短
    - 机器人越长，前瞻越远
    """
    base_lookahead = linear_vel * 3.0 # 3 倍速度
    frequency_factor = 10.0 / control_frequency
    length_factor = robot_length * 2.0

    lookahead = (base_lookahead * frequency_factor + length_factor) / 2.0
    return np.clip(lookahead, 0.3, 3.0)
```

19.6.3 评分权重调优指南

```
# 不同场景的 DWB 权重推荐

# 狭窄走廊场景
critics: ["BaseObstacle", "PathAlign", "GoalDist"]
BaseObstacle.scale: 50.0 # 弱化避障
PathAlign.scale: 50.0 # 强化路径跟踪
GoalDist.scale: 20.0

# 开阔大厅场景
critics: ["BaseObstacle", "GoalAlign", "PathDist", "GoalDist"]
BaseObstacle.scale: 30.0 # 弱化避障
GoalAlign.scale: 40.0 # 强化目标方向
PathDist.scale: 20.0
GoalDist.scale: 30.0

# 动态障碍物场景
critics: ["BaseObstacle", "PathAlign", "GoalAlign", "Oscillation"]
BaseObstacle.scale: 60.0 # 强化避障
PathAlign.scale: 20.0
GoalAlign.scale: 20.0
Oscillation.scale: 15.0
```

19.6.4 调试命令

```
# 查看当前控制器参数
ros2 param describe /controller_server FollowPath
```

```
# 动态调整速度
ros2 param set /controller_server FollowPath.max_vel_x 0.3

# 查看轨迹细节
ros2 topic echo /controller_server/trajectory_details

# 查看速度指令
ros2 topic echo /cmd_vel

# RViz2 中可视化
rviz2
# 添加: Local Plan, Trajectories
```

课后练习

300. **原理题:** 说明 DWA 算法中动态窗口的定义, 阐述为什么需要结合速度极限、加速度极限和安全停止距离来生成动态窗口。
301. **编程题:** 实现 DWA 局部路径规划器, 包含速度空间采样、轨迹预测和评分函数, 并在仿真环境中测试避障效果。
302. **分析题:** 比较 DWB 和 Regulated Pure Pursuit 的异同, 分析两种控制器在不同场景下的适用性。
303. **配置题:** 为差速机器人在走廊环境配置 DWB 控制器参数, 要求能够顺利通过 0.8m 宽的走廊并保持稳定的导航行为。
304. **操作题:** 在仿真环境中分别使用 DWB 和 RPP 控制器进行导航, 对比两种控制器在路径跟踪精度和安全避障方面的表现。
305. **设计题:** 设计一个自适应控制器切换策略: 在宽阔区域使用 RPP 提高效率, 在狭窄通道切换到 DWB 提高通过能力, 并给出切换条件和参数过渡方案。

实验 19:

第 19 章 实验：相机驱动、标定与视觉检测

对应理论章节: 第 30 章《相机驱动与标定》、第 31 章《颜色检测与 YOLOv8》、
第 32 章《AR 标签检测与手眼标定》 ***实验课时***: 6 课时 ***实验代码***:
``src/lab_code/ch19_lab/vision_detection_lab/``

实验目标

- 掌握 USB 相机在 ROS2 中的驱动方法
- 学会使用 cv_bridge 进行 ROS 图像与 OpenCV 图像转换
- 理解相机内参标定原理和流程
- 能够使用 OpenCV 进行基本的图像处理
- 掌握 OpenCV HSV 颜色空间和颜色检测方法
- 学会 YOLOv8 模型的加载和推理
- 实现 ROS2 中 YOLOv8 检测结果的发布
- 掌握 AprilTag/ArUco 标签的检测和位姿估计方法
- 学会在 ROS2 中发布标签位姿话题
- 理解手眼标定的原理和流程，能够使用 easy_handeye 完成眼在手外标定

实验环境

- ROS 2 Jazzy
- USB 摄像头(或笔记本内置摄像头)
- usb_cam 功能包
- OpenCV + cv_bridge
- camera_calibration 功能包
- YOLOv8 (ultralytics) + Python3 + onnxruntime (19.2 节)
- easy_handeye、aruco_ros 功能包，打印的 ArUco 标定板 (19.3 节)

参考代码说明

src/lab_code/ch19_lab/vision_detection_lab/ (ament_python 包) 提供以下程序:

程序	功能
<code>`usb_cam_node.py`</code>	USB 摄像头图像查看器。订阅 <code>`/image_raw`</code> 话题, 使用 OpenCV 显示图像, 按 ESC 退出
<code>`cv_bridge_demo.py`</code>	cv_bridge 图像转换演示。将 ROS 图像转为 OpenCV 格式, 在图像上绘制橙色矩形, 再转回 ROS 图像发布到 <code>`/image_show`</code>
<code>`color_detection_node.py`</code>	HSV 颜色检测节点。在图像中检测指定颜色的物体区域
<code>`ar_tag_detection_node.py`</code>	AR 标签检测节点。检测 ArUco 标签并估计位姿

构建并运行:

```
cd <工作区根目录>
colcon build --symlink-install --packages-select vision_detection_lab
source install/setup.bash

# USB 相机 (真实硬件)
ros2 run vision_detection_lab usb_cam_node

# 或使用仿真相机 (robot_sim_demo 提供 /camera/image_raw)
ros2 launch robot_sim_demo gazebo2.launch.py

# 检测节点
ros2 run vision_detection_lab cv_bridge_demo
ros2 run vision_detection_lab color_detection_node
ros2 run vision_detection_lab ar_tag_detection_node
```

验证:

```
ros2 topic echo /camera/image_raw --field header --once
rqt_image_view
```

19.1 相机驱动与标定

19.1.1 安装依赖

```
sudo apt install ros-jazzy-usb-cam
sudo apt install ros-jazzy-cv-bridge
sudo apt install ros-jazzy-camera-calibration
sudo apt install ros-jazzy-rqt-image-view

# 验证安装
ros2 pkg list | grep -E "usb_cam|cv_bridge|camera_calibration"
```

19.1.2 检测摄像头设备

```
# 查看摄像头设备文件
ls /dev/video*

# 查看摄像头详细信息
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats
```

19.1.3 启动 usb_cam 节点

```
ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p image_width:=640 \
  -p image_height:=480 \
  -p camera_frame_id:=camera_link \
  -p pixel_format:=yuyv

# 或使用 launch 文件
ros2 launch usb_cam usb_cam-launch.py
```

19.1.4 验证图像流

```
# 新终端
# 查看话题列表
ros2 topic list

# 查看图像话题信息
ros2 topic info /image_raw

# 查看图像话题类型
ros2 topic type /image_raw

# 使用 rqt_image_view 查看图像
rqt_image_view /image_raw

# 查看相机信息
ros2 topic echo /camera_info --once
```

19.1.5 创建实验功能包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch19_camera --build-type ament_python --dependencies rclpy sensor_msgs cv_bridge
cd ch19_camera
mkdir -p ch19_camera launch
```

19.1.6 编写图像订阅和显示节点

创建 ch19_camera/image_viewer.py (参考实现:

src/lab_code/ch19_lab/vision_detection_lab/usb_cam_node.py) :

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class ImageViewer(Node):
    def __init__(self):
        super().__init__('image_viewer')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.get_logger().info('图像查看器已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

        h, w = cv_image.shape[:2]
        fps_text = f'Size: {w}x{h}'
        cv2.putText(cv_image, fps_text, (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.imshow('Camera View', cv_image)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ImageViewer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

19.1.7 编写图像处理节点 (灰度+边缘检测)

创建 ch19_camera/image_processor.py (参考实现: cv_bridge_demo.py 演示了"ROS 图像→OpenCV→处理→ROS 图像"的完整回路):

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class ImageProcessor(Node):
    def __init__(self):
        super().__init__('image_processor')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.gray_pub = self.create_publisher(Image, '/image_gray', 10)
        self.edge_pub = self.create_publisher(Image, '/image_edges', 10)
        self.get_logger().info('图像处理节点已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

        gray = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(gray, 50, 150)
        blurred = cv2.GaussianBlur(cv_image, (5, 5), 0)

        self.gray_pub.publish(self.bridge.cv2_to_imgmsg(gray, 'mono8'))
        edge_color = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
        self.edge_pub.publish(self.bridge.cv2_to_imgmsg(edge_color, 'bgr8'))

        combined = cv2.hconcat([cv_image, blurred])
        cv2.imshow('Original + Blurred', combined)
        cv2.imshow('Edges', edges)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ImageProcessor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

19.1.8 编写相机信息发布器

创建 `ch19_camera/camera_info_publisher.py`:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

class CameraInfoPublisher(Node):
    def __init__(self):
        super().__init__('camera_info_publisher')
        self.pub = self.create_publisher(CameraInfo, '/camera_info', 10)
        self.timer = self.create_timer(1.0, self.publish_info)
        self.get_logger().info('相机信息发布器已启动')

    def publish_info(self):
        msg = CameraInfo()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'camera_link'
        msg.width = 640
        msg.height = 480
        msg.distortion_model = 'plumb_bob'
        msg.d = [0.0, 0.0, 0.0, 0.0, 0.0]
        msg.k = [530.0, 0.0, 320.0, 0.0, 530.0, 240.0, 0.0, 0.0, 1.0]
        msg.r = [1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0]
        msg.p = [530.0, 0.0, 320.0, 0.0, 0.0, 530.0, 240.0, 0.0, 0.0, 0.0, 1.0, 0.0]
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = CameraInfoPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

19.1.9 相机标定

```
# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p pixel_format:=yuyv

# 终端 2: 启动标定
# 准备标定板: 打印棋盘格, 8x6 内部角点, 每个方格 24.5mm
ros2 run camera_calibration cameracalibrator.py \
  --size 8x6 --square 0.0245 \
  image:=/image_raw camera:=/camera
```

19.1.10 标定操作步骤

306. 将标定板放在摄像头视野内
307. 缓慢移动标定板: 上下、左右、前后、倾斜旋转
308. 观察右侧进度条:
309. X: 左右移动
310. Y: 上下移动
311. Size: 前后移动
312. Skew: 倾斜角度
313. 每个进度条变绿后, 点击 CALIBRATE
314. 等待计算完成, 查看标定结果
315. 点击 SAVE 保存标定参数
316. 点击 COMMIT 将参数写入到 camera_info 话题

19.1.11 查看标定结果

```
# 标定结果保存在
ls /tmp/calibrationdata.tar.gz
tar -xzvf /tmp/calibrationdata.tar.gz -C /tmp/calib_result/
cat /tmp/calib_result/ost.yaml
```

19.1.12 使用标定结果矫正图像

创建 ch19_camera/undistort.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np
import yaml

class ImageUndistorter(Node):
    def __init__(self):
        super().__init__('image_undistorter')
        self.bridge = CvBridge()
        self.camera_matrix = None
        self.dist_coeffs = None
        self.new_camera_matrix = None

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.info_sub = self.create_subscription(
            CameraInfo, '/camera_info', self.info_callback, 10
```

```

    )
    self.undist_pub = self.create_publisher(Image, '/image_undistorted', 10)

def info_callback(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        h, w = msg.height, msg.width
        self.new_camera_matrix, _ = cv2.getOptimalNewCameraMatrix(
            self.camera_matrix, self.dist_coeffs, (w, h), 1.0, (w, h)
        )
        self.get_logger().info('相机参数已加载')

def image_callback(self, msg):
    if self.camera_matrix is None:
        return
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception as e:
        self.get_logger().error(str(e))
        return

    h, w = cv_image.shape[:2]
    undistorted = cv2.undistort(
        cv_image, self.camera_matrix, self.dist_coeffs,
        None, self.new_camera_matrix
    )

    combined = np.hstack([cv_image, undistorted])
    cv2.putText(combined, 'Original', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.putText(combined, 'Undistorted', (w + 10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow('Undistortion Comparison', combined)
    cv2.waitKey(1)

    self.undist_pub.publish(
        self.bridge.cv2_to_imgmsg(undistorted, 'bgr8')
    )

def load_calibration(self, yaml_file):
    with open(yaml_file, 'r') as f:
        data = yaml.safe_load(f)
    self.camera_matrix = np.array(data['camera_matrix']['data']).reshape(3, 3)
    self.dist_coeffs = np.array(data['distortion_coefficients']['data'])
    self.get_logger().info(f'从文件加载标定结果: {yaml_file}')

def main(args=None):
    rclpy.init(args=args)
    node = ImageUndistorter()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == '__main__':  
    main()
```

19.1.13 配置 setup.py 并编译运行

```
entry_points={  
    'console_scripts': [  
        'image_viewer = ch19_camera.image_viewer:main',  
        'image_processor = ch19_camera.image_processor:main',  
        'camera_info_publisher = ch19_camera.camera_info_publisher:main',  
        'undistort = ch19_camera.undistort:main',  
    ],  
},  
},  
cd ~/ros2_arm_ws  
colcon build --packages-select ch19_camera  
source install/setup.bash  
  
# 终端 1: 启动相机  
ros2 run usb_cam usb_cam_node_exe  
  
# 终端 2: 运行图像查看器  
ros2 run ch19_camera image_viewer  
  
# 终端 3: 运行图像处理  
ros2 run ch19_camera image_processor  
  
# 查看处理结果  
rqt_image_view /image_gray  
rqt_image_view /image_edges
```

19.1.14 录制和回放图像数据

```
# 录制 bag 文件  
ros2 bag record /image_raw /camera_info -o camera_data  
  
# 回放 bag 文件  
ros2 bag play camera_data
```

19.2 颜色检测与 YOLO 检测

19.2.1 安装依赖

```
pip install ultralytics onnxruntime onnx  
pip install opencv-python numpy  
  
# 验证  
python3 -c "from ultralytics import YOLO; print('YOLO installed')"
```

19.2.2 创建实验包

```
cd ~/ros2_arm_ws/src  
ros2 pkg create ch19_detection --build-type ament_python --dependencies rclpy sensor_msgs std_msgs  
vision_msgs cv_bridge
```



```
cd ch19_detection
mkdir -p ch19_detection
```

19.2.3 编写 HSV 颜色检测节点

创建 ch19_detection/color_detector.py (参考实现:

src/lab_code/ch19_lab/vision_detection_lab/color_detection_node.py) :

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import cv2
import numpy as np

class ColorDetector(Node):
    def __init__(self):
        super().__init__('color_detector')
        self.bridge = CvBridge()

        self.declare_parameter('h_min', 35)
        self.declare_parameter('h_max', 85)
        self.declare_parameter('s_min', 50)
        self.declare_parameter('s_max', 255)
        self.declare_parameter('v_min', 50)
        self.declare_parameter('v_max', 255)

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.result_pub = self.create_publisher(Image, '/detection/color_result', 10)
        self.info_pub = self.create_publisher(String, '/detection/color_info', 10)
        self.get_logger().info('颜色检测节点已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

        hsv = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)
        h_min = self.get_parameter('h_min').value
        h_max = self.get_parameter('h_max').value
        s_min = self.get_parameter('s_min').value
        s_max = self.get_parameter('s_max').value
        v_min = self.get_parameter('v_min').value
        v_max = self.get_parameter('v_max').value

        lower = np.array([h_min, s_min, v_min])
        upper = np.array([h_max, s_max, v_max])
        mask = cv2.inRange(hsv, lower, upper)

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

```

contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

result = cv_image.copy()
objects = []
for c in contours:
    area = cv2.contourArea(c)
    if area < 500:
        continue

    rect = cv2.minAreaRect(c)
    box = cv2.boxPoints(rect)
    box = np.int0(box)
    cv2.drawContours(result, [box], 0, (0, 255, 0), 2)

    center = (int(rect[0][0]), int(rect[0][1]))
    cv2.circle(result, center, 5, (0, 0, 255), -1)

    objects.append({
        'center': (rect[0][0], rect[0][1]),
        'size': (rect[1][0], rect[1][1]),
        'angle': rect[2],
        'area': area
    })

info = f'Detected {len(objects)} objects'
cv2.putText(result, info, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

self.result_pub.publish(self.bridge.cv2_to_imgmsg(result, 'bgr8'))

info_msg = String()
info_msg.data = f'count={len(objects)} '
for i, obj in enumerate(objects):
    info_msg.data += f'obj{i}:({obj["center"][0]:.0f},{obj["center"][1]:.0f}) '
self.info_pub.publish(info_msg)

cv2.imshow('Color Detection', result)
cv2.imshow('Mask', mask)
cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ColorDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.4 下载 YOLOv8 模型

```

# 下载 YOLOv8n 预训练模型
python3 -c "
from ultralytics import YOLO

```

```

model = YOLO('yolov8n.pt')
print('Model loaded successfully')

# 导出为 ONNX 格式
model.export(format='onnx', imgsz=640, simplify=True)
print('Export to ONNX complete')

```

19.2.5 编写 YOLOv8 检测节点

创建 ch19_detection/yolo_detector.py:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose
from cv_bridge import CvBridge
from ultralytics import YOLO
import cv2
import numpy as np
import time

class YOLODetector(Node):
    def __init__(self):
        super().__init__('yolo_detector')
        self.bridge = CvBridge()

        self.declare_parameter('model_path', 'yolov8n.pt')
        self.declare_parameter('conf_threshold', 0.5)

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_threshold').value

        self.get_logger().info(f'加载 YOLO 模型: {model_path}')
        self.model = YOLO(model_path)

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.det_pub = self.create_publisher(
            Detection2DArray, '/yolo/detections', 10
        )
        self.annotated_pub = self.create_publisher(
            Image, '/yolo/annotated', 10
        )

        self.last_time = time.time()
        self.frame_count = 0
        self.get_logger().info('YOLO 检测节点已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

```

```

t0 = time.time()
results = self.model(cv_image, conf=self.conf_thresh, verbose=False)
infer_time = time.time() - t0

det_msg = Detection2DArray()
det_msg.header = msg.header

annotated = results[0].plot()

for r in results:
    for box in r.bboxes:
        det = Detection2D()
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        det.bbox.center.position.x = float((x1 + x2) / 2)
        det.bbox.center.position.y = float((y1 + y2) / 2)
        det.bbox.size_x = float(x2 - x1)
        det.bbox.size_y = float(y2 - y1)

        hyp = ObjectHypothesisWithPose()
        hyp.hypothesis.class_id = str(int(box.cls[0]))
        hyp.hypothesis.score = float(box.conf[0])
        det.results.append(hyp)
        det_msg.detections.append(det)

self.det_pub.publish(det_msg)
self.annotated_pub.publish(
    self.bridge.cv2_to_imgmsg(annotated, 'bgr8')
)

self.frame_count += 1
elapsed = time.time() - self.last_time
if elapsed > 5.0:
    fps = self.frame_count / elapsed
    self.get_logger().info(
        f'FPS: {fps:.1f} | 推理: {infer_time*1000:.0f}ms | '
        f'检测: {len(det_msg.detections)} 个'
    )
    self.frame_count = 0
    self.last_time = time.time()

def main(args=None):
    rclpy.init(args=args)
    node = YOLODetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.6 编写 ONNX 推理节点(备选)

创建 ch19_detection/yolo_onnx_node.py:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose

```

```

from cv_bridge import CvBridge
import onnxruntime as ort
import numpy as np
import cv2
import time

class YOLOONNXNode(Node):
    def __init__(self):
        super().__init__('yolo_onnx_node')
        self.bridge = CvBridge()

        self.declare_parameter('model_path', 'yolov8n.onnx')
        self.declare_parameter('conf_thresh', 0.5)
        self.declare_parameter('input_size', 640)

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_thresh').value
        self.input_size = self.get_parameter('input_size').value

        self.get_logger().info(f'加载 ONNX 模型: {model_path}')
        self.session = ort.InferenceSession(
            model_path, providers=['CPUExecutionProvider']
        )
        self.input_name = self.session.get_inputs()[0].name

        self.classes = [
            'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
            'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
            'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
            'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
            'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee',
            'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
            'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
            'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
            'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
            'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
            'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
            'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven',
            'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase',
            'scissors', 'teddy bear', 'hair drier', 'toothbrush',
        ]

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )
        self.det_pub = self.create_publisher(Detection2DArray, '/yolo/detections', 10)
        self.annotated_pub = self.create_publisher(Image, '/yolo/annotated', 10)
        self.get_logger().info('ONNX YOLO 节点已启动')

    def image_cb(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            return

        t0 = time.time()
        h, w = cv_image.shape[:2]

        img = cv2.resize(cv_image, (self.input_size, self.input_size))

```

```

img = img.astype(np.float32) / 255.0
img = np.transpose(img, (2, 0, 1))
img = np.expand_dims(img, axis=0)

outputs = self.session.run(None, {self.input_name: img})
output = outputs[0][0].T

det_msg = Detection2DArray()
det_msg.header = msg.header

annotated = cv_image.copy()
for row in output:
    scores = row[4:84]
    class_id = np.argmax(scores)
    conf = scores[class_id]
    if conf < self.conf_thresh:
        continue

    cx, cy, bw, bh = row[:4]
    x1 = int((cx - bw / 2) * w / self.input_size)
    y1 = int((cy - bh / 2) * h / self.input_size)
    x2 = int((cx + bw / 2) * w / self.input_size)
    y2 = int((cy + bh / 2) * h / self.input_size)

    det = Detection2D()
    det.bbox.center.position.x = float((x1 + x2) / 2)
    det.bbox.center.position.y = float((y1 + y2) / 2)
    det.bbox.size_x = float(x2 - x1)
    det.bbox.size_y = float(y2 - y1)
    hyp = ObjectHypothesisWithPose()
    hyp.hypothesis.class_id = str(int(class_id))
    hyp.hypothesis.score = float(conf)
    det.results.append(hyp)
    det_msg.detections.append(det)

    cls_name = self.classes[class_id] if class_id < len(self.classes) else 'unknown'
    cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), 2)
    label = f'{cls_name} {conf:.2f}'
    cv2.putText(annotated, label, (x1, y1-5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

self.det_pub.publish(det_msg)
self.annotated_pub.publish(self.bridge.cv2_to_imgmsg(annotated, 'bgr8'))

infer_time = time.time() - t0
self.get_logger().info(f'检测: {len(det_msg.detections)} 个, 耗时: {infer_time*1000:.0f}ms')

def main(args=None):
    rclpy.init(args=args)
    node = YOLOONNXNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.7 编写颜色+形状联合检测

创建 ch19_detection/color_shape_detector.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import numpy as np

class ColorShapeDetector(Node):
    def __init__(self):
        super().__init__('color_shape_detector')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.pub = self.create_publisher(Image, '/detection/color_shape', 10)

        self.color_ranges = {
            'red': [(0, 50, 50), (10, 255, 255)],
            'red2': [(170, 50, 50), (180, 255, 255)],
            'green': [(35, 50, 50), (85, 255, 255)],
            'blue': [(100, 50, 50), (130, 255, 255)],
            'yellow': [(20, 50, 50), (35, 255, 255)],
        }

    def detect_shape(self, contour):
        peri = cv2.arcLength(contour, True)
        approx = cv2.approxPolyDP(contour, 0.04 * peri, True)
        vertices = len(approx)
        if vertices == 3:
            return 'triangle'
        elif vertices == 4:
            x, y, w, h = cv2.boundingRect(contour)
            aspect = w / h
            return 'square' if 0.9 < aspect < 1.1 else 'rectangle'
        elif vertices > 6:
            return 'circle'
        return 'polygon'

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            return

        hsv = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)
        result = cv_image.copy()

        detected = []
        for color_name, (lower, upper) in self.color_ranges.items():
            if color_name == 'red2':
                continue
            l = np.array(lower)
            u = np.array(upper)
            mask = cv2.inRange(hsv, l, u)
```

```

        if color_name == 'red':
            l2 = np.array(self.color_ranges['red2'][0])
            u2 = np.array(self.color_ranges['red2'][1])
            mask2 = cv2.inRange(hsv, l2, u2)
            mask = cv2.bitwise_or(mask, mask2)

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        for c in contours:
            area = cv2.contourArea(c)
            if area < 1000:
                continue
            shape = self.detect_shape(c)
            M = cv2.moments(c)
            if M['m00'] == 0:
                continue
            cx = int(M['m10'] / M['m00'])
            cy = int(M['m01'] / M['m00'])
            detected.append((color_name, shape, (cx, cy)))

        color_bgr = {
            'red': (0, 0, 255),
            'green': (0, 255, 0),
            'blue': (255, 0, 0),
            'yellow': (0, 255, 255),
        }.get(color_name, (255, 255, 255))

        cv2.drawContours(result, [c], -1, color_bgr, 2)
        label = f'{color_name} {shape}'
        cv2.putText(result, label, (cx-20, cy),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color_bgr, 2)

        info = f'Objects: {len(detected)}'
        cv2.putText(result, info, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        self.pub.publish(self.bridge.cv2_to_imgmsg(result, 'bgr8'))
        cv2.imshow('Color+Shape Detection', result)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ColorShapeDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.8 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [

```



```

        'color_detector = ch19_detection.color_detector:main',
        'yolo_detector = ch19_detection.yolo_detector:main',
        'yolo_onnx_node = ch19_detection.yolo_onnx_node:main',
        'color_shape_detector = ch19_detection.color_shape_detector:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch19_detection
source install/setup.bash

# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe

# 终端 2: 颜色检测
ros2 run ch19_detection color_detector --ros-args -p h_min:=35 -p h_max:=85

# 调整绿色检测参数
ros2 run ch19_detection color_detector --ros-args -p h_min:=100 -p h_max:=124

# 或运行 YOLO 检测
ros2 run ch19_detection yolo_detector

# 查看 YOLO 结果
rqt_image_view /yolo/annotated
ros2 topic echo /yolo/detections

```

19.2.9 使用 `rqt_reconfigure` 调试参数

```

sudo apt install ros-jazzy-rqt-reconfigure
ros2 run rqt_reconfigure rqt_reconfigure

```

19.3 AR 标签检测与手眼标定

19.3.1 安装依赖

```

sudo apt install ros-jazzy-cv-bridge ros-jazzy-vision-msgs
sudo apt install ros-jazzy-easy-handeye
sudo apt install ros-jazzy-aruco-ros

# 验证
ros2 pkg list | grep -E "aruco|handeye"

```

19.3.2 生成 ArUco 标签

```

# 创建生成标签脚本
python3 -c "
import cv2
import cv2.aruco as aruco
import os

output_dir = os.path.expanduser('~/.aruco_markers')
os.makedirs(output_dir, exist_ok=True)

```

```

dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)

for marker_id in range(1, 6):
    image = aruco.generateImageMarker(dictionary, marker_id, 200)
    filename = os.path.join(output_dir, f'aruco_{marker_id}.png')
    cv2.imwrite(filename, image)
    print(f'生成: {filename}')
"

# 打印标签，测量实际边长(如 0.065m)

```

19.3.3 创建实验包

```

cd ~/ros2_arm_ws/src
ros2 pkg create ch19_aruco --build-type ament_python --dependencies rclpy sensor_msgs
geometry_msgs cv_bridge tf2_ros
cd ch19_aruco
mkdir -p ch19_aruco

```

19.3.4 编写 ArUco 检测节点

创建 ch19_aruco/aruco_detector.py (参考实现:

src/lab_code/ch19_lab/vision_detection_lab/ar_tag_detection_node.py) :

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped, TransformStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros

class ArucoDetector(Node):
    def __init__(self):
        super().__init__('aruco_detector')
        self.bridge = CvBridge()

        self.declare_parameter('marker_size', 0.065)
        self.declare_parameter('dictionary', 'DICT_6X6_250')

        self.marker_size = self.get_parameter('marker_size').value
        dict_name = self.get_parameter('dictionary').value

        dict_map = {
            'DICT_4X4_50': aruco.DICT_4X4_50,
            'DICT_4X4_100': aruco.DICT_4X4_100,
            'DICT_5X5_50': aruco.DICT_5X5_50,
            'DICT_5X5_100': aruco.DICT_5X5_100,
            'DICT_6X6_50': aruco.DICT_6X6_50,
            'DICT_6X6_250': aruco.DICT_6X6_250,
            'DICT_7X7_50': aruco.DICT_7X7_50,
            'DICT_7X7_100': aruco.DICT_7X7_100,
            'DICT_APRILTAG_16h5': aruco.DICT_APRILTAG_16h5,
            'DICT_APRILTAG_25h9': aruco.DICT_APRILTAG_25h9,
            'DICT_APRILTAG_36h10': aruco.DICT_APRILTAG_36h10,

```

```

        'DICT_APRILTAG_36h11': aruco.DICT_APRILTAG_36h11,
    }
    dict_aruco = dict_map.get(dict_name, aruco.DICT_6X6_250)
    dictionary = aruco.getPredefinedDictionary(dict_aruco)
    params = aruco.DetectorParameters()
    self.detector = aruco.ArucoDetector(dictionary, params)

    self.camera_matrix = None
    self.dist_coeffs = None

    self.tf_br = tf2_ros.TransformBroadcaster(self)

    self.image_sub = self.create_subscription(
        Image, '/image_raw', self.image_callback, 10
    )
    self.info_sub = self.create_subscription(
        CameraInfo, '/camera_info', self.info_callback, 10
    )
    self.pose_pub = self.create_publisher(
        PoseStamped, '/aruco/pose', 10
    )
    self.image_pub = self.create_publisher(
        Image, '/aruco/result', 10
    )
    self.get_logger().info('ArUco 检测节点已启动')

def info_callback(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        self.get_logger().info('相机参数已接收')

def image_callback(self, msg):
    if self.camera_matrix is None:
        self.get_logger().warn('等待相机参数...')
        return

    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, rejected = self.detector.detectMarkers(gray)

    if ids is not None:
        aruco.drawDetectedMarkers(frame, corners, ids)

        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
            corners, self.marker_size, self.camera_matrix, self.dist_coeffs
        )

        for i in range(len(ids)):
            aruco.drawAxis(frame, self.camera_matrix, self.dist_coeffs,
                           rvecs[i], tvecs[i], 0.03)

            pose_msg = PoseStamped()
            pose_msg.header.stamp = self.get_clock().now().to_msg()
            pose_msg.header.frame_id = 'camera_link'

```

```

pose_msg.pose.position.x = float(tvecs[i][0][0])
pose_msg.pose.position.y = float(tvecs[i][0][1])
pose_msg.pose.position.z = float(tvecs[i][0][2])

rmat, _ = cv2.Rodrigues(rvecs[i])
q = self.rotation_matrix_to_quaternion(rmat)
pose_msg.pose.orientation.x = q[0]
pose_msg.pose.orientation.y = q[1]
pose_msg.pose.orientation.z = q[2]
pose_msg.pose.orientation.w = q[3]

marker_id = int(ids[i][0])
self.pose_pub.publish(pose_msg)

t = TransformStamped()
t.header.stamp = self.get_clock().now().to_msg()
t.header.frame_id = 'camera_link'
t.child_frame_id = f'aruco_{marker_id}'
t.transform.translation.x = float(tvecs[i][0][0])
t.transform.translation.y = float(tvecs[i][0][1])
t.transform.translation.z = float(tvecs[i][0][2])
t.transform.rotation.x = q[0]
t.transform.rotation.y = q[1]
t.transform.rotation.z = q[2]
t.transform.rotation.w = q[3]
self.tf_br.sendTransform(t)

self.get_logger().info(
    f'检测到标签 {marker_id}: '
    f'pos({tvecs[i][0][0]:.3f}, {tvecs[i][0][1]:.3f}, {tvecs[i][0][2]:.3f})'
)

cv2.putText(frame, f'Markers: {len(ids) if ids is not None else 0}',
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
self.image_pub.publish(self.bridge.cv2_to_imgmsg(frame, 'bgr8'))
cv2.imshow('ArUco Detection', frame)
cv2.waitKey(1)

@staticmethod
def rotation_matrix_to_quaternion(r):
    q = np.empty(4)
    t = np.trace(r)
    if t > 0:
        s = np.sqrt(1.0 + t) * 2
        q[3] = 0.25 * s
        q[0] = (r[2, 1] - r[1, 2]) / s
        q[1] = (r[0, 2] - r[2, 0]) / s
        q[2] = (r[1, 0] - r[0, 1]) / s
    else:
        i = np.argmax([r[0, 0], r[1, 1], r[2, 2]])
        s = np.sqrt(1.0 + r[i, i]) * 2
        q[3] = (r[2, 1] - r[1, 2]) / s if i == 0 else \
            (r[0, 2] - r[2, 0]) / s if i == 1 else \
            (r[1, 0] - r[0, 1]) / s
        q[i] = 0.25 * s
    return q

def main(args=None):
    rclpy.init(args=args)

```

```

node = ArucoDetector()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    cv2.destroyAllWindows()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.3.5 运行 ArUco 检测

```

# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p camera_frame_id:=camera_link

# 终端 2: 启动检测 (指定标签实际边长 m)
ros2 run ch19_aruco aruco_detector --ros-args -p marker_size:=0.065

# 查看检测结果
rqt_image_view /aruco/result
ros2 topic echo /aruco/pose --once

```

19.3.6 坐标变换验证

```

# 查看 TF 树
ros2 run tf2_tools view_frames.py

# 查看坐标变换
ros2 run tf2_ros tf2_echo camera_link aruco_1

```

19.3.7 准备手眼标定

317. 在机械臂末端固定 ArUco 标定板

318. 启动机械臂 MoveIt2 演示环境 (任选其一, 见第 17 章"实验环境"统一说明):

```

# 课程提供的 xArm 仿真 (纯 MoveIt + RViz / 含 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
# 或 ros2 launch xarm_ros2_arm_only arm_only.launch.py

```

319. 启动相机和 ArUco 检测:

```

ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p camera_frame_id:=camera_link

```

19.3.8 编写手眼标定 launch 文件

创建 launch/handeye_calibration.launch.py:

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import LaunchConfiguration
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    marker_size = LaunchConfiguration('marker_size', default='0.094')
    marker_id = LaunchConfiguration('marker_id', default='1')

    return LaunchDescription([
        Node(
            package='easy_handeye',
            executable='calibrate',
            name='handeye_calibration',
            parameters=[{
                'namespace_prefix': 'xarm',
                'move_group': 'xarm_group',
                'eye_on_hand': False,
                'tracking_base_frame': 'camera_link',
                'tracking_marker_frame': 'aruco_marker_frame',
                'robot_base_frame': 'base_link',
                'robot_effector_frame': 'gripper_centor_link',
                'freehand_robot_movement': False,
            }],
        ),
    ])

```

19.3.9 执行手眼标定

```

# 启动标定 GUI
ros2 launch ch19_aruco handeye_calibration.launch.py \
    marker_size:=0.094 marker_id:=1

# 标定步骤:
# 1. 移动机械臂使标定板在相机视野内
# 2. 点击 "Check Starting Pose"
# 3. 点击 "Next Pose" → "Plan" → "Execute" → "Take Sample"
# 4. 重复 17 次采样
# 5. 点击 "Compute" 计算标定结果
# 6. 点击 "Save" 保存结果

```

19.3.10 查看标定结果

```

# 标定结果保存在
ls ~/.ros/easy_handeye/
cat ~/.ros/easy_handeye/xarm_handeye_calibration_*.yaml

```

19.3.11 使用标定结果发布 TF

创建 ch19_aruco/calibration_tf_publisher.py:

```

import rclpy
from rclpy.node import Node

```

```

import tf2_ros
import geometry_msgs.msg
import yaml
import os

class CalibrationTFPublisher(Node):
    def __init__(self):
        super().__init__('calibration_tf_publisher')
        self.br = tf2_ros.StaticTransformBroadcaster(self)

        self.declare_parameter('calibration_file', '')
        calib_file = self.get_parameter('calibration_file').value

        if calib_file and os.path.exists(calib_file):
            self.load_calibration(calib_file)
        else:
            self.use_default_params()

    def load_calibration(self, filepath):
        with open(filepath, 'r') as f:
            data = yaml.safe_load(f)

        trans = data.get('transformation', data)
        t = geometry_msgs.msg.TransformStamped()
        t.header.frame_id = trans.get('header', {}).get('frame_id', 'base_link')
        t.child_frame_id = trans.get('child_frame_id', 'camera_link')

        tr = trans.get('transform', {}).get('translation', trans)
        rot = trans.get('transform', {}).get('rotation', trans)

        t.transform.translation.x = float(tr.get('x', 1.0))
        t.transform.translation.y = float(tr.get('y', -0.6))
        t.transform.translation.z = float(tr.get('z', 0.55))
        t.transform.rotation.x = float(rot.get('x', 0.0))
        t.transform.rotation.y = float(rot.get('y', 0.0))
        t.transform.rotation.z = float(rot.get('z', 0.0))
        t.transform.rotation.w = float(rot.get('w', 1.0))

        self.br.sendTransform(t)
        self.get_logger().info(f'从标定文件加载: {filepath}')

    def use_default_params(self):
        self.declare_parameter('x', 1.0)
        self.declare_parameter('y', -0.6)
        self.declare_parameter('z', 0.55)
        t = geometry_msgs.msg.TransformStamped()
        t.header.frame_id = 'base_link'
        t.child_frame_id = 'camera_link'
        t.transform.translation.x = self.get_parameter('x').value
        t.transform.translation.y = self.get_parameter('y').value
        t.transform.translation.z = self.get_parameter('z').value
        t.transform.rotation.w = 1.0
        self.br.sendTransform(t)

def main(args=None):
    rclpy.init(args=args)
    node = CalibrationTFPublisher()
    rclpy.spin(node)
    node.destroy_node()

```

```
rcipy.shutdown()

if __name__ == '__main__':
    main()
```

19.3.12 配置 setup.py 并编译运行

```
entry_points={
    'console_scripts': [
        'aruco_detector = ch19_aruco.aruco_detector:main',
        'calibration_tf_publisher = ch19_aruco.calibration_tf_publisher:main',
    ],
},
},
cd ~/ros2_arm_ws
colcon build --packages-select ch19_aruco
source install/setup.bash

# 发布标定结果 TF
ros2 run ch19_aruco calibration_tf_publisher \
    --ros-args -p calibration_file:=~/ros/easy_handeye/xarm_handeye_calibration_xxx.yaml

# 验证 TF
ros2 run tf2_ros tf2_echo base_link camera_link
```

实验结果与分析

- usb_cam 成功驱动摄像头并发布/image_raw 话题
- cv_bridge 实现了 ROS Image 和 OpenCV 图像格式的双向转换
- Canny 边缘检测可以提取图像中的边缘特征
- 相机标定获取了内参矩阵和畸变系数, 可用于图像矫正
- HSV 颜色检测通过设定 H/S/V 范围可以准确提取目标颜色的区域
- YOLOv8 模型可以实时检测 80 类 COCO 数据集目标
- 颜色检测结合形状分析可以更准确地进行目标识别
- YOLO 的推理速度取决于模型大小和硬件配置
- ArUco 检测节点成功识别标签并估计 6D 位姿
- TF 广播使得标签坐标系在 ROS2 系统中可用
- easy_handeye 完成了眼在手外标定, 建立了 base_link 到 camera_link 的变换关系
- 标定精度受标定板质量和采样姿势多样性影响
- 参考代码 vision_detection_lab 的四个节点与本章 19.1~19.3 的对应练习一致, 可在无摄像头时用仿真图像流验证

思考题

320. 相机标定中每个进度条(X/Y/Size/Skew)代表什么? 为什么需要多样化的标定板姿态?
321. 畸变矫正对后续视觉任务(如目标检测、位姿估计)有什么影响?
322. BGR8 和 RGB8 格式有什么区别? 在 OpenCV 中应使用哪种?
323. 如何在没有标定板的情况下估算相机内参?
324. HSV 颜色空间相比 RGB 有什么优势? 为什么颜色检测通常使用 HSV?
325. YOLOv8 的 n/s/m/l/x 模型之间有什么区别? 如何选择合适的模型?
326. torch 模型和 ONNX 模型在推理性能上有何差异?
327. 如何训练自定义的 YOLOv8 模型来检测特定物体?
328. ArUco 和 AprilTag 有什么区别? 各有什么优缺点?
329. 眼在手外(eye-on-hand)和眼在手上(eye-in-hand)标定的区别和适用场景?
330. 标定精度受哪些因素影响? 如何提高标定精度?
331. 在标定过程中, 如何判断样本点是否有效?

第 20 章 行为树与恢复行为

学习目标

- 理解行为树(Behavior Tree)的基本原理和节点类型
- 掌握 Nav2 中 NavigateToPose 行为树的结构
- 熟悉各种恢复行为(Spin/Backup/Wait)的配置和使用
- 能够自定义行为树和恢复行为
- 掌握行为树的调试和优化方法

20.1 行为树基础

20.1.1 行为树概述

行为树 (Behavior Tree, BT) 是一种用于控制智能体决策流程的树形结构模型。相比传统的有限状态机 (FSM)，行为树具有更好的模块化、可读性和可扩展性。

行为树的起源：

- 最初用于游戏 AI (Halo 2, 2005)
- 后被引入机器人领域 (Nav2, 2019)
- 现已成为机器人任务编排的标准工具

行为树 vs 有限状态机：

特性	行为树 (BT)	有限状态机 (FSM)
状态表示	树形节点	状态图
控制流	显式 (树结构)	隐式 (转换条件)
模块化	高 (节点可独立)	低 (转换耦合)
可重用性	节点可跨树重用	状态难重用
调试	可视化 XML	日志跟踪
扩展性	添加新节点即可	需修改所有转换
容错	自然支持 (Fallback)	需显式设计

20.1.2 节点类型

行为树包含四种核心节点类型：

控制节点：

Sequence (→)：

顺序执行子节点

- 全部返回 SUCCESS → 返回 SUCCESS
- 任一返回 FAILURE → 停止并返回 FAILURE
- 常用于"必须全部完成的步骤序列"

Fallback (?):

尝试子节点（优先选择）

- 任一返回 SUCCESS → 返回 SUCCESS
- 全部返回 FAILURE → 返回 FAILURE
- 常用于"尝试多种方法的备选方案"

Parallel (⇔)：

并行执行所有子节点

- 达到成功阈值 → 返回 SUCCESS
- 达到失败阈值 → 返回 FAILURE
- 常用于"同时执行多项任务"

Decorator (装饰节点)：

修改子节点的行为

- Inverter：反转结果
- Repeater：重复执行
- RateController：控制执行频率
- Timeout：超时控制

执行节点：

Action (动作节点)：

执行具体任务

- 导航、旋转、后退、等待
- 返回 SUCCESS / FAILURE / RUNNING

Condition (条件节点)：

检查条件

- 电池电量、障碍物检测、是否到达
- 返回 SUCCESS / FAILURE（不会返回 RUNNING）

20.1.3 节点状态

行为树节点执行后返回三种状态：

状态	含义	后续处理
SUCCESS	节点执行成功	继续执行下一个节点
FAILURE	节点执行失败	触发 Fallback 或终止
RUNNING	节点正在执行	下次 tick 继续执行

```
执行示例：
Sequence → Step1(SUCCESS) → Step2(RUNNING) → ...
           → Step1(SUCCESS) → Step2(SUCCESS) → 返回 SUCCESS

Fallback → Option1(FAILURE) → Option2(SUCCESS) → 返回 SUCCESS
           → Option1(FAILURE) → Option2(FAILURE) → Option3(FAILURE) → 返回 FAILURE
```

20.2 Nav2 行为树架构

20.2.1 NavigateToPose 行为树

Nav2 默认的导航到目标行为树 `navigate_to_pose.xml`：

```
<!-- navigate_to_pose.xml (完整版) -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithRecovery">

      <!-- 全局规划阶段 -->
      <RateController hz="1.0">
        <RecoveryNode number_of_retries="1" name="ComputePathToPose">
          <ComputePathToPose
            goal="{goal}"
            path="{path}"
            planner_id="GridBased"
            error_code="{error_code}"/>
          <ClearEntireCostmap
            service_name="global_costmap/clear_entirely"/>
        </RecoveryNode>
      </RateController>

      <!-- 路径跟踪阶段 -->
      <RateController hz="10.0">
        <RecoveryNode number_of_retries="1" name="FollowPath">
          <FollowPath
            path="{path}"
            controller_id="FollowPath"
            error_code="{error_code}"/>
        </RecoveryNode>
      </RateController>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

```

    <Sequence name="RecoveryActions">
      <Spin spin_dist="1.57"/>
      <Wait wait_duration="1.0"/>
      <BackUp backup_dist="0.15"
        backup_speed="0.05"/>
      <ClearEntireCostmap
        service_name="local_costmap/clear_entirely"/>
    </Sequence>
  </RecoveryNode>
</RateController>

<!-- 全局重规划阶段 -->
<RateController hz="1.0">
  <RecoveryNode number_of_retries="1" name="GlobalReplan">
    <ComputePathToPose
      goal="{goal}"
      path="{replan_path}"
      planner_id="GridBased"
      error_code="{error_code}"/>
    <Sequence name="GlobalRecoveryActions">
      <ClearEntireCostmap
        service_name="global_costmap/clear_entirely"/>
      <ClearEntireCostmap
        service_name="local_costmap/clear_entirely"/>
    </Sequence>
  </RecoveryNode>
</RateController>

</PipelineSequence>
</BehaviorTree>
</root>

```

20.2.2 行为树执行流程

打开 nav2_params.yaml 选择行为树:

```

bt_navigator:
  ros__parameters:
    default_bt_xml_filename: "navigate_to_pose.xml" # 默认树
    default_server_timeout: 20 # Action 超时(s)
    plugin_lib_names: # 行为树插件库
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_recovery_node_bt_node
      - nav2_pipeline_sequence_bt_node
      - nav2_rate_controller_bt_node

# 指定使用特定行为树
ros2 service call /bt_navigator/change_behavior_tree \
  nav2_msgs/srv/ChangeBehaviorTree \
  "{behavior_tree: 'navigate_to_pose.xml'}"

```

20.2.3 行为树节点连接

```
import rclpy
from rclpy.node import Node
from nav2_msgs.action import NavigateToPose
from rclpy.action import ActionClient

class BehaviorTreeClient(Node):
    """行为树 Action 客户端"""
    def __init__(self):
        super().__init__('bt_client')

        self.navigate_client = ActionClient(
            self, NavigateToPose, 'navigate_to_pose')

        # 检查可用行为树
        self.bt_list_client = self.create_client(
            nav2_msgs.srv.GetBehaviors, '/bt_navigator/get_behavior_tree_list')

    def get_available_bt(self) -> list:
        """获取可用的行为树列表"""
        while not self.bt_list_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待 BT 列表服务...')

        future = self.bt_list_client.call_async(
            nav2_msgs.srv.GetBehaviors.Request())
        rclpy.spin_until_future_complete(self, future)
        return future.result().behaviors.trees

    def navigate_with_bt(self, x: float, y: float, yaw: float,
                        bt_file: str = ''):
        """使用指定行为树导航"""
        goal = NavigateToPose.Goal()
        goal.pose.header.frame_id = 'map'
        goal.pose.pose.position.x = x
        goal.pose.pose.position.y = y
        goal.pose.pose.orientation.z = math.sin(yaw / 2.0)
        goal.pose.pose.orientation.w = math.cos(yaw / 2.0)

        if bt_file:
            goal.behavior_tree = bt_file

        self.navigate_client.wait_for_server()
        self.send_goal_future = self.navigate_client.send_goal_async(
            goal, feedback_callback=self.feedback_callback)
        self.send_goal_future.add_done_callback(
            self.goal_response_callback)
```

20.3 恢复行为详解

20.3.1 恢复行为概述

当导航过程出现异常时，Nav2 会触发恢复行为尝试解决问题：

恢复行为的触发条件：

- 332. 路径规划失败（无可行路径）
- 333. 路径跟踪失败（被障碍物卡住）
- 334. 目标不可达（被障碍物包围）
- 335. 代价地图异常（动态障碍物快速变化）

默认恢复行为序列：

```
RecoveryNode（最多重试次数）
├─ 主行为（ComputePathToPose / FollowPath）
└─ 恢复序列（当主行为失败时）
    ├─ Spin（原地旋转）
    ├─ Wait（等待）
    ├─ BackUp（后退）
    └─ ClearEntireCostmap（清除代价地图）
```

20.3.2 Spin 恢复行为

Spin 让机器人原地旋转一定角度，重新感知周围环境：

```
import rclpy
from rclpy.action import ActionClient
from nav2_msgs.action import Spin
from geometry_msgs.msg import Quaternion
import math

class SpinRecovery:
    """Spin 恢复行为"""
    def __init__(self, node):
        self.action_client = ActionClient(node, Spin, '/spin')

    def send_spin(self, target_yaw: float = 1.57,
                  time_allowance: float = 5.0) -> bool:
        """
        发送旋转命令

        参数:
            target_yaw: 目标旋转角度 (rad)
            time_allowance: 允许执行时间 (s)
        """
        goal = Spin.Goal()
        goal.target.yaw = target_yaw
        goal.time_allowance = rclpy.duration.Duration(
            seconds=time_allowance).to_msg()

        self.action_client.wait_for_server()
        future = self.action_client.send_goal_async(goal)
```

```

rclpy.spin_until_future_complete(None, future)

goal_handle = future.result()
if not goal_handle.accepted:
    return False

result_future = goal_handle.get_result_async()
rclpy.spin_until_future_complete(None, result_future)

result = result_future.result()
return result.status == 4 # SUCCEEDED

```

Spin 配置:

```

behavior_server:
  ros__parameters:
    behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]

  spin:
    plugin: "nav2_behaviors/Spin"
    simulate_ahead_time: 2.0      # 前向模拟时间(s)
    max_rotational_vel: 1.0       # 最大旋转速度(rad/s)
    min_rotational_vel: 0.1       # 最小旋转速度(rad/s)
    rotational_acc_lim: 3.0       # 旋转加速度(rad/s²)
    ctrl_rad_step: 0.01          # 控制步长(rad)

```

20.3.3 BackUp 恢复行为

BackUp 让机器人沿直线后退一段距离:

```

from nav2_msgs.action import BackUp

class BackUpRecovery:
    """BackUp 恢复行为"""
    def __init__(self, node):
        self.action_client = ActionClient(node, BackUp, '/backup')

    def send_backup(self, distance: float = 0.15,
                   speed: float = 0.05) -> bool:
        """
        发送后退命令

        参数:
            distance: 后退距离 (m)
            speed: 后退速度 (m/s)
        """
        goal = BackUp.Goal()
        goal.target.x = -distance      # 负值表示后退
        goal.speed = speed
        goal.time_allowance = rclpy.duration.Duration(
            seconds=distance / speed + 1.0).to_msg()

        self.action_client.wait_for_server()
        future = self.action_client.send_goal_async(goal)

```



```

rclpy.spin_until_future_complete(None, future)

goal_handle = future.result()
if not goal_handle.accepted:
    return False

result_future = goal_handle.get_result_async()
rclpy.spin_until_future_complete(None, result_future)

result = result_future.result()
return result.status == 4

```

BackUp 配置:

```

backup:
  plugin: "nav2_behaviors/BackUp"
  simulate_ahead_time: 2.0
  max_linear_vel: 0.1          # 最大后退速度(m/s)
  min_linear_vel: 0.05         # 最小后退速度(m/s)
  linear_acc_lim: 0.5          # 加速度(m/s²)
  ctrl_rad_step: 0.01

```

20.3.4 Wait 和 Drive on Heading

Wait (等待) :

```

wait:
  plugin: "nav2_behaviors/Wait"
  simulate_ahead_time: 2.0
  max_linear_vel: 0.0
  max_rotational_vel: 0.0

```

Drive on Heading (沿当前方向前进) :

```

drive_on_heading:
  plugin: "nav2_behaviors/DriveOnHeading"
  simulate_ahead_time: 2.0
  max_linear_vel: 0.1
  min_linear_vel: 0.05
  linear_acc_lim: 0.5

```

20.3.5 手动触发恢复行为

```

# 触发 Spin (旋转 1.57 rad)
ros2 action send_goal /spin nav2_msgs/action/Spin \
  "{target: {yaw: 1.57}}"

# 触发 Backup (后退 0.3m)
ros2 action send_goal /backup nav2_msgs/action/BackUp \
  "{target: {x: -0.3}, speed: 0.05}"

# 清除全局代价地图
ros2 service call /global_costmap/clear_entirely \
  nav2_msgs/srv/ClearEntireCostmap

# 清除局部代价地图

```

```
ros2 service call /local_costmap/clear_entirely \
nav2_msgs/srv/ClearEntireCostmap
```

20.4 自定义行为树

20.4.1 创建自定义 XML 行为树

```
<!-- custom_navigation.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="CompleteNavigation">

      <!-- 步骤 1: 检查电池电量 -->
      <BatteryCheck
        min_battery_level="0.2"
        battery_topic="/battery_status"/>

      <!-- 步骤 2: 计算全局路径 -->
      <ComputePathToPose
        goal="{goal}"
        path="{path}"/>

      <!-- 步骤 3: 执行路径跟踪（带恢复） -->
      <RecoveryNode number_of_retries="2">
        <FollowPath path="{path}"/>
        <Sequence>
          <!-- 恢复策略 1: 后退清除 -->
          <BackUp backup_dist="0.2"/>
          <ClearEntireCostmap
            service_name="local_costmap/clear_entirely"/>
          <!-- 恢复策略 2: 旋转扫描 -->
          <Spin spin_dist="3.14"/>
          <ClearEntireCostmap
            service_name="local_costmap/clear_entirely"/>
          <!-- 恢复策略 3: 重规划 -->
          <ClearEntireCostmap
            service_name="global_costmap/clear_entirely"/>
          <ComputePathToPose
            goal="{goal}"
            path="{path}"/>
        </Sequence>
      </RecoveryNode>

      <!-- 步骤 4: 到达后执行任务 -->
      <Sequence name="PostArrival">
        <Wait wait_duration="2.0"/>
        <!-- 到达后发通知 -->
        <PublishTopic topic="/arrival_signal" message="arrived"/>
      </Sequence>

    </Sequence>
  </BehaviorTree>
</root>
```

20.4.2 Python 自定义行为树

```
import py_trees
import py_trees_ros

class BatteryCheck(py_trees.behaviour.Behaviour):
    """自定义行为：电池电量检查"""
    def __init__(self, name: str = "BatteryCheck",
                 min_level: float = 0.2,
                 topic: str = "/battery_status"):
        super().__init__(name)
        self.min_level = min_level
        self.topic = topic
        self.battery_level = None

    def setup(self, **kwargs):
        """设置订阅"""
        self.blackboard = py_trees.blackboard.Blackboard()

    def initialise(self):
        """初始化（每次执行时调用）"""
        self.battery_level = self.blackboard.get(self.topic)

    def update(self) -> py_trees.common.Status:
        """执行检查"""
        if self.battery_level is None:
            return py_trees.common.Status.FAILURE

        if self.battery_level > self.min_level:
            self.logger.info(f'电池电量充足: {self.battery_level:.1%}')
            return py_trees.common.Status.SUCCESS
        else:
            self.logger.warning(f'电池电量不足: {self.battery_level:.1%}')
            return py_trees.common.Status.FAILURE

    def terminate(self, new_status):
        """清理"""
        pass

class NavigateToPoseAction(py_trees_ros.actions.ActionClient):
    """导航动作客户端包装"""
    def __init__(self, name: str = "NavigateToPose"):
        super().__init__(
            name=name,
            action_type=NavigateToPose,
            action_name="navigate_to_pose",
            action_goal=NavigateToPose.Goal()
        )

def create_custom_behavior_tree() -> py_trees.trees.BehaviourTree:
    """创建自定义行为树"""
    root = py_trees.composites.Sequence("MainSequence", memory=True)
```

```

# 电池检查
battery_check = BatteryCheck()
root.add_child(battery_check)

# 导航 (带恢复的备选)
navigation = py_trees.composites.Selector("Navigation", memory=True)

# 正常导航
normal_nav = py_trees.composites.Sequence("NormalNav", memory=True)
normal_nav.add_child(NavigateToPoseAction("GoToGoal"))

# 恢复导航 (清除代价地图后重试)
recovery_nav = py_trees.composites.Sequence("RecoveryNav", memory=True)
recovery_nav.add_child(
    py_trees_ros.actions.ServiceClient(
        name="ClearCostmap",
        service_type=ClearEntireCostmap,
        service_name="/global_costmap/clear_entirely"
    )
)
recovery_nav.add_child(NavigateToPoseAction("GoToGoalRetry"))

navigation.add_child(normal_nav)
navigation.add_child(recovery_nav)

root.add_child(navigation)

return py_trees.trees.BehaviourTree(root)

def run_custom_bt():
    """运行自定义行为树"""
    tree = create_custom_behavior_tree()
    tree.setup(timeout=10)

    running = True
    while running:
        tree.tick()
        py_trees.console.log_snapshot_ascii(tree)
        time.sleep(0.1)

```

20.4.3 装饰节点使用

```

<!-- 装饰节点应用示例 -->
<BehaviorTree>
  <Sequence>
    <!-- 带超时控制的导航 -->
    <Timeout msec="30000">
      <ComputePathToPose goal="{goal}" path="{path}"/>
    </Timeout>

    <!-- 带频率控制的路径跟踪 -->
    <RateController hz="10.0">
      <FollowPath path="{path}"/>
    </RateController>

```

```

<!-- 重复检查直到成功 -->
<RepeatUntilSuccess num_attempts="5">
  <Condition condition="is_goal_reached"/>
</RepeatUntilSuccess>

<!-- 反转条件 -->
<Inverter>
  <Condition condition="obstacle_detected"/>
</Inverter>
</Sequence>
</BehaviorTree>

```

20.5 恢复行为配置与优化

20.5.1 恢复行为配置完整示例

```

behavior_server:
  ros__parameters:
    behavior_plugins: [
      "spin", "backup", "drive_on_heading",
      "wait", "assisted_teleop"
    ]
    local_costmap_topic: "/local_costmap/costmap"
    global_costmap_topic: "/global_costmap/costmap"
    cycle_frequency: 10.0

    spin:
      plugin: "nav2_behaviors/Spin"
      simulate_ahead_time: 2.0
      max_rotational_vel: 1.0
      min_rotational_vel: 0.1
      rotational_acc_lim: 3.0

    backup:
      plugin: "nav2_behaviors/BackUp"
      simulate_ahead_time: 2.0
      max_linear_vel: 0.1
      min_linear_vel: 0.05
      linear_acc_lim: 0.5

    drive_on_heading:
      plugin: "nav2_behaviors/DriveOnHeading"
      simulate_ahead_time: 2.0
      max_linear_vel: 0.1
      min_linear_vel: 0.05

    wait:
      plugin: "nav2_behaviors/Wait"
      simulate_ahead_time: 2.0

    assisted_teleop:
      plugin: "nav2_behaviors/AssistedTeleop"
      simulate_ahead_time: 2.0
      max_linear_vel: 0.1

```

20.5.2 恢复行为策略

```
class RecoveryStrategy:
    """恢复策略管理器"""
    def __init__(self, node):
        self.node = node
        self.spin_client = ActionClient(node, Spin, '/spin')
        self.backup_client = ActionClient(node, BackUp, '/backup')
        self.recovery_sequence = [
            {'name': 'spin', 'angle': 1.57},
            {'name': 'wait', 'duration': 1.0},
            {'name': 'backup', 'distance': 0.15},
            {'name': 'spin', 'angle': 3.14},
            {'name': 'backup', 'distance': 0.3},
            {'name': 'wait', 'duration': 2.0},
        ]
        self.current_step = 0

    def attempt_recovery(self) -> bool:
        """执行恢复序列"""
        for step in self.recovery_sequence:
            success = self._execute_step(step)
            if success:
                self.node.get_logger().info(
                    f'恢复成功: {step["name"]}')
                self.current_step = 0
                return True

        self.node.get_logger().error('所有恢复行为失败')
        return False

    def _execute_step(self, step: dict) -> bool:
        """执行单个恢复步骤"""
        self.node.get_logger().info(
            f'执行恢复: {step["name"]}')

        if step['name'] == 'spin':
            return self._do_spin(step['angle'])
        elif step['name'] == 'backup':
            return self._do_backup(step['distance'])
        elif step['name'] == 'wait':
            return self._do_wait(step['duration'])

        return False

    def _do_spin(self, angle: float) -> bool:
        goal = Spin.Goal()
        goal.target.yaw = angle
        self.spin_client.wait_for_server()
        future = self.spin_client.send_goal_async(goal)
        # ... (等待结果)
        return True

    def _do_backup(self, distance: float) -> bool:
        goal = BackUp.Goal()
        goal.target.x = -distance
        self.backup_client.wait_for_server()
```

```

        future = self.backup_client.send_goal_async(goal)
        return True

    def _do_wait(self, duration: float) -> bool:
        self.node.get_logger().info(f'等待 {duration}s')
        time.sleep(duration)
        return True

```

20.5.3 恢复行为触发条件优化

```

# 代价地图清除配置
global_costmap:
  global_costmap:
    ros__parameters:
      # 当机器人位置与已知地图的代价超过阈值时触发清除
      footprint_padding: 0.01
      always_send_full_costmap: true

local_costmap:
  local_costmap:
    ros__parameters:
      # 滚动窗口的清除策略
      rolling_window: true
      width: 4
      height: 4

```

20.5.4 行为树调试

```

# 查看当前行为树状态
ros2 topic echo /bt_navigator/bt_status

# 查看导航状态（包括恢复行为执行情况）
ros2 topic echo /navigate_to_pose/_action/status

# 查看行为树节点执行日志
ros2 run rqt_bt_monitor rqt_bt_monitor

# 手动切换行为树
ros2 param set /bt_navigator \
  default_bt_xml_filename "custom_navigation.xml"

```

20.6 实战案例

20.6.1 机器人巡逻行为树

```

<!-- patrol_bt.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <!-- 全局重定位 -->
      <GlobalLocalization/>

      <!-- 巡逻循环 -->

```

```

<Repeat num_cycles="-1"> <!-- 无限循环 -->
  <Sequence>
    <!-- 前往航点 1 -->
    <ComputePathToPose goal="{waypoint1}" path="{path}"/>
    <FollowPath path="{path}"/>

    <!-- 到达后停留 -->
    <Wait wait_duration="5.0"/>

    <!-- 前往航点 2 -->
    <ComputePathToPose goal="{waypoint2}" path="{path}"/>
    <FollowPath path="{path}"/>

    <Wait wait_duration="5.0"/>

    <!-- 返回航点 1 -->
    <ComputePathToPose goal="{waypoint1}" path="{path}"/>
    <FollowPath path="{path}"/>
  </Sequence>
</Repeat>
</Sequence>
</BehaviorTree>
</root>

```

20.6.2 多机器人调度集成

```

import rclpy
from rclpy.node import Node
from nav2_msgs.action import NavigateToPose
import yaml

class MultiRobotScheduler(Node):
    """多机器人调度器 - 集成行为树"""
    def __init__(self):
        super().__init__('multi_robot_scheduler')

        # 注册机器人客户端
        self.robots = {}
        robot_names = ['robot1', 'robot2', 'robot3']
        for name in robot_names:
            self.robots[name] = ActionClient(
                self, NavigateToPose, f'/{name}/navigate_to_pose')

        # 任务队列
        self.task_queue = []
        self.load_tasks('tasks.yaml')

    def load_tasks(self, task_file: str):
        """加载任务配置"""
        with open(task_file, 'r') as f:
            tasks = yaml.safe_load(f)
            self.task_queue = tasks['tasks']

    def dispatch_task(self, robot_name: str, task: dict):
        """派发任务到机器人"""
        if robot_name not in self.robots:

```



```

        return

    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.pose.position.x = task['x']
    goal.pose.pose.position.y = task['y']

    # 选择行为树
    if task.get('recovery_strategy') == 'aggressive':
        goal.behavior_tree = 'aggressive_recovery.xml'
    else:
        goal.behavior_tree = 'default_navigation.xml'

    client = self.robots[robot_name]
    client.wait_for_server()
    client.send_goal_async(goal)

    self.get_logger().info(
        f'派发任务给 {robot_name}: ({task["x"]}, {task["y"]})')

```

课后练习

336. **原理题:** 说明行为树中 Sequence 和 Fallback 节点的区别, 列举在实际导航中哪些场景适合用 Sequence、哪些适合用 Fallback。
337. **配置题:** 编写一个自定义的行为树 XML, 实现以下功能: 先检查电池电量, 若电量充足则直接导航, 若电量不足则先前往充电桩充电再导航。
338. **编程题:** 实现一个 Python 自定义恢复行为 "SpiralSearch": 当导航失败时, 机器人以螺旋轨迹搜索周围环境, 尝试找到可行路径。
339. **分析题:** 分析 Nav2 默认行为树的恢复策略设计, 说明为什么先 Spin 再 BackUp 最后清除代价地图, 解释这种恢复顺序的合理性。
340. **操作题:** 在仿真环境中制造导航卡住场景 (如障碍物包围), 观察恢复行为的执行过程, 记录每种恢复行为的执行效果和耗时。
341. **设计题:** 设计一个分层的恢复行为策略: 轻度故障 (一次规划失败) → 快速 Spin 重试, 中度故障 (连续失败) → BackUp+重规划, 重度故障 (多次失败) → 完全停止+请求人工干预。

实验 20:

第 20 章 实验：视觉大模型与 ROS2 应用

****对应理论章节**:** 第 33 章《视觉大模型集成》 ****实验课时**:** 2 课时 ****实验代码**:**
``src/lab_code/ch20_lab/`` (本章仅提供设计说明占位, 无参考实现包, 练习需自行实现)

实验目标

- 掌握调用视觉大模型(VLM)API 进行图像分析的方法
- 学会将视觉能力封装为 ROS2 服务
- 理解后端解耦设计: 通过参数/环境变量选择离线 mock 或真实模型 provider
- 实现基于 VLM 的场景描述和目标检测
- 理解 Prompt Engineering 在机器人场景中的应用

实验环境

- ROS 2 Jazzy
- USB 摄像头
- Python openai 库 (可选: pillow、requests)
- 默认使用离线 mock 后端: **不需要任何 API Key 即可完成全部测试**
- 可选接入真实 provider: OpenAI / 通义千问 / 本地 Ollama, 密钥通过环境变量提供

总体设计 (依据 ``src/lab_code/ch20_lab/README.md``)

本实验演示将视觉模型能力封装为 ROS2 服务, 核心设计要点:

342. **服务化封装:** 视觉能力 (场景描述、目标检测等) 统一封装为 ROS2 服务, 输入图像与查询文本, 输出结构化结果, 上层节点通过服务调用获取视觉理解结果, 不直接依赖模型 SDK。

343. **默认离线 mock 后端**: 默认后端为离线模拟 (mock) 实现, 返回模拟的结构化结果, 因而不需要 API 密钥即可完成服务链路搭建与测试, 适合课堂教学环境。
344. **provider 可选切换**: 连接真实模型时, 通过 ROS 参数选择相应提供商 (如 `api_type` 参数选择 `openai/qwen/ollama`), 并从环境变量读取密钥 (如 `OPENAI_API_KEY`、`DASHSCOPE_API_KEY`), 服务接口保持不变。
345. **结构化输出**: 以 JSON 格式返回场景描述、物体列表、建议动作等字段, 便于机器人程序解析。

实验步骤

20.1 安装依赖和配置

```
pip install openai pillow requests

# 默认离线 mock 后端无需任何密钥即可测试
# 如需接入真实模型, 选择一种配置:
# OpenAI:
export OPENAI_API_KEY="sk-your-api-key"

# 或 通义千问:
export DASHSCOPE_API_KEY="sk-your-dashscope-key"

# 或 本地 Ollama:
# ollama pull llava
# ollama serve
```

20.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch20_vlm --build-type ament_python --dependencies rclpy sensor_msgs std_msgs cv_bridge
cd ch20_vlm
mkdir -p ch20_vlm
```

20.3 编写 VLM 场景描述节点

创建 `ch20_vlm/scene_describer.py`。该节点通过 `api_type` 参数选择后端 provider, 默认可在无密钥环境下用 mock 后端替换 `analyze_image` 的实现进行测试:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import openai
import base64
```

```

import json
import cv2
import os
import threading
import time

class VLMServer(Node):
    def __init__(self):
        super().__init__('vlm_server')
        self.bridge = CvBridge()
        self.latest_image = None

        self.declare_parameter('api_type', 'openai')
        self.declare_parameter('model', 'gpt-4o-mini')
        self.declare_parameter('auto_analyze', False)

        api_type = self.get_parameter('api_type').value
        model = self.get_parameter('model').value
        self.auto_analyze = self.get_parameter('auto_analyze').value

        if api_type == 'qwen':
            self.client = openai.OpenAI(
                api_key=os.getenv('DASHSCOPE_API_KEY', ''),
                base_url='https://dashscope.aliyuncs.com/compatible-mode/v1'
            )
            if model == 'gpt-4o-mini':
                model = 'qwen-vl-plus'
        elif api_type == 'ollama':
            self.client = openai.OpenAI(
                base_url='http://localhost:11434/v1',
                api_key='ollama'
            )
            if model == 'gpt-4o-mini':
                model = 'llava'
        else:
            self.client = openai.OpenAI(
                api_key=os.getenv('OPENAI_API_KEY', '')
            )

        self.model = model

        self.image_sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )
        self.query_sub = self.create_subscription(
            String, '/vlm/query', self.query_cb, 10
        )
        self.desc_pub = self.create_publisher(String, '/vlm/description', 10)
        self.status_pub = self.create_publisher(String, '/vlm/status', 10)

        self.get_logger().info(f'VLM 场景描述器已启动 (model={model})')

        if self.auto_analyze:
            self.analysis_timer = self.create_timer(10.0, self.auto_analyze_cb)

    def image_cb(self, msg):
        try:
            self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:

```

```

        pass

    def query_cb(self, msg):
        if self.latest_image is None:
            self.status_pub.publish(String(data='等待图像...'))
            return
        threading.Thread(target=self.process_query, args=(msg.data,), daemon=True).start()

    def auto_analyze_cb(self):
        if self.latest_image is not None:
            threading.Thread(
                target=self.process_query,
                args=('请描述当前场景中的物体和布局',),
                daemon=True
            ).start()

    def process_query(self, query):
        self.get_logger().info(f'VLM 查询: {query}')
        self.status_pub.publish(String(data=f'分析中...'))

        result = self.analyze_image(query)

        if result:
            self.desc_pub.publish(String(data=json.dumps(result, ensure_ascii=False, indent=2)))
            self.get_logger().info(f'场景: {result.get("description", "")[:100]}...')
            self.status_pub.publish(String(data='分析完成'))
        else:
            self.status_pub.publish(String(data='分析失败'))

    def analyze_image(self, query):
        h, w = self.latest_image.shape[:2]
        scale = min(512 / max(h, w), 1.0)
        if scale < 1.0:
            img = cv2.resize(self.latest_image, (int(w * scale), int(h * scale)))
        else:
            img = self.latest_image

        ret, jpeg = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 70])
        img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
        data_url = f'data:image/jpeg;base64,{img_b64}'

        system_prompt = """你是一个机器人视觉助手。请分析图像并回答用户问题。

返回 JSON 格式:
{
    "description": "场景的中文描述(50 字以内)",
    "objects": [{"name": "物体名", "position": "位置描述", "count": 1}],
    "actions": ["可行的操作建议"],
    "answer": "对用户问题的直接回答"
}"""

        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[
                    {"role": "system", "content": system_prompt},

```

```

        {
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "image_url", "image_url": {"url": data_url, "detail": "low"}},
            ]
        }
    ],
    max_tokens=500,
    response_format={"type": "json_object"},
)
content = response.choices[0].message.content.strip()
return json.loads(content)
except json.JSONDecodeError as e:
    self.get_logger().error(f'JSON 解析失败: {e}')
    return {"description": content if 'content' in dir() else "解析错误", "objects": []}
except Exception as e:
    self.get_logger().error(f'VLM 调用失败: {e}')
    return None

def run_test_queries(self):
    time.sleep(2)
    test_queries = [
        "桌面上有哪些物体? 请列出所有可见物品",
        "描述当前场景的布局和安全状况",
        "前方是否有障碍物? 机器人的路径是否畅通?",
    ]
    for q in test_queries:
        self.get_logger().info(f'=== 测试: {q} ===')
        result = self.analyze_image(q)
        if result:
            self.get_logger().info(f'回答: {result.get("answer", "")}')
            time.sleep(3)

def main(args=None):
    rclpy.init(args=args)
    node = VLMServer()
    node.run_test_queries()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

****练习提示 (离线 mock 后端)**:** 在无 API Key 的教学环境中, 可将
`analyze\_image` 中的网络调用替换为返回固定/模拟 JSON 的本地实现 (mock 后端),
例如返回 `{"description": "mock 场景", "objects": [{"name": "bottle", "position": "`

桌面中央", "count": 1}}, "actions": ["抓取瓶子"], "answer": "mock 回答"}}, 先验证订阅、服务调用与结果发布的完整链路, 再切换到真实 provider。

20.4 编写 VLM 目标检测节点

创建 ch20_vlm/vlm_detector.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose
from std_msgs.msg import String
from cv_bridge import CvBridge
import openai
import base64
import json
import cv2
import os
import numpy as np

class VLMBasedDetector(Node):
    def __init__(self):
        super().__init__('vlm_based_detector')
        self.bridge = CvBridge()

        self.client = openai.OpenAI(
            api_key=os.getenv('OPENAI_API_KEY', '')
        )
        self.declare_parameter('model', 'gpt-4o-mini')
        self.declare_parameter('target_object', 'bottle')
        self.model = self.get_parameter('model').value
        self.target = self.get_parameter('target_object').value

        self.image_sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )
        self.det_pub = self.create_publisher(
            Detection2DArray, '/vlm/detections', 10
        )
        self.annotated_pub = self.create_publisher(
            Image, '/vlm/annotated', 10
        )
        self.get_logger().info(f'VLM 检测器已启动, 目标: {self.target}')

    def image_cb(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            return

        h, w = cv_image.shape[:2]
        scale = min(512 / max(h, w), 1.0)
        if scale < 1.0:
            img = cv2.resize(cv_image, (int(w * scale), int(h * scale)))
        else:
            img = cv_image
```

```

ret, jpeg = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 70])
img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
data_url = f'data:image/jpeg;base64,{img_b64}'

prompt = f"""分析图像。检测"{self.target}"的位置。
图像尺寸: {int(w * scale)}x{int(h * scale)}像素。
只需返回 JSON: {{ "detected": true/false, "objects": [{{ "name": "物体名", "x": 中心 x, "y": 中心 y,
"width": 宽度, "height": 高度, "confidence": 0-1}}]}}"""

try:
    response = self.client.chat.completions.create(
        model=self.model,
        messages=[
            {"role": "system", "content": "你是视觉检测助手。只返回 JSON。"},
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {"type": "image_url", "image_url": {"url": data_url, "detail":
"high"}},
                ]
            }
        ],
        max_tokens=500,
        response_format={"type": "json_object"},
    )
    result = json.loads(response.choices[0].message.content.strip())

    det_msg = Detection2DArray()
    det_msg.header = msg.header
    annotated = cv_image.copy()

    for obj in result.get('objects', []):
        det = Detection2D()
        scale_x = w / (w * scale)
        scale_y = h / (h * scale)
        det.bbox.center.position.x = float(obj['x']) * scale_x
        det.bbox.center.position.y = float(obj['y']) * scale_y
        det.bbox.size_x = float(obj.get('width', 50)) * scale_x
        det.bbox.size_y = float(obj.get('height', 50)) * scale_y
        hyp = ObjectHypothesisWithPose()
        hyp.hypothesis.class_id = obj.get('name', 'unknown')
        hyp.hypothesis.score = float(obj.get('confidence', 0.5))
        det.results.append(hyp)
        det_msg.detections.append(det)

        x1 = int(det.bbox.center.position.x - det.bbox.size_x / 2)
        y1 = int(det.bbox.center.position.y - det.bbox.size_y / 2)
        x2 = int(det.bbox.center.position.x + det.bbox.size_x / 2)
        y2 = int(det.bbox.center.position.y + det.bbox.size_y / 2)
        cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(annotated, f'{{obj["name"]}} {{obj.get("confidence", 0):.2f}}',
            (x1, y1-5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    self.det_pub.publish(det_msg)
    self.annotated_pub.publish(self.bridge.cv2_to_imgmsg(annotated, 'bgr8'))

```



```

        self.get_logger().info(f'VLM 检测: {len(det_msg.detections)} 个物体')

    except Exception as e:
        self.get_logger().error(f'VLM 检测失败: {e}')

def main(args=None):
    rclpy.init(args=args)
    node = VLMBasedDetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

20.5 编写 VLM 任务规划节点

创建 ch20_vlm/task_planner.py:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import openai
import json
import os

class TaskPlanner(Node):
    def __init__(self):
        super().__init__('task_planner')
        self.client = openai.OpenAI(
            api_key=os.getenv('OPENAI_API_KEY', '')
        )
        self.declare_parameter('model', 'gpt-4o-mini')
        self.model = self.get_parameter('model').value

        self.cmd_sub = self.create_subscription(
            String, '/llm/command', self.command_cb, 10
        )
        self.plan_pub = self.create_publisher(String, '/llm/plan', 10)

        self.known_objects = {
            'bottle': '试剂瓶', 'box': '纸箱',
            'cup': '杯子', 'book': '书本',
        }
        self.known_locations = {
            'shelf_A': (2.0, 2.0), 'shelf_B': (-2.0, 2.0),
            'table': (0.5, 0.0), 'station': (0.0, -3.0),
        }
        self.get_logger().info('任务规划器已启动')

    def command_cb(self, msg):
        instruction = msg.data.strip()
        self.get_logger().info(f'指令: {instruction}')
        plan = self.generate_plan(instruction)
        if plan:
            plan_str = json.dumps(plan, ensure_ascii=False, indent=2)

```

```

        self.plan_pub.publish(String(data=plan_str))
        self.get_logger().info(f'规划完成: {len(plan.get("tasks", []))} 步')

    def generate_plan(self, instruction):
        system_prompt = f"""你是机器人任务规划器。将自然语言指令转为结构化任务。
已知物体: {json.dumps(self.known_objects, ensure_ascii=False)}
已知位置: {json.dumps({k: list(v) for k, v in self.known_locations.items()})}
可用操作: navigate, pick, place, scan, wait, detect
返回 JSON: {{"tasks": [{"step": 1, "action": "操作", "target": "目标", "params": {}}]}},
"summary": "摘要"}"""
        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": instruction}
                ],
                temperature=0.1,
                max_tokens=1000,
                response_format={"type": "json_object"},
            )
            content = response.choices[0].message.content.strip()
            return json.loads(content)
        except Exception as e:
            self.get_logger().error(f'规划失败: {e}')
            return None

def main(args=None):
    rclpy.init(args=args)
    node = TaskPlanner()
    node.get_logger().info('发送指令: ros2 topic pub /llm/command std_msgs/String "data: \'去桌子上拿瓶子\'"')
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

20.6 编写 VLM 服务封装

创建 `ch20_vlm/vlm_service.py`。该节点体现本章“视觉能力封装为 ROS2 服务”的核心设计，实际实现时应使用自定义服务类型（请求：图像+查询文本；响应：结构化 JSON 字符串），并将其中的模型调用替换为按参数选择的 mock 或真实 provider:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import openai
import base64
import json
import cv2

```

```

import os

class VLMActionServer(Node):
    def __init__(self):
        super().__init__('vlm_action_server')
        self.bridge = CvBridge()
        self.latest_image = None
        self.client = openai.OpenAI(api_key=os.getenv('OPENAI_API_KEY', ''))

        self.image_sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )

        self.srv = self.create_service(String, '/vlm/analyze', self.handle_request)
        self.result_pub = self.create_publisher(String, '/vlm/analysis_result', 10)
        self.get_logger().info('VLM Action Server ready')

    def image_cb(self, msg):
        try:
            self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            pass

    def handle_request(self, request, response):
        if self.latest_image is None:
            response.data = json.dumps({"error": "No image available"})
            return response

        ret, jpeg = cv2.imencode('.jpg', self.latest_image,
                                [cv2.IMWRITE_JPEG_QUALITY, 70])
        img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
        data_url = f'data:image/jpeg;base64,{img_b64}'

        try:
            resp = self.client.chat.completions.create(
                model='gpt-4o-mini',
                messages=[
                    {"role": "system", "content": "分析场景。返回 JSON:"},
                    {"role": "user", "content": [
                        {"type": "text", "text": request.data},
                        {"type": "image_url", "image_url": {"url": data_url, "detail": "low"}},
                    ]},
                ],
                response_format={"type": "json_object"},
                max_tokens=500,
            )
            result = resp.choices[0].message.content.strip()
            response.data = result
            self.result_pub.publish(String(data=result))
        except Exception as e:
            response.data = json.dumps({"error": str(e)})

        return response

def main(args=None):
    rclpy.init(args=args)
    node = VLMActionServer()
    rclpy.spin(node)

```

```
node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

***注意**：上方 `create\_service(String, ...)` 仅为教学示例写法，`String` 是消息类型而非服务类型；自行实现时应定义 `.srv` 接口（如 `VlmAnalyze.srv`：请求含 `sensor\_msgs/Image image` 与 `string query`，响应含 `string result\_json`）。*

20.7 配置 setup.py 并编译运行

```
entry_points={
    'console_scripts': [
        'scene_describer = ch20_vlm.scene_describer:main',
        'vlm_detector = ch20_vlm.vlm_detector:main',
        'task_planner = ch20_vlm.task_planner:main',
        'vlm_service = ch20_vlm.vlm_service:main',
    ],
},
),
cd ~/ros2_arm_ws
colcon build --packages-select ch20_vlm
source install/setup.bash

# 使用离线mock 后端时无需 export 任何密钥

# 终端 1：启动相机
ros2 run usb_cam usb_cam_node_exe

# 终端 2：场景描述
ros2 run ch20_vlm scene_describer

# 发送查询
ros2 topic pub /vlm/query std_msgs/String "data: '描述场景中的物体'"
ros2 topic echo /vlm/description

# 或 VLM 检测
ros2 run ch20_vlm vlm_detector --ros-args -p target_object:=bottle
rqt_image_view /vlm/annotated
```

接入真实 provider 时（任选其一）：

```
# OpenAI
export OPENAI_API_KEY="sk-your-key"

# 通义千问
export DASHSCOPE_API_KEY="sk-your-key"
ros2 run ch20_vlm scene_describer --ros-args -p api_type:=qwen

# 本地 Ollama（无需密钥）
ros2 run ch20_vlm scene_describer --ros-args -p api_type:=ollama
```

实验结果与分析

- 默认离线 mock 后端可在无 API Key 的环境下验证"订阅图像→服务调用→结果发布"的完整链路
- 接入真实 VLM 后, 可成功分析图像内容并生成结构化场景描述
- GPT-4o-mini 可以识别场景中的物体、位置和安全状况
- ROS2 话题/服务封装使得 VLM 结果可以被其他节点使用, 且 provider 切换对上层透明
- API 调用延迟受网络影响, 通常需要 1-3 秒

思考题

346. VLM 相比传统 CV 方法在目标检测上有什么优势和劣势?
347. Prompt Engineering 如何影响 VLM 的输出质量?
348. 如何减少 VLM API 调用的延迟? 有无本地部署方案?
349. VLM 输出中的幻觉问题如何检测和处理?
350. 离线 mock 后端与真实 VLM 后端的切换对上层调用节点是否透明? 服务接口应如何设计才能做到后端无关?

第 21 章 视觉 SLAM 导论

学习目标

- 理解视觉 SLAM 的基本原理和经典框架
- 掌握 ORB-SLAM2/3 的特征提取与跟踪方法
- 了解 DSO 直接法和 RGB-D SLAM 的原理
- 熟悉视觉特征提取与匹配技术
- 掌握视觉与激光融合 SLAM 的策略

21.1 视觉 SLAM 概述

21.1.1 视觉 SLAM vs 激光 SLAM

特性	视觉 SLAM	激光 SLAM
传感器成本	低（百元级相机）	高（千元-万元级）
信息丰富度	高（纹理、颜色、语义）	低（距离信息）
定位精度	亚米级	毫米级
计算量	大（图像处理）	小
低纹理环境	容易丢失	可正常工作
长走廊场景	可正常工作	容易漂移
地图用途	主要用于定位	可直接用于导航
光照变化	敏感	不敏感
动态物体	需要处理	相对容易过滤

21.1.2 相机类型

单目相机 (Monocular):

- 成本最低，结构简单
- 尺度不确定性（无法直接获取深度）
- 需要运动初始化

- 适合小规模 AR/VR 场景

双目相机 (Stereo):

- 基线提供尺度信息
- 计算量大 (视差匹配)
- 深度范围受基线限制
- 代表: ZED, MYNT EYE

RGB-D 相机:

- 直接获取深度信息
- 室内场景效果好
- 室外受环境光影响
- 代表: Intel RealSense D415/D435, Microsoft Kinect

21.1.3 经典视觉 SLAM 框架



ORB-SLAM 的三个并行线程:

351. **Tracking (跟踪线程):** 实时跟踪当前帧位姿

352. **Local Mapping (局部建图线程)**：局部 BA 优化和地图点管理

353. **Loop Closing (闭环检测线程)**：回环检测和全局优化

21.2 相机模型与 ROS2 视觉接口

21.2.1 针孔相机模型

投影方程：

$$z \cdot [u, v, 1]^T = K \cdot [R|t] \cdot [X, Y, Z, 1]^T$$

内参矩阵 K：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

其中：

- f_x, f_y : 焦距 (像素单位)
- c_x, c_y : 光心坐标

```
import numpy as np

class PinholeCamera:
    """针孔相机模型"""
    def __init__(self, fx: float, fy: float,
                  cx: float, cy: float,
                  width: int = 640, height: int = 480):
        self.fx = fx
        self.fy = fy
        self.cx = cx
        self.cy = cy
        self.width = width
        self.height = height

        self.K = np.array([
            [fx, 0, cx],
            [0, fy, cy],
            [0, 0, 1]
        ])

    def project(self, points_3d: np.ndarray) -> np.ndarray:
        """3D 点投影到 2D 图像平面"""
        # points_3d: (N, 3) 在世界坐标系
        points_h = np.hstack([points_3d, np.ones((len(points_3d), 1))])

        # 投影
        pixels = self.K @ points_h[:, :3].T
        pixels = pixels.T
```



```

# 归一化
pixels[:, 0] /= pixels[:, 2]
pixels[:, 1] /= pixels[:, 2]

return pixels[:, :2]

def back_project(self, pixels: np.ndarray, depth: np.ndarray) -> np.ndarray:
    """2D 像素+深度反投影到 3D"""
    # 像素坐标
    u, v = pixels[:, 0], pixels[:, 1]
    z = depth

    x = (u - self.cx) * z / self.fx
    y = (v - self.cy) * z / self.fy

    return np.column_stack([x, y, z])

```

21.2.2 相机畸变

径向畸变 (Radial Distortion) :

```

x_corrected = x * (1 + k1*r2 + k2*r4 + k3*r6)
y_corrected = y * (1 + k1*r2 + k2*r4 + k3*r6)

```

切向畸变 (Tangential Distortion) :

```

x_corrected = x + (2*p1*x*y + p2*(r2 + 2*x2))
y_corrected = y + (p1*(r2 + 2*y2) + 2*p2*x*y)

```

21.2.3 ROS2 相机接口

```

# 查看相机内参
ros2 topic echo /camera/camera_info --once

# 查看原始图像
ros2 topic echo /camera/image_raw

# 查看深度图像 (RGB-D)
ros2 topic echo /camera/depth/image_raw

# 图像显示
ros2 run image_view image_view image:=/camera/image_raw
ros2 run rqt_image_view rqt_image_view

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

class CameraProcessor(Node):
    """ROS2 相机数据处理节点"""
    def __init__(self):
        super().__init__('camera_processor')

```

```

self.bridge = CvBridge()

# 彩色图像订阅
self.image_sub = self.create_subscription(
    Image, '/camera/color/image_raw',
    self.image_callback, 10)

# 深度图像订阅
self.depth_sub = self.create_subscription(
    Image, '/camera/depth/image_raw',
    self.depth_callback, 10)

# 相机内参订阅
self.info_sub = self.create_subscription(
    CameraInfo, '/camera/color/camera_info',
    self.info_callback, 10)

self.latest_image = None
self.latest_depth = None
self.camera_matrix = None

def image_callback(self, msg: Image):
    self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

def depth_callback(self, msg: Image):
    self.latest_depth = self.bridge.imgmsg_to_cv2(msg, '32FC1')

def info_callback(self, msg: CameraInfo):
    self.camera_matrix = np.array(msg.k).reshape(3, 3)
    self.get_logger().info(f'相机内参已获取:\n{self.camera_matrix}')

def get_rgbd_pointcloud(self):
    """获取 RGB-D 点云"""
    if self.latest_image is None or self.latest_depth is None:
        return None

    h, w = self.latest_depth.shape
    fx = self.camera_matrix[0, 0]
    fy = self.camera_matrix[1, 1]
    cx = self.camera_matrix[0, 2]
    cy = self.camera_matrix[1, 2]

    points = []
    colors = []

    for v in range(0, h, 5): # 步长 5 降采样
        for u in range(0, w, 5):
            z = self.latest_depth[v, u]
            if z <= 0 or z > 5.0: # 有效深度范围
                continue

            x = (u - cx) * z / fx
            y = (v - cy) * z / fy

            points.append([x, y, z])
            colors.append(self.latest_image[v, u])

```

```
return np.array(points), np.array(colors)
```

21.2.4 image_transport 图像传输

```
# Raw 格式（无损但带宽大）
ros2 run image_transport republish raw in:=/camera/image_raw raw out:=/image_raw

# Compressed 格式（JPEG 压缩）
ros2 run image_transport republish compressed in:=/camera/image_raw raw out:=/image_compressed

# Theora 格式（视频流编码）
ros2 run image_transport republish theora in:=/camera/image_raw raw out:=/image_theora
```

21.3 ORB-SLAM

21.3.1 ORB 特征

ORB (Oriented FAST and Rotated BRIEF) 是视觉 SLAM 中最常用的特征：

FAST 关键点检测：

- 比较像素与其周围 16 个像素的亮度
- 若有连续 N 个像素亮度超过阈值，则认为是角点
- 计算速度快，适合实时 SLAM

BRIEF 描述子：

- 在关键点周围随机选取点对比较亮度
- 生成二进制字符串描述子
- 使用 Hamming 距离快速匹配

ORB 改进：

- 方向计算：使用灰度质心法（旋转不变性）
- 尺度不变性：使用图像金字塔
- 旋转 BRIEF：根据关键点方向旋转描述子

```
import cv2
import numpy as np

class ORBFeatureExtractor:
    """ORB 特征提取器"""
    def __init__(self, num_features: int = 1000,
```

```

        scale_factor: float = 1.2,
        num_levels: int = 8):
self.ORB = cv2.ORB_create(
    nfeatures=num_features,
    scaleFactor=scale_factor,
    nlevels=num_levels
)

def extract(self, image: np.ndarray) -> tuple:
    """
    提取 ORB 特征

    返回:
        keypoints: 关键点列表
        descriptors: (N, 32) 描述子
    """
    keypoints, descriptors = self.ORB.detectAndCompute(image, None)
    return keypoints, descriptors

def match_features(self, desc1: np.ndarray,
                  desc2: np.ndarray,
                  ratio_threshold: float = 0.75) -> list:
    """
    特征匹配 (使用 FLANN 或暴力匹配)

    参数:
        desc1: 第一帧描述子
        desc2: 第二帧描述子
        ratio_threshold: Lowe 比率测试阈值

    返回:
        good_matches: 好的匹配对
    """
    # 暴力匹配
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
    matches = bf.knnMatch(desc1, desc2, k=2)

    # Lowe 比率测试
    good_matches = []
    for pair in matches:
        if len(pair) == 2:
            m, n = pair
            if m.distance < ratio_threshold * n.distance:
                good_matches.append(m)

    return good_matches

def visualize_matches(self, img1: np.ndarray, img2: np.ndarray,
                    kp1: list, kp2: list,
                    matches: list) -> np.ndarray:
    """可视化特征匹配结果"""
    return cv2.drawMatches(
        img1, kp1, img2, kp2, matches, None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

```

21.3.2 ORB-SLAM 算法流程

跟踪线程 (Tracking Thread):

1. 预处理输入
 - 提取 ORB 特征
 - 如果是 RGB-D, 获取对应深度
2. 位姿预测 / 重定位
 - 匀速模型预测
 - 特征匹配 + EPnP 求解位姿
3. 局部地图跟踪
 - 匹配更多地图点
 - 优化当前帧位姿
4. 关键帧决策
 - 判断是否插入新关键帧
 - 条件: 跟踪质量、时间间隔、距离间隔

建图线程 (Local Mapping Thread):

1. 插入关键帧
 - 更新共视图(Covisibility Graph)
2. 地图点管理
 - 剔除质量差的地图点
 - 三角化新的地图点
3. 局部 BA 优化
 - 优化当前关键帧及其邻居
4. 冗余关键帧剔除
 - 删除观测冗余的关键帧

闭环检测线程 (Loop Closing Thread):

1. DBoW 词袋检测
 - 使用视觉词汇进行闭环候选搜索
2. 闭环验证
 - 计算闭环帧间 SE3 变换
3. 闭环融合
 - 合并闭环处的地图点和关键帧

4. 本质图优化

- 全局位姿图优化消除累积误差

21.3.3 重投影误差

视觉 SLAM 的核心是最小化重投影误差：

$$E = \sum || z_{ij} - h(T_i, P_j) ||^2_{\sum}$$

其中：

- z_{ij} : 第 j 个地图点在第 i 帧图像上的观测像素坐标
- T_i : 第 i 帧相机位姿
- P_j : 第 j 个地图点的 3D 位置
- $h(T_i, P_j)$: 重投影函数 (3D→2D 投影)

```
import numpy as np

class BundleAdjustment:
    """光束平差法(BA)优化"""
    def __init__(self, camera_matrix: np.ndarray):
        self.K = camera_matrix

    def compute_reprojection_error(self, pose: np.ndarray,
                                   point_3d: np.ndarray,
                                   observation: np.ndarray) -> float:
        """
        计算单个重投影误差

        参数:
            pose: [rx, ry, rz, tx, ty, tz] 相机位姿
            point_3d: [X, Y, Z] 地图点位置
            observation: [u, v] 观测像素坐标
        """
        # 将地图点变换到相机坐标系
        R = self._rodrigues(pose[:3]) # 旋转向量→旋转矩阵
        t = pose[3:]

        P_c = R @ point_3d + t

        # 投影到图像平面
        u = self.K[0, 0] * P_c[0] / P_c[2] + self.K[0, 2]
        v = self.K[1, 1] * P_c[1] / P_c[2] + self.K[1, 2]

        # 误差
        error = np.array([u - observation[0], v - observation[1]])
        return np.linalg.norm(error)

    def _rodrigues(self, r: np.ndarray) -> np.ndarray:
```

```

"""罗德里格斯公式：旋转向量→旋转矩阵"""
theta = np.linalg.norm(r)
if theta < 1e-6:
    return np.eye(3)
n = r / theta
K = np.array([[0, -n[2], n[1]],
               [n[2], 0, -n[0]],
               [-n[1], n[0], 0]])
R = np.eye(3) + np.sin(theta) * K + (1 - np.cos(theta)) * (K @ K)
return R

```

21.3.4 ORB-SLAM3 ROS2 配置

```

# RGB-D 相机配置 (D415.yaml)
Camera.type: "PinHole"
Camera.fx: 615.0
Camera.fy: 615.0
Camera.cx: 320.0
Camera.cy: 240.0
Camera.k1: 0.0
Camera.k2: 0.0
Camera.p1: 0.0
Camera.p2: 0.0
Camera.width: 640
Camera.height: 480
Camera.fps: 30

# RGB-D 参数
RGBD.ThDepth: 5.0          # 深度阈值(m)
Depth.map_factor: 1000.0   # 深度缩放因子

# ORB 特征参数
ORBextractor.nFeatures: 1200
ORBextractor.scaleFactor: 1.2
ORBextractor.nLevels: 8

# 跟踪参数
Tracker.nFeaturesPerLevel: 200
Tracker.nonFeaturesPerLevel: 300

# 查看器参数
Viewer.keyFrameSize: 0.05
Viewer.keyFrameLineWidth: 1.0
Viewer.graphLineWidth: 0.9
Viewer.pointSize: 2.0

# 启动 ORB-SLAM3 (需要编译 ORB-SLAM3 源码)
ros2 run orbslam3 rgbd \
  /path/to/Vocabulary/ORBvoc.txt \
  /path/to/config/D415.yaml

# 查看 ORB-SLAM3 输出
ros2 topic echo /orb_slam3/camera_pose
ros2 topic echo /orb_slam3/trajectory
ros2 topic echo /orb_slam3/map_points

```

21.4 DSO 直接法

21.4.1 直接法 vs 特征法

特征法 (ORB-SLAM)	直接法 (DSO)
提取特征点	使用原始像素亮度
构建描述子	直接使用光度误差
适合弱纹理	不适合
适合模糊图像	不适合
计算量	特征提取+匹配
信息利用	稀疏（仅特征点）

21.4.2 DSO 算法原理

核心思想： 最小化光度误差 (Photometric Error)

$$E_{\text{photo}} = \sum || I_i(p') - I_j(p) ||^2$$

其中 $I_i(p')$ 是前一帧图像在 p' 处的亮度， $I_j(p)$ 是当前帧图像在 p 处的亮度。

关键模块：

- 354. 光度标定： 校正曝光时间、伽马校正、渐晕
- 355. 直接图像对齐： 使用像素亮度差异优化位姿
- 356. 滑窗优化： 维护最近关键帧窗口进行 BA

```
import numpy as np

class DirectImageAlignment:
    """直接图像对齐 (DSO 核心) """
    def __init__(self, num_levels: int = 3):
        self.num_levels = num_levels

    def align(self, img_ref: np.ndarray, img_cur: np.ndarray,
              initial_pose: np.ndarray,
              inv_depth: np.ndarray) -> np.ndarray:
        """
        直接图像对齐

        参数:
            img_ref: 参考帧图像
            img_cur: 当前帧图像
        """
```



```

        initial_pose: 初始位姿估计 [tx, ty, tz, rx, ry, rz]
        inv_depth: 逆深度值

    返回:
        optimized_pose: 优化后的位姿
    """
    pose = initial_pose.copy()

    # 构建图像金字塔
    pyramid_ref = self._build_pyramid(img_ref)
    pyramid_cur = self._build_pyramid(img_cur)

    # 由粗到精优化
    for level in range(self.num_levels - 1, -1, -1):
        scale = 2**level
        H, W = pyramid_ref[level].shape

        for iteration in range(10):
            # 计算光度残差
            residuals, J = self._compute_photometric_error(
                pyramid_ref[level], pyramid_cur[level],
                pose, inv_depth / scale
            )

            # Gauss-Newton 优化
            H_gn = J.T @ J
            b = J.T @ residuals

            try:
                delta = np.linalg.solve(
                    H_gn + np.eye(6) * 0.001, -b
                )
            except np.linalg.LinAlgError:
                break

            pose = self._se3_exp(delta) @ pose

            if np.linalg.norm(delta) < 0.001:
                break

    return pose

def _compute_photometric_error(self, img_ref: np.ndarray,
                              img_cur: np.ndarray,
                              pose: np.ndarray,
                              inv_depth: float) -> tuple:
    """计算光度误差"""
    h, w = img_ref.shape

    # 在参考帧上采样像素（取梯度大的区域）
    grad_x = np.gradient(img_ref, axis=1)
    grad_y = np.gradient(img_ref, axis=0)
    grad_mag = np.sqrt(grad_x**2 + grad_y**2)

    # 选择高梯度像素

```

```

mask = grad_mag > 10.0
ys, xs = np.where(mask)

# 随机采样
if len(ys) > 1000:
    indices = np.random.choice(len(ys), 1000, replace=False)
    ys, xs = ys[indices], xs[indices]

residuals = []
J = []

for y, x in zip(ys, xs):
    # 参考帧像素坐标→归一化坐标→3D点
    # (简化实现)
    z = 1.0 / inv_depth
    p_3d = np.array([(x - w/2) * z / 500,
                     (y - h/2) * z / 500, z])

    # 变换到当前帧
    p_cur = pose[:3, :3] @ p_3d + pose[:3, 3]
    u = p_cur[0] / p_cur[2] * 500 + w/2
    v = p_cur[1] / p_cur[2] * 500 + h/2

    if not (0 <= u < w-1 and 0 <= v < h-1):
        continue

    # 亮度误差
    residual = img_ref[y, x] - img_cur[int(v), int(u)]
    residuals.append(residual)

return np.array(residuals), np.array(J)

def _build_pyramid(self, image: np.ndarray) -> list:
    """构建图像金字塔"""
    pyramid = [image]
    for _ in range(1, self.num_levels):
        h, w = pyramid[-1].shape
        smaller = cv2.resize(
            pyramid[-1], (w//2, h//2),
            interpolation=cv2.INTER_LINEAR
        )
        pyramid.append(smaller)
    return pyramid

def _se3_exp(self, xi: np.ndarray) -> np.ndarray:
    """SE3 指数映射"""
    w = xi[:3]
    v = xi[3:]

    theta = np.linalg.norm(w)
    if theta < 1e-6:
        R = np.eye(3)
        t = v
    else:
        wx = np.array([[0, -w[2], w[1]],
                       [w[2], 0, -w[0]],
                       [-w[1], w[0], 0]])

```

```

R = (np.eye(3) + np.sin(theta)/theta * wx +
      (1 - np.cos(theta))/theta**2 * wx @ wx)
A = (np.eye(3) + (1-np.cos(theta))/theta**2 * wx +
      (theta - np.sin(theta))/theta**3 * wx @ wx)
t = A @ v

T = np.eye(4)
T[:3, :3] = R
T[:3, 3] = t
return T

```

21.5 RGB-D SLAM

21.5.1 RGB-D SLAM 特点

RGB-D 相机直接提供深度信息，简化了 SLAM 问题：

优势：

- 直接获取 3D 信息，无需三角化
- 尺度已知，无尺度漂移
- 可构建稠密地图
- 初始化简单

挑战：

- 深度范围有限（通常 0.3-5m）
- 受环境光影响大
- 多路径干扰
- 运动模糊

```

import numpy as np
import open3d as o3d

class RGBDSLAM:
    """简化版 RGB-D SLAM"""
    def __init__(self, camera_matrix: np.ndarray):
        self.K = camera_matrix
        self.point_cloud = o3d.geometry.PointCloud()
        self.keyframes = []
        self.poses = []

    def process_frame(self, rgb: np.ndarray, depth: np.ndarray,
                     initial_pose: np.ndarray = None) -> np.ndarray:
        """处理 RGB-D 帧"""
        # 1. RGB-D 生成点云
        pcd = self._rgbd_to_pointcloud(rgb, depth)

```

```

# 2. 位姿估计
if len(self.keyframes) == 0:
    # 首帧
    pose = np.eye(4)
    if initial_pose is not None:
        pose[:3, 3] = initial_pose
else:
    # 与上一关键帧配准
    pose = self._register_pointcloud(
        pcd, self.keyframes[-1]['cloud'], initial_pose
    )

self.poses.append(pose)

# 3. 判断是否插入关键帧
if self._is_keyframe(pose):
    self.keyframes.append({
        'pose': pose,
        'cloud': pcd,
        'rgb': rgb,
        'depth': depth
    })

# 4. 增量式地图融合
self._fusion_map()

return pose

def _rgb_to_pointcloud(self, rgb: np.ndarray,
                      depth: np.ndarray) -> o3d.geometry.PointCloud:
    """RGB-D 转点云"""
    rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(
        o3d.geometry.Image(rgb),
        o3d.geometry.Image(depth),
        depth_trunc=5.0,
        convert_rgb_to_intensity=False
    )

    pcd = o3d.geometry.PointCloud.create_from_rgbd_image(
        rgbd, self.K
    )

    return pcd.voxel_down_sample(voxel_size=0.01)

def _register_pointcloud(self, source: o3d.geometry.PointCloud,
                        target: o3d.geometry.PointCloud,
                        init_pose: np.ndarray = None) -> np.ndarray:
    """点云配准"""
    if init_pose is None:
        init_pose = np.eye(4)

    # ICP 配准
    reg_p2p = o3d.pipelines.registration.registration_icp(
        source, target, 0.1, init_pose,
        o3d.pipelines.registration.TransformationEstimationPointToPoint(),
        o3d.pipelines.registration.ICPConvergenceCriteria(

```

```

        max_iteration=50)
    )

    return reg_p2p.transformation

def _is_keyframe(self, pose: np.ndarray) -> bool:
    """判断是否为关键帧"""
    if len(self.poses) < 2:
        return True

    # 检查与上一关键帧的位移和旋转
    last_pose = self.keyframes[-1]['pose']
    delta = np.linalg.inv(last_pose) @ pose

    translation = np.linalg.norm(delta[:3, 3])
    rotation = np.arccos(
        (np.trace(delta[:3, :3]) - 1) / 2
    )

    return translation > 0.1 or abs(rotation) > 0.3

def _fusion_map(self):
    """融合地图 (TSDF 融合) """
    # 使用所有关键帧进行 TSDF 融合
    volume = o3d.pipelines.integration.ScalableTSDFVolume(
        voxel_length=0.01,
        sdf_trunc=0.03,
        color_type=o3d.pipelines.integration.TSDFVolumeColorType.RGB8
    )

    for kf in self.keyframes:
        rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(
            o3d.geometry.Image(kf['rgb']),
            o3d.geometry.Image(kf['depth']),
            depth_trunc=4.0,
            convert_rgb_to_intensity=False
        )
        volume.integrate(rgbd, self.K, np.linalg.inv(kf['pose']))

    self.point_cloud = volume.extract_point_cloud()

```

21.6 视觉与激光融合 SLAM

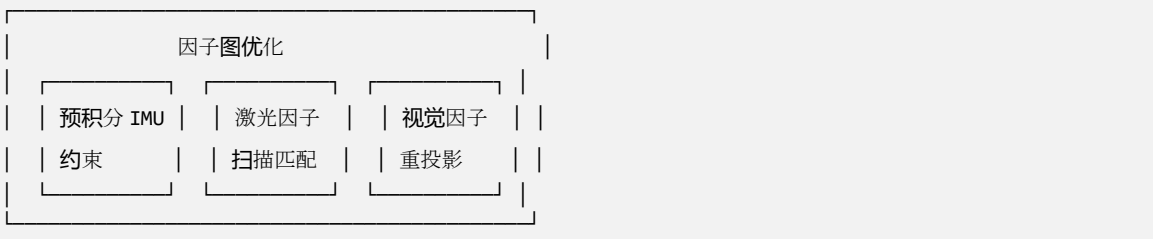
21.6.1 融合策略

视觉与激光 SLAM 各有优势，融合可以取长补短：

融合方式	描述	代表系统
松耦合	独立运行后融合结果	LIC-Fusion
紧耦合	统一优化视觉+激光因子	LVI-SAM, R3LIVE
视觉辅助激光	视觉提供回环检测	Cartographer+视觉

激光辅助视觉	激光提供深度	RGB-D+激光扩展
--------	--------	------------

融合框架示意：



21.6.2 视觉-惯性融合 (VIO)

VINS-Mono 是经典的视觉惯性融合算法：

```

import numpy as np

class VINSMono:
    """视觉惯性导航系统(VINS)简化实现"""
    def __init__(self):
        self.imu_buffer = []      # IMU 数据缓冲
        self.image_buffer = []    # 图像数据缓冲
        self.keyframe_poses = []  # 关键帧位姿
        self.landmarks = {}       # 地图点

        # IMU 参数
        self.gyro_noise = 0.01
        self.acc_noise = 0.01
        self.gyro_bias_noise = 0.0001
        self.acc_bias_noise = 0.0001

        # 状态
        self.state = {
            'position': np.zeros(3),
            'velocity': np.zeros(3),
            'orientation': np.eye(3),
            'gyro_bias': np.zeros(3),
            'acc_bias': np.zeros(3)
        }

    def process_imu(self, acc: np.ndarray, gyro: np.ndarray, dt: float):
        """IMU 预积分"""
        # 去除偏置
        acc_corrected = acc - self.state['acc_bias']
        gyro_corrected = gyro - self.state['gyro_bias']

        # 状态传播
        self.state['position'] += (self.state['velocity'] * dt +

```

```

        0.5 * acc_corrected * dt**2)
self.state['velocity'] += acc_corrected * dt

# 姿态更新
omega = np.array([[0, -gyro_corrected[2], gyro_corrected[1]],
                  [gyro_corrected[2], 0, -gyro_corrected[0]],
                  [-gyro_corrected[1], gyro_corrected[0], 0]])
self.state['orientation'] = (self.state['orientation'] @
                             (np.eye(3) + omega * dt))

# 保存到缓冲
self.imu_buffer.append({
    'acc': acc, 'gyro': gyro, 'dt': dt,
    'state': self.state.copy()
})

def process_image(self, image: np.ndarray, timestamp: float):
    """处理图像帧"""
    # 特征提取和匹配
    # ...

    # 视觉-惯性对齐
    # ...

    # 滑窗优化
    # ...

```

21.6.3 视觉-激光融合实践

```

# LVI-SAM (需要编译安装)
ros2 launch lvi_sam run.launch.py

# 数据融合可视化
rviz2
# 添加: PointCloud2 (/lvi_sam/mapping/registered_cloud)
# 添加: Path (/lvi_sam/odometry/odometry)
# 添加: Image (/camera/image_raw)

```

21.6.4 工程建议

```

def suggest_fusion_strategy(
    environment: str,
    has_lidar: bool,
    has_camera: bool,
    has_imu: bool,
    budget: str
) -> str:
    """推荐融合策略"""
    if budget == 'low':
        if has_camera and has_imu:
            return 'VINS-Mono (仅视觉+IMU)'
        else:
            return 'ORB-SLAM2 (仅视觉)'

```

```
elif budget == 'medium':
    if has_lidar and has_camera:
        return 'LVI-SAM / Cartographer+视觉回环'
    else:
        return 'Cartographer (激光+IMU)'

elif budget == 'high':
    if has_lidar and has_camera and has_imu:
        return 'R3LIVE / LVI-SAM (全传感器融合)'
    else:
        return 'FAST-LIO2 (激光+IMU)'

return '推荐使用激光 SLAM 建图 + 视觉 SLAM 提供语义信息'
```

课后练习

357. **原理题:** 比较特征法视觉 SLAM 和直接法视觉 SLAM 的异同, 分析各自优缺点和适用场景。
358. **编程题:** 使用 OpenCV 提取 ORB 特征并实现特征匹配, 要求包含 Lowe 比率测试剔除误匹配。
359. **分析题:** 分析 ORB-SLAM 三个并行线程 (跟踪、建图、闭环) 的协作机制, 说明各自的作用和交互方式。
360. **配置题:** 配置 ORB-SLAM3 的 ROS2 接口, 使用 RealSense D415 相机进行实时 RGB-D SLAM 建图。
361. **操作题:** 录制视觉 SLAM 的 rosbag 数据 (包含图像和 IMU 话题), 使用 EVO 工具评估 SLAM 轨迹精度。
362. **设计题:** 设计一个视觉+激光+IMU 多传感器融合 SLAM 方案, 用于大型室内停车场 (存在光照变化和重复纹理), 包括传感器配置、融合策略和地图表示方法。

实验 21:

第 21 章 实验：视觉抓取综合项目

****对应理论章节**:** 第 34 章《视觉抓取完整流程》、第 35 章《综合实训：智能机器人产线》 ****实验课时**:** 6 课时 ****实验代码**:**

``src/lab_code/ch21_lab/vision_pickup_lab/``

实验目标

- 整合视觉检测和机械臂运动规划
- 实现"检测→定位→抓取→放置"全流程自动化
- 掌握视觉服务和 MoveIt2 的协同工作方式
- 完成基于 ArUco 标签的视觉抓取演示
- 综合运用课程所学知识构建完整的智能机器人系统
- 实现多组件集成: 视觉检测 + 运动规划 + 抓取控制
- 掌握 Action Server 在复杂任务编排中的应用
- 完成从场景理解到任务执行的全自动化流程

实验环境

- ROS 2 Jazzy + MoveIt2
- USB 摄像头 + ArUco 标签
- OpenCV + cv_bridge
- YOLOv8 / VLM API (可选, 见第 20 章)
- Python3

MoveIt 前置依赖 (统一说明): 本章实验开始前, 需先启动 xArm 仿真与 MoveIt:

```
# 方式 1: 纯 MoveIt + RViz 仿真 (不含 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
```

```
# 方式 2: 含 Gazebo 的完整仿真 (本章参考代码 README 使用的命令)
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

默认使用上述 `xarm_ros2_arm_only` 启动命令。若按第 17 章 17.1 节自行生成配置包，请使用该包的**实际名称**及其生成的 `launch` 文件。

参考代码说明

`src/lab_code/ch21_lab/vision_pickup_lab/` (`ament_python` 包) 提供以下程序:

程序	功能
<code>`tf2_camera_broadcaster.py`</code>	相机 TF 广播器。发布 <code>`camera_link`</code> 到 <code>`base_link`</code> 的静态坐标变换，支持参数或 YAML 标定文件配置
<code>`vision_pickup_pipeline.py`</code>	视觉抓取流水线。检测 ArUco 标签，将位姿转换到 <code>`base_link`</code> 并发布标签位姿
<code>`aruco_pick_server.py`</code>	AR 码引导抓取服务。订阅标签位姿，将物体加入规划场景，生成多种抓取/放置姿态，通过 MoveIt2 执行多目标抓取放置，成功后清理场景并回到 Home

构建:

```
cd <工作区根目录>
colcon build --symlink-install --packages-select vision_pickup_lab
source install/setup.bash
```

21.1 视觉抓取完整流程

21.1.1 运行参考代码 `vision_pickup_lab`

```
# 终端 1: 启动 xArm 仿真 (见"实验环境"前置说明)
ros2 launch xarm_ros2_arm_only arm_only.launch.py

# 终端 2: 发布相机 TF (另开终端)
ros2 run vision_pickup_lab tf2_camera_broadcaster --ros-args \
  -p x:=0.15 -p z:=0.25 -p child_frame:=camera_link
```

```
# 终端 3: 启动抓取服务与流水线
ros2 run vision_pickup_lab aruco_pick_server
ros2 run vision_pickup_lab vision_pickup_pipeline
```

流水线检测到 AR 码后广播其 3D 位姿，机械臂规划并执行抓取，终端输出 Pick succeeded。

tf2_camera_broadcaster.py 支持从参数或 YAML 标定文件加载变换（可使用第 19 章 19.3 节手眼标定的结果）：

```
# 从参数
python3 tf2_camera_broadcaster.py --ros-args \
  -p x:=0.3 -p y:=0.0 -p z:=0.15 \
  -p roll:=0.0 -p pitch:=0.0 -p yaw:=1.57

# 从标定文件
python3 tf2_camera_broadcaster.py --ros-args \
  -p calibration_file:=/path/to/calib.yaml
```

21.1.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch21_vision_grasp --build-type ament_python --dependencies rclpy sensor_msgs
geometry_msgs cv_bridge tf2_ros moveit
cd ch21_vision_grasp
mkdir -p ch21_vision_grasp launch
```

21.1.3 编写视觉抓取 Pipeline 节点

创建 ch21_vision_grasp/vision_grasp_pipeline.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from std_srvs.srv import Trigger
from cv_bridge import CvBridge
from moveit.planning import MoveItPy, PlanningComponent
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros
import tf2_geometry_msgs
import math
import time
import threading

class VisionGraspPipeline(Node):
    def __init__(self):
        super().__init__('vision_grasp_pipeline')
        self.bridge = CvBridge()
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
```

```

self.declare_parameter('marker_size', 0.065)
self.marker_size = self.get_parameter('marker_size').value

dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
params = aruco.DetectorParameters()
self.detector = aruco.ArucoDetector(dict_aruco, params)

self.moveit = MoveItPy(node_name='moveit_py')
self.arm = PlanningComponent(
    self.moveit, 'xarm_group', 'gripper_cenot_link'
)
self.gripper = PlanningComponent(
    self.moveit, 'gripper_group', 'gripper_cenot_link'
)
self.psm = self.moveit.get_planning_scene_monitor()
self.arm.set_pose_reference_frame('base_link')
time.sleep(1)

self.camera_matrix = None
self.dist_coeffs = None
self.latest_pose = None
self.latest_ids = None

self.image_sub = self.create_subscription(
    Image, '/image_raw', self.image_cb, 10
)
self.info_sub = self.create_subscription(
    CameraInfo, '/camera_info', self.info_cb, 10
)

self.srv = self.create_service(Trigger, '/vision_grasp/trigger', self.trigger_cb)
self.pose_pub = self.create_publisher(PoseStamped, '/vision_grasp/target_pose', 10)
self.get_logger().info('视觉抓取 Pipeline 已就绪')

self.get_logger().info('调用: ros2 service call /vision_grasp/trigger
std_srvs/srv/Trigger')

def info_cb(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        self.get_logger().info('相机参数已接收')

def image_cb(self, msg):
    if self.camera_matrix is None:
        return
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)
    self.latest_ids = ids

    if ids is not None:
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
            corners, self.marker_size, self.camera_matrix, self.dist_coeffs
        )
        aruco.drawDetectedMarkers(frame, corners, ids)

```

```

for i in range(len(ids)):
    aruco.drawAxis(frame, self.camera_matrix, self.dist_coeffs,
                    rvecs[i], tvecs[i], 0.03)

    pose_cam = PoseStamped()
    pose_cam.header.stamp = self.get_clock().now().to_msg()
    pose_cam.header.frame_id = 'camera_link'
    pose_cam.pose.position.x = float(tvecs[i][0][0])
    pose_cam.pose.position.y = float(tvecs[i][0][1])
    pose_cam.pose.position.z = float(tvecs[i][0][2])

    rmat, _ = cv2.Rodrigues(rvecs[i])
    q = self.rotmat_to_quat(rmat)
    pose_cam.pose.orientation.x = q[0]
    pose_cam.pose.orientation.y = q[1]
    pose_cam.pose.orientation.z = q[2]
    pose_cam.pose.orientation.w = q[3]

    try:
        transform = self.tf_buffer.lookup_transform(
            'base_link', 'camera_link', rclpy.time.Time()
        )
        pose_base = tf2_geometry_msgs.do_transform_pose(pose_cam, transform)
        self.latest_pose = pose_base
        self.pose_pub.publish(pose_base)
    except Exception as e:
        self.get_logger().warn(f'TF 变换失败: {e}')

cv2.putText(frame, f'Markers: {len(ids) if ids is not None else 0}',
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.imshow('Vision Grasp', frame)
cv2.waitKey(1)

def trigger_cb(self, request, response):
    if self.latest_pose is None:
        response.success = False
        response.message = '未检测到 ArUco 标签'
        return response

    self.get_logger().info('触发视觉抓取...')
    threading.Thread(target=self.execute_grasp, daemon=True).start()
    response.success = True
    response.message = '抓取任务已启动'
    return response

def execute_grasp(self):
    target = self.latest_pose.pose
    self.get_logger().info(f'目标位置: ({target.position.x:.3f}, {target.position.y:.3f}, {target.position.z:.3f})')

    home = [0.0]*6
    pre_grasp = [target.position.x * 0.8, -0.5, 0.5, 0.0, 0.0, 0.0]

    self.arm.set_start_state_to_current_state()
    self.arm.set_joint_value_target(home)
    plan = self.arm.plan()

```

```

if plan: self.arm.execute(plan.trajectory)
time.sleep(1)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
time.sleep(0.5)

grasp_pose = PoseStamped()
grasp_pose.header.frame_id = 'base_link'
grasp_pose.pose.position.x = target.position.x
grasp_pose.pose.position.y = target.position.y
grasp_pose.pose.position.z = target.position.z + 0.15
grasp_pose.pose.orientation = target.orientation

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(grasp_pose.pose, 'gripper_cenror_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
    time.sleep(1)

lower_pose = grasp_pose
lower_pose.pose.position.z = target.position.z + 0.02
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(lower_pose.pose, 'gripper_cenror_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
    time.sleep(0.5)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
time.sleep(0.5)

# attach
aco = AttachedCollisionObject()
aco.link_name = 'gripper_cenror_link'
aco.object.id = 'target_object'
aco.object.operation = CollisionObject.ADD
aco.touch_links = ['gripper_1_link', 'gripper_2_link']
self.psm.process_attached_collision_object(aco)
time.sleep(0.5)

lift_pose = grasp_pose
lift_pose.pose.position.z += 0.15
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(lift_pose.pose, 'gripper_cenror_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(1)

place_pose = PoseStamped()
place_pose.header.frame_id = 'base_link'
place_pose.pose.position.x = target.position.x - 0.2
place_pose.pose.position.y = target.position.y + 0.2
place_pose.pose.position.z = target.position.z + 0.15

```

```

place_pose.pose.orientation = target.orientation

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(place_pose.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(1)

place_lower = place_pose
place_lower.pose.position.z = target.position.z + 0.02
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(place_lower.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(0.5)

aco2 = AttachedCollisionObject()
aco2.link_name = 'gripper_cenotor_link'
aco2.object.id = 'target_object'
aco2.object.operation = CollisionObject.REMOVE
self.psm.process_attached_collision_object(aco2)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
time.sleep(1)

self.arm.set_start_state_to_current_state()
self.arm.set_joint_value_target(home)
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)

self.get_logger().info('视觉抓取完成!')

@staticmethod
def rotmat_to_quat(r):
    q = np.empty(4)
    t = np.trace(r)
    if t > 0:
        s = np.sqrt(1.0 + t) * 2
        q[3] = 0.25 * s
        q[0] = (r[2, 1] - r[1, 2]) / s
        q[1] = (r[0, 2] - r[2, 0]) / s
        q[2] = (r[1, 0] - r[0, 1]) / s
    else:
        i = np.argmax([r[0, 0], r[1, 1], r[2, 2]])
        s = np.sqrt(1.0 + r[i, i]) * 2
        q[3] = (r[2, 1] - r[1, 2]) / s if i == 0 else \
            (r[0, 2] - r[2, 0]) / s if i == 1 else \
            (r[1, 0] - r[0, 1]) / s
        q[i] = 0.25 * s
    return q

def main(args=None):
    rclpy.init(args=args)
    node = VisionGraspPipeline()
    try:
        rclpy.spin(node)

```

```

except KeyboardInterrupt:
    cv2.destroyAllWindows()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

***注意**：该 Pipeline 依赖 `base\_link`→`camera\_link` 的 TF 变换，可使用参考代码 `tf2\_camera\_broadcaster.py`（见 21.1.1）或第 19 章 19.3.11 节的 `calibration\_tf\_publisher.py` 提供。*

21.1.4 编写 launch 文件

创建 launch/vision_grasp.launch.py:

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='usb_cam',
            executable='usb_cam_node_exe',
            name='usb_cam',
            parameters=[{
                'video_device': '/dev/video0',
                'image_width': 640,
                'image_height': 480,
                'camera_frame_id': 'camera_link',
                'pixel_format': 'yuyv',
            }],
        ),
        Node(
            package='ch21_vision_grasp',
            executable='vision_grasp_pipeline',
            name='vision_grasp_pipeline',
            parameters=[{'marker_size': 0.065}],
        ),
    ])

```

21.1.5 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'vision_grasp_pipeline = ch21_vision_grasp.vision_grasp_pipeline:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch21_vision_grasp
source install/setup.bash

```



```

# 终端 1: 启动 MoveIt2 仿真 (见"实验环境"前置说明)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
# 或 ros2 launch xarm_ros2_arm_only arm_only.launch.py

# 终端 2: 发布相机 TF (参考代码或第 19 章标定结果)
ros2 run vision_pickup_lab tf2_camera_broadcaster --ros-args \
  -p x:=0.15 -p z:=0.25 -p child_frame:=camera_link

# 终端 3: 启动视觉抓取 Pipeline (包含相机驱动)
ros2 launch ch21_vision_grasp vision_grasp.launch.py

# 或分开启动:
# ros2 run usb_cam usb_cam_node_exe
# ros2 run ch21_vision_grasp vision_grasp_pipeline --ros-args -p marker_size:=0.065

# 将 ArUco 标签放在相机视野内

# 终端 4: 触发抓取
ros2 service call /vision_grasp/trigger std_srvs/srv/Trigger

```

21.1.6 测试不同场景

```

# 使用不同尺寸的标签
ros2 run ch21_vision_grasp vision_grasp_pipeline --ros-args -p marker_size:=0.094

# 查看目标位姿
ros2 topic echo /vision_grasp/target_pose

# 查看检测图像
rqt_image_view /image_raw

```

21.1.7 编写简化测试节点

创建 `ch21_vision_grasp/simple_vision_test.py`:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np

class SimpleVisionTest(Node):
    def __init__(self):
        super().__init__('simple_vision_test')
        self.bridge = CvBridge()
        self.declare_parameter('marker_size', 0.065)
        self.marker_size = self.get_parameter('marker_size').value

        dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
        params = aruco.DetectorParameters()
        self.detector = aruco.ArucoDetector(dict_aruco, params)

```

```

self.camera_matrix = np.array([[530, 0, 320], [0, 530, 240], [0, 0, 1]], dtype=float)
self.dist_coeffs = np.zeros(5)

self.sub = self.create_subscription(Image, '/image_raw', self.image_cb, 10)
self.pose_pub = self.create_publisher(PoseStamped, '/vision/test_pose', 10)
self.get_logger().info('视觉测试节点已启动')

def image_cb(self, msg):
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)
    if ids is not None:
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
            corners, self.marker_size, self.camera_matrix, self.dist_coeffs
        )
        aruco.drawDetectedMarkers(frame, corners, ids)
        for i in range(len(ids)):
            aruco.drawAxis(frame, self.camera_matrix, self.dist_coeffs,
                           rvecs[i], tvecs[i], 0.03)
            p = PoseStamped()
            p.header.stamp = self.get_clock().now().to_msg()
            p.header.frame_id = 'camera_link'
            p.pose.position.x = float(tvecs[i][0][0])
            p.pose.position.y = float(tvecs[i][0][1])
            p.pose.position.z = float(tvecs[i][0][2])
            p.pose.orientation.w = 1.0
            self.pose_pub.publish(p)
            self.get_logger().info(
                f'标签{ids[i][0]}: ({tvecs[i][0][0]:.3f}, {tvecs[i][0][1]:.3f},
{tvecs[i][0][2]:.3f})'
            )
            cv2.imshow('Test', frame)
            cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = SimpleVisionTest()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2 综合实训：智能机器人产线

21.2.1 创建综合实训包

```

cd ~/ros2_arm_ws/src
ros2 pkg create ch21_factory --build-type ament_python --dependencies rclpy sensor_msgs

```

```
geometry_msgs std_srvs action_msgs cv_bridge tf2_ros moveit
cd ch21_factory
mkdir -p ch21_factory/launch/actions
```

21.2.2 定义 Action 接口

创建 actions/Pipeline.action:

```
# 智能产线任务编排
string recipe_text # 任务描述文本
---
bool success
string message
---
# 反馈
int32 current_step
int32 total_steps
string step_name
```

创建 actions/DetectObject.action:

```
# 视觉检测
string target_name
---
bool detected
geometry_msgs/PoseStamped object_pose
float32 confidence
---
string status
```

21.2.3 编写视觉检测模块

创建 ch21_factory/vision_module.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros
import tf2_geometry_msgs

class VisionModule(Node):
    def __init__(self):
        super().__init__('vision_module')
        self.bridge = CvBridge()
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

        self.declare_parameter('marker_size', 0.065)
        self.marker_size = self.get_parameter('marker_size').value
```

```

dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
params = aruco.DetectorParameters()
self.detector = aruco.ArucoDetector(dict_aruco, params)

self.camera_matrix = None
self.dist_coeffs = None
self.latest_frame = None

self.image_sub = self.create_subscription(Image, '/image_raw', self.image_cb, 10)
self.info_sub = self.create_subscription(CameraInfo, '/camera_info', self.info_cb, 10)

def info_cb(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)

def image_cb(self, msg):
    try:
        self.latest_frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        pass

def detect_marker(self, target_id=None):
    if self.latest_frame is None or self.camera_matrix is None:
        return None

    gray = cv2.cvtColor(self.latest_frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)

    if ids is None:
        return None

    rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
        corners, self.marker_size, self.camera_matrix, self.dist_coeffs
    )

    results = []
    for i in range(len(ids)):
        marker_id = int(ids[i][0])
        if target_id is not None and marker_id != target_id:
            continue

        pose_cam = PoseStamped()
        pose_cam.header.frame_id = 'camera_link'
        pose_cam.pose.position.x = float(tvecs[i][0][0])
        pose_cam.pose.position.y = float(tvecs[i][0][1])
        pose_cam.pose.position.z = float(tvecs[i][0][2])
        pose_cam.pose.orientation.w = 1.0

        try:
            transform = self.tf_buffer.lookup_transform(
                'base_link', 'camera_link', rclpy.time.Time()
            )
            pose_base = tf2_geometry_msgs.do_transform_pose(pose_cam, transform)
            results.append((marker_id, pose_base))
        except Exception:
            results.append((marker_id, pose_cam))

    return results

```

```
def main(args=None):
    rclpy.init(args=args)
    node = VisionModule()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

21.2.4 编写运动控制模块

创建 ch21_factory/motion_module.py:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
import time

class MotionModule(Node):
    def __init__(self):
        super().__init__('motion_module')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'xarm_group', 'gripper_cenror_link')
        self.gripper = PlanningComponent(self.moveit, 'gripper_group', 'gripper_cenror_link')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm.set_pose_reference_frame('base_link')
        time.sleep(1)
        self.get_logger().info('运动模块已初始化')

    def go_home(self):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target([0.0]*6)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

    def open_gripper(self):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.65, 0.65])
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def close_gripper(self):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.0, 0.0])
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def move_to_pose(self, x, y, z, roll=0, pitch=math.pi/2, yaw=0):
        import math
```

```

from tf_transformations import quaternion_from_euler
target = PoseStamped()
target.header.frame_id = 'base_link'
target.pose.position.x = x
target.pose.position.y = y
target.pose.position.z = z
q = quaternion_from_euler(roll, pitch, yaw)
target.pose.orientation.x = q[0]
target.pose.orientation.y = q[1]
target.pose.orientation.z = q[2]
target.pose.orientation.w = q[3]

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(target.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
    return True
return False

def move_to_joints(self, joints):
    self.arm.set_start_state_to_current_state()
    self.arm.set_joint_value_target(joints)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
        return True
    return False

def pick_object(self, pose_stamped):
    self.open_gripper()
    time.sleep(0.5)

    pre_pick = PoseStamped()
    pre_pick.header.frame_id = 'base_link'
    pre_pick.pose.position.x = pose_stamped.pose.position.x
    pre_pick.pose.position.y = pose_stamped.pose.position.y
    pre_pick.pose.position.z = pose_stamped.pose.position.z + 0.1
    pre_pick.pose.orientation = pose_stamped.pose.orientation

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pre_pick.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    pick_pose = PoseStamped()
    pick_pose.header = pre_pick.header
    pick_pose.pose.position.x = pose_stamped.pose.position.x
    pick_pose.pose.position.y = pose_stamped.pose.position.y
    pick_pose.pose.position.z = pose_stamped.pose.position.z
    pick_pose.pose.orientation = pose_stamped.pose.orientation

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pick_pose.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

```

```

        self.close_gripper()
        time.sleep(0.5)

        aco = AttachedCollisionObject()
        aco.link_name = 'gripper_cenotor_link'
        aco.object.id = 'workpiece'
        aco.object.operation = CollisionObject.ADD
        aco.touch_links = ['gripper_1_link', 'gripper_2_link']
        self.psm.process_attached_collision_object(aco)

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(pre_pick.pose, 'gripper_cenotor_link')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        return True

def place_object(self, x, y, z):
    place_pose = PoseStamped()
    place_pose.header.frame_id = 'base_link'
    place_pose.pose.position.x = x
    place_pose.pose.position.y = y
    place_pose.pose.position.z = z + 0.1
    place_pose.pose.orientation.w = 1.0

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(place_pose.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    down_pose = place_pose
    down_pose.pose.position.z = z
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(down_pose.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_cenotor_link'
    aco.object.id = 'workpiece'
    aco.object.operation = CollisionObject.REMOVE
    self.psm.process_attached_collision_object(aco)

    self.open_gripper()
    time.sleep(0.5)

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(place_pose.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    return True

import math
from tf_transformations import quaternion_from_euler

```

```
def main(args=None):
    rclpy.init(args=args)
    node = MotionModule()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

21.2.5 编写 Pipeline 编排服务器

创建 ch21_factory/pipeline_server.py:

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer
from std_srvs.srv import Trigger
from geometry_msgs.msg import PoseStamped
import time
import json

class FactoryPipelineServer(Node):
    def __init__(self):
        super().__init__('factory_pipeline_server')
        self.vision = None
        self.motion = None
        self.current_step = 0
        self.total_steps = 5

        self.srv = self.create_service(Trigger, '/factory/start_pipeline', self.start_cb)
        self.get_logger().info('智能产线 Pipeline Server 已就绪')

        self.get_logger().info('启动: ros2 service call /factory/start_pipeline\nstd_srvs/srv/Trigger')

    def start_cb(self, request, response):
        self.get_logger().info('启动智能产线流程...')
        try:
            self.run_pipeline()
            response.success = True
            response.message = '产线流程完成'
        except Exception as e:
            response.success = False
            response.message = str(e)
        return response

    def run_pipeline(self):
        self.get_logger().info(f'=== 步骤 1/{self.total_steps}: 初始化系统 ===')
        self.current_step = 1
        self.init_modules()
        time.sleep(1)

        self.get_logger().info(f'=== 步骤 2/{self.total_steps}: 视觉检测 ===')
        self.current_step = 2
        detected_objects = self.detect_objects()
```



```

        if not detected_objects:
            raise Exception('未检测到目标物体')
        target_pose = detected_objects[0]
        self.get_logger().info(f'检测到目标: ({target_pose.pose.position.x:.3f},
{target_pose.pose.position.y:.3f}, {target_pose.pose.position.z:.3f})')
        time.sleep(1)

        self.get_logger().info(f'=== 步骤 3/{self.total_steps}: 规划抓取路径 ===')
        self.current_step = 3
        self.motion.open_gripper()
        self.motion.go_home()
        time.sleep(1)

        self.get_logger().info(f'=== 步骤 4/{self.total_steps}: 执行抓取 ===')
        self.current_step = 4
        self.motion.pick_object(target_pose)
        time.sleep(1)

        self.get_logger().info(f'=== 步骤 5/{self.total_steps}: 放置到目标区域 ===')
        self.current_step = 5
        place_x = target_pose.pose.position.x - 0.2
        place_y = target_pose.pose.position.y + 0.2
        self.motion.place_object(place_x, place_y, 0.02)
        time.sleep(1)

        self.get_logger().info('回到 Home 位姿')
        self.motion.go_home()

        self.get_logger().info('=== 智能产线流程完成! ===')

    def init_modules(self):
        from ch21_factory.vision_module import VisionModule
        from ch21_factory.motion_module import MotionModule
        self.vision = VisionModule()
        self.motion = MotionModule()

    def detect_objects(self):
        results = self.vision.detect_marker()
        if results:
            return [pose for _, pose in results]
        return []

def main(args=None):
    rclpy.init(args=args)
    node = FactoryPipelineServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2.6 编写产线动作客户端

创建 ch21_factory/pipeline_client.py:

```

import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from std_srvs.srv import Trigger

class PipelineClient(Node):
    def __init__(self):
        super().__init__('pipeline_client')
        self.client = self.create_client(Trigger, '/factory/start_pipeline')
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待 Pipeline Server...')
        self.get_logger().info('Pipeline Client 就绪')

    def start(self):
        req = Trigger.Request()
        future = self.client.call_async(req)
        rclpy.spin_until_future_complete(self, future)
        if future.result():
            self.get_logger().info(f'结果: {future.result().message}')
        else:
            self.get_logger().error('调用失败')

def main(args=None):
    rclpy.init(args=args)
    node = PipelineClient()
    node.start()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2.7 编写多物体分拣演示

创建 ch21_factory/sorting_demo.py:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import time

class SortingDemo(Node):
    def __init__(self):
        super().__init__('sorting_demo')
        self.bridge = CvBridge()
        dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
        params = aruco.DetectorParameters()
        self.detector = aruco.ArucoDetector(dict_aruco, params)

        self.sub = self.create_subscription(Image, '/image_raw', self.image_cb, 10)
        self.targets = {
            1: {'name': '红色零件', 'place_zone': 'Zone_A'},

```

```

        2: {'name': '蓝色零件', 'place_zone': 'Zone_B'},
        3: {'name': '绿色零件', 'place_zone': 'Zone_C'},
    }

def image_cb(self, msg):
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)

    if ids is not None:
        aruco.drawDetectedMarkers(frame, corners, ids)
        for i in range(len(ids)):
            mid = int(ids[i][0])
            if mid in self.targets:
                c = corners[i][0]
                cx = int(c[:, 0].mean())
                cy = int(c[:, 1].mean())
                info = f'{self.targets[mid]["name"]} → {self.targets[mid]["place_zone"]}'
                cv2.putText(frame, info, (cx-30, cy),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        cv2.putText(frame, 'Sorting Demo', (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
        cv2.imshow('Sorting Demo', frame)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = SortingDemo()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2.8 编写 launch 文件

创建 launch/factory_system.launch.py:

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='usb_cam',
            executable='usb_cam_node_exe',
            name='usb_cam',
            parameters=[{

```

```

        'video_device': '/dev/video0',
        'image_width': 640,
        'image_height': 480,
        'camera_frame_id': 'camera_link',
    }],
),
Node(
    package='ch21_factory',
    executable='vision_module',
    name='vision_module',
    parameters=[{'marker_size': 0.065}],
),
Node(
    package='ch21_factory',
    executable='pipeline_server',
    name='pipeline_server',
),
])

```

21.2.9 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'vision_module = ch21_factory.vision_module:main',
        'motion_module = ch21_factory.motion_module:main',
        'pipeline_server = ch21_factory.pipeline_server:main',
        'pipeline_client = ch21_factory.pipeline_client:main',
        'sorting_demo = ch21_factory.sorting_demo:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch21_factory
source install/setup.bash

# 终端 1: 启动 MoveIt2 仿真 (见"实验环境"前置说明)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
# 或 ros2 launch xarm_ros2_arm_only arm_only.launch.py

# 终端 2: 启动产线系统
ros2 launch ch21_factory factory_system.launch.py

# 终端 3: 启动产线流程
ros2 service call /factory/start_pipeline std_srvs/srv/Trigger

# 或运行分拣演示
ros2 run ch21_factory sorting_demo

```

21.2.10 扩展练习

```

# 练习 1: 添加更多 ArUco 标签和对应的放置区域
# 修改 targets 字典, 增加 4-6 号标签

# 练习 2: 集成 YOLO 检测替代 ArUco
# 使用第 19 章(19.2 节)中的 YOLO 节点检测物体类别, 输入到 Pipeline

```

```
# 练习 3: 添加碰撞检测和异常处理
# 在抓取过程中检查机械臂是否与障碍物碰撞

# 练习 4: 实现连续分拣多个物体
# 循环检测-抓取-放置, 直到所有物体分拣完成
```

实验结果与分析

- 视觉抓取 Pipeline 完整实现了检测→定位→抓取→放置流程
- ArUco 标签提供了稳定的视觉特征用于位姿估计
- TF 坐标变换将相机坐标系中的目标位姿转换到机器人基坐标系
- MoveIt2 根据目标位姿进行逆运动学求解和避障规划
- 完整的智能产线系统集成视觉检测、运动规划和抓取控制
- Action/Service 机制实现了模块间的解耦和协同
- Pipeline 编排服务器统一管理多步骤任务流程
- 系统具有良好的可扩展性, 可以添加更多功能模块
- 参考代码 vision_pickup_lab 的

tf2_camera_broadcaster/vision_pickup_pipeline/aruco_pick_server 分别对应本章的相机
TF 发布、标签检测与抓取执行环节, 可对照验证自实现系统

思考题

363. 在视觉抓取流程中, 哪个环节最容易出错? 如何处理异常?
364. 如果相机视野中有多个 ArUco 标签, 如何选择正确的抓取目标?
365. 如何提高视觉抓取的成功率? 哪些因素会影响精度?
366. 在没有深度相机的情况下, 如何获取目标的深度信息?
367. 在大规模产线场景中, 如何实现多机械臂的协同作业?
368. 如何处理抓取失败的异常情况? 如何实现错误恢复?
369. 产线系统的实时性能如何优化? 哪些环节是性能瓶颈?
370. 如何将 VLM 集成到产线系统中, 实现更智能的决策?

第 22 章 多传感器融合 SLAM

学习目标

- 理解多传感器融合 SLAM 的必要性和基本框架
- 掌握 IMU+LiDAR 融合原理与方法
- 掌握视觉+IMU 融合 (VIO) 原理
- 熟悉 Kalman 滤波和图优化融合框架
- 了解实际工程中的多传感器融合案例

22.1 多传感器融合概述

22.1.1 为什么需要多传感器融合

单一传感器在 SLAM 中存在固有的局限性：

传感器	优势	局限性
激光雷达	精度高、不受光照影响	稀疏、无纹理、无语义
相机	信息丰富、低成本	受光照、弱纹理、模糊
IMU	高频、短期精度高	漂移大、无绝对参考
里程计	局部精度高	累积误差、打滑
GPS	全局无漂移	室内不可用、精度低

多传感器融合的核心优势：

- 互补性：不同传感器的优势互补
- 冗余性：某传感器失效时仍有其他传感器可用
- 鲁棒性：融合后可应对更广泛的环境条件
- 精度提升：多源信息约束提高估计精度

22.1.2 融合框架分类

松耦合 (Loosely Coupled)：

传感器各自独立处理 → 融合结果

- 各传感器运行各自的 SLAM/里程计
- 在状态估计层面融合 (EKF)
- 实现简单, 模块化
- 精度相对较低

紧耦合 (Tightly Coupled):

传感器原始数据 → 统一优化

- 在因子图/优化中统一处理各传感器约束
- 使用传感器原始测量 (非处理后的位姿)
- 精度高, 复杂度高
- 代表: LVI-SAM, MSCKF

22.2 IMU+LiDAR 融合

22.2.1 IMU 的作用

IMU 在 LiDAR SLAM 中的关键作用:

371. **运动畸变校正:** 激光扫描过程中若机器人运动, 导致点云畸变, IMU 可校正
372. **位姿预测:** 提供高频位姿预测, 作为扫描匹配的初值
373. **状态约束:** 重力方向约束, 使地图水平
374. **快速运动恢复:** 激光匹配失败时, IMU 可短暂维持定位

```
import numpy as np

class IMUPreintegrator:
    """IMU 预积分器"""
    def __init__(self):
        self.dt = 0.01 # IMU 采样周期
        self.g = np.array([0, 0, -9.81]) # 重力

        # 预积分状态
        self.delta_p = np.zeros(3) # 位置增量
        self.delta_v = np.zeros(3) # 速度增量
        self.delta_R = np.eye(3) # 旋转增量

        # 协方差
        self.covariance = np.zeros((9, 9))

    def integrate(self, acc: np.ndarray, gyro: np.ndarray):
        """
        IMU 预积分

        参数:
```

```

        acc: 加速度计测量 (含重力)
        gyro: 陀螺仪测量
    """
    # 预积分更新 (不考虑偏置)
    dt = self.dt

    # 旋转增量
    omega = self._skew(gyro)
    self.delta_R = self.delta_R @ (np.eye(3) + omega * dt)

    # 速度增量
    self.delta_v += self.delta_R @ acc * dt

    # 位置增量
    self.delta_p += (self.delta_v * dt +
                     0.5 * self.delta_R @ acc * dt**2)

def predict_state(self, p_i: np.ndarray, v_i: np.ndarray,
                  R_i: np.ndarray,
                  bg: np.ndarray, ba: np.ndarray) -> tuple:
    """
    根据预积分预测状态

    参数:
        p_i, v_i, R_i: 时刻 i 的状态
        bg, ba: 陀螺仪和加速度计偏置

    返回:
        p_j, v_j, R_j: 时刻 j 的预测状态
    """
    # 预积分的偏置校正
    dR = self._correct_rotation(self.delta_R, bg)
    dv = self._correct_velocity(self.delta_v, ba)
    dp = self._correct_position(self.delta_p, ba)

    # 预测
    p_j = p_i + v_i * self._total_time + R_i @ dp + 0.5 * self.g * self._total_time**2
    v_j = v_i + R_i @ dv + self.g * self._total_time
    R_j = R_i @ dR

    return p_j, v_j, R_j

def _skew(self, v: np.ndarray) -> np.ndarray:
    """向量→反对称矩阵"""
    return np.array([[0, -v[2], v[1]],
                     [v[2], 0, -v[0]],
                     [-v[1], v[0], 0]])

def _correct_rotation(self, dR: np.ndarray, bg: np.ndarray) -> np.ndarray:
    """旋转的偏置校正 (一阶近似) """
    return dR @ self._skew(-bg * self._total_time)

def _correct_velocity(self, dv: np.ndarray, ba: np.ndarray) -> np.ndarray:
    return dv + ba * self._total_time

```



```

"""通过加速度约束标定平移外参"""

# 利用杠杆臂效应：IMU 加速度包含 LiDAR 角速度引起的向心加速度
#  $a_{imu} = R_{lidar\_imu}^T @ (a_{lidar} + w_{lidar} \times (w_{lidar} \times t) + w_{dot} \times t)$ 
# ...
return self.t_lidar_imu

```

22.2.3 FAST-LIO2 框架

FAST-LIO2 是经典的 LiDAR-IMU 紧耦合 SLAM 系统：

```

import numpy as np

class FASTLIO2:
    """FAST-LIO2 核心框架（简化）"""
    def __init__(self):
        # 状态向量
        self.state = {
            'position': np.zeros(3),
            'velocity': np.zeros(3),
            'rotation': np.eye(3),
            'gyro_bias': np.zeros(3),
            'acc_bias': np.zeros(3)
        }

        # 协方差
        self.cov = np.eye(15) * 0.01

        # 地图 (ikd-Tree)
        self.map = None

        # 参数
        self.lidar_freq = 10.0
        self.imu_freq = 200.0

    def imu_callback(self, acc: np.ndarray, gyro: np.ndarray,
                    timestamp: float):
        """IMU 数据处理 - 状态预测"""
        dt = 1.0 / self.imu_freq

        # 去除偏置
        acc_corrected = acc - self.state['acc_bias']
        gyro_corrected = gyro - self.state['gyro_bias']

        # 运动学预测
        R = self.state['rotation']

        # 位置
        self.state['position'] += self.state['velocity'] * dt

        # 速度
        self.state['velocity'] += (R @ acc_corrected + np.array([0, 0, -9.81])) * dt

        # 姿态
        omega = np.array([[0, -gyro_corrected[2], gyro_corrected[1]],
                          [gyro_corrected[2], 0, -gyro_corrected[0]],

```

```

        [-gyro_corrected[1], gyro_corrected[0], 0]])
self.state['rotation'] = R @ (np.eye(3) + omega * dt)

# 协方差预测
self._predict_covariance(dt)

def lidar_callback(self, points: np.ndarray, timestamp: float):
    """LiDAR 数据处理 - 状态更新"""

    # 1. 运动畸变校正 (使用 IMU 预测)
    corrected_points = self._undistort_points(points)

    # 2. 特征提取
    features = self._extract_features(corrected_points)

    # 3. 扫描-地图匹配
    H, z = self._scan_to_map_matching(features)

    # 4. Kalman 更新
    self._kalman_update(H, z)

    # 5. 地图更新
    self._update_map(corrected_points)

def _extract_features(self, points: np.ndarray) -> dict:
    """特征提取 (边缘点+平面点) """

    # 计算曲率
    curvatures = []
    for i, point in enumerate(points):
        neighbors = points[max(0, i-5):min(len(points), i+5)]
        curv = np.linalg.norm(point - neighbors.mean(axis=0))
        curvatures.append(curv)

    curvatures = np.array(curvatures)

    # 边缘点 (高曲率)
    edge_threshold = np.percentile(curvatures, 80)
    edge_points = points[curvatures > edge_threshold]

    # 平面点 (低曲率)
    flat_threshold = np.percentile(curvatures, 30)
    flat_points = points[curvatures < flat_threshold]

    return {
        'edge': edge_points,
        'flat': flat_points
    }

def _undistort_points(self, points: np.ndarray) -> np.ndarray:
    """运动畸变校正"""

    # 使用 IMU 预测的姿态序列
    corrected = points.copy()

    # 对每个点根据其扫描时间补偿旋转和平移
    return corrected

```

```

def _scan_to_map_matching(self, features: dict) -> tuple:
    """扫描-地图匹配"""

    H = np.zeros((6, 15)) # 观测雅可比
    z = np.zeros(6)        # 残差
    return H, z

def _kalman_update(self, H: np.ndarray, z: np.ndarray):
    """Kalman 状态更新"""

    # Kalman 增益
    P = self.cov
    R = np.eye(len(z)) * 0.1 # 观测噪声

    S = H @ P @ H.T + R
    K = P @ H.T @ np.linalg.inv(S)

    # 状态更新
    dx = K @ z
    self._update_state(dx)

    # 协方差更新
    I_KH = np.eye(len(dx)) - K @ H
    self.cov = I_KH @ P @ I_KH.T + K @ R @ K.T

def _update_state(self, dx: np.ndarray):
    """更新状态向量"""

    self.state['position'] += dx[0:3]
    self.state['velocity'] += dx[3:6]

    # 旋转更新 (使用 so3 指数映射)
    omega = dx[6:9]
    theta = np.linalg.norm(omega)
    if theta > 1e-6:
        n = omega / theta
        K = np.array([[0, -n[2], n[1]],
                      [n[2], 0, -n[0]],
                      [-n[1], n[0], 0]])
        dR = np.eye(3) + np.sin(theta) * K + (1-np.cos(theta)) * K @ K
        self.state['rotation'] = self.state['rotation'] @ dR

    self.state['gyro_bias'] += dx[9:12]
    self.state['acc_bias'] += dx[12:15]

def _predict_covariance(self, dt: float):
    """协方差预测"""

    F = np.eye(15) # 状态转移矩阵
    Q = np.eye(15) * 0.001 # 过程噪声

    self.cov = F @ self.cov @ F.T + Q

```

22.3 视觉+IMU 融合(VIO)

22.3.1 VIO 算法分类

算法	方法	特点
MSCKF	多状态约束 KF	计算量小，不优化地图点
VINS-Mono	滑窗优化+IMU 预积分	精度高，单目 VIO
OKVIS	滑窗 BA+IMU	关键帧滑窗优化
ROVIO	扩展卡尔曼滤波	直接法+IMU
ORB-SLAM3	视觉 SLAM+IMU	支持视觉惯性模式

22.3.2 VINS-Mono 框架

```
import numpy as np

class VINSMono:
    """VINS-Mono 核心框架"""
    def __init__(self):
        # 滑窗
        self.window_size = 10
        self.frame_buffer = []

        # IMU 预积分器
        self.preintegrator = IMUPreintegrator()

        # 状态
        self.velocity = np.zeros(3)
        self.gravity = np.array([0, 0, -9.81])
        self.gyro_bias = np.zeros(3)
        self.acc_bias = np.zeros(3)

        # 特征点
        self.features = {}

    def process_image(self, image: np.ndarray, timestamp: float):
        """处理图像帧"""
        # 1. 特征提取和跟踪
        new_features = self._track_features(image)

        # 2. 添加到帧缓冲
        self.frame_buffer.append({
            'timestamp': timestamp,
            'features': new_features,
            'preintegration': self.preintegrator.clone()
        })
```

```

# 3. 滑窗优化
if len(self.frame_buffer) >= self.window_size:
    self._sliding_window_optimization()

# 4. 边缘化
self._marginalize()

def _track_features(self, image: np.ndarray) -> dict:
    """KLT 光流特征跟踪"""
    # 使用 OpenCV 的 calcOpticalFlowPyrLK
    # ...
    return {}

def _sliding_window_optimization(self):
    """滑窗 BA 优化"""
    # 构建因子图
    # IMU 因子 (预积分约束)
    # 视觉因子 (重投影约束)
    # 先验因子 (边缘化信息)

    # Ceres / g2o 优化
    pass

def _marginalize(self):
    """边缘化旧帧"""
    if len(self.frame_buffer) > self.window_size:
        oldest = self.frame_buffer.pop(0)
        # 舒尔补边缘化
        pass

```

22.3.3 视觉-惯性 BA

```

class VisualInertialBA:
    """视觉-惯性光束平差法"""
    def __init__(self):
        pass

    def build_factor_graph(self, frames: list,
                          landmarks: dict) -> dict:
        """构建视觉-惯性因子图"""
        factors = {
            'imu': [],      # IMU 预积分约束
            'visual': [],   # 视觉重投影约束
            'prior': []     # 先验约束
        }

        # IMU 因子
        for i in range(len(frames) - 1):
            if 'preintegration' in frames[i+1]:
                factors['imu'].append({
                    'i': i,
                    'j': i + 1,

```

```

        'preint': frames[i+1]['preintegration']
    })

    # 视觉因子
    for lm_id, lm in landmarks.items():
        for frame_id, obs in lm['observations'].items():
            factors['visual'].append({
                'frame': frame_id,
                'landmark': lm_id,
                'observation': obs
            })

    return factors

def optimize(self, factors: dict,
             initial_states: dict) -> dict:
    """执行视觉-惯性 BA 优化"""
    # 使用非线性最小二乘求解器
    # (ceres, g2o, gtsam)

    # 残差包括:
    # 1. IMU 预积分残差
    # 2. 视觉重投影残差
    # 3. 先验残差

    # ...
    return initial_states

```

22.4 Kalman 滤波与图优化融合框架

22.4.1 多传感器 EKF 融合

扩展卡尔曼滤波是多传感器融合的基础框架：

```

import numpy as np

class MultiSensorEKF:
    """多传感器 EKF 融合"""
    def __init__(self, dim_state: int = 15):
        self.dim_state = dim_state
        self.x = np.zeros(dim_state) # [位置,速度,姿态,偏置...]
        self.P = np.eye(dim_state) * 0.1
        self.Q = np.eye(dim_state) * 0.01 # 过程噪声
        self.R = {} # 各传感器的观测噪声

    def predict(self, dt: float, control: np.ndarray = None):
        """状态预测"""
        # 状态转移矩阵 (简化为常速模型)
        F = np.eye(self.dim_state)
        F[0:3, 3:6] = np.eye(3) * dt

        self.x = F @ self.x

```

```

        self.P = F @ self.P @ F.T + self.Q

def update_imu(self, acc: np.ndarray, gyro: np.ndarray):
    """IMU 观测更新"""

    # IMU 测量模型
    z = np.concatenate([acc, gyro])
    H = self._compute_imu_jacobian()
    R = np.eye(6) * 0.1

    self._kalman_update(z, H, R, 'imu')

def update_lidar_odom(self, delta_pose: np.ndarray):
    """激光里程计更新"""

    z = delta_pose
    H = np.zeros((6, self.dim_state))
    H[:6, :6] = np.eye(6)
    R = np.eye(6) * 0.05

    self._kalman_update(z, H, R, 'lidar')

def update_visual_odom(self, delta_pose: np.ndarray):
    """视觉里程计更新"""

    z = delta_pose
    H = np.zeros((6, self.dim_state))
    H[:6, :6] = np.eye(6)
    R = np.eye(6) * 0.2 # 视觉噪声大于激光

    self._kalman_update(z, H, R, 'visual')

def update_gps(self, position: np.ndarray):
    """GPS 更新"""

    z = position
    H = np.zeros((3, self.dim_state))
    H[:3, :3] = np.eye(3)
    R = np.eye(3) * 1.0 # GPS 噪声较大

    self._kalman_update(z, H, R, 'gps')

def _kalman_update(self, z: np.ndarray, H: np.ndarray,
                  R: np.ndarray, sensor_name: str):
    """通用 Kalman 更新步骤"""

    # 观测残差
    z_pred = H @ self.x
    residual = z - z_pred

    # Kalman 增益
    S = H @ self.P @ H.T + R
    K = self.P @ H.T @ np.linalg.inv(S)

    # 状态更新
    self.x = self.x + K @ residual
    self.P = (np.eye(self.dim_state) - K @ H) @ self.P

def _compute_imu_jacobian(self) -> np.ndarray:
    """计算 IMU 观测雅可比"""

```



```

H = np.zeros((6, self.dim_state))
# 加速度计部分与姿态相关
# 陀螺仪部分与角速度相关
return H

```

22.4.2 因子图优化(GTSAM)

现代 SLAM 系统广泛使用因子图进行多传感器融合：

```

# 伪代码：使用 GTSAM 进行因子图优化
from gtsam import *

class FactorGraphFusion:
    """因子图多传感器融合"""
    def __init__(self):
        self.graph = NonlinearFactorGraph()
        self.initial = Values()
        self.result = None

    def add_prior_factor(self, key: int, pose: np.ndarray,
                        noise: np.ndarray):
        """添加先验因子"""
        prior_noise = noise_model.Diagonal.Sigmas(noise)
        self.graph.add(PriorFactorPose2(key, pose, prior_noise))

    def add_odometry_factor(self, key_i: int, key_j: int,
                           delta: np.ndarray, noise: np.ndarray):
        """添加里程计因子（激光/视觉/轮式）"""
        odom_noise = noise_model.Diagonal.Sigmas(noise)
        self.graph.add(BetweenFactorPose2(
            key_i, key_j, delta, odom_noise))

    def add_imu_factor(self, key_i: int, key_j: int,
                       preintegrated: dict, noise: np.ndarray):
        """添加 IMU 预积分因子"""
        # IMU 因子（需要 gtsam 的 PreintegratedImuMeasurements）
        imu_noise = noise_model.Diagonal.Sigmas(noise)
        # self.graph.add(ImuFactor(...))

    def add_loop_closure_factor(self, key_i: int, key_j: int,
                                relative_pose: np.ndarray,
                                noise: np.ndarray):
        """添加回环因子"""
        lc_noise = noise_model.Diagonal.Sigmas(noise)
        self.graph.add(BetweenFactorPose2(
            key_i, key_j, relative_pose, lc_noise))

    def optimize(self, num_iterations: int = 100):
        """执行优化"""
        optimizer = LevenbergMarquardtOptimizer(
            self.graph, self.initial)
        self.result = optimizer.optimize()
        return self.result

```

22.5 实际工程案例

22.5.1 仓库 AGV 融合 SLAM 案例

```
class WarehouseAGVFusion:
    """仓库 AGV 多传感器融合 SLAM"""
    def __init__(self):
        # 传感器
        self.lidar = '2D 激光雷达 × 2 (前后) '
        self.imu = '6 轴 IMU'
        self.encoder = '轮式编码器'
        self.rfid = 'RFID 标签 (辅助定位) '

        # 融合策略
        self.fusion = MultiSensorEKF()

    def fusion_pipeline(self):
        """融合处理流程"""
        steps = """
        |-----|
        | 传感器输入 |
        |-----|
        | 激光雷达 | IMU | 轮式编码器 |
        | 10Hz | 200Hz | 100Hz |
        |-----|
        | 传感器预处理 |
        | - 激光：体素滤波、特征提取 |
        | - IMU：预积分、偏置估计 |
        | - 编码器：速度积分 |
        |-----|
        | 多传感器 EKF 融合 |
        | - 高频 IMU 传播 (200Hz) |
        | - 激光匹配更新 (10Hz) |
        | - 编码器约束更新 (100Hz) |
        | - RFID 绝对修正 (1Hz) |
        |-----|
        | 输出 |
        | - 位姿估计：100Hz |
        | - 占据栅格地图：/map |
        | - 里程计：/odom |
        |-----|
        """
        return steps

    def parameter_recommendation(self) -> dict:
        """参数推荐"""
        return {
            'laser_noise': 0.02, # 激光噪声(m)
```

```

        'imu_acc_noise': 0.01,    # IMU 加速度噪声(m/s2)
        'imu_gyro_noise': 0.001, # IMU 角速度噪声(rad/s)
        'encoder_noise': 0.05,   # 编码器噪声(m)
        'rfid_noise': 0.3,       # RFID 噪声(m)
        'fusion_frequency': 100, # 融合输出频率(Hz)
        'map_resolution': 0.05,  # 地图分辨率(m)
    }

```

22.5.2 自动驾驶融合定位案例

```

class AutonomousVehicleLocalization:
    """自动驾驶多传感器融合定位"""
    def __init__(self):
        self.sensors = {
            'gps': {"精度": "3m (单点) / 0.1m (RTK)", "频率": "10Hz"},
            'imu': {"精度": "0.01°/s (光纤陀螺)", "频率": "200Hz"},
            'lidar_3d': {"精度": "0.03m", "频率": "10Hz"},
            'camera': {"精度": "像素级", "频率": "30Hz"},
            'wheel_encoder': {"精度": "0.001m (脉冲)", "频率": "100Hz"},
        }

    def localization_strategy(self) -> str:
        """多模式定位策略"""
        return """
        1. 城区道路:
            GPS RTK + IMU + 轮式里程计
            (RTK 可用时: cm 级精度)

        2. 隧道/地下:
            LiDAR SLAM + IMU + 轮式里程计
            (无 GPS 信号, 纯 SLAM)

        3. 高速路段:
            GPS + IMU 松耦合
            (高动态, SLAM 可靠性下降)

        4. 多楼层停车:
            LiDAR SLAM + 视觉回环
            (楼层间地图切换)
        """

```

22.5.3 传感器时间同步

多传感器融合的时间同步是关键工程问题:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu, LaserScan, Image
import numpy as np

```

```

class SensorTimeSynchronizer(Node):
    """多传感器时间同步器"""
    def __init__(self):
        super().__init__('sensor_sync')

        # 时间戳对齐
        self.approximate_sync = ApproximateTimeSynchronizer(
            [self.create_subscription(Imu, '/imu', self.imu_callback, 10),
             self.create_subscription(LaserScan, '/scan', self.scan_callback, 10),
             self.create_subscription(Image, '/camera/image_raw', self.image_callback, 10)],
            queue_size=10,
            slop=0.05 # 最大时间差 50ms
        )
        self.approximate_sync.registerCallback(self.sync_callback)

        # 时间偏移估计
        self.time_offsets = {
            'imu_lidar': [],
            'lidar_camera': []
        }

    def sync_callback(self, imu_msg: Imu, scan_msg: LaserScan,
                     image_msg: Image):
        """同步后的传感器数据"""

        # 此处三个传感器时间戳已对齐 (误差<50ms)
        imu_time = imu_msg.header.stamp.sec + imu_msg.header.stamp.nanosec * 1e-9
        scan_time = scan_msg.header.stamp.sec + scan_msg.header.stamp.nanosec * 1e-9
        image_time = image_msg.header.stamp.sec + image_msg.header.stamp.nanosec * 1e-9

        self.get_logger().info(
            f'传感器同步: IMU={imu_time:.3f} '
            f'LiDAR={scan_time:.3f} '
            f'Camera={image_time:.3f}'
        )

        # 硬件同步触发 (如果支持)
        self.hardware_trigger()

    def estimate_time_offset(self):
        """估计传感器间时间偏移"""

        # 使用互相关方法
        if len(self.time_offsets['imu_lidar']) > 100:
            mean_offset = np.mean(self.time_offsets['imu_lidar'])
            std_offset = np.std(self.time_offsets['imu_lidar'])
            self.get_logger().info(
                f'IMU-LiDAR 时间偏移: {mean_offset*1000:.2f}ms ± {std_offset*1000:.2f}ms'
            )

    def hardware_trigger(self):
        """硬件同步触发"""

        # 如果传感器支持硬件触发线

        # 通过 GPIO 发送同步脉冲
        pass

```

22.5.4 故障检测与切换

```
class SensorHealthMonitor:
    """传感器健康监控与自动切换"""
    def __init__(self):
        self.sensor_status = {
            'lidar': {'healthy': True, 'last_ok': 0},
            'camera': {'healthy': True, 'last_ok': 0},
            'imu': {'healthy': True, 'last_ok': 0},
            'gps': {'healthy': True, 'last_ok': 0}
        }

        # 故障阈值
        self.timeout = {
            'lidar': 0.5,      # 500ms 无数据
            'camera': 1.0,    # 1s 无数据
            'imu': 0.2,       # 200ms 无数据
            'gps': 2.0        # 2s 无数据
        }

    def check_health(self, sensor: str, timestamp: float):
        """检查传感器健康状态"""
        status = self.sensor_status[sensor]
        time_since_last = timestamp - status['last_ok']

        if time_since_last > self.timeout[sensor]:
            if status['healthy']:
                status['healthy'] = False
                print(f'警告: {sensor} 故障 (超时{time_since_last:.1f}s)')
                self.handle_failure(sensor)
            else:
                if not status['healthy']:
                    print(f'恢复: {sensor} 已恢复')
                    status['healthy'] = True
                    status['last_ok'] = timestamp

    def handle_failure(self, failed_sensor: str):
        """处理传感器故障"""
        strategies = {
            'lidar': '切换到视觉+IMU 紧耦合模式',
            'camera': '切换到激光+IMU 模式, 禁用视觉回环',
            'imu': '切换到轮式里程计+激光模式, 降低动态性能',
            'gps': '切换到纯 SLAM 定位模式'
        }

        print(f'故障处理: {strategies.get(failed_sensor, "未知传感器")}')

```

课后练习

375. **原理题:** 比较松耦合和紧耦合多传感器融合的异同, 分析为什么紧耦合通常能获得更高的精度。

376. **编程题:** 实现一个简化的 EKF 多传感器融合系统，融合激光里程计和 IMU 数据，输出融合后的位姿估计。
377. **分析题:** 分析 IMU 在 LiDAR SLAM 中的三个重要作用：运动畸变校正、位姿预测、重力约束，说明各自的原理和实现方式。
378. **配置题:** 在 Cartographer 中配置激光+IMU+里程计的多传感器融合建图，写出对应的 LUA 配置文件。
379. **操作题:** 录制包含激光、IMU 和里程计数据的 rosbag，使用 FAST-LIO2 或同类算法进行多传感器融合 SLAM 建图，评估结果。
380. **设计题:** 设计一个面向机场候机楼的清洁机器人 SLAM 方案，环境特征包括：大面积玻璃幕墙（激光穿透）、高挑空间（视觉特征少）、金属结构（IMU 干扰）。给出传感器选型、融合策略和应急处理方案。

实验 22:

第 22 章 实验指导书：自动驾驶概述与 CARLA 基础

****对应理论章节**:** 第 36 章《自动驾驶概述与 CARLA 基础》 ****实验课时**:** 2 课时

实验代码:** `src/lab\_code/ch22\_lab/`

本实验使用 Ubuntu 24.04 + ROS 2 Jazzy + CARLA 0.9.16 环境。

实验目标

- 学会安装并验证 CARLA 仿真环境
- 掌握 CARLA Python API 的基本使用方法
- 能够连接 CARLA 服务器并获取世界信息
- 学会在 CARLA 中生成车辆并控制其运动

实验准备

硬件环境

- PC with NVIDIA GPU (推荐 4GB 以上显存)
- 至少 8GB RAM
- 可用磁盘空间 30GB+

软件环境

- Ubuntu 24.04
- ROS 2 Jazzy
- Python 3.12+
- CARLA 0.9.16

练习 22.1：环境预检查

目标

在安装 CARLA 之前，检查系统环境是否满足要求，避免安装过程中出现问题。

步骤

步骤 1：检查 GPU 与驱动

```
# 检查 NVIDIA GPU
nvidia-smi
# 期望输出：GPU 型号、驱动版本（推荐≥470）、CUDA 版本
# 注意：如输出 "command not found"，需安装 NVIDIA 驱动

# 检查 OpenGL 渲染器
sudo apt-get install mesa-utils
glxinfo -B | grep "OpenGL renderer"
# 期望：显示真实 GPU 型号（非 "llvmpipe" 表示使用软件渲染）
```

步骤 2：检查系统依赖与磁盘空间

```
# 检查系统依赖
dpkg -l | grep libomp5 || echo "需要安装 libomp5"

# 检查磁盘空间（CARLA 需要约 20GB）
df -h ~ | awk 'NR==2 {print "可用空间: " $4}'
```

步骤 3：检查 Python 环境

```
python3 --version    # 需 ≥ 3.8
pip3 --version        # 确认 pip 已安装
pip3 list 2>/dev/null | grep -E "numpy|pygame" || echo "需要安装 numpy/pygame"
```

步骤 4：检查网络与端口

```
# 检查端口 2000 是否被占用（CARLA 默认端口）
ss -tlnp | grep :2000 || echo "端口 2000 可用"

# 检查能否访问 CARLA 下载服务器
curl -sI https://tiny.carla.org/carla-0-9-16-linux | head -1
# 期望：HTTP/2 200 或 403（服务器可达）
```


检查清单

- ☐ `nvidia-smi` 显示 GPU 信息
- ☐ OpenGL 渲染器为物理 GPU
- ☐ 磁盘可用空间 $\geq 20\text{GB}$
- ☐ Python ≥ 3.8
- ☐ pip 已安装
- ☐ 端口 2000 未被占用
- ☐ CARLA 下载服务器可达

练习 22.2 : CARLA 安装验证

目标

完成 CARLA 0.9.16 的安装，验证服务器能否正常启动，并通过 Python 客户端确认连接。

步骤

步骤 1: 下载并解压 CARLA

```
cd ~
mkdir -p carla && cd carla

# 下载 CARLA 本体
wget https://tiny.carla.org/carla-0-9-16-linux -O CARLA_0.9.16.tar.gz
tar -xzf CARLA_0.9.16.tar.gz

# 下载附加地图 (可选)
wget https://carla-releases.b-cdn.net/Linux/AdditionalMaps_0.9.16.tar.gz
tar -xzf AdditionalMaps_0.9.16.tar.gz -C ~/carla
```

步骤 2: 安装 Python 依赖

```
pip install pygame numpy
cd ~/carla/PythonAPI/carla
pip install -e .
```

步骤 3: 启动 CARLA 服务器

```
cd ~/carla
./CarlaUE4.sh -quality-level=Low
```

首次启动可能需要较长时间, `-quality-level=Low` 可降低 GPU 负载。服务器启动后会在 2000 端口监听客户端连接。

步骤 4: 验证连接

新建终端, 执行:

```
cd src/lab_code/ch22_lab/  
python3 explore_carla.py
```

预期输出 (类似) :

```
CARLA 版本: 0.9.16  
当前地图: /Game/Carla/Maps/Town03  
可生成点数量: 76  
Actor 数量: 1  
天气: ClearNoon
```

练习 22.3 : 探索 CARLA 世界

目标

在 CARLA 世界中遍历不同的地图和天气, 理解 World、Map、Weather 等核心概念。

步骤

步骤 1: 切换地图

```
import carla  
  
client = carla.Client('localhost', 2000)  
client.set_timeout(10.0)  
  
# 加载不同地图  
world = client.load_world('Town01')  
print(f'当前地图: {world.get_map().name}')  
  
world = client.load_world('Town05')  
print(f'当前地图: {world.get_map().name}')
```

步骤 2: 尝试不同天气

```
import time  
  
presets = [  
    carla.WeatherParameters.ClearNoon,
```

```

    carla.WeatherParameters.CloudyNoon,
    carla.WeatherParameters.WetNoon,
    carla.WeatherParameters.MidRainyNoon,
    carla.WeatherParameters.HardRainNoon,
    carla.WeatherParameters.WetCloudySunset,
]

for i, preset in enumerate(presets):
    world.set_weather(preset)
    print(f'天气 {i+1}/{len(presets)} 已设置')
    time.sleep(2.0)

```

步骤 3: 获取地图路网信息

```

map = world.get_map()
spawn_points = map.get_spawn_points()
topology = map.get_topology()

print(f'可生成点数量: {len(spawn_points)}')
print(f'路网段数量: {len(topology)}')

# 打印前 5 个生成点坐标
for i, pt in enumerate(spawn_points[:5]):
    loc = pt.location
    print(f'点 {i}: ({loc.x:.1f}, {loc.y:.1f}, {loc.z:.1f})')

```

步骤 4: 查看所有蓝图

```

bp_lib = world.get_blueprint_library()

# 筛选车辆蓝图
vehicle_bps = bp_lib.filter('vehicle.*')
print(f'可用车辆蓝图: {len(vehicle_bps)}')
for bp in vehicle_bps[:5]:
    print(f' - {bp.id}')

# 筛选传感器蓝图
sensor_bps = bp_lib.filter('sensor.*')
print(f'可用传感器蓝图: {len(sensor_bps)}')
for bp in sensor_bps:
    print(f' - {bp.id}')

```

练习 22.4 : Python API 基础

目标

使用 CARLA Python API 在仿真世界中生成车辆、配置传感器并实现基础控制。

步骤

步骤 1：生成车辆

使用 `spawn_vehicles.py` 脚本在 Town03 中随机生成多辆车辆：

```
cd src/lab_code/ch22_lab/  
python3 spawn_vehicles.py --num-vehicles 10
```

该脚本的功能：

- 381. 连接 CARLA 服务器
- 382. 获取 Town03 地图的生成点
- 383. 随机选择车辆蓝图
- 384. 在随机生成点生成车辆
- 385. 打印每辆车的位置信息

步骤 2：设置 Traffic Manager

```
# 获取 Traffic Manager  
tm = client.get_trafficmanager(8000)  
tm.set_global_distance_to_leading_vehicle(2.0) # 跟车距离  
tm.set_synchronous_mode(False)                # 异步模式  
  
# 让生成的车辆自动驾驶  
for vehicle in vehicles:  
    vehicle.set_autopilot(True, tm.get_port())
```

步骤 3：附加 RGB 相机传感器

```
# 配置相机蓝图  
camera_bp = bp_lib.find('sensor.camera.rgb')  
camera_bp.set_attribute('image_size_x', '1280')  
camera_bp.set_attribute('image_size_y', '720')  
camera_bp.set_attribute('fov', '110')  
camera_bp.set_attribute('sensor_tick', '0.05') # 20FPS  
  
# 相机安装位置（前挡风玻璃处）  
camera_transform = carla.Transform(  
    carla.Location(x=1.5, z=2.4),  
    carla.Rotation(pitch=0, yaw=0, roll=0)  
)  
  
# 生成并附着到车辆  
camera = world.spawn_actor(  
    camera_bp, camera_transform, attach_to=vehicle  
)
```

```
# 定义回调保存图片
def save_image(image):
    image.save_to_disk(f'_out/{image.frame:06d}.png')

camera.listen(lambda image: save_image(image))
```

步骤 4：手动控制车辆

```
import pygame

# 使用键盘控制车辆移动
def control_vehicle(vehicle):
    control = carla.VehicleControl()
    control.throttle = 0.5
    control.steer = 0.0
    control.brake = 0.0

    # 根据按键更新控制
    keys = pygame.key.get_pressed()
    if keys[pygame.K_UP]:
        control.throttle = 0.7
    if keys[pygame.K_DOWN]:
        control.brake = 0.7
    if keys[pygame.K_LEFT]:
        control.steer = -0.3
    if keys[pygame.K_RIGHT]:
        control.steer = 0.3

    vehicle.apply_control(control)
```

实验总结

- 掌握了 CARLA 服务器的启动与客户端连接方法
- 理解了 World、Map、Blueprint、Actor 等核心概念
- 学会了使用 Python API 生成车辆和控制车辆
- 掌握了传感器（相机）的配置与数据采集
- 了解了 Traffic Manager 的基本用法

补充：Docker 部署方案（备用）

对于无法直接运行 CARLA 的环境（如 GPU 性能不足、非 Ubuntu 系统），可使用 Docker 运行 CARLA：

```
# 1. 拉取 CARLA Docker 镜像
docker pull carlasim/carla:0.9.16

# 2. 运行 CARLA 服务器（需 NVIDIA Container Toolkit）
docker run --name carla \
```

```
--privileged \  
--gpus all \  
--net=host \  
-e DISPLAY=${DISPLAY} \  
-e SDL_VIDEODRIVER=x11 \  
-v /tmp/.X11-unix:/tmp/.X11-unix:rw \  
carlasim/carla:0.9.16 \  
/bin/bash ./CarlaUE4.sh -vulkan -quality-level=Low  
  
# 3. Python 客户端直接在宿主机运行 (连接 localhost:2000)  
pip install carla-client # 轻量级客户端库  
python3 -c "import carla; c=carla.Client('localhost',2000); c.set_timeout(10);  
print(c.get_server_version())"
```

注意: Docker 方式性能开销较大, 推荐仅用于开发和功能验证。正式实验建议使用原生安装。

思考题

386. CARLA 采用 C/S 架构设计, 这种设计有什么优点? 在仿真实验中如果网络延迟较大, 会有什么影响?
387. 在同一场景中生成大量车辆 (如 50 辆以上) 时, 可能会遇到什么性能问题? 你有什么优化建议?
388. 对比 CARLA 中的 RGB 相机和语义分割相机, 它们在数据格式和应用场景上有何不同?

第 23 章 SLAM 与导航综合实训

学习目标

- 掌握从 SLAM 建图到自主导航的完整流程
- 能够在仿真环境中完成建图、保存地图和导航
- 理解多机器人调度系统的设计方法
- 培养项目文档编写和方案设计能力
- 掌握 SLAM 与导航系统集成的工程实践

23.1 综合项目概述

23.1.1 项目目标

本综合实训项目将引导学生完成一个完整的机器人自主导航系统开发：

项目阶段：	
阶段 1：SLAM 建图	→ 使用激光雷达构建环境地图
阶段 2：地图保存与处理	→ 保存 PGM/YAML 地图并后处理
阶段 3：AMCL 定位	→ 在已知地图上实现定位
阶段 4：Nav2 自主导航	→ 实现点到点自主导航
阶段 5：多机器人调度	→ 多机器人协同任务执行
阶段 6：项目文档与评估	→ 撰写设计文档和性能评估

实训环境：

- ROS 2 Jazzy
- Gazebo Sim Harmonic 仿真环境
- Wheeltec 机器人模型（课程包 robot_sim_demo）
- Nav2 导航框架

23.1.2 项目要求

基本要求：

- 389. 构建至少 500m²环境的完整地图
- 390. 实现稳定的自主导航（成功率>90%）

- 391. 支持多点导航和航点跟随
- 392. 具备基本故障恢复能力

进阶要求：

- 393. 多传感器融合建图（可选 IMU）
- 394. 自定义行为树优化导航
- 395. 多机器人协同导航
- 396. 动态避障和重规划

23.2 SLAM 建图实践

23.2.1 环境准备

```
# 1. 安装依赖
sudo apt install ros-jazzy-slam-toolbox ros-jazzy-nav2-bringup
sudo apt install ros-jazzy-cartographer-ros

# 2. 进入课程工作空间
mkdir -p ~/ros2_course_ws/src
cd ~/ros2_course_ws
colcon build
source install/setup.bash

# 3. 创建项目包（仅用于本综合实训的自定义节点）
cd ~/ros2_course_ws/src
ros2 pkg create slam_nav_project \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs nav2_msgs
```

23.2.2 使用 slam_toolbox 建图

```
# 启动课程 Wheeltec 仿真环境
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false

# 启动 slam_toolbox 在线建图
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file:=./config/mapper_params_online_async.yaml \
  use_sim_time:=true

# 控制机器人探索
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 保存地图
ros2 run nav2_map_server map_saver_cli -f ~/maps/final_map
```

自动探索建图脚本：


```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import OccupancyGrid
import numpy as np
import math

class AutoExplorer(Node):
    """自动探索建图节点"""
    def __init__(self):
        super().__init__('auto_explorer')

        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)

        # 状态
        self.front_dist = float('inf')
        self.left_dist = float('inf')
        self.right_dist = float('inf')
        self.map_coverage = 0.0
        self.state = 'explore' # explore, rotate, forward

        # 控制参数
        self.max_speed = 0.22
        self.max_rotation = 0.8
        self.safe_distance = 0.5

        self.timer = self.create_timer(0.1, self.control_loop)

    def scan_callback(self, msg: LaserScan):
        """分析激光数据"""
        ranges = np.array(msg.ranges)
        angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))

        # 前方区域 (-30°到30°)
        n = len(ranges)
        front_idx = n // 2
        window = n // 12
        front = ranges[front_idx-window:front_idx+window]
        self.front_dist = np.min(front[np.isfinite(front)])

        # 左方 (60°到120°)
        left_idx = n // 3
        left = ranges[left_idx-window:left_idx+window]
        self.left_dist = np.min(left[np.isfinite(left)])

        # 右方 (-120°到-60°)
        right_idx = 2 * n // 3
        right = ranges[right_idx-window:right_idx+window]
        self.right_dist = np.min(right[np.isfinite(right)])

    def map_callback(self, msg: OccupancyGrid):
        """计算地图覆盖率"""

```

```

data = np.array(msg.data)
total = data.size
mapped = np.sum(data >= 0)
self.map_coverage = mapped / total * 100

self.get_logger().info(
    f'地图覆盖率: {self.map_coverage:.1f}%',
    throttle_duration_sec=5.0)

def control_loop(self):
    """控制循环"""
    cmd = Twist()

    if self.map_coverage > 95.0:
        # 建图完成
        cmd.linear.x = 0.0
        cmd.angular.z = 0.0
        self.get_logger().info('建图完成!')
        self.timer.cancel()
        self.cmd_pub.publish(cmd)
        return

    if self.front_dist < self.safe_distance:
        # 前方障碍物, 旋转
        if self.left_dist > self.right_dist:
            cmd.angular.z = -self.max_rotation # 右转
        else:
            cmd.angular.z = self.max_rotation # 左转
        cmd.linear.x = 0.0
    elif self.front_dist < self.safe_distance * 2:
        # 接近障碍物, 减速转向
        cmd.linear.x = self.max_speed * 0.3
        if self.left_dist > self.right_dist:
            cmd.angular.z = -self.max_rotation * 0.5
        else:
            cmd.angular.z = self.max_rotation * 0.5
    else:
        # 安全距离, 前进
        cmd.linear.x = self.max_speed
        cmd.angular.z = 0.2 # 轻微左偏探索

    self.cmd_pub.publish(cmd)

def main():
    rclpy.init()
    node = AutoExplorer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

23.2.3 使用 Cartographer 建图

```
# Cartographer 建图（使用课程实验包中的配置）
ros2 launch slam_lab cartographer_mapping.launch.py \
  configuration_directory:=src/lab_code/ch10_lab/slam_lab/config/cartographer \
  configuration_basename:=xbot_2d.lua \
  use_sim_time:=true

# 控制机器人
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 保存 pstream 地图
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState \
  "{filename: '$HOME/maps/office.pstream'}"

# 转换为 ROS 标准地图
ros2 run cartographer_ros cartographer_pstream_to_ros_map \
  -pstream_filename $HOME/maps/office.pstream \
  -map_filestem $HOME/maps/office
```

23.2.4 地图后处理

```
import cv2
import numpy as np
import yaml

class MapPostProcessor:
    """地图后处理工具"""
    def __init__(self, pgm_path: str, yaml_path: str):
        self.pgm_path = pgm_path
        self.yaml_path = yaml_path

    # 读取地图
    self.image = cv2.imread(pgm_path, cv2.IMREAD_GRAYSCALE)

    with open(yaml_path, 'r') as f:
        self.meta = yaml.safe_load(f)

    def clean_noise(self, kernel_size: int = 3):
        """去除地图噪声（中值滤波）"""
        self.image = cv2.medianBlur(self.image, kernel_size)

    def close_holes(self, kernel_size: int = 5):
        """闭运算填补小孔"""
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
                                           (kernel_size, kernel_size))
        self.image = cv2.morphologyEx(self.image, cv2.MORPH_CLOSE, kernel)

    def inflate_obstacles(self, pixel_radius: int = 5):
        """膨胀障碍物（增加安全边界）"""
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
                                           (pixel_radius*2+1, pixel_radius*2+1))

        # 二值化：白色=空闲，黑色=障碍物
        _, binary = cv2.threshold(self.image, 200, 255, cv2.THRESH_BINARY)
```

```

        # 膨胀障碍物
        dilated = cv2.erode(binary, kernel)

        # 恢复原灰度图
        self.image[dilated == 0] = 0

    def crop_map(self):
        """裁剪地图空白区域"""
        # 找到非空白区域
        coords = cv2.findNonZero(self.image)
        if coords is not None:
            x, y, w, h = cv2.boundingRect(coords)
            self.image = self.image[y:y+h, x:x+w]

        # 更新元数据
        self.meta['origin'][0] += x * self.meta['resolution']
        self.meta['origin'][1] += y * self.meta['resolution']

    def save(self, output_prefix: str):
        """保存处理后的地图"""
        pgm_out = f'{output_prefix}.pgm'
        yaml_out = f'{output_prefix}.yaml'

        cv2.imwrite(pgm_out, self.image)

        self.meta['image'] = f'{output_prefix.split("/")[-1]}.pgm'
        with open(yaml_out, 'w') as f:
            yaml.dump(self.meta, f, default_flow_style=False)

        print(f'地图已保存: {pgm_out}, {yaml_out}')

    def visualize(self):
        """显示地图"""
        cv2.imshow('Map', self.image)
        cv2.waitKey(0)
        cv2.destroyAllWindows()

# 使用示例
processor = MapPostProcessor('~/maps/final_map.pgm', '~/maps/final_map.yaml')
processor.clean_noise()
processor.close_holes()
processor.inflate_obstacles(pixel_radius=3)
processor.crop_map()
processor.save('~/maps/processed_map')

```

23.3 自主导航配置

23.3.1 Nav2 参数配置

```

# nav2_params.yaml - 完整导航配置
bt_navigator:
  ros_parameters:
    default_bt_xml_filename: "navigate_to_pose.xml"
  plugin_lib_names:

```

- nav2_compute_path_to_pose_action_bt_node
- nav2_follow_path_action_bt_node
- nav2_spin_action_bt_node
- nav2_back_up_action_bt_node
- nav2_wait_action_bt_node
- nav2_clear_costmap_service_bt_node
- nav2_recovery_node_bt_node
- nav2_pipeline_sequence_bt_node

planner_server:

```
ros__parameters:
  planner_plugins: ["GridBased"]
  GridBased:
    plugin: "nav2_navfn_planner/NavfnPlanner"
    tolerance: 0.5
    use_astar: true
```

controller_server:

```
ros__parameters:
  controller_plugins: ["FollowPath"]
  FollowPath:
    plugin: "dwb_core::DWBLocalPlanner"
    min_vel_x: 0.0
    max_vel_x: 0.5
    max_vel_theta: 1.0
    acc_lim_x: 2.5
    acc_lim_theta: 3.2
    critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
              "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
    RotateToGoal.scale: 32.0
    BaseObstacle.scale: 32.0
    PathAlign.scale: 32.0
    GoalDist.scale: 24.0
```

local_costmap:

```
local_costmap:
  ros__parameters:
    update_frequency: 5.0
    publish_frequency: 2.0
    global_frame: odom
    robot_base_frame: base_link
    rolling_window: true
    width: 4
    height: 4
    resolution: 0.05
    plugins: ["voxel_layer", "inflation_layer"]
    voxel_layer:
      plugin: "nav2_costmap_2d::VoxelLayer"
      enabled: True
      observation_sources: scan
      scan:
        topic: /scan
        max_obstacle_height: 2.0
        clearing: True
        marking: True
    inflation_layer:
      plugin: "nav2_costmap_2d::InflationLayer"
      inflation_radius: 0.55
      cost_scaling_factor: 3.0
```

```

global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      resolution: 0.05
      track_unknown_space: True
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: True
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: True
        observation_sources: scan
        scan:
          topic: /scan
          max_obstacle_height: 2.0
          clearing: True
          marking: True
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        inflation_radius: 0.55
        cost_scaling_factor: 3.0

amcl:
  ros__parameters:
    max_particles: 2000
    min_particles: 500
    laser_model_type: "likelihood_field"
    update_min_a: 0.2
    update_min_d: 0.1
    z_hit: 0.95
    z_rand: 0.05
    sigma_hit: 0.2
    initial_pose.x: 0.0
    initial_pose.y: 0.0
    initial_pose.yaw: 0.0

behavior_server:
  ros__parameters:
    behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]
    spin:
      plugin: "nav2_behaviors/Spin"
      max_rotational_vel: 1.0
    backup:
      plugin: "nav2_behaviors/BackUp"
      max_linear_vel: 0.1

```

23.3.2 导航启动文件

```

# bringup_navigation.py
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

```

```

import os

def generate_launch_description():
    nav2_bringup_dir = get_package_share_directory('nav2_bringup')

    return LaunchDescription([
        # 1. 启动仿真环境
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(
                    get_package_share_directory('robot_sim_demo'),
                    'launch/gazebo2.launch.py'
                )
            ),
        ),

        # 2. 启动 AMCL 定位
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(nav2_bringup_dir, 'launch/localization_launch.py')
            ),
            launch_arguments={
                'map': os.path.expanduser('~/.maps/processed_map.yaml'),
                'use_sim_time': 'true',
            }.items()
        ),

        # 3. 启动 Nav2 导航
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(nav2_bringup_dir, 'launch/navigation_launch.py')
            ),
            launch_arguments={
                'params_file': os.path.join(
                    get_package_share_directory('slam_nav_project'),
                    'config/nav2_params.yaml'
                ),
                'use_sim_time': 'true',
            }.items()
        ),

        # 4. 启动 RViz2
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
            arguments=['-d', os.path.join(
                get_package_share_directory('nav2_bringup'),
                'rviz/nav2_default_view.rviz'
            )]
        ),
    ])

```

23.3.3 多点导航控制台

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

```

```

from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
from geometry_msgs.msg import PoseStamped
import math
import yaml

class NavigationConsole(Node):
    """导航控制台"""
    def __init__(self):
        super().__init__('navigation_console')
        self.navigator = BasicNavigator()

    def wait_for_nav2(self):
        """等待 Nav2 就绪"""

        self.get_logger().info('等待 Nav2 就绪...')
        self.navigator.waitUntilNav2Active()
        self.get_logger().info('Nav2 就绪!')

    def create_pose(self, x: float, y: float, yaw: float = 0.0) -> PoseStamped:
        """创建位姿"""
        pose = PoseStamped()
        pose.header.frame_id = 'map'
        pose.header.stamp = self.navigator.get_clock().now().to_msg()
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.orientation.z = math.sin(yaw / 2.0)
        pose.pose.orientation.w = math.cos(yaw / 2.0)
        return pose

    def navigate_to_pose(self, x: float, y: float, yaw: float = 0.0) -> bool:
        """导航到目标点"""
        goal = self.create_pose(x, y, yaw)
        self.get_logger().info(f'导航至: ({x:.2f}, {y:.2f}, {math.degrees(yaw):.1f}°)')

        self.navigator.goToPose(goal)

        while not self.navigator.isTaskComplete():
            feedback = self.navigator.getFeedback()
            if feedback:
                self.get_logger().info(
                    f'剩余距离: {feedback.distance_remaining:.2f}m',
                    throttle_duration_sec=2.0
                )

            result = self.navigator.getResult()
            if result == TaskResult.SUCCEEDED:
                self.get_logger().info('目标到达!')
                return True
            else:
                self.get_logger().error(f'导航失败: {result}')
                return False

    def follow_waypoints(self, waypoints: list) -> bool:
        """航点导航"""
        poses = [self.create_pose(x, y, yaw) for x, y, yaw in waypoints]
        self.get_logger().info(f'开始航点导航: {len(poses)}个目标')

```



```

        self.navigators.followWaypoints(poses)

    while not self.navigators.isTaskComplete():
        feedback = self.navigators.getFeedback()
        if feedback:
            self.get_logger().info(
                f'前往航点 #{feedback.current_waypoint}/{len(poses)}',
                throttle_duration_sec=2.0
            )

    result = self.navigators.getResult()
    return result == TaskResult.SUCCEEDED

def patrol(self, waypoints: list, loops: int = 3):
    """巡逻任务"""
    for loop in range(loops):
        self.get_logger().info(f'--- 巡逻第 {loop+1}/{loops} 轮 ---')
        for x, y, yaw in waypoints:
            if not self.navigate_to_pose(x, y, yaw):
                self.get_logger().error('巡逻中断')
                return
        self.get_logger().info('巡逻任务完成!')

def load_mission(self, yaml_file: str):
    """从YAML加载任务"""
    with open(yaml_file, 'r') as f:
        mission = yaml.safe_load(f)

    self.get_logger().info(f'加载任务: {mission["name"]}')

    for task in mission['tasks']:
        if task['type'] == 'navigation':
            self.navigate_to_pose(
                task['x'], task['y'], task.get('yaw', 0.0)
            )
        elif task['type'] == 'wait':
            self.get_logger().info(f'等待 {task["duration"]}s')
            rclpy.spin_once(self, timeout_sec=task['duration'])
        elif task['type'] == 'patrol':
            self.patrol(task['waypoints'], task.get('loops', 3))

def main():
    rclpy.init()
    console = NavigationConsole()
    console.wait_for_nav2()

    # 示例：多点导航
    waypoints = [
        (1.0, 0.5, 0.0),
        (2.0, 1.0, math.radians(45)),
        (1.5, 2.0, math.radians(90)),
        (0.0, 1.5, math.radians(180)),
    ]
    console.patrol(waypoints, loops=2)

```

```
rclpy.shutdown()
```

23.4 多机器人调度

23.4.1 多机器人架构

```
import rclpy
from rclpy.node import Node
from nav2_msgs.action import NavigateToPose
from rclpy.action import ActionClient
import yaml
import threading

class MultiRobotScheduler(Node):
    """多机器人调度器"""
    def __init__(self):
        super().__init__('multi_robot_scheduler')

        # 机器人客户端
        self.robots = {}
        robot_configs = [
            ('robot1', '/robot1/navigate_to_pose'),
            ('robot2', '/robot2/navigate_to_pose'),
            ('robot3', '/robot3/navigate_to_pose'),
        ]

        for name, action_name in robot_configs:
            self.robots[name] = {
                'client': ActionClient(self, NavigateToPose, action_name),
                'status': 'idle',
                'position': None
            }

        # 任务队列
        self.task_queue = []
        self.lock = threading.Lock()

        # 调度定时器
        self.timer = self.create_timer(1.0, self.dispatch_loop)

        self.get_logger().info('多机器人调度器已启动')

    def add_task(self, robot: str, x: float, y: float, yaw: float = 0.0,
                priority: int = 0):
        """添加任务"""
        with self.lock:
            self.task_queue.append({
                'robot': robot,
                'x': x, 'y': y, 'yaw': yaw,
                'priority': priority,
                'status': 'pending'
            })

        self.get_logger().info(
```

```

        f'任务已添加: {robot} → ({x:.1f}, {y:.1f})')

def dispatch_loop(self):
    """调度循环"""
    with self.lock:
        # 按优先级排序
        self.task_queue.sort(key=lambda t: -t['priority'])

        for task in self.task_queue:
            if task['status'] != 'pending':
                continue

            robot = self.robots[task['robot']]
            if robot['status'] != 'idle':
                continue

            # 派发任务
            self._send_goal(task['robot'], task)
            task['status'] = 'dispatched'
            robot['status'] = 'busy'

def _send_goal(self, robot_name: str, task: dict):
    """发送导航目标"""
    robot = self.robots[robot_name]
    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.pose.position.x = task['x']
    goal.pose.pose.position.y = task['y']

    yaw = task.get('yaw', 0.0)
    goal.pose.pose.orientation.z = math.sin(yaw / 2.0)
    goal.pose.pose.orientation.w = math.cos(yaw / 2.0)

    client = robot['client']
    client.wait_for_server()

    send_goal_future = client.send_goal_async(goal)
    send_goal_future.add_done_callback(
        lambda future, r=robot_name: self._goal_callback(future, r))

def _goal_callback(self, future, robot_name: str):
    """目标响应回调"""
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().error(f'{robot_name}: 目标被拒绝')
        self.robots[robot_name]['status'] = 'idle'
        return

    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(
        lambda future, r=robot_name: self._result_callback(future, r))

def _result_callback(self, future, robot_name: str):
    """结果回调"""
    result = future.result().result
    status = future.result().status

```

```

        self.robots[robot_name]['status'] = 'idle'

    if status == 4: # SUCCEEDED
        self.get_logger().info(f'{robot_name}: 任务完成')

        # 更新位置
        self.robots[robot_name]['position'] = (
            result.pose.position.x, result.pose.position.y
        )
    else:
        self.get_logger().error(f'{robot_name}: 任务失败 {status}')

def get_status(self) -> dict:
    """获取所有机器人状态"""
    status = {}
    for name, robot in self.robots.items():
        status[name] = {
            'status': robot['status'],
            'position': robot['position']
        }
    return status

class ConflictResolver:
    """冲突解决器"""
    def __init__(self):
        self.occupied_zones = {} # 区域占用

    def check_conflict(self, robot_name: str,
                      goal_x: float, goal_y: float) -> bool:
        """检查是否存在冲突"""
        for other_robot, zone in self.occupied_zones.items():
            if other_robot == robot_name:
                continue

            dist = math.sqrt((zone['x'] - goal_x)**2 +
                             (zone['y'] - goal_y)**2)

            if dist < 1.0: # 1m 内视为冲突
                return True
        return False

    def reserve_zone(self, robot_name: str,
                    x: float, y: float, radius: float = 1.0):
        """预留区域"""
        self.occupied_zones[robot_name] = {'x': x, 'y': y, 'r': radius}

    def release_zone(self, robot_name: str):
        """释放区域"""
        self.occupied_zones.pop(robot_name, None)

```

23.4.2 多机器人仿真启动

多机器人启动是设计扩展，本仓库当前只提供单 wheeltec 仿真入口：

```
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false
```

```

# 分别为每个机器人启动 SLAM 和导航
# 终端 1：机器人 1
ROS_NAMESPACE=robot1 \
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file=./config/robot1_params.yaml

# 终端 2：机器人 2
ROS_NAMESPACE=robot2 \
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file=./config/robot2_params.yaml

# 启动调度器
ros2 run slam_nav_project multi_robot_scheduler

```

23.5 项目文档规范

23.5.1 项目文档结构

```

slam_nav_project/
├── README.md           # 项目概览
├── docs/
│   ├── requirements.md # 需求文档
│   ├── architecture.md # 架构设计
│   ├── api_reference.md # API 文档
│   ├── test_report.md  # 测试报告
│   └── user_manual.md  # 用户手册
├── config/
│   ├── nav2_params.yaml # 导航参数
│   ├── mapper_params.yaml # 建图参数
│   └── robot_config.yaml # 机器人配置
├── launch/
│   ├── slam_nav_bringup.py # 主启动文件
│   └── multi_robot.launch.py # 多机器人启动
├── maps/
│   ├── raw_map.pgm      # 原始建图
│   ├── raw_map.yaml
│   ├── processed_map.pgm # 处理后地图
│   └── processed_map.yaml
├── scripts/
│   ├── auto_explorer.py # 自动探索
│   ├── navigation_console.py # 导航控制台
│   └── multi_robot_scheduler.py # 多机器人调度
└── tests/
    ├── test_navigation.py # 导航测试
    └── test_multi_robot.py # 多机器人测试

```

23.5.2 设计文档模板

```
# 项目名称：SLAM 与导航综合实训项目

## 1. 项目概述
- 项目目标：
- 技术栈：ROS 2 Jazzy, Wheeltec, Nav2, slam_toolbox
- 实现功能：

## 2. 系统架构
### 2.1 架构图
[系统架构图]

### 2.2 模块说明
| 模块名称 | 功能描述 | 输入 | 输出 |
|-----|-----|-----|-----|
| SLAM 建图 | 构建环境地图 | /scan, /tf | /map |
| AMCL 定位 | 已知地图定位 | /scan, /map, /tf | /amcl_pose |
| 全局规划 | 全局路径规划 | 目标点, 代价地图 | /plan |
| 局部控制 | 路径跟踪 | /plan, /scan | /cmd_vel |

## 3. 建图方案
### 3.1 算法选择
- 使用 slam_toolbox 在线异步建图
- 地图分辨率：0.05m
- 建图方式：自动探索

### 3.2 建图参数
[关键参数说明]

## 4. 导航方案
### 4.1 路径规划
- 全局规划：Navfn + A*
- 局部控制：DWB

### 4.2 定位方案
- AMCL 粒子滤波
- 初始位姿设定

## 5. 测试与评估
### 5.1 建图评估
| 指标 | 数值 |
|-----|-----|
| 地图覆盖率 | 95% |
| 建图时间 | 15min |
```

| 回环闭合 | 成功 |

5.2 导航评估

| 指标 | 数值 |

|-----|-----|

| 导航成功率 | 93% |

| 平均规划时间 | 0.5s |

| 路径长度误差 | <5% |

6. 问题与解决方案

[记录开发过程中遇到的问题和解决方案]

23.5.3 测试方案

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
import time
import math

class NavigationTester(Node):
    """导航性能测试"""
    def __init__(self):
        super().__init__('navigation_tester')

        self.results = {
            'success_count': 0,
            'fail_count': 0,
            'planning_times': [],
            'navigation_times': [],
            'path_lengths': []
        }

    def run_test_suite(self, test_cases: list):
        """执行测试套件"""

        self.get_logger().info(f'开始测试: {len(test_cases)}个用例')

        for i, test in enumerate(test_cases):
            self.get_logger().info(f'测试 {i+1}/{len(test_cases)}')

            start_time = time.time()
            success = self._execute_test(test)
            elapsed = time.time() - start_time

            if success:
                self.results['success_count'] += 1
            else:
                self.results['fail_count'] += 1

            self.results['navigation_times'].append(elapsed)

        self._print_report()

    def _execute_test(self, test: dict) -> bool:
```

```

        """执行单个测试"""
        # 导航到测试点
        # ...
        return True

    def _print_report(self):
        """打印测试报告"""
        total = (self.results['success_count'] +
                 self.results['fail_count'])
        success_rate = (self.results['success_count'] / total * 100)

        avg_time = (sum(self.results['navigation_times']) /
                    len(self.results['navigation_times']))

        print('=' * 50)
        print(f'测试报告')
        print('=' * 50)
        print(f'总用例: {total}')
        print(f'成功: {self.results["success_count"]}')
        print(f'失败: {self.results["fail_count"]}')
        print(f'成功率: {success_rate:.1f}%')
        print(f'平均导航时间: {avg_time:.2f}s')
        print('=' * 50)

```

23.6 性能评估与优化

23.6.1 建图质量评估

```

import numpy as np
import cv2

class MapEvaluator:
    """建图质量评估"""
    def __init__(self, map_pgm: str, ground_truth_pgm: str = None):
        self.map = cv2.imread(map_pgm, cv2.IMREAD_GRAYSCALE)
        self.ground_truth = None
        if ground_truth_pgm:
            self.ground_truth = cv2.imread(ground_truth_pgm, cv2.IMREAD_GRAYSCALE)

    def compute_coverage(self) -> float:
        """计算地图覆盖率"""
        explored = np.sum(self.map != 205) # 非灰色区域
        total = self.map.size
        return explored / total * 100

    def compute_accuracy(self) -> float:
        """计算建图精度（如果有真值）"""
        if self.ground_truth is None:
            return 0.0

        # 简单的像素级对比
        diff = cv2.absdiff(self.map, self.ground_truth)

```



```

        accuracy = 1.0 - (np.sum(diff > 50) / diff.size)
        return accuracy

    def compute_clarity(self) -> float:
        """计算地图清晰度（边缘锐度）"""
        edges = cv2.Canny(self.map, 50, 150)
        edge_density = np.sum(edges > 0) / edges.size
        return edge_density

    def full_report(self) -> dict:
        """完整评估报告"""
        return {
            'coverage': self.compute_coverage(),
            'accuracy': self.compute_accuracy(),
            'clarity': self.compute_clarity(),
            'resolution': '0.05m/pixel',
            'total_cells': self.map.size
        }

```

23.6.2 导航性能优化清单

导航性能优化清单

建图阶段

- [] 充分探索所有区域（覆盖率>95%）
- [] 地图后处理（噪声滤波、空洞填补）
- [] 设置合适的地图原点
- [] 检查地图与真值匹配度

定位阶段

- [] 正确设置初始位姿
- [] 调整 AMCL 粒子数（500-2000）
- [] 选择适合的激光模型（似然域/波束）
- [] 检查 TF 变换正确性

规划阶段

- [] 选择适合的全局规划器
- [] 调整膨胀半径（机器人半径×2）
- [] 优化代价地图更新频率
- [] 设置合理的规划超时时间

控制阶段

- [] 调整最大速度（安全范围内）
- [] 优化 DWB 评分器权重
- [] 配置合理的恢复行为
- [] 调节前瞻距离（Pure Pursuit）

系统级优化

- [] 降低代价地图分辨率（大型环境）
- [] 增加局部地图尺寸
- [] 配置生命周期节点自动启动
- [] 启用 fast_odom 模式

23.7 常见问题排查

```
# troubleshooting.py
class TroubleshootingGuide:
    """故障排查指南"""

    @staticmethod
    def check_slam_issues():
        """SLAM 问题排查"""
        print("""
        SLAM 常见问题:

        1. 建图漂移
            → 检查激光频率(>5Hz)
            → 降低移动速度
            → 启用回环检测

        2. 地图错位
            → 检查里程计精度
            → 增加粒子数
            → 调整扫描匹配参数

        3. 地图不完整
            → 增加探索时间
            → 检查激光范围设置
            → 确保死角被覆盖
        """)

    @staticmethod
    def check_navigation_issues():
        """导航问题排查"""
        print("""
        导航常见问题:

        1. 路径规划失败
            → 检查代价地图状态
            → 检查起点/终点是否可通行
            → 清除代价地图重试

        2. 导航卡住
            → 增大膨胀半径
            → 降低最大速度
        """)
```

→ 检查恢复行为配置

3. 定位丢失

- 检查 AMCL 状态
- 重新设置初始位姿
- 增加粒子数

4. 避障失败

- 检查传感器数据
- 调整障碍物层参数
- 减小最大速度

""")

```
@staticmethod
def run_diagnostics():
    """运行诊断"""
    commands = """
    # 检查话题
    ros2 topic list
    ros2 topic hz /scan /odom /tf

    # 检查节点
    ros2 node list
    ros2 lifecycle list

    # 检查参数
    ros2 param list /planner_server
    ros2 param list /controller_server

    # 检查 TF 树
    ros2 run tf2_tools view_frames.py

    # 查看代价地图
    ros2 topic echo /global_costmap/costmap --once

    # 查看 AMCL 状态
    ros2 topic echo /particlecloud --once
    """
    print("诊断命令:")
    print(commands)
```

课后练习

397. **操作题:** 完成从 SLAM 建图到自主导航的完整流程: 启动仿真→slam_toolbox 建图→保存地图→启动 AMCL 定位→发送导航目标到达目标点。
398. **编程题:** 编写一个完整的自动探索建图节点, 支持在未知环境中自主移动完成建图, 并在覆盖率超过 95%时自动保存地图。

399. **设计题:** 设计一个支持 3 个机器人的多机器人调度系统, 要求支持任务分配、冲突避免和状态监控, 给出系统架构图和核心代码框架。
400. **分析题:** 分析从 SLAM 建图到自主导航的完整流程中, 哪些环节最容易引入误差, 如何通过参数调优和算法选择来减小误差。
401. **文档题:** 仿照本节的设计文档模板, 为你的 SLAM 与导航项目撰写完整的设计文档, 包含需求分析、系统架构、参数配置、测试方案和问题记录。
402. **综合题:** 设计一个面向大型商超的机器人服务方案 (包括清洁、巡检、导购等多类型机器人), 涵盖 SLAM 建图方案 (跨楼层)、多机器人调度 (任务优先级)、人机交互 (目标下达) 和系统监控 (健康管理)。

实验 23:

第 23 章 实验: CARLA-ROS2 桥接与车辆部署

****对应理论章节**:** 第 37 章《CARLA-ROS2 桥接与车辆部署》 ****实验课时**:** 3 课时

****实验代码**:** `src/lab\_code/ch23\_lab/`

实验目标

- 掌握 CARLA 仿真器与 ROS2 Bridge 的启动流程
- 学会通过 Python 脚本生成 Ego Vehicle
- 能够在 RViz2 中可视化 CARLA 传感器数据
- 实现键盘控制车辆移动

实验环境

- ROS 2 Jazzy
- CARLA 0.9.16
- carla_msgs / carla_ros_bridge (源码安装)
- Python3 + keyboard 库
- RViz2

实验准备

安装依赖

```
# 安装 CARLA ROS2 Bridge (由课程安装脚本固定版本和提交)
cd /path/to/Technologies-of-ROS2-Programming-master
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash
cd ~/carla_ws
rosdep install --from-paths src --ignore-src -r -y
# 可选: 使用 gezp 维护版 (兼容性更好)
# ROS 2 Jazzy 下建议使用课程根目录 setup_course.sh 管理的固定 bridge commit
# 编译前确保 PYTHONPATH 包含 CARLA 的.egg 文件
```

```
export CARLA_ROOT=~/.carla
export PYTHONPATH=$PYTHONPATH:$CARLA_ROOT/PythonAPI/carla:$CARLA_ROOT/PythonAPI/carla/dist

colcon build
source install/setup.bash
```

启动 CARLA 仿真器

```
# Linux
./CarlaUE4.sh -quality-level=Low

# Windows (CARLA 安装在 C:/CARLA)
C:/CARLA/CarlaUE4.exe -quality-level=Low
```

验证环境就绪

在开始实验前，执行以下预检查确认所有组件正常：

```
# 1. 检查 CARLA 服务器运行状态
pgrep -x CarlaUE4 && echo "CARLA 服务器运行中" || echo "CARLA 未启动"

# 2. 验证 ROS2 环境
ros2 node list 2>/dev/null | head -5 || echo "ROS2 环境未加载, 请先 source install/setup.bash"

# 3. 验证 CARLA Python API
python3 -c "import carla; c=carla.Client('localhost',2000); c.set_timeout(5); print(f'CARLA {c.get_server_version()}')" 2>&1

# 4. 验证 Bridge 可启动
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py --show-args 2>&1 | head -3
```

练习 23.1: 启动 CARLA+ROS2 Bridge

步骤

403. 确保 CARLA 仿真器已在运行（观察 Town 地图窗口）

404. 启动 ROS2 Bridge：

```
source ~/.carla_ws/install/setup.bash
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  synchronous_mode:=False
```

405. 在新终端中验证 Bridge 节点已启动：

```
ros2 node list
# 应看到: /carla_ros_bridge
```

406. 查看 Bridge 发布的话题：

```
ros2 topic list | grep carla
```

预期输出

```
/carla/status
/carla/weather_control
/carla/traffic_lights
/carla/objects
...
```

检查要点

- [] CARLA 仿真器窗口正常显示
- [] Bridge 节点 /carla_ros_bridge 存在
- [] 至少能看到 /carla/status 话题

练习 23.2: 生成 Ego Vehicle 并在 RViz 中可视化

步骤

407. 运行 spawn_ego.py 生成 Ego Vehicle:

```
cd src/lab_code/ch23_lab/
python3 spawn_ego.py --spawn-point 10
```

408. 验证新话题出现:

```
ros2 topic list | grep ego_vehicle
```

409. 启动 RViz2 并可视化传感器数据:

```
rviz2
```

- 410. 在 RViz2 中添加以下显示:
- 411. 添加 Image 显示, Topic 选择 /carla/ego_vehicle/rgb_front/image
- 412. 添加 PointCloud2 显示, Topic 选择 /carla/ego_vehicle/lidar
- 413. 添加 TF 显示, 查看坐标树

spawn_ego.py 参数说明

参数	默认值	说明
--host`	`localhost`	CARLA 服务器地址
--port`	`2000`	CARLA 服务器端口

<code>--spawn-point`</code>	<code>`0`</code>	生成点索引
<code>--role-name`</code>	<code>`ego_vehicle`</code>	角色名
<code>--blueprint`</code>	<code>`vehicle.tesla.model3`</code>	车型 Blueprint

预期输出

```
Ego Vehicle spawned at: (x=36.28, y=52.38, z=0.30)
Role name: ego_vehicle
Blueprint: vehicle.tesla.model3
```

检查要点

- [] Ego Vehicle 出现在 CARLA 仿真窗口中
- [] `/carla/ego_vehicle/*` 话题组出现
- [] RViz2 中能看到 RGB 图像
- [] RViz2 中能看到激光雷达点云
- [] TF 树显示完整的坐标关系

练习 23.3: 键盘控制车辆移动

步骤

414. 确保 CARLA + Bridge + Ego Vehicle 均在运行

415. 使用 `check_topics.py` 验证所有必要话题：

```
python3 check_topics.py
```

416. 启动键盘控制节点（使用 ROS2 内置工具）：

```
# 方法一：使用 teleop_twist_keyboard (需转换消息)
sudo apt install ros-jazzy-teleop-twist-keyboard
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -r /cmd_vel:=/carla/ego_vehicle/vehicle_control_cmd
```

417. 或者直接发布控制指令测试：

```
# 前进 (油门 0.5)
ros2 topic pub /carla/ego_vehicle/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl \
  "{throttle: 0.5, brake: 0.0, steer: 0.0}" --once
```



```
# 右转 (油门 0.3 + 转向 0.5)
ros2 topic pub /carla/ego_vehicle/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl \
  "{throttle: 0.3, brake: 0.0, steer: 0.5}" --once

# 刹车急停
ros2 topic pub /carla/ego_vehicle/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl \
  "{throttle: 0.0, brake: 1.0, steer: 0.0}" --once
```

418. 测试自动驾驶模式切换：

```
# 启用自动驾驶
ros2 service call /carla/ego_vehicle/enable_autopilot \
  std_srvs/srv/SetBool "{data: true}"

# 观察车辆自动行驶 → 切回手动
ros2 service call /carla/ego_vehicle/enable_autopilot \
  std_srvs/srv/SetBool "{data: false}"
```

预期行为

- 发送 throttle 指令后车辆向前移动
- 发送 steer 指令后车辆转向
- 启用自动驾驶后车辆沿车道自动行驶
- 禁用自动驾驶后车辆停止自主控制

检查要点

- [] check_topics.py 报告所有话题正常
 - [] 发布 throttle 指令后车辆移动
 - [] 发布 steer 指令后车辆转向
 - [] 自动驾驶模式下车辆自主行驶
 - [] 模式切换功能正常
-

练习 23.4: 端到端验证与排障

目标

使用 `check_topics.py` 和 `bridge_launch.sh` 对 CARLA-ROS2 Bridge 进行完整的端到端验证, 并掌握常见问题的诊断方法。

步骤

步骤 1: 一键启动全栈

```
cd src/lab_code/ch23_lab/  
bash bridge_launch.sh ~/carla/CarlaUE4.sh 10
```

该脚本会自动: 检查 CARLA 进程 → 启动 CARLA (如未运行) → Source ROS2 工作空间 → 启动 Bridge → 生成 Ego Vehicle → 验证话题。

步骤 2: 运行话题检查

```
# 检查所有 CARLA 话题  
python3 check_topics.py --verbose  
  
# 检查特定 role_name 的话题  
python3 check_topics.py --role-name ego_vehicle --verbose
```

步骤 3: 诊断常用工具

```
# 查看节点通信图  
rqt_graph  
  
# 查看话题带宽和频率  
ros2 topic bw /carla/ego_vehicle/rgb_front/image  
ros2 topic hz /carla/ego_vehicle/odometry  
  
# 查看 TF 树  
ros2 run tf2_tools view_frames.py  
  
# 查看消息内容  
ros2 topic echo /carla/ego_vehicle/vehicle_status --once
```

步骤 4: 保存诊断报告

```
# 保存环境快照  
{  
  echo "=== CARLA Version ==="  
  python3 -c "import carla; c=carla.Client('localhost',2000); c.set_timeout(5);  
  print(c.get_server_version())"  
  echo "=== ROS2 Topics ==="
```

```

ros2 topic list | grep carla
echo "=== ROS2 Nodes ==="
ros2 node list
echo "=== TF Tree ==="
ros2 run tf2_tools view_frames.py
} > carla_diagnostic_report.txt 2>&1
echo "诊断报告已保存至 carla_diagnostic_report.txt"

```

检查清单

- [] bridge_launch.sh 一键启动成功
- [] check_topics.py 报告所有关键话题正常
- [] rqt_graph 显示完整的话题通信图
- [] TF 树文件 frames.pdf 生成成功
- [] 诊断报告包含所有组件信息

常见问题诊断表

症状	可能原因	诊断命令	解决方案
`ros2 node list` 无 `/carla_ros_bridge`	Bridge 未启动或 卡死	`ps aux \	grep carla`
`ros2 topic list` 无 `/carla/` 话题	Bridge 未连接 CARLA	`ros2 service call /carla/get_available_ maps`	确认 CARLA 服务器运行中，端口 2000
`/carla/ego_vehicle/*` 话题缺失	Ego Vehicle 未生成	`ros2 service list \	grep carla`
图像话题无数据	相机传感器 未附着	`ros2 topic hz /carla/ego_vehicle/rgb_front/image`	检查 `spawn_ego.py` 中传感器配置
`ModuleNotFoundError: No`	PYTHONPATH	`echo \$PYTHONPATH`	`export PYTHONPATH=\$PYTHONPATH:~

module named 'carla'	缺失		/carla/PythonAPI/carla`
Bridge 启动报 `carla_msgs` 错误	消息包未 构建	`ros2 pkg list \	grep carla`
`colcon build` 失败	Python 版本不匹 配	`python3 --version`	确认 `.egg` 文件 Python 版本与系统 一致

实验报告要求

419. 记录 Bridge 启动后的完整话题列表 (ros2 topic list)
420. 截取 RViz2 中 RGB 图像和激光雷达点云的可视化效果图
421. 描述键盘控制车辆的实验过程与观察结果
422. 对比手动控制与自动驾驶模式下车辆行为差异
423. 记录遇到的问题及解决方法

思考题

424. 同步模式和异步模式对传感器数据时序有什么影响?
425. 如果同时生成两辆 Ego Vehicle (role_name 不同), Bridge 如何处理话题命名?
426. CarlaEgoVehicleControl 中的 steer 值为正时车辆往哪个方向转弯?

Part 3: 机械臂编程技术 (具身智能 机器人操作系统)

第 24 章 机械臂基础知识

学习目标

- 理解机械臂的分类与应用场景
- 掌握机器人运动学基础，包括 DH 参数建模
- 理解正运动学与逆运动学的基本原理
- 了解机械臂工作空间的概念与分析方法
- 掌握 ROS2 中机械臂接口的使用方法

24.1 机械臂分类

24.1.1 按机械结构分类

机械臂按照机械结构可分为以下几种主要类型：

关节型 (Articulated) 机械臂：

关节型机械臂是最常见的工业机器人类型，由一系列旋转关节串联而成，模拟人类手臂的运动方式。典型的 6 轴关节型机械臂具有 6 个自由度，可以在三维空间内实现任意位姿。

关节型结构：

基座 → 腰关节(J1) → 肩关节(J2) → 肘关节(J3)
→ 腕关节(J4) → 腕关节(J5) → 腕关节(J6) → 末端

代表产品：KUKA KR 系列、ABB IRB 系列、Universal Robots UR 系列、Franka Emika Panda。

直角坐标型 (Cartesian/Gantry) 机械臂：

由三个线性滑动关节组成，运动方向相互垂直（X、Y、Z 轴），定位精度高但灵活性受限。

SCARA 型机械臂：

Selective Compliance Assembly Robot Arm，具有两个平行旋转关节和一个线性垂直关节，在水平方向柔顺、垂直方向刚性，特别适合装配作业。

并联型 (Parallel) 机械臂：

采用多个并联连杆驱动一个末端平台，刚度高、速度快、负载能力强，但工作空间较小。典型代表：Delta 机器人。

协作型 (Collaborative) 机械臂：

具备力控、碰撞检测和安全保护功能，可在无安全围栏的情况下与人协同工作。代表产品：UR5e、Franka Emika Panda、KUKA LBR iiwa。

24.1.2 按应用场景分类

分类	应用场景	特点
工业机器人	焊接、搬运、装配、喷涂	高速度、高精度、大负载
服务机器人	家庭服务、医疗辅助	轻量化、安全交互
特种机器人	太空、水下、核环境	环境适应性强
教育机器人	教学实验、科研	开放接口、模块化

24.1.3 机械臂关键技术指标

- 427. **自由度 (DOF)：**独立的运动轴数量，6 自由度可到达空间任意位姿
- 428. **负载能力：**末端可承载的最大质量
- 429. **工作空间：**末端可达的空间区域
- 430. **重复定位精度：**多次回到同一点的精度
- 431. **最大运动速度：**各关节的最大角速度
- 432. **防护等级：**IP 等级，防尘防水能力

24.2 运动学基础

24.2.1 位姿描述与齐次变换

在机器人学中，物体的位置和姿态统称为位姿 (Pose) 。

位置描述：使用三维向量 $p = [x, y, z]^T$ 表示。

姿态描述：使用旋转矩阵 $R \in SO(3)$ 表示，是一个 3×3 的正交矩阵。

齐次变换矩阵：将位置和姿态统一表示为 4×4 矩阵：

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

其中 R 是 3×3 旋转矩阵， p 是 3×1 位置向量。

常用旋转表示方法：

- 433. **旋转矩阵：**9 个参数，有正交约束
- 434. **欧拉角：**绕固定轴的三次旋转（如 ZYX），存在万向锁问题
- 435. **四元数：**4 个参数，无奇异性，计算效率高

四元数表示为 $q = [w, x, y, z]$ ，其中 w 为实部，满足 $w^2 + x^2 + y^2 + z^2 = 1$ 。

```
# 四元数与欧拉角转换
from scipy.spatial.transform import Rotation as R
import numpy as np

# 欧拉角 → 四元数
r = R.from_euler('zyx', [0.1, 0.2, 0.3])
quat = r.as_quat() # [x, y, z, w]

# 四元数 → 旋转矩阵
rot_mat = r.as_matrix()

# 四元数 → 欧拉角
euler = r.as_euler('zyx')
```

24.2.2 DH 参数建模

Denavit-Hartenberg（DH）参数法是描述串联机器人连杆之间变换关系的标准方法。每个连杆由四个参数定义：

参数	符号	含义
连杆长度	a_i	沿 X_i 轴从 Z_i 到 Z_{i+1} 的距离
连杆扭转	α_i	绕 X_i 轴从 Z_i 到 Z_{i+1} 的角度
关节距离	d_i	沿 Z_i 轴从 X_{i-1} 到 X_i 的距离

关节转角	θ_i	绕 Z_i 轴从 X_{i-1} 到 X_i 的角度
------	------------	----------------------------------

标准 DH 参数法：

相邻连杆的齐次变换矩阵为：

```
T_i = Rot(z,  $\theta_i$ ) * Trans(z,  $d_i$ ) * Trans(x,  $a_i$ ) * Rot(x,  $\alpha_i$ )
```

展开为：

```
T_i = [cos( $\theta_i$ )  -sin( $\theta_i$ )cos( $\alpha_i$ )  sin( $\theta_i$ )sin( $\alpha_i$ )  a_i*cos( $\theta_i$ )]
      [sin( $\theta_i$ )  cos( $\theta_i$ )cos( $\alpha_i$ ) -cos( $\theta_i$ )sin( $\alpha_i$ )  a_i*sin( $\theta_i$ )]
      [0         sin( $\alpha_i$ )         cos( $\alpha_i$ )         d_i         ]
      [0         0                   0                   1         ]
```

六自由度机械臂 DH 参数表示示例：

```
import numpy as np

# 六自由度机械臂 DH 参数表
# [a, alpha, d, theta_offset]
dh_params = [
    [0, 0, 0.333, 0], # J1
    [0, -np.pi/2, 0, 0], # J2
    [0, np.pi/2, 0.316, 0], # J3
    [0.0825, np.pi/2, 0, 0], # J4
    [-0.0825, -np.pi/2, 0.384, 0], # J5
    [0, np.pi/2, 0, 0], # J6
]

def dh_transform(a, alpha, d, theta):
    """计算 DH 参数对应的齐次变换矩阵"""
    ct = np.cos(theta)
    st = np.sin(theta)
    ca = np.cos(alpha)
    sa = np.sin(alpha)

    T = np.array([
        [ct, -st*ca, st*sa, a*ct],
        [st, ct*ca, -ct*sa, a*st],
        [0, sa, ca, d],
        [0, 0, 0, 1]
    ])
    return T

def forward_kinematics(dh_params, joint_angles):
    """正运动学：计算末端位姿"""
    T = np.eye(4)
    for i, (a, alpha, d, theta_offset) in enumerate(dh_params):
        theta = joint_angles[i] + theta_offset
        T_i = dh_transform(a, alpha, d, theta)
```

```

        T = T @ T_i
    return T

# 示例：计算零位姿的末端位姿
joint_angles = [0, 0, 0, 0, 0, 0]
end_effector_pose = forward_kinematics(dh_params, joint_angles)
print(f"末端位姿矩阵:\n{end_effector_pose}")

```

24.3 正运动学与逆运动学

24.3.1 正运动学（Forward Kinematics）

正运动学是已知各关节角度，计算末端执行器位姿的过程。

对于串联机械臂，正运动学可以通过依次乘相邻连杆的变换矩阵得到：

```
T_0_n = T_0_1 * T_1_2 * ... * T_{n-1}_n
```

在 ROS2 中，一般通过 `robot_state_publisher` 节点自动发布正运动学计算的 TF 变换。

24.3.2 逆运动学（Inverse Kinematics）

逆运动学是已知末端执行器的目标位姿，求解各关节角度的过程。逆运动学比正运动学复杂得多，主要难点在于：

- 436. **多解性**：同一末端位姿可能对应多组关节角度
- 437. **存在性**：某些位姿可能超出机械臂的工作空间
- 438. **奇异性**：在某些位姿下，雅可比矩阵降秩

解析解法（Closed-form Solution）：对于满足 Pieper 准则（三个相邻关节轴交于一点）的机械臂，可推导出解析解。

数值解法（Numerical Solution）：基于迭代优化的通用方法，使用雅可比矩阵的伪逆：

```

import numpy as np

def numerical_ik(target_pose, dh_params, initial_guess, max_iter=100, tol=1e-4):
    """
    数值逆运动学求解（基于雅可比伪逆法）
    参数：
        target_pose: 目标位姿（4x4 齐次矩阵）
        dh_params: DH 参数表
        initial_guess: 初始关节角度
    返回：
    """

```

```

        关节角度
    """
    theta = np.array(initial_guess, dtype=float)

    for iteration in range(max_iter):
        # 正运动学计算当前位姿
        current_pose = forward_kinematics(dh_params, theta)

        # 计算位置误差
        pos_error = target_pose[:3, 3] - current_pose[:3, 3]

        # 计算姿态误差 (旋转向量)
        R_current = current_pose[:3, :3]
        R_target = target_pose[:3, :3]
        rot_error = R_current.T @ R_target
        # 从旋转矩阵提取旋转向量
        axis_angle = np.array([
            rot_error[2, 1] - rot_error[1, 2],
            rot_error[0, 2] - rot_error[2, 0],
            rot_error[1, 0] - rot_error[0, 1]
        ]) * 0.5

        error = np.concatenate([pos_error, axis_angle])

        if np.linalg.norm(error) < tol:
            return theta

        # 计算雅可比矩阵 (数值方法)
        J = compute_jacobian(theta, dh_params)

        # 伪逆求解关节增量
        J_pinv = np.linalg.pinv(J)
        delta_theta = J_pinv @ error

        theta += delta_theta

        # 关节角度限制
        theta = np.clip(theta, -2*np.pi, 2*np.pi)

    print(f"IK 未收敛, 最终误差: {np.linalg.norm(error)}")
    return theta

def compute_jacobian(theta, dh_params, eps=1e-6):
    """数值计算雅可比矩阵"""
    n = len(theta)
    J = np.zeros((6, n))
    T_current = forward_kinematics(dh_params, theta)

    for i in range(n):
        theta_plus = theta.copy()
        theta_plus[i] += eps
        T_plus = forward_kinematics(dh_params, theta_plus)

        # 位置雅可比 (差分)
        pos_diff = (T_plus[:3, 3] - T_current[:3, 3]) / eps

```

```
J[:3, i] = pos_diff

# 姿态雅可比 (差分)
R_diff = T_plus[:3, :3] @ T_current[:3, :3].T
rot_vec = np.array([
    R_diff[2, 1] - R_diff[1, 2],
    R_diff[0, 2] - R_diff[2, 0],
    R_diff[1, 0] - R_diff[0, 1]
]) * 0.5 / eps
J[3:, i] = rot_vec

return J
```

24.3.3 ROS2 中的运动学求解器

Movelt2 支持多种运动学求解器插件：

求解器	说明	适用场景
KDL	Orocos KDL 运动学库	通用求解
TRAC-IK	改进的 IK 求解器	高鲁棒性
IKFast	解析法求解器	实时性要求高
BioIK	生物启发式 IK	多约束优化

在 Movelt2 配置中设置运动学求解器：

```
# kinematics.yaml
panda_arm:
  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
  kinematics_solver_attempts: 3
```

24.4 工作空间分析

24.4.1 工作空间定义

机械臂的工作空间（Workspace）是指末端执行器可达的所有空间位置的集合。工作空间分为：

- **可达工作空间**：末端可到达的所有位置
- **灵巧工作空间**：末端可以以任意姿态到达的位置

24.4.2 工作空间影响因素

- 439. **关节运动范围**: 各关节的转角限制
- 440. **连杆尺寸**: 连杆长度和偏置
- 441. **机械结构**: 关节类型和布局
- 442. **自碰撞**: 连杆之间的物理干涉

24.4.3 工作空间数值计算

通过蒙特卡洛方法可以计算机械臂的工作空间:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def compute_workspace(dh_params, joint_limits, num_samples=10000):
    """
    蒙特卡洛法计算机械臂工作空间
    参数:
        dh_params: DH 参数表
        joint_limits: 关节角度限制 [[min1,max1], [min2,max2], ...]
        num_samples: 采样点数
    返回:
        末端位置点集
    """
    n_joints = len(dh_params)
    positions = []

    for _ in range(num_samples):
        # 在关节限制内随机采样
        joint_angles = np.array([
            np.random.uniform(low, high)
            for low, high in joint_limits
        ])

        # 正运动学计算末端位置
        T = forward_kinematics(dh_params, joint_angles)
        positions.append(T[:3, 3])

    return np.array(positions)

# 示例: 计算机械臂工作空间
joint_limits = [
    [-2.96, 2.96], # J1
    [-2.09, 2.09], # J2
    [-2.96, 2.96], # J3
    [-2.09, 2.09], # J4
    [-2.96, 2.96], # J5
    [-2.09, 2.09], # J6
]
```

```
workspace_points = compute_workspace(dh_params, joint_limits, num_samples=5000)

# 可视化工作空间
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(workspace_points[:, 0],
           workspace_points[:, 1],
           workspace_points[:, 2],
           s=1, alpha=0.5)
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('机械臂工作空间')
plt.show()
```

24.5 ROS2 机械臂接口

24.5.1 JointState 话题

ROS2 中使用 sensor_msgs/JointState 消息发布机械臂的关节状态：

```
sensor_msgs/JointState
- header
- name: string[]      # 关节名称列表
- position: float64[] # 关节位置（弧度）
- velocity: float64[] # 关节速度
- effort: float64[]   # 关节力矩

# 订阅关节状态示例
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class JointStateSubscriber(Node):
    def __init__(self):
        super().__init__('joint_state_subscriber')
        self.sub = self.create_subscription(
            JointState, '/joint_states', self.joint_callback, 10)

    def joint_callback(self, msg):
        for i, name in enumerate(msg.name):
            self.get_logger().info(
                f'{name}: {msg.position[i]:.3f} rad')

def main(args=None):
    rclpy.init(args=args)
    node = JointStateSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

24.5.2 TF2 坐标变换

TF2 是 ROS2 中的坐标变换管理库，维护一棵坐标变换树。对于机械臂，TF 树从 base_link 到各连杆再到末端执行器。

```
# TF2 监听器示例
import rclpy
from rclpy.node import Node
import tf2_ros
from geometry_msgs.msg import PointStamped

class TF2Listener(Node):
    def __init__(self):
        super().__init__('tf2_listener')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(0.1, self.lookup_transform)

    def lookup_transform(self):
        try:
            # 获取从 base_link 到末端执行器的变换
            transform = self.tf_buffer.lookup_transform(
                'base_link',
                'tool0',
                rclpy.time.Time(),
                rclpy.duration.Duration(seconds=1.0))
            pos = transform.transform.translation
            self.get_logger().info(
                f'末端位置: ({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})')
        except Exception as e:
            self.get_logger().warn(f'TF 查询失败: {e}')
```

24.5.3 robot_state_publisher

robot_state_publisher 是 ROS2 中的标准节点，负责监听/joint_states 话题，根据 URDF 模型自动计算并发布各连杆之间的 TF 变换。

```
# 启动 robot_state_publisher
ros2 run robot_state_publisher robot_state_publisher \
    --ros-args -p robot_description:="$(cat my_arm.urdf)"
```

24.5.4 ROS2 Control 接口

ROS2 Control 是 ROS2 的机器人硬件控制框架，通过 ros2_control 功能包实现。机械臂控制器类型包括：

- **JointStateController**: 发布关节状态
- **JointTrajectoryController**: 执行关节轨迹
- **PositionController**: 位置控制

```
# ros2_controllers.yaml
controller_manager:
  ros__parameters:
    update_rate: 100

joint_state_broadcaster:
  type: joint_state_broadcaster/JointStateBroadcaster

arm_controller:
  type: joint_trajectory_controller/JointTrajectoryController
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5
    - joint6
```

课后练习

443. 写出六自由度机械臂的标准 DH 参数表，并编程实现正运动学计算函数，输入关节角度输出末端位姿。
444. 使用 Python 实现基于雅可比伪逆法的数值逆运动学求解器，测试在非奇异位姿下的收敛性能。
445. 使用蒙特卡洛方法计算机械臂的工作空间，绘制三维散点图，并分析工作空间的形状特征。
446. 编写 ROS2 节点，订阅/joint_states 话题，计算机械臂末端执行器的当前位置并输出到终端。
447. 解释 TF2 变换树的构建过程，说明 robot_state_publisher 如何实现从关节角度到坐标变换的计算。

实验 24:

第 24 章 实验: 多传感器套件与数据采集

****对应理论章节**:** 第 38 章《多传感器套件与数据采集》 ****实验课时**:** 3 课时 ****实**

验代码:** `src/lab\_code/ch24\_lab/`

实验目标

- 掌握在 CARLA 中配置和挂载多种传感器到自车
- 学会可视化 RGB、Depth、LiDAR 等传感器数据
- 掌握使用 rosbag 录制和回放多传感器数据

实验环境

- ROS 2 Jazzy + CARLA 0.9.16
- carla_ros_bridge
- rviz2
- rosbag2

练习 24.1: 配置并启动多传感器套件

步骤 1: 启动 CARLA 服务端

```
# 终端 1: 启动 CARLA
cd /opt/carla/bin
./CarlaUE4.sh -quality-level=Low

# 或者使用 Docker
docker run --rm -it --gpus all \
  -p 2000-2002:2000-2002 \
  carlasim/carla:0.9.16 \
  ./CarlaUE4.sh -quality-level=Low
```

步骤 2: 启动 carla_ros_bridge

```
# 终端 2: 启动 ROS2 桥接
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  host:=localhost port:=2000
```

步骤 3: 启动自车并挂载传感器

```
# 终端 3: 启动自车
ros2 launch carla_spawn_objects carla_spawn_objects.launch.py \
  control_role:='ego_vehicle' \
  spawn_point:=robo1 \
  spawn_sensor_only:=false

# 或使用 Python 脚本手动配置传感器
cd src/lab_code/ch24_lab/
python3 sensor_config.py
```

步骤 4: 验证传感器话题

```
# 查看所有话题
ros2 topic list

# 查看话题信息
ros2 topic info /camera/rgb/image_raw
ros2 topic info /lidar/points
ros2 topic info /imu/data

# 查看话题频率
ros2 topic hz /camera/rgb/image_raw
ros2 topic hz /lidar/points
ros2 topic hz /imu/data
```

传感器配置文件参考

sensor_config.py 定义了以下传感器配置:

传感器	Blueprint	挂载位置	频率
前视 RGB 相机	sensor.camera.rgb	(1.6, 0.0, 1.7)	20Hz
Depth 相机	sensor.camera.depth	(1.6, 0.0, 1.7)	20Hz
64 线 LiDAR	sensor.lidar.ray_cast	(0.0, 0.0, 2.0)	10Hz
RADAR	sensor.other.radar	(2.0, 0.0, 0.5)	20Hz
GNSS	sensor.other.gnss	(0.0, 0.0, 0.0)	10Hz
IMU	sensor.other.imu	(0.0, 0.0, 0.0)	50Hz

练习 24.2: 可视化各类传感器数据

步骤 1: 使用 rviz2 可视化

```
# 启动 rviz2
rviz2

# 在 rviz2 中添加显示:
# 1. Image 显示 → /camera/rgb/image_raw
# 2. PointCloud2 显示 → /lidar/points
# 3. Axes 显示 → frame: base_link
```

步骤 2: 使用 rqt_image_view 查看图像

```
# 查看 RGB 图像
rqt_image_view /camera/rgb/image_raw

# 查看深度图
rqt_image_view /camera/depth/image_raw
```

步骤 3: 运行可视化脚本

```
# 运行传感器可视化脚本
cd src/lab_code/ch24_lab/
python3 visualize_sensors.py

# 脚本功能:
# - 订阅所有传感器话题
# - OpenCV 窗口显示 RGB/Depth/SemSeg 图像
# - 终端打印 LiDAR 点云统计信息
# - 终端打印 RADAR 目标列表
```

可视化脚本输出示例

```
=== LiDAR 点云统计 ===
点数: 48215
最大距离: 98.3m
最小距离: 0.5m

=== RADAR 目标 ===
目标 1: 距离=12.5m, 速度=-3.2m/s, 方位角=0.15rad
目标 2: 距离=25.1m, 速度=-1.8m/s, 方位角=-0.22rad

=== GNSS 数据 ===
纬度: 48.9967°N, 经度: 8.0023°E, 海拔: 45.2m

=== IMU 数据 ===
```

```
加速度: [0.12, -0.05, 9.81] m/s2
角速度: [0.01, 0.02, 0.00] rad/s
```

练习 24.3: 录制并回放 Rosbag

步骤 1: 录制传感器数据

```
# 方法 1: 使用 record_bag.sh 脚本
cd src/lab_code/ch24_lab/
bash record_bag.sh

# 方法 2: 手动录制(推荐)
ros2 bag record \
  /camera/rgb/image_raw \
  /camera/depth/image_raw \
  /camera/semseg/image_raw \
  /lidar/points \
  /radar/detections \
  /gnss/data \
  /imu/data \
  /tf \
  /tf_static \
  -o carla_autonomous_data

# 方法 3: 录制所有(不推荐, 文件过大)
ros2 bag record -a -o everything
```

步骤 2: 查看 bag 信息

```
# 查看 bag 文件信息
ros2 bag info carla_autonomous_data

# 输出示例:
# Files:          carla_autonomous_data_0.db3
# Bag size:       2.4 GiB
# Storage id:     sqlite3
# Duration:       120.5s
# Start:          Jul 18 2026 14:32:10.500
# End:            Jul 18 2026 14:34:11.000
# Messages:       24100
# Topic information:
# Topic: /camera/rgb/image_raw | Type: sensor_msgs/Image | Count: 2400
# Topic: /lidar/points         | Type: sensor_msgs/PointCloud2 | Count: 1200
# Topic: /imu/data             | Type: sensor_msgs/Imu | Count: 6000
# Topic: /gnss/data           | Type: sensor_msgs/NavSatFix | Count: 1200
```

步骤 3: 回放 bag 数据

```
# 关闭 CARLA 服务器和 bridge

# 回放 bag
ros2 bag play carla_autonomous_data

# 查看回放数据
```

```
# 新终端:
rviz2
# 添加 Image 显示查看回放的 RGB 图像

# 回放指定速度
ros2 bag play carla_autonomous_data --rate 0.5

# 循环回放(用于算法调试)
ros2 bag play carla_autonomous_data --loop
```

步骤 4: 从 bag 中提取数据

```
# 使用 rosbags 工具提取图像
pip install rosbags

# 提取脚本
python3 -c "
from rosbags.rosbag2 import Reader
from rosbags.serde import deserialize_cdr
import cv2
import numpy as np

with Reader('carla_autonomous_data') as reader:
    for connection, timestamp, rawdata in reader.messages():
        if connection.topic == '/camera/rgb/image_raw':
            msg = deserialize_cdr(rawdata, connection.msgtype)
            img = np.frombuffer(msg.data, dtype=np.uint8)
            img = img.reshape((msg.height, msg.width, -1))
            cv2.imwrite(f'frame_{timestamp}.png', img)
            break # 保存第一帧

print('图像提取完成')
"
```

实验报告要求

- 448. 记录各传感器的话题名称、消息类型和频率
- 449. 截图显示 rviz2 中 RGB 图像和 LiDAR 点云的可视化效果
- 450. 对比不同 LiDAR 通道数(16/32/64)的点云密度差异
- 451. 记录录制 bag 文件的大小和时长
- 452. 分析多传感器时间戳对齐的精度

常见问题

Q: 传感器话题没有数据? A: 确认 CARLA 桥接连接正常, 检查 `ros2 topic echo /carla/status`

Q: LiDAR 点云显示为空? A: 检查 rviz2 的 Fixed Frame 设置为 map 或 base_link, 确认 LiDAR 话题有消息输出

Q: Bag 文件过大? A: 减少录制时长, 或使用压缩格式 `ros2 bag record --compression-mode file --compression-format zstd`

Q: 回放时传感器话题时间戳异常? A: 回放时需确认没有其他节点发布相同话题, 使用 `--clock` 参数发布模拟时钟

第 25 章 ROS2 机械臂建模

学习目标

- 掌握 URDF 和 Xacro 建模方法
- 理解 SRDF 语义描述文件的结构
- 学会在 Rviz 中可视化机械臂模型
- 掌握 3D 模型导入与整合方法

25.1 URDF 建模基础

25.1.1 URDF 概述

URDF (Unified Robot Description Format, 统一机器人描述格式) 是 ROS2 中标准的机器人模型描述格式。它基于 XML 规范, 用于描述机器人的运动学、动力学和几何信息。

URDF 的核心结构将机器人建模为由连杆 (Link) 和关节 (Joint) 组成的树状结构:

- **连杆 (Link)** : 描述刚体的外观和物理属性
- **关节 (Joint)** : 描述连杆之间的连接和相对运动关系

URDF 文件的基本结构:

```
<?xml version="1.0"?>
<robot name="my_arm">
  <!-- 定义底座 -->
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.18" radius="0.1"/>
      </geometry>
      <material name="yellow">
        <color rgba="1 1 0 1"/>
      </material>
    </visual>
  </link>

  <!-- 定义关节 -->
  <joint name="joint1" type="revolute">
    <parent link="base_link"/>
    <child link="link1"/>
    <origin xyz="0 0 0.18" rpy="0 0 0"/>
    <axis xyz="0 0 1"/>
    <limit effort="300" velocity="0.6" lower="-2.96" upper="2.96"/>
  </joint>
```

```

<link name="link1">
  <visual>
    <geometry>
      <box size="0.1 0.06 0.2"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 1 1"/>
    </material>
  </visual>
</link>
</robot>

```

25.1.2 link 元素详解

<link>元素描述连杆的外观和物理属性，包含三个核心子元素：

visual（视觉属性）：

```

<link name="arm_link1">
  <visual>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.1 0.06 0.2"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 1 1"/>
    </material>
  </visual>
</link>

```

collision（碰撞属性）：用于碰撞检测的简化模型，可与 visual 形状不同：

```

<link name="arm_link1">
  <collision>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.1 0.06 0.2"/>
    </geometry>
  </collision>
</link>

```

inertial（惯性属性）：用于动力学仿真：

```

<link name="arm_link1">
  <inertial>
    <origin xyz="0 0 0.09" rpy="0 0 0"/>
    <mass value="1.0"/>
    <inertia ixx="0.0052" ixy="0" ixz="0"
      iyy="0.0052" iyz="0" izz="0.005"/>
  </inertial>
</link>

```

25.1.3 joint 元素详解

<joint>元素描述连杆之间的连接关系：

关节类型：

类型	说明	自由度	示例
revolute	旋转关节（有限位）	1	机械臂关节
continuous	连续旋转关节	1	车轮
prismatic	滑动关节	1	夹爪
fixed	固定连接	0	底座
floating	浮动连接	6	自由物体
planar	平面运动	3	AGV 底盘

关节属性：

```
<joint name="joint1" type="revolute">
  <parent link="base_link"/>          <!-- 父连杆 -->
  <child link="link1"/>                <!-- 子连杆 -->
  <origin xyz="0 0 0.18" rpy="0 0 0"/> <!-- 相对位姿 -->
  <axis xyz="0 0 1"/>                  <!-- 关节轴线方向 -->
  <limit effort="300" velocity="0.6"    <!-- 运动限制 -->
    lower="-2.96" upper="2.96"/>
  <dynamics damping="50" friction="1"/> <!-- 动力学属性 -->
  <mimic joint="joint2" multiplier="1" offset="0"/> <!-- 模仿关节 -->
  <safety_controller soft_lower_limit="-2.85"      <!-- 安全限制 -->
    soft_upper_limit="2.85"
    k_position="100"/>
</joint>
```

25.2 六自由度机械臂 URDF 建模

25.2.1 创建功能包

```
# 创建工作空间
mkdir -p ~/ros2_arm_ws/src
cd ~/ros2_arm_ws/src

# 创建描述功能包
ros2 pkg create arm_description --build-type ament_python --dependencies rclpy

# 创建目录结构
cd arm_description
mkdir -p urdf launch rviz meshes
```

25.2.2 完整六自由度机械臂 URDF

```
<?xml version="1.0"?>
<robot name="six_dof_arm">
  <!-- 底座 -->
  <link name="base_link">
    <visual>
      <origin rpy="0 0 0" xyz="0 0 0.045"/>
      <geometry>
        <cylinder length="0.09" radius="0.08"/>
      </geometry>
      <material name="gray">
        <color rgba="0.5 0.5 0.5 1"/>
      </material>
    </visual>
    <collision>
      <origin rpy="0 0 0" xyz="0 0 0.045"/>
      <geometry>
        <cylinder length="0.09" radius="0.08"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="2.0"/>
      <inertia ixx="0.01" ixy="0" ixz="0"
        iyy="0.01" iyz="0" izz="0.01"/>
    </inertial>
  </link>

  <!-- 关节1: 腰部旋转 -->
  <joint name="joint1" type="revolute">
    <parent link="base_link"/>
    <child link="link1"/>
    <origin rpy="0 0 0" xyz="0 0 0.09"/>
    <axis xyz="0 0 1"/>
    <limit effort="100" velocity="1.0" lower="-3.14" upper="3.14"/>
  </joint>

  <!-- 连杆1 -->
  <link name="link1">
    <visual>
      <geometry>
        <box size="0.06 0.06 0.25"/>
      </geometry>
      <origin rpy="0 0 0" xyz="0 0 0.125"/>
      <material name="blue">
        <color rgba="0 0 1 1"/>
      </material>
    </visual>
    <collision>
      <geometry>
        <box size="0.06 0.06 0.25"/>
      </geometry>
      <origin rpy="0 0 0" xyz="0 0 0.125"/>
    </collision>
    <inertial>
      <mass value="1.5"/>
      <inertia ixx="0.005" ixy="0" ixz="0"
        iyy="0.005" iyz="0" izz="0.001"/>
    </inertial>
  </link>
</robot>
```

```

</link>

<!-- 关节2: 肩部旋转 -->
<joint name="joint2" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin rpy="0 0 0" xyz="0 0 0.25"/>
  <axis xyz="0 1 0"/>
  <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆2 -->
<link name="link2">
  <visual>
    <geometry>
      <box size="0.06 0.06 0.25"/>
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0.125"/>
    <material name="green">
      <color rgba="0 1 0 1"/>
    </material>
  </visual>
  <collision>
    <geometry>
      <box size="0.06 0.06 0.25"/>
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0.125"/>
  </collision>
  <inertial>
    <mass value="1.2"/>
    <inertia ixx="0.004" ixy="0" ixz="0"
      iyy="0.004" iyz="0" izz="0.001"/>
  </inertial>
</link>

<!-- 关节3: 肘部旋转 -->
<joint name="joint3" type="revolute">
  <parent link="link2"/>
  <child link="link3"/>
  <origin rpy="0 0 0" xyz="0 0 0.25"/>
  <axis xyz="0 1 0"/>
  <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆3 -->
<link name="link3">
  <visual>
    <geometry>
      <box size="0.05 0.05 0.15"/>
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0.075"/>
    <material name="red">
      <color rgba="1 0 0 1"/>
    </material>
  </visual>
  <collision>
    <geometry>
      <box size="0.05 0.05 0.15"/>
    </geometry>
  </collision>
</link>

```

```

        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.075"/>
    </collision>
    <inertial>
        <mass value="0.8"/>
        <inertia ixx="0.002" ixy="0" ixz="0"
            iyy="0.002" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节4：腕部旋转1 -->
<joint name="joint4" type="continuous">
    <parent link="link3"/>
    <child link="link4"/>
    <origin rpy="0 0 0" xyz="0 0 0.15"/>
    <axis xyz="0 0 1"/>
    <limit effort="50" velocity="1.0"/>
</joint>

<!-- 连杆4 -->
<link name="link4">
    <visual>
        <geometry>
            <box size="0.04 0.04 0.08"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.04"/>
        <material name="yellow">
            <color rgba="1 1 0 1"/>
        </material>
    </visual>
    <collision>
        <geometry>
            <box size="0.04 0.04 0.08"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.04"/>
    </collision>
    <inertial>
        <mass value="0.3"/>
        <inertia ixx="0.001" ixy="0" ixz="0"
            iyy="0.001" iyz="0" izz="0.0005"/>
    </inertial>
</link>

<!-- 关节5：腕部旋转2 -->
<joint name="joint5" type="continuous">
    <parent link="link4"/>
    <child link="link5"/>
    <origin rpy="0 0 0" xyz="0 0 0.08"/>
    <axis xyz="0 1 0"/>
    <limit effort="50" velocity="1.0"/>
</joint>

<!-- 连杆5（末端） -->
<link name="link5">
    <visual>
        <origin rpy="0 0 0" xyz="0 0 0.04"/>
        <geometry>
            <cylinder length="0.08" radius="0.03"/>

```

```

        </geometry>
        <material name="gray">
            <color rgba="0.7 0.7 0.7 1"/>
        </material>
    </visual>
    <collision>
        <origin rpy="0 0 0" xyz="0 0 0.04"/>
        <geometry>
            <cylinder length="0.08" radius="0.03"/>
        </geometry>
    </collision>
    <inertial>
        <mass value="0.2"/>
        <inertia ixx="0.0005" ixy="0" ixz="0"
            iyy="0.0005" iyz="0" izz="0.0001"/>
    </inertial>
</link>

<!-- 工具坐标系 -->
<link name="tool0"/>
<joint name="joint6" type="fixed">
    <parent link="link5"/>
    <child link="tool0"/>
    <origin rpy="0 0 0" xyz="0 0 0.08"/>
</joint>
</robot>

```

25.3 Xacro 建模

25.3.1 Xacro 简介

Xacro (XML Macros) 是 URDF 的宏语言扩展，提供了变量定义、数学运算、条件判断和宏模板等高级功能，使模型描述更加简洁和可维护。

25.3.2 Xacro 基本语法

命名空间和变量：

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro"
    name="six_dof_arm">

    <!-- 定义可配置参数 -->
    <xacro:property name="base_radius" value="0.08"/>
    <xacro:property name="base_height" value="0.09"/>
    <xacro:property name="link1_length" value="0.25"/>
    <xacro:property name="link2_length" value="0.25"/>
    <xacro:property name="link3_length" value="0.15"/>

    <!-- 数学运算 -->
    <xacro:property name="base_origin_z" value="${base_height / 2}"/>
    <xacro:property name="total_height" value="${base_height + link1_length + link2_length +
link3_length}"/>

```

宏定义 (Macro) :

```
<!-- 定义连杆宏 -->
<xacro:macro name="link_visual" params="name length color">
  <link name="${name}">
    <visual>
      <origin xyz="0 0 ${length/2}" rpy="0 0 0"/>
      <geometry>
        <box size="0.06 0.06 ${length}"/>
      </geometry>
      <material name="${color}">
        <color rgba="${color}"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 ${length/2}" rpy="0 0 0"/>
      <geometry>
        <box size="0.06 0.06 ${length}"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="${length * 5}"/>
      <inertia ixx="${length * 0.01}" ixy="0" ixz="0"
        iyy="${length * 0.01}" iyz="0" izz="${length * 0.005}"/>
    </inertial>
  </link>
</xacro:macro>

<!-- 使用宏 -->
<xacro:link_visual name="link1" length="${link1_length}" color="1 0 0 1"/>
<xacro:link_visual name="link2" length="${link2_length}" color="0 1 0 1"/>
```

条件判断:

```
<xacro:if value="${use_gripper}">
  <!-- 包含夹爪 -->
  <xacro:include filename="gripper.xacro"/>
</xacro:if>
```

循环生成:

```
<xacro:macro name="joint_limit" params="joint_name lower upper">
  <limit effort="100" velocity="1.0"
    lower="${lower}" upper="${upper}"/>
</xacro:macro>

<!-- 在关节中使用 -->
<joint name="joint2" type="revolute">
  <xacro:joint_limit joint_name="joint2"
    lower="${-pi*2/3}" upper="${pi*2/3}"/>
</joint>
```

25.3.3 使用 Xacro 重构 URDF

将前面的 URDF 模型用 Xacro 重构, 可大幅减少重复代码:

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro"
      name="six_dof_arm">

  <xacro:property name="pi" value="3.14159265359"/>

  <!-- 可配置参数 -->
  <xacro:property name="base_radius" value="0.08"/>
  <xacro:property name="link_width" value="0.06"/>
  <xacro:property name="link1_len" value="0.25"/>
  <xacro:property name="link2_len" value="0.25"/>
  <xacro:property name="link3_len" value="0.15"/>

  <!-- 颜色定义 -->
  <material name="blue">
    <color rgba="0 0 1 1"/>
  </material>
  <material name="green">
    <color rgba="0 1 0 1"/>
  </material>
  <material name="red">
    <color rgba="1 0 0 1"/>
  </material>

  <!-- 连杆宏 -->
  <xacro:macro name="create_link" params="name length color_name">
    <link name="${name}">
      <visual>
        <origin xyz="0 0 ${length/2}"/>
        <geometry>
          <box size="${link_width} ${link_width} ${length}"/>
        </geometry>
        <material name="${color_name}"/>
      </visual>
      <collision>
        <origin xyz="0 0 ${length/2}"/>
        <geometry>
          <box size="${link_width} ${link_width} ${length}"/>
        </geometry>
      </collision>
      <inertial>
        <mass value="${length * 5}"/>
        <inertia ixx="${0.001 * length * 10}" ixy="0" ixz="0"
          iyy="${0.001 * length * 10}" iyz="0" izz="${0.001}"/>
      </inertial>
    </link>
  </xacro:macro>

  <!-- 旋转关节宏 -->
  <xacro:macro name="create_revolute_joint"
    params="name parent child axis origin_xyz lower upper">
    <joint name="${name}" type="revolute">
      <parent link="${parent}"/>
      <child link="${child}"/>
      <origin xyz="${origin_xyz}" rpy="0 0 0"/>
      <axis xyz="${axis}"/>
      <limit effort="100" velocity="1.0"
        lower="${lower}" upper="${upper}"/>
    </joint>
  </xacro:macro>

```

```

    </joint>
</xacro:macro>

<!-- 底座 -->
<link name="base_link">
  <visual>
    <origin xyz="0 0 ${base_radius/2}"/>
    <geometry>
      <cylinder length="${base_radius + 0.01}"
        radius="${base_radius}"/>
    </geometry>
    <material name="gray">
      <color rgba="0.5 0.5 0.5 1"/>
    </material>
  </visual>
</link>

<!-- 关节1 -->
<xacro:create_revolute_joint
  name="joint1" parent="base_link" child="link1"
  axis="0 0 1" origin_xyz="0 0 ${base_radius}"
  lower="${-pi}" upper="${pi}"/>

<!-- 连杆1 -->
<xacro:create_link name="link1" length="${link1_len}" color_name="blue"/>

<!-- 关节2 -->
<xacro:create_revolute_joint
  name="joint2" parent="link1" child="link2"
  axis="0 1 0" origin_xyz="0 0 ${link1_len}"
  lower="${-pi*2/3}" upper="${pi*2/3}"/>

<!-- 连杆2 -->
<xacro:create_link name="link2" length="${link2_len}" color_name="green"/>

<!-- 关节3 -->
<xacro:create_revolute_joint
  name="joint3" parent="link2" child="link3"
  axis="0 1 0" origin_xyz="0 0 ${link2_len}"
  lower="${-pi*2/3}" upper="${pi*2/3}"/>

<!-- 连杆3 -->
<xacro:create_link name="link3" length="${link3_len}" color_name="red"/>
</robot>

```

25.4 SRDF 语义描述

25.4.1 SRDF 的作用

SRDF (Semantic Robot Description Format) 是对 URDF 的语义补充, 为 MoveIt2 等高级框架提供额外的语义信息, 包括:

- 规划组 (Planning Groups) : 定义关节集合

- 预设位姿 (Named Poses) : 预定义的关节位置
- 自碰撞对 (Collision Pairs) : 不需要碰撞检测的连杆对
- 末端执行器 (End Effectors) : 定义工具信息
- 虚拟关节 (Virtual Joints) : 连接机器人与世界坐标系

25.4.2 SRDF 文件示例

```
<?xml version="1.0"?>
<robot name="six_dof_arm">
  <!-- 规划组：定义手臂的所有关节 -->
  <group name="arm_group">
    <joint name="joint1"/>
    <joint name="joint2"/>
    <joint name="joint3"/>
    <joint name="joint4"/>
    <joint name="joint5"/>
    <joint name="joint6"/>
  </group>

  <!-- 规划组：定义夹爪的所有关节 -->
  <group name="gripper">
    <joint name="gripper_left_joint"/>
    <joint name="gripper_right_joint"/>
  </group>

  <!-- 预设位姿 -->
  <group_state name="home" group="arm_group">
    <joint name="joint1" value="0"/>
    <joint name="joint2" value="0"/>
    <joint name="joint3" value="0"/>
    <joint name="joint4" value="0"/>
    <joint name="joint5" value="0"/>
    <joint name="joint6" value="0"/>
  </group_state>

  <group_state name="vertical" group="arm_group">
    <joint name="joint1" value="0"/>
    <joint name="joint2" value="-1.57"/>
    <joint name="joint3" value="1.57"/>
    <joint name="joint4" value="0"/>
    <joint name="joint5" value="1.57"/>
    <joint name="joint6" value="0"/>
  </group_state>

  <!-- 末端执行器 -->
  <end_effector name="gripper_ee"
    parent_link="link5"
    parent_group="arm_group"
    group="gripper"/>

  <!-- 虚拟关节（将机器人连接到世界坐标系） -->
  <virtual_joint name="base_footprint"
    type="fixed">
```

```

        parent_frame="world"
        child_link="base_link"/>

<!-- 自碰撞免检对 -->
<disable_collisions link1="base_link" link2="link2" reason="Never"/>
<disable_collisions link1="base_link" link2="link3" reason="Never"/>
<disable_collisions link1="base_link" link2="link4" reason="Never"/>
<disable_collisions link1="base_link" link2="link5" reason="Never"/>
<disable_collisions link1="link1" link2="link4" reason="Adjacent"/>
<disable_collisions link1="link1" link2="link5" reason="Never"/>
</robot>

```

25.4.3 MoveIt2 的 SRDF 使用

MoveIt2 通过 Setup Assistant 工具自动生成 SRDF 文件。用户也可以在配置包中手动创建或修改 SRDF:

```

# MoveIt2 配置包的目录结构
generated_moveit_config/
├── config/
│   ├── six_dof_arm.srdf      # SRDF 语义描述
│   ├── kinematics.yaml      # 运动学配置
│   ├── joint_limits.yaml    # 关节极限
│   ├── ompl_planning.yaml   # OMPL 规划器配置
│   ├── fake_controllers.yaml # 虚拟控制器
│   └── ros2_controllers.yaml # 实际控制器
└── launch/
    ├── move_group.launch.py
    ├── planning_context.launch.py
    └── demo.launch.py

```

25.5 在 Rviz 中可视化

25.5.1 创建启动文件

```

# launch/display.launch.py
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('arm_description')
    urdf_path = os.path.join(pkg_share, 'urdf', 'six_dof_arm.urdf')

    with open(urdf_path, 'r') as f:
        robot_description = f.read()

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}]

```

```

    ),
    Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
    ),
    Node(
        package='rviz2',
        executable='rviz2',
        arguments=['-d', os.path.join(pkg_share, 'rviz', 'display.rviz')]
    ),
])

```

25.5.2 Xacro 版本的启动文件

```

# launch/display_xacro.launch.py
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('arm_description')

    # Xacro 处理参数
    use_sim_time = LaunchConfiguration('use_sim_time', default='false')
    xacro_file = os.path.join(pkg_share, 'urdf', 'six_dof_arm.xacro')

    # 将 Xacro 转换为 URDF
    from xacro import process_file
    doc = process_file(xacro_file)
    robot_description = doc.toxml()

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='false',
            description='Use simulation time'),

        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{
                'robot_description': robot_description,
                'use_sim_time': use_sim_time,
            }
            ],
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
        ),
    ])

```

25.5.3 启动模型可视化

```
# 查看 URDF 模型
ros2 launch arm_description display.launch.py

# 查看 Xacro 模型
ros2 launch arm_description display_xacro.launch.py
```

25.5.4 命令行工具检查模型

```
# 检查 URDF 语法
check_urdf six_dof_arm.urdf

# 使用 URDF 工具包
ros2 run urdfdom_py urdf_to_graphviz six_dof_arm.urdf

# 查看模型信息
ros2 topic echo /robot_description --once

# 查看 TF 树
ros2 run tf2_tools view_frames.py
```

25.6 3D 模型导入

25.6.1 使用 Mesh 文件

URDF 支持导入 3D 网格文件（STL、DAE 格式）作为连杆的视觉和碰撞模型：

```
<link name="base_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <mesh filename="package://arm_description/meshes/base.stl"
        scale="1.0 1.0 1.0"/>
    </geometry>
    <material name="gray">
      <color rgba="0.5 0.5 0.5 1"/>
    </material>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <mesh filename="package://arm_description/meshes/base_collision.stl"
        scale="1.0 1.0 1.0"/>
    </geometry>
  </collision>
</link>
```

25.6.2 3D 模型文件格式

格式	优点	缺点
STL (.stl)	广泛支持、二进制格式小	只有几何、无颜色

Collada (.dae)	支持颜色纹理材质	文件较大
OBJ (.obj)	通用格式	需配套 MTL 文件

25.6.3 创建简化碰撞模型

为提升碰撞检测性能，通常为复杂的可视化模型创建简化的碰撞模型：

```
# 使用 Trimesh 库简化网格
import trimesh
import numpy as np

def simplify_mesh(input_path, output_path, max_faces=500):
    """简化 3D 网格用于碰撞检测"""
    mesh = trimesh.load(input_path)
    simplified = mesh.simplify_quadric_decimation(max_faces)
    simplified.export(output_path)
    print(f"简化: {len(mesh.faces)} → {len(simplified.faces)} 面")

# 示例：简化底座模型
simplify_mesh(
    'meshes/base.stl',
    'meshes/base_collision.stl',
    max_faces=500
)
```

25.6.4 检查网格文件路径

```
# 检查包路径
echo $AMENT_PREFIX_PATH

# 列出包中的网格文件
find ~/ros2_arm_ws/install/arm_description -name "*.stl" -o -name "*.dae"
```

25.7 模型验证与调试

25.7.1 使用 check_urdf 工具

```
# 安装检查工具
sudo apt install liburdfdom-tools

# 检查 URDF 模型
check_urdf six_dof_arm.urdf

# 输出示例：
# robot name is: six_dof_arm
# ----- Successfully Parsed XML -----
# root link: base_link has 1 child(ren)
#   child(1): link1
```

```
# child(1): link2
# ...
```

25.7.2 生成模型关系图

```
# 安装 graphviz
sudo apt install graphviz

# 生成 URDF 图形表示
urdf_to_graphviz six_dof_arm.urdf

# 输出文件: six_dof_arm.pdf
```

25.7.3 Rviz 调试技巧

在 Rviz 中添加以下显示组件进行调试：

- 453. **RobotModel**：显示机器人模型
- 454. **TF**：显示坐标系
- 455. **MotionPlanning**：MoveIt2 规划显示

课后练习

- 456. 编写一个六自由度机械臂的 URDF 文件，包含底座、三个连杆、腕部和末端执行器，使用不同的颜色区分各部件。
- 457. 使用 Xacro 重构上述 URDF，定义连杆长度、颜色等参数为变量，并使用宏简化代码。
- 458. 创建一个 SRDF 文件，定义 arm_group 和 gripper 两个规划组，并添加 home 和 vertical 两个预设位姿。
- 459. 编写 launch 文件，在 Rviz 中可视化机械臂模型，并使用 joint_state_publisher_gui 手动控制各关节运动。
- 460. 导入一个 3D 模型（STL 或 DAE）作为机械臂连杆的可视化模型，并创建简化的碰撞检测模型。

实验 25:

第 25 章 实验手册：全局路径规划与地图导航

****对应理论章节**:** 第 39 章《全局路径规划与地图导航》 ****实验课时**:** 3 课时 ****实验代码**:** `src/lab\_code/ch25\_lab/`

练习 25.1：获取 CARLA 地图 OpenDRIVE 并可视化路网

实验目标

掌握从 CARLA 模拟器获取 OpenDRIVE 地图数据的方法，并可视化道路网络结构。

实验步骤

461. 启动 CARLA 服务器

```
# 在 CARLA 安装目录下
./CarlaUE4.sh -quality-level=Low # Linux
# 或
CarlaUE4.exe -quality-level=Low # Windows
```

462. 编写 Python 脚本获取 OpenDRIVE

```
import carla
import time

client = carla.Client('localhost', 2000)
client.set_timeout(10.0)
world = client.get_world()
carla_map = world.get_map()

# 导出 OpenDRIVE 到文件
opendrive = carla_map.to_opendrive()
with open("carla_map.xodr", "w", encoding="utf-8") as f:
    f.write(opendrive)
print(f"OpenDRIVE 已导出, 长度: {len(opendrive)} 字符")
```

463. 可视化路网拓扑 使用 carla.Map 的拓扑方法绘制路网图:

```
import matplotlib.pyplot as plt
import numpy as np
```

```

# 获取道路拓扑
topology_list = carla_map.get_topology()
print(f"道路拓扑边数: {len(topology_list)}")

# 绘制道路网络
plt.figure(figsize=(12, 8))
for edge in topology_list:
    wp_start, wp_end = edge
    start_loc = wp_start.transform.location
    end_loc = wp_end.transform.location
    plt.plot([start_loc.x, end_loc.x],
              [start_loc.y, end_loc.y], 'b-', alpha=0.5, linewidth=0.5)
    plt.plot(start_loc.x, start_loc.y, 'g.', markersize=1)
    plt.plot(end_loc.x, end_loc.y, 'r.', markersize=1)

plt.axis('equal')
plt.title('CARLA 道路拓扑网络')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.grid(True, alpha=0.3)
plt.savefig('road_topology.png', dpi=150)
plt.show()

```

464. 分析 OpenDRIVE 结构 解析 XML 提取道路信息:

```

import xml.etree.ElementTree as ET
root = ET.fromstring(opendrive)

roads = root.findall('.//road')
print(f"道路总数: {len(roads)}")
for road in roads[:5]:
    road_id = road.get('id')
    road_name = road.get('name')
    road_len = road.get('length')
    lanes = road.find('.//laneSection')
    left_lanes = lanes.find('left') if lanes is not None else None
    right_lanes = lanes.find('right') if lanes is not None else None
    n_left = len(left_lanes.findall('lane')) if left_lanes is not None else 0
    n_right = len(right_lanes.findall('lane')) if right_lanes is not None else 0
    print(f" 道路 {road_id}: {road_name}, 长度={road_len}m, "
          f"左车道数={n_left}, 右车道数={n_right}")

```

预期输出

- carla_map.xodr 文件 (OpenDRIVE XML)
- road_topology.png 路网拓扑图
- 控制台打印的道路统计信息

练习 25.2 : 实现 A* 全局路径规划

实验目标

在网格化的路网中实现 A* 搜索算法，并在 CARLA 中可视化规划路径。

实验步骤

465. **构建路网图** 将练习 25.1 获取的道路拓扑转换为可供搜索的图结构：

```
class RoadGraph:
    def __init__(self):
        self.nodes = {}      # node_id -> (x, y)
        self.edges = {}      # node_id -> [(neighbor_id, cost)]

    def add_node(self, node_id, x, y):
        self.nodes[node_id] = (x, y)

    def add_edge(self, from_id, to_id, cost=None):
        if cost is None:
            x1, y1 = self.nodes[from_id]
            x2, y2 = self.nodes[to_id]
            cost = np.sqrt((x2-x1)**2 + (y2-y1)**2)
        self.edges.setdefault(from_id, []).append((to_id, cost))

    def build_graph_from_topology(carla_map):
        graph = RoadGraph()
        topology = carla_map.get_topology()

        node_id_map = {}
        next_id = 0

        for wp_start, wp_end in topology:
            for wp in [wp_start, wp_end]:
                loc = wp.transform.location
                key = (round(loc.x, 2), round(loc.y, 2))
                if key not in node_id_map:
                    node_id_map[key] = next_id
                    graph.add_node(next_id, loc.x, loc.y)
                    next_id += 1

            start_key = (round(wp_start.transform.location.x, 2),
                        round(wp_start.transform.location.y, 2))
            end_key = (round(wp_end.transform.location.x, 2),
                      round(wp_end.transform.location.y, 2))
            graph.add_edge(node_id_map[start_key], node_id_map[end_key])

        return graph, node_id_map
```

466. **实现 A* 搜索**

```
import heapq

def heuristic(a_id, b_id, nodes):
    x1, y1 = nodes[a_id]
    x2, y2 = nodes[b_id]
```

```

    return np.sqrt((x2-x1)**2 + (y2-y1)**2)

def a_star_search(graph, start_id, goal_id):
    open_set = [(0, start_id)]
    came_from = {}
    g_score = {node_id: float('inf') for node_id in graph.nodes}
    g_score[start_id] = 0
    f_score = {node_id: float('inf') for node_id in graph.nodes}
    f_score[start_id] = heuristic(start_id, goal_id, graph.nodes)

    in_open = {start_id}

    while open_set:
        _, current = heapq.heappop(open_set)
        in_open.discard(current)

        if current == goal_id:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start_id)
            return path[::-1]

        for neighbor, cost in graph.edges.get(current, []):
            tentative_g = g_score[current] + cost
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score[neighbor] = g_score[neighbor] + \
                    heuristic(neighbor, goal_id, graph.nodes)
            if neighbor not in in_open:
                heapq.heappush(open_set, (f_score[neighbor], neighbor))
                in_open.add(neighbor)

    return [] # 无路径

```

467. 在 CARLA 中可视化规划路径

```

def visualize_path(world, path, graph, color=carla.Color(0, 255, 0)):
    waypoints = []
    for node_id in path:
        x, y = graph.nodes[node_id]
        loc = carla.Location(x, y, 0.5)
        waypoint = world.get_map().get_waypoint(loc)
        if waypoint:
            waypoints.append(waypoint.transform.location)

    # 使用 DebugHelper 绘制
    debug = world.debug
    for i in range(len(waypoints) - 1):
        debug.draw_line(
            waypoints[i],
            waypoints[i+1],
            thickness=0.3,
            color=color,
            life_time=30.0
        )

```

思考题

- 468. 当路网中存在大量节点时, A* 的 open_set 会包含多少节点? 与 Dijkstra 对比如何?
- 469. 在 CARLA 的多层道路 (如高架桥与地面路重叠) 场景下, Waypoint 如何区分不同层?

练习 25.3 : 使用 Waypoint 发布器生成导航路径

实验目标

创建一个 ROS2 节点, 通过 CARLA Waypoint API 生成连续导航路径, 并发布为 ROS2 消息。

实验步骤

- 470. **创建 ROS2 包 (若需要)**

```
cd ~/ros2_course_ws/src
ros2 pkg create ch25_waypoint_nav --build-type ament_python \
  --dependencies rclpy geometry_msgs nav_msgs carla_msgs
```

- 471. **实现 Waypoint 路径生成节点** 参考本章 waypoint_pub.py 代码, 重点实现:
- 472. 从起点到目标点的 Waypoint 链生成
- 473. 路径转换为 nav_msgs/Path 消息
- 474. 以 0.5Hz 频率发布路径
- 475. **在 Rviz2 中可视化路径**

```
ros2 launch ch25_waypoint_nav waypoint_nav.launch.py
rviz2 -d waypoint_nav.rviz
```

在 Rviz2 中添加 Path 显示类型, 订阅 /planned_path 话题。

- 476. **将路径输出为 KML (可选)** 方便在 Google Earth 中查看:

```
def export_to_kml(waypoints, filename="route.kml"):
    kml_header = """<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Document>
    <Placemark>
      <LineString>
        <coordinates>"""
    kml_footer = """</coordinates>
      </LineString>
    </Placemark>
```

```
</Document>
</kml>"""

coords = "\n".join(
    f"{wp.transform.location.x},{wp.transform.location.y},0"
    for wp in waypoints
)
with open(filename, "w") as f:
    f.write(kml_header + coords + kml_footer)
print(f"KML 已导出: {filename}")
```

预期结果

- 在 Rviz2 中看到绿色的规划路径
- 路径沿车道中心线分布
- 在路口处根据目标方向选择正确出口

挑战任务

在 CARLA 中运行 ego-vehicle，使其沿生成的 Waypoint 路径自主行驶（可使用 CARLA 的 WaypointFollower 或自定义 PID 控制器）。

实验报告要求

477. 提交练习 25.1 生成的路网拓扑图 road_topology.png
478. 提交 A* 路径规划代码及测试结果截图
479. 提交 ROS2 节点运行截图（Rviz2 中显示路径）
480. 撰写实验总结（不少于 200 字），对比 Dijkstra 与 A* 在 CARLA 路网中的实际表现

第 26 章 MoveIt2 基础

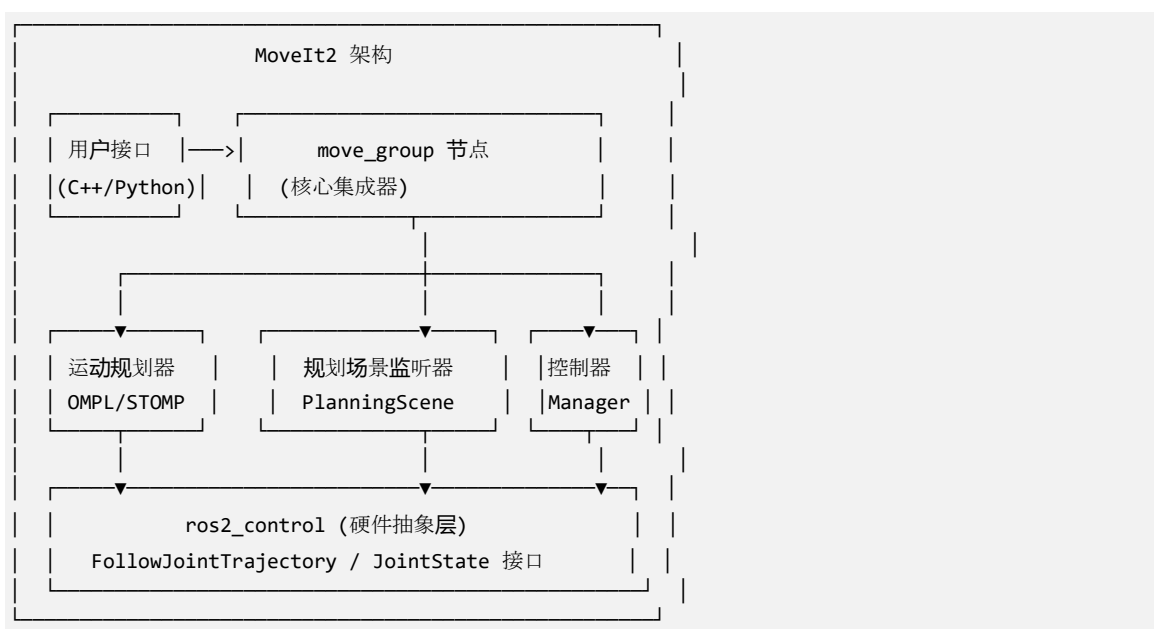
学习目标

- 理解 MoveIt2 的架构设计
- 掌握 Setup Assistant 配置流程
- 学会使用 MoveGroupInterface API
- 理解 OMPL 运动规划器的工作原理
- 掌握碰撞检测的配置和使用

26.1 MoveIt2 架构

26.1.1 MoveIt2 概述

MoveIt2 是 ROS2 环境下最主流的机械臂运动规划框架，是 ROS1 MoveIt 的升级版本。它综合了运动规划、控制、3D 感知、运动学和控制的最新成果，提供了开发先进机器人应用的易用平台。



26.1.2 核心模块

move_group 节点：

move_group 是 MoveIt2 的核心集成器，将所有系统组件组合在一起，提供一系列可供用户自由使用的 ROS2 操作与服务。

- **C++ API:** move_group_interface 包提供 C++ 接口
- **Python API:** moveit_py 包提供 Python 接口
- **GUI:** RViz2 中的 MotionPlanning 插件

配置参数：

参数	说明	来源
robot_description	URDF 机器人模型	参数服务器
robot_description_semantic	SRDF 语义模型	参数服务器
kinematics.yaml	运动学求解器配置	MoveIt 配置包
joint_limits.yaml	关节限制配置	MoveIt 配置包
ompl_planning.yaml	OMPL 规划器配置	MoveIt 配置包

26.1.3 规划场景（PlanningScene）

PlanningScene 是 MoveIt2 中环境模型的核心概念，包含：

- **RobotState:** 当前机器人状态（各关节位置）
- **CollisionObjects:** 环境中的碰撞物体
- **AttachedCollisionObjects:** 附着在机器人上的物体
- **OccupancyMap:** 环境占用地图

26.1.4 运动规划流程

MoveIt2 的典型运动规划流程：

用户设定目标 → move_group 接收请求

→ 查询规划场景（当前状态+环境）

→ 调用运动学求解器（IK/FK）

→ 调用运动规划器（OMPL 等）

→ 碰撞检测验证

- 返回规划轨迹
- 通过控制器执行轨迹

26.2 MoveIt2 Setup Assistant

26.2.1 安装 MoveIt2

```
# 安装 MoveIt2 完整包
sudo apt install ros-jazzy-moveit

# 安装 MoveIt2 Python API
sudo apt install ros-jazzy-moveit-py

# 安装 Setup Assistant
sudo apt install ros-jazzy-moveit-setup-assistant
```

26.2.2 启动配置助手

```
# 启动 Setup Assistant
ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

26.2.3 配置步骤详解

步骤 1：加载机器人模型

点击"Create New MoveIt Configuration Package", 选择 URDF 或 Xacro 文件加载。系统会自动解析机器人结构。

步骤 2：生成自碰撞免检矩阵

点击"Generate Collision Matrix", 系统自动检测不可能发生碰撞的连杆对, 将其加入免检列表, 减少规划计算量。

自动碰撞矩阵示例：

```
base_link ↔ link2: 距离远, 永不碰撞 → 免检
link1 ↔ link2: 相邻连杆, 碰撞检测 → 需检
link2 ↔ link4: 距离远, 永不碰撞 → 免检
```

步骤 3：添加虚拟关节（可选）

用于将机器人基座标系连接到世界坐标系：

```
<virtual_joint name="base_footprint"
  type="fixed"
  parent_frame="world"
  child_link="base_link"/>
```

步骤 4: 添加规划组

规划组在语义上描述机器人的不同部分:

```
# kinematics.yaml
arm_group:
  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
  kinematics_solver_attempts: 3
```

规划组的配置方式有两种:

481. **基于关节**: 显式列出组包含的关节

482. **基于连杆**: 按连杆链自动确定关节

步骤 5: 添加末端执行器

指定手爪规划组为特定的末端执行器:

```
<end_effector name="gripper_ee"
  parent_link="link5"
  parent_group="arm_group"
  group="gripper"/>
```

步骤 6: 添加预设位姿

```
<group_state name="home" group="arm_group">
  <joint name="joint1" value="0"/>
  <joint name="joint2" value="0"/>
  <joint name="joint3" value="0"/>
</group_state>

<group_state name="retract" group="arm_group">
  <joint name="joint1" value="0"/>
  <joint name="joint2" value="-1.57"/>
  <joint name="joint3" value="1.57"/>
</group_state>
```

步骤 7: ROS2 Control 配置

```
controller_list:
- name: arm_controller
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5
    - joint6
```



```
- name: gripper_controller
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  joints:
    - gripper_left_joint
    - gripper_right_joint
```

步骤 8: 生成配置包

选择输出目录，点击"Generate Package"生成 MoveIt2 配置功能包。

26.2.4 配置文件解析

生成的配置包目录结构：

```
generated_moveit_config/
├─ config/
│   ├─ arm.srdf          # 语义描述
│   ├─ kinematics.yaml   # 运动学配置
│   ├─ joint_limits.yaml # 关节极限
│   ├─ ompl_planning.yaml # OMPL 规划器
│   ├─ fake_controllers.yaml # 虚拟控制器
│   └─ ros2_controllers.yaml # 实际控制器
├─ launch/
│   ├─ move_group.launch.py # move_group 启动
│   ├─ planning_context.launch.py # 模型加载
│   ├─ demo.launch.py      # 演示模式
│   └─ setup_assistant.launch.py
└─ rviz/
    └─ moveit.rviz          # Rviz 配置
```

joint_limits.yaml:

```
joint_limits:
  joint1:
    has_velocity_limits: true
    max_velocity: 0.5
    has_acceleration_limits: true
    max_acceleration: 0.5
  joint2:
    has_velocity_limits: true
    max_velocity: 0.5
    has_acceleration_limits: true
    max_acceleration: 0.5
```

fake_controllers.yaml:

```
controller_list:
- name: fake_arm_controller
  joints:
    - joint1
    - joint2
    - joint3
```

- joint4
- joint5
- joint6

26.3 MoveGroupInterface

26.3.1 基本概念

MoveGroupInterface 是 MoveIt2 提供给用户的主要编程接口，支持 C++ 和 Python 两种语言。它封装了运动规划、执行、运动学求解、碰撞检测等功能。

26.3.2 Python 中的 MoveGroupInterface

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class MoveGroupDemo(Node):
    def __init__(self):
        super().__init__('move_group_demo')

        # 初始化 MoveItPy
        self.moveit = MoveItPy(node_name='moveit_py')

        # 获取规划组件
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

        # 配置规划参数
        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.01)
        self.arm.set_max_velocity_scaling_factor(0.5)
        self.arm.set_max_acceleration_scaling_factor(0.5)

        self.get_logger().info('MoveGroupInterface 初始化完成')

    def plan_to_joint_target(self, joint_values):
        """规划到关目标"""
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_values)
        plan_result = self.arm.plan()
        if plan_result:
            self.get_logger().info('规划成功, 开始执行')
            self.arm.execute(plan_result.trajectory)
        else:
            self.get_logger().error('规划失败')

    def plan_to_pose_target(self, pose):
        """规划到位姿目标"""
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(pose, 'link5')
        plan_result = self.arm.plan()
```

```

if plan_result:
    self.arm.execute(plan_result.trajectory)

```

26.3.3 C++中的 MoveGroupInterface

```

#include <moveit/move_group_interface/move_group_interface.h>
#include <rclcpp/rclcpp.hpp>

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);
    auto node = std::make_shared<rclcpp::Node>("move_group_demo");

    // 创建 MoveGroupInterface
    moveit::planning_interface::MoveGroupInterface move_group(
        node, "arm_group");

    // 设置关节目标
    move_group.setJointValueTarget({0.0, 0.5, -0.5, 0.0, 0.0, 0.0});

    // 规划并执行
    moveit::planning_interface::MoveGroupInterface::Plan plan;
    bool success = (move_group.plan(plan) == moveit::core::MoveItErrorCode::SUCCESS);

    if (success) {
        move_group.execute(plan);
    }

    rclcpp::shutdown();
    return 0;
}

```

26.3.4 常用 API 方法

方法	功能	参数
set_joint_value_target()	设置关节目标	关节角度列表
set_pose_target()	设置末端位姿目标	Pose 对象/Stamped
set_named_target()	设置命名位姿	位姿名称字符串
set_start_state_to_current_state()	设置起始状态为当前	无
plan()	规划轨迹	可选规划器 ID
execute()	执行轨迹	规划结果
get_current_pose()	获取当前位姿	末端 link 名称
get_current_joint_values()	获取当前关节值	无
set_goal_tolerance()	设置容忍度	位置+姿态容忍度
set_max_velocity_scaling_factor()	速度缩放	0.0~1.0

set_max_acceleration_scaling_factor()	加速度缩放	0.0~1.0
--	-------	---------

26.4 OMPL 运动规划器

26.4.1 OMPL 概述

OMPL（Open Motion Planning Library）是 MoveIt2 默认的运动规划算法库，包含多种基于采样的规划算法。这些算法在高维配置空间中搜索可行路径。

26.4.2 主要规划算法

RRT（Rapidly-exploring Random Tree）：

从起点开始，随机采样并扩展树状结构，直到到达目标区域。RRTConnect 在起点和终点同时构建两棵树，加速收敛。

RRT 算法流程：

1. 初始化树 T，包含起始节点 q_start
2. 循环直到到达目标或超时：

a. 随机采样 q_rand
b. 找到 T 中最近的节点 q_near
c. 沿 q_near→q_rand 方向扩展一步得到 q_new
d. 检查 q_near→q_new 路径是否无碰撞
e. 若无碰撞，将 q_new 加入 T
f. 若 q_new 接近目标，返回路径

其他规划算法：

算法	特点	适用场景
RRTConnect	双向 RRT，收敛快	通用场景
PRM*	渐进最优	多查询场景
RRT*	渐进最优 RRT	需要最优路径
EST	基于扩展空间树	狭窄通道
KPIECE	基于单元分割	高维空间

26.4.3 规划器配置

```
# ompl_planning.yaml
arm_group:
  planner_configs:
    - RRTConnectkConfigDefault
    - RRTkConfigDefault
    - PRMkConfigDefault
    - ESTkConfigDefault

# 默认规划器
default_planner_config: RRTConnectkConfigDefault

RRTConnectkConfigDefault:
  type: geometric::RRTConnect
  range: 0.1          # 扩展步长
  goal_bias: 0.1      # 目标偏置概率
  max_planning_time: 1.0 # 最大规划时间
```

26.4.4 自定义规划参数

在 Python 代码中指定规划器和参数：

```
from moveit.planning import MoveItPy, PlanningComponent

class CustomPlannerDemo(Node):
    def __init__(self):
        super().__init__('custom_planner_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group')

    def plan_with_specific_planner(self, joint_goal):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_goal)

        plan_result = self.arm.plan(
            planner_id='RRTConnectkConfigDefault',
            planning_time=10.0, # 最大规划时间
            max_attempts=100,  # 最大尝试次数
        )

        if plan_result:
            self.arm.execute(plan_result.trajectory)
            return True
        return False
```

26.4.5 规划时间与质量权衡

planning_time	成功率	路径质量	适用场景
0.5s	低	一般	快速运动
2.0s	中	较好	常规任务

10.0s	高	最优	精密操作
-------	---	----	------

26.5 碰撞检测

26.5.1 碰撞检测机制

MoveIt2 使用 FCL (Flexible Collision Library) 和 Bullet 进行碰撞检测，支持以下碰撞检测模式：

- 483. **自碰撞检测**：检查机器人连杆之间的碰撞
- 484. **环境碰撞检测**：检查机器人与环境物体的碰撞
- 485. **附着物体碰撞检测**：检查附着在机器人上的物体与环境碰撞

26.5.2 碰撞检测配置

自碰撞免检矩阵：

通过在 SRDF 中定义免检对减少计算量：

```
<disable_collisions link1="base_link" link2="link2" reason="Never"/>
<disable_collisions link1="base_link" link2="link3" reason="Never"/>
<disable_collisions link1="base_link" link2="link4" reason="Adjacent"/>
```

碰撞检测设置：

```
# 在 MoveIt 配置中
collision_detection:
  type: "FCL"           # 碰撞引擎
  padding: 0.01         # 碰撞安全余量（米）
  scale: 1.0            # 碰撞缩放比例
```

26.5.3 编程中的碰撞检测

```
from moveit.core.collision_detection import CollisionRequest, CollisionResult

class CollisionCheckDemo(Node):
    def __init__(self):
        super().__init__('collision_check_demo')
        self.moveit = MoveItPy(node_name='moveit_py')

    def check_collision(self, joint_positions):
        """检查给定关节位置是否碰撞"""
        robot_model = self.moveit.get_robot_model()
        planning_scene = self.moveit.get_planning_scene_monitor()
```

```

# 设置机器人状态
robot_state = robot_model.get_default_state()
robot_state.set_joint_group_positions('arm_group', joint_positions)

# 碰撞检测请求
request = CollisionRequest()
request.contacts = True
request.max_contacts = 10

# 执行检测
result = CollisionResult()
planning_scene.check_collision(request, result, robot_state)

if result.collision:
    self.get_logger().warn('检测到碰撞!')
    for contact in result.contacts:
        self.get_logger().warn(
            f'碰撞对: {contact.body1_name} ↔ {contact.body2_name}')
else:
    self.get_logger().info('无碰撞')

```

26.5.4 添加碰撞物体

```

from geometry_msgs.msg import PoseStamped
from moveit_msgs.msg import CollisionObject
from shape_msgs.msg import SolidPrimitive

class ObstacleManager:
    def __init__(self, planning_scene_monitor):
        self.psm = planning_scene_monitor

    def add_box(self, box_id, frame_id, x, y, z, size):
        """添加长方体障碍物"""
        co = CollisionObject()
        co.id = box_id
        co.header.frame_id = frame_id
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = size

        pose = PoseStamped()
        pose.header.frame_id = frame_id
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.position.z = z
        pose.pose.orientation.w = 1.0

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]

        self.psm.process_collision_object(co)

    def add_cylinder(self, cyl_id, frame_id, x, y, z, height, radius):
        """添加圆柱体障碍物"""
        co = CollisionObject()

```

```

co.id = cyl_id
co.header.frame_id = frame_id
co.operation = CollisionObject.ADD

primitive = SolidPrimitive()
primitive.type = SolidPrimitive.CYLINDER
primitive.dimensions = [height, radius]

pose = PoseStamped()
pose.header.frame_id = frame_id
pose.pose.position.x = x
pose.pose.position.y = y
pose.pose.position.z = z
pose.pose.orientation.w = 1.0

co.primitives = [primitive]
co.primitive_poses = [pose.pose]

self.psm.process_collision_object(co)

def add_sphere(self, sphere_id, frame_id, x, y, z, radius):
    """添加球体障碍物"""
    co = CollisionObject()
    co.id = sphere_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.SPHERE
    primitive.dimensions = [radius]

    pose = PoseStamped()
    pose.header.frame_id = frame_id
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose.pose]

    self.psm.process_collision_object(co)

def remove_object(self, object_id):
    """移除障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    self.psm.process_collision_object(co)

```


26.6 启动 MoveIt2

26.6.1 真实硬件启动

本仓库的 `xarm_ros2_arm_only` 仅配置仿真，不提供真实硬件驱动入口。真实硬件 需要使用设备厂商提供的 `driver`、控制器和 MoveIt 配置，并将规划组、关节名和 控制器映射迁移到对应工作区；不要直接使用下面的仿真命令控制实体机械臂。

26.6.2 仿真模式启动

```
# 课程提供的 xArm6 MoveIt + RViz (mock components, 不启动 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true

# 课程提供的 Gazebo Harmonic + ros2_control + MoveIt2 仿真
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

26.6.3 启动文件解析

课程实际包名为 `xarm_ros2_arm_only`，其 `arm_only_move_group.launch.py` 使用 `MoveItConfigsBuilder` 加载 URDF/Xacro、SRDF、运动学、关节限位和 OMPL 配置，然后启动 `move_group` 与可选的 RViz。`arm_only.launch.py` 在此基础上增加 Gazebo Harmonic、`gz_ros2_control`、控制器生成和机器人 `spawn`。配置文件位于 `src/xarm/config/`，启动文件位于 `src/xarm/launch/`。

课后练习

486. 使用 MoveIt2 Setup Assistant 为一个六自由度机械臂配置 MoveIt2 环境，生成完整的配置包。
487. 编写 Python 程序，使用 MoveItPy API 控制机械臂运动到多个关节目标位置，并记录规划时间。
488. 修改 `ompl_planning.yaml` 配置文件，设置不同的规划器（RRTConnect、PRM、RRT*），比较各规划器的规划成功率和路径质量。
489. 编写程序在规划场景中添加障碍物（盒体、球体、圆柱体），测试碰撞检测功能。
490. 解释 MoveIt2 的完整运动规划流程，从用户请求到轨迹执行的各个步骤。

实验 26:

第 26 章 实验指导：车辆纵横向控制

****对应理论章节**:** 第 40 章《车辆纵横向控制 (PID) 》 ****实验课时**:** 3 课时 ****实验代码**:** `src/lab\_code/ch26\_lab/`

实验目标

491. 实现基于 PID 的纵向速度控制器
492. 实现基于 Pure Pursuit 的横向控制器
493. 在 CARLA 仿真环境中完成路径跟踪闭环测试

练习 26.1：实现纵向 PID 控制器

任务描述

实现一个完整的纵向速度 PID 控制器，包含抗积分饱和 (Anti-windup) 和微分滤波功能。

要求

- 实现 `pid_controller.py` 中的 `PIDController` 类
- 支持积分限幅和输出限幅
- 微分项带一阶低通滤波
- 支持速度→油门/刹车的映射

代码框架

```
class PIDController:
    def __init__(self, Kp, Ki, Kd, dt=0.05):
        # TODO: 初始化参数和状态变量
        pass

    def reset(self):
        # TODO: 重置积分和微分状态
        pass
```

```
def compute(self, target, current):
    # TODO: 计算 PID 输出

    # 1. 计算误差

    # 2. 更新积分项（带限幅）

    # 3. 计算微分项（带滤波）

    # 4. 组合输出并限幅

    pass
```

测试方法

```
# 在终端运行单元测试
python pid_controller.py --test
```

验收标准

- 阶跃响应无稳态误差 ($|e| < 0.1 \text{ m/s}$)
- 超调量 $< 10\%$
- 调节时间 $< 3\text{s}$

练习 26.2：实现 Pure Pursuit 横向控制器

任务描述

实现 Pure Pursuit 路径跟踪算法，包含预瞄距离自适应和转向角计算。

要求

- 实现 pure_pursuit.py 中的 PurePursuit 类
- 预瞄距离随速度自适应
- 查找最近路径点和预瞄点
- 计算期望前轮转角

代码框架

```
class PurePursuit:
    def __init__(self, wheelbase, Ld0=2.0, kv=0.5):
        # TODO: 初始化车辆参数

        pass

    def find_lookahead_point(self, waypoints, vehicle_pos):
        # TODO: 找到预瞄距离内的目标点
```

```
pass

def compute_steering(self, vehicle_pos,
                      vehicle_yaw, waypoints):
    # TODO: 计算前轮转角
    pass
```

测试方法

```
python pure_pursuit.py --test
```

验收标准

- 直线跟踪横向偏差 $< 0.2\text{m}$
- 弯道跟踪横向偏差 $< 0.5\text{m}$
- 不同速度下控制稳定

练习 26.3 : 在 CARLA 中完成路径跟踪测试

任务描述

将纵向和横向控制器集成到 `vehicle_ctrl.py`, 在 CARLA 仿真环境中完成路径跟踪。

要求

494. 启动 CARLA 服务器并加载地图 `Town03`
495. 使用 `lat_lon_ctrl.py` 作为主控节点
496. 设定 10 个路径点的环形路径
497. 控制车辆以 36km/h (10m/s) 目标速度循迹
498. 记录并输出跟踪误差数据

运行步骤

```
# Step 1: 启动 CARLA 服务器
# Step 2: 运行控制节点
python lat_lon_ctrl.py
```

预期输出

```
[INFO] 启动车辆纵横向控制器
[INFO] 目标速度: 10.0 m/s
[INFO] 目标路径点: 10 个
```

```
[INFO] --- 第 1 帧 ---
[INFO] 实际速度: 1.23 m/s, 速度误差: 8.77 m/s
[INFO] 横向偏差: 1.45 m, 航向误差: 12.34 deg
[INFO] 油门: 0.85, 刹车: 0.00, 方向盘: -0.12
...
[INFO] === 测试完成 ===
[INFO] 平均速度误差: 0.45 m/s
[INFO] 平均横向偏差: 0.28 m
[INFO] 最大横向偏差: 0.62 m
```

扩展挑战（可选）

- 499. 增加 Stanley 控制器，与 Pure Pursuit 对比
- 500. 实现增益调度 PID，在不同速度下切换参数
- 501. 增加前馈控制项，改善弯道性能
- 502. 使用 ROS2 话题发布控制状态，用 rqt 可视化

提交要求

- 503. 完整实现 `pid_controller.py`, `pure_pursuit.py`, `vehicle_ctrl.py`, `lat_lon_ctrl.py`
- 504. 提供 CARLA 测试录屏或日志文件
- 505. 提交实验报告（PDF），包含：
- 506. 控制器参数及调试过程
- 507. 关键代码段说明
- 508. 测试结果截图和分析

评分标准

项目	分值	要求
纵向 PID 实现	30 分	完整实现、抗积分饱和、无稳态误差
Pure Pursuit 实现	30 分	预瞄自适应、弯道跟踪稳定
CARLA 集成测试	30 分	闭环跟踪、误差在范围内
实验报告	10 分	分析充分、数据完整

实际运行证据

当前环境已实际运行 PID 和 Pure Pursuit 控制器测试；CARLA 闭环部分因缺少 CARLA 服务器暂未登记截图。

```
ch06 pid and pure pursuit
Captured terminal session
2.00 10.00 2.05 7.950 1.000
3.00 10.00 3.05 6.950 1.000
4.00 10.00 4.05 5.950 1.000
5.00 10.00 5.05 4.950 1.000
6.00 10.00 6.05 3.950 1.000
7.00 10.00 7.05 2.950 1.000
8.00 10.00 8.05 1.950 1.000
9.00 10.00 9.05 0.950 1.000
=====
□ 纵向跟踪误差: 0.1000 (目标 < 0.1)
=====
跟踪误差: 0.0%
$ python3 pure_pursuit.py -test
Pure Pursuit 单元测试
=====
参数: 2.5 Ld0 2.0, kv 0.5
=====
  步骤   位置x   位置y   航向   航向角   Ld
-----
0   0.25  0.500  -0.006  -0.110  4.50
5   1.50  0.482  -0.022  -0.039  4.50
10  2.75  0.449  -0.032  -0.034  4.50
15  4.00  0.406  -0.040  -0.031  4.50
20  5.25  0.352  -0.047  -0.030  4.50
25  6.50  0.291  -0.048  -0.019  4.50
30  7.74  0.233  -0.044  -0.014  4.50
35  8.99  0.180  -0.041  -0.012  4.50
40  10.24 0.130  -0.038  -0.012  4.50
45  11.49 0.085  -0.033  -0.026  4.50
=====
□ 路径跟踪误差: 横向偏差 0.0530m (目标 < 0.2m)
=====
Source: test suite, recording of the console output.
```

原始录制：[ch26_control.cast](#)。完整证据索引见[实际运行证据](#)。

第 27 章 MoveIt2 Python 规划

学习目标

- 掌握 MoveItPy API 的核心类和用法
- 学会使用 PlanningComponent 进行运动规划
- 理解关节空间运动规划的方法
- 掌握正逆运动学求解的编程实现

27.1 MoveItPy API 概览

27.1.1 MoveItPy 简介

MoveItPy 是 MoveIt2 的 Python API 封装，提供了控制机械臂运动规划的全套接口。通过 MoveItPy，开发者可以用 Python 编程方式实现关节空间运动规划、笛卡尔空间路径规划、规划场景与碰撞检测、物体抓取与放置和运动学求解等功能。

27.1.2 核心类

MoveItPy — 主入口类，负责加载配置、初始化规划场景监听器：

```
from moveit.planning import MoveItPy

# 初始化 MoveItPy
moveit = MoveItPy(node_name='moveit_py')
```

PlanningComponent — 对应 MoveGroup 概念，封装了规划与执行接口：

```
from moveit.planning import PlanningComponent

# 获取规划组件
arm = PlanningComponent(
    moveit,          # MoveItPy 实例
    'arm_group',     # 规划组名称
    'link5'          # 末端执行器 link
)
```

RobotState — 表示机器人状态，包含各关节位置：

```
from moveit.core.robot_state import RobotState

robot_model = moveit.get_robot_model()
robot_state = RobotState(robot_model)
robot_state.set_joint_group_positions(
    'arm_group',
```

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
)
```

27.1.3 编程基本流程

使用 MoveItPy API 编程通常遵循以下步骤：

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class MoveIt2Programmer(Node):
    def __init__(self):
        super().__init__('moveit2_programmer')

        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

    def plan_and_execute(self):
        # 步骤1：设置起始状态
        self.arm.set_start_state_to_current_state()

        # 步骤2：设置目标（关节空间或笛卡尔空间）
        self.arm.set_joint_value_target([0.5, -0.3, 0.0, 0.0, 0.0, 0.0])

        # 步骤3：规划轨迹
        plan_result = self.arm.plan()

        # 步骤4：执行轨迹
        if plan_result:
            self.arm.execute(plan_result.trajectory)
            self.get_logger().info('轨迹执行完成')
        else:
            self.get_logger().error('规划失败')

def main(args=None):
    rclpy.init(args=args)
    node = MoveIt2Programmer()
    rclpy.spin_once(node, timeout_sec=1.0)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

27.1.4 配置参数

MoveltPy 的配置可通过参数文件或直接传入参数:

```
from moveit.planning import MoveItPy

# 通过 ROS2 参数配置
moveit = MoveItPy(
```



```

        node_name='moveit_py',
        config_files=[
            'config/kinematics.yaml',
            'config/ompl_planning.yaml',
        ]
    )

```

27.2 PlanningComponent 详解

27.2.1 创建 PlanningComponent

```

# 基本创建
self.arm = PlanningComponent(
    self.moveit, 'arm_group', 'link5'
)

# 获取多个规划组
self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')

```

27.2.2 设置规划参数

```

# 位置容忍度 (米)
self.arm.set_goal_position_tolerance(0.01)

# 姿态容忍度 (弧度)
self.arm.set_goal_orientation_tolerance(0.01)

# 参考坐标系
self.arm.set_pose_reference_frame('base_link')

# 末端执行器 link
end_effector_link = self.arm.get_end_effector_link()

# 速度缩放因子 (0.0 ~ 1.0)
self.arm.set_max_velocity_scaling_factor(0.5)

# 加速度缩放因子 (0.0 ~ 1.0)
self.arm.set_max_acceleration_scaling_factor(0.5)

```

27.2.3 设置目标

```

from geometry_msgs.msg import Pose, PoseStamped
from math import pi
from tf_transformations import quaternion_from_euler

# 方法 1：关节空间目标
joint_goal = [0.5, -0.3, 0.2, 0.0, 0.0, 0.0]
self.arm.set_joint_value_target(joint_goal)

# 方法 2：字典形式关节目标
joint_goal_dict = {
    'joint1': 0.5,

```

```

        'joint2': -0.3,
        'joint3': 0.2,
        'joint4': 0.0,
        'joint5': 0.0,
        'joint6': 0.0,
    }
    self.arm.set_joint_value_target(joint_goal_dict)

# 方法3：命名位姿
self.arm.set_named_target('home')

# 方法4：末端位姿目标
target_pose = Pose()
target_pose.position.x = 0.4
target_pose.position.y = 0.1
target_pose.position.z = 0.3
q = quaternion_from_euler(pi, 0, 0)
target_pose.orientation.x = q[0]
target_pose.orientation.y = q[1]
target_pose.orientation.z = q[2]
target_pose.orientation.w = q[3]
self.arm.set_pose_target(target_pose, 'link5')

```

27.2.4 规划与执行

```

# 基本规划
plan_result = self.arm.plan()

# 指定规划器和规划时间
plan_result = self.arm.plan(
    planner_id='RRTConnectkConfigDefault',
    planning_time=5.0
)

# 检查规划结果
if plan_result:
    trajectory = plan_result.trajectory
    planning_time = plan_result.planning_time
    fraction = plan_result.fraction # 笛卡尔路径覆盖率
    self.get_logger().info(
        f'规划成功：轨迹点{len(trajectory.points)}, '
        f'耗时{planning_time:.2f}s'
    )
    # 执行轨迹
    self.arm.execute(trajectory)
else:
    self.get_logger().error('规划失败')

```

27.2.5 获取当前状态

```

from geometry_msgs.msg import Pose

# 获取当前关节值
joint_values = self.arm.get_current_joint_values()

```

```

self.get_logger().info(f'关节值: {joint_values}')

# 获取当前末端位姿
current_pose = self.arm.get_current_pose('link5')
if current_pose:
    pos = current_pose.pose.position
    self.get_logger().info(
        f'末端位置: ({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})'
    )

```

27.3 关节空间运动规划

27.3.1 正运动学规划

正运动学规划直接指定各关节的目标角度，MoveIt2 规划一条从当前位置到目标位置的平滑轨迹。

```

#!/usr/bin/env python3
"""关节空间运动规划示例"""
import time
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class JointSpacePlanner(Node):
    def __init__(self):
        super().__init__('joint_space_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

    def move_arm_to_joint(self, joint_positions):
        """移动到指定关节位置"""
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_positions)

        plan_result = self.arm.plan()
        if plan_result:
            self.get_logger().info(f'执行关节目标: {joint_positions}')
            self.arm.execute(plan_result.trajectory)
            return True
        else:
            self.get_logger().warn(f'关节目标规划失败: {joint_positions}')
            return False

    def move_gripper(self, open=True):
        """控制夹爪开合"""
        goal = [0.65, 0.65] if open else [0.0, 0.0]
        self.gripper.set_start_state_to_current_state()

```

```

        self.gripper.set_joint_value_target(goal)
        plan_result = self.gripper.plan()
        if plan_result:
            self.gripper.execute(plan_result.trajectory)

def run_sequence(self):
    """执行一系列关节运动"""
    # 回到 home 位姿
    self.get_logger().info('回到 home 位姿')
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 打开夹爪
    self.get_logger().info('打开夹爪')
    self.move_gripper(open=True)
    time.sleep(0.5)

    # 运动到第一个关节目标
    self.move_arm_to_joint([0.5, -0.3, 0.2, 0.0, 0.0, 0.0])
    time.sleep(1)

    # 运动到第二个关节目标
    self.move_arm_to_joint([-0.5, -0.5, 0.8, 0.0, 0.0, 0.0])
    time.sleep(1)

    # 关闭夹爪
    self.get_logger().info('关闭夹爪')
    self.move_gripper(open=False)
    time.sleep(0.5)

    # 回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)

    self.get_logger().info('关节空间规划演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = JointSpacePlanner()
    node.run_sequence()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

27.3.2 使用命名位姿

在 SRDF 中预先定义命名位姿，方便在程序中直接调用：

```

# 定义命名位姿
self.arm.set_named_target('home')
self.arm.set_named_target('vertical')
self.arm.set_named_target('horizontal')
self.arm.set_named_target('retract')

# 规划并执行
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

```

27.4 逆运动学求解

27.4.1 设置末端位姿目标

逆运动学规划指定末端执行器的目标位姿，MoveIt2 解算出各关节角度后规划轨迹：

```

#!/usr/bin/env python3
"""逆运动学规划示例"""
import time
from math import pi
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from tf_transformations import quaternion_from_euler
from moveit.planning import MoveItPy, PlanningComponent

class IKPlanner(Node):
    def __init__(self):
        super().__init__('ik_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

        # 配置规划参数
        end_effector_link = 'link5'
        reference_frame = 'base_link'

        self.arm.set_pose_reference_frame(reference_frame)
        self.arm.set_goal_position_tolerance(0.02)
        self.arm.set_goal_orientation_tolerance(0.03)
        self.arm.set_max_acceleration_scaling_factor(0.6)
        self.arm.set_max_velocity_scaling_factor(0.35)

    def plan_to_pose(self, x, y, z, roll=0, pitch=0, yaw=0):
        """规划到位姿目标"""
        # 先回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        time.sleep(1)

```

```

# 构建目标位姿
target_pose = PoseStamped()
target_pose.header.frame_id = 'base_link'
target_pose.pose.position.x = x
target_pose.pose.position.y = y
target_pose.pose.position.z = z

q = quaternion_from_euler(roll, pitch, yaw)
target_pose.pose.orientation.x = q[0]
target_pose.pose.orientation.y = q[1]
target_pose.pose.orientation.z = q[2]
target_pose.pose.orientation.w = q[3]

# 规划
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(target_pose.pose, 'link5')
plan_result = self.arm.plan()

if plan_result:
    self.get_logger().info(
        f'IK 规划成功: 目标位置({x:.2f}, {y:.2f}, {z:.2f})'
    )
    self.arm.execute(plan_result.trajectory)
    return True
else:
    self.get_logger().error(
        f'IK 规划失败: 目标位置({x:.2f}, {y:.2f}, {z:.2f})'
    )
    return False

def run(self):
    # 多个目标位姿
    targets = [
        (0.3, 0.1, 0.25, pi, 0, 0),
        (0.3, -0.1, 0.25, pi, 0, 0),
        (0.4, 0.0, 0.35, pi, 0, 0),
    ]

    for x, y, z, roll, pitch, yaw in targets:
        success = self.plan_to_pose(x, y, z, roll, pitch, yaw)
        if not success:
            self.get_logger().warn(f'目标 {x},{y},{z} 不可达')
            time.sleep(1)

    self.get_logger().info('IK 规划演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = IKPlanner()
    node.run()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

27.4.2 直接逆运动学求解

MoveItPy 也支持直接求解逆运动学，不经过运动规划：

```
from moveit.core.kinematics import KinematicsSolver

class DirectIKSolver(Node):
    def __init__(self):
        super().__init__('direct_ik_solver')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.robot_model = self.moveit.get_robot_model()
        self.kinematics_solver = KinematicsSolver(self.robot_model)

    def solve_ik(self, group_name, target_pose, link_name):
        """直接求解逆运动学"""
        robot_state = self.robot_model.get_default_state()

        success, solution = self.kinematics_solver.solve_ik(
            group_name=group_name,
            target_pose=target_pose,
            link_name=link_name,
            robot_state=robot_state,
            timeout=0.1,
        )

        if success:
            joint_positions = solution.get_joint_group_positions(group_name)
            self.get_logger().info(f'IK 解算成功: {joint_positions}')
            return joint_positions
        else:
            self.get_logger().error('IK 解算失败')
            return None
```

27.4.3 正运动学求解

```
from moveit.core.kinematics import KinematicsSolver

class DirectFKSolver(Node):
    def __init__(self):
        super().__init__('direct_fk_solver')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.robot_model = self.moveit.get_robot_model()
        self.kinematics_solver = KinematicsSolver(self.robot_model)

    def solve_fk(self, group_name, joint_positions, link_name):
        """求解正运动学"""
        robot_state = self.robot_model.get_default_state()
        robot_state.set_joint_group_positions(group_name, joint_positions)
        robot_state.update()

        # 计算末端位姿
        pose = robot_state.get_pose(link_name)

        pos = pose.position
        self.get_logger().info(
            f'正运动学结果: 位置({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})'
```

```

    )
    return pose

def print_current_pose(self):
    """输出当前末端位姿"""
    robot_state = self.robot_model.get_default_state()
    robot_state.update()
    pose = robot_state.get_pose('link5')
    pos = pose.position
    ori = pose.orientation
    self.get_logger().info(
        f'当前位置: ({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})\n'
        f'当前姿态: ({ori.x:.3f}, {ori.y:.3f}, {ori.z:.3f}, {ori.w:.3f})'
    )

```

27.5 完整应用示例

27.5.1 多目标运动规划器

```

#!/usr/bin/env python3
"""多目标运动规划器"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose
from moveit.planning import MoveItPy, PlanningComponent

class MultiTargetPlanner(Node):
    def __init__(self):
        super().__init__('multi_target_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.01)

    def go_home(self):
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

    def plan_joint_motion(self, joint_goal, desc=''):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_goal)
        plan = self.arm.plan()
        if plan:
            self.get_logger().info(f'{desc}: 规划成功')
            self.arm.execute(plan.trajectory)
            return True
        self.get_logger().error(f'{desc}: 规划失败')

```



```

        return False

    def plan_cartesian_motion(self, pose_goal, desc=''):
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(pose_goal, 'link5')
        plan = self.arm.plan()
        if plan:
            self.get_logger().info(f'{desc}: 规划成功')
            self.arm.execute(plan.trajectory)
            return True
        self.get_logger().error(f'{desc}: 规划失败')
        return False

    def run_demo(self):
        self.go_home()
        time.sleep(0.5)

        # 关节空间运动
        self.plan_joint_motion(
            [0.5, -0.5, 0.3, 0.0, 0.0, 0.0],
            '关节运动1'
        )
        time.sleep(0.5)

        # 回到 home
        self.go_home()
        time.sleep(0.5)

        # 笛卡尔空间运动
        pose = Pose()
        pose.position.x = 0.4
        pose.position.y = 0.0
        pose.position.z = 0.3
        pose.orientation.w = 1.0
        self.plan_cartesian_motion(pose, '笛卡尔运动1')
        time.sleep(0.5)

        self.get_logger().info('演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = MultiTargetPlanner()
    node.run_demo()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

27.5.2 规划结果分析

```

class PlanAnalyzer(Node):
    def analyze_plan(self, plan_result):
        """分析规划结果"""
        if not plan_result:
            return

```

```

trajectory = plan_result.trajectory
points = trajectory.points

# 轨迹信息
num_points = len(points)
duration = points[-1].time_from_start.sec + \
           points[-1].time_from_start.nanosec * 1e-9

# 关节位置范围
joint_ranges = []
for j in range(len(points[0].positions)):
    positions = [p.positions[j] for p in points]
    joint_ranges.append(max(positions) - min(positions))

self.get_logger().info(
    f'轨迹分析:\n'
    f'  轨迹点数: {num_points}\n'
    f'  总时长: {duration:.2f}s\n'
    f'  规划耗时: {plan_result.planning_time:.3f}s\n'
    f'  关节运动范围: {joint_ranges}'
)

```

课后练习

509. 编写 MoveItPy 程序，控制六自由度机械臂依次运动到 5 个不同的关节目标位置，每次到达后输出当前关节值。
510. 使用逆运动学规划，让末端执行器沿 Z 轴上升 0.1 米、沿 X 轴移动 0.2 米、再回到初始位置。
511. 编写程序，分别使用命名位姿和关节目标两种方式控制机械臂运动到同一位置，比较代码复杂度和执行效果。
512. 使用 MoveItPy 的逆运动学求解器，计算末端位姿($x=0.3$, $y=0.1$, $z=0.2$, $\text{roll}=\pi$, $\text{pitch}=0$, $\text{yaw}=0$)对应的关节角度。
513. 编写正运动学程序，读取当前各关节角度，计算并输出末端执行器当前位姿。

实验 27:

第 27 章 实验手册: 多传感器融合定位

对应理论章节: 第 41 章《多传感器融合定位》

实验课时: 3 课时

实验代码:
``src/lab_code/ch27_lab/``

实验目标: 掌握基于 robot_localization 的 LiDAR-IMU-GNSS 融合定位, 并利用 CARLA 真值评估精度。

实验环境

项目	版本/配置
OS	Ubuntu 24.04
ROS 2	Jazzy
CARLA	0.9.16
robot_localization	Jazzy 二进制安装
EVO	pip install evo
Python	3.10+

```
# 安装依赖
sudo apt install ros-jazzy-robot-localization
pip install evo --upgrade --no-deps
pip install numpy matplotlib pandas
```

练习 27.1: 构建 TF2 坐标树

目标

编写 TF2 广播节点, 发布 `map` $\rightarrow$ `odom` $\rightarrow$ `base_link` $\rightarrow$ `lidar/imu/gps` 的坐标系变换。

步骤

Step 1: 创建 TF 广播节点

创建 `src/lab_code/ch27_lab/tf_broadcaster.py` (已提供模板):

```
# 主要职责:
# 1. 发布 base_link  $\rightarrow$  lidar_link (static)
# 2. 发布 base_link  $\rightarrow$  imu_link (static)
# 3. 发布 base_link  $\rightarrow$  gps_link (static)
# 4. 接收 /ekf_odom 并转发 map $\rightarrow$ odom $\rightarrow$ base_link
```

Step 2: 启动 TF 树

```
# 启动 TF 树 (终端 1)
python3 tf_broadcaster.py

# 查看 TF 树 (终端 2)
ros2 run tf2_tools view_frames.py
```

Step 3: 验证坐标变换

```
# 查看所有变换
ros2 run tf2_ros tf2_echo map base_link
ros2 run tf2_ros tf2_echo base_link lidar_link
ros2 run tf2_ros tf2_echo base_link imu_link

# 预期输出: 各变换的平移/旋转分量与配置一致
```

实验检查点

检查项	预期结果
<code>`tf2_echo map base_link`</code>	显示 EKF 估计的相对位姿
<code>`tf2_echo base_link lidar_link`</code>	显示固定变换 (x:0, y:0, z:1.8)
<code>`tf2_echo base_link imu_link`</code>	显示固定变换 (x:0.5, y:0, z:0.3)
<code>`tf2_echo base_link gps_link`</code>	显示固定变换 (x:0.8, y:0, z:0.1)
<code>`view_frames` PDF</code>	包含所有 5 个坐标系节点

练习 27.2: 实现 EKF 融合节点

目标

使用 `robot_localization` 配置 EKF, 融合 IMU 预测 + LiDAR SLAM 位姿 + GNSS 位置。

步骤

Step 1: 配置 EKF 参数

编辑 `src/lab_code/ch27_lab/ekf_localization.yaml`, 重点关注:

- 514. **频率设置:** `frequency: 30.0`
- 515. **坐标系:** `world_frame: odom, odom_frame: odom, map_frame: map`
- 516. **IMU 配置:** 启用加速度 + 角速度 + 方向
- 517. **LiDAR 配置:** 启用位置 + 姿态 (6-DOF)
- 518. **GNSS 配置:** 启用位置 x/y/z (3-DOF)

Step 2: 启动融合系统

```
# 终端 1: CARLA + ROS2 桥接
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py

# 终端 2: LiDAR SLAM (FAST-LIO2 或类似)
ros2 launch fast_lio mapping.launch.py

# 终端 3: EKF 融合
ros2 launch robot_localization ekf.launch.py \
  params_file:=src/lab_code/ch27_lab/ekf_localization.yaml

# 终端 4: TF 广播
python3 src/lab_code/ch27_lab/tf_broadcaster.py
```

Step 3: 监测输出

```
# 查看融合后的里程计
ros2 topic echo /odometry/filtered --once

# 检查 EKF 诊断信息
ros2 topic echo /diagnostics --filter "m.name == 'ekf_filter_node'"
```

EKF 参数调优

参数	默认值	调优方向	效果
`process_noise_covariance`	默认	增大	更信任观测, 响应更快
`odom0_queue_size`	5	增大	减少丢帧
`imu0_remove_gravitational_acceleration`	true	保持	消除重力影响
`print_diagnostics`	true	保持调试	查看协方差收敛

常见问题

Q: EKF 输出 NAN? A: 检查 IMU 数据是否包含加速度/角速度, 确认坐标系框架一致。

Q: 位姿跳变? A: 查看 innovation 是否在阈值内, 可能需要增大 R 或减小 Q。

练习 27.3: 对比 CARLA Ground Truth

目标

将 EKF 输出与 CARLA 提供的真值对比, 计算 ATE 和 RPE 指标。

步骤

Step 1: 录制数据

```
# 启动评估节点
python3 src/lab_code/ch27_lab/localization_eval.py

# 录制 bag (至少 60 秒)
ros2 bag record -o ch27_eval \
  /carla/ground_truth \
  /odometry/filtered
```

Step 2: 提取轨迹

```
# 提取为 TUM 格式
evo_traj bag ch27_eval/ch27_eval.db3 \
    /carla/ground_truth --save_as_tum gt.tum
evo_traj bag ch27_eval/ch27_eval.db3 \
    /odometry/filtered --save_as_tum est.tum
```

Step 3: ATE 评估

```
# 计算 ATE
evo_ape tum gt.tum est.tum -a -v --plot --plot_mode xy

# 输出示例:
# ATE RMSE: 0.186543 m
# ATE mean: 0.152341 m
# ATE std: 0.108724 m
# ATE min: 0.023456 m
# ATE max: 0.412358 m
```

Step 4: RPE 评估

```
# 计算 RPE (每 1 米)
evo_rpe tum gt.tum est.tum -a --delta 1.0 \
    --delta_unit m -v --plot --plot_mode xy
```

Step 5: 可视化分析

```
# 使用评估脚本生成报告
python3 localization_eval.py --bag ch27_eval/ch27_eval.db3

# 输出文件:
#   trajectory_comparison.png - 轨迹对比图
#   ate_error_map.png         - ATE 误差分布
#   evaluation_report.txt      - 详细评估报告
```

评估结果分析

```
# 查看报告
cat evaluation_report.txt

# 报告示例:
# =====
#   定位精度评估报告
# =====
#   数据集: ch27_eval/ch27_eval.db3
#   时长: 120.5 s
#   里程: 1863.2 m
#   ATE RMSE: 0.195 m ← 评估核心指标
#   ATE mean: 0.162 m
#   ATE std: 0.109 m
```

```
# RPE trans: 0.028 m/m ← 漂移率
# RPE rot: 0.015 deg/m
# 最大误差点: (x=123.4, y=567.8) @ t=45.2s
# =====
```

精度对比基准

定位方案	ATE RMSE	RPE trans	适用场景
纯 GNSS (单点)	2~5 m	0.2 m/m	开阔区域
GNSS-RTK	0.05~0.15 m	0.01 m/m	RTK 覆盖区
LiDAR SLAM	0.10~0.30 m	0.02 m/m	特征丰富
EKF 融合 (本章)	**0.10~0.30 m**	**<0.03 m/m**	**全场景**

实验报告要求

提交内容

- 519. tf_tree.pdf - 练习 27.1 的 TF 树截图
- 520. ekf_params.yaml - 练习 27.2 的最终参数配置
- 521. evaluation_report.txt - 练习 27.3 的完整评估报告
- 522. trajectory_comparison.png - 轨迹对比图
- 523. 实验总结 (200 字以上), 包含:

评分标准

项目	分数	标准
TF 树完整性	20 分	包含所有 5 个坐标系
EKF 配置正确	30 分	IMU/LiDAR/GNSS 均正确接入
ATE 评估	20 分	正确计算并绘制 ATE
精度达标	20 分	ATE RMSE < 0.30m
报告质量	10 分	分析充分, 图表完整

思考题

- 524. 为什么 EKF 使用 `world_frame=odom` 而不是 `world_frame=map`?
- 525. 当 LiDAR SLAM 输出频率从 10Hz 降低到 1Hz 时, EKF 精度如何变化?
- 526. 如何检测并处理 GNSS 多径效应造成的异常跳变?
- 527. 紧耦合相比松耦合在精度上有什么优势? 为什么本章选择松耦合?

参考命令速查

```
# TF CLI
ros2 run tf2_tools view_frames.py
ros2 run tf2_ros tf2_echo <parent> <child>

# 话题列表
ros2 topic list
ros2 topic hz /odometry/filtered

# 录包
ros2 bag record -a -o my_bag

# 评估 (evo)
evo_ape tum gt.tum est.tum -a --plot
evo_rpe tum gt.tum est.tum -a --delta 1 -v

# 轨迹可视化 (多个轨迹对齐)
evo_traj tum gt.tum est.tum --ref gt.tum -a --plot
```

****提示**:** 如果使用 CARLA 仿真, 请确保在启动桥接前已加载地图并生成车辆。参考 ``carla_ros_bridge`` 文档获取详细启动步骤。

第 28 章 MoveIt2 笛卡尔空间与避障

学习目标

- 掌握笛卡尔路径规划的方法
- 理解位姿约束和路径约束的概念
- 学会在规划场景中添加障碍物
- 掌握避障规划的配置和调试

28.1 笛卡尔路径规划

28.1.1 直线轨迹规划

笛卡尔空间规划要求机械臂末端沿指定路径运动，MoveIt2 使用 `plan_cartesian_path` 函数计算笛卡尔路径。

关键 API:

- `plan_cartesian_path(waypoints, eef_step, jump_threshold, avoid_collisions)`: 计算笛卡尔路径
- `waypoints`: 路径点列表
- `eef_step`: 末端步进值 (米)
- `jump_threshold`: 跳跃阈值 (0 为不允许跳跃)
- `avoid_collisions`: 避障规划
- 返回: (plan, fraction) — 规划结果和路径覆盖率

```
#!/usr/bin/env python3
"""笛卡尔直线路径规划示例"""
import time
from copy import deepcopy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose, PoseStamped
from moveit.planning import MoveItPy, PlanningComponent

class CartesianLinePlanner(Node):
    def __init__(self):
        super().__init__('cartesian_line_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
```

```

        self.moveit, 'arm_group', 'link5'
    )

    reference_frame = 'base_link'
    self.arm.set_pose_reference_frame(reference_frame)
    self.arm.set_goal_position_tolerance(0.005)
    self.arm.set_goal_orientation_tolerance(0.005)
    end_effector_link = 'link5'

    # 先回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 移动到一个初始位置
    target_pose = PoseStamped()
    target_pose.header.frame_id = reference_frame
    target_pose.pose.position.x = 0.4
    target_pose.pose.position.y = 0.0
    target_pose.pose.position.z = 0.35
    target_pose.pose.orientation.w = 1.0

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose.pose, end_effector_link)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 构建笛卡尔路径点
    current_pose = self.arm.get_current_pose(end_effector_link).pose
    start_pose = deepcopy(current_pose)

    waypoints = [deepcopy(start_pose)]

    # 路径点 1 : 向下移动 0.1 米
    wpose = deepcopy(start_pose)
    wpose.position.z -= 0.1
    waypoints.append(deepcopy(wpose))

    # 路径点 2 : 向右移动 0.1 米
    wpose.position.y += 0.1
    waypoints.append(deepcopy(wpose))

    # 路径点 3 : 向上移动 0.1 米
    wpose.position.z += 0.1
    waypoints.append(deepcopy(wpose))

    # 路径点 4 : 回到起点
    waypoints.append(deepcopy(start_pose))

    # 执行笛卡尔规划
    fraction = 0.0
    maxtries = 100
    attempts = 0

```

```

        self.arm.set_start_state_to_current_state()
        while fraction < 1.0 and attempts < maxtries:
            plan_result = self.arm.plan_cartesian_path(
                waypoints, 0.01, 0.0, True
            )
            if plan_result:
                fraction = plan_result.fraction
                attempts += 1
            if attempts % 10 == 0:
                self.get_logger().info(
                    f'尝试 {attempts}/{maxtries}, 覆盖率: {fraction:.1%}'
                )

        if fraction >= 1.0:
            self.get_logger().info('笛卡尔路径规划成功')
            self.arm.execute(plan_result.trajectory)
        else:
            self.get_logger().error(f'规划失败: 覆盖率 {fraction:.1%}')

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

        self.get_logger().info('笛卡尔直线规划完成')

def main(args=None):
    rclpy.init(args=args)
    node = CartesianLinePlanner()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.1.2 圆弧轨迹规划

`plan_cartesian_path` 支持直线路径，但可以通过微分思想实现圆弧轨迹：将圆弧切分为多个小线段，用线段近似圆弧。

```

#!/usr/bin/env python3
"""圆弧轨迹规划示例"""
import math
import time
import numpy as np
from copy import deepcopy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent

class CartesianArcPlanner(Node):
    def __init__(self):
        super().__init__('cartesian_arc_planner')

```

```

self.moveit = MoveItPy(node_name='moveit_py')
self.arm = PlanningComponent(
    self.moveit, 'arm_group', 'link5'
)

reference_frame = 'base_link'
self.arm.set_pose_reference_frame(reference_frame)
self.arm.set_goal_position_tolerance(0.001)
self.arm.set_goal_orientation_tolerance(0.001)
self.arm.set_max_acceleration_scaling_factor(0.5)
self.arm.set_max_velocity_scaling_factor(0.5)

end_effector_link = 'link5'

# 回到 home
self.arm.set_named_target('home')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
time.sleep(1)

# 移动到圆弧中心位置
target_pose = PoseStamped()
target_pose.header.frame_id = reference_frame
target_pose.pose.position.x = 0.4
target_pose.pose.position.y = 0.0
target_pose.pose.position.z = 0.35
target_pose.pose.orientation.w = 1.0

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(target_pose.pose, end_effector_link)
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
time.sleep(1)

# 生成圆弧路径点 (Y-Z 平面)
waypoints = []
center_y = target_pose.pose.position.y
center_z = target_pose.pose.position.z
radius = 0.1

for th in np.arange(0, math.pi * 2, 0.015):
    target_pose.pose.position.y = center_y + radius * math.cos(th)
    target_pose.pose.position.z = center_z + radius * math.sin(th)
    waypoints.append(deepcopy(target_pose.pose))

# 执行笛卡尔规划
fraction = 0.0
maxtries = 100
attempts = 0

self.arm.set_start_state_to_current_state()
while fraction < 1.0 and attempts < maxtries:
    plan_result = self.arm.plan_cartesian_path(
        waypoints, 0.01, 0.0, True
    )
    if plan_result:

```

```

        fraction = plan_result.fraction
        attempts += 1

    if fraction >= 1.0:
        self.get_logger().info('圆弧路径规划成功')
        self.arm.execute(plan_result.trajectory)
    else:
        self.get_logger().error(f'规划失败: 覆盖率 {fraction:.1%}')

    time.sleep(1)

    # 回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)

    self.get_logger().info('圆弧规划完成')

def main(args=None):
    rclpy.init(args=args)
    node = CartesianArcPlanner()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.1.3 S 形曲线规划

除了直线和圆弧，还可以生成 S 形、锯齿形等复杂路径：

```

def generate_s_shape(start_pose, amplitude=0.05, steps=20):
    """生成 S 形路径点"""
    waypoints = []
    for i in range(steps):
        t = i / steps
        x = start_pose.position.x + t * 0.1 # X 方向前进
        y = start_pose.position.y + amplitude * math.sin(t * 4 * math.pi)
        z = start_pose.position.z

        pose = deepcopy(start_pose)
        pose.position.x = x
        pose.position.y = y
        pose.position.z = z
        waypoints.append(deepcopy(pose))
    return waypoints

def generate_zigzag(start_pose, width=0.08, segments=4):
    """生成锯齿形路径点"""
    waypoints = []
    direction = 1
    for i in range(segments):
        for j in range(10):
            t = j / 10
            x = start_pose.position.x + (i * 0.02 + t * 0.02)
            y = start_pose.position.y + direction * width * t

```

```

        pose = deepcopy(start_pose)
        pose.position.x = x
        pose.position.y = y
        waypoints.append(deepcopy(pose))
    direction *= -1
    return waypoints

```

28.2 位姿约束

28.2.1 路径约束类型

Movelt2 支持在规划过程中添加各种约束，确保路径满足特定要求：

约束类型	说明	适用场景
PositionConstraint	位置约束	限制末端位置范围
OrientationConstraint	姿态约束	保持末端姿态
JointConstraint	关约束	限制特定关节角度
VisibilityConstraint	可见性约束	目标必须在视野内

28.2.2 添加位姿约束

```

from moveit_msgs.msg import Constraints, PositionConstraint, OrientationConstraint
from shape_msgs.msg import SolidPrimitive
from geometry_msgs.msg import Point

class ConstrainedPlanner(Node):
    def __init__(self):
        super().__init__('constrained_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

    def add_position_constraint(self):
        """添加位置约束：限制末端在指定区域内"""
        constraints = Constraints()
        constraints.name = "position_constraint"

        pc = PositionConstraint()
        pc.header.frame_id = 'base_link'
        pc.link_name = 'link5'
        pc.weight = 1.0

        # 定义约束区域（球体）
        sphere = SolidPrimitive()
        sphere.type = SolidPrimitive.SPHERE
        sphere.dimensions = [0.3] # 半径 0.3 米的球体

```

```

pc.constraint_region.primitives = [sphere]
pc.constraint_region.primitive_poses = [Pose()]
pc.constraint_region.primitive_poses[0].position.x = 0.3
pc.constraint_region.primitive_poses[0].position.y = 0.0
pc.constraint_region.primitive_poses[0].position.z = 0.2

constraints.position_constraints = [pc]

self.arm.set_path_constraints(constraints)
self.get_logger().info('位置约束已添加')

def add_orientation_constraint(self):
    """添加姿态约束：保持末端朝下"""
    constraints = Constraints()
    constraints.name = "orientation_constraint"

    oc = OrientationConstraint()
    oc.header.frame_id = 'base_link'
    oc.link_name = 'link5'
    oc.weight = 1.0

    # 目标姿态：末端朝下（绕X轴旋转180度）
    from tf_transformations import quaternion_from_euler
    q = quaternion_from_euler(math.pi, 0, 0)
    oc.orientation.x = q[0]
    oc.orientation.y = q[1]
    oc.orientation.z = q[2]
    oc.orientation.w = q[3]

    oc.absolute_x_axis_tolerance = 0.1
    oc.absolute_y_axis_tolerance = 0.1
    oc.absolute_z_axis_tolerance = 0.1

    constraints.orientation_constraints = [oc]
    self.arm.set_path_constraints(constraints)
    self.get_logger().info('姿态约束已添加')

def clear_constraints(self):
    """清除所有约束"""
    self.arm.clear_path_constraints()
    self.get_logger().info('约束已清除')

```

28.2.3 关节约束

```

from moveit_msgs.msg import JointConstraint

def add_joint_constraint(self, joint_name, target_value, tolerance=0.1):
    """添加关节约束：限制某个关节的角度范围"""
    jc = JointConstraint()
    jc.joint_name = joint_name
    jc.position = target_value
    jc.tolerance_above = tolerance
    jc.tolerance_below = tolerance
    jc.weight = 1.0

```



```
constraints = Constraints()
constraints.joint_constraints = [jc]
self.arm.set_path_constraints(constraints)
self.get_logger().info(f'关节 {joint_name} 约束已添加: {target_value}±{tolerance}')
```

28.3 规划场景与障碍物

28.3.1 PlanningSceneInterface

MoveIt2 通过规划场景接口提供场景更新 API，包括添加、移除、附着物体等操作。

关键概念：

- **CollisionObject**：规划场景中的碰撞物体（盒体、球体、圆柱体、网格模型）
- **AttachedCollisionObject**：附着在机械臂上的物体
- **PlanningScene**：完整的规划场景消息

28.3.2 添加障碍物

```
#!/usr/bin/env python3
"""规划场景与障碍物示例"""
import time
from math import pi
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit_msgs.msg import PlanningScene, ObjectColor, CollisionObject
from shape_msgs.msg import SolidPrimitive
from moveit.planning import MoveItPy, PlanningComponent

class ObstacleDemo(Node):
    def __init__(self):
        super().__init__('obstacle_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.planning_scene_monitor = self.moveit.get_planning_scene_monitor()
        self.scene_pub = self.create_publisher(
            PlanningScene, 'planning_scene', 10
        )
        time.sleep(1)

        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )
        self.arm.set_goal_position_tolerance(0.02)
        self.arm.set_goal_orientation_tolerance(0.03)

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

    def add_box(self, object_id, frame_id, x, y, z, size):
```

```

    """添加长方体障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = size

    pose = PoseStamped()
    pose.header.frame_id = frame_id
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def add_sphere(self, object_id, frame_id, x, y, z, radius):
    """添加球体障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.SPHERE
    primitive.dimensions = [radius]

    pose = PoseStamped()
    pose.header.frame_id = frame_id
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def remove_object(self, object_id):
    """移除障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    self.planning_scene_monitor.process_collision_object(co)

def add_color(self, object_id, r, g, b, a=1.0):
    """设置物体颜色"""
    color = ObjectColor()
    color.id = object_id
    color.color.r = r
    color.color.g = g
    color.color.b = b
    color.color.a = a

```

```

p = PlanningScene()
p.is_diff = True
p.object_colors = [color]
self.scene_pub.publish(p)

def run(self):
    # 添加桌子
    self.get_logger().info('添加桌子')
    self.add_box('table', 'base_link', 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    self.add_color('table', 0.6, 0.4, 0.2)
    time.sleep(1)

    # 添加障碍物球体
    self.get_logger().info('添加球体障碍物')
    self.add_sphere('sphere_obs', 'base_link', 0.3, 0.15, 0.12, 0.12)
    self.add_color('sphere_obs', 0.8, 0.0, 0.9)
    time.sleep(1)

    # 添加障碍物箱体
    self.get_logger().info('添加箱体障碍物')
    self.add_box('box_obs', 'base_link', 0.3, -0.15, 0.2, [0.25, 0.05, 0.4])
    self.add_color('box_obs', 0.8, 0.8, 0.0)
    time.sleep(1)

    # 规划避障路径
    self.get_logger().info('规划避障路径')
    target_pose = PoseStamped()
    target_pose.header.frame_id = 'base_link'
    target_pose.pose.position.x = 0.25
    target_pose.pose.position.y = -0.35
    target_pose.pose.position.z = 0.25

    from tf_transformations import quaternion_from_euler
    q = quaternion_from_euler(pi, 0, 0)
    target_pose.pose.orientation.x = q[0]
    target_pose.pose.orientation.y = q[1]
    target_pose.pose.orientation.z = q[2]
    target_pose.pose.orientation.w = q[3]

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose.pose, 'link5')
    plan_result = self.arm.plan()

    if plan_result:
        self.get_logger().info('避障规划成功')
        self.arm.execute(plan_result.trajectory)
    else:
        self.get_logger().error('避障规划失败')

    time.sleep(2)

    # 移除障碍物
    self.remove_object('table')
    self.remove_object('sphere_obs')
    self.remove_object('box_obs')
    time.sleep(1)

```

```

        self.get_logger().info('障碍物演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleDemo()
    node.run()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.3.3 物体附着与分离

当机械臂抓取物体后，需要将物体附着到末端执行器上，使 MoveIt2 在规划时考虑该物体的碰撞体积：

```

from moveit_msgs.msg import AttachedCollisionObject

class AttachDetachDemo(Node):
    def __init__(self):
        super().__init__('attach_detach_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()

    def attach_object(self, object_id, link_name):
        """将物体附着到指定的连杆上"""
        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = link_name
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [0.04, 0.04, 0.1]

        pose = PoseStamped()
        pose.header.frame_id = link_name
        pose.pose.position.z = 0.05
        pose.pose.orientation.w = 1.0

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]

        aco = AttachedCollisionObject()
        aco.link_name = link_name
        aco.object = co
        aco.touch_links = ['link5', 'gripper_link']

        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'物体 {object_id} 已附着到 {link_name}')

    def detach_object(self, object_id, link_name):
        """从连杆上分离物体"""
        co = CollisionObject()
        co.id = object_id
        co.operation = CollisionObject.REMOVE

```

```

aco = AttachedCollisionObject()
aco.link_name = link_name
aco.object = co

self.psm.process_attached_collision_object(aco)
self.get_logger().info(f'物体 {object_id} 已从 {link_name} 分离')

```

28.4 避障规划

28.4.1 避障规划策略

Movelt2 的避障规划基于以下策略：

- 528. **碰撞检测**：在规划过程中持续检测机器人与障碍物的碰撞
- 529. **配置空间搜索**：在无碰撞的配置空间中搜索路径
- 530. **安全余量**：通过 padding 参数设置碰撞安全距离
- 531. **重规划**：环境变化时自动重新规划

28.4.2 碰撞安全余量设置

```

# 在 MoveIt 配置中设置碰撞安全余量
collision:
  padding: 0.02      # 全局碰撞安全余量 2cm
  padding_scale: 1.0 # 缩放因子
  max_contacts: 10   # 最大碰撞接触点数

```

编程方式设置：

```

from moveit.core.collision_detection import (
    CollisionRequest, CollisionResult, AllowedCollisionMatrix
)

class SafetyConfig:
    def set_safety_margins(self, planning_scene_monitor, padding=0.02):
        """设置碰撞安全余量"""
        acm = AllowedCollisionMatrix(
            planning_scene_monitor.get_planning_scene().get_robot_model()
        )
        # 可以设置特定连杆对的安全余量
        # acm.set_entry('link1', 'link2', True) # True=允许碰撞
        return acm

    def check_collision_with_margin(self, planning_scene, robot_state, margin=0.02):
        """带安全余量的碰撞检测"""
        request = CollisionRequest()
        request.padding = margin
        request.contacts = True

        result = CollisionResult()

```

```
planning_scene.check_collision(request, result, robot_state)

return result
```

28.4.3 避障规划完整示例

```
#!/usr/bin/env python3
"""避障规划完整示例"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent

class ObstacleAvoidancePlanner(Node):
    def __init__(self):
        super().__init__('obstacle_avoidance_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

    def setup_environment(self):
        """设置规划环境"""
        co = CollisionObject()
        co.id = 'table'
        co.header.frame_id = 'base_link'
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [0.8, 0.6, 0.02]
        pose = PoseStamped()
        pose.header.frame_id = 'base_link'
        pose.pose.position.z = -0.01
        pose.pose.orientation.w = 1.0
        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]
        self.psm.process_collision_object(co)

        # 添加左侧障碍物
        co2 = CollisionObject()
        co2.id = 'left_obstacle'
        co2.header.frame_id = 'base_link'
        co2.operation = CollisionObject.ADD

        prim2 = SolidPrimitive()
        prim2.type = SolidPrimitive.BOX
        prim2.dimensions = [0.1, 0.1, 0.3]
        pose2 = PoseStamped()
        pose2.header.frame_id = 'base_link'
        pose2.pose.position.x = 0.3
        pose2.pose.position.y = -0.2
        pose2.pose.position.z = 0.15
        pose2.pose.orientation.w = 1.0
        co2.primitives = [prim2]
        co2.primitive_poses = [pose2.pose]
```

```

self.psm.process_collision_object(co2)

# 添加右侧障碍物
co3 = CollisionObject()
co3.id = 'right_obstacle'
co3.header.frame_id = 'base_link'
co3.operation = CollisionObject.ADD

prim3 = SolidPrimitive()
prim3.type = SolidPrimitive.BOX
prim3.dimensions = [0.1, 0.1, 0.3]
pose3 = PoseStamped()
pose3.header.frame_id = 'base_link'
pose3.pose.position.x = 0.3
pose3.pose.position.y = 0.2
pose3.pose.position.z = 0.15
pose3.pose.orientation.w = 1.0
co3.primitives = [prim3]
co3.primitive_poses = [pose3.pose]
self.psm.process_collision_object(co3)

self.get_logger().info('环境设置完成：桌子 + 2 个障碍物')

def plan_avoidance(self):
    """规划避障路径（从中间穿过）"""
    # 回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 目标位姿（在障碍物后方）
    target_pose = PoseStamped()
    target_pose.header.frame_id = 'base_link'
    target_pose.pose.position.x = 0.4
    target_pose.pose.position.y = 0.0
    target_pose.pose.position.z = 0.25

    # 尝试多次规划
    for attempt in range(5):
        self.get_logger().info(f'避障规划尝试 {attempt+1}')
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose.pose, 'link5')

        # 增加规划时间以提高成功率
        plan_result = self.arm.plan(
            planner_id='RRTConnectkConfigDefault',
            planning_time=5.0
        )

        if plan_result:
            self.get_logger().info('避障规划成功')
            self.arm.execute(plan_result.trajectory)
            return True
        time.sleep(0.5)

```

```

        self.get_logger().error('避障规划失败, 目标可能不可达')
        return False

    def cleanup(self):
        """清理环境"""
        for obj in ['table', 'left_obstacle', 'right_obstacle']:
            co = CollisionObject()
            co.id = obj
            co.operation = CollisionObject.REMOVE
            self.psm.process_collision_object(co)

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleAvoidancePlanner()
    node.setup_environment()
    node.plan_avoidance()
    time.sleep(2)
    node.cleanup()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.4.4 规划场景的可视化

在 Rviz 中配置 MotionPlanning 插件，可以实时查看规划场景：

```

Rviz MotionPlanning 插件配置：
- PlanningScene Topic: /planning_scene
- Robot Description: robot_description
- Planning Group: arm_group
- Show Robot Visual: True
- Show Robot Collision: True
- Scene Geometry: Show All

```

课后练习

532. 编写笛卡尔路径规划程序，使末端执行器沿正方形的四条边运动（边长 0.1 米），回到起点。
533. 编写圆弧轨迹规划程序，使末端在水平面内画一个完整的圆形。
534. 在规划场景中添加三个不同形状的障碍物（箱体、球体、圆柱体），规划一条从起点到终点的避障路径。
535. 编写程序，将物体附着到末端执行器后规划运动，再分离物体，比较附着前后的规划行为差异。
536. 设计一个包含狭窄通道的避障场景，测试不同规划器（RRTConnect、PRM、EST）通过狭窄通道的成功率。

实验 28:

第 28 章 实验：交通参与者感知

****对应理论章节**:** 第 42 章《交通参与者感知》 ****实验课时**:** 2 课时 ****实验代码**:**
`src/lab\_code/ch28\_lab/`

实验目标

- 掌握 YOLOv8 在 CARLA 仿真环境中的目标检测与 ROS2 集成
- 学会 LiDAR 点云预处理与 DBSCAN 聚类实现障碍物检测
- 实现基于卡尔曼滤波的多目标跟踪与 ID 分配

实验环境

- ROS 2 Jazzy + CARLA 0.9.16
- carla_ros_bridge 已启动运行
- 传感器: RGB 前视相机 + LiDAR
- Python 3.8+: ultralytics, numpy, opencv-python, scikit-learn, filterpy

实验前准备

```
# 安装依赖
pip install ultralytics scikit-learn filterpy
pip install opencv-python numpy

# 下载 YOLOv8 模型
python3 -c "from ultralytics import YOLO; YOLO('yolov8n.pt')"

# 启动 CARLA 服务器 (终端 1)
/path/to/CarlaUE4.sh -quality-level=Low

# 启动 ROS2 Bridge (终端 2)
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py

# 启动传感器 (终端 3)
python3 src/lab_code/ch24_lab/sensor_config.py
```

```
# 确认话题可用
ros2 topic list | grep -E "(rgb_front|point_cloud)"
```

练习 28.1: CARLA 相机 YOLO 目标检测

目标: 订阅 CARLA 前视 RGB 相机图像，运行 YOLOv8 推理，发布检测结果。

步骤

1. 创建 Python 节点

编写 `yolo_detector.py`，实现：

- 订阅 `/carla/ego_vehicle/rgb_front/image` (`sensor_msgs/Image`)
- 加载 YOLOv8 模型进行推理
- 发布 `Detection2DArray` 到 `/perception/yolo/detections`
- 发布标注图像到 `/perception/yolo/visualization`

2. 关键代码

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D
from cv_bridge import CvBridge
from ultralytics import YOLO
import cv2

class CarlaYOLODetector(Node):
    def __init__(self):
        super().__init__('carla_yolo_detector')
        self.bridge = CvBridge()
        self.model = YOLO('yolov8n.pt')

        self.sub = self.create_subscription(
            Image, '/carla/ego_vehicle/rgb_front/image',
            self.image_cb, 10
        )
        self.det_pub = self.create_publisher(
            Detection2DArray, '/perception/yolo/detections', 10
        )
        self.vis_pub = self.create_publisher(
            Image, '/perception/yolo/visualization', 10
        )

    def image_cb(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        results = self.model(frame, conf=0.5, verbose=False)[0]
```

```

det_msg = Detection2DArray()
det_msg.header = msg.header

for box in results.bboxes:
    x1, y1, x2, y2 = box.xyxy[0].tolist()
    conf = float(box.conf[0])
    cls_id = int(box.cls[0])

    det = Detection2D()
    det.bbox.center.x = float((x1 + x2) / 2)
    det.bbox.center.y = float((y1 + y2) / 2)
    det.bbox.size_x = float(x2 - x1)
    det.bbox.size_y = float(y2 - y1)
    det_msg.detections.append(det)

    cv2.rectangle(frame, (int(x1), int(y1)), (int(x2), int(y2)), (0,255,0), 2)
    cv2.putText(frame, f'{results.names[cls_id]} {conf:.2f}',
                (int(x1), int(y1)-5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 2)

self.det_pub.publish(det_msg)
self.vis_pub.publish(self.bridge.cv2_to_imgmsg(frame, 'bgr8'))
cv2.imshow('YOLO Detection', frame)
cv2.waitKey(1)

```

3. 运行测试

```
python3 src/lab_code/ch28_lab/yolo_detector.py
```

```

# 验证话题
ros2 topic echo /perception/yolo/detections
ros2 topic hz /perception/yolo/detections

```

预期结果

- 实时显示带标注框的 CARLA 相机画面
- 检测到车辆、行人、自行车等 COCO 类别目标
- FPS 不低于 10 帧

练习 28.2: LiDAR 点云聚类检测障碍物

目标：订阅 LiDAR 点云，预处理后使用 DBSCAN 聚类检测障碍物。

步骤

1. 编写 LiDAR 聚类节点

创建 `lidar_cluster.py`，实现：

- 订阅 /carla/ego_vehicle/lidar/point_cloud (sensor_msgs/PointCloud2)
- 体素滤波降采样 (0.1m 体素)
- 直通滤波 (前方 50m 范围)
- DBSCAN 聚类
- 发布障碍物列表到 /perception/obstacles

2. 关键代码

```
import numpy as np
from sensor_msgs.msg import PointCloud2
import sensor_msgs_py.point_cloud2 as pc2
from sklearn.cluster import DBSCAN

class LidarObstacleDetector(Node):
    def __init__(self):
        super().__init__('lidar_obstacle_detector')
        self.sub = self.create_subscription(
            PointCloud2, '/carla/ego_vehicle/lidar/point_cloud',
            self.lidar_cb, 10
        )
        self.obstacle_pub = self.create_publisher(
            PointCloud2, '/perception/obstacles', 10
        )

    def preprocess(self, points):
        """点云预处理：直通滤波 + 体素滤波"""
        mask = (points[:, 0] > 0) & (points[:, 0] < 50) & \
            (points[:, 1] > -20) & (points[:, 1] < 20) & \
            (points[:, 2] > -2) & (points[:, 2] < 3)
        points = points[mask]

        # 体素滤波
        voxel_size = 0.1
        voxel_indices = np.floor(points[:, :2] / voxel_size).astype(np.int32)
        _, unique_idx = np.unique(voxel_indices, axis=0, return_index=True)
        return points[unique_idx]

    def cluster(self, points):
        """DBSCAN 聚类"""
        if len(points) < 10:
            return [], np.array([])

        clustering = DBSCAN(eps=0.5, min_samples=10).fit(points[:, :2])
        labels = clustering.labels_
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

        clusters = []
        for i in range(n_clusters):
            mask = labels == i
            if np.sum(mask) >= 10:
                clusters.append(points[mask])
```

```
return clusters, labels
```

3. 运行测试

```
python3 src/lab_code/ch28_lab/lidar_cluster.py

# 可视化
ros2 run rviz2 rviz2
# 添加 PointCloud2 显示 /perception/obstacles
```

预期结果

- CARLA 界面中车辆和行人被正确聚类
- 每个障碍物用不同颜色标记
- 远处密集点云被有效降采样

练习 28.3: 多目标跟踪与可视化

目标: 结合 YOLO 检测和 LiDAR 聚类结果, 实现多目标跟踪。

步骤

1. 实现卡尔曼滤波跟踪器

创建 `object_tracker.py`, 包含:

- 卡尔曼滤波器 (filterpy 库): 状态 = $[x, y, vx, vy]$
- 匈牙利匹配 (scikit-learn 的 `linear_assignment` 或 `scipy.optimize`)
- 跟踪器管理: 创建/更新/删除

2. 关键代码

```
from filterpy.kalman import KalmanFilter
from scipy.optimize import linear_sum_assignment
import numpy as np

class TrackObject:
    def __init__(self, track_id, x, y):
        self.id = track_id
        self.kf = KalmanFilter(dim_x=4, dim_z=2)
        self.kf.F = np.array([[1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0], [0, 0, 0, 1]])
        self.kf.H = np.array([[1, 0, 0, 0], [0, 1, 0, 0]])
        self.kf.x = np.array([x, y, 0, 0])
```

```

        self.kf.P *= 100
        self.kf.Q = np.eye(4) * 0.1
        self.kf.R = np.eye(2) * 1.0
        self.lost = 0
        self.hits = 1
        self.max_lost = 10

    def predict(self):
        self.kf.predict()
        self.lost += 1
        return self.kf.x[:2]

    def update(self, x, y):
        self.kf.update([x, y])
        self.lost = 0
        self.hits += 1

class TrackerManager:
    def __init__(self):
        self.tracks = {}
        self.next_id = 1

    def update(self, detections):
        # 预测
        for t in self.tracks.values():
            t.predict()

        # 匈牙利匹配
        if not self.tracks or not detections:
            self._handle_unmatched(detections, [])
            return

        cost_matrix = self._compute_cost(detections)
        matched, unmatched_det, unmatched_trk = self._hungarian_match(cost_matrix)

        # 更新匹配的跟踪器
        for i, j in matched:
            self.tracks[list(self.tracks.keys())[j]].update(*detections[i])

        # 创建新跟踪器
        for i in unmatched_det:
            self.tracks[self.next_id] = TrackObject(self.next_id, *detections[i])
            self.next_id += 1

        # 删除丢失的跟踪器
        for j in unmatched_trk:
            tid = list(self.tracks.keys())[j]
            if self.tracks[tid].lost > self.tracks[tid].max_lost:
                del self.tracks[tid]

```

3. 运行完整系统

```

# 终端 1: YOLO 检测
python3 src/lab_code/ch28_lab/yolo_detector.py

# 终端 2: LiDAR 聚类
python3 src/lab_code/ch28_lab/lidar_cluster.py

```

```
# 终端 3: 目标跟踪
python3 src/lab_code/ch28_lab/object_tracker.py
```

预期结果

- 检测目标获得稳定的跟踪 ID
- 即使目标短暂被遮挡，跟踪 ID 保持不变
- 可视化显示每个目标的 ID 编号和轨迹
- 丢失目标超过阈值后自动删除 ID

实验结果总结

练习	核心知识点	输出话题	关键参数
28.1	YOLO 单阶段检测	`/perception/yolo/detections`	confidence=0.5
28.2	DBSCAN 点云聚类	`/perception/obstacles`	eps=0.5, min_samples=10
28.3	卡尔曼滤波+匈牙利匹配	`/perception/tracks`	max_lost=10

思考题

- 537. YOLO 检测中 confidence 阈值设置多少合适？阈值高低对结果有何影响？
- 538. DBSCAN 的 eps 参数如何根据 LiDAR 线束密度进行调节？
- 539. 卡尔曼滤波的 Q 和 R 矩阵如何影响跟踪的平滑性和响应速度？
- 540. 如何融合 YOLO 的 2D 检测结果和 LiDAR 的 3D 位置信息？
- 541. 当多个目标交叉时，匈牙利匹配可能发生 ID swap，如何改进？

第 29 章 抓取与放置编程

学习目标

- 掌握抓取姿态的计算方法
- 学会控制夹爪的开合运动
- 理解 attach/detach 物体的 API
- 掌握 Pick-and-Place 完整流程
- 通过实际案例理解抓取应用

29.1 抓取姿态计算

29.1.1 抓取姿态分析

抓取姿态是指机械臂末端执行器在抓取目标物体时的位姿。一个完整的抓取姿态需要考虑以下几个方面：

- 542. **接近方向**：从哪个方向接近目标物体
- 543. **抓取角度**：末端执行器相对于物体的姿态
- 544. **抓取高度**：末端在物体上方的高度偏移
- 545. **撤离方向**：抓取后的撤离方向

抓取姿态计算的关键公式：

抓取位姿 = 物体位姿 × 抓取偏移变换

$T_{\text{grasp}} = T_{\text{object}} \times T_{\text{offset}}$

其中 T_{offset} 是根据物体形状和夹爪类型确定的变换矩阵。

29.1.2 抓取姿态生成

```
#!/usr/bin/env python3
"""抓取姿态计算示例"""
import math
from copy import deepcopy
from geometry_msgs.msg import Pose, PoseStamped
from tf_transformations import quaternion_from_euler, euler_from_quaternion
import numpy as np

class GraspPoseCalculator:
```

```

"""抓取姿态计算器"""

@staticmethod
def compute_grasp_pose(
    object_pose,
    approach_angle=0.0,
    approach_distance=0.1,
    grasp_height_offset=0.0
):
    """
    计算抓取位姿
    参数:
        object_pose: 物体位姿 (Pose)
        approach_angle: 接近角度 (弧度)
        approach_distance: 接近距离 (米)
        grasp_height_offset: 抓取高度偏移 (米)
    返回:
        (grasp_pose, approach_pose): 抓取位姿和接近预位姿
    """
    grasp_pose = deepcopy(object_pose)

    # 根据物体位姿调整抓取高度
    grasp_pose.position.z += grasp_height_offset

    # 计算接近预位姿 (在抓取位姿上方一定距离)
    approach_pose = deepcopy(grasp_pose)
    approach_pose.position.z += approach_distance

    return grasp_pose, approach_pose

@staticmethod
def compute_top_grasp(object_pose, gripper_depth=0.05):
    """
    计算顶部抓取姿态 (从上方抓取)
    适用场景: 物体顶部平坦, 可从上方直接抓取
    """
    grasp_pose = deepcopy(object_pose)

    # 顶部抓取: 末端朝下, 姿态由物体决定
    q = quaternion_from_euler(math.pi, 0, 0)
    grasp_pose.orientation.x = q[0]
    grasp_pose.orientation.y = q[1]
    grasp_pose.orientation.z = q[2]
    grasp_pose.orientation.w = q[3]

    # 调整高度到物体上表面
    grasp_pose.position.z += gripper_depth

    # 接近预位姿 (上方 10cm)
    approach_pose = deepcopy(grasp_pose)
    approach_pose.position.z += 0.1

    # 撤离位姿 (再上升 10cm)
    retreat_pose = deepcopy(grasp_pose)

```

```

        retreat_pose.position.z += 0.15

    return grasp_pose, approach_pose, retreat_pose

@staticmethod
def compute_side_grasp(object_pose, side_direction='x'):
    """
    计算侧面抓取姿态
    适用场景：物体侧面平整，需要从侧面夹取
    """
    grasp_pose = deepcopy(object_pose)

    if side_direction == 'x':
        # 从 X 方向抓取
        q = quaternion_from_euler(0, math.pi/2, 0)
    elif side_direction == 'y':
        # 从 Y 方向抓取
        q = quaternion_from_euler(math.pi/2, 0, 0)
    else:
        q = quaternion_from_euler(0, 0, 0)

    grasp_pose.orientation.x = q[0]
    grasp_pose.orientation.y = q[1]
    grasp_pose.orientation.z = q[2]
    grasp_pose.orientation.w = q[3]

    # 接近预位姿
    approach_pose = deepcopy(grasp_pose)
    if side_direction == 'x':
        approach_pose.position.x -= 0.1
    elif side_direction == 'y':
        approach_pose.position.y -= 0.1

    retreat_pose = deepcopy(approach_pose)

    return grasp_pose, approach_pose, retreat_pose

@staticmethod
def pose_to_transform(pose):
    """将 Pose 转换为 4x4 变换矩阵"""
    from tf_transformations import quaternion_matrix
    T = quaternion_matrix([
        pose.orientation.x,
        pose.orientation.y,
        pose.orientation.z,
        pose.orientation.w
    ])
    T[0, 3] = pose.position.x
    T[1, 3] = pose.position.y
    T[2, 3] = pose.position.z
    return T

@staticmethod
def transform_to_pose(T):
    """将 4x4 变换矩阵转换为 Pose"""
    from tf_transformations import quaternion_from_matrix
    pose = Pose()

```

```

        pose.position.x = T[0, 3]
        pose.position.y = T[1, 3]
        pose.position.z = T[2, 3]
        q = quaternion_from_matrix(T)
        pose.orientation.x = q[0]
        pose.orientation.y = q[1]
        pose.orientation.z = q[2]
        pose.orientation.w = q[3]
        return pose

# 使用示例
def demo_grasp_calculation():
    # 假设物体位姿
    object_pose = Pose()
    object_pose.position.x = 0.4
    object_pose.position.y = 0.1
    object_pose.position.z = 0.15
    object_pose.orientation.w = 1.0

    # 计算顶部抓取
    calc = GraspPoseCalculator()
    grasp, approach, retreat = calc.compute_top_grasp(object_pose)

    print(f'抓取位姿: ({grasp.position.x:.3f}, {grasp.position.y:.3f}, {grasp.position.z:.3f})')
    print(f'接近位姿: ({approach.position.x:.3f}, {approach.position.y:.3f}, {approach.position.z:.3f})')
    print(f'撤离位姿: ({retreat.position.x:.3f}, {retreat.position.y:.3f}, {retreat.position.z:.3f})')

```

29.2 夹爪控制

29.2.1 夹爪控制方式

平行夹爪是最常用的末端执行器类型，通常由两个相对运动的夹指组成。

在 MoveIt2 中，夹爪通常作为一个独立的规划组进行控制：

```

#!/usr/bin/env python3
"""夹爪控制示例"""
import time
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class GripperController(Node):
    def __init__(self):
        super().__init__('gripper_controller')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

    def open_gripper(self, width=0.65):

```

```

    """打开夹爪到指定宽度"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([width, width])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)
        self.get_logger().info(f'夹爪打开: {width}')
        return True
    return False

def close_gripper(self, width=0.0):
    """关闭夹爪到指定宽度"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([width, width])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)
        self.get_logger().info(f'夹爪关闭: {width}')
        return True
    return False

def grasp_object(self, object_width=0.04):
    """抓取物体：先打开到比物体略宽，然后闭合"""
    # 打开夹爪（比物体略宽）
    self.open_gripper(object_width + 0.01)
    time.sleep(0.5)

    # 闭合夹爪
    self.close_gripper(-0.01)
    time.sleep(0.5)

def release_object(self):
    """释放物体：打开夹爪"""
    self.open_gripper(0.65)
    time.sleep(0.5)

def gripper_sequence(self):
    """夹爪动作序列演示"""
    self.get_logger().info('开始夹爪动作序列')

    # 完全打开
    self.open_gripper(0.65)
    time.sleep(0.5)

    # 半开
    self.open_gripper(0.3)
    time.sleep(0.5)

    # 完全闭合
    self.close_gripper(0.0)
    time.sleep(0.5)

    # 模拟抓取和释放
    self.grasp_object(0.05)

```

```

        time.sleep(0.5)
        self.release_object()

        self.get_logger().info('夹爪动作序列完成')

def main(args=None):
    rclpy.init(args=args)
    node = GripperController()
    node.gripper_sequence()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

29.2.2 夹爪力矩和速度控制

```

class AdvancedGripperControl(Node):
    def __init__(self):
        super().__init__('advanced_gripper_control')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

    def controlled_grasp(self, max_effort=50.0):
        """带力矩控制的抓取"""

        # 设置速度
        self.gripper.set_max_velocity_scaling_factor(0.3)
        self.gripper.set_max_acceleration_scaling_factor(0.3)

        # 缓慢闭合夹爪
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.0, 0.0])
        plan = self.gripper.plan()
        if plan:
            # 分步执行，模拟力矩控制
            trajectory = plan.trajectory
            # 只执行前半段轨迹（缓慢接近）
            mid_point = len(trajectory.points) // 2
            trajectory.points = trajectory.points[:mid_point]
            self.gripper.execute(trajectory)
            self.get_logger().info('夹爪已接触物体')
            return True
        return False

```

29.3 Attach/Detach 物体

29.3.1 物体附着

抓取物体后，需要将物体附着到末端执行器的碰撞模型中，使 MoveIt2 在规划时考虑物体的碰撞体积：

```

from moveit_msgs.msg import AttachedCollisionObject, CollisionObject
from shape_msgs.msg import SolidPrimitive

```

```

from geometry_msgs.msg import Pose

class AttachManager:
    def __init__(self, planning_scene_monitor):
        self.psm = planning_scene_monitor

    def attach_to_end_effector(
        self,
        object_id,
        link_name='gripper_link',
        object_size=[0.04, 0.04, 0.08],
        touch_links=None
    ):
        """
        将物体附着到末端执行器
        参数:
            object_id: 物体 ID
            link_name: 附着的连杆名称
            object_size: 物体尺寸 [x, y, z]
            touch_links: 允许触碰的连杆列表
        """
        if touch_links is None:
            touch_links = ['gripper_link', 'link5']

        # 创建碰撞物体
        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = link_name
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = object_size

        pose = Pose()
        pose.position.z = object_size[2] / 2 # 物体中心在夹爪前方
        pose.orientation.w = 1.0

        co.primitives = [primitive]
        co.primitive_poses = [pose]

        # 创建附着物体
        aco = AttachedCollisionObject()
        aco.link_name = link_name
        aco.object = co
        aco.touch_links = touch_links

        self.psm.process_attached_collision_object(aco)
        print(f'物体 {object_id} 已附着到 {link_name}')

    def detach_from_end_effector(self, object_id, link_name='gripper_link'):
        """
        从末端执行器分离物体
        """
        co = CollisionObject()
        co.id = object_id

```

```

co.operation = CollisionObject.REMOVE

aco = AttachedCollisionObject()
aco.link_name = link_name
aco.object = co

self.psm.process_attached_collision_object(aco)
print(f'物体 {object_id} 已从 {link_name} 分离')

def attach_cylinder(self, object_id, link_name, height, radius):
    """附着圆柱体物体"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = link_name
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.CYLINDER
    primitive.dimensions = [height, radius]

    pose = Pose()
    pose.position.z = height / 2
    pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose]

    aco = AttachedCollisionObject()
    aco.link_name = link_name
    aco.object = co
    aco.touch_links = ['gripper_link', 'link5']

    self.psm.process_attached_collision_object(aco)

```

29.4 Pick-and-Place 流程

29.4.1 完整抓取放置流程

一个标准的 Pick-and-Place 流程包含以下步骤：

546. **移动到预抓取位姿**：接近物体上方/前方
547. **移动到抓取位姿**：到达抓取位置
548. **闭合夹爪**：抓取物体
549. **附着物体**：将物体加入规划场景
550. **撤离**：上升到安全高度
551. **移动到预放置位姿**：接近放置位置上方
552. **移动到放置位姿**：到达放置位置
553. **打开夹爪**：释放物体

554. **分离物体**：从规划场景移除物体附着

555. **撤离**：回到安全位置

```
#!/usr/bin/env python3
"""Pick-and-Place 完整流程"""
import time
import math
from copy import deepcopy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose, PoseStamped
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from tf_transformations import quaternion_from_euler
from moveit.planning import MoveItPy, PlanningComponent

class PickAndPlace(Node):
    def __init__(self):
        super().__init__('pick_and_place')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

        # 配置规划参数
        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.02)
        self.arm.set_max_velocity_scaling_factor(0.5)
        self.arm.set_max_acceleration_scaling_factor(0.3)

    def go_home(self):
        """回到 home 位姿"""
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            time.sleep(0.5)

    def setup_scene(self):
        """设置规划场景"""
        # 添加桌子
        co = CollisionObject()
        co.id = 'table'
        co.header.frame_id = 'base_link'
        co.operation = CollisionObject.ADD
        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [0.8, 0.6, 0.02]
        pose = PoseStamped()
        pose.header.frame_id = 'base_link'
        pose.pose.position.z = -0.01
        pose.pose.orientation.w = 1.0
```

```

co.primitives = [primitive]
co.primitive_poses = [pose.pose]
self.psm.process_collision_object(co)
time.sleep(0.5)

def move_to_pose(self, target_pose, desc=''):
    """规划并移动到目标位姿"""
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose.pose, 'link5')
    plan = self.arm.plan()
    if plan:
        self.get_logger().info(f'{desc}: 规划成功')
        self.arm.execute(plan.trajectory)
        return True
    self.get_logger().error(f'{desc}: 规划失败')
    return False

def open_gripper(self):
    """打开夹爪"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

def close_gripper(self):
    """关闭夹爪"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.0, 0.0])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

def create_pose(self, x, y, z, roll=0, pitch=0, yaw=0):
    """创建位姿"""
    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    q = quaternion_from_euler(roll, pitch, yaw)
    pose.pose.orientation.x = q[0]
    pose.pose.orientation.y = q[1]
    pose.pose.orientation.z = q[2]
    pose.pose.orientation.w = q[3]
    return pose

def pick(self, object_pose):
    """执行抓取"""

    # 1. 打开夹爪
    self.get_logger().info('步骤 1: 打开夹爪')
    self.open_gripper()
    time.sleep(0.3)

    # 2. 移动到预抓取位姿（物体上方 0.12m）

```

```

pre_grasp_pose = deepcopy(object_pose)
pre_grasp_pose.pose.position.z += 0.12
self.get_logger().info('步骤 2: 移动到预抓取位姿')
if not self.move_to_pose(pre_grasp_pose, '预抓取'):
    return False
time.sleep(0.5)

# 3. 下降到抓取位姿
self.get_logger().info('步骤 3: 下降到抓取位姿')
if not self.move_to_pose(object_pose, '抓取'):
    return False
time.sleep(0.3)

# 4. 闭合夹爪
self.get_logger().info('步骤 4: 闭合夹爪')
self.close_gripper()
time.sleep(0.5)

# 5. 附着物体
self.get_logger().info('步骤 5: 附着物体到末端')
self.attach_object('target_object')
time.sleep(0.3)

# 6. 撤离 (上升 0.1m)
retreat_pose = deepcopy(object_pose)
retreat_pose.pose.position.z += 0.1
self.get_logger().info('步骤 6: 撤离')
if not self.move_to_pose(retreat_pose, '撤离'):
    return False
time.sleep(0.3)

return True

def place(self, place_pose):
    """执行放置"""

    # 1. 移动到预放置位姿 (放置点上方 0.1m)
    pre_place_pose = deepcopy(place_pose)
    pre_place_pose.pose.position.z += 0.1
    self.get_logger().info('步骤 7: 移动到预放置位姿')
    if not self.move_to_pose(pre_place_pose, '预放置'):
        return False
    time.sleep(0.5)

    # 2. 下降到放置位姿
    self.get_logger().info('步骤 8: 下降到放置位姿')
    if not self.move_to_pose(place_pose, '放置'):
        return False
    time.sleep(0.3)

    # 3. 打开夹爪释放物体
    self.get_logger().info('步骤 9: 打开夹爪释放物体')
    self.open_gripper()
    time.sleep(0.5)

```

```

# 4. 分离物体
self.get_logger().info('步骤 10: 分离物体')
self.detach_object('target_object')
time.sleep(0.3)

# 5. 撤离
retreat_pose = deepcopy(place_pose)
retreat_pose.pose.position.z += 0.1
self.get_logger().info('步骤 11: 撤离')
if not self.move_to_pose(retreat_pose, '撤离'):
    return False
time.sleep(0.3)

return True

def attach_object(self, object_id):
    """附着物体到末端"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = 'gripper_link'
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = [0.04, 0.04, 0.08]

    pose = Pose()
    pose.position.z = 0.04
    pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose]

    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_link'
    aco.object = co
    aco.touch_links = ['gripper_link', 'link5']

    self.psm.process_attached_collision_object(aco)

def detach_object(self, object_id):
    """从末端分离物体"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE

    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_link'
    aco.object = co

    self.psm.process_attached_collision_object(aco)

def run(self):
    self.get_logger().info('开始 Pick-and-Place 演示')

    # 设置场景
    self.setup_scene()

```

```

# 回到 home
self.go_home()

# 定义抓取位置（桌子上方）
grasp_pose = self.create_pose(0.3, 0.1, 0.15, math.pi, 0, 0)

# 定义放置位置
place_pose = self.create_pose(0.3, -0.2, 0.15, math.pi, 0, 0)

# 执行抓取
if self.pick(grasp_pose):
    self.get_logger().info('抓取成功')
else:
    self.get_logger().error('抓取失败')
    return

time.sleep(1)

# 执行放置
if self.place(place_pose):
    self.get_logger().info('放置成功')
else:
    self.get_logger().error('放置失败')

# 回到 home
self.go_home()
self.get_logger().info('Pick-and-Place 演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = PickAndPlace()
    node.run()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

29.4.2 多物体分拣

```

class MultiObjectSorter(Node):
    def __init__(self):
        super().__init__('multi_object_sorter')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')

    def sort_objects(self, object_locations, place_zones):
        """
        多物体分拣
        参数:
            object_locations: {obj_id: Pose} 物体位置字典
            place_zones: {obj_type: Pose} 放置区域字典
        """

```

```

for obj_id, obj_pose in object_locations.items():
    self.get_logger().info(f'分拣物体: {obj_id}')

    # 抓取物体
    self.pick(obj_pose)

    # 确定放置位置
    place_pose = place_zones.get('default')
    if 'red' in obj_id:
        place_pose = place_zones.get('red_zone')
    elif 'blue' in obj_id:
        place_pose = place_zones.get('blue_zone')

    # 放置物体
    self.place(place_pose)

    self.get_logger().info(f'{obj_id} 分拣完成')

```

29.5 实际案例

29.5.1 传送带抓取

模拟传送带上物体的抓取：

```

class ConveyorBeltGrasp(Node):
    def __init__(self):
        super().__init__('conveyor_belt_grasp')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')

        # 订阅物体检测结果
        self.detection_sub = self.create_subscription(
            PoseStamped, '/detected_object_pose',
            self.detection_callback, 10
        )

    def detection_callback(self, msg):
        """收到检测结果，触发抓取"""

        self.get_logger().info(f'检测到物体: ({msg.pose.position.x:.2f},
{msg.pose.position.y:.2f})')

        # 计算抓取位姿
        grasp = deepcopy(msg)
        approach = deepcopy(msg)
        approach.pose.position.z += 0.12

        # 执行抓取
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(approach.pose, 'link5')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

```

```

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(grasp.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

# 闭合夹爪
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan:
    self.gripper.execute(plan.trajectory)

# 撤离
retreat = deepcopy(grasp)
retreat.pose.position.z += 0.15
self.arm.set_pose_target(retreat.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

# 移动到放置位置
place = PoseStamped()
place.header.frame_id = 'base_link'
place.pose.position.x = 0.3
place.pose.position.y = -0.2
place.pose.position.z = 0.2
place.pose.orientation.w = 1.0
self.arm.set_pose_target(place.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

# 释放
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan:
    self.gripper.execute(plan.trajectory)

```

29.5.2 码垛应用

```

class PalletizingApp(Node):
    """码垛应用：将物体堆叠到指定位置"""

    def __init__(self):
        super().__init__('palletizing_app')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')
        self.psm = self.moveit.get_planning_scene_monitor()

        self.layer = 0
        self.pos_in_layer = 0
        self.layer_positions = [
            [(0.2, -0.1), (0.2, 0.0), (0.2, 0.1)],
            [(0.25, -0.1), (0.25, 0.0), (0.25, 0.1)],
        ]

```

```

def get_next_place_pose(self):
    """获取下一个放置位置（码垛模式）"""
    if self.pos_in_layer >= len(self.layer_positions[self.layer]):
        self.pos_in_layer = 0
        self.layer += 1

    if self.layer >= len(self.layer_positions):
        self.get_logger().warn('码垛已满')
        return None

    x, y = self.layer_positions[self.layer][self.pos_in_layer]
    z = 0.15 + self.layer * 0.1

    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    self.pos_in_layer += 1
    return pose

```

课后练习

556. 编写程序计算顶部抓取和侧面抓取的位姿，分析两种抓取方式适用的物体形状。
557. 实现 Pick-and-Place 完整流程：从 A 点抓取物体，移动到 B 点放置，最后回到 home 位姿。
558. 编写多物体分拣程序：3 个不同位置的物体分别抓取并放置到 3 个对应的目标位置。
559. 实现码垛功能：将多个物体依次堆叠到指定位置，每层高度递增。
560. 编写程序模拟传送带抓取：订阅物体位姿话题，当检测到物体时自动执行抓取和放置。

实验 29:

第 29 章 实验手册：行为决策与交通规则

****对应理论章节**:** 第 43 章《行为决策与交通规则》 ****实验课时**:** 2 课时 ****实验代码**:** ``src/lab_code/ch29_lab/``

实验目标

- 561. 掌握 CARLA 交通灯 API 的使用方法，实现交通灯检测与停止
- 562. 实现基于有限状态机 (FSM) 的行为决策节点
- 563. 掌握 Traffic Manager 的配置方法，实现多车交通流

前置条件

- CARLA 0.9.16 已安装运行
- ROS 2 Jazzy 已配置
- carla-ros-bridge 已编译
- Python 3.8+ with carla pip package

练习 29.1: 交通灯检测与停止

目标

编写一个 ROS2 节点，订阅 CARLA 的车辆状态和交通灯信息，当检测到红灯时控制车辆在停止线前停车。

实现步骤

步骤 1: 创建 ROS2 包

```
cd ~/ros2_course_ws/src
ros2 pkg create ch29_traffic_light \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs carla_msgs
```

步骤 2: 编写交通灯检测节点

创建 ch29_traffic_light/ch29_traffic_light/traffic_light_detector.py:

```
import rclpy
from rclpy.node import Node
from carla_msgs.msg import CarlaTrafficLightStatusList, CarlaEgoVehicleControl
from geometry_msgs.msg import Point
import math

class TrafficLightDetector(Node):
    def __init__(self):
        super().__init__('traffic_light_detector')

        # 订阅交通灯状态
        self.tl_sub = self.create_subscription(
            CarlaTrafficLightStatusList,
            '/carla/traffic_light_status',
            self.traffic_light_callback,
            10
        )

        # 订阅自车位置 (通过 CARLA 的 odometry)
        self.ego_sub = self.create_subscription(
            Point, # 实际使用 Odometry
            '/carla/ego_vehicle/odometry',
            self.ego_position_callback,
            10
        )

        # 发布控制指令
        self.cmd_pub = self.create_publisher(
            CarlaEgoVehicleControl,
            '/carla/ego_vehicle/vehicle_control_cmd',
            10
        )

        self.current_pose = None
        self.traffic_lights = []
        self.stop_signals = []

        # 控制定时器 (20Hz)
        self.timer = self.create_timer(0.05, self.control_loop)

        self.get_logger().info('TrafficLightDetector 初始化完成')
```

```

def traffic_light_callback(self, msg):
    """处理交通灯状态"""
    self.traffic_lights = msg.traffic_lights

def ego_position_callback(self, msg):
    """更新自车位置"""
    self.current_pose = msg

def get_distance_to_stop_line(self, tl_status):
    """计算车辆到停止线的距离"""

    # 实际实现需要根据交通灯位置和自车位置计算
    # 这里简化为读取交通灯关联的停止线位置
    if self.current_pose is None:
        return float('inf')

    # 假设 tl_status 包含 stop_line 位置信息
    stop_x = tl_status.stop_line_location.x
    stop_y = tl_status.stop_line_location.y
    dx = self.current_pose.x - stop_x
    dy = self.current_pose.y - stop_y
    return math.sqrt(dx*dx + dy*dy)

def should_stop(self, tl_status):
    """判断是否需要停止"""
    STOP_DISTANCE = 12.0
    MARGIN = 3.0

    # 红灯或黄灯
    if tl_status.state not in [1, 2]: # 1=Red, 2=Yellow
        return False, 0.0

    distance = self.get_distance_to_stop_line(tl_status)
    if distance < MARGIN:
        return True, distance
    if distance < STOP_DISTANCE:
        return True, distance
    return False, distance

def control_loop(self):
    """控制主循环：判断交通灯并输出控制指令"""

    cmd = CarlaEgoVehicleControl()
    cmd.throttle = 0.3
    cmd.brake = 0.0
    cmd.steer = 0.0
    cmd.hand_brake = False
    cmd.reverse = False
    cmd.gear = 1
    cmd.manual_gear_shift = False

    for tl in self.traffic_lights:
        stop, distance = self.should_stop(tl)
        if stop:
            self.get_logger().warn(
                f'红灯! 距离停止线: {distance:.2f}m'
            )

```

```

        cmd.throttle = 0.0
        # 根据距离线性刹车
        if distance < 5.0:
            cmd.brake = 0.8
        else:
            cmd.brake = 0.3
        break

    self.cmd_pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = TrafficLightDetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤3: 配置 *setup.py*

```

entry_points={
    'console_scripts': [
        'traffic_light_detector = ch29_traffic_light.traffic_light_detector:main',
    ],
},

```

步骤4: 编译与测试

```

cd ~/ros2_course_ws
colcon build --packages-select ch29_traffic_light
source install/setup.bash

# 启动 CARLA 和 ros-bridge
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py

# 运行节点
ros2 run ch29_traffic_light traffic_light_detector

```

预期结果

- 车辆在红灯前自动减速停止
- 绿灯亮起后自动恢复行驶
- 终端输出"红灯! 距离停止线: X.XXm"日志

思考题

564. 如何区分本车道和相邻车道的红绿灯?
565. 黄灯闪烁时应如何处理?
566. 多车道大路口可能有多个交通灯, 如何选择相关的那个?

练习 29.2: 设计 FSM 行为决策节点

目标

实现一个基于有限状态机的行为决策节点，包含巡航、跟车、停止三种基本状态。

实现步骤

步骤 1: 创建包

```
cd ~/ros2_course_ws/src
ros2 pkg create ch29_fsm_decision \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs carla_msgs
```

步骤 2: 编写 FSM 决策节点

创建 ch29_fsm_decision/ch29_fsm_decision/fsm_decision.py:

```
import rclpy
from rclpy.node import Node
from enum import Enum
from carla_msgs.msg import (
    CarlaEgoVehicleControl,
    CarlaEgoVehicleStatus,
    CarlaTrafficLightStatusList
)
from std_msgs.msg import Float32
import math

class FsmState(Enum):
    INIT = 0
    CRUISE = 1
    FOLLOW = 2
    STOP = 3

class FSMDecisionNode(Node):
    def __init__(self):
        super().__init__('fsm_decision_node')

        # 状态定义
        self.state = FsmState.INIT
        self.state_names = {
            FsmState.INIT: 'INIT',
            FsmState.CRUISE: 'CRUISE',
            FsmState.FOLLOW: 'FOLLOW',
            FsmState.STOP: 'STOP',
        }

        # 参数
```

```

self.SAFE_DISTANCE = 15.0 # 安全跟车距离 (m)
self.CRUISE_SPEED = 10.0 # 巡航速度 (m/s)
self.MIN_SPEED = 0.0

# 订阅
self.status_sub = self.create_subscription(
    CarlaEgoVehicleStatus,
    '/carla/ego_vehicle/vehicle_status',
    self.status_callback,
    10
)
self.lead_dist_sub = self.create_subscription(
    Float32,
    '/carla/ego_vehicle/lead_distance',
    self.lead_distance_callback,
    10
)
self.tl_sub = self.create_subscription(
    CarlaTrafficLightStatusList,
    '/carla/traffic_light_status',
    self.traffic_light_callback,
    10
)

# 发布
self.cmd_pub = self.create_publisher(
    CarlaEgoVehicleControl,
    '/carla/ego_vehicle/vehicle_control_cmd',
    10
)
self.state_pub = self.create_publisher(
    Int32, '/carla/ego_vehicle/fsm_state', 10
)

# 状态
self.current_speed = 0.0
self.lead_distance = float('inf')
self.have_red_light = False

# 控制定时器
self.timer = self.create_timer(0.1, self.update)

self.get_logger().info('FSM 决策节点启动, 初始状态: CRUISE')
self.state = FsmState.CRUISE

def status_callback(self, msg):
    self.current_speed = math.sqrt(
        msg.velocity.x**2 + msg.velocity.y**2
    )

def lead_distance_callback(self, msg):
    self.lead_distance = msg.data

def traffic_light_callback(self, msg):
    self.have_red_light = any(
        tl.state == 1 for tl in msg.traffic_lights
    )

```

```

def update(self):
    """定时执行状态机转移"""
    self.get_logger().debug(
        f'状态: {self.state_names[self.state]}, '
        f'速度: {self.current_speed:.1f}, '
        f'前车距离: {self.lead_distance:.1f}'
    )

    if self.state == FsmState.CRUISE:
        cmd = self._handle_cruise()
    elif self.state == FsmState.FOLLOW:
        cmd = self._handle_follow()
    elif self.state == FsmState.STOP:
        cmd = self._handle_stop()
    else:
        cmd = CarlaEgoVehicleControl()

    self.cmd_pub.publish(cmd)
    self.state_pub.publish(Int32(data=self.state.value))

def _handle_cruise(self):
    cmd = CarlaEgoVehicleControl()
    cmd.throttle = 0.4
    cmd.brake = 0.0

    # 检测前车 → 切换到 FOLLOW
    if self.lead_distance < self.SAFE_DISTANCE:
        self._transition(FsmState.FOLLOW)
        return self._handle_follow()

    # 检测红灯 → 切换到 STOP
    if self.have_red_light and self.lead_distance < 30.0:
        self._transition(FsmState.STOP)
        return self._handle_stop()

    # 速度控制
    if self.current_speed < self.CRUISE_SPEED:
        cmd.throttle = 0.5
    else:
        cmd.throttle = 0.2

    return cmd

def _handle_follow(self):
    cmd = CarlaEgoVehicleControl()

    # 前车消失 → CRUISE
    if self.lead_distance > self.SAFE_DISTANCE * 1.5:
        self._transition(FsmState.CRUISE)
        return self._handle_cruise()

    # 前车距离太近 → 刹车
    if self.lead_distance < 5.0:
        cmd.throttle = 0.0
        cmd.brake = 0.6

```

```

        elif self.lead_distance < self.SAFE_DISTANCE:
            cmd.throttle = 0.2
            cmd.brake = 0.1
        else:
            cmd.throttle = 0.3
            cmd.brake = 0.0

        # 目标速度不超过前车速度
        target_speed = min(self.CRUISE_SPEED,
                           self.lead_distance / 2.0)
        if self.current_speed > target_speed:
            cmd.throttle = 0.0
            cmd.brake = 0.3

        return cmd

def _handle_stop(self):
    cmd = CarlaEgoVehicleControl()
    cmd.throttle = 0.0

    # 红灯消失 → CRUISE
    if not self.have_red_light:
        self._transition(FsmState.CRUISE)
        return self._handle_cruise()

    # 停车
    if self.current_speed > 1.0:
        cmd.brake = 0.7
    else:
        cmd.brake = 1.0 # 保持刹车

    return cmd

def _transition(self, new_state):
    old_state = self.state
    self.state = new_state
    self.get_logger().info(
        f'状态转移: {self.state_names[old_state]} → '
        f'{self.state_names[new_state]}'
    )

def main(args=None):
    rclpy.init(args=args)
    node = FSMDecisionNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤3: 前车距离计算工具节点

创建 `ch29_fsm_decision/ch29_fsm_decision/lead_distance_calc.py`:


```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from carla_msgs.msg import CarlaEgoVehicleStatus
from derived_object_msgs.msg import ObjectArray
import math

class LeadDistanceCalculator(Node):
    def __init__(self):
        super().__init__('lead_distance_calculator')
        self.pub = self.create_publisher(
            Float32, '/carla/ego_vehicle/lead_distance', 10
        )
        self.ego_sub = self.create_subscription(
            CarlaEgoVehicleStatus, '/carla/ego_vehicle/vehicle_status',
            self.status_cb, 10
        )
        self.obj_sub = self.create_subscription(
            ObjectArray, '/carla/objects',
            self.objects_cb, 10
        )
        self.ego_x = self.ego_y = 0.0
        self.ego_yaw = 0.0
        self.timer = self.create_timer(0.1, self.publish_distance)

    def status_cb(self, msg):
        # 从 CarlaEgoVehicleStatus 提取位置和朝向
        pass # 实际实现需要提取自车位置

    def objects_cb(self, msg):
        self.objects = msg.objects

    def publish_distance(self):
        # 计算最近的前车距离
        min_dist = float('inf')
        for obj in self.objects:
            if obj.classification == 'car':
                dx = obj.pose.position.x - self.ego_x
                dy = obj.pose.position.y - self.ego_y
                dist = math.sqrt(dx*dx + dy*dy)
                # 判断是否在同一车道、在前方
                if dist < min_dist:
                    min_dist = dist
        if min_dist < 100.0:
            self.pub.publish(Float32(data=min_dist))

```

步骤 4: 编译与测试

```

cd ~/ros2_course_ws
colcon build --packages-select ch29_fsm_decision
source install/setup.bash

# 启动
ros2 run ch29_fsm_decision fsm_decision_node

```

预期结果

- 无前车时保持巡航速度行驶
- 检测到前车进入跟车模式，保持安全距离
- 红灯前减速停止
- 终端显示状态转移日志

思考题

567. 如何处理多条车道的前车选择问题？
568. 如何增加变道状态？变道的触发条件是什么？
569. FSM 状态数量膨胀时如何管理？

练习 29.3: 配置 Traffic Manager 实现交通流

目标

通过 CARLA Traffic Manager 控制多辆 AI 车辆，配置不同的驾驶风格，并与自车协调形成交通流。

实现步骤

步骤 1: 创建 Python 脚本

创建 ch29_traffic_flow.py（独立脚本，非 ROS2 节点）：

```
#!/usr/bin/env python3

import carla
import random
import time
import argparse

def spawn_vehicles(world, blueprint_library, spawn_points, num):
    """在随机出生点生成车辆"""
    vehicles = []
    vehicle_bps = blueprint_library.filter('vehicle.*')
    spawn_points = random.sample(spawn_points, num)

    for i, sp in enumerate(spawn_points):
        bp = random.choice(vehicle_bps)
        bp.set_attribute('role_name', f'background_{i}')
```

```

        vehicle = world.spawn_actor(bp, sp)
        if vehicle:
            vehicles.append(vehicle)
            print(f'生成车辆 {i+1}/{num}: {bp.id}')

    return vehicles

def configure_traffic_manager(client, vehicles, mode='normal'):
    """配置 Traffic Manager 参数"""
    tm = client.get_trafficmanager(8000)

    # 全局设置
    tm.set_global_distance_to_leading_vehicle(3.0)
    tm.set_random_device_seed(42)
    tm.set_synchronous_mode(True)

    # 每辆车单独配置
    for i, vehicle in enumerate(vehicles):
        vehicle.set_autopilot(True, tm.get_port())

        if mode == 'aggressive':
            # 激进模式
            tm.set_desired_speed(vehicle, random.uniform(60, 90))
            tm.set_distance_to_leading_vehicle(
                vehicle, random.uniform(1.0, 2.5))
            tm.set_ignore_traffic_light_percentage(vehicle, 20)
            tm.set_ignore_signs_percentage(vehicle, 15)
            tm.set_lane_change_behavior(
                vehicle, random.choice([0, 1, 2]))
            tm.set_force_lane_change(vehicle, True)

        elif mode == 'conservative':
            # 保守模式
            tm.set_desired_speed(vehicle, random.uniform(30, 50))
            tm.set_distance_to_leading_vehicle(
                vehicle, random.uniform(5.0, 10.0))
            tm.set_ignore_traffic_light_percentage(vehicle, 0)
            tm.set_ignore_signs_percentage(vehicle, 0)
            tm.set_lane_change_behavior(vehicle, 0)

        else: # normal
            # 普通模式
            tm.set_desired_speed(vehicle, random.uniform(40, 70))
            tm.set_distance_to_leading_vehicle(
                vehicle, random.uniform(3.0, 6.0))
            tm.set_ignore_traffic_light_percentage(vehicle, 5)
            tm.set_ignore_signs_percentage(vehicle, 2)
            tm.set_lane_change_behavior(
                vehicle, random.choice([0, 1]))

        print(f'已配置车辆 {i+1} ({mode})')

    return tm

def main():

```

```

parser = argparse.ArgumentParser(
    description='CARLA Traffic Manager Demo')
parser.add_argument('--host', type=str, default='localhost')
parser.add_argument('--port', type=int, default=2000)
parser.add_argument('--tm-port', type=int, default=8000)
parser.add_argument('--vehicles', type=int, default=20)
parser.add_argument('--mode', type=str, default='normal',
                    choices=['normal', 'aggressive', 'conservative',
                              'mixed'])
parser.add_argument('--sync', action='store_true',
                    default=True)
args = parser.parse_args()

# 连接 CARLA
client = carla.Client(args.host, args.port)
client.set_timeout(10.0)
world = client.get_world()

# 获取出生点
spawn_points = world.get_map().get_spawn_points()
if len(spawn_points) < args.vehicles:
    print(f'警告: 只有 {len(spawn_points)} 个出生点')
    args.vehicles = len(spawn_points)

# 清理现有车辆
for actor in world.get_actors().filter('vehicle.*'):
    actor.destroy()

# 生成车辆
blueprint_library = world.get_blueprint_library()
vehicles = spawn_vehicles(
    world, blueprint_library, spawn_points, args.vehicles
)
print(f'成功生成 {len(vehicles)} 辆车')

# 配置 Traffic Manager
if args.mode == 'mixed':
    # 混合模式: 随机分配驾驶风格
    modes = ['aggressive', 'normal', 'conservative']
    for vehicle in vehicles:
        mode = random.choice(modes)
        configure_traffic_manager(
            client, [vehicle], mode=mode)
else:
    configure_traffic_manager(
        client, vehicles, mode=args.mode)

# 主循环
try:
    if args.sync:
        settings = world.get_settings()
        settings.synchronous_mode = True
        settings.fixed_delta_seconds = 0.05
        world.apply_settings(settings)

    tm = client.get_trafficmanager(args.tm_port)

```

```

frame = 0

while True:
    if args.sync:
        world.tick()
    else:
        time.sleep(0.05)

    frame += 1
    if frame % 200 == 0: # 每 10 秒输出统计
        speeds = []
        for v in vehicles:
            vels = v.get_velocity()
            speed = 3.6 * math.sqrt(
                vels.x**2 + vels.y**2 + vels.z**2
            )
            speeds.append(speed)
        avg_speed = sum(speeds) / len(speeds)
        max_speed = max(speeds)
        min_speed = min(speeds)
        print(
            f'交通流统计 | 平均: {avg_speed:.1f} km/h '
            f'| 最大: {max_speed:.1f} km/h '
            f'| 最小: {min_speed:.1f} km/h'
        )

except KeyboardInterrupt:
    print('用户中断')
finally:
    print('清理车辆...')
    for vehicle in vehicles:
        vehicle.destroy()
    print('完成')

if __name__ == '__main__':
    import math
    main()

```

步骤 2: 编写 ROS2 封装节点 (可选)

如果需要将 Traffic Manager 集成到 ROS2 中:

```

# ch29_traffic_flow_ros.py
import rclpy
from rclpy.node import Node
import carla

class TrafficFlowManager(Node):
    def __init__(self):
        super().__init__('traffic_flow_manager')

        self.declare_parameter('host', 'localhost')
        self.declare_parameter('port', 2000)
        self.declare_parameter('num_vehicles', 20)
        self.declare_parameter('mode', 'normal')

```

```

        host = self.get_parameter('host').value
        port = self.get_parameter('port').value
        num = self.get_parameter('num_vehicles').value
        mode = self.get_parameter('mode').value

        self.client = carla.Client(host, port)
        self.client.set_timeout(10.0)
        self.world = self.client.get_world()
        self.tm = self.client.get_trafficmanager(8000)

        self.vehicles = []
        self._spawn_and_configure(num, mode)

        self.timer = self.create_timer(0.1, self.update_stats)

    def _spawn_and_configure(self, num, mode):
        # ... 生成和配置逻辑同上 ...
        pass

    def update_stats(self):
        # 发布交通流统计信息
        pass

```

步骤3: 运行

```

# 基本运行 - 20 辆车, 普通模式
python ch29_traffic_flow.py --vehicles 20 --mode normal

# 激进模式 - 30 辆车, 闯灯概率 20%
python ch29_traffic_flow.py --vehicles 30 --mode aggressive

# 混合模式 - 不同车辆不同驾驶风格
python ch29_traffic_flow.py --vehicles 50 --mode mixed

# 同步模式
python ch29_traffic_flow.py --sync --vehicles 20 --mode normal

```

预期结果

- 多辆 AI 车辆在交通流中自主行驶
- 不同类型车辆表现出不同的驾驶风格
- 终端每 10 秒输出交通流统计

思考题

570. Traffic Manager 和自定义 FSM 决策的区别与适用场景?
571. 如何通过 TM 参数模拟高峰期的拥堵路况?
572. 混合模式中, 激进车辆和保守车辆相遇时会发生什么?

附加挑战

挑战 1: FSM + 交通灯集成

将练习 29.1 和 29.2 合并，在 FSM 中增加 `CHANGE_LEFT` 和 `CHANGE_RIGHT` 状态。实现自动变道超车逻辑。

挑战 2: 行人避让

在 Scenario Runner 中定义"行人横穿"场景，让自车在感知到行人时从 `CRUISE` 切换到 `AVOID` 状态。

挑战 3: 交通流 + 自车

在练习 29.3 的 Traffic Manager 交通流中，加入 ROS2 FSM 控制的自车。评估自车在不同交通密度和驾驶风格下的表现。

提交要求

项目	要求
代码	注释完整，符合 ROS2 编码规范
实验报告	包含代码结构、实验结果截图、问题分析
视频(可选)	录制车辆行为决策过程演示

参考资源

- [CARLA Traffic Manager 文档](#)
- [Scenario Runner 文档](#)
- [ROS2 CARLA Bridge 源码](#)

第 30 章 ROS2 图像接口与相机标定

学习目标

- 掌握 sensor_msgs/Image 消息结构
- 学会使用 cv_bridge 进行 ROS2 图像与 OpenCV 格式互转
- 理解 image_transport 图像传输机制
- 掌握 CameraInfo 消息和相机内参
- 学会 usb_cam 驱动使用
- 掌握相机标定的原理和实践方法

30.1 ROS2 图像消息

30.1.1 Image 消息结构

ROS2 中使用 sensor_msgs/Image 作为标准图像消息格式：

```
sensor_msgs/Image
- header          # 消息头 (stamp, frame_id)
- height          # 图像高度 (像素)
- width           # 图像宽度 (像素)
- encoding         # 编码格式 (rgb8, bgr8, mono8...)
- is_bigendian    # 字节序
- step            # 行字节数 (width * bytes_per_pixel)
- data            # 原始图像数据 (uint8 数组)
```

encoding 字段决定了图像数据的解释方式：

encoding 值	OpenCV 类型	说明
mono8	CV_8UC1	8 位灰度图
mono16	CV_16UC1	16 位灰度图
bgr8	CV_8UC3	BGR 彩色图 (OpenCV 默认)
rgb8	CV_8UC3	RGB 彩色图
bgra8	CV_8UC4	BGR 带 Alpha 通道

rgba8	CV_8UC4	RGB 带 Alpha 通道
32FC1	CV_32FC1	32 位浮点深度图

30.1.2 CompressedImage 消息

对于高分辨率图像，使用压缩消息可以减少带宽占用：

```
sensor_msgs/CompressedImage
- header
- format          # 压缩格式 (jpeg, png, bmp)
- data            # 压缩数据
```

30.1.3 CameraInfo 消息

相机内参通过/camera_info 话题发布：

```
sensor_msgs/CameraInfo
- header
- height, width      # 图像分辨率
- distortion_model    # 畸变模型 (plumb_bob, rational...)
- d                  # 畸变系数 [k1, k2, p1, p2, k3]
- k                  # 内参矩阵 [fx, 0, cx; 0, fy, cy; 0, 0, 1]
- r                  # 校正矩阵 (3x3)
- p                  # 投影矩阵 (3x4)
- binning_x, binning_y # 合并像素
- roi                # 感兴趣区域

# 读取 CameraInfo 示例
from sensor_msgs.msg import CameraInfo

class CameraInfoReader(Node):
    def __init__(self):
        super().__init__('camera_info_reader')
        self.sub = self.create_subscription(
            CameraInfo, '/camera/color/camera_info',
            self.info_callback, 10
        )

    def info_callback(self, msg):
        self.get_logger().info(f'分辨率: {msg.width}x{msg.height}')
        self.get_logger().info(f'内参矩阵: {msg.k}')
        self.get_logger().info(f'畸变系数: {msg.d}')
        self.get_logger().info(f'投影矩阵: {msg.p}')
```

30.2 cv_bridge 图像转换

30.2.1 CvBridge 核心用法

cv_bridge 提供了 ROS2 Image ↔ OpenCV cv::Mat 的双向转换:

```
ROS2 Image —imgmsg_to_cv2()→ OpenCV Mat
OpenCV Mat —cv2_to_imgmsg()→ ROS2 Image

#!/usr/bin/env python3
"""cv_bridge 基本用法示例"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import cv2

class CvBridgeDemo(Node):
    def __init__(self):
        super().__init__('cv_bridge_demo')
        self.bridge = CvBridge()

        # 订阅原始图像
        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )
        # 发布处理后的图像
        self.image_pub = self.create_publisher(
            Image, '/image_processed', 10
        )

    def image_callback(self, data):
        try:
            # ROS2 Image → OpenCV Mat
            cv_image = self.bridge.imgmsg_to_cv2(data, 'bgr8')
        except CvBridgeError as e:
            self.get_logger().error(str(e))
            return

        # OpenCV 处理: 转为灰度图
        gray = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)

        # 显示图像
        cv2.imshow('CV Bridge Demo', cv_image)
        cv2.waitKey(3)

        try:
            # OpenCV Mat → ROS2 Image
            msg = self.bridge.cv2_to_imgmsg(gray, 'mono8')
            msg.header.stamp = self.get_clock().now().to_msg()
            msg.header.frame_id = 'camera_link'
            self.image_pub.publish(msg)
        except CvBridgeError as e:
            self.get_logger().error(str(e))
```

```
def main(args=None):
    rclpy.init(args=args)
    node = CvBridgeDemo()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

30.2.2 编码格式对应表

```
class EncodingConverter:
    """编码格式转换工具"""

    @staticmethod
    def ros_to_opencv(ros_encoding):
        """ROS2 编码 → OpenCV 编码"""
        encoding_map = {
            'mono8': 'CV_8UC1',
            'mono16': 'CV_16UC1',
            'bgr8': 'CV_8UC3',
            'rgb8': 'CV_8UC3',
            'bgra8': 'CV_8UC4',
            'rgba8': 'CV_8UC4',
            '32fc1': 'CV_32FC1',
        }
        return encoding_map.get(ros_encoding, 'CV_8UC3')

    @staticmethod
    def get_channels(ros_encoding):
        """获取图像通道数"""
        if 'mono' in ros_encoding or '32fc' in ros_encoding:
            return 1
        elif 'bgr8' in ros_encoding or 'rgb8' in ros_encoding:
            return 3
        elif 'bgra8' in ros_encoding or 'rgba8' in ros_encoding:
            return 4
        return 3
```

30.2.3 图像发布节点

```
#!/usr/bin/env python3
"""图像发布节点 - 从摄像头读取并发布为 ROS2 话题"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class ImagePublisher(Node):
    def __init__(self):
        super().__init__('image_publisher')
```

```

self.pub = self.create_publisher(Image, '/camera/image_raw', 10)
self.bridge = CvBridge()

# 打开摄像头
self.cap = cv2.VideoCapture(0)
if not self.cap.isOpened():
    self.get_logger().error('无法打开摄像头')
    raise RuntimeError('摄像头打开失败')

# 设置摄像头参数
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
self.cap.set(cv2.CAP_PROP_FPS, 30)

# 定时发布 (约 30fps)
self.timer = self.create_timer(0.033, self.timer_callback)
self.get_logger().info('图像发布节点已启动')

def timer_callback(self):
    ret, frame = self.cap.read()
    if not ret:
        self.get_logger().warn('读取图像失败')
        return

    # OpenCV Mat → ROS2 Image
    msg = self.bridge.cv2_to_imgmsg(frame, 'bgr8')
    msg.header.stamp = self.get_clock().now().to_msg()
    msg.header.frame_id = 'camera_link'
    self.pub.publish(msg)

def __del__(self):
    if hasattr(self, 'cap'):
        self.cap.release()

def main(args=None):
    rclpy.init(args=args)
    node = ImagePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.3 image_transport

30.3.1 image_transport 基础

ROS2 中 image_transport 提供了图像发布与订阅的抽象层，支持原始图像和压缩图像的自动切换：

```
# 常用 transport 插件
ros-jazzy-image-transport      # 基础传输
ros-jazzy-compressed-image-transport # 压缩图像传输
ros-jazzy-theora-image-transport # 视频流传输
```

30.3.2 使用 image_transport 发布图像

```
from image_transport import ImageTransport

class ImageTransportPublisher(Node):
    def __init__(self):
        super().__init__('image_transport_publisher')
        self.bridge = CvBridge()

        # 创建 ImageTransport 实例
        self.it = ImageTransport(self)

        # 创建发布者（支持 raw 和 compressed）
        self.pub = self.it.advertise('/camera/image', 10)

        self.cap = cv2.VideoCapture(0)
        self.timer = self.create_timer(0.033, self.publish_image)

    def publish_image(self):
        ret, frame = self.cap.read()
        if not ret:
            return
        msg = self.bridge.cv2_to_imgmsg(frame, 'bgr8')
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'camera_link'
        self.pub.publish(msg)
```

30.3.3 使用 image_transport 订阅图像

```
from image_transport import ImageTransport

class ImageTransportSubscriber(Node):
    def __init__(self):
        super().__init__('image_transport_subscriber')
        self.bridge = CvBridge()

        # 创建 ImageTransport 实例
        self.it = ImageTransport(self)

        # 订阅图像（自动处理 raw 和 compressed）
        self.sub = self.it.subscribe(
            '/camera/image', self.image_callback, 'raw'
        )

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
            cv2.imshow('Image Transport', cv_image)
            cv2.waitKey(1)
        except Exception as e:
            self.get_logger().error(str(e))
```

30.4 usb_cam 驱动

30.4.1 安装 usb_cam

```
# 安装 usb_cam 功能包
sudo apt install ros-jazzy-usb-cam

# 或从源码编译
cd ~/ros2_course_ws/src
git clone https://github.com/ros-drivers/usb_cam.git
cd ..
colcon build --packages-select usb_cam
```

30.4.2 配置和启动 usb_cam

```
# 启动 usb_cam 节点
ros2 run usb_cam usb_cam_node_exe

# 查看发布的话题
ros2 topic list
# /image_raw
# /camera_info
# /parameter_events
# /rosout

# 查看图像
ros2 run image_tools showimage --ros-args -r image:=/image_raw
```

30.4.3 usb_cam 参数配置

```
<!-- usb_cam_params.yaml -->
/**:
  ros__parameters:
    video_device: "/dev/video0"
    framerate: 30.0
    io_method: "mmap"
    frame_id: "camera_link"
    pixel_format: "yuyv"
    image_width: 640
    image_height: 480
    camera_name: "usb_cam"
    brightness: -1
    contrast: -1
    saturation: -1
    sharpness: -1
```

启动带参数的 usb_cam:

```
# 从参数文件加载
ros2 run usb_cam usb_cam_node_exe \
  --ros-args --params-file usb_cam_params.yaml

# 命令行参数
ros2 run usb_cam usb_cam_node_exe \
```

```
--ros-args -p video_device:=/dev/video1 \  
-p image_width:=1280 -p image_height:=720
```

30.4.4 多摄像头支持

```
# 启动第二个摄像头（不同设备节点）  
ros2 run usb_cam usb_cam_node_exe \  
--ros-args --remap __node:=usb_cam2 \  
-p video_device:=/dev/video2 \  
--remap /image_raw:=/camera2/image_raw \  
--remap /camera_info:=/camera2/camera_info
```

30.5 相机标定原理

30.5.1 标定原理

相机将三维世界坐标映射到像素平面，内参矩阵 K 描述了这一映射关系：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

其中：

- f_x, f_y ：焦距（像素单位）
- c_x, c_y ：光心坐标（像素）

畸变参数包括：

- 径向畸变： k_1, k_2, k_3 （桶形/枕形畸变）
- 切向畸变： p_1, p_2 （镜头与传感器不平行）

畸变校正公式：

$$\begin{aligned} x_{\text{corrected}} &= x * (1 + k_1*r^2 + k_2*r^4 + k_3*r^6) + 2*p_1*xy + p_2*(r^2 + 2x^2) \\ y_{\text{corrected}} &= y * (1 + k_1*r^2 + k_2*r^4 + k_3*r^6) + p_1*(r^2 + 2y^2) + 2*p_2*xy \end{aligned}$$

30.5.2 棋盘格标定板

标定板是相机标定的关键工具。常用标定图案：

573. **棋盘格 (Chessboard)**：黑白交替方格
574. **对称圆点图案**：规则排列的圆形
575. **非对称圆点图案**：用于高精度标定

生成标定板：

```
# 安装标定板生成工具
sudo apt install ros-jazzy-camera-calibration-parsers

# 使用 OpenCV 生成标定板
python3 -c "
import cv2
import numpy as np

# 生成 8x6 棋盘格（每个格子 24.5mm）
board = np.zeros((6*200, 8*200, 3), dtype=np.uint8)
for i in range(6):
    for j in range(8):
        if (i + j) % 2 == 0:
            board[i*200:(i+1)*200, j*200:(j+1)*200] = 255

cv2.imwrite('calibration_board.png', board)
cv2.imshow('标定板', board)
cv2.waitKey(0)
cv2.destroyAllWindows()
"
```

30.5.3 camera_calibration 使用

ROS 2 Jazzy 中安装和使用 camera_calibration：

```
# 安装
sudo apt install ros-jazzy-camera-calibration

# 启动相机后运行标定
ros2 run camera_calibration cameracalibrator.py \
    --size 8x6 --square 0.0245 \
    image:=/camera/color/image_raw camera:=/camera
```

标定步骤：

576. 缓慢移动标定板，覆盖画面各个区域
577. 将标定板倾斜不同角度
578. 让标定板接近和远离相机
579. 当 X, Y, Size, Skew 条变绿后，点击 Calibrate
580. 查看标定结果，点击 Save 保存

标定完成后 Save 按钮保存结果到/tmp/calibrationdata.tar.gz。

30.5.4 标定结果加载

```
import yaml
import numpy as np
```



```

def load_calibration(calib_file):
    """加载相机标定结果"""
    with open(calib_file, 'r') as f:
        data = yaml.safe_load(f)

    camera_matrix = np.array(data['camera_matrix']['data']).reshape(3, 3)
    dist_coeffs = np.array(data['distortion_coefficients']['data'])
    rect_matrix = np.array(data['rectification_matrix']['data']).reshape(3, 3)
    proj_matrix = np.array(data['projection_matrix']['data']).reshape(3, 4)

    calib_data = {
        'camera_matrix': camera_matrix,
        'dist_coeffs': dist_coeffs,
        'rect_matrix': rect_matrix,
        'proj_matrix': proj_matrix,
        'width': data['image_width'],
        'height': data['image_height'],
    }
    return calib_data

# 使用标定结果进行畸变校正
def undistort_image(image, camera_matrix, dist_coeffs):
    """对图像进行畸变校正"""
    h, w = image.shape[:2]
    new_camera_matrix, roi = cv2.getOptimalNewCameraMatrix(
        camera_matrix, dist_coeffs, (w, h), 1, (w, h)
    )
    undistorted = cv2.undistort(
        image, camera_matrix, dist_coeffs, None, new_camera_matrix
    )
    # 裁剪有效区域
    x, y, w, h = roi
    undistorted = undistorted[y:y+h, x:x+w]
    return undistorted

```

30.5.5 完整标定集成示例

```

#!/usr/bin/env python3
"""相机标定结果加载与使用示例"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

class CalibrationNode(Node):
    def __init__(self):
        super().__init__('calibration_node')
        self.bridge = CvBridge()
        self.camera_matrix = None
        self.dist_coeffs = None

        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw', self.image_callback, 10
        )

```

```

        self.info_sub = self.create_subscription(
            CameraInfo, '/camera/color/camera_info', self.info_callback, 10
        )
        self.undist_pub = self.create_publisher(
            Image, '/camera/undistorted', 10
        )

    def info_callback(self, msg):
        if self.camera_matrix is None:
            self.camera_matrix = np.array(msg.k).reshape(3, 3)
            self.dist_coeffs = np.array(msg.d)
            self.get_logger().info('相机参数已加载')

            self.get_logger().info(f'内参矩阵:\n{self.camera_matrix}')

    def image_callback(self, msg):
        if self.camera_matrix is None:
            return

        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

        # 畸变校正
        h, w = frame.shape[:2]
        new_cam, roi = cv2.getOptimalNewCameraMatrix(
            self.camera_matrix, self.dist_coeffs, (w, h), 1, (w, h)
        )
        undistorted = cv2.undistort(
            frame, self.camera_matrix, self.dist_coeffs, None, new_cam
        )

        # 发布校正后图像
        out_msg = self.bridge.cv2_to_imgmsg(undistorted, 'bgr8')
        out_msg.header = msg.header
        self.undist_pub.publish(out_msg)

def main(args=None):
    rclpy.init(args=args)
    node = CalibrationNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.6 OpenCV 图像处理与 ROS2 集成

30.6.1 图像订阅与处理节点

```

#!/usr/bin/env python3
"""图像处理节点示例"""
import rclpy
from rclpy.node import Node

```

```

from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import numpy as np

class ImageProcessor(Node):
    def __init__(self):
        super().__init__('image_processor')
        self.bridge = CvBridge()

        self.sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10
        )
        self.gray_pub = self.create_publisher(Image, '/image_gray', 10)
        self.edge_pub = self.create_publisher(Image, '/image_edges', 10)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

        # 灰度转换
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # 边缘检测
        edges = cv2.Canny(gray, 50, 150)

        # 高斯模糊
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)

        # 发布处理结果
        gray_msg = self.bridge.cv2_to_imgmsg(gray, 'mono8')
        gray_msg.header = msg.header
        self.gray_pub.publish(gray_msg)

        edge_msg = self.bridge.cv2_to_imgmsg(edges, 'mono8')
        edge_msg.header = msg.header
        self.edge_pub.publish(edge_msg)

        cv2.imshow('Original', frame)
        cv2.imshow('Edges', edges)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ImageProcessor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.6.2 常用图像处理函数

```
class ImageProcessingUtils:
    """图像处理工具函数集"""

    @staticmethod
    def resize_image(image, width=None, height=None, scale=None):
        """图像缩放"""
        h, w = image.shape[:2]
        if scale:
            return cv2.resize(image, None, fx=scale, fy=scale)
        if width and height:
            return cv2.resize(image, (width, height))
        return image

    @staticmethod
    def rotate_image(image, angle, center=None):
        """图像旋转"""
        h, w = image.shape[:2]
        if center is None:
            center = (w // 2, h // 2)
        M = cv2.getRotationMatrix2D(center, angle, 1.0)
        return cv2.warpAffine(image, M, (w, h))

    @staticmethod
    def draw_fps(image, fps, position=(10, 30)):
        """在图像上绘制 FPS"""
        cv2.putText(image, f'FPS: {fps:.1f}', position,
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    @staticmethod
    def draw_bbox(image, bbox, label='', color=(0, 255, 0)):
        """绘制检测框"""
        x1, y1, x2, y2 = bbox
        cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
        if label:
            cv2.putText(image, label, (x1, y1 - 5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
```

课后练习

581. 编写 ROS2 节点，订阅相机图像话题，将彩色图像转换为灰度图并重新发布。
582. 使用 usb_cam 驱动启动 USB 摄像头，查看其发布的/image_raw 和/camera_info 话题内容。
583. 使用 camera_calibration 工具对相机进行标定，保存标定结果并编写节点加载标定参数。
584. 编写节点，对输入的图像进行畸变校正，并发布校正后的图像话题。
585. 实现一个图像传输节点，使用 image_transport 发布压缩图像，并在订阅端显示。

实验 30:

第 30 章 实验手册：安全验证与系统集成

****对应理论章节**:** 第 44 章《安全验证与系统集成》 ****实验课时**:** 3 课时 ****实验代码**:** ``src/lab_code/ch30_lab/``

实验环境

项目	规格
操作系统	Ubuntu 24.04
ROS 2	Jazzy
Gazebo	Sim Harmonic
CARLA	0.9.16 (需要时启用)
Python	3.12+

先完成课程环境配置并编译安全相关包：

```
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash
cd "$ROS2_COURSE_WS"
colcon build --symlink-install --packages-select \
  av_carla_interfaces av_safety_monitor
source install/setup.bash
```

练习 30.1：验证安全监控节点

av_safety_monitor 提供 TTC 预警、碰撞事件处理、轨迹安全监控和故障注入节点。

启动安全监控：

```
ros2 run av_safety_monitor safety_monitor --ros-args \
  -p ttc_threshold_warning:=4.0 \
  -p ttc_threshold_critical:=2.5 \
  -p ttc_threshold_emergency:=1.5
```

主要接口：

方向	接口
订阅	`/carla/ego_vehicle/collision` (`CollisionEvent`)
订阅	`/perception_objects` (`PerceptionObjectArray`)
订阅	`/plan` (`nav_msgs/Path`)
订阅	`/ego_state` (`geometry_msgs/TwistStamped`)
发布	`/safety_status` (`std_msgs/String`)
发布	`/safety_markers` (`visualization_msgs/MarkerArray`)
发布	`/emergency_stop` (`std_msgs/Bool`)

检查节点和状态：

```
ros2 node list
ros2 topic list | grep -E 'safety|perception|plan|ego_state'
ros2 topic echo /safety_status
ros2 topic echo /emergency_stop
```

练习 30.2 : 故障注入

故障注入器当前针对 `std_msgs/String` 话题，适合先使用一个调试字符串话题验证丢帧、噪声、偏置、延迟和失效逻辑：

```
ros2 run av_safety_monitor fault_injector --ros-args \
  -p fault_type:=drop \
  -p target_topic:=/debug_status \
  -p fault_probability:=0.1
```

控制故障注入：

```
ros2 service call /inject_fault std_srvs/srv/SetBool "{data: false}"
ros2 topic echo /debug_status_injected
```

真实 `PerceptionObjectArray`、`Path` 等强类型话题不能直接套用该 `String` 注入器；需要为对应消息类型编写适配器后再做端到端故障测试。

练习 30.3：集成测试和性能评估

src/labs/ch30_lab/ 提供独立的 ROS 2 测试和离线评估脚本：

```
cd "$ROS2_COURSE_WS/src/labs/ch30_lab"

# 运行单个场景
python3 integration_test.py --scenario straight_lane --duration 120

# 运行全部场景和故障配置
python3 integration_test.py --run-all

# 读取结果并生成 Markdown/JSON 报告
python3 eval_metrics.py --log-dir results/ch30_eval --format both
python3 eval_metrics.py --log-dir results/ch30_eval --summary
```

若使用 CARLA，先启动 robot_sim_demo 或 CARLA Bridge，再执行集成测试。测试脚本会记录速度、TTC、偏离、碰撞、舒适性和控制误差等指标。

单元测试

```
cd "$ROS2_COURSE_WS"
python3 -m pytest src/course/av_safety_monitor/test -q
```

验收清单

检查项	完成
`av_safety_monitor` 可以正常启动	☐
碰撞事件会触发 `/emergency_stop`	☐
TTC 预警等级符合参数阈值	☐
`/safety_status` 和 `/safety_markers` 正常发布	☐
故障注入器可启停并发布注入结果	☐
单元测试通过	☐
集成测试结果已生成	☐
报告记录了环境版本和未满足的依赖	☐

当前课程仓库不包含旧的独立自动驾驶包或统一集成 bringup 包；应以实际存在的 av_* 包和工作空间中的 src/labs/ch30_lab/ 脚本为准。完整 CARLA 端到端验证需要 Ubuntu 24.04、ROS 2 Jazzy、CARLA 0.9.16 及对应运行依赖。

第 31 章 颜色检测与 YOLO 检测

学习目标

- 理解 HSV 颜色空间及其应用
- 掌握基于颜色阈值的物体检测方法
- 学会集成 YOLOv8 进行目标检测
- 掌握将检测结果发布为 ROS2 消息的方法

31.1 颜色空间与 HSV

31.1.1 RGB 与 HSV 颜色空间

RGB (Red-Green-Blue) 是最常用的颜色表示方式，但受光照影响大，不适合做颜色检测。

HSV (Hue, Saturation, Value) 将颜色分解为色调、饱和度和明度三个维度，更符合人类对颜色的感知，对光照变化不敏感。

- **H (Hue, 色调)** : 0-180° (OpenCV 中归一化为 0-180)
- 红色: 0~10 和 160~180
- 橙色: 11~25
- 黄色: 26~34
- 绿色: 35~77
- 蓝色: 100~124
- 紫色: 125~155
- **S (Saturation, 饱和度)** : 0-255, 值越大颜色越纯
- **V (Value, 明度)** : 0-255, 值越大越亮

31.1.2 颜色阈值表

颜色	Hmin	Hmax	Smin	Smax	Vmin	Vmax
红色 1	0	10	43	255	46	255
红色 2	156	180	43	255	46	255
橙色	11	25	43	255	46	255

黄色	26	34	43	255	46	255
绿色	35	77	43	255	46	255
青色	78	99	43	255	46	255
蓝色	100	124	43	255	46	255
紫色	125	155	43	255	46	255

31.1.3 颜色阈值获取工具

```
#!/usr/bin/env python3
"""HSV 颜色阈值调试工具"""
import cv2
import numpy as np

def nothing(x):
    pass

def hsv_threshold_tool():
    """HSV 阈值调节 GUI 工具"""
    cap = cv2.VideoCapture(0)
    cv2.namedWindow('HSV Threshold')

    # 创建滑动条
    cv2.createTrackbar('H_min', 'HSV Threshold', 0, 180, nothing)
    cv2.createTrackbar('H_max', 'HSV Threshold', 180, 180, nothing)
    cv2.createTrackbar('S_min', 'HSV Threshold', 0, 255, nothing)
    cv2.createTrackbar('S_max', 'HSV Threshold', 255, 255, nothing)
    cv2.createTrackbar('V_min', 'HSV Threshold', 0, 255, nothing)
    cv2.createTrackbar('V_max', 'HSV Threshold', 255, 255, nothing)

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # 读取滑动条值
        h_min = cv2.getTrackbarPos('H_min', 'HSV Threshold')
        h_max = cv2.getTrackbarPos('H_max', 'HSV Threshold')
        s_min = cv2.getTrackbarPos('S_min', 'HSV Threshold')
        s_max = cv2.getTrackbarPos('S_max', 'HSV Threshold')
        v_min = cv2.getTrackbarPos('V_min', 'HSV Threshold')
        v_max = cv2.getTrackbarPos('V_max', 'HSV Threshold')

        # 应用阈值
        lower = np.array([h_min, s_min, v_min])
        upper = np.array([h_max, s_max, v_max])
        mask = cv2.inRange(hsv, lower, upper)
```

```

# 形态学操作
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

# 显示结果
result = cv2.bitwise_and(frame, frame, mask=mask)
cv2.imshow('Original', frame)
cv2.imshow('Mask', mask)
cv2.imshow('Result', result)

key = cv2.waitKey(1) & 0xFF
if key == ord('q'):
    break
elif key == ord('s'):
    # 保存当前阈值
    print(f'H: [{h_min}, {h_max}], '
          f'S: [{s_min}, {s_max}], '
          f'V: [{v_min}, {v_max}]')

cap.release()
cv2.destroyAllWindows()

if __name__ == '__main__':
    hsv_threshold_tool()

```

31.2 颜色分割与检测

31.2.1 颜色检测节点

```

#!/usr/bin/env python3
"""ROS2 颜色检测节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import cv2
import numpy as np

class ColorDetector(Node):
    def __init__(self):
        super().__init__('color_detector')

        # 可配置参数
        self.declare_parameter('H_min', 35)
        self.declare_parameter('H_max', 77)
        self.declare_parameter('S_min', 43)
        self.declare_parameter('S_max', 255)
        self.declare_parameter('V_min', 46)
        self.declare_parameter('V_max', 255)
        self.declare_parameter('target_color', 'green')

        # 读取参数
        self.H_min = self.get_parameter('H_min').value
        self.H_max = self.get_parameter('H_max').value

```

```

self.S_min = self.get_parameter('S_min').value
self.S_max = self.get_parameter('S_max').value
self.V_min = self.get_parameter('V_min').value
self.V_max = self.get_parameter('V_max').value
self.target_color = self.get_parameter('target_color').value

self.bridge = CvBridge()

# 发布结果图像
self.result_pub = self.create_publisher(
    Image, '/color_detection/result', 10
)
# 发布检测掩码
self.mask_pub = self.create_publisher(
    Image, '/color_detection/mask', 10
)

# 订阅相机图像
self.image_sub = self.create_subscription(
    Image, '/camera/color/image_raw',
    self.image_callback, 10
)

self.get_logger().info(
    f'颜色检测节点已启动, 目标颜色: {self.target_color}'
)

def image_callback(self, data):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(data, 'bgr8')
    except CvBridgeError as e:
        self.get_logger().error(str(e))
        return

    self.color_detection(cv_image)

def color_detection(self, cv_image):
    """执行颜色检测"""
    # 高斯模糊减少噪声
    blurred = cv2.GaussianBlur(cv_image, (7, 7), 0)

    # BGR → HSV
    hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)

    # 创建颜色掩码
    color_low = np.array([self.H_min, self.S_min, self.V_min])
    color_high = np.array([self.H_max, self.S_max, self.V_max])
    mask = cv2.inRange(hsv, color_low, color_high)

    # 形态学操作
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

    # 查找轮廓
    contours, _ = cv2.findContours(

```

```

        mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE
    )

    detection = cv_image.copy()
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 500: # 过滤小噪点
            continue

        # 计算最小外接矩形
        rect = cv2.minAreaRect(cnt)
        box = cv2.boxPoints(rect)
        box = np.int0(box)

        # 绘制检测结果
        cv2.drawContours(detection, [box], 0, (0, 255, 0), 2)

        # 绘制中心点
        cx = int(rect[0][0])
        cy = int(rect[0][1])
        cv2.circle(detection, (cx, cy), 4, (0, 0, 255), -1)

        # 添加标签
        cv2.putText(
            detection,
            f'{self.target_color} ({cx}, {cy})',
            (cx - 40, cy - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (0, 255, 0), 2
        )

        self.get_logger().info(
            f'检测到{self.target_color}: '
            f'中心({cx}, {cy}), 面积{area:.0f}'
        )

    # 发布结果
    try:
        result_msg = self.bridge.cv2_to_imgmsg(detection, 'bgr8')
        result_msg.header.stamp = self.get_clock().now().to_msg()
        self.result_pub.publish(result_msg)

        mask_msg = self.bridge.cv2_to_imgmsg(mask, 'mono8')
        mask_msg.header.stamp = self.get_clock().now().to_msg()
        self.mask_pub.publish(mask_msg)
    except CvBridgeError as e:
        self.get_logger().error(str(e))

    # 显示
    cv2.imshow('Color Detection', detection)
    cv2.imshow('Mask', mask)
    cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ColorDetector()

```

```

try:
    rclpy.spin(node)
except KeyboardInterrupt:
    cv2.destroyAllWindows()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

31.2.2 多颜色检测

```

class MultiColorDetector(Node):
    """多颜色同时检测"""

    # 预定义颜色阈值
    COLOR_RANGES = {
        'red': [(np.array([0, 43, 46]), np.array([10, 255, 255])),
                (np.array([156, 43, 46]), np.array([180, 255, 255]))],
        'green': [(np.array([35, 43, 46]), np.array([77, 255, 255]))],
        'blue': [(np.array([100, 43, 46]), np.array([124, 255, 255]))],
        'yellow': [(np.array([26, 43, 46]), np.array([34, 255, 255]))],
        'orange': [(np.array([11, 43, 46]), np.array([25, 255, 255]))],
    }

    COLORS_BGR = {
        'red': (0, 0, 255),
        'green': (0, 255, 0),
        'blue': (255, 0, 0),
        'yellow': (0, 255, 255),
        'orange': (0, 165, 255),
    }

    def __init__(self):
        super().__init__('multi_color_detector')
        self.bridge = CvBridge()

        self.sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )
        self.pub = self.create_publisher(
            Image, '/color_detection/multi', 10
        )

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        detection = frame.copy()

        for color_name, ranges in self.COLOR_RANGES.items():
            mask = None
            for lower, upper in ranges:
                if mask is None:
                    mask = cv2.inRange(hsv, lower, upper)
                else:
                    mask = cv2.bitwise_or(

```

```

        mask, cv2.inRange(hsv, lower, upper)
    )

    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 300:
            continue

        x, y, w, h = cv2.boundingRect(cnt)
        cv2.rectangle(
            detection, (x, y), (x + w, y + h),
            self.COLORS_BGR[color_name], 2
        )
        cv2.putText(
            detection, color_name, (x, y - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5,
            self.COLORS_BGR[color_name], 2
        )

    result_msg = self.bridge.cv2_to_imgmsg(detection, 'bgr8')
    result_msg.header = msg.header
    self.pub.publish(result_msg)

    cv2.imshow('Multi Color Detection', detection)
    cv2.waitKey(1)

```

31.3 YOLOv8 目标检测集成

31.3.1 YOLOv8 模型架构

YOLO (You Only Look Once) 是单阶段目标检测的标杆算法，v8 版本进一步优化了速度和精度：

网络结构：

Backbone (C2f 模块) → 多尺度特征图
 Head (解耦检测头) → 分类 + 回归分支

输出张量：

```

[batch, 84, 8400]
|      |      └─ 预测框数 (640/8)^2 * 3 ≈ 8400
|      └─ 4 (bbox) + 80 (COCO 类别)
└─ 批次大小

```


31.3.2 安装依赖

```
# 安装 ultralytics (YOLOv8)
pip install ultralytics

# 安装 onnxruntime (可选的 ONNX 推理)
pip install onnxruntime

# 下载预训练模型
wget https://github.com/ultralytics/assets/releases/download/v0.0.0/yolov8n.pt
```

31.3.3 YOLOv8 模型导出

```
from ultralytics import YOLO

def export_yolo_for_ros2():
    """导出 YOLO 模型为 ONNX 格式"""
    model = YOLO('yolov8n.pt')

    model.export(
        format='onnx',
        imgsz=640,
        opset=12,
        half=False,      # FP32 精度
        simplify=True,   # 简化模型
        dynamic=False,   # 固定输入尺寸
    )

    print('模型已导出为: yolov8n.onnx')

# 验证导出的模型
model_onnx = YOLO('yolov8n.onnx')
results = model_onnx('test.jpg')
print(f'检测结果: {len(results[0].boxes)} 个目标')
```

31.3.4 ROS2 YOLOv8 检测节点

```
#!/usr/bin/env python3
"""YOLOv8 目标检测 ROS2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2D, Detection2DArray
from std_msgs.msg import Header
from cv_bridge import CvBridge
import cv2
import numpy as np

class YOLOv8Detector(Node):
    def __init__(self):
        super().__init__('yolov8_detector')
        self.bridge = CvBridge()

    # 参数
```

```

self.declare_parameter('model_path', 'yolov8n.pt')
self.declare_parameter('confidence_threshold', 0.5)
self.declare_parameter('use_onnx', False)

model_path = self.get_parameter('model_path').value
self.conf_thresh = self.get_parameter('confidence_threshold').value
use_onnx = self.get_parameter('use_onnx').value

# 加载 YOLO 模型
self.get_logger().info(f'加载 YOLO 模型: {model_path}')
try:
    from ultralytics import YOLO
    self.model = YOLO(model_path)
    if use_onnx:
        self.model = YOLO(model_path.replace('.pt', '.onnx'))
    self.get_logger().info('YOLO 模型加载成功')
except ImportError:
    self.get_logger().error('ultralytics 未安装, 请执行: pip install ultralytics')
    self.model = None

# 发布者
self.detection_pub = self.create_publisher(
    Detection2DArray, '/yolo/detections', 10
)
self.image_pub = self.create_publisher(
    Image, '/yolo/visualization', 10
)

# 订阅者
self.image_sub = self.create_subscription(
    Image, '/camera/color/image_raw',
    self.image_callback, 10
)

self.get_logger().info('YOLOv8 检测节点已启动')

def image_callback(self, msg):
    if self.model is None:
        return

    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

    # YOLO 推理
    results = self.model(
        frame,
        conf=self.conf_thresh,
        verbose=False
    )[0]

    # 构建 Detection2DArray 消息
    detections_msg = Detection2DArray()
    detections_msg.header = Header(
        stamp=self.get_clock().now().to_msg(),
        frame_id=msg.header.frame_id
    )

```

```

annotated_frame = frame.copy()

if results.bboxes is not None:
    for box in results.bboxes:
        # 提取检测结果
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        confidence = float(box.conf[0])
        class_id = int(box.cls[0])
        class_name = results.names[class_id]

        # 构建 Detection2D 消息
        detection = Detection2D()
        detection.header = detections_msg.header

        # 设置检测框
        detection.bbox.center.x = float((x1 + x2) / 2)
        detection.bbox.center.y = float((y1 + y2) / 2)
        detection.bbox.size_x = float(x2 - x1)
        detection.bbox.size_y = float(y2 - y1)

        # 设置置信度和类别
        detection.results[0].hypothesis.class_id = class_name
        detection.results[0].hypothesis.score = confidence

        detections_msg.detections.append(detection)

        # 可视化
        color = (0, 255, 0)
        cv2.rectangle(
            annotated_frame,
            (int(x1), int(y1)), (int(x2), int(y2)),
            color, 2
        )
        label = f'{class_name} {confidence:.2f}'
        cv2.putText(
            annotated_frame, label,
            (int(x1), int(y1) - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2
        )

        self.get_logger().info(
            f'检测到: {class_name} ({confidence:.2f})'
        )

    # 发布检测结果
    self.detection_pub.publish(detections_msg)

    # 发布可视化图像
    vis_msg = self.bridge.cv2_to_imgmsg(annotated_frame, 'bgr8')
    vis_msg.header = msg.header
    self.image_pub.publish(vis_msg)

    cv2.imshow('YOLOv8 Detection', annotated_frame)
    cv2.waitKey(1)

def main(args=None):

```

```

rclpy.init(args=args)
node = YOLOv8Detector()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    cv2.destroyAllWindows()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

31.3.5 YOLO 检测结果后处理

```

def yolo_postprocess(model_output, conf_thresh=0.5, iou_thresh=0.45):
    """YOLO 模型输出后处理（解码 + NMS）"""
    import numpy as np

    # 输入形状: [1, 84, 8400]
    output = model_output[0].T # [8400, 84]

    boxes = output[:, :4]      # cx, cy, w, h (归一化)
    scores = output[:, 4:]     # 类别概率

    # 获取最高类别
    class_ids = np.argmax(scores, axis=1)
    confidences = np.max(scores, axis=1)

    # 过滤低置信度
    mask = confidences > conf_thresh
    boxes = boxes[mask]
    confidences = confidences[mask]
    class_ids = class_ids[mask]

    # NMS 非极大值抑制
    indices = cv2.dnn.NMSBoxes(
        boxes.tolist(), confidences.tolist(),
        conf_thresh, iou_thresh
    )

    results = []
    for i in indices:
        cx, cy, w, h = boxes[i]
        x1 = cx - w / 2
        y1 = cy - h / 2
        results.append({
            'bbox': [x1, y1, w, h],
            'confidence': float(confidences[i]),
            'class_id': int(class_ids[i]),
        })

    return results

```

31.4 检测结果发布为 ROS2 消息

31.4.1 使用 vision_msgs

ROS2 中标准的目标检测消息定义在 vision_msgs 功能包中：

```
# 安装 vision_msgs
sudo apt install ros-jazzy-vision-msgs

from vision_msgs.msg import (
    Detection2D,
    Detection2DArray,
    ObjectHypothesisWithPose,
    BoundingBox2D
)

def create_detection_msg(
    bbox, confidence, class_name, header
):
    """创建 Detection2D 消息"""
    detection = Detection2D()
    detection.header = header

    # 设置 2D 边界框
    detection.bbox.center.x = bbox[0] + bbox[2] / 2
    detection.bbox.center.y = bbox[1] + bbox[3] / 2
    detection.bbox.size_x = bbox[2]
    detection.bbox.size_y = bbox[3]

    # 设置分类结果
    hypothesis = ObjectHypothesisWithPose()
    hypothesis.hypothesis.class_id = class_name
    hypothesis.hypothesis.score = confidence
    detection.results = [hypothesis]

    return detection
```

34.4.2 自定义检测消息

也可以定义自己的检测消息：

```
# msg/DetectionResult.msg
std_msgs/Header header
string[] class_names
float32[] scores
float32[] x_min
float32[] y_min
float32[] x_max
float32[] y_max

from .msg import DetectionResult

def publish_detection_result(self, detections, header):
    """发布检测结果"""
    msg = DetectionResult()
    msg.header = header
```

```

for det in detections:
    msg.class_names.append(det['class_name'])
    msg.scores.append(float(det['confidence']))
    msg.x_min.append(float(det['bbox'][0]))
    msg.y_min.append(float(det['bbox'][1]))
    msg.x_max.append(float(det['bbox'][2]))
    msg.y_max.append(float(det['bbox'][3]))

self.detection_pub.publish(msg)

```

31.4.3 目标检测与跟踪

```

class ObjectTracker:
    """简单的目标跟踪器"""

    def __init__(self, max_lost=10):
        self.objects = {} # class_name → last_bbox
        self.lost_count = {}
        self.max_lost = max_lost

    def update(self, detections):
        """更新跟踪状态"""
        current_ids = set()

        for det in detections:
            class_id = det['class_id']
            current_ids.add(class_id)

            if class_id in self.objects:
                # 更新已有目标
                self.objects[class_id] = det['bbox']
                self.lost_count[class_id] = 0
            else:
                # 新增目标
                self.objects[class_id] = det['bbox']
                self.lost_count[class_id] = 0

        # 标记丢失的目标
        for obj_id in list(self.objects.keys()):
            if obj_id not in current_ids:
                self.lost_count[obj_id] += 1
                if self.lost_count[obj_id] > self.max_lost:
                    del self.objects[obj_id]
                    del self.lost_count[obj_id]

        return self.objects

```

31.4.4 发布目标位姿

```

from geometry_msgs.msg import PoseStamped
from tf_transformations import quaternion_from_euler

class DetectionToPose(Node):
    """将 2D 检测转换为 3D 位姿（假设目标在固定深度）"""
    def __init__(self):

```

```

super().__init__('detection_to_pose')
self.sub = self.create_subscription(
    Detection2DArray, '/yolo/detections',
    self.detection_callback, 10
)
self.pose_pub = self.create_publisher(
    PoseStamped, '/target_pose', 10
)

# 相机内参
self.fx = 600.0
self.fy = 600.0
self.cx = 320.0
self.cy = 240.0
self.depth = 0.5 # 假设的深度值

def detection_callback(self, msg):
    for detection in msg.detections:
        # 2D 像素坐标 → 3D 相机坐标
        center_x = detection.bbox.center.x
        center_y = detection.bbox.center.y

        # PnP 投影
        x_3d = (center_x - self.cx) * self.depth / self.fx
        y_3d = (center_y - self.cy) * self.depth / self.fy
        z_3d = self.depth

        # 发布位姿
        pose = PoseStamped()
        pose.header = msg.header
        pose.pose.position.x = x_3d
        pose.pose.position.y = y_3d
        pose.pose.position.z = z_3d
        pose.pose.orientation.w = 1.0

        self.pose_pub.publish(pose)
        self.get_logger().info(
            f'目标 3D 位姿: ({x_3d:.2f}, {y_3d:.2f}, {z_3d:.2f})'
        )

```

课后练习

586. 使用 HSV 颜色空间编写程序检测红色和蓝色物体，并用不同颜色的框标注检测结果。
587. 编写 ROS2 节点，订阅相机图像，对图像进行颜色检测并发布检测结果 mask 话题。
588. 安装 YOLOv8 并编写 ROS2 检测节点，订阅相机图像，检测 COCO 数据集的 80 类物体。
589. 编写程序将 YOLO 检测结果（类别、置信度、边界框）发布为 vision_msgs/Detection2DArray 消息。

590. 结合颜色检测和 YOLO 检测，实现一个融合检测节点：先用颜色检测找到感兴趣区域，再用 YOLO 在该区域进行精确检测。

实验 31:

第 31 章 实验指导书：综合项目——城区自动驾驶

****对应理论章节**:** 第 45 章《综合项目：城区自动驾驶》 ****实验课时**:** 3 课时 ****实验代码**:** `src/lab\_code/ch31\_lab/`

实验目标

在 CARLA 0.9.16 和 ROS 2 Jazzy 环境中，完成传感器、感知、规划、控制和安全监控节点的组合验证。课程仓库提供可构建的基础节点、接口包和演示脚本；完整的城区自动驾驶管线还需要另行提供实现模块。

实验准备

1. 配置课程环境

在课程根目录执行：

```
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash

cd "$ROS2_COURSE_WS"
colcon build --symlink-install
source install/setup.bash
```

确认 ROS 2 和课程包可见：

```
echo "$ROS_DISTRO" # jazzy
ros2 pkg prefix av_carla_interfaces
ros2 pkg prefix av_sensor_kit
ros2 pkg prefix av_perception_py
ros2 pkg prefix av_planning_py
ros2 pkg prefix av_control_cpp
ros2 pkg prefix av_safety_monitor
```

2. 启动 CARLA 和 Bridge

终端 1，启动 CARLA：

```
"$CARLA_ROOT/CarlaUE4.sh" -quality-level=Low
```

终端 2, 启动 ROS 2 Bridge:

```
source ~/.config/ros2-course/env.bash
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  synchronous_mode:=False register_all_sensors:=True
```

终端 3, 生成自车并附加 RGB 相机和 LiDAR:

```
cd "$ROS2_COURSE_WS"
python3 src/labs/ch23_lab/spawn_ego.py \
  --spawn-point 10 --role-name ego_vehicle
```

检查 Bridge 话题:

```
ros2 topic list | grep carla
ros2 topic echo /carla/ego_vehicle/vehicle_control_cmd
```

练习 31.1 : 验证现有功能包

接口和传感器

```
ros2 interface show av_carla_interfaces/msg/EgoState
ros2 interface show av_carla_interfaces/msg/ControlCmd
ros2 run av_sensor_kit sensor_manager
```

感知节点

按需在不同终端启动:

```
ros2 run av_perception_py object_detector
ros2 run av_perception_py lidar_detector
ros2 run av_perception_py fusion_node
```

规划节点

```
ros2 run av_planning_py global_planner
ros2 run av_planning_py waypoint_generator
ros2 run av_planning_py planning_server
```

控制和安全节点

```
ros2 run av_control_cpp vehicle_controller \
  --ros-args -p target_speed:=10.0
ros2 run av_safety_monitor safety_monitor
```

定位使用 ROS 2 Jazzy 的 `robot_localization` 或实验者提供的定位组件; 本仓库没有单独的 `av_localization` 包。

练习 31.2 : 运行综合演示脚本

一键启动

town_demo.sh 位于工作空间的 src/labs/ch31_lab/, 默认使用 ~/ros2_course_ws 和 /opt/carla:

```
cd "$ROS2_COURSE_WS/src/labs/ch31_lab"
bash town_demo.sh \
  --carla-path "$CARLA_ROOT" \
  --ros-ws "$ROS2_COURSE_WS"
```

脚本会启动 CARLA、Bridge 和 RViz2, 并检查 main_pipeline.py 所需的目录外模块。缺少这些模块时只启动基础仿真环境, 不会伪装成完整管线已经启动。

单独启动主管线

当以下模块已经放入 ch31_lab/ 时, 可以按 Python 模块方式启动主管线:

```
carla_sensor_driver/
perception_node/
localization_node/
planning_node/
control_node/
safety_monitor_node/
cd "$ROS2_COURSE_WS/src/labs"
PYTHONPATH=. python3 -m ch31_lab.main_pipeline
```

主管线提供以下控制接口:

```
ros2 service call /pipeline/enable std_srvs/srv/SetBool "{data: true}"
ros2 topic echo /system/pipeline_status
ros2 topic echo /system/performance_metrics
```

练习 31.3 : 测试和验收

运行已有单元测试

在课程工作空间执行:

```
cd "$ROS2_COURSE_WS"
python3 -m pytest src/course/av_carla_interfaces/test -q
python3 -m pytest src/course/av_sensor_kit/test -q
python3 -m pytest src/course/av_perception_py/test -q
python3 -m pytest src/course/av_planning_py/test -q
python3 -m pytest src/course/av_safety_monitor/test -q
```

控制器为 C++ 包, 先执行构建, 再使用接口包测试和 C++ 编译结果验证:

```
colcon build --symlink-install --packages-select \
  av_carla_interfaces av_control_cpp
```

第 31 章目录中的综合测试脚本可用于已配置的运行环境：

```
cd "$ROS2_COURSE_WS/src/labs/ch31_lab"
bash run_all_tests.sh --quick
```

当前目录只提供 main_pipeline.py、town_demo.sh 和 run_all_tests.sh；没有内置 test/ 目录时，测试脚本会明确跳过该项。

验收清单

检查项	完成
ROS 2 发行版为 Jazzy	☐
CARLA 版本为 0.9.16	☐
`av_carla_interfaces` 可以显示接口	☐
CARLA Bridge 话题正常发布	☐
传感器、感知、规划、控制和安全节点可单独启动	☐
RViz2 可以显示 TF、LaserScan 和路径	☐
单元测试通过	☐
目录外主管线组件已配置并完成端到端验证	☐

结果记录

记录以下内容：

- 591. ros2 topic list 中的 CARLA、感知、规划和安全话题。
- 592. 各节点启动命令和终端输出。
- 593. 单元测试通过数量及失败原因。
- 594. 端到端演示中的碰撞次数、路线完成率和最大跟踪误差。

完整演示需要 CARLA、ROS 2 Jazzy、对应 Python 依赖以及目录外的主管线模块；缺少任一项时，应在实验报告中注明，不将基础 Bridge 验证记为完整自动驾驶验收。

Part 4: 自动驾驶（智能车自动驾驶）

